Combined Transaction JSON Data

Contains 29 JSON file(s) found in the uploaded ZIP archive. Each file is shown as readable, formatted text below.

1. transactions/t_10.json


```
{
  "video_id": "55aenUfSIhQ",
  "title": "Best Explanation Of Merge K Sorted Lists | HEAP DSA Pattern",
  "language": "en",
  "model": "large-v3",
  "duration": 1472,
  "segments": [
    {
      "start": 0.0,
      "end": 26.8,
      "text": "I was not in the mood to make this video today, but the thing is that the world
does not work with the mind, neither with my mind nor with your mind. Always do one thing in
the day that you do not want to do, okay? Whether it is to study, whether it is to go to the
gym or not to run Instagram, whatever it is, whether it is to eat or not to eat, any work. It
strengthens the willpower, like the body is made by lifting weight, so doing all these things
trains the willpower that okay, there is some resistance."
    },
    {
      "start": 26.8,
      "end": 54.91,
      "text": "Otherwise, you will always do what you want to do. We have been sleeping since
morning, we haven't done anything. So, I didn't feel like doing anything, we will go and sleep
now. But this is one thing that you have to do, do it. Do it in any way, do it by hitting your
mind, do it by getting angry, do it by doing something, do it by getting sad, but do it. So,
do these little things, this will make your mind stronger gradually. If you do only those
things that your mind wants to do, then your mind wants to see everyone's reel. See, the thing
is the same thing, it is a simple thing."
    },
    {
      "start": 54.91,
      "end": 81.7,
      "text": "So, let's start and see. It's a little problem-easy. It's a problem with syntax
too. I mean, how to write that heap, you have a problem with that. So, for that, you will get
all the code words in the description, you will get the question link. So, don't worry about
the syntax, learn the concept. Let's start. So, those who are following from the beginning,
will remember something after seeing this question. Because when we used to do two-pointers
many years ago, we had done this. Merge two-pointers, we had done it in a way, but..."
    },
    {
      "start": 81.7,
      "end": 109.94,
      "text": "Now, Merge to Sorted Array is not Merge to Sorted Array, it is Merge to Sorted
Array. We can call it as Increased Version or Hard Version. Let's start thinking. First of all,
 let's remember what we used to do in Merge to Sorted Array. So, what used to happen in Merge
to Sorted Array is that you used to get two arrays like this. Right? So, 1, 3 and 7 used to be
one array. And 2, 6 and 8 used to be one different array. Now, what did we do? And if we write
its index here, then what did we do?"
    },
    {
      "start": 109.94,
      "end": 139.62,
      "text": "you used to make a big array first because their size is 3 so for the whole
array you will need an array of 6 sizes so you used to take an array of 6 sizes here 1,2,3,4,5
1,2,3 and by doing this you used to make an array after that what we used to do in 2 pointer
approach what we used to do we used to keep one pointer here whose name used to be i we used
to keep one pointer here whose name used to be j now what decision we used to take that
brother look we have to fill the whole array"
    },
    {
      "start": 139.62,
      "end": 163.74,
      "text": "In some way or the other, you have to fill the whole area. Let's concentrate on
filling this space now. Because this space is empty now, so let's concentrate on filling this
first one. Now who will come in the first space? The smallest element of both the areas can
come. Okay. Now in both the areas, we used to see that there is 1 on 1. Here is 2. So we used
to see that which element can't come for sure. So we used to say that 3 and 7, do anything."
    },
    {
      "start": 163.74,
```

```
      "end": 191.12,
      "text": "It can't come to the first position. Why can't it come? Because if someone from 3 and 7 has to come to the first, then where will 1 go? 1 is small, so the sorted one will not sit. With the same logic, we used to say that 6 and 8, no matter what they do, no matter what they eat, it can't come to the first position. Why? Because 6 and 8, whether it is bigger than anyone or not, it is definitely bigger than 2. We used to decide that for the first position, only 2 claimants are left. Which 2 claimants are left? Either 1 or 2."
    },
    {
      "start": 191.12,
      "end": 216.29,
      "text": "Who will come from 1 and 2? So we used to see that who is the smaller one from array of i and b of j? We used to see that the smaller one is this one. We used to lift it up and put it here. What did we do by lifting it up? We used to put it here. And we used to increase i. What did we do by increasing i? We used to increase it. This used to become i. We used to do the same thing. Now we will fill this space. So 1's work is finished. On this space, using the same logic,"
    },
    {
      "start": 216.29,
      "end": 241.74,
      "text": "7 can't do anything. 6 and 8 can't do anything. Who can come? Who has a chance to come? 3 can come or 2 can come. Who will come from both? The one who is small will come. We used to see that he is small. So what we used to do? We used to write 2 here. And similarly, whoever we used to write, we used to increase j in it. We used to do it like this. Until both the arrays were finished, you used to get a sorted array here."
    },
    {
      "start": 241.74,
      "end": 271.25,
      "text": "Now the problem is that you don't have 2 sorted arrays, you have k sorted arrays. k can be 2, k can be 3, k can be 100. If we take a pointer for each array and compare it, then you will face a lot of problems. For example, let's try this on 3 arrays. Let's try to merge 3 sorted arrays. One is done, and the other is done. 1, 0, 1, 0, 1, 0, 1."
    },
    {
      "start": 271.25,
      "end": 299.41,
      "text": "1, 6, 2, 5, and 2 and 8. These are your three sorted arrays. Similarly, you will make a very big array. Similarly, you will make a very big array. Okay. 0, 1, 2, 3, 4, 5. Now, because there are three sorted arrays here, then you will have to take three pointers. Two pointers won't work. It was in two pointers. Now, you will have to do three pointers. So, if you do three pointers, then I will keep it here. J will keep it here. K will keep it here. With similar logic,"
    },
    {
      "start": 299.41,
      "end": 321.97,
      "text": "We will try to fill this space. Which element can come in this space? So in this space, with similar logic, either this can come, or this can come, or this can come. Nothing else can come. Now you have to compare between these three. Which one is smaller than these three? Which one is smaller than these three? And suppose you come to know that the i is smaller. So you write the i here. You will increase the i here."
    },
    {
      "start": 321.97,
      "end": 348.66,
      "text": "We will increase the i here. Then you will have to do the same thing on all three. So tracking will be very difficult. You must be thinking that we will do it on all three arrays. There is only i, j, k. By doing something, we will increase the minimum i and add it. This is taking 3 paise. Now assume 100 are sorted arrays. How many are sorted arrays? 100. 100 are sorted arrays. Now this 2 pointer approach has become your 100 pointer approach. You might not realize it but it has become."
    },
    {
      "start": 348.66,
      "end": 378.35,
      "text": "Earlier i and j were there, now you will need 100 variables together. 100 variables, from a1 to a100, the names of the variables. Every time you will have to track which is the minimum. Every time which is the minimum, you will have to increase the minimum. So this will become a very bulky program. Do you understand? Keeping 100 variables, if something goes wrong somewhere, then we will go and debug that where did it go wrong, who knows what happened there. So this will become very bulky. We will apply the same thing, same logic, everything is the same, but"
    },
    {
      "start": 378.42,
```

```
      "end": 399.71,
      "text": "Let's see what we are essentially doing here. The logic is the same, everything is the same, that there will be a pointer in every array. So initially what are you doing? Initially what are you doing? Suppose we talk about 4 arrays. 1, 2 and like this. So 0 and 1. This is 0 and 1. 0 and 1."
    },
    {
      "start": 399.71,
      "end": 426.88,
      "text": "And this is 0 and 1. Let's talk about 4 arrays. To fill the first place, which are the candidates of each array? The first one of each array. The first one of each array. The first one of each array. Means the one with 0 in each array. This. This. This. Who wants to go from this? Who wants to go in the real one? The smallest one. The smallest one. Okay? After that, what do we do? Whoever you send, whoever you put in the main array, we increase that pointer."
    },
    {
      "start": 426.88,
      "end": 455.92,
      "text": "Let's increase that pointer to 1 and let's repeat this until all the arrays are finished. What can we use for this? What you have to do is, on every point, what do you want? On every point, what do you want? The smallest, the smallest, in any set of values. How many values are there here? 4 values. This, this, this and this. There are set of values. On every point, on which are you interested? You are interested in the smallest value."
    },
    {
      "start": 455.92,
      "end": 486.27,
      "text": "What does this heap mean? It means heap data structure. If you want the largest or the smallest data structure at every point, then you use heap data structure. Now you will see how easy it is to use heap data structure. I will show you how easy it is to use heap data structure. I will show you how easy it is to use heap data structure. I will show you how easy it is to use heap data structure. I will show you how easy it is to use heap data structure. I will show you how easy it is to use heap data structure. I will show you how easy it is to use heap data structure. I will show you how easy it is to use heap data structure."
    },
    {
      "start": 486.27,
      "end": 509.18,
      "text": "Make one more array. 2, 4, 6 and 8. Make one more array. 0, 9 and 10. Okay? 0, 1, 2, 3. 0, 1, 2, 3. 0, 1, 2. Okay? What will we do? First of all, do you want the smallest value in each? Okay."
    },
    {
      "start": 509.18,
      "end": 533.44,
      "text": "So, first of all, let's see what type of heap you want. At every point, are you looking for the biggest value or the smallest value? What are you looking for? We are looking for the smallest value. We will make a min heap. Okay. What will we keep in that min heap? What will we keep in that min heap? So, someone can say that, okay, we will keep all the values. We will put 1, 2 and 0 in the heap. We can put. For the first time, you will get it. For the first time, you will get 0."
    },
    {
      "start": 533.44,
      "end": 560.27,
      "text": "But after getting 0, what you used to do? If you remember 2 pointer, then suppose what you used to do after getting this, you used to go to next. Means you used to start looking at 9. So what used to happen in the next set of values? 1, 2 and 9, if 0 has gone. Similarly, if you used to get 1, then the next set of values used to happen, sorry, if you used to get 2, then the next set used to happen 3, 4 and 9. So whichever array you used to choose the value, you used to point at the next one. This used to happen in 2 pointer."
    },
    {
      "start": 560.27,
      "end": 590.03,
      "text": "So, in your heap, you have to do the same thing. In the beginning, you can take 1, 2, 0 because it is happening from here in the beginning. But when you select someone, select a small one and put it in the main array, then you will have to move to the next one in that array. So, what can we store in our heap? And you should know from which array you have chosen. Which array? There are three arrays here. It is possible that you have 100 sorted arrays. So, it is possible that you have chosen a value from array number 57. So, you have to go to the next one in the 57th array. Do you understand what I am saying?"
    },
    {
      "start": 590.06,
      "end": 615.44,
```

```
      "text": "Same pointer approach. In 2, it would have been easy to do i and j, but here
100 is sorted array. If you take something from array number 55, then you will have to move to
the next index of array number 55. Okay? So, here you will have to track 2-3 things. The first
thing is value. Value means the original value on which you will apply minimum heap. The thing
on which minimum will be applied. After that, what will you do?"
    },
    {
      "start": 615.66,
      "end": 646.9,
      "text": "Array number, if you think of it as a matrix, if you think of it as a matrix,
like a 2D matrix, which is also given in the question, then what is this array? What is its
row number? This is row 0, this is row 1, and this is row 2. So you can say row number, either
array number or row number, you can say anything. One thing you will need is row. Row. Next
thing, this pointer, on which index are you pointing? On which index of each row, means on
each column, column, column, column."
    },
    {
      "start": 647.15,
      "end": 676.82,
      "text": "If we put these three things in our heap and make a rule that according to whom
this minimum heap is? This minimum heap is according to my value, according to the real value.
So what will we put in the beginning? We will put three things in the beginning. What are the
three things? First of all, the value, this one, 1. Which row is this? This row number is 0.
Okay. In which column? In the beginning, all of them will remain 0 because in the beginning,
remember, in the two-pointer, i and j both were on 0 index. So in the beginning, both were
there. Okay. What will we put in the next? This one."
    },
    {
      "start": 676.82,
      "end": 707.25,
      "text": "2. Which row is this? It is 1. And which column is this? It is 0 column. Okay.
What will we enter next? 0. Value. Which row is this? It is 2 row. And which column is this?
It is 0 column. You have entered this in your heap. You have entered this in your heap. Your
heap has been made. In the beginning, in the beginning, what did we do in 2 pointer? int i is
equal to 0, j is equal to 0. We are doing the same thing. We have entered all those with 0. We
have entered all those with 0 in our heap. Is it okay? Okay. How will this heap be made? We
will give its code. Don't worry about it."
    },
    {
      "start": 707.31,
      "end": 736.46,
      "text": "Now, what did you do in 2-pointer? Once it is done, then you decide which one
is smaller. The smaller one will go to your main array. So, what will you have to do in this?
Nothing will have to be done. Just pop it. If you pop it, then you will get the smallest value.
 So, when you pop it, then see which value is the smallest? 0. Because the minheap is made
according to the value. This entry will come out. This entry will come out. If this entry
comes out, then what will you know? See, 0 to 0 came out. 0 to 0."
    },
    {
      "start": 736.46,
      "end": 765.15,
      "text": "What is there in this? The value is 0, so you will keep 0 in your main array.
You will keep 0 where your main array is being created. The work of 0 is done. Now, what you
have to do is to increase this pointer. But which array's pointer is this? That will be known
as your row. Row is equal to 2. Row is equal to 2. Okay. Where was this pointer just now? It
will tell you this column. This pointer was just now on this. What you have to do is to make
this column plus plus. In this row, in this row, by doing plus plus,"
    },
    {
      "start": 765.15,
      "end": 791.81,
      "text": "If you want to push the next value, i.e. this 9 in the heap, then what will you
push? You will push the array of which row was it? Row and column plus 1, i.e. array of 2, i.e.
 array of 2's 1. Array of 2's 1, see 9. What will go in your heap? This has popped in your
heap. What has gone in your heap? See, 9, 2, 1."
    },
    {
      "start": 791.81,
      "end": 819.54,
      "text": "Automatically, the two-pointer that you had done, is being achieved. Next time
when you will pull this, next time when you will see, which one is the smallest? The smallest
is 1. Bring out 1, 0, 0. Bring out 1, 0, 0. 1, 0, 0. What is the value in this? 1. Put 1 in
your main array, in the result array. Now what do you have to do? Which row's array was this?
Which row was it? 0. What was the column? 0. What do you have to do? What do you have to do?
Array of row,"
    },
    {
```

```
      "start": 820.05,
      "end": 849.17,
      "text": "column plus 1 means array of 0 means 1 means this value 3 so what will you do? we have popped this and pushed it value is 3 and row is 0 so you will keep doing this and till when will you keep doing this? suppose what you are doing array of row and column minus 1 if you ever get to know that the existing column suppose the existing column is the end of the array that is the end of the array how will you get to know the end of the array?"
    },
    {
      "start": 849.26,
      "end": 877.63,
      "text": "you will have n rows in the array and m columns if you ever get to know that the column is equal to m-1 means you have reached the last of that array you have reached the last of that array, you used to do the loop there too while i less than n and j less than m is the same thing so if the column is equal to m-1 then what will we do? we will not do anything and if there is nothing left then we will not do anything, we will go ahead so when your heap will be completely finished"
    },
    {
      "start": 877.63,
      "end": 908.37,
      "text": "when your heap will be empty then all the things will be in your array we will explain this with a small example we will explain this with a small dry run you might have understood it but anyway take one array 1 and 2 0 and 1 make it 3 take one more array 2 and 4 0 and 1 suppose there are two sorted arrays write its row number row is equal to 0 is equal to 1 in the beginning"
    },
    {
      "start": 908.37,
      "end": 935.04,
      "text": "What will we push in our heap? Value, row and column. So, what is its value? 1, row number 0 and column is 0. This is pushed. Then, in the next also, for all arrays, initially push the first one of all arrays. Push the first one of all arrays initially because we will start after that. So, its value is 2, row is 1 and 0. Let's start. Let's start and remove. So, what will you remove first? 1, 0, 0. Okay?"
    },
    {
      "start": 935.04,
      "end": 964.82,
      "text": "So, in your main array, in your result array, first of all, push the value. After that, see what is its row? Zero. What is its column? Zero. Is it correct? What is your column? Yes, what is your column in this? n is equal to 2, m is equal to 2. There are 2 rows, 2 columns in this whole matrix. Has your column become equal to m minus 1? No, your column has not become equal to m minus 1. So, what will you push? Array of row and column minus, column plus 1. Meaning, array of 0 and 1. Meaning,"
    },
    {
      "start": 964.82,
      "end": 996.08,
      "text": "What happened to array of 0 and 1? This one. So what did you push? 3. So remove this. And what did you push? 3, 0, 1. Okay? Let's move forward. Come, let's move forward. Come, let's move forward. Next time when you will pull, what will you pull? 2, 1, 0. 2, 1, 0. So first of all, put this in your array. Where is your array? Put this result array in 2. And what did you push? Array of row, column plus 1. See that the column is not finished. So what will you push? Array of 1."
    },
    {
      "start": 996.08,
      "end": 1023.68,
      "text": "1 means 4. Nothing. Remove this. 4 is row 1 and column 1. It's over. Do it again. The heap is not over yet. Till the heap is not over, we will continue because till then the elements will remain. If you do the next pull, you will get 3, 0, 1. 3, 0, 1. First of all, push 3 in the result. And, and, and, is your column, your row is 0. Row will remain 0. Column is equal to 1."
    },
    {
      "start": 1023.68,
      "end": 1047.98,
      "text": "Is your column equal to m-1? Yes, because you can see that we have reached the end. Where did it go? Oh, sorry. We have reached the end here. There is nothing left after this. If there is nothing left after this, then don't do anything. Pop it and leave it. Pop it and leave it. You can't do anything, right? After this, you can't increase the pointer in this array, right? The work of this array is over. Do the next pull. 4, 1, 1. First of all, push 4 in your answer."
    },
    {
      "start": 1047.98,
```

```
      "end": 1077.55,
      "text": "And what is the column of this also? 1. So is this also finished? Yes, this is
also finished. So you don't have anything to push before this. Delete this. Your heap is empty,
 close it. See carefully that your result array has become 1, 2, 3, 4. Yes, my result array
has become 1, 2, 3, 4. So it is a similar concept. In the case of the same two-pointer, why
are we using the heap column? Every time, it will not be so much fun, right? That you are
comparing the first of 100 arrays, then it will be very long. That's why we are just using the
heap to make it easy."
    },
    {
      "start": 1077.84,
      "end": 1110.13,
      "text": "So let's write a heap. How do we write a heap? We have taught you about pairs,
C++, Java, Python. We will show you C++. You will get the code of Java and Python in this doc.
There you can see how a heap is made. So make a class in C++. Because you have to store three
things. You don't have to store pairs, you have to store three things. So in the class node,
we will keep one value, one row, and one call."
    },
    {
      "start": 1110.29,
      "end": 1147.54,
      "text": "and by making a constructor of node, int v int r c and even if we don't do this,
 it will be fine yes, we will do it, no problem after that, value is equal to v, row is equal
to r column is equal to c ok, you have made this, this class is made how will you make your
comparator? struct compare, struct cmp"
    },
    {
      "start": 1151.38,
      "end": 1185.76,
      "text": "operator, all this is explained in the pair method, you will also get this code,
 so don't worry that you don't understand anything, if you don't understand, then it will come,
 const nodeA, const, this is just syntax, once it is remembered, then it will happen, return,
we will also give it to you for java python, it is the same, java python, in java you will
have to use that trick of a-b-b-a, in python you will have to write that lt function by
yourself,"
    },
    {
      "start": 1185.76,
      "end": 1220.78,
      "text": "close this so you have to write this in the beginning you have to make a heap
what does it say? you have to make a heap of 3 sizes and according to the value it should be a
min heap ok now let's write the real code so what will be given to you? you will have to
return a vector and you will be given a 2D array what does this 2D array say to you? it has a
lot of sorted arrays that means its row is sorted every day this is what it says vector int oh
sorry it won't be like this vector vector"
    },
    {
      "start": 1221.46,
      "end": 1252.19,
      "text": "int, int, a. Okay? So, first of all, what to do? n is equal to a dot size.
Means, how many rows are there in this? int, m is equal to a0 dot size. We are assuming that
you know 2D matrix. Let's make it. Priority, q. Now, what type of priority q do we have to
make? You have to make it in the type of node. This node that we have made, this node. This
node says that this node is a data structure. You understand that in which three things can be
stored."
    },
    {
      "start": 1252.19,
      "end": 1284.03,
      "text": "We can store two things in pair, one thing in int, and many things in array.
There are three things in node. What are they? These three things. This is what it means. We
have to make a node type and vector node because we want to give a comparator function in it.
So, you make cmp pq. For i is equal to zero, i less than n, i plus plus. What did we say?
Insert the first one of every sorted array. How will we insert?"
    },
    {
      "start": 1284.03,
      "end": 1313.74,
      "text": "pq.push make a curly bracket what do you have to insert? what do you have to
insert? of every array of every array means of every row the initial element of every row the
initial element means array of i 0 this is the value then you have to insert the row what is
the row? i you are going on that row i is going from 0 to n and column number which column is
being inserted first? the column of 0 that's it, it is inserted"
    },
    {
      "start": 1314.03,
      "end": 1346.93,
```

```
      "text": "now make an output array for yourself now make an output array for yourself and
let's start the loop so while pq.not empty means until it is empty, until you keep running
what you have to do first of all node n is equal to pq.top this is similar like if you make
int's pq int p is equal to pq.top this is of pq type and this is of node type so node n is
equal to node is a data structure node is just like int or string you have made it ok"
    },
    {
      "start": 1346.93,
      "end": 1379.18,
      "text": "pq.pop. Okay. Let's extract all the things. int v is equal to n.value. Let's
remove it like this. There are three things in n. There is value, row and column in n. int row
is equal to n.row. int column is equal to n.column. What is the first step? The first step is
to do rest.pushback. Put it in your result first. After that, if"
    },
    {
      "start": 1379.66,
      "end": 1410.67,
      "text": "column is less than m-1. So, pq.push, what do we have to push? First of all,
the column of that row, plus 1. And we don't have to push only this. And the row will remain
the same. Which column? The one with column plus 1. The next one, the next one. There are
three things, don't leave one by pushing one. There will be three things in the heap, and this
curly bracket presses those three things in a node. Close this."
    },
    {
      "start": 1411.44,
      "end": 1441.25,
      "text": "Return, Rest. That's it. The program is finished. Look at this. Java, Python.
You will get the code. Today, we will give the code in the doc. See, there is no problem in
giving this type of code because it is a syntax-heavy code. Because we have understood what to
do, how to do it, how to do the syntax, how to change the three of the node. We understand
that. We will give that to you. So, this was Merge K Sorted Arrays. If you have understood
this, then definitely try one thing. There is a question whose name is Merge K Sorted Lists.
That is the question of the link list. So,"
    },
    {
      "start": 1441.25,
      "end": 1471.18,
      "text": "What is the number of this? LC... LC23 It's a hard question Same question Merge
K Sorted Lists Means, Link List Not Array, Link List Tell me Array because it's okay To
understand the concept, Array is very good Because a relation is coming from two pointers So
this is it So definitely go and code it You will get its link Merge K Sorted Array You will
get the link in the sheet So definitely do that Okay, let's go"
    }
  ]
}
```


2. transactions/t_11.json


```
{
  "video_id": "5IV8CgWnf04",
  "title": "Rotate A Linked List | Master DSA in 2026 With DSA Patterns",
  "language": "en",
  "model": "large-v3",
  "duration": 2374,
  "segments": [
    {
      "start": 0.0,
      "end": 25.2,
      "text": "Although this question is quite hard because it is asked in Microsoft,
Bloomberg etc. Easy questions are not asked there. But because you are reading from me, you
will feel that this question is easy of the easy. Means, it is more easy than easy. So, those
who are coming for the first time, watch the playlist from the beginning, don't watch it in
the middle. You will get the code etc. in the comments. And today's question is Rotate a link
list. Along with it, this last video of the link list reversal that was going on for many days
is going to happen. Let's move ahead."
    },
    {
      "start": 25.42,
      "end": 55.23,
      "text": "Yes, now there are two approaches to this. One approach is in which you use
reverse in actual. We will see that too later because it is necessary to see that too. And
there is one approach which is very simple. We were making it just now, then it came to our
mind that we can also make it like this. Because we were going to reverse it first and teach
you. Reversal is used in this, but we thought it would be very complicated. Means there are a
lot of edge cases. If you want to see that approach, then see it till the end. We will discuss
that too. But the right approach to make it is very easy. Means it is very easy, very
```

```
intuitive, very easy to understand."
    },
    {
      "start": 55.23,
      "end": 83.23,
      "text": "Let's go. So, what does the question tell you? The question doesn't tell you
anything. The question tells you that you have a linked list. Okay? It's an obvious thing that
if there is a reverse linked list, then you will have to have a linked list. Your question
says that this is it. Make one more. Let's go. Along with this linked list, you are given a k
is equal to 2. What does this k mean? You have to rotate this linked list to the right. How?
You have to rotate it. What does rotating mean?"
    },
    {
      "start": 83.23,
      "end": 111.86,
      "text": "Rotate to the right. Rotate to the right. What does this mean? I will tell you.
K is equal to 2 is given, right? Yes, K is equal to 2 is given. You have to rotate in two
places. First of all, every node, by taking every node, rotate in one right. For example, this
one node will come in second place. Which place was this? This was the first place. Now it has
come in second place. Why? Because you rotated in right. So when you rotate every child in
right, then the child who will stand in first place will come in second place. Similarly,"
    },
    {
      "start": 111.89,
      "end": 141.07,
      "text": "The kid who stands on 2nd place will be 3rd. The kid who stands on 3rd place
will be 4th. The kid who stands on 4th place will be 5th. And the kid who stands on 5th place
will be 6th. But, but, but, the kid who stands on 6th place, where will he go? You can't make
the kid on 7th place because there are 6 kids. Why would you make the kid on 7th place for 6
kids? So, the kid who stands on last place, where will he go? On 1st place. The kid who stands
on last place, where will he go? On 1st place."
    },
    {
      "start": 141.07,
      "end": 165.65,
      "text": "How many times did you rotate this? Once, once. Do this thing again because you
have to rotate it twice. Do this thing again. The child who is in the first place, who is now
6, will come in the second place. The second one will come in the third place. The third one
will come in the fourth place. The fourth one will come in the fifth place. The fifth one will
come in the sixth place. And, and, and, and, sorry, there was 5 here. There was 5 here."
    },
    {
      "start": 165.71,
      "end": 193.31,
      "text": "5 came from there. Yes. Now, see, where will this child go in the 6th place? He
will go to the 1st place. Because see, the 1st place is empty. So, where will the 5th child
go? Here. That's it. Once this link list is made, attach the arrow. Attach the arrow. And, and,
 the new head of this, the new head, which you think is 5 right now, return it. So, this is
the question. Rotate the link list. Okay."
    },
    {
      "start": 193.31,
      "end": 221.07,
      "text": "Similar question is also used in array. Similar question is also used in
rotating array. Rotate an array to the right by k. But this question is a little hard. But
when you are told to rotate a linked list, this question is very easy. This question is very
easy for linked list. Why? Because it is the property of linked list. The property of linked
list is that you can break it from anywhere and connect that part anywhere. What is the
property of linked list? For example, if you talk about array. If you talk about array, there
is an array or vector or anything."
    },
    {
      "start": 221.07,
      "end": 247.42,
      "text": "so you can't do it like this that you break it from here means you break one
part and bring this part in the beginning and join it this is difficult to do in array this is
not difficult it is impossible but but in linked list it is very easy to do this for example
by doing this you get a linked list by doing this you get a linked list you want to break the
last two nodes break the last two nodes separately so you can break it just null it it is
broken after that"
    },
    {
      "start": 247.42,
      "end": 259.73,
      "text": "If you want to make these two notes stand in the beginning, then you can do it.
You can do it. What can you do? Point the next note on this and put it on the head."
    },
```

```
    {
      "start": 259.73,
      "end": 280.66,
      "text": "What you can do is, you break the last part of the linked list and put it back.
This is very easy to do. In linked list, it is not easy to do it in array, it is impossible.
In array, you can't break these two blocks and put them in the last, it is not possible. But
because in linked list, every block is of chain type, so you can do it easily. For example,"
    },
    {
      "start": 280.66,
      "end": 309.58,
      "text": "If you have seen the train, if you travel by train, then sometimes you will
hear that the train is getting delayed because the engine is changing, the engine is going
from there to there. So what does the train do? See, in the beginning, the engine stays here.
Suppose in the beginning the engine is here and all its bogies are connected like this. All
its bogies are connected. Now the train was going home, but suppose the train has to go there
now. So what do these people do? These people break the engine. They break the engine from
here. This one removes the chain, the train is also connected to the chain and"
    },
    {
      "start": 309.78,
      "end": 337.28,
      "text": "By lifting the engine, where does he put it? First, he lifts the engine and
puts it here. And by doing this, he adds something. Now the train starts moving. So it is very
easy to do it in the train. Why is it easy to do it in the train? Because what is the train?
It is a linked list. The train is connected to everyone. So the linked list is now. Come, now
let's see what to solve this question. So what is happening in this question? What do you
understand by K-Rotate? We saw that when our linked list looked like this. 1, 2, 3, 4, 5."
    },
    {
      "start": 337.28,
      "end": 367.3,
      "text": "6. This was your original linked list. So whenever you will ask any linked list
question, then the main thing in linked list question is connections. Connections means the
wire which is connected, you have to modify to connect this wire, you have to change this wire,
 you have to do whatever you want to do. Let's see that when you will rotate it 2 times, k is
equal to 2, means if you will rotate it 2 times, then what will happen? What happened above?
What happened above? 5 and 6 came in front and 1, 2, 3, 4 remained as it is. So let's write
its answer."
    },
    {
      "start": 367.3,
      "end": 397.09,
      "text": "Assume 5, 6, 1, 2, 3, 4. Add an arrow. Now see, if you are solving any linked
list question, then you have to see the linked list and the linked list that we want in the
answer. What is the difference between them? What is the difference between them? What is the
connection and which connection do we need to make? So, let's see. What? What is this
connection?"
    },
    {
      "start": 397.09,
      "end": 421.7,
      "text": "Is this arrow, arrow 1, which is from 5 to 6, which is your answer, this is
your final answer, and this is your input. So has the arrow from 5 to 6 changed? No, the arrow
from 5 to 6 has not changed, you can see, 5 to 6 was there in the original as well as in your
answer, so there is no need to do anything in this. But, has the one from 6 to 1 changed? Yes,
because in the original, 6 had no arrow, 6 used to point to the null, but"
    },
    {
      "start": 421.7,
      "end": 451.38,
      "text": "In the change, 6 started pointing to 1. So we will see this. We will see this.
We know that we have to change. Has the arrow from 1 to 2 changed? No, no, no. The arrow from
1 to 2 has not changed. Because it is in the answer too. It was here too. So very good. Has
the arrow from 2 to 3 changed? No, no, no. The arrow from 2 to 3 has not changed. The arrow
from 2 to 3 was there and here too. Has the arrow from 3 to 4 changed? No, no, no. The arrow
from 3 to 4 has not changed. It is in the answer too. It was in the input too. But,"
    },
    {
      "start": 451.38,
      "end": 481.5,
      "text": "Has the arrow of 4 changed? Yes. The arrow of 4 points to 5 in the input. But
in the answer, the arrow of 4 points to NULL. So how many arrows did you get to know? How many
connections do you need to change? One connection you need to change is 6 to 1. You will have
to make this. You will have to make this. Because look carefully. In the answer, it is 6 to 1.
It was not in the input. It was not in the input. More, more, more."
    },
```

```
    {
      "start": 481.5,
      "end": 502.35,
      "text": "You will have to remove the connection of 4 from anyone. You will have to
remove it. You will have to remove it. Because in the input, 4 was pointing to someone. It was
pointing to 5. But in the answer, 4 does not point to anyone. In the answer, 4 does not point
to anyone. If you do these two things, then all your connections will be correct. All your
connections will be correct."
    },
    {
      "start": 502.45,
      "end": 527.6,
      "text": "All the connections will be correct, right? What was the difference between
your answer and your input? There was a difference of only two arrows, right? Now if both the
arrows are correct, then your answer and your input will match. Yes, the question arises, what
will be the head? What will be the head? What will be the head? So let's see what will be the
head, let's go ahead and see. Come on. So much has been understood, now come. Now what do you
have to do? See what you have to do. If you had such a link list, now we will start talking
about the number. Now we will start talking."
    },
    {
      "start": 527.6,
      "end": 555.95,
      "text": "On K. K can be anything. K is not always 2. K can be anything. Okay. You got to
know that K is 2. What did you do in T2? What did you do in K2? What will be your answer? What
will be your answer? Your answer will be this. 5, 6, 1, 2, 3, 4. This is your answer. This is
your answer. Okay. Now let's see how this answer is made. Now how is it dependent on K? How is
it dependent on K? See."
    },
    {
      "start": 556.24,
      "end": 579.98,
      "text": "What did you do in this linked list? If you look carefully at what you did, you
broke the last two notes, broke the last two note part and added it in the beginning. Added it
in the beginning. Okay? Have you done this? Have you done this? Okay. And what did you do? The
four notes of the beginning, the four notes of the beginning, picked it up and did it in the
second."
    },
    {
      "start": 581.71,
      "end": 606.16,
      "text": "first part is done in second and second part is done in first how did you know
that we have to bring 4 at the beginning and 2 at the end? by K if we would have taken 7 notes
in this then see what would have happened 1,2,3,4,5,6,7 and K is equal to 2"
    },
    {
      "start": 606.16,
      "end": 634.96,
      "text": "we are taking 2 because 2 is easy to understand so if you rotate it 2 times
then what will be your answer? your answer will be 7 6 1 2 3 4 5 see what happened see what
happened carefully see what happened carefully pick up the last 2 nodes and do it in the
beginning and pick up the 5 nodes in the beginning and do it in the second so what is
happening? we have to pick up the last k nodes"
    },
    {
      "start": 634.96,
      "end": 662.72,
      "text": "k nodes, we have to do our work on this and in the beginning, n-k nodes this is
your first part which you want to send in the second part so we got to know that k means that
you have to do some work on the last k nodes now your question will get clear slowly now what
we saw that which connection we have to make and which we don't have to make let's see which
connections you make let's go"
    },
    {
      "start": 662.72,
      "end": 695.66,
      "text": "Make a big linked list so that there is no confusion. We don't want any doubt.
We have made a linked list of 8 sizes. Okay? Okay? k is equal to 2. Let's see what you have to
do. First of all, you have to take the last two notes and put them in the beginning. 7 and 8.
And then you have to do the first 6 notes later. Okay? Okay?"
    },
    {
      "start": 696.21,
      "end": 727.57,
      "text": "Let's see what we need and what we have to make. The first thing which is
different in your input and answer is 8.next is equal to 1. You have to do this because this
arrow is in your answer and that arrow is not in your input. One more thing is that you have
```

```
to do 6.next is equal to null. Why?"
    },
    {
      "start": 727.57,
      "end": 757.68,
      "text": "6 has something in the input, but 6 has nothing in the answer. So, you have to
null 6.next. What should be your new head? What should be your new head? Your new head should
be 7. 7. What does 7 mean? 7 is the first node of the last two parts. So, what is 7 here? 7th
node. Your 7th node should be your head."
    },
    {
      "start": 757.68,
      "end": 786.96,
      "text": "So, see what happened with Gouar. What was your number of nodes? 8. Your k is
equal to 2. Let's write it in n and k. We have just seen that we have to do this for the 8th
node, this for the 6th node, and this for the 7th node. But, if we don't see it and make it,
then let's try to write it in a formula. Let's see what happens every time. We will take more
examples. We will take n and k separately and see if it matches or not. So, see. Your number
of nodes is 8."
    },
    {
      "start": 786.96,
      "end": 815.94,
      "text": "Your k is 2. First of all, let's say 8.next is equal to 1. 8.next is equal to
null. Sorry, 1. 8.next is equal to 1. So, see, what is 8? 8 is its last node. And what is 1? 1
is its first node. So, can we say that the first thing you have to do is last.next is equal to
head."
    },
    {
      "start": 815.94,
      "end": 844.06,
      "text": "Last dot next is equal to head. Look carefully. When there were 6 nodes, sorry,
when there were 7 nodes, then what was it? When there were 6 nodes, then what was it? 6 dot
next is equal to 1. Look here too, the number of nodes was 6, which means last node, which is
6, dot next is equal to 1, dot next is equal to 1, head. One thing is clear that last dot next
is always happening on head. The first thing is done. The first connection is connected. And,"
    },
    {
      "start": 844.06,
      "end": 873.15,
      "text": "See, 6.next is equal to null. Now, 6 is the 6th node here. How do we know this
today? 6.next is equal to null. Why did this 6.next is equal to null happen? What was this 6?
See, this 6 was the last node of this part. How many nodes were there in this part? n-k nodes.
Its last node, its last node. Means, which number node will it be? n-k node.next."
    },
    {
      "start": 873.15,
      "end": 900.21,
      "text": "is equal to null. 8 minus 2. 6th node next is equal to null. Your work is also
done. What is the third work left? Making the head of 7th node. How did 7th come? See.
Whichever node, its last. Now who will make the head? The first one of last k nodes. So can we
call it like this? N minus k plus 1 node."
    },
    {
      "start": 900.27,
      "end": 929.5,
      "text": "What will happen to you? Your head will be like this. According to me, these
are the three formulas. Come, let's test it more. Let's take another example and see if it
works or not. Come, make a link list of 7. 1, 2, 3, 4, 5, 6, 7. Okay? This is your input link
list. We are just seeing that what we thought, the formula we thought, is it correct or not?
And this is the only way. No one remembers. We don't remember that we have to do this or that."
    },
    {
      "start": 929.5,
      "end": 958.67,
      "text": "Think about it. If you want to make an arrow like this, then what will you make
for it? Let's think about it. Okay, this is it. Your number of nodes is 7. Assume that k is
equal to 3. Let's not take 2, let's take 3. We are taking 3 randomly. This was also not
planned from the beginning. Let's go. First of all, we say to head the next of last. So, what
is the last node of this? 7th node. It says to head the next of this. Let's make the next of
this a head. Let's see if we get the answer or not."
    },
    {
      "start": 958.93,
      "end": 987.2,
      "text": "N-kth node dot next is equal to null. N-kth node dot next is equal to null. N-
kth node dot next is equal to null. N-kth node dot next is equal to null. N-kth node dot next
```

```
is equal to null. N-kth node dot next is equal to null."
    },
    {
      "start": 987.2,
      "end": 1011.31,
      "text": "Which means, make the 5th node your new head. So what is the 5th node? This is
your new head. And return this. Let's see if it works or not. Let's see how this linked list
will look like. First comes 5, then 6, then 7, after 7 directly 1, then 2, then 3."
    },
    {
      "start": 1011.31,
      "end": 1039.73,
      "text": "Then 4, isn't this done? Isn't this rotate by 3 places? 3 notes of last came
first, 4 notes of beginning came later, so it's done, rotate by 3 places. Rotate by 3 places.
Yes, yes, rotate by 3 places, means everything is correct. Means, what you have to do, you
have to do 3 steps, first of all, you have to do last.next is equal to head. You have to do
this first step, after that,"
    },
    {
      "start": 1039.73,
      "end": 1055.98,
      "text": "you need n-kth node so that you can null its next and you also need n-kth node
because you have to make its head"
    },
    {
      "start": 1055.98,
      "end": 1083.66,
      "text": "You have to make it a head, that's why. And what do you need to do all this?
You need n. What is the number of nodes? You will know the number of nodes. That's why you
will subtract n minus k, then you will subtract n minus k plus 1. So you need the number of
nodes as well. You also need last. Last means last node. Last means which node? Last node,
last node, last node. So you will have to subtract all this. Once you have subtracted all this,
 then you can do your answer in a three-step process. But there is one more doubt in this.
There is a doubt or you can say there is a trick in this."
    },
    {
      "start": 1084.05,
      "end": 1109.14,
      "text": "Suppose I have 3 nodes. So you are assuming that my k is less than n. This is
not true. Your k can be anything. Suppose your k says that k is equal to 4. Now what do you
do? Now k is equal to 4. So what did we say earlier that if there is any linked list then in
the beginning there are n minus k parts and in the end there are k parts. We have to do this
and that in this."
    },
    {
      "start": 1109.36,
      "end": 1139.84,
      "text": "Now, if you see, your n is equal to 3 and k is equal to 4. So, what will be the
n minus k part of the beginning? 3 minus 4 minus 1 part. Minus 1 part doesn't make sense. In
this, that formula will be useless. Come, let's see what we will do in this. Come, let's solve
it manually. k is equal to 4, right? You know to rotate by yourself, right? Guess every node
on the right. Write the last node on the first. Come, let's do it once. If we rotate it once,
it will look like this. This is done once. Okay? We have to do it four times, right? Let's do
it. Do it one more time. So, this will become 3."
    },
    {
      "start": 1139.84,
      "end": 1168.35,
      "text": "This will be 1, this will be 2. Okay, 2 times are done. Do it 2 more times.
Bring 2 here, bring 3 here, bring 1 here. How did you do it? Like this. And 1 there, okay? 3
times are done. Do it one more time. 1, 2, 3. How did this happen? Here, here, 3 there. Yes,
so this is your answer. This is your answer. How many times? Rotate 4 times. Rotate 4 times.
So how to rotate 4 times? Look, think carefully, open your mind."
    },
    {
      "start": 1168.35,
      "end": 1198.02,
      "text": "This was your original. This was your original. This one, had to rotate once.
This one, had to rotate twice. And this one, see, 1, 2, 3, it became its original again. It
became its original again. Rotating three times, it became its original again. It became its
original again. And after this, 3, 1, 2. See, 3."
    },
    {
      "start": 1198.02,
      "end": 1226.06,
      "text": "1, 2, 3, 1, 2. It became the same again. It became the same again. What does it
mean? What does it mean? If you have a linked list. If you have a linked list. 1, 2, 3. This
```

```
is k is equal to 0. This is original. This is original. When you did it on k is equal to 1, then it became 3, 1, 2. Is it good? When you did it on k is equal to 2, then what became? 3, 1. "
    },
    {
      "start": 1226.06,
      "end": 1241.34,
      "text": "2. Okay. But as soon as you reached k is equal to 3, this list became original again. 1, 2, 3 became. It got reset again. And when you do k is equal to 4, then what will happen? Again 3, 1, 2."
    },
    {
      "start": 1241.34,
      "end": 1267.18,
      "text": "Again, 3, 1, 2. Meaning, K is equal to 4 times rotating. It is same as K is equal to 1 times rotating. Why? Because on K is equal to 3, it was reset. What do you understand from this? K is equal to 0 times. Meaning, rotating 0 times. And K is equal to 3 times rotating. Same thing."
    },
    {
      "start": 1267.18,
      "end": 1292.82,
      "text": "K is equal to 1 time rotate and K is equal to 4 time rotate is also same. K is equal to 2 time rotate and K is equal to 5 time rotate is also same. Means, if you have any K, then you can do K is equal to K modulus N. What will happen with this? Suppose, you have K is equal to 4 and N is equal to 3. What will you make K?"
    },
    {
      "start": 1292.82,
      "end": 1320.21,
      "text": "4 modulus 3 is equal to 1. So, you rotate k is equal to 1 time. Because it will be less than 3. You know how to do this. You don't know how to rotate 4 times. Because our entire logic is based on this. On n minus k and k. So, if k will be greater than n, then how will we do it? So, we can use this trick. That k is equal to 4 times rotating. It's same as k is equal to 1 time rotating. Why? Because when you do k is equal to k modulus n, then it gets smaller. And we have seen, see,"
    },
    {
      "start": 1320.21,
      "end": 1349.47,
      "text": "k is equal to 0 times is same as k is equal to 3. k is equal to 4 is same as k is equal to 1. And k is equal to 5 is same as k is equal to 2. So, you can do this. So, as soon as we take out n, what will we do? k is equal to k modulus n. To handle whom? When k is greater than n. Because it is possible. Let's go. So, you want these things. Now, let's see how these things will come out. So, let's make a linked list. Let's make a linked list."
    },
    {
      "start": 1349.47,
      "end": 1380.51,
      "text": "Now we will make the real one, not just the one-two one. So let's make a 5 size link list. This is done. 100, 200, 300, 400, and 500. So this is done. 1, 2, 3, 4, 5. You have given such a link list. So let's write the address in it too. Okay. Let's see. Now all the things are calculations. What are all the things? It is a simple calculation. That from where to start t, from where to stop t."
    },
    {
      "start": 1380.51,
      "end": 1408.3,
      "text": "The whole concept is done, so we will try to do the same. Okay? What did we see? Which three things did we need? Where did it go? Did you forget? Yes, these three things were needed. First of all, what did you need? N. And first of all, you needed last. Last note is also needed, N is also needed. So, there is a very good way to remove it. You can remove both at once. See, you can do that first reverse once and remove N. Then reverse once and remove the last note. But there is a very good way to do both. Okay."
    },
    {
      "start": 1408.98,
      "end": 1434.26,
      "text": "First of all, we will see that if head is equal to null, then return head. What is there in that? There is nothing to do in that. Okay. If head is equal to null, then return null. So, there is one in head. There is one node in head. So, what will we do? We will take n is equal to 1. There is one node, right? Head is not null, so there is one node at least. And for the start, we will take last is equal to head."
    },
    {
      "start": 1434.26,
      "end": 1463.31,
```

```
      "text": "For the start, what is last? Last is our first node. Let's assume there is one
node. So when there is one node, then there is our last head. After that, if we do this, while,
 while, last not equal to null. See, you need last node. When you used to do traverse, what
did you do in traverse? While t not equal to null. In that, you wanted t to be null, then we
will stop. But here, as soon as your last is 500, then you have to stop. And what is the
property of last node? What is the property of last node? Last dot next."
    },
    {
      "start": 1463.54,
      "end": 1488.94,
      "text": "It is always null. That is why it is the last node. If it is not null, then why
is it the last? It is the second last. What is the property of the last node? It is null. We
will write while last.next not equal to null. It means till it is not null, it is the last
node. And what will we do? n++ and last is equal to last.next."
    },
    {
      "start": 1490.1,
      "end": 1518.53,
      "text": "Will this work or not? The simplest way is to write it in a copy. No one
remembers this. No one in the world remembers this. Every time a person thinks about it, we
also think about it. We also don't remember. It may be wrong. Let's go and see. In the
beginning, n is equal to 1, last is equal to n. Meaning, n is equal to 1 and last is equal to
100. Let's run it. Is last's next null? Is 100's next null? No, no, no. It will go inside.
What will it do to n? 2."
    },
    {
      "start": 1518.53,
      "end": 1545.1,
      "text": "What will you do to the last? Last connects 200. Very good. Check again. Is the
last connect null? Is the 200 connect null? No, it is not. What will you do to the end? You
will increase it. What will you do to the 200? 300. Yes. Check again. Is the last connect
null? Is the 300 connect null? No. Increase the end. Increase the last. Check again. Is the
last connect null? Is the 400 connect null? No, it is not 400. It is 500. Increase the end."
    },
    {
      "start": 1545.1,
      "end": 1574.62,
      "text": "Increase it. Now, what is the next null of last? Yes, yes, yes. The next null
of 500. Stop. The while loop is broken. See, what did you get? n is equal to 5 and last is
equal to 500. Did you get it? Both things were found at the same time. Both things were found
at the same time. If you want, you can do it differently. But if both things were found at the
same time, then do it at the same time. Come on. Last was found. n was found. What did you
get? Last was found and n was found. Now, what do you want? Now, what do you want?"
    },
    {
      "start": 1574.62,
      "end": 1604.21,
      "text": "Now, what do you want? n minus kth node and n minus k plus 1th node. Yes, you
want these two. How will you get these two? See, n minus kth means, what do you want? You know
that n is equal to 5. 5 minus 2. k is equal to 2. k is equal to 2. So, you want 3rd node. And
n minus k plus 1 means 5 minus 2 plus 1. 4th node. Tell me one thing."
    },
    {
      "start": 1604.21,
      "end": 1632.62,
      "text": "when we take out the third node and if we take out the third node and if we
take out the third node and if we take out the third node and if we take out the third node
and if we take out the third node and if we take out the third node and if we take out the
third node and if we take out the third node and if we take out the third node and if we take
out the third node and if we take out the third node and if we take out the third node and if
we take out the third node and if we take out the third node and if we take out the third node
and if we take out the third node and if we take out the third node and if we take out the
third node and if we take out the third node and if we take out the third node and if we take
out the third node and if we take out the third node and if we take out the third node and if
we take out the third node and if we take out the third node and if we take out the third node
and if we take out the third node and"
    },
    {
      "start": 1632.62,
      "end": 1656.24,
      "text": "Our k is equal to 0. It is possible that if k was 5, then k would have become 0.
 5 modulus 5. So what will we do? Nothing. It will not make any difference. If you are
rotating something 0 times, then what will happen? Nothing will happen. Return head. So there
is no need to do this. There is no need to do this. But if this is not the case, then you will
have to take out n minus k. Come, let's see how n minus k comes out. Come."
    },
    {
```

```
      "start": 1656.24,
      "end": 1686.66,
      "text": "Now let's see how n- comes out. Make a link-place again. There is no such thing
in making a link-place that it is good again and again. Make it. We will copy and see what we
have to do. This is 100, 200, 300, 400, 500. 1, 2, 3, 4, 5. This is 200, this is 300, this is
400, this is 500 and this is null. Remember what we have taken out."
    },
    {
      "start": 1686.66,
      "end": 1716.03,
      "text": "N's value is 5. Last's value is 500. And K's value is 2. So 2 mod 5, 2 will
remain. What do we want? What do we want? Int C is equal to what? N minus K. N minus K. N
minus K. Okay. Okay. N minus K means third node. Third node. How do we remove this? Note star.
T is equal to head. We will write the whole code. We are just writing this so that you know.
Okay."
    },
    {
      "start": 1716.03,
      "end": 1742.43,
      "text": "int count is equal to 1 means head and count is equal to 1 ok come while t
means while t not equal to null if if count equal equal to c means count equal equal to n
minus k break it immediately so for example if you want first node then you break head also if
you want first node"
    },
    {
      "start": 1742.43,
      "end": 1762.58,
      "text": "Now let's break on head. But what node do you want? You want n-k which means
third node. It hasn't happened yet. No problem. Count plus plus and t is equal to t dot next.
Like this you can remove any nth node. Let's see. Try this too. In the beginning, what will be
the count? 1."
    },
    {
      "start": 1762.58,
      "end": 1792.54,
      "text": "In the beginning, T will be 100. Check it. Count is equal to C. C means C is
equal to 3. You need third node. You can give some other name. It doesn't matter. So, is C 3?
No, C is not 3. Third node is not there. Increase the count. Make T 200. Is 2 equal to 3? No,
no, no. Make it 3. Make it 300. Is 3 equal to 3? Yes, yes, yes. Break, break, break. See what
you got. T is equal to 300. Your T is here. T is equal to 300. T is equal to"
    },
    {
      "start": 1792.54,
      "end": 1821.25,
      "text": "300. Which node did you get? We got n-kth node. What is its value? 300. What
else did you need? n-k plus 1. Why? To make a head. To make a head. To make a head. Now, we
need to make a head, right? So, what can we do? Node result is equal to t dot next. t dot next.
 t dot next was the node, n-k plus 1. Why did we need that? To make a head. So, we made it.
What value is stored in rest?"
    },
    {
      "start": 1821.25,
      "end": 1848.22,
      "text": "400. 400. Okay. Now, let's do our work. One work is done. Two more works are
left. What two works are left? Last.next is equal to head. What is last? 500. 500.next is
equal to 100. Come. Connect the wire. Connect the wire. Connected. Connected. Very good. And
what did we have to do with n-kth? What did we have to make n-kth? Null. Null. Null. Means,
whatever node it is,"
    },
    {
      "start": 1848.22,
      "end": 1878.61,
      "text": "So you can name it whatever you want, but 300.next is equal to null. Let's make
300.next null, then it will break. Null. And what is the rest? The rest is 400. So which head
did you make? 400. And return rest. Now see, what will happen? If you return, then what
increase did you make? First 4, then 5, then 1, then 2, then 3. Did it rotate 2 times? Yes, it
did. The last 2 came here, the first 3 came here."
    },
    {
      "start": 1878.61,
      "end": 1911.15,
      "text": "So, it's a simple program. Come, let's do the program. Come, let's do the
program. Come, let's write the program. So, let's start. What will you be given? Note star.
Any function will be given. And you will be given a head. And you will be given an int k. We
will give the code. If you don't agree, you say, no, copy and paste, then it's fine. This is
also a complete code, by the way. We will not write anything in it that you will not be able
to run in reality. First of all, if."
```

```
    },
    {
      "start": 1911.5,
      "end": 1941.22,
      "text": "head equal equal to null return null if there is nothing then what will you
rotate? there is no use in rotating but now first of all what we have to do? last is equal to
head we have to remove last and int n is equal to 1 because there is one node because head is
null then there is one node while what was the condition of last? next of last is always null
so till it is not null till it is not null"
    },
    {
      "start": 1941.22,
      "end": 1971.23,
      "text": "till then what we have to do is n++ and last is equal to last dot sorry last
dot next last dot next see why we are working so hard here you don't have to write such code
on ipad right we want you to get used to it it is very important to write code on copy or
somewhere else before typing this habit is very important to us it doesn't mean that we write
that ok remove last no till this habit is not applied till it remains that no the code is
written there"
    },
    {
      "start": 1971.23,
      "end": 1990.93,
      "text": "If someone gives you, then you will copy from there. No, it is a wrong habit.
My register is full of copies, in which we have written the code by hand. Until you get the
habit of writing by hand, you will not be able to learn. What will happen if you copy my code?
I mean, if you copy my code, what will be the benefit? I don't understand. You always give
code."
    },
    {
      "start": 1990.93,
      "end": 2019.38,
      "text": "there is a problem in the code, if we give it, then it will be the same code
that will be submitted, if it will not be submitted, then you will say that no, you have
written it wrong here, the submit did not happen, you have to submit the same code, so get the
habit of writing by hand, by yourself, now you have got the last one, ok, now what do you want,
 now what do you want, now you want n minus kth, so first of all, int, count is equal to 1,
did you take count is equal to 1 or did you take 0 above, you started with 1,"
    },
    {
      "start": 2019.57,
      "end": 2049.58,
      "text": "and if you want n-k then do that note star t is equal to head this t will tell
you whether you have got n-k or not write while t not equal to null if what yes one thing is
left now you have got n now you have got n now you have got n yes what you have to do k is
equal to k modulus n see one thing is left"
    },
    {
      "start": 2049.58,
      "end": 2078.19,
      "text": "What will you have to do? k is equal to and k modulus n. k modulus n. After
this, if k is equal to 0, if k comes, then what? There is no need to do anything. Return head.
Return head. There is no need to do anything special. There is no need to do anything special.
So, some people will say that k is equal to 0. Why didn't you check above? Because, because,
this can happen. n is equal to 5. And k is equal to 5. So, here k is not 0. But, when you will
do 5 modulus 5, it will become 0. So, it is not 0."
    },
    {
      "start": 2078.19,
      "end": 2109.65,
      "text": "It became zero. That means effectively it is zero. That is why it is important
to do it in the last. Now it is done. Return head up. Now come. Note star. T is equal to head.
For whom? To get the note of n minus k. Count is equal to 1. While t not equal to null. What
do you have to check? If count equal equal to n minus k. Break. Break. Only then you will get
n minus k. Otherwise."
    },
    {
      "start": 2109.65,
      "end": 2138.21,
      "text": "No problem. Count plus plus. t is equal to t dot next. That's it. Now you have
got all two things. Now you are ready. You have to do three steps. What three steps? Come,
let's do it. You have to do three steps. First of all, last dot next is equal to head. First
step is over. Note star result is equal to t dot next. Why t dot next?"
    },
    {
      "start": 2138.21,
      "end": 2164.99,
```

```
      "text": "t represents n-k and the head represents n-k plus 1 and the fourth one is the
rest and after that, what did you have to do for n-k? you had to make the next null t.next is
equal to null and return rest that's it, this is a small program this is a small program, this
program will be submitted"
    },
    {
      "start": 2164.99,
      "end": 2195.2,
      "text": "We will give it in the comments. We don't know what you will do with it, but we
will give it. No problem, we will give it. This will be submitted. So, you have learnt this
for the rotator inklis. This is the easiest way. You have learnt this for the rotator inklis.
Yes, we had said that we will also tell you about the reverse one. So, what is there in the
reverse one? See. The reverse one works like this, but it has a lot of edge case. Honestly, it
has a lot of edge case. But what was to be done in it? See. 1, 2, 3, 4, 5."
    },
    {
      "start": 2195.2,
      "end": 2218.86,
      "text": "what we did to it, how we thought about it, we thought like this, that suppose,
in this also you have to take out n, n is equal to 5, and you have to do k is equal to k more
or less, but after that what you do, you once reverse the entire linked list, reverse the
entire linked list, so it will look like this, 5, 4, 3, 2, 1, now if you say then there is no
answer, no there is no answer, reverse it, after that, there is a trick,"
    },
    {
      "start": 2219.34,
      "end": 2245.38,
      "text": "Keep the curve on the head and reverse it by K times. How will it look like? It
will look like this. 5, 4. It will look like this. We are not telling you to reverse this 4
for the first time. We are giving you homework. You have to do it yourself. Store your head.
This is your new head. Now, what do you have to do?"
    },
    {
      "start": 2245.38,
      "end": 2275.06,
      "text": "Reverse the next 3 notes as well. Reverse the next 3 notes as well. Reverse the
next 3 notes as well. And the first note that you got, you have to store it as well. Add it
here. Add it here. And return it. So, effectively, it becomes 4, 5, 1, 2, 3. But this is a bit
complex. Obviously, the thing is... Sorry, don't add it here. Don't add 5 to 3. Add 5 to the
prev of the 2nd reversed note. Add it to that. Add it here."
    },
    {
      "start": 2275.06,
      "end": 2301.01,
      "text": "If you do this effectively, then first comes 4, 5, 1, 2, 3. You have to reverse
it 2-3 times, but it will be done like this. It is a bit complex. Let's assume that it is a
bit complex. But where does it come from? It comes from array. When we were solving it in
array, we did this. Assume that your array was like this. Assume that your array looks like
this. 1, 2. This works very well in array. 0, 1, 2, 3, 4."
    },
    {
      "start": 2301.01,
      "end": 2332.02,
      "text": "n is equal to 5 and k is equal to 2 so what we used to do is we used to reverse
the whole array first 5 4 3 2 1 first step after that we used to reverse the two elements of
the beginning k elements of the beginning 4 5 the rest used to remain the same after that we
used to reverse the last k and minus k means we used to reverse the last 3 we used to reverse
the last 3 4 5 1 2 so after reversing 3 times we get the answer"
    },
    {
      "start": 2332.02,
      "end": 2345.09,
      "text": "1st Reverse is Full 2nd Reverse is K at the beginning 3rd Reverse is K at the
end"
    },
    {
      "start": 2345.09,
      "end": 2373.87,
      "text": "these three will be reversed and you will get the answer, you have to do the
same thing here but this approach was easy in array, it was hard in linked list, why? because
you had to add that link thing, so this approach was a little hard in linked list, the upper
approach was very good in linked list, the upper approach will be very hard in array that how
will you join, you can't add the last, the upper approach was very good in linked list, the
lower approach, the reverse one works very well in array, but you can do it in linked list, we
have done it, you also try, it is a little edge case, put a little mind, you can do it like
this, let's meet tomorrow, right?"
    }
```

```
  ]
}
```


3. transactions/t_12.json


```
{
  "video_id": "5Net0Dd7NAA",
  "title": "Solve Any HEAP Question By This Template | Top K frequent Elements Leetcode",
  "language": "en",
  "model": "large-v3",
  "duration": 2467,
  "segments": [
    {
      "start": 0.0,
      "end": 18.99,
      "text": "Hello, if you are new to this channel, then go and check the description. Even
if you are an old kid, if you haven't seen the last video of the Heap on Pairs series, then
turn off today's video right here. You won't understand anything, just watch that and then you
will understand. If you haven't seen this, then you won't understand the code and you won't
understand what we are doing."
    },
    {
      "start": 18.99,
      "end": 36.18,
      "text": "we will recommend you to watch this entire playlist because today we are going
to do two questions and we will make you do the first question and the second question is a
challenge for you that if you have understood the first question well then think about the
second question yourself 10% change will make the second question and do all this and see"
    },
    {
      "start": 36.24,
      "end": 56.91,
      "text": "Why are we doing this? It's not that we don't have time to teach. We can teach
and we will teach but when we teach from someone, let's say we give an example that we were
teaching from Stryver. So Stryver would have done more than 500 questions in his whole YouTube.
 We haven't solved 500, we have solved 1500. So why did that happen? If we would have just
watched his video and would have done the questions he was asking, then..."
    },
    {
      "start": 56.91,
      "end": 78.61,
      "text": "We don't learn, right? So by doing all this, you learn by yourself. If we just
tell you, you will listen to it, you will note it down in a copy and write it down, then you
will be able to learn very little. So this is a chance for you that yes, this will give you
confidence that it happened by itself. And we do all this because, we don't do it to
demotivate you or anything like that, but to boost your confidence."
    },
    {
      "start": 78.61,
      "end": 107.17,
      "text": "If we can do a similar 10-20% change with another pattern, then you will
understand that yes, we have got that concept very well. So watch this video, this question.
Till the end of this question, we will tell you a trick of heap, what to recognize, which heap
to put, max, min, where to put max, where to put min, how to do it, everything will be
understood, so let's start. Come, first we will explain the question, listen to the question
very carefully, we will tell you in Hindi."
    },
    {
      "start": 107.17,
      "end": 137.07,
      "text": "You are given an array and what do you have to return in this array? Frequent
elements of the top. Frequent elements of the top means, See, there are many elements in this
array. In this array, you must be seeing that if we talk about unique elements, then there are
elements of 1, 2, 3, 4. There must be some frequency of every element. Every element must have
come many times. For example, if we talk about 1, then 1 has come here once, 2 has come, 3 has
come. So, we write once in the next that 1 has come 3 times. It's a good thing. See 2. Once,
twice,"
    },
    {
      "start": 137.07,
      "end": 164.0,
      "text": "3 times. Nothing else? Okay. 2 has also come 3 times. Let's see 3. Where is 3?
It has come 1 time here. 1 time, 1 time, 1 time. So 3 has come 4 times. 3 has come total 4
times. And how many times 4 has come? So 4 has come here 1 time? It has come here 1 time. So
how many times 4 has come? It has come 2 times. Is it correct? Okay. For example, you have
been given in this question that k is equal to 2. The value of k is equal to 2 has been given
```

```
in the question. Okay. What do you have to tell?"
    },
    {
      "start": 164.0,
      "end": 178.29,
      "text": "Top K frequent elements, means these are all their frequencies. In these, the
top 2 frequent elements, see, top 2 frequent elements, first of all, there is 3. Most of the
times, 3 has come. 1 has become 3. Let's talk about the next one. Suppose, 4 times, it has
come only once."
    },
    {
      "start": 178.29,
      "end": 203.22,
      "text": "3 times you will see that 1 and 2, both are 3 times. You can give whatever you
want from 1 and 2. So you can say that 1. So we can see that 3 and 1 are top 2 frequent
elements. How many? Top 2. Why are you saying top 2? Because k is equal to 2. If k is 3, then
you would have given 2 as answer. If k is 4, then you would have given 4 as answer. So this is
your question. And the very important thing is that from n log n,"
    },
    {
      "start": 203.22,
      "end": 232.51,
      "text": "solve in less complexity means there should not be n log n it is about time
complexity there should not be n log n it is written in the question you can see that it is
written in the follow up that it should be made in less than n log n ok so top k frequent
element so first of all why this is a heap question so remember the pattern of the first day
if you have given any array and you have any top k or most k something like this top k
frequent greater k greatest k largest if this happens then it is a heap question and"
    },
    {
      "start": 232.51,
      "end": 258.32,
      "text": "After that, it seems that there is a max and a min. There is a trick for this,
which we will tell you in the end. But first, let's think a little about how we will do this.
Assume that you do not have the knowledge of the heap. Assume that you do not have the
knowledge of the heap, then what would you do? First of all, it is a matter of frequency here.
The frequency of the element will be decided according to which is the top k. The element is
not the greatest largest. The element's frequency has to be decided according to which is the
top k. And that element has to be returned."
    },
    {
      "start": 258.32,
      "end": 288.21,
      "text": "So the most simple solution that is coming to my mind is that if we sort the
array, If we sort the array, then my array would look like this. 1, 1, 1, 2, 2, 2, 3, 3, 3, 3,
4 and 4."
    },
    {
      "start": 288.21,
      "end": 317.33,
      "text": "0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11. If we had sorted it, then the array would
have become like this. Even now, we will not be able to do anything. What we can do now is
that we can remove the frequency of every element. How can we remove the frequency of every
element? Now see, all the ones that are 1 will come together. All the ones that are 1 will
come together. All the ones that are 2 will come together. All the ones that are 3 will come
together. All the ones that are 4 will come together. By doing this, we can remove the
frequency of 1 that 1 is 3 times."
    },
    {
      "start": 317.33,
      "end": 333.89,
      "text": "We can find out the frequency of 2, that 2 has come 3 times. We can find out
the frequency of 3, that 3 has come 4 times. And we can also find out the frequency of 4, that
4 has come 2 times. Okay. But there will be no benefit from this. Because we don't want
everyone's frequency. We want the top K. There is no benefit from sorting this as well."
    },
    {
      "start": 333.89,
      "end": 360.98,
      "text": "first thing and in sorting already nlogin will be used so you don't have to
make it in nlogin it is written in the question so there is no use in sorting because this
frequency you can take out normally if you will do it on array if you will use hash map then
you can take it out normally also so ok there is no use in taking out the frequency of every
element so now we need the frequent element of k ok now let's think about what else can be
done now let's see what can be done in this now"
    },
    {
```

```
      "start": 360.98,
      "end": 376.4,
      "text": "After looking at top k, we can understand that there will be heap here. But how will heap look like? Why? Will it look like max heap? Or will it look like min heap? And one more problem is that will we keep in heap? When we did kth largest element, then we kept number in heap. What will we keep here? Let's see what we will keep here."
    },
    {
      "start": 376.91,
      "end": 403.02,
      "text": "First of all, you have to see what the top k is. In the previous question, what was the kth largest element? So it was on the element that which is the kth largest element. Here you have to tell which is the top k frequency element. First of all, according to me, you should know the frequency of every element and it should be stored somewhere. You should know the frequency of every element and it should be stored somewhere. So what can we do in a hash map?"
    },
    {
      "start": 403.18,
      "end": 427.55,
      "text": "in one hash map, first we have to store every element and its frequency that 1 has come 3 times, 2 has come 3 times, 3 has come 4 times, and 4 has come 2 times, so we have to store this in the hash map, by putting a loop once, how much time will it take, it will take order of n time, so we are still not on n log n, it will take order of n time, now it is of top, we have to use heap,"
    },
    {
      "start": 427.55,
      "end": 454.64,
      "text": "Intuitively, it comes to our mind that the top k is the maximum k. So why don't we use maxheap? If we use maxheap, then it can work. Absolutely it can work. So see. After that, what will we keep in the heap? Can't we keep elements in the heap? If you insert 1 in the heap and then 2, then there will be no benefit. In the heap, you will have to insert the frequency. The thing on which you need the top k or the kth largest, you will have to insert that thing in the heap."
    },
    {
      "start": 454.8,
      "end": 484.4,
      "text": "But just inserting the frequency won't work either. For example, if you insert 3, 3, 4, 2 in the heap, and you get some answer in the end, then how will you know whose frequency is 3, 3, 4, 2? In HashMap, you can find out the frequency of 1. You can't find out whose frequency is 3. You can't search by value in HashMap. You can search by key in HashMap. You can ask, how many times is the frequency of 3? It's 4 times. You can't ask, which element's frequency is 4? That element will tell us."
    },
    {
      "start": 484.4,
      "end": 505.26,
      "text": "This doesn't happen in HashMap. So, in heap, you won't be able to work only with elements because you don't have to do anything with elements. You won't be able to work only with frequency because you need top frequency. It's a good thing. You will insert it in heap. But, what will happen after inserting? From there, suppose you get the answer in the last. Suppose, you get the answer in heap in the last. 4 and 3."
    },
    {
      "start": 505.26,
      "end": 534.5,
      "text": "It is the greatest frequency. 4 and 3 are mostly 2 frequencies. But whose frequency is this 4? Which element's frequency is this? We don't know. Which element's frequency is this 3? We don't know. You have to return the element in the final answer. You don't have to return the frequency. That's why, in heap, we have to insert both the things. What does both the things mean? In heap, we have to insert a pair. That's why we had taught heap on pairs in the previous video. Because this is very important in this question. This question cannot be made without a pair. In heap,"
    },
    {
      "start": 534.5,
      "end": 562.66,
      "text": "We will have to insert two things. First, we will have to insert the frequency. First, we will have to insert the frequency and which element is that frequency. How will this be inserted? By putting a loop on the hash map. If you put a loop on the hash map, then you get both the things. You get the key and you get the value. Value and key, we will insert them in the heap. By doing this, we will have to insert in the heap. Okay. So, we got to know what to insert in the heap. Okay. But,"
    },
    {
```

```
      "start": 562.66,
      "end": 581.25,
      "text": "Now, what will we do? First of all, is it a max heap or a min heap? Intuitively,
 it comes to my mind that if the top K is asked, then we will take a max heap. We can take it.
See what happens. Suppose you make a max heap. You make a max heap here. This is a heap. What
kind of heap is this? It is a max heap."
    },
    {
      "start": 581.25,
      "end": 599.34,
      "text": "Max is the pair here. The rule of both frequency and element is defined. That
it should be max according to the frequency and it should be max according to the element. So,
let's say that take both as max. Take both as max. Remember what we were studying yesterday or
the day before yesterday. That's why we are saying again and again that if you haven't seen it,
 then you won't understand it at all."
    },
    {
      "start": 599.86,
      "end": 619.18,
      "text": "How does the heap decide which element should go up from this pair? First, it
will look at the first. If the first is different, then according to the first, if it is max,
then it will be max. If it is min, then it will go to the min top. If the first is same, then
it will look at the second. If the second is min, then it will go up and down accordingly."
    },
    {
      "start": 619.34,
      "end": 649.34,
      "text": "Let's assume for a while that we make both as max just for simplicity purpose.
That we have made both as max. Now let's start inserting. So first of all, this comes, 1 and 3.
 So how will we insert this? We will make a pair in which the frequency will remain first and
the element will remain later. So what will we make? 3 and 1. Okay, so this is the first
insert. What is the second? 3 and 2. First, its frequency is 3. And which element's frequency
is 3? 2 car 3. Okay, this is also inserted. 3 and 4 means?"
    },
    {
      "start": 649.34,
      "end": 673.66,
      "text": "4 is the frequency and which element is it? This is also inserted. After that,
4 and 2 means 2 and 4 means that the frequency is 2 and the element is 4. So, you can see that
we have inserted the entire hash map in the heap. Now, what is this heap? This is the max heap.
 It is according to you, according to the need. How many entries do you want? You want top k.
Top k or top 2. In this question, you want top 2. So, we will not do anything."
    },
    {
      "start": 673.66,
      "end": 703.44,
      "text": "We will pop two entries from the heap. Which two entries will pop? First of all,
 in this max heap, who will be the first to pop? Who will be sitting at the top? Whose first
is the most? Whose first is the most? 4 and 3's first is the most. So when you will pop the
first, what will you get? 4 and 3. What does this 4 and 3 tell you? There was one element
whose frequency was 4. And which element's frequency was 4? 3's frequency was 4. So you can
write 3 in your answer. Very good. So now this element has disappeared from the heap."
    },
    {
      "start": 703.44,
      "end": 723.98,
      "text": "What will be the next element? See, for the next element, if we look at the
first, the first is the same of two things, 3 and 3, both are the same, but which one will be
sitting at the top? Whose second will be more? Why? Because we are saying that you stay at the
max on the first, and according to the second, you also behave like a max heap. So whose is
more? Two is more. So you will get this entry, which one?"
    },
    {
      "start": 723.98,
      "end": 743.7,
      "text": "3 and 2. Again, what does this entry tell you? The frequency of one element is
3. And which element is that? It is 2. And what will you find out from this? 2. So, you can
write 2 in your answer. You got the answer. You wanted the top K. You popped it twice. You got
the answer. You can return this very good. This question will be made. Very good. The question
is solved. But, but, but."
    },
    {
      "start": 743.7,
      "end": 767.38,
      "text": "Now let's take a look at the time complexity. We had told you that there are
only 3 operations in heap. Which 3 operations are there in heap? The first operation in heap
is to see the top. After that there is push or insert which you insert inside. After that what
```

```
happens is pop. In pop or java it is called offer."
    },
    {
      "start": 767.38,
      "end": 795.28,
      "text": "The time complexity of top is order of 1. It is a constant operation. But what
is the time complexity of push and pop? It is log n. What is this n? This n is not the size of
your array. Remember that this n is not the size of your array. It is the size of the heap.
What is the maximum size of the heap? The maximum size of the heap is decided by how much it
can grow in a time. Okay. So we have done this. We have seen this. Now let's see what you are
doing."
    },
    {
      "start": 795.28,
      "end": 819.7,
      "text": "First of all, we have to make a hash map in this. First of all, we have to make
a hash map with array. Whose? For frequency. So, in that, what is the order of n time? Because
if you do it once, then it will be n once. We will talk about space later. Let's do time only.
Okay. Plus. In heap, what are we doing after that? We are looping on every entry. We are
looping on every entry in hash map. Okay. So, at max, how many entries can there be? At max,
how many entries can there be? Number of unique elements. See, how big is your hash map?"
    },
    {
      "start": 819.7,
      "end": 847.49,
      "text": "1, 2, 3, 4. How much is the size of HashMap? 4. Where did this 4 come from?
Because in your array, the number of unique elements was 4. But the number of unique elements
of the array can also be N. If every element comes once, then the size of your HashMap and the
size of the array will be the same. Everyone's frequency will be 1. So let's assume that the
worst possible size of HashMap can be N. What is N? The size of the array is okay. N. What are
you doing for every entry? One, you are pushing. Where? You are pushing in the heap."
    },
    {
      "start": 847.49,
      "end": 863.18,
      "text": "so what is the time complexity of push? one push log n, n means this is not the
size of n array, the maximum size of heap can be at max so see how much the size of your heap
can be? the size of your hash map can be as much as the size of your heap, why? because"
    },
    {
      "start": 863.18,
      "end": 892.54,
      "text": "The size of your hash map, you are pushing every entry, every key value pair of
the hash map in the heap. So how much can the size of the heap increase? It can increase up to
the size of the hash map. And how much can the size of the hash map be? What did we see in the
worst case? The size of the hash map can be equal to the size of the array. Why? Because if
every element comes once, then the size of the hash map is equal to the size of the array. The
size of the hash map is the same as the size of the heap. So how much can the size of the heap
increase? It can increase up to the size of the array. It means that n can increase. So,"
    },
    {
      "start": 892.54,
      "end": 913.22,
      "text": "How much will it be to push and pop once? Log N. Now this N is the size of your
array. How many times are you pushing? You are pushing N times. Why? Because how many entries
are there in the hash map? There are N entries. If you are pushing every time, then N log N.
So see, your N log N came here. You don't have to do it in N log N."
    },
    {
      "start": 913.22,
      "end": 941.63,
      "text": "This is not allowed. We can do it. It will be submitted too. But n log n is not
allowed. Or n log n will stop you from interviewing. No, brother. This max heap that you are
doing. You are pushing every entry in that heap. After that, k times is popped. We are doing
its time complexity. See that too. Let's do it. After that, after pushing, what are you doing?
You are doing k times pop so that you get the answer. So you are doing k times pop. How much
is 1 pop? Log n. n means array size. k log n."
    },
    {
      "start": 941.63,
      "end": 967.14,
      "text": "Don't get confused here again and again. The log n of the heap tells you the
maximum size of the heap. Because in this condition, the maximum size of the heap and the
maximum size of the array can be the same. That's why I am writing n, otherwise I don't have
to write. Now your k log n will be less than n log n. So here you get to know that your time
complexity will be n log n. You don't have to do n log n, it is not allowed."
    },
```

```
    {
      "start": 967.14,
      "end": 996.88,
      "text": "Okay, now we know that we can't take max heap because to take max heap means
that we have to push sub and remove kmax. So, this is a problem. So, let's solve the problem.
It will be made. We will submit it and it will be made. But let's see if there is any other
way. So, for simplicity, we write our hash map like this. This was our hash map in which we
were storing elements and their frequency. Okay. 1 came here 3 times."
    },
    {
      "start": 996.88,
      "end": 1026.35,
      "text": "2 came here 3 times. Let's do this to teach you that 2 came here 1 time, 3 came
here 4 times, and 4 came here 2 times. Okay. This is your hash map. What do you want? You want
top 2 frequent elements. We saw that max heap didn't work. Why didn't it work? It worked.
Nlogin happened. We didn't have to do it in nlogin. So, now see what we can do. Now we will
tell you that trick that whenever in heap, whenever in heap,"
    },
    {
      "start": 1026.35,
      "end": 1057.57,
      "text": "you get this, kth largest, kth largest, kth biggest, means big, big words,
biggest, kth greatest, or top k, or most frequent k, like this, means, what happens with k, if
it is a big term, what should be max, always in these cases, minheap will be used, always in
these cases, minheap will be used, and sometimes you get, kth smallest, kth smallest,"
    },
    {
      "start": 1057.57,
      "end": 1074.48,
      "text": "or kth, shortest, whatever you can think of as parivachi of chhote bottom k
means lowest k means whatever you can think of as parivachi of chhote in this case maxheap
will be used"
    },
    {
      "start": 1074.48,
      "end": 1089.46,
      "text": "In this case, Maxheap will be used. Why didn't we tell you this earlier? Why
did we show you that Maxheap can also be used? But why didn't we tell you this? Because you
have to think that why can't it be used? Maxheap will take time to unlock. So remember this."
    },
    {
      "start": 1089.71,
      "end": 1117.87,
      "text": "If Keth is big, then we have to use min heap. If Keth is big, then we have to
use max heap. If Keth is large, then we have to use min heap. If Keth is small, then we have
to use max heap. Now I will tell you why this will happen. Just remember this. If Keth is the
biggest problem, then don't use max heap. No interviewer will allow you to use it. It will be
very easy."
    },
    {
      "start": 1117.87,
      "end": 1147.38,
      "text": "But it won't allow you to add interviewers. Okay? Remember this. And let's see
what can be the min heap in this. What can be the min heap? Let's go. There is a hash map.
What do you want? Top 2. Okay. Top 2 frequent elements. Okay? So, let's assume that there are
6 elements in this array. Let's assume that there are 6 elements in this array. Let's take 6
elements. Let's assume that consider an array which has 6 unique elements. Which has 6 unique
elements. Like this. 5, 6."
    },
    {
      "start": 1147.38,
      "end": 1176.21,
      "text": "How many times has 5 come? 5 has come once and 6 has come twice. Is it correct?
Very good. Let's not do 6 twice. Let's do 6 as 6. 6 has also come 4 times. According to this
question, what should be your answer? Your answer should be 3 and 6 because these are the top
2 frequent elements. So, let's try to write this."
    },
    {
      "start": 1176.56,
      "end": 1206.16,
      "text": "Let's sort the order of these elements with the order of their frequency. So
see, who has come the most times? The most times is 6. Okay? Less than that or equal to that?
After that, 3 has come. After that, which one has come? 1 has come, which has come 3 times.
After that, 5 has come, which has come 1 time. And in the last, 2 has come, which has come 1
time. Below this, we write the frequency. This has come 4 times, this has come 4 times. 1 has
come 3 times. Is anything left? 4, 4 is left."
    },
```

```
    {
      "start": 1206.61,
      "end": 1234.05,
      "text": "4 has come twice, 4 is small or what is small, sorry. So we are just sorting
this by the order of their frequency. After that, what has come, 4 has come twice and after
that 5 and 2 have come once. 5 and 2 have come once, 4 has come twice. So this is sorted, ok.
What is your answer? Your answer is this one. Top 2, top 2, 3 and 6 are your answers, ok. Look
carefully."
    },
    {
      "start": 1234.05,
      "end": 1263.52,
      "text": "6, now we will ask a question in every element that this element here, this
element here is 6, how many elements is it smaller than? According to the frequency, now
everything is from the frequency, now don't look at their values. 6, how many elements is it
smaller than? So you will get the answer that 6 is smaller than 0 elements. There is no
element bigger than 6. 6 is smaller than 0 elements. Now let's talk about 3. How many elements
is 3 smaller than? How many elements is 3 smaller than in terms of this? So now you will say
that it is 4, this is also 4, it is not smaller than anyone."
    },
    {
      "start": 1263.52,
      "end": 1285.66,
      "text": "Assume that according to this order, how many people are there in front of it?
Then you will get the answer that it is smaller than one person. This person, 1, is smaller
than how many people? It is smaller than two people. How many people are smaller than 4? How
many people are there in front of 4 who are sitting at a higher frequency than it? 3 people
are sitting. 5? How many people are sitting in front of 5? 4 people are sitting and in front
of 2? 5 people are sitting. Okay."
    },
    {
      "start": 1285.66,
      "end": 1312.1,
      "text": "What do you want? What do you want? What is in your answer? In your answer,
there are 3 and 6. Meaning, when k is equal to 2, you want such elements which are either
smaller than 0, or there are 0 elements in front of it, meaning, they are smaller than 0
elements, or they are smaller than 1 element. Or they are smaller than 1 element. If you get
an element which is smaller than 2 elements, then you don't want it in your answer. Why?
Because it won't be able to come in top 2."
    },
    {
      "start": 1312.1,
      "end": 1338.02,
      "text": "What is the meaning of top 2? First and second. Who is behind first? No one,
behind 0. Who is behind second? Behind 1. But the third person, third is behind 2, so he will
not come in top 2. That is why we need these two. Now see, see a very important thing,
understand a very important thing. We make a min heap. We make a min heap. What do we do? We
make a min heap. Just make it. Make it and see."
    },
    {
      "start": 1338.02,
      "end": 1366.38,
      "text": "Means what will happen more or less? There will be no question and answer, then
it will not happen. No problem, this also did not happen. Okay. Let's make a min-heap. What is
this? Min-heap. What type of heap is it? It is of the same type. Frequency and element. For
simplicity, min will work in both. Means according to first also it is min-heap and according
to second also it is min-heap. You are reminding me of yesterday's lecture. Min-heap is made.
What you do is, this"
    },
    {
      "start": 1366.38,
      "end": 1396.3,
      "text": "Two elements of the beginning. Because how much do we need in the beginning? We
need top k elements. We need top k elements. Pick up two elements of the beginning and put
them in the heap. Pick up two elements of the beginning from the hash map and put them in the
heap. Meaning, pick up this man and pick up this man. No matter if there is an answer or not.
One or two answers are not there. No problem. Put them in. Put them in. You have to see every
element, right? Put them in. So, put them in. What happened to this? Three and one happened.
And this? One and two happened. Frequency and element got put in. Okay. How much is the size
of heap now? How much is the size of heap? Two."
    },
    {
      "start": 1396.4,
      "end": 1425.22,
      "text": "Is it correct? Okay. Now let's look at the next element. Let's look at this. 3
and 4 means that the frequency of 3 is 4. It means that what you want to put? You want to put
4 and 3 in the heap. Okay. Now tell me one thing. You want to put 4 and 3 in the heap. Where
```

```
do you want to put them? You want to put them in the heap. Okay. What will happen at the top of your heap now? What will happen at the top of your heap now? This will happen at the top of your heap now."
    },
    {
      "start": 1425.22,
      "end": 1448.51,
      "text": "Why? Because it is a min-heap. Which is the minimum frequency? 1 and 2. Is 4 and 3 greater than 1 and 2? Yes, it is greater than 4 and 3. The ones you want to put, 4 and 3, and the ones at the top of your heap, 1 and 2, you can see that it is greater."
    },
    {
      "start": 1448.51,
      "end": 1478.11,
      "text": "Now notice an important thing about 1 and 2. 1 and 2 are sitting at the top of the current heap. The size of the heap is 2. 1 and 2 sitting at the top means that 1 and 2 will be the minimum in this heap. 1 and 2 is sitting at the top of this heap. So 1 and 2 will be guaranteed to be smaller than how many people? 1 and 2 will be smaller than this man."
    },
    {
      "start": 1478.11,
      "end": 1505.89,
      "text": "1,2 is the minimum in the size of 2 heaps. In the group of 2 people, it is the poorest. So it will definitely be poorer than how many people? It will definitely be poorer than 1 person. And we saw that 1 and 2 is also smaller than 4 and 3. Because we just saw that we wanted to include 4 and 3. We saw that 1 and 2 is smaller than 4 and 3. So 1 and 3 is guaranteed to be smaller than how many people? It is smaller than 2 people."
    },
    {
      "start": 1505.89,
      "end": 1523.38,
      "text": "One from this person and one from the friend of his hip. Because the size of the hip is 2. 1 and 2 was the minimum there. It means 1 and 2 is smaller than 3 and 1 is smaller than 4 and 3. 1 and 2 is guaranteed to be smaller than 2 people."
    },
    {
      "start": 1523.86,
      "end": 1553.46,
      "text": "Remember the above. Did you need an element in your answer which is smaller than 2 people? No. Because who will be smaller than 2 people? He will be the third person. Max to max. Max to max will be the third. Which element did you need? Which is smaller than 0 people or maximum smaller than 1 person. But here we can see that the top of the heap, 1 and 2, is guaranteed to be smaller than 2 people. 1 from this and 1 from this. So can 1 and 2 ever be your answer? No."
    },
    {
      "start": 1553.46,
      "end": 1583.76,
      "text": "Because 1 and 2 will never be able to come in the top 2 because it is smaller than 2 people. The person who comes in the top 2 cannot be smaller than 2 people. Either he will be smaller than 1 in the 2nd position or he will be smaller than 0 in the 1st position. So what to do? 1 and 2 cannot be your answer at all. What to do with 1 and 2? Pop. Oh sorry. What to do with 1 and 2? Pop. Remove it. It is a useless element. Absolutely useless. And who to enter? Who to enter? 4 and 3 to enter. 4 and 3 to enter. 4 and 3 to enter."
    },
    {
      "start": 1583.76,
      "end": 1611.65,
      "text": "Okay? This man's leaf is clean. This man is gone. Okay. Come to the next item. 4 and 2. Means, frequency 4 and element 2. Okay, brother? Frequency 4 and element... What is frequency 4? Sorry. Frequency... Yes. Frequency 2 and element 4. Frequency 2 and element 4. Yes. Okay? Repeat the same exercise. Which one will be sitting at the top of your heap? 3 and 1. 3 and 1."
    },
    {
      "start": 1611.65,
      "end": 1640.4,
      "text": "How is 3 and 1 still the same? Did we pop someone else? Oh man! This is... Yes! We had to pop 1 and 2, right? 1 and... Yes, okay, okay, okay. We popped it correctly. We popped it correctly, okay. We popped this and put 4 and 3 in it. We put 4 and 3 in it. Okay. Now come to 4 and 2. What does 4 and 2 mean? How much is its frequency? 2. And how much is its element? 4. Okay?"
    },
    {
      "start": 1640.59,
      "end": 1659.23,
      "text": "How much is the element? 4. Look at the top of your heap. Always look at the
```

```
top of your heap. Who will be at the top of your heap? This element. Who will be at the top of your heap? What does this element say? Frequency 3, element 4. So, this is the element of your top. This is the element of your current. Look, is its frequency smaller than this?"
    },
    {
      "start": 1659.23,
      "end": 1684.93,
      "text": "Yes, yes, yes. Your current element is smaller than your top. Now, now, you have come to know about your current element. See, your current element is smaller than the top of the heap. It means that the element on the top of the heap is guaranteed to be smaller than one person. We have known this because the size of the heap is 2. That 3 and 1,"
    },
    {
      "start": 1684.93,
      "end": 1710.05,
      "text": "this current is guaranteed to be smaller than one person so this current is guaranteed to be in second position at max and this current is also smaller than your current means this current is guaranteed to be smaller than two people one from the top of your heap and the other from the top of your heap means this is the relation the top of your heap"
    },
    {
      "start": 1710.05,
      "end": 1739.49,
      "text": "is smaller than any x because it is at the top of the heap. Who goes to the top of the heap? Minimum element goes, so it will be small. And your current is also smaller than your top. Your current is also smaller than your top. Meaning, can your current ever be the answer? No, your current is definitely smaller than 2 logos. The one smaller than 2 logos cannot be the answer to your top 2. Absolutely not. Ignore the current. Ignore the current at all. Your current can never be the answer. What will we do to the current?"
    },
    {
      "start": 1739.49,
      "end": 1761.41,
      "text": "Ignore it. Absolutely ignore it. Move on. What do you get? The frequency of 5 is 1. What do you want to insert? 1 frequency and element 5. Repeat the same thing. This is your current. Who is at the top of the heap? 3 and 1. 3 and 1. Is your current frequency smaller than the top of the heap? Yes, it is. Meaning, your current..."
    },
    {
      "start": 1761.41,
      "end": 1791.04,
      "text": "It is smaller than your top. Your top is already smaller than x. So what will happen? How many people is your current smaller than? Guaranteed. It can be more than 2 people.  It is definitely smaller than 2 people. You don't need this current in your answer. This answer can never be made. Remove it completely. Ignore it completely. Let's move on. Let's move on. Come on. Let's move on. Let's move on. 6 and 4 means frequency is 4 and element is 6. Okay? What is this?"
    },
    {
      "start": 1791.04,
      "end": 1803.65,
      "text": "This is your current. Who is sitting at the top of your heap? 3 and 1. Is your current bigger than your top? Yes, yes, yes, it is bigger. Now, what did you find out about your top?"
    },
    {
      "start": 1803.65,
      "end": 1829.68,
      "text": "The top element was already smaller than 1x because the size of the heap was 2. This minimum element was already smaller than x in the size of 2. And this is also smaller than this. This is also smaller than your curve. It means that the top element of your heap is guaranteed to be smaller than 2 logos. It can't be in the answer because it won't be able to come in the top 2. What will we do? This can't be the answer. Remove it. And put 4 and 6."
    },
    {
      "start": 1830.45,
      "end": 1857.15,
      "text": "Now all your elements are finished. In the last, which elements are left in your heap? See, only 2 elements are left in your heap. You started with 2 and finished with 2. 4, 6 means? Frequency 4. Whose frequency is 4? 6's frequency is 4. What will go in your answer? 6. And 4, 3. Frequency 4. But which element? 3. Wasn't your answer 6 and 3 according to this? Yes, your answer was 6 and 3."
    },
    {
      "start": 1857.15,
      "end": 1876.82,
      "text": "So did you get the answer by thinking of a small trick? Which element is
```

```
required which can be smaller than how many people? If we are talking about k is equal to 2, then if an element is smaller than 2 people, then it can never be your answer. Who can be your answer? Which is either smaller than 1 or smaller than someone. You will be in the top 2. If k is equal to 3,"
    },
    {
      "start": 1877.01,
      "end": 1899.12,
      "text": "Same logic, if an element is smaller than 3 people, it cannot be your answer, then it will not be able to come in the top 3. Who can come in the top 3? Either third person who is smaller than 2 people, second person who is smaller than 1, first person who is not smaller than anyone. So you got the answer by doing this. Let's analyze the time complexity of this with me. First of all, you will need order of n to make a hash map, there is no room for it."
    },
    {
      "start": 1899.38,
      "end": 1911.47,
      "text": "The size of the hashmap, you are doing something for every element, you are doing a loop on the hashmap, you are doing a loop on the hashmap, okay, if you are doing a loop, then it will take n. Every time, you are looking at the top, okay."
    },
    {
      "start": 1911.47,
      "end": 1935.97,
      "text": "either you are ignoring it or you are pushing it with a pop or you are pushing it with a pop there will be some log in the pop push but what will that log be? will that log be n? no! the size of the heap, remember the size of the heap did it ever go above 2? no! we never allowed the size of the heap to go above k so what will happen? n log k, not n log n"
    },
    {
      "start": 1935.97,
      "end": 1964.72,
      "text": "In the previous example, in maxheap, the heap size increased. The heap size became the array size. But we never allowed the heap size to exceed k. If an element was inserted, it was inserted after popping. At any point of time, the heap size was not allowed to exceed 2. What is the time complexity of this? n log k, which is better than n log n. So this is a solution that will be accepted. Whose solution is this? It is the solution of minheap."
    },
    {
      "start": 1964.72,
      "end": 1993.66,
      "text": "Look at the dry run of the smallest example. Look at the smallest example, it will be completely clear. So, what is the smallest example? Your array is like this, 1, 2, 1, 2, 1, 2, 3, 1, 3, and 2. Let's make a hash map quickly. In the hash map, how many times is 1? 4 times. How many times is 2? 1, 2, 3, 4 times. How many times is 3? 2 times. Make a min heap."
    },
    {
      "start": 1993.66,
      "end": 2023.52,
      "text": "Make a min heap. What have you done in the min heap? How many elements do you need? k is equal to 2. Top k elements. Insert the two items of the beginning in the min heap. Don't think what is there in the beginning. Just for your understanding, we insert this. First, we insert this. 1 and 4 means we insert 4 and 1. 4 and 1. And we insert this. Because hash map is not sorted. It can come in any order. Just for understanding, we insert this. So, we insert 2 and 3 too. 2 and 3. Is it correct?"
    },
    {
      "start": 2023.52,
      "end": 2037.78,
      "text": "Let's move ahead. How many elements are left in the hash map? One element. Let's process this. 2 and 4 means frequency 4 and element 2. You want to see this. Which element would be sitting on your top right now? Min one. 2 and 3. 2 and 3."
    },
    {
      "start": 2038.1,
      "end": 2059.47,
      "text": "Is this 4 bigger than your 2? Is this 4 bigger than your 2? Yes, it is bigger than your 2. Yes, it is bigger than your 2. What does this mean? This top element will already be smaller than any x. What is x? One element. Because heap's size is 2. So this k-1 element is definitely smaller because it is on the min. And this is also smaller than this."
    },
    {
      "start": 2059.47,
      "end": 2088.51,
      "text": "So how many people will be smaller than this 2,3 item? How many people will be
```

```
guaranteed to be smaller than this 2,3 item? One will be smaller than this and the other will be smaller than 2 people. I cannot be smaller than 2 people in the top 2. Do you want this 2, 3? No. Pop 2,3 and enter 4,2. 4,2 can be the answer. All the items are finished. Which 2 items are left in your heap? One is 4,1 which means frequency 4 element 1."
    },
    {
      "start": 2088.51,
      "end": 2120.0,
      "text": "And 4, 2. Frequency 4, element 2. 1 and 2. This was not your answer. 1's frequency is 4, 2's frequency is 4. So, in top 2, it is 1 and 2. We will just return it. Very good. So, let's hope that you understood. Come, let's write the code. So, first of all, what will be given to you? One vector will be given. First of all, you have to return an integer and vector. Means, you have to return an array. And you will also be given a vector. Vector."
    },
    {
      "start": 2120.0,
      "end": 2144.56,
      "text": "It will be given. Int k will also be given. Okay. Come. Int k is equal to, sorry. Let's take int n. n is equal to a dot size. Okay. Now you have to make a heap. You have to make a heap of pairs. You have to make a heap of pairs. How? Both should be max. Both should be max. So go and see tomorrow's code. We don't remember that code. We will also see that."
    },
    {
      "start": 2144.56,
      "end": 2170.46,
      "text": "If you are in C++, then write your own comparator function. In that, everything is told line by line that which pair do you want, which pair do you want, max, max, min, max, all kinds of combinations are told. So what do you want? You want the heap of min and min. In both, you want the heap of min and min. Okay? Okay. So make a heap. If you are in Python, class pair it in the LT function. If you are in Java, class pair it in the"
    },
    {
      "start": 2170.46,
      "end": 2197.6,
      "text": "integer, a-b or b-a whatever you want to do so this is easy, you only need int and int we have told you about int and string in this it is easy, you only need int and int so here we are writing make heap heap of pairs what kind of heap? which is min and min means according to first also min and according to second also min now what we have to do? let's make hash map let's make hash map we will give you its code if you want"
    },
    {
      "start": 2197.6,
      "end": 2224.91,
      "text": "What will be the hash map? It will store the frequency of every element. So, let's store the frequency of every element. I is equal to 0. I less than N. I plus plus. And F. A I plus plus. Is the frequency work done? The frequency work is done. Okay. Now, what to do? We have to loop on the hash map. And we have to run it. So, let's loop. For. Auto. I. F."
    },
    {
      "start": 2225.23,
      "end": 2261.22,
      "text": "After this, what is stored in the hash map? Element and its frequency. So, an int element is equal to i.first. As you remove java and python, int frequency is equal to i. second. Okay? To enter in heap, we make a pair of this. To enter in heap, we make a pair of this. Okay? To enter in heap, Sorry, I got a call. So, what we used to do is, remember what we used to try to do."
    },
    {
      "start": 2261.22,
      "end": 2291.78,
      "text": "sorry, we used to insert the k elements of the hash map without any resistance we used to insert it without any resistance we didn't check anything in it whether it is current or not so first of all we will do one thing, we will make a pair of what you have to insert so look at the pair, this is how it is made so what we will do is if you are in java, do new pair and if you are in python, do new pair and if you are in c++, do this and make a pair pair int ger is equal to"
    },
    {
      "start": 2291.78,
      "end": 2322.93,
      "text": "what you have to do is, remember, frequency element, frequency and element, means, frequency and element, you have to do that, first of all, we will do one thing, if pq. size is less than k, means, you have not processed k till now, when you will process 2, then hip size will be 2, but if you have not processed, means, in first or second, don't do anything, directly push it, directly push it, which, current, and continue it,"
    },
```

```
    {
      "start": 2322.93,
      "end": 2351.76,
      "text": "Let's move forward. But if this is not the case, but if this is not the case,
but if this is not the case, it means that the PQ size is done. Now what would we do? If the
frequency of the current, if the frequency of the current is less than the frequency of your
top, then we would have ignored the current because the current fit is smaller than two people.
 If this is not the case, we would have popped the top and pushed the current. So we will
check it like this. What is the current? What is the current? Frequency and element. So it
means if,"
    },
    {
      "start": 2352.08,
      "end": 2381.84,
      "text": "current.first, which will be its frequency, is smaller than pq.top.first. Means,
 from the element at the top of pq, if it is small, what do we have to do? Nothing. Ignore.
Continue. Continue. Nothing. Ignore. If not, what do we have to do? Remove that top one. Pop
it. Pop it because the top one is wrong. The top one will be smaller than 2 logos. So, that
can't be the answer. And, pq."
    },
    {
      "start": 2381.97,
      "end": 2413.6,
      "text": "Push. What push? Current. Close this hash map loop. Now in your heap, what did
we see in the last? That only two elements will be left. If it is K2, then K elements will be
left. Which will be of highest frequency. For a result, take this. Vector. Int. Res. And empty
this heap. Just like we empty the stack, the heap is also emptied. PQ.Empty. What do you need
in res? Frequency or element? Frequency or element? Element."
    },
    {
      "start": 2413.6,
      "end": 2443.49,
      "text": "So, res.pushback, whatever is in the top of pq, because you can only find the
top, you can't find the element, you can't find it by looking at the details. The second of
the top. Why second? We need the element. What did we put in the heap? Frequency and element.
Second one. And pq.pop. This is over. And in the last, return res. That's it. Time complexity.
We have seen n log."
    },
    {
      "start": 2443.49,
      "end": 2464.88,
      "text": "k, k, not n, space complexity order of n, because anyways we will make a hash
map, the size of your array can be more or less than the hash map, so this is the discussion,
so do make this question, and the next question is in the sheet, which is the frequent words
of the top, do try that, otherwise we will tell you tomorrow, okay, let's go."
    }
  ]
}
```


4. transactions/t_13.json


```
{
  "video_id": "5PrzZfhk1OQ",
  "title": "The BST Validation Problem Everyone Gets Wrong",
  "language": "en",
  "model": "large-v3",
  "duration": 2531,
  "segments": [
    {
      "start": 0.0,
      "end": 29.38,
      "text": "Today I have very less time, so today we will do only two questions on BST.
Both are very good questions, very medium to hard level questions. So the first question will
be a little weird, the second question will be related. Let's start quickly. The first
question is, you have to validate. Who do you have to validate? You have to validate BST. What
is BST? Binary Search Tree. What does validate mean? That you will be given a route, any BST
can be given to you. By doing this, any BST will be given."
    },
    {
      "start": 29.38,
      "end": 59.68,
      "text": "You just have to tell true or false. True or false. This is the meaning of
validate. You have to tell true or false. If, if, if, this BST is correct, what does correct
mean? The rule of BST is that its root is left small, right big. Means if this whole rule is
followed in the whole tree, then you have to return true. If this rule breaks anywhere, then
you have to tell false. As an example, if you are given a BST like this, here is 10, here is 5,
```

```
 here is 15, here is 7, here is 2."
    },
    {
      "start": 59.68,
      "end": 85.28,
      "text": "and here is 12 and here is 18. So, look carefully whether it is BST or not.
There are many ways to see BST. One way is that, see here is 10. The left of 10 is 5 and this
is 15. So, it is working perfectly. Its left, root and right should work in this order. Now, 5
is there. 5's left is 2. So, 2 is smaller than 5 and smaller than 10. Very good. 5's right
should be bigger than that. But, it should not be so big in the right that it should be bigger
than 10. It should be smaller than 10."
    },
    {
      "start": 85.28,
      "end": 114.22,
      "text": "So, 7 is bigger than 5 and smaller than 10. Very good. So, whatever is on the
left of 10 is smaller than 10. It was till now. In the same way, 15 is written on the right of
10. So, it's a good thing. It should be bigger on the right. On the left of 15, it should be
smaller than 15. But, it should be bigger than 10. Whatever is on the left of 15, now it is 12.
 So, it should be bigger than 10 on the right of 10. But, it should be smaller than 15 on the
left of 15. So, it's fine. This is also fine. 12 is in the middle. 18 is on the right of 15.
So, 18 should obviously be the biggest. So, you must be thinking that"
    },
    {
      "start": 114.22,
      "end": 142.88,
      "text": "Very good BST, so you will return true in this. Now, there are many ways to do
this. I mean, there are many ways. But the way that we like the most and the best way, we will
tell you. And we will tell you the other one. We will not code the other one. If you want to
code it, then you can code it yourself and see. But the way that we like the most is that we
have been telling you for many days that the inorder of BST, what? The inorder of BST, it
changes it to a sorted array."
    },
    {
      "start": 142.88,
      "end": 167.47,
      "text": "It turns it into a sorted array. If this BST is bad from anywhere, means if
there is a bad BST or a wrong BST from anywhere, then you will not get this sorted array. How
do we write in order? Let's write it. What is in order? Left, then root, then right. This is
how in order is written, right? Very good. So how is in order written? Let's write it like
this. First of all, we will come to 10. So 10 will not give a chance to our left. Then we will
come to 5."
    },
    {
      "start": 167.47,
      "end": 194.06,
      "text": "5 will first give a chance to its left, then it will come to 2. If there is no
one on the left of 2, then 2 will be printed by itself. Okay? Now, if we go back to 5, then 5
has given a chance to its left, so it will be printed by itself. After that, 5 will give a
chance to its right, 7. Now, if there is no left of 7, then 7 will be printed by itself. If
there is no right of 7, then nothing will happen. Back to 5, 5's work is done. Back to 10, now
10 has given a chance to its left, so 10 will be printed by itself. After that, 10 will give a
chance to its right."
    },
    {
      "start": 194.06,
      "end": 217.94,
      "text": "15. Now, 15 will not be printed by itself. First, we have to give the left
child a chance. So, first it will come on 12. There is no left or right for 12. So, 12 will be
printed by itself. After 12, it will come on 15. Now, 15 will be printed. After 15 is printed
by itself, it will give the right child a chance. So, it will become 18. So, look carefully.
Is this a sorted array? 2, 5, 7, 10, 12, 15, 18. Yes, this is a sorted array. If this BST was
bad from anywhere, from anywhere, let's assume that, let's do one thing, this 15, we,"
    },
    {
      "start": 217.94,
      "end": 244.51,
      "text": "What do we make? We make this 15 as 9. We make this 15 as 9. Now see, when we
print in order in this order, where there is 15, there will be 9. So where there is 15, write
9 there. Now see, is this a sorted array? No, it is not a sorted array. Why not? Because BST
was bad. If BST was bad, then obviously this sorted array will not be made. It will never be
made. So the easiest way is to take one vector. Take one vector int. Okay? Run in order."
    },
    {
      "start": 244.51,
      "end": 272.78,
      "text": "Run the inorder program on this root. What do you have to do? Nothing. First of
```

all, what does inorder say? Give me a chance on the left. Okay. After that, we will push this. res.pushback. Or whatever your array list is. To whom? To the root data. This is what inorder is. Inorder is that first call the same function on the left. And call the same function on the right. Below. In the middle. In the middle. This part of the middle is the root part. So you can do whatever work you want to do here."
    },
    {
      "start": 272.78,
      "end": 302.1,
      "text": "When we do this, then what will happen? It will convert the whole BST into this vector. And what will this vector hold? In order. Because we are doing it in this way. We are doing it in this way. After that, you can check on this vector. After that, you can check on this vector. How can you check? Now, if any array is sorted or not, how do you know? You will get any vector. Suppose you get such a vector. 2, 5. And here 4. And then here 10 and 12. So, you can see that this is not a sorted array. This is not a sorted array. So, how will we check it? It is very simple. 4."
    },
    {
      "start": 302.1,
      "end": 328.54,
      "text": "I is equal to 0. I less than res dot size minus 1. I plus plus. Why minus 1? Let's tell you. So how will we check this? Look. What is the identification of sorted array? That every element is smaller than the previous one. Every element is smaller than the previous one. Only smaller than the previous one. So we will do if res I. If any element is bigger than the previous element."
    },
    {
      "start": 328.54,
      "end": 353.92,
      "text": "If it is bigger than 5-4, then it is not a sorted array. If it is bigger, and it is said strictly, then even if it is equal, then it is not a sorted array. Here it is strict, it means that the values do not repeat. It should be bigger. So if it is bigger than your previous one, then you can immediately do return false here. You can do return false, let it run. And if nothing like this happens, then it means that the entire array is sorted."
    },
    {
      "start": 353.92,
      "end": 381.33,
      "text": "Is this array sorted or not? We think everyone knows this. We might have studied this in recursion and you should know this by now. This is a very basic thing. You can do one thing that you put the whole tree in the result first. After that, you check in the result whether it is sorted or not. But this approach will not work in the interview. This approach may work in the interview. If we are the interviewer, then it won't work. Why won't it work? Because you are using this extra thing. The vector int res, you are using this extra."
    },
    {
      "start": 381.33,
      "end": 410.03,
      "text": "You have already given a tree, so why are you using it extra? What will we do to do this? Let's think. Now you don't have to write the whole tree in a vector. Doing it in order is right, but don't write the whole tree. Look, what were we doing earlier? Earlier we did a lot of work. Earlier we made the whole tree in a vector or an array. For this, we ran in order. After that, we checked in this array that if any person is bigger than their predecessor,"
    },
    {
      "start": 410.03,
      "end": 440.14,
      "text": "So, so, so, what did we say? False. In total, we have made the tree a vector or an array. We checked in the array that if there is something wrong, if it is not looking sorted, then we returned false. You can do the same thing without making a vector. You can do the same thing without making a vector. How can we do it? Let's see. Let's say, let's say, what are we saying? Whenever you don't understand anything in the tree, the smallest tree is made of 3 nodes. Let's see, we make a tree. We write here 2, 1, and"
    },
    {
      "start": 440.14,
      "end": 467.46,
      "text": "What should I write? 2. I can't understand this. 3 and 2. Okay? Is this a tree? Is this a BST? No, this is not a BST. Why not? Look. Root is 3. Root can be anything. 3 is smaller than 3 on the left of 3. Very good. But 3 is not bigger than 3 on the right of 3. So this is not a BST. If you make an array of this, what will it be? 1, 3, 2. Is this a sorted array? This is not a sorted array. So this is not a tree. So the same thing you are doing,"
    },
    {
      "start": 467.46,
      "end": 497.68,

```
      "text": "what you were doing here, you can also do it during the inorder. How? Let me tell you. What is happening here? In a way, what is happening here is that we are checking two elements every time. We are checking i with i plus 1. Instead of doing this, we could do one more thing in this array. We could do one more thing in this array. We could do one more thing in this array. We could do one more thing in this array. We could do one more thing in this array. We could do one more thing in this array. We could do one more thing in this array."
    },
    {
      "start": 497.68,
      "end": 521.7,
      "text": "If the i of the rest is smaller than i-1, then we can make the return false. What does this mean? See, it comes to 2. It won't come to 2, since i starts from 1, so it comes to 5. Is 5 smaller than the previous one? No. So, it's very good. It comes to 4. Is 4 smaller than the previous 5? Yes, it is. This shouldn't happen in the sorted array. In the sorted array,"
    },
    {
      "start": 521.7,
      "end": 551.04,
      "text": "Every element should be bigger than the first one. If there is a sorted array in which 4 is coming and before 4, 6 is written, then this is not a sorted array. So you can do this trick. Exactly, exactly the same thing you can do in order too. Without making an array. How? At any time, at any time in order, you know the root. You know the data of the root. You know the root in order. Okay. What you have to check is that if, if this,"
    },
    {
      "start": 551.04,
      "end": 574.61,
      "text": "Chota hai. Ye chota hai. Kisse? Apne previous wale se. Apne previous wale se. Apne previous wale node se. Previous ka kya matlab hai? Ki in order me. In order me. In order me. In order me. Is root ke. Pahle kaunsa root visit hua tha? Agar ye usse chota ya equal to b hai. Equal to b hai. To aap directly ek answer me false kar sakte hai. Phir sir batara hai."
    },
    {
      "start": 574.61,
      "end": 603.38,
      "text": "In order, if you come to a certain route, before that circle, there must have been a circle visited, right? You store that in the array. What do you store in the array? You store route by route in the array. So, suppose you are in a certain circle now, suppose you are on 2. Before this, which circle must have come in order on which we must have worked? If the data of the route becomes smaller than that, smaller than the previous one, then it will be false. So, we will do the same trick. We will use the same trick. How? See."
    },
    {
      "start": 603.47,
      "end": 630.06,
      "text": "In the beginning, the inorder will start from the root. So what will be the previous in the beginning? Nothing will happen in the previous. Previous means that in the inorder, when you are doing the inorder, then what was the node before this root? So when the inorder hasn't started yet, then what will be the previous? It will be null. So see what we will do. First of all, where will the inorder call? From the root. Obviously, it starts from the root. See what we will do. See. And see how the prev changes and how we get to know. So first of all,"
    },
    {
      "start": 630.06,
      "end": 657.04,
      "text": "Root is equal to 3 will be called. Inorder is called. So what does inorder do? First of all, it gives a chance to its left child. Who will it give a chance to? Root is equal to 1. After that, it will process itself. What will it process? We will see later. And after that, it will also give a chance to its right child. Is it a good thing? Okay. Let's do this. Now root is equal to 1 will come. Root is equal to 1 will come. Fun. Root is equal to 1 will come. Is it null? No, it is not null."
    },
    {
      "start": 657.04,
      "end": 686.35,
      "text": "So, first of all, who will get a chance? 1. Now, you have come to 1. Who? Your left child. So, if you give a chance to your left child, then this will become root is equal to null. Because its left child is not there. Now, nothing happens in null. There is only return in null. So, okay. Something happened. Now, it will do its own processing. Now, let's see what processing it will do. Now, now. Look. If here you get to know that previous is null. If you get to know that previous is null, then what does it mean? That this circle, this circle of 1, is the first node. This is the first node in order."
    },
    {
      "start": 686.35,
```

```
      "end": 712.77,
      "text": "The first node is in order, which means this person, i is equal to 0, i is
equal to 0. Before this, there is no one, then only the previous can be null. The previous
cannot be null like this. So the first thing is that if the previous is null, then this is the
first node. Now what we have thought is that we will compare the root with our previous and
find out. Now the first node, first node does not have any previous. So here you will not do
anything, here you will not do anything. But, but, we can do one thing, if previous."
    },
    {
      "start": 712.77,
      "end": 742.32,
      "text": "Equal equal to null. Means this is the first node. What will you do in this?
Now, while going, make the previous root. Make the previous root. What will happen? We will
tell you. What will happen? Watch. So when you do this, previous root. What is the root now? 1,
 1, 1. What is previous? Previous is a global variable. We will write it above. So what
happened to previous from null? Previous became 1. Previous became 1. Okay? Previous is null,
so you cannot do any comparison. Okay. After that, now this is done. Process is done. Now,"
    },
    {
      "start": 742.32,
      "end": 770.13,
      "text": "It will give a chance to its right child. So, this is also null. It has 2, 1.
And there is no right child. So, it will become null. So, null will not work. This whole
function is over. Now, where are you? Now, you have come here. Now, you have come here. Now,
look carefully. This root is equal to 3. You went from here to 1. 1 is over. Now, what is
previous? Previous holds exactly the same value as it has been processed before. Before this,
it was 1. Before this, it was 1. Now, previous holds it. Okay?"
    },
    {
      "start": 770.13,
      "end": 790.62,
      "text": "Now, when you check here that if previous is null, previous is not null because
3 is not the first node, 3 is the second node. Before this, a circle process has been done.
Now you can check, what can you check? If, if the data of root, if now see 3 is coming after
previous, then first previous node would have come, after that root would have come."
    },
    {
      "start": 790.62,
      "end": 820.37,
      "text": "So what does sorted data mean? If the root is bigger than this, then it is fine.
 But if the root is smaller than previous or equal to it, then it is not BST. If it is smaller
or equal to Prev's data, then what will we do? Answer is equal to false. And after that,
whatever work we did here on previous is equal to root, we have to do this work in this too.
Whether previous is null or not, previous is equal to root, we have to do it."
    },
    {
      "start": 820.37,
      "end": 848.93,
      "text": "Previous is equal to root. So, we have to do it. So, after doing this, see. Now,
 what is previous? 1. Here, what was it? Now, the code is not complete. You must be thinking,
what did I write? I am just explaining to you. In Hindi, I am explaining to you what I want to
say. Now, what was previous? 1. Now, you were in 3. So, is 3 smaller than 1? No, 3 is not
smaller than 1. So, till now, nothing. And till now, you can see that it is fine. This part
was fine, so it is still fine. After that, what happened? Previous is equal to root. So, what
became after changing previous? 3."
    },
    {
      "start": 848.93,
      "end": 870.9,
      "text": "What is the previous variable? 3. Because 1's work is done. Now, what will be
the call? fun root 2. Now see what will happen. When you come here, root is equal to 2. So 2.
2 will give a chance to your left child, but if left child is not there, then it will be null.
Now, what will it check? If my root's data, which is 2,"
    },
    {
      "start": 870.9,
      "end": 889.92,
      "text": "It is smaller than or equal to previous data. And now what does previous hold?
The circle which was executed before this. What was executed before this? Root is equal to 3.
So what does previous hold? 3. So is 2 smaller than 3? Yes, 2 is smaller than 3. There is a
mistake here. And if there is a mistake here,"
    },
    {
      "start": 889.92,
      "end": 918.18,
      "text": "So, the answer is equal to false. Here, there is a mistake because 2 is on the
right side of 3. And if it is on the right side of 3, then it should have been bigger than 3.
```

2 is smaller than the previous one. So, the game is over here. The answer is equal to false. If you do the same thing by making an array, then how would its array be made? Its array would be made like this. First, 1 would come. Then, 3 would come. Then, 2 would come. You do the same thing in the array. 0, 1, 2. You start from here. You see, is it bigger than the last one? Yes, it is bigger than the last one."
    },
    {
      "start": 918.18,
      "end": 947.74,
      "text": "Then here, is it bigger than the last one? No, it is not bigger than the last one, but it should have been. So you say here, false. So the same thing that you do in array, we are doing here with the help of a previous. Its code is very simple. Come, let's see its code. So you will need two things. In the beginning, what will you need? One, previous is equal to null. In the beginning, when in order has not started, then previous will be null. And you will need an answer. So in the beginning, we will assume that it is valid. If there is a mistake, then we will make it false. If not, then we will not make it. So it is fine."
    },
    {
      "start": 947.74,
      "end": 976.3,
      "text": "Now, let's start our in-order. What will be passed in the in-order? Nothing, just the root. You have written the previous above, right? Sorry, I didn't want to give this. Yes. First of all, it's simple. If the root is null, then there is no need to do anything. You can't do anything. Now, the in-order is starting. In-order means, first of all, give a chance to your left child. Give a chance to the left of the root. You gave a chance. Now, let's play our game. First of all, if..."
    },
    {
      "start": 976.72,
      "end": 1006.27,
      "text": "If prev is equal to null, then what do we have to do? What do we have to do? Prev equal to null means this is the first circle. So, we don't have to do much. Just put prev in root. Prev is equal to root so that we can use it next time. Else, if this is not the case, then you have to compare. So, what will we write in compare? If the data of root is smaller than the previous one or is equal to it? Is it smaller or is it equal to it? So, what does this mean? Answer is equal to false."
    },
    {
      "start": 1006.27,
      "end": 1036.53,
      "text": "Okay? If it's big, then it's okay. If it's big, then it's okay. The answer was true. If it's big, then it's a good thing. If it's small, then the answer is equal to false. And in this case too, you should do prev is equal to root. Because next time it's less. Prev is equal to root, so you have to do it. Otherwise, you won't be able to maintain the previous. That will have to be done every time. Turn it off. I think it's done. So you had given the left a chance. Now give the right a chance. Now give the right a chance. Okay? And turn it off. Okay? Main... Oh man."
    },
    {
      "start": 1037.23,
      "end": 1065.39,
      "text": "What is happening? No problem. If you want to give a chance to the main, then give a chance to the main. How will we do it in the main? Fun, or whatever your function is, inorder function name, and return answer. That's it. Look at this small program. So, what is this? This is your processing unit. This is your processing. First of all, you give a chance to the left. Then, in the last, you give a chance to the right. And in the middle, from here to here,"
    },
    {
      "start": 1065.39,
      "end": 1094.46,
      "text": "or root processing. So this way you will know if it is valid or not. So this is its best approach. If you understand the array approach, you can do it, but this is better. And you have to understand this because it will be useful in the next question too. It will be useful in the next question too. What is the next question? Let's explain it. The next question is, recover binary search tree. The first one was validate. Now recover. So what does recover mean? That something has gone wrong. You have to correct that wrong. So see, let's move a little different from BST. Okay."
    },
    {
      "start": 1094.46,
      "end": 1119.49,
      "text": "Let's assume that you have given a binary search tree. By doing this, anything can happen. Let's make a BST of 10, 15, 5, 2, 7, and 20. This is a valid BST. You can see that everything is going well. Now what has been told to you? This was valid. In this, any two nodes,"
    },

```
    {
      "start": 1119.49,
      "end": 1147.62,
      "text": "Someone has swapped it. A thief has swapped it. A thief has swapped it. What
has a thief done? Suppose we swap them. This one and this one. So, bring 15 here and bring 5
here. Okay. Now, is this BST? No, it is not BST. How will you know? Try using Validate. You
will know in this order. Now, what is your work? And it is said that only two notes have been
swapped. It is not that many... Two notes have been swapped by mistake. So, what has happened
to 5 and 15?"
    },
    {
      "start": 1147.62,
      "end": 1173.52,
      "text": "you have to find both of them and then you have to fix the BST by swapping
their data again means if you had found both of them by chance if someone had told you then
you would have swapped their data again so what would have happened if you had swapped them
again? this would have been 5 again and this would have been 15 again now this BST would have
been fixed because you have swapped both of them"
    },
    {
      "start": 1173.52,
      "end": 1203.3,
      "text": "then it will end and BST will be correct. This is the statement of the question.
 It is a little difficult to think of it as BST. It is very difficult. It is difficult to
explain it as BST. What should we say? BST and Sorted Array are the same. We call BST's
inorder as Sorted Array. We can make Sorted Array as BST. So, instead of thinking of BST,
let's think of Sorted Array. Now, if we write its inorder, then what will come? If we write
its inorder, then it will come 2, 5, 7, 10, 15, and 20."
    },
    {
      "start": 1203.3,
      "end": 1231.87,
      "text": "This is the array, right? Let's think in this array. Suppose someone tells you
that there is no BST. Suppose there is an array. There is a simple array. Do one thing. In
this array, we have swapped two indexes here and there. So earlier it was a sorted array. If
you swap it, it becomes unsorted. Find them and correct them. Find them and correct them. So
let's think in this array. We will think in the array. The work that has to be done in the
array, we will do it in the tree. We did the same in Validate. We did the same in Validate. We
thought in the array that what will we do by making an array. When we found out in that, it
was done in the tree. We will follow the same here. Come."
    },
    {
      "start": 1231.87,
      "end": 1257.01,
      "text": "So we have to write this array down. So suppose you have an array. Instead of
doing this, we do this. Suppose there is a tree like this. 1, 2, 3, 4, 5, 6, and 7. This is a
good tree. It is a sorted tree. Very good. Now in this array, I have told you that any two
notes will be swapped. Any two notes will be swapped. So to understand this, we will"
    },
    {
      "start": 1257.01,
      "end": 1285.81,
      "text": "Let's swap 3 and 6. Let's swap 3 and 6. See what happens. Look carefully. Open
your eyes and open your mind. 1, 2, 6, 4, 5, 3 and 7. This is your sorted array which has been
swapped. We have swapped any 2. Now, in the upper array, how did you check the sorted array?
How did you check the sorted array? For i is equal to 0,"
    },
    {
      "start": 1285.81,
      "end": 1307.1,
      "text": "i less than, whatever is the size of your array, and i plus plus, and if array
of i, array of i, make it minus 1. If array of i is bigger, bigger than your array i plus 1,
then we used to say that it is not sorted. Not sorted. It means it is wrong. It means that if
a person is bigger than his previous one,"
    },
    {
      "start": 1307.1,
      "end": 1336.85,
      "text": "then the wire wouldn't be sorted, it would be wrong. So let's see, in this
array, which is not swapped, which is not swapped, it is sorted, it is perfect sorted, there
is no error in it. If someone tells you that there is a wrong point in it, then obviously the
wrong point in it is 0. There is no wrong point in it. If you pick up any element, then you
will get a smaller one than the previous one. If you pick up anyone, then you will get a
smaller one than the previous one. That is why the array is sorted, that there is no wrong
work in it. Pick up anyone, if the previous one is smaller, then it will work."
    },
    {
```

```
      "start": 1336.88,
      "end": 1366.59,
      "text": "If someone asks you in this, in which we have swapped 2, how wrong is it? Let's
see how wrong it is. Let's start from here. Is 1 smaller than its predecessor? Yes, it is
smaller than its predecessor. Is 2 smaller than its predecessor? Yes, it is smaller than its
predecessor. But, is 6 smaller than its predecessor? No, no, no. We got one wrong here. What
was wrong there? 6 should have been smaller than its predecessor, but 6 is bigger than its
predecessor. So, we can say one wrong is"
    },
    {
      "start": 1366.59,
      "end": 1393.02,
      "text": "One is wrong. One is wrong. Okay? But, but, but, let's see more. We might get
more. 4. Is 4 smaller than our previous one? Yes, it is smaller. Absolutely correct. 5. Is 5
smaller than our previous one? No, 5 is bigger than our previous one. So, actually, there is
not one wrong point. Two wrong points have been created. If you swap two, then it is not like
the array is just ruined from one place. Two different points have been ruined. After that,"
    },
    {
      "start": 1393.02,
      "end": 1418.83,
      "text": "Three. Three is smaller than its own. So here, wrong one. Where did you get one
wrong? You got one wrong here. It increased from six to four. And the second wrong. Where did
you get wrong two? You got it here. When five also increased from three. So you got two wrongs.
 Let's see how you always get two wrongs."
    },
    {
      "start": 1418.83,
      "end": 1446.21,
      "text": "Once you get to know this, then suppose you have got two wrongs. Now how are
the two wrongs? This element is bigger than the previous one. So if we write these four
circles as the first of the wrong ones. What is the first of the wrong ones? Six. What is the
second of the wrong ones? The first second of the wrong ones means where did we get it and
what was the first second in it?"
    },
    {
      "start": 1446.21,
      "end": 1476.42,
      "text": "You must have done this to find the error. Array of i should be greater than
array of i plus 1. In this, this one has become first. Meaning, on the left side, you can
write left and right, or first and second. If you want, you can write left and right, or first
and second. And this has become second. We are saying this, that the wrong one has happened.
The wrong one has happened. Look, I am sweating. It has become very hot. The first of the
wrong one is 6. What is the second of the wrong one? 4. Okay. And what is the first of the
wrong two?"
    },
    {
      "start": 1476.42,
      "end": 1505.82,
      "text": "Galat 2 ka first hai 5. Aur galat 2 ka second kya hai? Second hai 3. Ye aaya.
Acha. Ab isme kaunsa 2 node swap hua tha shuru me? Kya hua tha? 6 aur 3. 6 aur 3 matlab galat
1 first aur galat 2 second. Toh agar aap in dono ko swap kar dete toh aapko array sahi ho jata.
 Isme is example me kamse kam. Kamse kam is example me o array sahi ho jata. Ek aur example
dekh lete hain. Ek aur example dekh lete hain. Ki kya hamesha aisa hi hota hai kya?"
    },
    {
      "start": 1505.82,
      "end": 1535.87,
      "text": "Let's see another example. Let's make a big array. 1, 2, 3, 4, 5, 6, 7, 8.
Okay? This is a good array. What do we do in this? We swap 1 and 8. Who? Who? We swap 1 and 8.
What will be the array in this? First comes 8, then 2, 3, 4, 5, 6, 7, and 1. Done? Both of
these are swapped. Someone may ask you in this, how much is wrong?"
    },
    {
      "start": 1535.87,
      "end": 1564.85,
      "text": "How many wrongs will you get? You will get 1 wrong. 8 is bigger than your
previous one. It is wrong. 7 is bigger than your previous one. So here also you got 2 wrongs.
Here also you got 2 wrongs. Come here and write. Wrong 1 and wrong 1's first is 8. What is
wrong 1's second? 2. Very good. What is wrong 2's first? 7. And what is wrong 2's second?"
    },
    {
      "start": 1564.85,
      "end": 1594.32,
      "text": "1. Okay. Which 2 was swapped? 1 and 8. Meaning, wrong first and wrong 1's first
and wrong 2's second. If you swap these two, you will get the answer. Was it the same above?
Yes, it was the same above. So, now we know what to do. If you both found the wrong one, wrong
```

```
1 and wrong 2, wrong 1's first and wrong 2's second, how will you write it? You will do nothing. What will happen to wrong 1's first? As soon as you get a wrong one, you get a less wrong one. If array of i,"
    },
    {
      "start": 1594.32,
      "end": 1616.06,
      "text": "is greater than the array of i plus 1. So this is what happened. This is first, this is second. So what is the first of the wrong one? Array of i. And what is the second of the wrong one? Array of... We will discuss this later. We will do it like this. We are not writing this on our own. But we can see a pattern. Which two people will always get the answer by swapping?"
    },
    {
      "start": 1616.06,
      "end": 1645.5,
      "text": "The first of the wrong one and the second of the wrong two. Here also, the first of the wrong one and the second of the wrong two. But, there is a small twist here. Did you understand till here? It's a very good thing. It was difficult. It's a small twist. Till now, the swapping we were doing, we were doing it of the far away elements. See, far away elements. We were doing it intentionally. We were doing it of the far away elements. If we swap two adjacent elements, then see what will happen. See what will happen. Adjacent means the ones that were together. Swapping them means, now anyone can swap, right? Swap does not mean that the adjacent will not happen. It can happen."
    },
    {
      "start": 1645.5,
      "end": 1674.37,
      "text": "Come. 1, 2, 3, 4, 5, 6. What do we do in this? We swap 3 and 4. 3 and 4 can also be done. Obviously, it can be done. Come. Swap it. 1, 2, 4, 3, 5, 6. See what is wrong in this. 1. What is wrong in 1? No. 1 is smaller than its predecessor. 2. 2 is also smaller than its predecessor. But 4 is bigger than its predecessor. So 4, 3. So what did you get in this? Wrong."
    },
    {
      "start": 1674.37,
      "end": 1702.56,
      "text": "What did you get from 1? You got 4. What did you get from the wrong 1? You got 3. Very good. Now let's move ahead to find the wrong 2. 3. Have you increased from the previous one? No. 5. Have you increased from the previous one? No. There is no previous one of 6. So, see, a very important thing happened. Here, you got the wrong 2's first and the wrong 2's second. You didn't get this."
    },
    {
      "start": 1702.56,
      "end": 1730.74,
      "text": "Why didn't you get it? Because when you swap adjacent elements, you get only two wrong ones. You get the first and second of the wrong one. You get the wrong one. If you swap, something will be wrong. But here you didn't get two wrong ones. On the top, you used to get two points where the array used to break. In that, you get the first and second. So you used to get four things. You used to get two pairs. Here you are getting only one wrong point, which is of these two. You are not getting the wrong two."
    },
    {
      "start": 1730.77,
      "end": 1754.16,
      "text": "So if in any case you didn't get the wrong 2 first and wrong 2 second, then what will you do? Simple. Swap these two. 4 and 3. 4 and 3 was the swap. Swap these two. Swap these two. How will you know? You will take a counter. Wrong counter. Wrong is equal to 0. Wrong is equal to 0. As soon as you get a wrong, if the wrong is already 0,"
    },
    {
      "start": 1754.16,
      "end": 1784.13,
      "text": "So update the wrong one first and the wrong one second. And make the wrong one plus plus. If you get and there is a wrong one in it. You get one more wrong point. Means this is the second. Update these two. And make the wrong one plus plus and make it two. Now come out and check. That if you have got only one wrong. You will get one. It is not possible that you will not get wrong. So you are doing sorted here and there. You will get it. If you are getting wrong one. Who to swap? These two. Because it means that there is no other wrong. Adjacent element was swapped. If."
    },
    {
      "start": 1784.13,
      "end": 1809.84,
      "text": "If we are getting wrong is equal to 2, then what do we have to do? We have to do the first of the wrong one and the second of the wrong two. If we have written this, now we
```

hope that you have understood what to do. This whole thing happened on array. When you have array, then you do this work. Let's write the code first and see what happens in the code, what will happen in the code. So the first thing we will do is our BST."
    },
    {
      "start": 1809.84,
      "end": 1839.31,
      "text": "We will make a vector of the sorted array. How? Do you know how to make it? Yes, I have told you a lot of times. I have told you immediately. We have made its vector. Suppose you have a vector. What will you do? Let's see. You have a vector. First of all, what will you do? Int. Now, obviously, if two circles have been swapped in BST, then it is the same in this array. It is the same in the sorted array."
    },
    {
      "start": 1839.31,
      "end": 1868.85,
      "text": "Two swaps have been done. It is possible that adjacent swaps have been done in which only one is wrong. It is possible that distant swaps have been done in which only two are wrong. You will not get three wrong. Try different permutation combinations. You will not get three wrong. Okay? Wrong is equal to zero. Now, how many things do you want? You take four things. You take four things. Int. We write this as G1's first. Means, wrong one's first. In the beginning, don't give any value. And, what do we do in this? You want four things, right? So, G1's first is done."
    },
    {
      "start": 1868.85,
      "end": 1897.84,
      "text": "G1 is the second. Now, the first is the wrong 2. You can name it as you like. You can left-write it. And the second is the wrong 2. Very good. Now, let's do the loop. What do you want to check? If the array of i is bigger than your previous one, then what happens? This is wrong. In this, what happens? This is its first and this is its second. So, run the loop according to that. i is equal to 0. i less than. Let's assume that our array's name is a."
    },
    {
      "start": 1897.84,
      "end": 1926.54,
      "text": "A dot size. Where is A made of? It is made of BST. Minus 1. Don't go till the last because here you are doing I plus 1. You will crash there. Last index. So stop first. Because there is no one beyond the last index. So what will we check? I plus plus. Now, now, now see. If area of I gets bigger than the previous one. So it means it is wrong. It clearly means it is wrong. But which is wrong? Let's see it inside. If"
    },
    {
      "start": 1926.64,
      "end": 1951.94,
      "text": "If the error is equal to 0, if the error is still 0, which was 0, which means this is the first error, this is the first error, then what will we do? g1 first is equal to array of i. Okay. g1 second is equal to array of i plus 1. And, wrong plus plus. Now, it is wrong. Now, it should be wrong 1. Okay?"
    },
    {
      "start": 1951.94,
      "end": 1979.7,
      "text": "Else. Else. Else. This can only mean one thing. If the error is not 0, then the error is 1. It means you have got the second error. So, you have got the second error. So, G2 first. This means the first error has been made. Its pair has been made. Update the second error. Update the second error. And G2 second is equal to array of I plus 1. Okay. You can do the wrong plus plus. No. Yes, do it. Yes, do it. Because we will check the last one. Wrong plus plus. This is done. And."
    },
    {
      "start": 1979.7,
      "end": 2008.51,
      "text": "This is done and this for loop is closed. After this, after this, now check, what did we say? There will be at least one mistake. If, if, wrong, if, wrong, 1, sorry, wrong, wrong means how many times has it been wrong? Is equal equal to 1, means there is only one mistake. There is only one mistake, we had to swap this. Obviously, in one mistake, there will be no wrong 2, there will be only wrong 1. We had to swap these two. We had to swap these two. So what will we do in this?"
    },
    {
      "start": 2008.51,
      "end": 2037.63,
      "text": "In this, we can do something like G1 first and G1's second. Okay? Swap is done. Else, if this is not the case, there are two mistakes. In which of the two mistakes did you fill the pattern? Yes. G1's first and G2's second. So swap them. Swap them. G1's first and G2's second. Yes. G2's second. Very good. And return it."

```
    },
    {
      "start": 2037.63,
      "end": 2067.58,
      "text": "So, this is solved for array. This is a very simple code. Let's hope that there will be no doubt in this. Why did you do this? It will be done with a very simple explanation. Now, the thing is that you can do this. You can do this. You can make a tree of BST. You can do this. But, the same problem will come in this. The interviewer will say that the array that you are making. The array that you are making. Don't make it. Do it somewhere else. Means, do it in BST only. The problem that came in validate. The problem that came in validate. Don't make an array. Whatever you are doing in array."
    },
    {
      "start": 2067.58,
      "end": 2095.55,
      "text": "Do it in BST. Do it with PREV or anything else. Do it in BST. Do it during in-order. Come, let's do it during in-order. Come. Now, to do it during in-order, here we will apply the same formula. So, what will we do? Let's see what we will do. Let's write the code. We will dry run it. Similar, similar. We will do it during in-order in the area we are doing. First of all, now let's know what we want. We want previous. What do we want? We want previous. Why?"
    },
    {
      "start": 2095.55,
      "end": 2125.04,
      "text": "Because you will look at root and previous. In this, you were looking at i and i plus 1. In that, you will look at root and previous. So, you need one previous. Node, star, previous is equal to null. Now, you need four things. You had taken four things above. G1, G2, here and there. Now, here you had taken integer. Because R is integer. But binary search tree is not integer. It has big circle. It is node. So, what to do? Take those four things. Node, star, G1 first."
    },
    {
      "start": 2125.3,
      "end": 2155.41,
      "text": "is equal to null. Let it be null in the beginning. Node star, G1's second is equal to null. Let's hope that you are getting the naming convention. I like doing this. If you want, you can give something else. G2's first means it was wrong. One more null. And G2's second null. The same work that was there. Take it up. Very good. Take it. Let's start the inorder and see what we can do."
    },
    {
      "start": 2155.47,
      "end": 2186.93,
      "text": "In order, start void. Give it a name. Fun. Fun of void. And node star root. Okay. First of all, if root is null, then you don't have to do anything. Return. Okay. The first thing to do in order is to give the left one a chance. Give the left one a chance. Give. Give the left of root. Left is done. Now come. Now let's do our processing."
    },
    {
      "start": 2186.93,
      "end": 2215.65,
      "text": "First of all, the same thing that we did in the validate, if the previous is null, if the previous is null, now see, what will we check in this? What will we check in this? In the sorted order, what were we checking? In the upper array, we were checking i with i plus 1, in this we will do the opposite, in this we will do the opposite, like this, assume this is your 0, 1, if i is like this, then when will this be wrong? When in the sorted array, i should have been greater than i minus 1."
    },
    {
      "start": 2215.65,
      "end": 2243.6,
      "text": "But if an element is smaller than the last one, then it will be wrong. So what will happen? It will be wrong. That means, if the root is smaller than the previous data, then it will be counted as a mistake. So what will happen? It will be counted as a mistake. So if the previous is null, then there is no use. Then what should you do? Make previous equal to root."
    },
    {
      "start": 2243.89,
      "end": 2274.27,
      "text": "You can't do anything, right? Because there was no one behind it. It's the first node. Same. Validate. Else. Else. Else. Okay. Now, there is something or the other. Come, let's identify the mistake. If root's data, root's data is previous and root, root is before previous. So, root should have been bigger than this for sorted. But, if it is smaller than this, if it is smaller than this, very good. What do we call this? Mistake. Mistake. This mistake has happened."
```

```
    },
    {
      "start": 2274.27,
      "end": 2300.7,
      "text": "Now, do the same thing. Take the int above as a false. To make it simple. Very simple. False is equal to 0. Do the same thing. If false is equal to 0. Meaning, what should be the update? G1 first and G1 second. Meaning, your first mistake. Your first mistake. Now, look very carefully. Root is here. Meaning, root has come in order later."
    },
    {
      "start": 2300.7,
      "end": 2330.54,
      "text": "And previous is the previous node. So, previous is first. And what is root? Second. What is root? Second. The mistake is that root should have been bigger than previous. But root became smaller than previous. So, what is this mistake? What is this mistake? First and second. What is this mistake? Previous is your first. And root is your second. Same mistake. See, this is the mistake. This is the mistake. This is your root. And this is your second."
    },
    {
      "start": 2330.54,
      "end": 2358.22,
      "text": "Previous. Okay? This is your root here. And this is your previous. So the root is smaller than the previous. There is a mistake here. Who will be first? Previous. Who will be second? Root. If the wrong is equal to 0, it means your first mistake is going to happen. So G1 first is equal to prev. And G1 second is equal to root. And what will we do? Wrong plus plus. Now the mistake is done. Wrong plus plus."
    },
    {
      "start": 2358.22,
      "end": 2386.54,
      "text": "Very good. Now, below this, below this, should I write it here? Let's write it below this. Else, else, the only meaning of else is that this is your second mistake. Same thing, you have to do the same thing. But now, you have to do it for G2. G2 first is equal to prev, and G2 second is equal to root, and wrong plus plus. Now, this is your second mistake. You have stopped your second mistake, else. Now, you have to stop the mistake part."
    },
    {
      "start": 2386.7,
      "end": 2417.01,
      "text": "And close one more else. This is the else. Look at the bracket. We will make the code smaller and show you. Done. The whole processing is done. The whole processing is not done yet. The whole processing is not done yet. Before doing this work, before going to the next, you should do previous is equal to root one more time here. Now it is closed. Because why are you doing it? You did it in validate. Previous is equal to root. Do it in the last. Next time it will be useful. Now the whole processing is over. Now give your right child a chance. Now give your right child. The code is a little lengthy."
    },
    {
      "start": 2417.1,
      "end": 2443.14,
      "text": "It's not hard, right? We are doing what was above. We will make the whole code small and show you how. And we will also do dry run. And close this. Now, how will you call it from the main function? What will you call it from the main function? We will call our fun and root. Now, same thing, same thing. If, now when this will run completely, then it will be wrong update, everything will be updated. If you come to know that there is only one mistake, there is only one mistake, then whom do we swap?"
    },
    {
      "start": 2443.14,
      "end": 2469.39,
      "text": "G1 first and G2 second, because G2 will not be there, because there is only one mistake. In this, we don't have to swap this, we have to swap the data of the node. Keep in mind that both are node pointers, all four are node pointers. So let me tell you what the data of G1 first means, and the data of G1 second should be swapped. Swap, this means that the number of both the circles is written,"
    },
    {
      "start": 2469.39,
      "end": 2499.47,
      "text": "I'll erase it and write it here and there. Swap is done. Else. Else. Else. This else means that two mistakes have been made. Who will you have to swap? The data of G1 first. And the data of G2 second. G2, as we have seen above, that is what has to be done. After this, you can return. Make this code small and see. It looks like this. It's not too big. I mean, it should be this big. See, this is your code. See. This is your code. You will get its code in the comments. Both of them. Validator of both of them."
    },
```

```
    {
      "start": 2499.89,
      "end": 2526.9,
      "text": "So this is your program. Try it once. We don't think there is any need to try
it. It is similar. If you want to do it with an array, then you can do it. But don't learn it.
It won't work in an interview. You will have to do it like this in an interview. And it is not
that hard. I don't think it is. So these were two questions. This is a very good question of
BST. There are not many questions after this in BST. Tomorrow is the last lecture of TREE.
According to me, we will teach you construction in it. How to learn construction."
    },
    {
      "start": 2526.9,
      "end": 2530.7,
      "text": "And after that we can move on to graph or dp."
    }
  ]
}
```


5. transactions/t_14.json


```
{
  "video_id": "6XSrjOmTJio",
  "title": "Today You Will Know Why I Am The Best DSA Teacher | DP Series Episode 14",
  "language": "en",
  "model": "large-v3",
  "duration": 1952,
  "segments": [
    {
      "start": 0.78,
      "end": 24.06,
      "text": "Today's lecture is a very long and satisfying lecture because today we are
doing 4 questions together. In any video before today, we have not studied 4 questions
together, we have studied 3 questions together. DP on Stocks is a very famous topic and many
people have already studied it. It has 4 questions. Buy and sell starts from 1 and goes up to
4. There are 5 questions but 5 questions are probably in the lead quote premium. I don't have
it in the lead quote premium."
    },
    {
      "start": 24.06,
      "end": 48.21,
      "text": "1 is easy, 2 is medium, 3 and 4 are hard. So how do people teach this? They
teach 1 one day, 2 the next day, 3 the next day and 4 the next day. But this is not the right
way to teach. We will teach you all these 4 together. And by teaching together, it doesn't
mean that we will teach one by one. No. By the time we teach you YNCELL STOCKS 1, you must
have understood so many things."
    },
    {
      "start": 48.21,
      "end": 77.76,
      "text": "After this, after understanding 1, no matter how much time it takes, 10 minutes,
 15 minutes, 20 minutes, once you understand 1 properly, in 2, 3, 4, you just have to copy and
paste. What do you have to do? You just have to copy and paste and change the value of one
variable. Whose value do you have to change? You have to change the value of one variable and
change one or two lines. Okay? So, I mean, give your full focus to understanding this. Don't
think that this is easy. Some people will think that this question is easy. Why are you
wasting so much time in this? It could have been like this."
    },
    {
      "start": 77.76,
      "end": 102.64,
      "text": "But you don't have to understand this. Your goal is to understand this. So we
are getting ready for 4 from 1. Listen carefully to 1. You won't understand 1. You won't
understand anything. Okay? So what does the question tell you? What does buy and sell stock
mean? That you have been given an array. Okay? You have been given an array. What did you
give? Array. Okay. The numbers written in the array, what are those numbers? Those numbers are
prices. Prices. Whose prices are they?"
    },
    {
      "start": 102.64,
      "end": 132.13,
      "text": "Stocks ke prices hai. For example, agar aapko koi aisa area diya hua hai. 7, 1,
5, 3, 6 aur 4. 0, 1, 2, 3, 4, 5. Har din ye stock ke aapko prices diya hua hai. Har din prices
matlab, pehle din ye price hai, dusre din ye, tisre din ye, ye, ye, falana falana prices diya
hua stock ke. Acha. Aapka kaam kya hai? Aapka kaam kya karna hai? Aapke paas ek mauka hai.
Aapke paas kitna mauka hai? Ek mauka hai ki aap,"
    },
```

```
    {
      "start": 132.13,
      "end": 150.99,
      "text": "You can buy the stock once. Don't worry about the price, you can buy it no
matter how expensive it is. And you can sell it once. How many times? You can sell it once. So
what is your job? You will wake up on the 0th day. On what? You will wake up on the 0th day."
    },
    {
      "start": 150.99,
      "end": 171.92,
      "text": "Every day you will see the price of the stock. The power you have in the whole
day is of one time. You can buy it if you want. For example, if you buy the stock on day 0,
then you will get 7 rupees."
    },
    {
      "start": 171.92,
      "end": 198.38,
      "text": "You don't have the option to buy. You can buy only one stock. You can sell it
if you want to. That is also up to you. You can sell it if you want to. For example, one day
you went crazy and sold it for Rs.3 on day 3. How much? You sold it for Rs.3. What was your
net profit? It was minus Rs.4. You bought it for Rs.7 and sold it for Rs.3. What was your
profit? Minus Rs.4."
    },
    {
      "start": 198.38,
      "end": 223.3,
      "text": "You have to find your maximum profit possible. What do you have to find?
Maximum profit possible. If you know your maximum profit possible is minus 4 rupees, in that
case, you have to return zero. Because why will someone make a profit of minus? Make a profit
of zero, don't do anything. So this is your question. Like what should be the answer to this
question? You should buy it here. What should you buy?"
    },
    {
      "start": 223.3,
      "end": 251.41,
      "text": "You should buy on 1. So, how much did you buy? You bought a stock of 1 rupee.
And, you should sell it on this day. When it is of 6 rupee, you should sell it on this day. So,
 how much is your net profit? You can make a profit of 5 rupee. So, this is your question. Is
it right? Let's go. Now, it seems very simple to listen to this. People will think that we
will do one thing. Generally, what do we do? This is a real world problem. Generally, when you
buy, you want to"
    },
    {
      "start": 251.41,
      "end": 281.15,
      "text": "Let's buy at the lowest price. If any person does business, then he wants to
buy at the lowest price. And when he sells, then he wants to sell at the highest price. How
much? He wants to sell at the highest price. This is what business is. What is the gain in 9-
10? Selling price minus cost price. So, we have to reduce the cost price and increase the
selling price. So, someone will say, okay, let's do one thing in this. Let's find the minimum
value of the array. Okay, brother, find it. So, suppose you found out that this array's"
    },
    {
      "start": 281.15,
      "end": 300.82,
      "text": "Someone said that the minimum value is 1 rupee. So we will buy it in 1 rupee.
What else did someone say? That we will also find the maximum value of this area. And we will
know that it is 7 rupee. Now this is not possible. You will say that 7-1 profit is 6. No. Now
you are saying that we will buy on the minimum day. So you buy on day 1."
    },
    {
      "start": 300.82,
      "end": 319.7,
      "text": "You are saying that you will sell it on the maximum day. How can it be possible
to sell it for 7 rupees? It is not possible that you buy it on 22nd June and sell it on 20th
June. So you don't have to travel time. If you buy it on 22nd June, you have to sell it on the
next day. You can sell it on 22nd June. Now someone will also say that let's give him a
solution. Let's find the minimum value."
    },
    {
      "start": 319.7,
      "end": 349.79,
      "text": "They will say, okay, find it. And whatever is the maximum value in front of
that minimum, whatever is the maximum value in front of that, we will sell it. So, someone
will say, okay, sir, have you bought it in minimum 1? Whatever is the maximum value in front
of 1, which is 6, we will sell it at 6. So, I have 5 rupees. The answer is also given in this.
So, why don't we make it with this? Why don't we make it with this? Let me explain it to you.
```

```
See. This logic is sound. This logic means, it should come to mind. It should come to everyone.
 What you are saying is, let's assume that this is 1. After this, it is 2, 3, and?"
    },
    {
      "start": 349.79,
      "end": 379.55,
      "text": "Four. Okay. Zero. Let's do it like this. This is your logic, right? Zero, one,
two, three, four, five. Let's assume that this is one minimum value. What should be your
logic? Let's say we have found the minimum which is one rupee. Let's buy it in one rupee. The
maximum which is four in front of one. Let's sell it in four. Net profit is three rupees. This
is your logic. Okay. But now assume that we have written this for four days intentionally.
Before this, before one, let's assume that this is written. So,"
    },
    {
      "start": 379.55,
      "end": 407.66,
      "text": "And 1 lakh. 100 and 1 lakh is written. Now what will we do? Now should its
answer be 3? No. What should its answer be? You should buy it for 100 rupees. And you should
sell it for 1 lakh rupees. Means your answer should be 99,900 rupees. How much are you taking
out? 3 rupees. That's why it won't work. We tell all this because you shouldn't have any doubt
in your mind. That this one is not told. They won't understand. You should know why it won't
work."
    },
    {
      "start": 407.66,
      "end": 422.29,
      "text": "So you understood that this will not work. It will not work by removing minimum
and maximum. You can do whatever you want. Now see. Now we will start explaining this question
to you. What is to be done in this? What can you do in this? See. First of all, this question."
    },
    {
      "start": 422.29,
      "end": 443.66,
      "text": "This is a question of choice. We have been telling you from the beginning that
whenever you see the problem of DP, then see if it is a question of choice, is it a question
of free will? Can you think of such an algorithm that will work every time? This means that if
this minimum-maximum algorithm worked every time, then this would not have been a question of
DP, then you would have done some math type of question. But here we understand that no."
    },
    {
      "start": 443.66,
      "end": 471.49,
      "text": "Such an algorithm cannot be made in the beginning that we will buy the minimum
and sell the maximum. No, because it is dependent. You are getting to know the price of the
stock every day. So you will always have a choice. What will be the choice? How many choices
do you have in this question? There is one choice that you buy it. There is one choice that
you buy it. And there is one choice that you sell it. What is one choice? That you sell it.
You always have this choice."
    },
    {
      "start": 471.49,
      "end": 497.6,
      "text": "So what is the way to solve this? Look, buy and sell. There are two
transactions. How many transactions are there? There are two transactions. How many? There are
two transactions. This is two transactions. Okay, two transactions. Okay, tell me one thing.
This is a normal thing that if a person has not bought yet, if a person has not bought yet,
then can he sell?"
    },
    {
      "start": 497.6,
      "end": 524.62,
      "text": "If a person has not bought, then how will he sell? If you haven't bought
anything, will you sell it? You can't sell your hand. So if you haven't bought, then you can't
sell. If you have bought, then he has the option to sell. Everyone understands this. This is
not a rocket science. The option to sell opens only after buying. There is no option to sell
in the beginning. If you don't buy a car, you won't be able to sell it. That's a good thing."
    },
    {
      "start": 524.62,
      "end": 553.87,
      "text": "So in this question, what is your limitation? You have been told that you can
buy once and sell once. What can you do once? You can sell. What will we do in this? We will
make this buy-sell a transaction. What? Once buy, once sell. How much is the total
transaction? How much is the total transaction allowed? Two transactions are allowed. Okay?
One buy, one sell. Okay. In the beginning, let's assume that when your transaction"
    },
    {
```

```
      "start": 553.87,
      "end": 581.14,
      "text": "is equal to 2. When your transaction is equal to 2, it means that you have not
yet bought. You have not bought anything in the beginning. Till then, the transaction will
remain 2. You will only have the option of buying. Only the option of buying will remain open.
Yes, yes. Only the option of buying will remain open. Once you buy, the option will remain
open. It means you can do two things."
    },
    {
      "start": 581.14,
      "end": 609.07,
      "text": "Either you buy that day or you do nothing. It is not necessary to buy, but
until you have two transactions, you do not have the option to sell. If you buy, if you choose
one day, one day you wake up and you feel that today we will buy, then what will we do? You
had two transactions, one buy and one sell. When your buy transaction is done,"
    },
    {
      "start": 609.07,
      "end": 633.87,
      "text": "Now, how many transactions are left? One transaction is left. If you have one
transaction left, then you don't have the option to buy. You have already bought, which means
you cannot buy twice. Now, you have the option to sell or to do nothing. Now, you don't have
the option to buy. Let me tell you again."
    },
    {
      "start": 633.87,
      "end": 659.18,
      "text": "In the beginning, we thought that we have two transactions. By the time you see
that there are two transactions, you only have the option to buy. Buy or nothing. Once you buy,
 we will reduce the transaction from one to one because one transaction is done. When your
transaction is one, you only have the option to sell. Now you don't have the option to buy.
Either you can sell or you can do nothing."
    },
    {
      "start": 659.18,
      "end": 682.78,
      "text": "If you sell, we will reduce the transaction by one more and make it zero. Now
you have zero transaction allowed. If you have zero transaction allowed, now you have only one
thing left to do, which is nothing. Now you can't do anything. Now your profit is going to be
zero."
    },
    {
      "start": 682.78,
      "end": 704.34,
      "text": "Whatever you had to do, you did it. You did buy once, you did sell once. Now
you have nothing left to do. So does everyone understand this or not? When we were saying that
you have a choice, then what is the choice? How is the choice? We will decide on the basis of
this transaction. Two transactions means either buy or do nothing."
    },
    {
      "start": 704.34,
      "end": 733.89,
      "text": "One transaction means either sell or don't do anything. Zero transaction means
don't do anything and you are not allowed to do anything. So this can be our recursion. What
kind of recursion can it be? We will pass two things in recursion. What will we pass? First of
all, I. On which day are you? On which day are you seeing the stock? And we will pass a K
value. What does K mean? Number of transaction left. What? Number of transaction left. Yes.
Yes. Number of transaction left."
    },
    {
      "start": 733.89,
      "end": 763.17,
      "text": "According to K, we will decide what choices are available and we will write
them. Sorry, the light went off. So, according to K, we will decide what is available and what
is not available. If you try to buy someday, suppose you try to buy someday, 0, 1, 2. Suppose
this is 1, this is 2 and this is 3. Suppose when I is equal to 0 and your K is equal to 2,"
    },
    {
      "start": 763.17,
      "end": 787.23,
      "text": "You decide that you will buy today. So what will happen by buying? What will
happen by buying? What will happen by buying? On your purse, you will get minus 1. You will
have to do plus minus 1. Or in simple words, you will have to reduce minus 1 from whatever you
have. Because you are buying. Buying reduces money."
    },
    {
      "start": 787.23,
```

```
      "end": 806.26,
      "text": "Assume that when I2 is there, you say that we will sell now. Because when I2 is there and K1 is there, then you say that K1 is there, now we want to sell. When you sell, then you will do plus 3. By selling, the money increases. By buying, the money decreases. This is what happens. Assume that you have 100 rupees."
    },
    {
      "start": 806.26,
      "end": 832.75,
      "text": "You said that today we will book for 50 rupees. So your bank will do minus 50. After that you said that okay, now we will sell it for 70 rupees. So your bank will do plus 70. That's why 120 and that's why you will say that it was a profit of 20 rupees. So whenever you buy, then what will we do? Minus AI. By way, minus AI. Your money will be less. When you sell, then what will you add to it? Plus AI. That much money."
    },
    {
      "start": 832.75,
      "end": 856.59,
      "text": "will increase. This is your profit. This is your profit. Automatically, when you do this, you get a profit of Rs. 20. You don't have to do anything separately. Is it right? In the beginning, you have this. Is it right? It is Rs. 0. It is minus 50. It is plus 70. How much profit has come? Rs. 20. In the last, whatever profit will be left, whatever maximum profit will be left, we will return it. Come, let's write its recursion."
    },
    {
      "start": 856.59,
      "end": 885.73,
      "text": "So let's hope that you understand the entire Ramayan because it is going to be very important in the future. Now you must be thinking that we are doing a little overkill, that means we should not have done this much. There is also a fall loop to solve this. You can do a simple fall loop, but that is not the aim. The interviewer will not ask you this, the interviewer will ask you stock 2 or 3 or 4, okay? He will not ask you 1. That is why I am saying that you are interested in this story, okay? Let's go. INT fun, what will we pass in this? First of all, we will enter vector int."
    },
    {
      "start": 885.73,
      "end": 916.0,
      "text": "A, which is your price array. After that, you, N, how many are there? I, I means on which day are there? And, I and T, K. K, okay? For this question, what is K? 2. How many transactions do you have? 2. Because it is written that you can buy once and sell once. So, how much did you get in total? 2 transactions. Okay. First of all, if I equals to N, means you went beyond the whole day, then what is it? You can't do anything with 0, right? After that, what will you do? The stock market is over. It is closed. So, what to do? 0."
    },
    {
      "start": 916.0,
      "end": 944.69,
      "text": "Now, if K is equal to 0, what did we teach? If there is 0 transaction left, what does K tell? How much transaction is left? If there is 0 left, then how much will be the profit? Nothing, brother, nothing, nothing. Return 0. It's over. Now you have to decide whether you will sell or buy. K is not 0, it means K is something. K is either 2 or 1. Now you have to decide whether you will sell or buy. What power do you have? So if, if,"
    },
    {
      "start": 944.69,
      "end": 973.73,
      "text": "K is equal to 2. Which means this is a case of buying. So you have two choices. The first choice is to buy. What does it mean to buy? Fun, AN, I plus 1. And if you buy, your transaction will be less than 1. You are using a transaction. What will you do? K minus 1. And K minus 1."
    },
    {
      "start": 973.73,
      "end": 1003.17,
      "text": "Minus AI. What did we say? When you buy something, you do minus AI in it. So this was the first choice in which you bought. What is the second choice? You will not do anything. You will pass it. Means we will not buy. So if you will not buy, then you go straight ahead. Neither this will change, nor anything will be minus. Two choices. Out of these two, out of these two, whatever is the maximum, we will return it. This is over. Now, now, else, else, else."
    },
    {
      "start": 1003.17,
      "end": 1026.77,
      "text": "What does this mean? You are in the sell category. If K is not 0, K is not 0, K is not 2, then K will be 1. Now you have the power to sell. Same thing, first you can sell if
```

```
you want. A, N, I plus 1 and if you sell, your transaction will be less."
    },
    {
      "start": 1026.77,
      "end": 1050.13,
      "text": "Now, if you are selling, then you will do plus AI. If you are selling, then
your money will increase. It was less above. In buy, it will increase in less sell. You also
have another choice here that you do nothing. You leave it and go ahead. It is not necessary
to sell. It is not necessary at all. Leave K as it is. And return maximum of C1 and C2. Take
this."
    },
    {
      "start": 1050.13,
      "end": 1077.7,
      "text": "Else is over, this is over. Look at this code carefully, stick this code
carefully in your mind and understand. Let's come to the main function. What will we do in the
main function? Let's come to the main function. int, this is your stock function. This is your
stocks function. Vector, you are given only one thing, a. Okay. First of all, int n is equal
to a dot size. Okay. Okay."
    },
    {
      "start": 1077.7,
      "end": 1103.95,
      "text": "Now, we will make a k from our own mind. How much should be in this question? k
means number of transactions are allowed. So, from our own mind, write k is equal to 2. It's
done. Now, use it. Now, what will we do? int gain is equal to our function. We will call. How
will we call the function? a, n. When will it start? From 0th day. And, pass k. After this,
take care of one thing."
    },
    {
      "start": 1103.95,
      "end": 1128.88,
      "text": "If this gain is negative, suppose there is no way to make a profit, then what
will you do? You won't do anything. Max of zero and gain. We don't need this. We think that
the one who is not going to do anything will also work. Let's check this quickly that we don't
need to do anything."
    },
    {
      "start": 1128.88,
      "end": 1157.6,
      "text": "There is no need to do this. We think that our case is that we will not do
anything. It is our choice. Why will we take a negative value? If you don't want to do
anything. So if we will not do anything, then we are accepting it. Okay, my internet is gone.
Okay, there is no need for this. There is no need for this. If there is, then see. I don't
think so. My code has not run. My net is gone. No, no, do this. Do this. If you will not do
anything, then you will not do anything. Take this."
    },
    {
      "start": 1157.6,
      "end": 1181.57,
      "text": "The question of buying and selling stocks is over. Yes, now this is a recursion.
 You have to convert it to DP. You have to convert it to tabulation. We don't want to focus on
that. You don't have to do anything in that. What is this? There is I and K. So what type do
you have? You have to make a DP array of I plus 1 and K plus 1. And you have to memoize it. We
are not telling you in that right now because it has been going on for a long time. It is just
of 2-3 lines. My goal for you is"
    },
    {
      "start": 1181.57,
      "end": 1211.04,
      "text": "There are three problems to be taught in the future. So this problem has become
like this. Memoirize in this. Memoirize and then tabulate too. You must have learned that too.
But now this problem has happened. Now see how much time it will take to do 2, 3, 4. Let's
move ahead. Buy and sell stocks 2. Now come to 2. Buy and sell stocks 2. Read this question.
One meaning of the question is. In the last question, how much was it allowed for you to buy
and sell? In this, unlimited transaction is allowed. Yes, unlimited transaction means."
    },
    {
      "start": 1211.04,
      "end": 1238.26,
      "text": "Buy as much as you want, sell as much as you want. But, it doesn't mean that
you buy 3 stocks and sell all 3. No. Sell one by one. It means buy, sell that. If you want,
then buy. Then sell. Then buy, then sell. It should be in this order. It shouldn't be that you
buy 10 stocks at once and sell them in a line. No, don't do that."
    },
    {
      "start": 1238.26,
```

```
      "end": 1266.78,
      "text": "Kharido, becho, kharido, becho, kharido, becho. Ye unlimited hai. Ye cheez
unlimited hai. Matlab, jo aapko number of transactions tha kya? Number of transactions, pichle
question me do tha. Isme number of transactions infinite hai. Matlab, jitne man karat number
kharido beta. Lekin order me, order me. Kharidna hai, phir bechna hai. Aisa bhi nahi karna hai
ki pehle hi bech diye. Wo to against the rule hai. Matlab, ye wala cheez unlimited hai. Ab ye
toh bilkul naya question hai. Iske liye toh phir 20 minute dena padega."
    },
    {
      "start": 1266.78,
      "end": 1291.55,
      "text": "You don't have to. You don't have to. In the last question, we learned a small
trick to solve it. See, what was happening in the last question? In the last question, we were
starting with k is equal to 2. So, when k is equal to 2, you had the option to buy. It was a
buy or nothing option. When we used to buy, we used to do minus 1 in this and make it k is
equal to 1."
    },
    {
      "start": 1291.55,
      "end": 1313.46,
      "text": "Now, when k is equal to 1, then we used to say that now you have the option to
sell. Okay. After selling, we used to make it minus 1 again. And when k is equal to 0, then we
used to say that now you can't do anything. This is what we were doing. Okay. In this question,
 we won't be able to do this. You see, the meaning of infinite transaction is not that you
pass k to infinite."
    },
    {
      "start": 1313.46,
      "end": 1339.07,
      "text": "Then how will the DP array be made? There is no such thing as infinite. If some
people are thinking that we will pass K is equal to int max, then the DP array that you made,
the DP array that we made, you must have made it, you must have made it with confidence. This
was, now from where to bring infinite memory in this, we cannot do that. There is no need to
do that. Why don't we do one thing? Tell me one thing."
    },
    {
      "start": 1339.07,
      "end": 1366.48,
      "text": "k is equal to 2 only pass in this also k is equal to 2 only pass but someone
will say that 2 transaction is not allowed infinite is allowed ok in buy pass k is equal to 2
ok as it is as it is pass now when you buy then you make it k is equal to 1 till there also ok
in k is equal to 1 you sell also in k is equal to 1 you sell also but but after selling after
selling"
    },
    {
      "start": 1366.48,
      "end": 1395.1,
      "text": "Here, instead of doing minus 1, make it again k is equal to 2. Meaning, make it
again k is equal to 2. Here, this is done. What do we want to say? In the beginning, you pass
k is equal to 2. Keep the same check that when k is equal to 2, we will buy. It's a good thing.
 After buying, you make it k is equal to 1. It's fine till there too. In k is equal to 1, the
same code is checked that k is equal to 1 means sell. But,"
    },
    {
      "start": 1395.1,
      "end": 1419.09,
      "text": "After selling, make it again K is equal to 2. Don't make it K is equal to 0.
The program will stop from 0. Make it again K is equal to 2. What will happen with that? It
will automatically convert to infinite. Why are we saying this? Suppose you had K is equal to
2. You bought 1. This became K is equal to 1. This became K is equal to 1. Means you bought 1."
    },
    {
      "start": 1419.09,
      "end": 1443.23,
      "text": "Now you have the option to sell. You sell it again and make it 2 again. Now you
have sold one. Now your case has become 2 again. That means you have the power to buy again.
If you want, you can buy again and make it 1 again. Then 1 is done. Now you have the power to
sell again. If you want, you can sell it again and make it 2."
    },
    {
      "start": 1443.23,
      "end": 1469.76,
      "text": "See, this has become a cycle in itself. So, by changing only one word
automatically, this two transaction thing will become an infinite transaction. Where to
change? Look, buy is as it is. This is a buy, let it go. If you want this, then remove it. It
will not be needed because we will not let K0 happen. If you don't want, then don't remove it."
    },
```

```
    {
      "start": 1469.76,
      "end": 1499.15,
      "text": "By is fine. Cell is also fine. You just have to change one place. After cell, when we were doing this k minus 1, here again pass 2. Here again make it 2. After k is equal to 1, again make it 2. The program is over. We said that it will be very easy to understand 2. It is over. There will not be a single line change in the main function. Here 1 minus is made plus. Your program is over. Go and run it. And go and buy and sell top 2."
    },
    {
      "start": 1499.15,
      "end": 1527.47,
      "text": "Let's move on to buy and sell stocks 3. We used to say that all the hard work will be in one. The rest is not even small. It is useless. Both are fake questions. Now come to buy and sell stocks 3. This is a hard question. This hard question means that the lead quote is hard. It is not hard for us. It is not hard for us at all. Don't think of it as hard. Let's read the question 3 and see what it says."
    },
    {
      "start": 1527.6,
      "end": 1557.02,
      "text": "You have given the same error. What is this question? You can do at most two transactions. How many? You can do two transactions. You can do two transactions. Now, in this question, the meaning of transaction is opposite. In this question, the meaning of transaction is opposite. The meaning of two transactions. One transaction means this one. Buy, sell. These people, the lead code people,"
    },
    {
      "start": 1557.02,
      "end": 1576.37,
      "text": "One transaction counts. And one more buy and sell. One more buy and sell. This is counted as one transaction. So, you have two chances to talk about this. That means, buy and sell that once. Buy and sell that twice. So, this is the question."
    },
    {
      "start": 1576.37,
      "end": 1605.89,
      "text": "This question is a very important question. This question is also a very important question that we will also tell you. Now, for their transaction, our transaction, what was our transaction? Buy means one transaction, sell means one transaction. So, if you are allowed two, means you get one buy, one sell, one buy, one sell. So, what is the total? Our, which we were writing K, you are allowed total K is equal to four. Yes, you are allowed K is equal to four. Now, now, now,"
    },
    {
      "start": 1605.89,
      "end": 1617.89,
      "text": "When you will start from k is equal to 4. So in the beginning. Now see. Now the code which we wrote in the first one will break. Because what the first code is checking? It was checking only 3 things. k is equal to 0."
    },
    {
      "start": 1617.89,
      "end": 1642.93,
      "text": "K is equal to 1, K is equal to 2. In this, he used to say nothing. In this, he used to say sell. In this, he used to say buy. But now, we have come to K is equal to 4. Because in total, how many things are allowed? 4 things. You can buy and sell 2 times. 4 things are allowed. So, we have to change it a little. Let's see what we have to change. In the beginning, K is equal to 4 will remain. You can only buy. You can only buy here."
    },
    {
      "start": 1642.93,
      "end": 1669.76,
      "text": "By buying, you will make it K is equal to 3. Now, if you have bought once, then you can sell it. By selling, you will make it K is equal to 2. Now, you will make it less in sell. Now, in sell, it is not like you will add it again. That was done in infinite. Now, you will make it normal. What you were doing, less in buy, less in sell. K is equal to 2 has come. You can buy again. After that, you will come to K is equal to 1. K is equal to 1, you will sell again."
    },
    {
      "start": 1669.76,
      "end": 1699.55,
      "text": "K is equal to zero. And once K is equal to zero, it will end. This is the cycle of this. So, notice one thing. Here, you can buy. Here, you can buy. Meaning, in 4 and 2, when K is equal to 4 and 2, you can buy. And here, sell. And here, sell. Meaning, when K is equal to 3 and 1, you can sell. Can you see anything? When, when,"
```

```
    },
    {
      "start": 1699.55,
      "end": 1728.54,
      "text": "K is even, you can buy. When K is odd, you can sell. See this. See this. This
is it, right? Nothing else is happening. When K is even, you can buy on 4 and 2. 4 and 2. When
K is odd, you can sell. On 3 and 1. This is it, right? So, do this. What do you do? Remove
your code. Remove your code."
    },
    {
      "start": 1728.54,
      "end": 1758.0,
      "text": "Let's change some things. First of all, the cell we did for infinite, let's
make it k-1 again. Now, now, let's leave k is equal to 0. Here, instead of checking equal to 2,
 k modulus to equal equal to 0, means even. What does this mean? Even. Yes, yes, even. And, if
not, let's leave the else one the same. Look at this code. Look at this code. And,"
    },
    {
      "start": 1758.0,
      "end": 1786.93,
      "text": "In our main function, here we had kept k is equal to 2. Here we have kept k is
equal to 4. Here we have kept k is equal to 4. This is the end of the code of buy and sell
stocks 3. I had to change the code this much. I had to understand this much. So, there must
not have been any problem in this. This is my full hope. This is a hard question by the way.
This is a very hard question. Let's go to 4. Let's go to 4. Buy. Let's go to 4. Buy. Sell
stocks."
    },
    {
      "start": 1786.93,
      "end": 1816.99,
      "text": "4. Let's see what 4 will give. It will take less time in 4. It will take less
time in 4. What does 4 give? Okay. 4 says same is less. Now, now, now. In 3, in 3, how much?
We are talking about the transaction of the lead code. 2 transactions were allowed. How much?
2 transactions were allowed. Transaction means the lead code. By cell counts 1 transaction. So
what did we do to it? We made k is equal to 4. We did 2 into. By cell, by cell, 2 into will
happen. This question, what does it tell you?"
    },
    {
      "start": 1816.99,
      "end": 1846.43,
      "text": "You are allowed K transactions. In this question, you will get a K. What does
the transaction mean? What does the lead code understand? It understands buy and sell as one
transaction. So what will we do? We will do nothing. We will directly do K is equal to K into
K into 2. And we will print the code of 3. The game is over. This is what he is saying. Only
this has changed in it. In 3, you were allowed 2. In our world, 2 into 2. 4 was allowed."
    },
    {
      "start": 1846.43,
      "end": 1875.86,
      "text": "In this question, K transaction is allowed. So how much will be allowed in our
world? K into 2 transaction. The rest is the same. What difference does it make? When you
solve for K is equal to 4, then won't the code K is equal to 8 work? Won't the code K is equal
to 8 work? Absolutely it will work. Why won't it work? What to do? Come simple. In your main
function, take out your main function. Where is the main function? Now instead of K is equal
to 4, and now in this function, you will also get K. You will get K."
    },
    {
      "start": 1875.86,
      "end": 1905.42,
      "text": "You will also get it. Simply what to do? Remove the int here. And K is equal to
K into 2. It is over. Buy and sell stocks 4 is over. As it is, let the recursion remain. It is
over. Buy and sell stocks 4 too. How many minutes did it take? It took 2 minutes to solve a
hard question. So that's why we said that understand 1. 2, 3, 4. It will fall like that. It
will fall like a domino. So go and solve all the questions. What to solve? First solve."
    },
    {
      "start": 1905.42,
      "end": 1930.0,
      "text": "And your job is to memorize the first code in DP and do tabulate. You should
also do tabulate by yourself. We didn't tell that today because that was not my main focus
today that you have to teach memoization. No, we have taught that. You have to teach them how
to build anything, how to think anything from the ground up. So if you start solving 4
directly, then you think what is he teaching? Right? If we teach it separately that first
teach 1."
    },
    {
      "start": 1930.0,
```

```
      "end": 1951.5,
      "text": "Now until you learn 1, the next day you come to learn 2, then you know that you
have forgotten 1. Now you have learned 2, then the next day you took a break to learn 3, then
you forgot everything. So you should not do it like that. That's why my goal was that in one
slot, one to four, we will tell you in one shot. So solve this and now 15 days of DP is over.
You have learned DP like work. What work like work? You have learned very good DP. Okay, see
you in the next one."
    }
  ]
}
```


6. transactions/t_15.json


```
{
  "video_id": "6ZtNHvGeEu4",
  "title": "Easiest Graphs question Surrounded Regions | DSA Patterns 2026",
  "language": "en",
  "model": "large-v3",
  "duration": 1044,
  "segments": [
    {
      "start": 0.0,
      "end": 28.62,
      "text": "Hello kids, you are watching Pado with Pratyush. My name is Pratyush and today
I didn't want to make a video but this question was shown somewhere in the comments that I
don't understand the surrounding region, please make it. So it is a very easy question, we
will tell it immediately. It is not possible that if you have come while reading this graph
series, then you will not be able to make it, you will not be able to understand it, it is
absolutely not possible. We have already done a very hard question from it, the question is a
piece of cake, we will do it at an easy level, provided that you are coming while reading it."
    },
    {
      "start": 28.62,
      "end": 57.62,
      "text": "Let's quickly explain the question to you. What is the Surrounded Region? The
question will be a little hard for you because the question that has been explained in the
leetcode is very bad. The leetcode does not know how to explain the question. I don't know
what they do, but it's okay. There is also a problem with English, so it's okay. You have been
given a matrix. If it is a question of graph, then there will be only two things. Either you
will be given a graph, meaning either you will be given edges or you will be given a matrix.
We have built both the methods. In this condition, you have been given a matrix."
    },
    {
      "start": 57.65,
      "end": 85.79,
      "text": "Matrix means a 2D array. Matrix means a simple 2D array. We treat it as a graph.
 0, 1, 2, 3. 0, 1, 2, 3. Now, what is there in this matrix? There are two types of characters
in this matrix. Note that this is a character matrix, not an integer matrix. One character is
x. One character is x. And one character is o. One character is o. It is not 0, it is o."
    },
    {
      "start": 85.79,
      "end": 113.17,
      "text": "We will write x and o. We will quickly make x and 0 in this. So this is x, x, x,
 x, x, o, o, x. I am telling you again that it is not 0, it is o. x, x, o, x, x, o, x, x. This
is your matrix. You have to find the surrounded regions in this. What is to be found? We have
to do something in it. Let me tell you what it means. Surrounded regions of o. See."
    },
    {
      "start": 113.17,
      "end": 140.27,
      "text": "In Hindi, we tell you a question that what does a question want to ask you? In
this, you must be seeing a lot of O's. One is visible, one is visible, one is visible, and one
is visible. Okay? Okay. Look, what does Surrounded Region mean? This X, you understand this X
as a monster, that X will slowly eat this zero. You understand X as anything, understand it as
water, it as an animal, understand it as anything. What will happen slowly?"
    },
    {
      "start": 140.27,
      "end": 168.56,
      "text": "X will eat this. Who will it eat? It will eat the zero. If X wraps the zero
somewhere, then it will eat the zero. Which zero will be saved in this? After the whole
process, which zero will be saved? The zero which is on the edge. The zero which is on the
edge will be saved. It will be saved or it will run away. It will get help from somewhere.
Whichever zero is on the edge. What does edge mean?"
    },
```

```
    {
      "start": 168.56,
      "end": 197.71,
      "text": "This is the edge. This is the edge. This is the edge. And this is the edge. It
means, either the first row is done, or the first column is done, or the last column, or the
last row, or the last column. The edge means, on the boundary, on the boundary of the matrix,
the zero will be saved. It will not be able to eat the X because it will be helped from
somewhere. It will be helped from outside. But, whatever O or whatever zero is, the one who
will be surrounded by X inside, it will be eaten and it will also become X. What will it
become? It will become X. You just have to do this."
    },
    {
      "start": 197.71,
      "end": 226.88,
      "text": "that the zeroes that are going to be saved, the zeroes that are going to be
saved, one second. So what we have to tell you is that the zero that is on the edge, we have
changed the diagram a little bit to make it easier to understand, it will be saved, it will
not be able to eat the X. And the zero that is on the edge, it will not be saved alone, it
will move in its four directions. And the zero that is connected to it, the zero that is
connected to it, means that you can go anywhere from an edge zero, as much as you can spread
in the four directions,"
    },
    {
      "start": 226.88,
      "end": 256.5,
      "text": "all the zeros will be saved in the end. What does this mean? In this matrix, in
the beginning, we have given you this zero. How many zeros are there? One, two, and three.
Which zero do you see on the edge? Which zero do you see on the edge? What does edge mean? I
am telling you again. First row means this one. Last row, first column and last column. This
is called edge or boundary zero. So here you will say that we see this zero. This zero is
visible which is still unvisited. In the beginning, everything is unvisited."
    },
    {
      "start": 256.5,
      "end": 278.75,
      "text": "Now what we will do is we will make it something else. We will make it
something else. Let's make it a hash. We will make it a hash. We will make it a hash. Now you
don't have to stop here. You don't have to stop here. Can you move somewhere else from this
hash? In four directions. Up, down, right, left. So see."
    },
    {
      "start": 278.75,
      "end": 288.0,
      "text": "We can't move below this because the world ends below this. Very good. We can't
move beyond this because we don't want to save X. X is a monster. Okay."
    },
    {
      "start": 288.0,
      "end": 315.33,
      "text": "We can't move it to the right because x is in the right, so we don't have to
save x. But above this, what have you got? Above this, you have got a zero. So that zero will
also be saved. Why will that zero be saved? Because it was connected to a zero which was on
the boundary. So if help reaches this zero, then it will also save all its connections. So
what will we do? We will go above this and go to the next zero."
    },
    {
      "start": 315.33,
      "end": 336.34,
      "text": "Go to the next zero and make a hash here as well. Why are we making a hash? To
make a visited mark and to show that it is saved. You can do something else instead of a hash.
Any character other than x and zero, any character other than x and o, write it here. Now you
have come to this hash. Can you go somewhere else and save another zero from this hash? Let's
see."
    },
    {
      "start": 336.34,
      "end": 359.55,
      "text": "Can you go up? No, there is an X above. Can you go left? No, there is an X on
the left. Can you go right? No, there is an X on the right. Can you go down? No, the one below
is already a hash, so it is visited. You could have just gone towards 0 or 0. So after this,
you are not saving anyone, so you will stop. Now you will stop. Now do you see any other 0 or
any other O which is left on the edge?"
    },
    {
      "start": 359.55,
      "end": 389.17,
      "text": "No, one was this one. We have done its work that if help reaches it, then it
```

```
will be safe from its own monster and will save all its friends. Those who are connected to it,
 those who are connected to it, in which direction? In four directions, up, down, right, left,
it will also save them from this monster. Now your game is over. Now see, this one which was
zero, it is not connected to any boundary zero. It is not connected to any boundary zero. Now
you will say that it is not connected like this."
    },
    {
      "start": 389.17,
      "end": 414.34,
      "text": "Diagonal connection is not allowed. You just have to make a connection up and
down. So you see this is in a way surrounded by all four sides. Surrounded by what? Surrounded
by X. So this X will be eaten. After this is completed, which zeros will be saved? This zero,
which you have made a hash, will be saved. Because you are doing this. You are starting DFS
from the boundary zero."
    },
    {
      "start": 414.34,
      "end": 441.46,
      "text": "And it saves itself. It saves all its connected zeros. One will be saved. One
will be saved. So what do you have to do in the end? What do you have to do in the end? Where
there is a hash. Where there is a hash. You have to remove it and make it zero. Make it zero.
And the rest of them. This zero. Now your x will be eaten. And it will make it x. It will make
it x. The rest of them have to make x. The rest of them."
    },
    {
      "start": 441.46,
      "end": 464.34,
      "text": "Make it x. What else? Make it x. Your final matrix will look like this. See.
This zero is eaten so it becomes x. These two zeros are saved. Why are these zeros saved?
Because where was this zero? On the edge. And it also saved this zero. If by chance there was
a zero here, then this would have been saved too. So this would have been saved too. Why?
Because you would have gone from here to here. You would have gone up from here."
    },
    {
      "start": 464.34,
      "end": 491.78,
      "text": "So any zero of any boundary has the power to save all the zeros around it
because it will help it from here. The zeros that are unable to save, the zeros that are
unable to save till the end, like this zero is unable to save till the end, this will become x,
 the surrounding x will be covered and eaten. That is why the name of this question is
surrounded region. This zero was surrounded by x, so it was eaten by x. So it is a very simple
question."
    },
    {
      "start": 491.78,
      "end": 520.5,
      "text": "You just had to understand the hard part of the question. Now there is no hard
part. Now you must have understood what to do. You have to start DFS from the boundary O."
    },
    {
      "start": 520.59,
      "end": 548.43,
      "text": "Normally, when we were studying the number of islands, we would maintain a
visited array that we have seen this. Instead of that, what you should do is, make every zero
a character of your choice. Hash, dollar, rupeeya, whatever you want. Whatever looks different,
 so that you know that it is saved. In the end, we have to save it and convert it back to zero.
 Do this. You have to do this with every boundary zero."
    },
    {
      "start": 548.43,
      "end": 576.18,
      "text": "Because it is possible that every boundary zero has the power to save multiple
people. If any boundary zero is already a hash, then don't do it. If you are getting zero on
the boundary, then what do you have to do from there? You have to start the DFA. In which
direction? In four directions. You already know all this because of the number of islands.
After all this happens, in the whole matrix, wherever there is a hash, convert it back to zero
or O because they will be saved in the end. And the rest of them,"
    },
    {
      "start": 576.18,
      "end": 606.18,
      "text": "The rest of them, what will you make? Make them x. Because the x is x. The
zeros that will be saved, the zeros that will not be able to be hashed at the end, what is
that zero? That zero is surrounded by monsters. So, x will eat them and make them x. This is
the only small thing in this. Its code is just like number of islands. So, what were we doing
to write number of islands? First of all, we were writing a Boolean valid function. First of
all, a valid function. Why? So that we don't get ahead of the board."
```

```
    },
    {
      "start": 606.18,
      "end": 635.26,
      "text": "Those who are watching the video from the beginning will remember all this. We
believe that you must have remembered all these things. There should be no doubt in this. If i
is negative and i is n and j is negative and j is greater than or equal to m, then return is
false. This is not valid. Valid means that you don't have to go there. It can happen in DFS
that you are trying to get out of the board. You won't go. If this is not the case, then
return true."
    },
    {
      "start": 635.26,
      "end": 664.58,
      "text": "Return. True. Let's make an array of x and y. Why? For direction. Remember the
number of islands we told. So, the trick of Stryver Bhaiya. His trick was very good. The same
trick. That you don't have to work hard to go in all four directions. Don't have to work hard.
What is this? x coordinate of all four directions. And y coordinate of all four directions.
Very good. Come, let's write our DFS. It will be a very simple DFS. Void."
    },
    {
      "start": 664.58,
      "end": 695.01,
      "text": "DFS. What will be passed in DFS? Your matrix. What kind of graph is given to
you? Care type graph is given to you. int n, int m, int i, int j. This will happen. i, j means
you are here. What was the first thing you said? Make this hash. Make this i, j hash. So that
it gets visited. If it gets visited, then make it like this. Now what to do?"
    },
    {
      "start": 695.01,
      "end": 724.1,
      "text": "Look at its neighbors. How? Look at the neighbors of all four directions. k is
equal to 0. k less than 4. k plus plus. How do we look at the neighbors of all four
directions? int row is equal to i plus i plus x of k. So this will be your row. int column is
equal to j plus y of k. So you will get this column. Check it. If this is valid. If this row
and column is valid. If this"
    },
    {
      "start": 724.1,
      "end": 753.89,
      "text": "row and column is valid and and if it is 0 if it is 0 we don't have to go
towards hash we don't have to go towards hash and we don't have to go towards x so array of
row column row column equal equal to O O write O here again and again we made a mistake we
also started writing 0 remember to write O it is confusing but remember to write capital O ok
if it is like this then do DFS on it array of n m row n column so it will just become DFS it
is nothing"
    },
    {
      "start": 753.89,
      "end": 783.84,
      "text": "After starting everything, your for is closed. Return it here. So this is your
DFS. How did you write the main program? You don't have to write anything in the main program.
In the main program, you have to solve a void. You have to give a character array. You have to
do this and give an array. Take out its size. Size means number of rows. Take out a.size."
    },
    {
      "start": 783.84,
      "end": 813.71,
      "text": "Take out the size of the column. We will also take out the size of the column.
What did we say? From whom do you have to start DFS? From the boundary zero. From which? From
the boundary zero. Boundary zero means first row, first column, last row and last column.
Let's do it one by one and see all of them. What does it mean to see all of them? First of all,
 we will see the first row zero. First row O. We will see O. So see. Int i, j. What will we
do? For j is equal to zero."
    },
    {
      "start": 813.71,
      "end": 843.87,
      "text": "j less than m, j plus plus. Whose is this? What is this? First row. What is
this? First row. First row. First row means when i is equal to 0, it will take the first row.
When i is equal to 0, it will take the first row. So if array of 0, j equal equal to 0. What
do you have to do? DFS array n, m, 0, j. What is this? First row. Last row."
    },
    {
      "start": 843.87,
      "end": 873.7,
      "text": "Last row. Last row means, everything will remain exactly the same. Is equal to
```

```
0, j less than m, j plus plus. In the last row, array of 0, no, array of n minus 1, j. Equal
equal to O. Then call DFS. DFS, array, n, m, n minus 1 and j. So this is the last row zero's
take care. First column. First column. First column."
    },
    {
      "start": 873.7,
      "end": 904.48,
      "text": "Just do the opposite. Now increase the i. Take the i from 0 to n. We hope that
you are able to understand. What does it mean to take the first row? i is equal to 0 and j is
everything. So you will take the first row. What does it mean to take the last row? Fix the i
on n-1 and take the j completely. What does it mean to take the first column? Fix the j on 0
and take the i completely. So that's what we are doing. If array of i 0 is the first column.
Equal equal to o. DFS. What? Array n."
    },
    {
      "start": 904.48,
      "end": 934.03,
      "text": "M I 0. This is done. Last column. Last column. Just do the same thing. For I is
equal to 0. I less than N. I plus plus. If array of I. M minus 1. M minus 1. Last column.
Equal equal to O. DFS. Array. N. M. I. M minus 1. Very good. You have done the entire boundary.
 What will happen by doing the entire boundary?"
    },
    {
      "start": 934.03,
      "end": 964.62,
      "text": "All the boundary zeros will be hashed. All their friends will be hashed.
Meaning, all the connections, where, where, where. Meaning, if any O is connected to the edge,
then it will be hashed. After this, those who will be saved, they will be eaten by X. After
this, those who are hashed, you have to make them zero. So, what will we do in the last step?
We will loop the entire array. I is equal to zero. I less than N. I plus plus. For. J is equal
to zero."
    },
    {
      "start": 964.62,
      "end": 989.57,
      "text": "j less than n, j plus plus. Now, the people who have made hash, if array of i,
j equal equal to hash, what do they have to make? Zero. They will be left. They will be left.
They will be left. Oh, oh, else, array of i, j is equal to cross. The rest of them, cross will
remain cross."
    },
    {
      "start": 989.57,
      "end": 1019.52,
      "text": "If there is a zero, which is saved, it means that this X is going to eat it.
After doing this, return it. So look at this. The code is a bit long. We believe that the code
is a bit long. But it is long because you are writing four loops here, so it has become long.
By the way, all this is boilerplate code. Boilerplate means that it is a very normal DFS code.
So see this code from here. See this code. This is DFS. This is void solve. See this. There
are four types of loops."
    },
    {
      "start": 1019.52,
      "end": 1043.63,
      "text": "And this is what you do in the last. Time complexity order of m into n. There
is not much space used in space complexity. Space must have been used in DFS. Yes, space will
be used in DFS for stack. So that will happen in m into n worst case. So this was the
surrounded region. Very easy, very fake question. But very famous question. Very good question.
 So go and do this. And see you tomorrow."
    }
  ]
}
```


7. transactions/t_16.json


```
{
  "video_id": "7gOCx1vbx0k",
  "title": "Linked List Reversal In the Easiest Way Possible | DSA Patterns Course 2026 CPP
JAVA Python",
  "language": "en",
  "model": "large-v3",
  "duration": 3143,
  "segments": [
    {
      "start": 0.0,
      "end": 24.26,
      "text": "How are you all? You are watching Paro with Pratyush. My name is Pratyush
```

```
Narayan. And you must have read the link list of this reversal a lot of times. But what many people do is that they make you remember that do this, do that, do that. You don't have to remember. You need to understand this reversal very well. We will explain it with such examples that you will never forget it in your life. You will not be able to forget it even if you want to. Before starting, those who have come for the first time, please see in the description."
    },
    {
      "start": 24.26,
      "end": 45.94,
      "text": "If you guys watch this video, then watch it completely. We say again and again that don't increase my view. What will happen to you if you increase my view? If my view increases after seeing you for 2 minutes, then nothing more will happen. We don't get that much money for one view. Don't waste 2 minutes. Watch 4 reels in 2 minutes. Watch a good web series in 5 minutes. Don't waste time because"
    },
    {
      "start": 46.42,
      "end": 76.3,
      "text": "We don't understand the reason for this. There should be no reason to look at half of it. Either it is possible that you are very busy. It doesn't happen that B.Tech people are very busy because we have also done B.Tech. It is not that we came directly to the job. We have done B.Tech and we know that no one is busy. And if you think that you are getting very busy in going to class for 2-3 hours a day and making notes, then don't study. And after a year, your job will take place in the call center. Your shift in the call center will be 15 hours per day. And the salary is Rs. 12,000."
    },
    {
      "start": 76.3,
      "end": 105.39,
      "text": "So we work for 15 hours a day, we get 3 lakh rupees per month. You will work for 15 hours, you will get 12,000 rupees per month. So your time will be very cheap, your time will be cheaper than that of a rickshaw driver. If a rickshaw driver drives a rickshaw for 15 hours, then he will earn more than 12,000 rupees. So you will become a very cheap person. So if you want to do that, then you can do that too. That is also a way of life, many lakhs of people do it. It is not that there is no job in BPO, lakhs of people are doing jobs in BPO. In call center etc., in selling credit cards etc. So okay, you can do all that. Let's go."
    },
    {
      "start": 105.39,
      "end": 127.18,
      "text": "Now, why is reversal a separate pattern? Reversal is a separate pattern because once you get a reversal, there are a lot of questions related to it. I mean, you will have to reverse in everything, but different things are being told that reverse this way, reverse that way, break it from here and reverse it. I mean, it will be complex. Its question can be complex, it won't be difficult. You will never..."
    },
    {
      "start": 127.18,
      "end": 154.1,
      "text": "It won't seem like we don't know what to do. You will always feel that we don't know what to do. How to do it? There is a problem in that. There is a problem in writing the code. What to do? So its question will always be complex. It won't be difficult. It will never be difficult. Hard question too. Lead code hard question is also complex. It is not difficult. If someone tells you to speak in Hindi, you will tell him immediately. But how to implement becomes hard. So that's why it is complex."
    },
    {
      "start": 154.1,
      "end": 178.99,
      "text": "So, what is the meaning of reverse arrow? See, a link list is a train. So, first assume that the engine is in the beginning. After that, you have AC bogey, then sleeper, and in the end, general. Someone tells you to reverse this train completely. What do you do to this train? Reverse it completely. Means, do this to this arrow, do this to this arrow, do this to this arrow, and do general first. Means, reverse it completely. So,"
    },
    {
      "start": 178.99,
      "end": 208.98,
      "text": "Now to reverse, there is a very simple algorithm, you can also remember it, but there is no need to remember it, just understand it like this. Now you have to go and change the arrow of every bogey, every bogey's arrow means you have to change the connection of every bogey. So first of all, you need a mechanic, you need a mechanic, what do you need? Mechanic or iron, whatever you need to change the wire of the bogey. Keep one more helper, one helper of this."
    },
    {
```

```
      "start": 210.22,
      "end": 238.56,
      "text": "It doesn't know much, but it has a very special work. Let's see what is its special work. First of all, what will your mechanic do? Your mechanic will go and stand here. Where will your mechanic go? On the engine. He will stand on the first one. Now its work is to turn this arrow and turn it back. Turn it and turn it back. Towards whom? Towards the helper. Yes. So what is the work of the helper? Helper always"
    },
    {
      "start": 238.56,
      "end": 265.12,
      "text": "It will stand on the back bogie. It will stand on the back bogie. What is the mechanic's job? It will take the arrow of the bogie and turn it towards the helper. So, what will your mechanic do? It will take this arrow and what will it do? It will turn it towards the back. Towards whom? Towards the back. Now, okay, this job is done. Okay, this arrow has been turned. Very good. Move the helper forward. Move the helper forward. Bring the helper to the AC bogie."
    },
    {
      "start": 265.12,
      "end": 293.44,
      "text": "Sorry, not the helper, move the mechanic forward. Now, because the mechanic has moved forward, what is the helper's job? The helper's job is to stay on the back bogie. You have to pick up the helper, you have to pick up the helper, and where do you have to bring him? You have to bring him on the engine. See, still, where is the mechanic standing? Where is the mechanic standing? The bogie's wire has to be changed, the bogie's connection has to be changed. Where is the helper standing? The helper is standing on the back one. The helper was already standing on the back one, that's why we had turned it here. You can consider it null because"
    },
    {
      "start": 293.44,
      "end": 317.42,
      "text": "There was nothing in front of the engine, so we did it in the back. Okay. Same work, same work, the mechanic will do the same work again. What will he do? This AC wire connected to the sleeper here, he will turn it and put it on the engine. Why? Because the helper was standing there. So, where the helper is standing, he will tell him to put his wire there. Now, what does the mechanic have to do? The mechanic has to move forward. The mechanic has to move forward simply."
    },
    {
      "start": 317.42,
      "end": 346.77,
      "text": "Mechanic has to move forward and stand on the sleeper. But helper also has to move forward. Because helper is standing on the back boogie. Helper is standing on the back boogie. So where will helper come? He will come on the AC. What will mechanic do? Mechanic will remove the sleeper wire. And where will he attach it? He will attach it on the AC. Because helper is standing there. Now what will mechanic do? Mechanic will move forward. Mechanic will move forward. Mechanic will move forward and come on the general."
    },
    {
      "start": 346.77,
      "end": 372.08,
      "text": "What will helper do? Helper will also move forward. Helper's job is to always help the one at the back. Always help him. Come here. What do we have to do? There was no wire attached in front of the general. There was no wire attached in front but we have to attach the wire to the back sleeper. Let's attach it to the back sleeper. Now,"
    },
    {
      "start": 372.08,
      "end": 396.72,
      "text": "Mechanic will move forward, but as soon as mechanic moves forward, then this train has gone out, now there is no bogie left, and helper has also moved forward. Helper has also moved forward. Helper has also moved forward, on whom? On General. Now, it is over, if there is no bogie left, then mechanic will not do anything alone. So things are over, but look carefully what happened in this. What happened in this train, see."
    },
    {
      "start": 397.01,
      "end": 421.26,
      "text": "The engine has turned and has gone behind which was null or whatever it was. AC used to point the sleeper first, but now AC points to your engine. Sleeper used to point to the jungle first, but now sleeper is pointing to AC. General was on the last bogey, but now general is pointing to sleeper."
    },
    {
      "start": 421.62,
```

```
      "end": 448.85,
      "text": "Now, if someone asks you to draw this link list, then you can draw it like this.
 How will you draw this link list? First of all, your general. Your general. Who is the
general going to point? The sleeper. General is going to point the sleeper. Yes. Who is the
sleeper going to point? The AC. Sleeper is going to point the AC. Very good. Who is the AC
going to point? The engine. The engine. The engine."
    },
    {
      "start": 449.07,
      "end": 475.41,
      "text": "Who is being pointed at by the engine? No one, at the back, at the empty space,
at the null, at the null, so no one is being pointed at. Now see, the linked list has rotated,
it has reversed completely. How did it look earlier? Engine, AC, sleeper, general. Now how
does it look? General, sleeper, AC, engine. Now, who was the head of this linked list earlier?
The head was here, the engine was its head. Now who should be its head? General should be its
head because"
    },
    {
      "start": 475.41,
      "end": 504.82,
      "text": "the link list will start from general obviously, you took a lot from head you
said reverse the whole thing so the last bogey which was in the last will come first so it
will become head but how will you get this head? see your mechanic has gone out of the bogey
means your mechanic has become null in a way but who is your helper standing on? he is
standing on general who is your helper standing on? he is standing on general that's why
that's why if we do this return"
    },
    {
      "start": 504.82,
      "end": 527.31,
      "text": "my helper so what do we get which is a new head which is a new head or new
bogey which has come on the head see because helper is standing exactly on general no matter
how many times you do it no matter how big the train you take but helper will always stand on
general and the whole list will be around this is the logic of reversing this is the logic of
reversing"
    },
    {
      "start": 527.31,
      "end": 558.19,
      "text": "this is about train but these things are not called as mechanic or helper they
are called something else so we will tell you how it works in real in linked list we have
explained this to give an example come let's make a linked list small and cute make a linked
list see we like the whole question because there is no hard question in it means everything
is simple here and there we have to change the address how complex it will be so 1 2 3"
    },
    {
      "start": 558.19,
      "end": 588.56,
      "text": "4. You should remember the linked list. If you don't remember the linked list,
then go and revise the previous one. We have already taught you the linked list once. What
happens and how it happens. Introduction. Okay? This means it is null. Let's go. What did we
have to take? You have to reverse it and return the new head. You will always get a new head.
Because the head is here now. The head is here now. On 1. But when you reverse it, your head
will be this. It will be the same. Obviously, if you reverse 1, 2, 3, 4, then what? If it
becomes 4, 3, 2, 1, then it will be 4 head. Okay? What did you have to take? Mechanic or
helper."
    },
    {
      "start": 588.56,
      "end": 618.21,
      "text": "You can call it anything, mechanic or helper name, but its standard name, what
is its standard name? You have to take a curve variable, what? Curve. Curve means current,
current variable. And you have to take one variable which is previous. Previous, okay? What do
you have to put in current? The head. Because where will you start? Mechanic, from which bogey
did you start? From the first bogey, so head. And previous, initially, where was your helper
standing? Initially, where was your helper standing? Initially, where was your helper
standing?"
    },
    {
      "start": 618.21,
      "end": 644.21,
      "text": "at an empty space, that's why the engine was running at an empty space
initially your previous will be at null ok, that's it, let's go now come, just like you ran
the mechanic, now run it so first of all, curr is equal to 100 now see, there are two things
curr means your mechanic, previous means your helper so what did you do in this? what did the
mechanic do?"
    },
```

```
    {
      "start": 644.46,
      "end": 674.03,
      "text": "First of all, the mechanic who used to stand on the bogie, he used to remove the wire and send it to the previous one, the helper one. So this is one thing. After that, the mechanic had to move forward. How does it move forward? From the next. It moves forward from the next. But what are you doing? What do you have to do by breaking the next? You have to do it backwards. So how will you move forward? How will you move forward? How will you jump on the bogie? For this, for this, whatever is the bogie in front,"
    },
    {
      "start": 674.03,
      "end": 704.38,
      "text": "store this wire before moving on to the next one because earlier the AC was going to the sleeper so you know that we are on the AC now we have to jump forward and then go to the sleeper so that we can reverse the sleeper but you are in the process of breaking the AC wire you are in the process of connecting the AC to the engine then how will you jump? this place will be empty that's why store the next bogie that we have to go there that's it and how do we store it? simple we"
    },
    {
      "start": 704.38,
      "end": 735.04,
      "text": "Take a variable, node star, next, not next, you can give any name of your choice, which is current.next. Okay, means, whatever node is in front of current, we have stored it in one place. Okay, now do your work, what do you do? First of all, reverse this bogey's wire, turn it and put it back, how will you do it? Curve.next is equal to prev, means, wherever helper is standing, then prev, turn it completely and put it back. What happened to this? What happened to this? One thing happened to this, see."
    },
    {
      "start": 735.04,
      "end": 762.54,
      "text": "Your curve is here, at 100. What did you do? You turned curve.next and turned it backwards. Where is it backwards? At null. This is null. Now, increase both of them. First, increase helper. So, prem is equal to curve. Because, where does helper have to come? The bogey on which the mechanic is standing, come there. And increase the mechanic, then the same thing will happen. And, curve is equal to next."
    },
    {
      "start": 762.58,
      "end": 792.91,
      "text": "Cur is equal to next. Why? Because it will go to the next one. So, see. Now, on your curve, it will be 200. This is the program. If you are wondering, this is the program. This. This. 4 lines. 1, 2, 3, 4. Okay. Let's go. Let's run it once. Right? Now, what will be on your prev? 100. Okay. Come. Do the same thing. Let's see on curve is equal to 200. Cur is equal to 200. First of all, take out next. Next of 200. What is it? 300. Very good."
    },
    {
      "start": 792.91,
      "end": 822.45,
      "text": "After that, remove the wire of this, that is, turn the wire back completely. Where will it go? What will be the next point of 200 on 100? It will become like this. Done? Very good. After that, increase the curve, increase the previous, increase the curve. So, what will come in the previous? The value of the curve. 200. What will come in the curve? The value of the next. We will jump on the next node. Very good. Now, come here again, run again. Curve is equal to 300."
    },
    {
      "start": 822.45,
      "end": 852.35,
      "text": "First of all, take out next because you have to go there, 400, very good, keep it. Now what you have to do is, you have to turn the next of curr to previous, so turn it. Where did you remove this wire and turn it to 200? Done, that's it. Increase the previous, means previous is equal to curr. Turn the previous to 300. Increase the curr to next. Increase the curr to next, 4. Okay, we are at 400. Curr is equal to 400. Next is equal to null, it is very important, see. Next is equal to curr.next, now it is null, okay."
    },
    {
      "start": 852.35,
      "end": 882.38,
      "text": "After that, where do we have to turn the previous to? Although there is no wire in front of it, but we will connect the wire to the previous one. So, what will happen? This will happen here. Yes, sorry, we will change this too. This was on 100, this was on 200. Now this wire has come here, so this is 300. We could not change it with the help of arrow. Okay, the work is done. After that, what do we have to do? We have to move the helper forward, that is, we have to move the previous forward. So, what will we write in the previous?"
```

```
    },
    {
      "start": 882.38,
      "end": 910.35,
      "text": "400 and we have to move Kerr forward but next is null so it's okay Kerr is
equal to next as we had written so Kerr is null as soon as Kerr is null it means that Kerr is
here who was Kerr? it was the mechanic when the mechanic is in an empty space means the train
is over now the next train is over if there is no other bogie then what will you remove? so as
soon as Kerr is null it means that your program is completely over as soon as Kerr is null
your program is completely over"
    },
    {
      "start": 910.35,
      "end": 938.53,
      "text": "Now you have to return the head of the new list. But how will you return the
head? Where is its head? This is its head because earlier it was 1, 2, 3, 4. Reverse it to 4,
3, 2, 1. The last node is the head. Who is standing on the last node? Previous is standing. So
what will you do? Return. Return. Previous. Don't do it. If you don't do it, then you will
know that the train is over."
    },
    {
      "start": 938.53,
      "end": 967.66,
      "text": "Previous helper, where did helper stand? On the last bogey. So you have turned
it. This is its code. Once. Formally see its code. You will have a node star like this. This
function will be called reverse. You will be given a head. Okay. You need 2-3 things. First of
all, you need only 2 things. First of all, you need a curr. Which is on the head. Very good.
After that you need a"
    },
    {
      "start": 967.66,
      "end": 997.34,
      "text": "Previous which is our helper. You can give its name as per your understanding.
If it is in the beginning, it will be null because there is nothing in the beginning. There is
nothing in the previous. Okay? How long do we have to run it? Until the curr is null. While
curr is not equal to null. Very good. First of all, you have to remove the next of curr.
Remove the next of curr. Because you are going to change the next of curr. So, how will you go
to the next? You will change it. After that, turn the wire. Turn the wire. Hold the wire and
turn it back. Where? How will you turn it back?"
    },
    {
      "start": 997.34,
      "end": 1023.6,
      "text": "Previous. Now, teach both previous and cur. So, first teach previous. Bring it
on cur and bring cur on next. Your while loop is over. What will you return in the last?
Previous. Not cur, previous. Because previous will be on your last node. This program is so
small that it has reverse. Order of n is its time complexity. Order of 1 is its space
complexity."
    },
    {
      "start": 1023.92,
      "end": 1053.52,
      "text": "you guys have done the first question of the patent sheet of the link list
reversal which is what? if we see the description 206 206 is what you have to do the reversal
link list so you guys have done this the next question is going to be a little hard so listen
carefully listen to this question very carefully because it can be a very different type of
question it will look like you have given a list like this we have not made a complete list
but to make you understand first understand the question that what exactly you have to do and
how you have to do it"
    },
    {
      "start": 1053.52,
      "end": 1083.26,
      "text": "This is the linked list. Okay. 1, 2, 3, 4, 5. It has 5 nodes. But, you have to
reverse it. But you don't have to reverse it completely. If you had reversed it completely,
then it would have been very easy that you would have taken curve here and prep here. Like we
did last time, then it would have turned like this. This linked list. And return 5. Very good.
But, there is a small catch in this. What is the catch in this? You have two numbers. What?
Left is equal to 2. Right is equal to 4. Okay. What is this number? This number, 1, 2, 3, 4, 5,
 does not represent this. What is this number?"
    },
    {
      "start": 1083.26,
      "end": 1106.9,
      "text": "These two numbers are positions. What? Positions. Yes. Confusion. It doesn't
represent 1, 2, 3, 4, 5. What are these two numbers? Positions. To explain it properly, we
will erase it. Because we saw that many people had a problem in the description of this lead
```

```
quote. Assume that your list is like this. 10, 20, 30, 40."
    },
    {
      "start": 1106.9,
      "end": 1131.04,
      "text": "and 50, now there will be no confusion left has 2 and 4 and the position is
started from 1 means if you start it from 1 then 1, 2, 3, 4, 5 this is your left and this is
your right you are given these two numbers not nodes, just integer numbers so you know left
and right that 20 is 40, what you have to do is"
    },
    {
      "start": 1131.04,
      "end": 1157.33,
      "text": "you have to reverse this part from left to right not the entire link list but
the middle part and the rest of the link list will remain the same so let's see the answer to
this first of all, this middle part will be reversed and when it will be reversed, then how
will it look? 40 30 20"
    },
    {
      "start": 1157.33,
      "end": 1187.73,
      "text": "How will it look after reversing? But don't change the rest of the link list.
The rest of the 10 will remain here. 50 was here, 50 will remain here. And the arrow should
also be like this. This is yours. You have to return this. That means, reverse the middle one.
The rest of the link list, the left and right part, the one that is not reversed, will remain
the same. It will keep pointing the way it used to. Your answer should be this. You have to
return head. You have to return head."
    },
    {
      "start": 1187.92,
      "end": 1213.97,
      "text": "so this is your question and if you are a beginner then this is a fairly
complex question it is not that easy that's why you can see that it is a medium level question
let's think about what to do in this what is this that we should reverse from left to right
and do something like this we didn't learn this, we learned how to reverse the whole linked
list let's think about this so see"
    },
    {
      "start": 1213.97,
      "end": 1243.7,
      "text": "It is not a big deal to reverse from left to right. It is not a big deal to
reverse from left to right. Why is it not a big deal? See, when you had to reverse completely,
when you had to reverse completely, then what did you do? You put the curve on the head. Okay.
And this is your while loop. Now see, till the time you will run your while loop, till then
you will reverse. Reverse means what? To lift the string and join the back. The back one to
the front, the front one to the front. This is the reverse. So, the more you used to run the
while loop, the more it would reverse. So, what did we write in the while loop? While."
    },
    {
      "start": 1243.7,
      "end": 1272.43,
      "text": "Curve not equal to null. Okay. This used to happen. Okay. Assume. Assume. You
have such a linked list. You have such a linked list. Assume. We are trying to tell you that
the problem in your mind is not the real problem of this question. You must be thinking that
how will someone reverse left to right. See. 100. 200. 300. 400. Assume here is 1."
    },
    {
      "start": 1272.85,
      "end": 1304.37,
      "text": "and this points to this, this to this, this to this, and this does not point to
anyone. Okay, this is your linked list, very good. Now, if you are told to reverse the entire
linked list, then you know how to do it normally, I have taught you how to do it 5-6 minutes
ago, but, we say, do one thing, you reverse it twice, do it twice, do it twice, you start to
reverse from the beginning, but if it happens twice, then stop, if it happens twice, then stop.
 Let's see what it means to reverse it twice."
    },
    {
      "start": 1305.04,
      "end": 1334.94,
      "text": "First of all, what will be your Kerr? 100. What will be your Prev? Null. You
have to do it twice, so let's do it for the first time. First of all, take out its next and
keep it at 200. The one of Kerr's 100, oh sorry, the one of Kerr's 100, the next of 100, turn
it. Where do you turn it? Turn it on Null. This is Null. Okay. This does not point to anyone.
Okay. It happened once. Remember once. It happened once."
    },
    {
      "start": 1334.94,
```

```
      "end": 1364.85,
      "text": "now increase the curve to 200 and also increase the previous curve to 100 very
good do it one more time next to 300 now turn the next of the curve turn the next of the curve
here also turn it to previous to previous to previous to previous to previous to previous to
previous to previous to previous to previous to previous to previous to previous to previous
to previous to previous to previous to previous to previous to previous to previous to
previous to previous to previous to"
    },
    {
      "start": 1366.06,
      "end": 1390.8,
      "text": "reverse means you have reversed two nodes and if someone ask you to return its
head then you return this one because prebha is sitting here so you do it here so the rest of
the linked list is broken but you have reversed two nodes so you can reverse number of times
you can reverse till where you want"
    },
    {
      "start": 1390.86,
      "end": 1420.16,
      "text": "You have to reverse it as much as you want or as many times as you want. It is
not necessary that you have to reverse it completely. No, we have to reverse it twice. So we
will do two nodes. We have to do it three times. We have to do it four times. We have to do it
four times. We have to reverse it to a particular node. So we, while current not equal to null.
 No, we will write something else. We will write like this. While current not equal to 400.
What will happen with this? Until 400, the entire linked list will be reversed."
    },
    {
      "start": 1420.16,
      "end": 1452.56,
      "text": "This is what it means, right? So, you can reverse as far as you want. You can,
you can. Okay. Now, now. Now, let's come to this question. In this question, if you are given
left and right, then how many times will you have to reverse? Let's see how many times you
have to reverse. 10, 20, 30, 40, 50. Phases by phases, it's going on. Okay. This is your 1, 2,
4, 5. What have you been told? Left and right, 2 and 4."
    },
    {
      "start": 1452.82,
      "end": 1481.81,
      "text": "What does this mean? To whom? To this, to this, to this. 3 nodes. You will have
to change the wire of 3 nodes. What will you have to change of 3 nodes? You will have to
change the wire of 3 nodes. You will have to change the wire of 3 nodes. Yes. So how much?
Right minus left. Yes. Plus 1. Plus 1. So many times. So many times you will have to change
the wire. Reverse means you have to change the wire. We will see the rest later. What will
happen? 4 minus 2 plus 1."
    },
    {
      "start": 1481.87,
      "end": 1514.26,
      "text": "3 times. You can see, you have to reverse on 3 notes. 2 to 4 means 3 times.
That's it. So, one thing is that you have to reverse 3 times. From where to start? From left
to right. Okay. Now, there is a very easy case in this that if left is equal to right, if left
is equal to right, then what will you have to do? You don't have to do anything. See, 10, 20,
30, 40. Assume it is 4. This is 1, 2, 3, 4."
    },
    {
      "start": 1514.26,
      "end": 1540.4,
      "text": "It tells you that left is equal to 2 and right is equal to 2. Do this. So what
is it? You have to reverse this. How many times? 2 minus 2 plus 1. You have to reverse one
node. What does it mean to reverse one node? It doesn't mean anything. When you write, if you
reverse one node, it will become one node. And you have to leave the rest as it is. You have
to write this as it is. So,"
    },
    {
      "start": 1540.4,
      "end": 1569.3,
      "text": "Like this, like this, like this. This is the same linked list, right? These two
linked lists are same, right? There is no difference. So, you understood one thing that if
left is equal to equal to right, then what to do immediately? Return head. Return head. There
is no need to do anything because why are you reversing one node? One node is reversed in the
middle, what is that? Nothing happens. Let's go. This is done. One easy case, easy case. Now
you are in the main program that now you have understood times, that what is the number of
times and what happens, how it happens. We can do it if we want to. Now come, let's see. Now
come."
    },
    {
      "start": 1569.3,
```

```
      "end": 1599.89,
      "text": "Let's do it. If, if, I mean, this can be the answer that you know the meaning
of left. You know that left is 2. You know that right is 4. You will get to know times with
this. You will get to know times with this. What will you get to know? That you have to do it
three times. So, you can do one thing that when, when you, you see, left is equal to 2, means
second node. You have to come to the second node and reverse. How many times? Three times."
    },
    {
      "start": 1599.92,
      "end": 1631.81,
      "text": "three times one thing you can do is when you count you can traverse the linked
list and count that when you reach the second note when you reach the second note when you
reach the second note when you reach the second note when you reach the second note when you
reach the second note when you reach the second note when you reach the second note when you
reach the second note when you reach the second note when you reach the second note when you
reach the second note when you reach the second note when you reach the second note when you
reach the second note when you reach the second note when you reach the second note when you
reach the second note when you reach the second note when you reach the second note when you
reach the second note when you reach the second note when you reach the second note when you
reach the second note when you reach the second note when you reach the second note when you
reach the second note when you reach the second note when you reach the second note when you
reach the second note when you reach the second note when you reach the second note when you
reach the second note when you reach the second note when you reach the second note when you
reach the second note when you reach the second note when you reach the second note when you
reach the second note when you reach the second note"
    },
    {
      "start": 1631.81,
      "end": 1661.84,
      "text": "3, 4, 5. When you reach the second node, then take the reverse program. How
many times? 3 times. The whole program is the same. Just that while curve not equal to null,
instead of while times minus minus. Let's see if it is 3 times. When you reach here, you will
know that you have come to the second node. How? We will take a count variable that you have
come to the second node. So, let's start our reverse program 3 times. Exactly 3 times. So,
let's see. First of all, when we will come, then what will be our curve? Curve is equal to 200.
"
    },
    {
      "start": 1661.84,
      "end": 1693.9,
      "text": "And previous is equal to null. Let's write reverse like this. Let's start the
reverse program like this. First of all, this will be deleted. And this will start pointing at
null. Okay. It happened once. After deleting the previous one, it will start pointing here. It
happened twice. After deleting the previous one, it will start pointing here. It happened
three times. Now, now, now, when you will do this, when you will have done it three times,
your curve will be equal to 500 on this."
    },
    {
      "start": 1694.19,
      "end": 1718.88,
      "text": "500 means 5th node or whatever its address is, your PREV will be 1 behind 400
here and see your reverse is done, your reverse is done, your reverse is done but your
complete linked list, let's see how your complete linked list is looking, let's see how your
complete linked list is looking, in the very beginning which is 10, we didn't touch 10, so 10
is pointing to 20,"
    },
    {
      "start": 1718.88,
      "end": 1747.97,
      "text": "20 is pointing at NULL. 20 is pointing at NULL. 20 is pointing at NULL, so
nothing. Okay. Let's see further. 30. 30 is pointing at 20. It is, right? Okay. 40. It is
pointing at 30. Yes. Okay. And 50 is alone. 40 is not pointing at 50, so your linked list has
become like this. Your linked list has become like this, but you didn't want this. This is a
very useless linked list. What has become of this? I mean, look. From here to here, it is
correct."
    },
    {
      "start": 1747.97,
      "end": 1777.81,
      "text": "40, 30, 20. Very good, very good, very good. But, you didn't need this. What
did you need? You needed this. What did you need? I will give you the link. You needed this,
if you don't remember. You needed this. In the middle, your reverse portion should be there.
We don't have any problem with that. What should be there in the middle? 40, 30, 20. After
that, the rest of the portion should be as it is. So, see, what changes you will get from
these upper links? How? When you will reverse normally,"
    },
    {
      "start": 1777.81,
      "end": 1807.31,
```

```
      "text": "What changes do you need to make? Look at this. 10 is pointing at what? At 40, not at 20. It was pointing at 20 at the top because we didn't change 10. So if you remove 10, if you remove 10, at which point do you point? In some way? 40. You have done it. 40 is pointing at 30 and above it is also pointing at 40 at 30. 30 at 20 and above it is also pointing at 30 at 20. But, it is pointing at 20 at 50 and above it is pointing at 20 at null because you had reversed it and you had nulled it. If,"
    },
    {
      "start": 1807.31,
      "end": 1836.72,
      "text": "We will remove 20 and point the next 20 to 50. And what is head? Your head should be 10. Now let's see how the linked list is made. First comes 10. Okay. After 10 comes 40. Very good. After 40 comes 30. Very good. After 30 comes 20. Very good. After 20 comes 50. Yes. Exactly same linked list. Exactly same linked list. So you will have to change some things. You will have to change some things. What? You will have to do some connections here and there. Let's see what you have to do."
    },
    {
      "start": 1836.88,
      "end": 1871.44,
      "text": "Let's see what we have to do. This makes this question hard, this makes this question hard. What do you have to do? See. 10, 20, 30, 40, 50. Okay. When you reverse, it will be like this. 20, sorry. When you reverse, your upper one will be reversed. 40, 30, 20. Okay. Your 10 is still here and 50 is here. These two were not reversed."
    },
    {
      "start": 1871.44,
      "end": 1901.3,
      "text": "So all the things between 10 and 50 have been reversed. Now where do you have to make a connection? One more thing, when you have reversed above, after reversing, your previous is equal to 40. Because 40 is your new head. Because if we reverse 20, 30, 40, 40 will become a new head. Your curve will be at 50. Your curve will be at 50. Okay. So what do we have to do? This 10, what is this 10? This 10,"
    },
    {
      "start": 1901.68,
      "end": 1928.82,
      "text": "This is the first node, which means it is just the first node to reverse. If you are told to reverse from 2, left is equal to 2, then this 10 is left minus 1, this is one node, which means it has not been reversed till here, it will be reversed after this. If we call this before, we call this before, assume that you have it. Okay, that's it. What you have to do is go from 10 to 140, which means"
    },
    {
      "start": 1929.01,
      "end": 1958.77,
      "text": "Before before matlab jis node ke pehle tak reverse nahi hua hai uske baad reverse shuru hua hai before dot next is equal to kya 40 40 kaha hai prev prev yeh hogaya yeh ban jayega baaki same hai aur aur aur 20 20 20 20 20 kaunsa node hai 20 kaunsa 20 toh kahi hai nahi abhi tak toh 20 kahi bhi nahi hai lekin 20 ka jo jana chahiye woh kaha pe jana chahiye 20 ka jo"
    },
    {
      "start": 1958.77,
      "end": 1987.44,
      "text": "things should go to where? things of 20 should go to 50 what was 20? 20 was your left node 20 was your left node on which you were standing and on which you started reversing so let's do this too let's give it a name that we call it last this is last because it can be taken out, it is last so what happens? last.next is equal to, where does it have to go? 50, where is 50 on current?"
    },
    {
      "start": 1987.6,
      "end": 2017.97,
      "text": "if you do these two things and after that do return head then you will get the answer your linked list is ready because you started from left and reversed 3 times after that you connected the other connections which were broken so your linked list can be made let's see how we will do it you must have understood what to do and how to do it"
    },
    {
      "start": 2017.97,
      "end": 2054.48,
      "text": "But if you say that we will keep this here, we will keep that there, we will get to know here, how will we get to know? Come, let's see how we will get to know. This is a very important question, okay? Okay. First of all, your linked list will look like this. 10, 20, 30, 40. Let's make a proper linked list of this. You won't like this, right? It is better to make a proper linked list. 1, 2, 3, 4, 5."
```

```
    },
    {
      "start": 2054.96,
      "end": 2085.62,
      "text": "20, 30, 40, 50. This is 100, 200, 300, 400, 500. This points to 200, this
points to 300, this points to 400, this points to 500, this doesn't point to anyone. Okay. You
are given the value of left, 1, sorry 2. The value of right is 4."
    },
    {
      "start": 2085.62,
      "end": 2116.59,
      "text": "Very good. Now there is an edge case, we will come to that later. First look at
the normal case, there is an edge case. Left is 2, right is 4. Means, you have to start from
the second node. Start from the second node. Reverse 3 times. Reverse 3 times. How is 3 times?
4 minus 2 plus 1. You have to do this. So you have reached the second node. How will you know
that? You have reached the second node."
    },
    {
      "start": 2116.59,
      "end": 2139.25,
      "text": "How will you know that? So, we will tell you how to know that. T is equal to
100. This is a temporary variable to traverse because if you traverse then only you will know.
And remember one thing. It is okay to reverse from the second node but you also needed this
node. You also needed this node. You also needed this node. What was this? This was the first
node."
    },
    {
      "start": 2139.6,
      "end": 2169.84,
      "text": "You have to reverse from the second note. But you also need this note. Why?
Because you will have to add this connection later. This is before. Before.next is equal to
previous. You will have to do this later. You need before. For this, what is coming in my mind.
 You might have a different algorithm in your mind. So see. If you know the first note. If you
know that at this time we are standing on first. At this time we are standing on first note.
So you will know that"
    },
    {
      "start": 2169.84,
      "end": 2204.37,
      "text": "The next node is the second node. What is the next node? It is the second node.
Or you can do it in another way that you take a before first. You take a before first. Before
is equal to null. Okay. Every time, every time, when t comes, every time when t is equal to
100 comes, what do you do? And take a position variable. What do you take? Position. Position
is equal to 0. Okay. Come. t is equal to 100. Let's take it from 1 because position is going
to start from 1."
    },
    {
      "start": 2204.37,
      "end": 2236.69,
      "text": "The position is going to start from 1. If t comes, what will we check? You can
check the position in the linked list. You can check whatever is coming in your mind. But what
to check? If equal equal to 2. Because left 2 is there. Position 2 means that you had to start
from here. You had to start from here. So you started from here. But until that 2 is not there.
 Until that 2 is not there. Suppose we don't write like this. Until that 2 is not there."
    },
    {
      "start": 2238.06,
      "end": 2268.85,
      "text": "Less than 2. Less than 2. Means that you have not reached 2 yet. So, what you
do? Before is equal to t. Before is equal to t. t is equal to t dot next. And continue. And
continue. What will happen with this? Till you will not reach left. Till you will not reach
left. Till then your before is equal to t will be there. What will be the benefit of this? Now,
 suppose here you write that when your position 2 is done. So, see. Earlier we see that when
not. Now, what is your position?"
    },
    {
      "start": 2268.85,
      "end": 2293.25,
      "text": "1. Is 1 less than 2? Yes, it is less than 2. So, what is your before? T. So,
what is your before? 100. What is your before? 100. Okay. T is equal to 200. Yes, sorry. One
thing is left. Now, you are using position. So, you will do that too. And pause. Plus plus.
Then continue. Then what will we do? Continue. Yes."
    },
    {
      "start": 2293.25,
      "end": 2323.09,
      "text": "Now see what will happen. In the beginning, pos1 was 1. Is 1 less than 2? Yes,
```

```
it is 1 less than 2. That means you haven't reached the left yet. You are the first one. So, the first one is before is equal to T parallel. So, before is equal to 100. T is equal to T dot next. So, T200 will come in the while loop. Pos++. So, is it pos? Now, next time when you will check if pos2 is less than 2. No, it is not less than 2. It has become pos2. Now, you need to start your reverse. Now, you need to start your reverse. And see,"
    },
    {
      "start": 2323.09,
      "end": 2351.31,
      "text": "magically or very importantly, when your t is equal to 200 happened here, at that time, you are here. At that time, who did you need? You needed this 10. Not in the last, but it will happen. And see, your before is 100. Because what were you doing? Until you were not reaching the left, you were doing before is equal to t. Meaning, before in a way, the t you are standing on, you will know that you have to reverse it from here. Your before will remain on the first node because you will need it later."
    },
    {
      "start": 2351.31,
      "end": 2379.98,
      "text": "Now you know that pos is equal to 2 means this one is false. This one is false means pos is equal to 2. Now you have to start doing. What to do? You have to start reversing. How do we reverse? How many times do you have to reverse from this node? 3 times. So first of all, a curve is equal to, on which the curve used to be? On the head. But you don't have to keep the head. You have to reverse from this node. This is your head. Curve is equal to 200. Means curve is equal to t. Prev is equal to null."
    },
    {
      "start": 2380.18,
      "end": 2410.61,
      "text": "And what will we do? While, yes, we will take out one int. Times is equal to what? Right minus left plus 1. This many times. So while times minus minus. Yes. After that, what we do reverse, we have to do that only. Next is equal to curve dot next. Curve dot next is equal to prev. Turn back. Bring prev to curve."
    },
    {
      "start": 2411.12,
      "end": 2442.3,
      "text": "and move the curve forward close this what will happen with this? see what will happen with this first of all, curve is equal to 200 your curve will be here first of all, this will rotate this will rotate and what will become? null so this will not happen anywhere after that, it happened once remember this once next time, what will happen? the one with 300 will rotate where will the one with 300 reach? at 200 at 200 this became 200 it happened twice, remember next time, what will happen?"
    },
    {
      "start": 2442.3,
      "end": 2472.9,
      "text": "400 will also rotate. Where will 400 rotate and reach? On 300. On this? On 300. So it reached here. It happened 3 times. 3 times happened. 3 times happened. But when you would have gone out of 3 times, where would your curve be? On 500. Curve is always ahead. Where would your PREV be? 400. Okay. Where would your curve be? Now your link list looks like this. Your link list looks like this. Now what do you need? You need to add some arrows. What arrows do you need to add?"
    },
    {
      "start": 2472.9,
      "end": 2502.21,
      "text": "10, on which should we point? on 40 means the link list node which was in last in the reverse part, it must have come first so 10 should be pointed on that, where is it? it is in prev, where is 10? it is in before, so after everything is done before.next ok first we will add the one over there, see add the one over there, now 2, the one over there 2 it is pointing on null, but where should it point in real? on 5, this"
    },
    {
      "start": 2502.21,
      "end": 2528.29,
      "text": "Where was the point? At 50 or 500. The later node, which didn't get reversed. There is a before node, after which it got reversed. There is an after node, before which it got reversed. So where is that node? It is in Kerr. Where is 200? It is in T."
    },
    {
      "start": 2528.29,
      "end": 2556.9,
      "text": "It came at 200, that's when we started reversing. So T will remain at 200, right? So what can we do? T dot next is equal to curve. Means, T dot next, 200's next, where will it go? At 500. At 500. Where? At 500. At 500. It went. I am not making an error because it will be a mess. After that, after that, one thing happened. One thing happened. Means, this
```

```
one joined. This one joined. Very good."
    },
    {
      "start": 2556.9,
      "end": 2583.26,
      "text": "Let's add this one. 10 to 40. Where will you get 10? Before. That's why we took
out before. That's why we took out before. Before. Dot next. Dot next is equal to where?
Previous. Yes. Before dot next is equal to previous. Before dot next is equal to previous.
What is previous? 400. 400. That's it. After that, your work is done. Your work is done. See.
It got reversed."
    },
    {
      "start": 2583.26,
      "end": 2616.46,
      "text": "The link list is connected here and there. No matter how big the link list is
in the future, there is no need for it. It is done. It has been reversed. Return. Now, let's
see how your link list is made. Let's see how your link list is made. First of all, it starts
with 10. So, first of all, it is 10. 10 goes to 400. So, it becomes 40. 40 goes to 300. So, it
becomes 30. 30 goes to 200. So, it becomes 20. 20 goes to 500. So, it becomes 50. See. It is
reversed. This middle part is reversed. Very good. So, you can write this. So, how can you
write this? Come, let's see."
    },
    {
      "start": 2616.46,
      "end": 2650.02,
      "text": "This is the dry run. Let's see the code. The code is also not that special.
First of all, you will get one head and you will have to return one head. If you don't
understand this, then please tell us in the comments. We will explain it again. It was a very
good question. So, node fun, you have got one head. You have got one head. Okay. There is an
edge case in this. Now we are saying again and again that there is an edge case. We will tell
you. If some people have thought about it, then it is a very good thing. First of all, if head
is equal to null, means the linked list is empty, then return null."
    },
    {
      "start": 2650.02,
      "end": 2684.85,
      "text": "You don't have to do anything. Very good. We had given one more easy case. If
left equal equal to right. So what will we do? We will reverse one. Nothing will happen to it.
Return head. Now your main program has started. So node t is equal to head. We will have to
take one before. In the beginning, take before as null. We will have to take one pause as well.
 What? For position. Because we will have to find out where to cut. We will take one int."
    },
    {
      "start": 2686.26,
      "end": 2720.03,
      "text": "Let's start our loop. While t not equal to null. First of all, what was the
check? If my boss is less than left. Means you have never reached that left. So what did you
do? Before is equal to t. t is equal to t dot next. Boss plus plus. Means you have not reached
there. So keep updating before and wait for reaching there. You have not reached left. So why
will you reverse? Very good."
    },
    {
      "start": 2720.03,
      "end": 2751.55,
      "text": "And continue. And continue. If it is not like this. If it is not like this.
Then while loop comes down. Below is its continuation. Below is its continuation. If it is not
like this. If your pause 2 has happened. Means. You have come till here. Means your pause is
equal to 2. Or. Pause is equal to left. Very good. Before is stored. Okay. So reverse yourself.
 Note star curve. Is equal to t. Check. Note star. Prev. Is equal to null."
    },
    {
      "start": 2751.55,
      "end": 2788.72,
      "text": "Same as it was reversed. And how many times will we do int? Times is equal to
right minus left plus 1. Right or left is given in input. While times minus minus. What did we
do? Node star next is equal to curve dot next. Because you had to go ahead. Curve dot next is
equal to prev. Very good. Prev is equal to curve. Very good."
    },
    {
      "start": 2788.75,
      "end": 2819.92,
      "text": "This while loop is done. It is reversed. You are still in that. It is reversed.
But what do you have to do now? You don't have to do anything. You have to add some kind of
strings. What strings did you add? See. First of all, t.next is equal to curr. Why? Because t
was standing at 200. t was standing at 200. Where does it have to go? It has to go to curr. It
has to go to 500. We had seen that this will work. So, wherever t was standing, first of all,
t.next is equal to curr. Okay."
```

```
    },
    {
      "start": 2819.92,
      "end": 2852.34,
      "text": "after that before.next is equal to prev before. is equal to prev and return
head while loop is closed and after that you also close the function because you don't want to
return anything so this is done now now this is done this will work very good this is the
correct program t.next is equal to what are you doing this is your before node and this is
your after node"
    },
    {
      "start": 2852.34,
      "end": 2883.87,
      "text": "In the middle, there was a reverse. Now, we are joining this wire and then this
wire. So, we are doing the same. Okay. But, but, but. What is the edge case in this? If by
mistake, you give left is equal to 1, then? If by mistake, you give left is equal to 1, then?
Now, what does left is equal to 1 mean? Suppose, it is like this. 1, 2, 3, 4. Suppose, left is
equal to 1 in this. Right is equal to, and this is like this. 1 is here. 2."
    },
    {
      "start": 2883.87,
      "end": 2915.66,
      "text": "3 4 right is equal to let's say 2 right is equal to 2 you have to reverse from
1 to 2 you have to reverse from 1 to 2 so when you will reverse it then it will look like this
when you will reverse it then it will look like this 2 it will start doing 1 point and the
rest will remain the same now now now at this time this is fine this was your t so you will do
t dot next is equal to curr your curr will remain here so this wire is connected but the next
wire you used to connect"
    },
    {
      "start": 2915.66,
      "end": 2935.52,
      "text": "before.next is equal to prev, when you will reverse it, then prev is equal to,
prev is equal to 2, you will try before.next is equal to prev, before where is it, before
where is it, before what was it called, the first part of the linked list which was not
reversed, but here there is no part which is not reversed, it is reversed from the beginning,
because left 1 is given,"
    },
    {
      "start": 2935.52,
      "end": 2962.67,
      "text": "Left 1 means that one part of the beginning has been reversed. What should have
been the head here? Prev. What should have been the head of the new linked list? Prev. Because
it has been reversed from the beginning. Earlier, why did we return the head? Because one part
of the beginning had not been reversed. One had not been reversed. Some part of the middle had
been reversed. Here, the part of the beginning has been reversed. So, on the head, it is prev.
And when you do this before.next is equal to prev, then what is before? Before is nothing.
Before will be null. Before will be null."
    },
    {
      "start": 2962.7,
      "end": 2991.34,
      "text": "Why will it be null? Because how did you make before? How did you check before?
If pos is less than left, how did pos start? Where did pos start? From 1. So as soon as you
start the program, is 1 less than 1? No, no, no. You will directly come here to reverse. And
where did you start before? From null. So before is null. You are saying null.next is equal to
prep. That's why you don't have to do this. You have to"
    },
    {
      "start": 2991.76,
      "end": 3017.62,
      "text": "t.next is equal to cot t, what was t? t was the node from which you started to
reverse, so that node automatically went to the last that node should point to the next node
where cur is remaining, so if cur is null then it will point to the next null, if cur is a
node then it will point to that, but but but but you have to take care of before, what you
have to take care of? see you have to check if"
    },
    {
      "start": 3020.14,
      "end": 3051.26,
      "text": "if before means if there is any value in before means if there is any part
which is not reversed then it is fine then it is fine before dot next is equal to pref and the
head will remain the same head because if one part is not reversed then its head will remain
normal so you have to return what you had to do head but if it is not like this if it is not
like this else there is no need of else it will return if it is not like this then it is fine
then nothing return"
    },
```

```
    {
      "start": 3051.26,
      "end": 3081.55,
      "text": "Because this will be your new head. This will be your new head. See. This will
be your new head. Take a very big link. 1, 2, 3, 4, 5, 6. Assume that someone tells you that
left is equal to 1. Right is equal to 4. This is your left. 2, 3, 4. We have to reverse them.
If we reverse them, it will be like this. 4, 3, 2, 1."
    },
    {
      "start": 3081.55,
      "end": 3111.1,
      "text": "and the ones here will remain the same 5 and 6 have to be kept the same first
of all what will you do? t, t will be here because you started reversing from here so t.next
is equal to curr so this will be correct but there is no before so what is the head of this
linked list? the head of this linked list is 4 where did we get 4 from? 4 will be on previous
we saw that 4 was on previous you will have to return previous here when there is no before
then return previous so this was a small edge case when? when left is equal to 1 how did we
know?"
    },
    {
      "start": 3111.1,
      "end": 3140.0,
      "text": "If there is something before, if there is no null before, then there will not
be left 1, because something will happen in the future, because the update will happen before.
If there is null before, then there will be left 1. So this is your small code. Try to code it
yourself. If you understand any other algorithm, you can do that too. This is not a hard and
fast rule, this is what you have to do. But do code, it was a very good question. It was a
complex question, not hard. I mean, okay, you can't say hard, it can be complex, right?"
    },
    {
      "start": 3140.0,
      "end": 3142.4,
      "text": "Thank you very much for coming. See you tomorrow."
    }
  ]
}
```


8. transactions/t_17.json


```
{
  "video_id": "7ztFXqo8nZo",
  "title": "My 4 Year DSA Learning JOURNEY | Mistakes | Striver Love Babbar Pepcoding Apna
College",
  "language": "en",
  "model": "large-v3",
  "duration": 861,
  "segments": [
    {
      "start": 0.0,
      "end": 27.06,
      "text": "If someone's way of studying DSA on the whole internet has been full of
mistakes, it has been very long, it has been full of many challenges, then it has been mine.
It was 2021, in which we had written the first line of code in C++, after that, Mtech, then
placement in startup, then Amazon's interview, then it didn't happen there, then 30-40 lakhs
in Akamai, so this whole journey has a lot of teachers involved in it, whoever you are,
brother, sister, sir, whoever you see, whoever the big people are,"
    },
    {
      "start": 27.06,
      "end": 56.78,
      "text": "Everyone is involved in this in phases. We have learnt from everyone. We have
learnt from everyone's mistakes. We have learnt from everyone what to do and what not to do.
And today when we can teach people, when people tell us that you teach very well, your way of
thinking is very good, we are not afraid of taking a live class that if someone asks then what
will we do. So this is a very long journey. And today we know a lot of things that we did not
know at that time. So let's listen to a story. So everything started as we said in 2021."
    },
    {
      "start": 56.78,
      "end": 77.06,
      "text": "In 2021, this video of Lovebubberji, complete DS Algo in 450 questions. So, out
of the 12-11 lakh people who watched this, we were one of them. So, at that time, we came to
know what is DSA? It is a company like Amazon. The app that we run is really a company. It
gets so much money. It gets 50 lakh rupees. Its office is so big. So,"
    },
    {
```

```
      "start": 77.06,
      "end": 101.65,
      "text": "We have also seen this video. After watching this video, the first thought that
comes to mind is that yes, it would have been fun to study DSA. Like nowadays people think
that it would have been fun to study AI. At that time, no one knew about DSA. College students
did not teach. Seniors did not know. Juniors did not know. So how would anyone know? No one
knew. So, although Lovebubber did not teach anything at that time. Lovebubber used to post
videos of the same type that Amazon's office is like this, clothes are like this, money is
like this, but"
    },
    {
      "start": 101.65,
      "end": 125.81,
      "text": "If they wouldn't have uploaded those videos, then a whole generation wouldn't
have started DSA. We are talking about a whole generation, not just one or two kids. Because
of them, lakhs of kids got inspired to study DSA. Thousands of teachers got inspired like me
to get them to study DSA or to do something or the other. So for the push to get them started,
we are very thankful to Labbabbar. We got to learn showmanship from this man. What is
showmanship?"
    },
    {
      "start": 125.81,
      "end": 150.9,
      "text": "How do you show your things? I mean, when Lovebubber came, he was the first
YouTuber of his kind who used to talk about this kind of things. Before that, there was no
free resource, no one knew, 1-2 courses used to run here and there, local, offline. Apart from
that, there was no other information that how is the placement done, is DSA asked or not, what
all is asked. For this thing, a whole generation should be thankful for Lovebubber. Anyway,"
    },
    {
      "start": 150.9,
      "end": 180.86,
      "text": "Now, we have seen this video, but in this video, as we have told again and
again, what is the problem, you see. The first question of this array is reverse array. Okay,
someone can make it if syntax comes. The second question is maximum minimum element of an
array. Okay. The third question is find kth max. Now, kth max contains priority cube. So, this
is not made by anyone. The next question is consist of only 0, 1, 2. Sort the array without
any sorting algorithm. This is a question of Dutch National Flag. It is a question of two
pointer. This is an obvious thing that will not be made by any child."
    },
    {
      "start": 180.86,
      "end": 197.73,
      "text": "So, here the students get demotivated and leave it. We also left it. After that,
 what did we feel? When my M.Tech started in 2021, we thought that first of all, we should
learn DSA. At that time, a course of GFG used to come, a self-paced course. At that time, its
price was around 3000 rupees. Today, it must have become very less. Today, it is available for
999 rupees."
    },
    {
      "start": 197.73,
      "end": 220.19,
      "text": "At that time, we took this course of 3000. And without understanding anything,
the video of the teacher, which Sandeep sir used to teach in the video, he used to ask to make
a question. So, you can see 438 of 1419. So, there were 1400 questions in this whole course.
But, we kept making, making, making. That year, if we couldn't understand anything, we copied
them. If we couldn't understand anything, we took it. Anyway, it was a brute force approach."
    },
    {
      "start": 220.19,
      "end": 249.47,
      "text": "that keep making questions, this was the hope that it will come at some point
but what happened is that 438 questions were made but if someone would have asked that if you
have made 438 then tell me the first question how did you make it so he had no idea, we had to
tell him that we have to see the notes, it flew away from his mind so what happened during
this what happened during this that at least my syntax got clear Sandeep sir should be given
this credit that my syntax got clear completely concept was not coming at all"
    },
    {
      "start": 249.47,
      "end": 271.47,
      "text": "Intuition was not built at all. But the fear of writing syntax, how to write
the code, how to write the vector, how to write the heap, how to write the string, how to
write the function in the string, everything became clear. If someone had told us to write
this, this, this, then we were in a state of writing it. So this credit goes to Sandeep sir.
We started this course in 2021, the whole 2021 went into it."
    },
```

```
    {
      "start": 271.47,
      "end": 293.73,
      "text": "After this, in 2022, we understood that there is no point in taking this course
forward because it is not possible. We are being questioned but we are not able to understand.
After that, we got a sir whose name was Sumit Malik. Sumit Malik used to run PEP coding. In
today's date, PEP coding is a part of Scalar Academy but at that time, Sumit sir"
    },
    {
      "start": 293.73,
      "end": 321.81,
      "text": "There is a special thing about Sumit sir that he knows DSA very well. He is a
very magnetic person. Magnetic means a very charismatic person. When you will say this, you
will enjoy listening to it. Even if you don't want to study DSA, you will feel that let's
listen to him. He is a very good speaker. His personality is very cheerful and bold. So, Sumit
sir was a very magnetic person. At this time, his course used to come out. His course was in
three parts. Level 1, Level 2, Level 3."
    },
    {
      "start": 321.81,
      "end": 346.86,
      "text": "They used to say that take 1200 questions or make them. 300 in level 1, 600 in
level 2 and 300 in level 3. What will happen if you make 1200 questions? Another good thing
about Sumit is that he never used to lie. He never used to say that you make 1200 questions so
that you can make 1200 one question. No. Their core proposition, selling point was that if you
make 1200 questions in any way, then"
    },
    {
      "start": 346.86,
      "end": 374.54,
      "text": "The question that will be asked in the interview, if you have already seen the
question, its probability will increase a lot. If you make 1200, then there is very little
chance that you will get to see a new question, because the question will come from this 1200.
Because there are so many questions in it. That year, the whole year of 2022, the year of
placement, we followed pep coding, made level 1, made level 2, made level 3. Till that time,
even at that time, I was not able to understand anything, that is, the intuition was not built,
 a new question was not made by itself, but"
    },
    {
      "start": 374.54,
      "end": 403.12,
      "text": "What used to happen was that there was a momentum, there was a rush to make a
question. That year, you can see my GFG profile. Even now, my rank of IIT Dhanbad is 23.
Meaning, 624 problems, 2168 scores. We passed out from IIT Dhanbad, it's been 2-2.5 years.
Till now, this is the 23rd rank. At that time, when we were in college, this was the rank 1.
In the entire IIT Dhanbad, we were the one who made the most questions. I don't know about the
questions, but the questions were made. Why were they made? Because Sumit sir's promise was
that"
    },
    {
      "start": 403.12,
      "end": 429.38,
      "text": "The questions that we will get asked in our course will be the same. And when
the interview happened, when the placement of IIT came, it happened exactly as Sumit sir said.
Okay? That's why we are saying that Sumit sir never made such a selling point that if you do
200 of mine, you will do the rest of the questions. No. His selling point was always that you
do 1200 of my questions and then you will get the question you saw in the interview. And that
happened. We did not make 1200 questions, we could make 624 out of it."
    },
    {
      "start": 429.38,
      "end": 455.57,
      "text": "But, but, but, what happened? The two questions that were asked in the
interview were the same two questions that we had already asked. By that time, my condition
was such that almost, almost, I had 400, 500, 600 questions that I had already asked. Meaning,
you will ask a question and I will tell you the answer. We didn't know why that concept was
used here, but we will write the whole code and the whole code will be run. So, by doing this,
my placement was placed in Sandvine. Sandvine was a start-up company. Very good thing."
    },
    {
      "start": 455.7,
      "end": 483.23,
      "text": "Now, we got a placement in Sandvine. Now we started working in Sandvine. It's
been a year since we started working. We forgot the entire DSA. Obviously, when you write the
question, you will forget the question. If you don't remember the film from 3 months ago, then
you will never remember the DSA question from a year ago. Right? After that, we came to
Sandvine. Now, we got a problem that we have to switch. To switch, we have to study DSA again.
```

```
Let's go. Now, I didn't want to do this pep coding level 1 and level 2 again. After that, we met Stryver."
    },
    {
      "start": 483.23,
      "end": 509.3,
      "text": "There is no need of any identification of Stryver, at least not from me. At that time, there were a lot of playlists of Stryver. One of them was A to Z DSA course. There was a DP series, a tree series, a lot of series. After that, we wanted to do the same thing again that by looking at the questions, we started solving random questions. We picked up a sheet, we picked up a video of Stryver, we started solving that question randomly. While doing this, 2024 came. Now, what happened in 2024?"
    },
    {
      "start": 509.3,
      "end": 532.53,
      "text": "In 2024, my interview with Amazon was scheduled. My interview with Amazon was scheduled in January or February this year. Till then, I had no intuition, no pattern knowledge, no thinking process was developed. Even at that time, my condition was that whatever question is made, remember it. If the same question comes, we will make it. In the meantime, somewhere from Instagram, a reel came."
    },
    {
      "start": 532.53,
      "end": 554.7,
      "text": "There was a reel, in reel it was said about patents, I don't know whose reel it was, Indian or foreign, I don't remember. Like you guys scroll the reel, we were also scrolling the reel. It was also written there that comment, you will get the patent sheet, we also commented, I got the patent sheet. After reading that patent sheet, we realized for the first time that the question we are asking, there is no need to ask."
    },
    {
      "start": 554.7,
      "end": 582.3,
      "text": "Every question follows a certain pattern. So whatever we understand today, the first time we read in that Github repo. So we read that sheet carefully. We read that sheet carefully for a week or two. After that, after that, a lot of things became clear to me that the 500 questions that are already remembered, there was no need to remember them. If we had read pattern-wise from the beginning, there was no need to remember anything. There was no need to remember 1000-2000 questions. After that, after that, we started this DSA journey again."
    },
    {
      "start": 582.3,
      "end": 611.87,
      "text": "Solved that pattern sheet again. Where? On the lead code. Solved that pattern sheet again. Solved almost 300-400 questions again. Okay? After that, after that, when Amazon's interview came very close, very close, when 5 days were left, 6 days were left, at that time, there is a portal of Striverji, there is a portal called Take You Forward, there is a course on it. The name of that course is DSA Quick Revision. So, DSA Quick Revision has 80 problems. We solved 80 problems in 3 days or in 3 or 4 days."
    },
    {
      "start": 611.87,
      "end": 637.58,
      "text": "You can see, my Amazon interview was going to happen in March. So from here, we started from March 21, 15, 10, 16, 19, 13, 10, 34, means we solved 30-30 problems in a day. So during this whole week, we solved the problem while applying pattern knowledge. How many? Approximately 100-200 problems, I don't remember how many."
    },
    {
      "start": 637.71,
      "end": 666.67,
      "text": "Then Amazon's interview happened. People will say that Amazon's interview didn't happen. So you don't know. It wasn't like that. Amazon's interview happened. The first round went very well. In the first round, 100% success rate. The second round Amazon's schedule happened. But that interview never happened. Now you can see. This is a revised confirmation email. Now it will be 8th April. But first it happened in March. Then it was rescheduled. Then it came in April. And this interview never happened. Till now, till June 10, 2025,"
    },
    {
      "start": 666.67,
      "end": 693.87,
      "text": "They have the same message that we have recruited you, they will do something for you. But now it doesn't seem like it will happen. What happened after this? After this, Akamai's interview came. Akamai's interview came in September, August or September. By that
```

```
time, my pattern knowledge, my thinking process, my intuition had developed so much that there was no problem in Akamai's interview. There were two dsa rounds in Akamai, four questions were asked in both the dsa rounds. Four questions."
    },
    {
      "start": 693.87,
      "end": 720.26,
      "text": "We got to know what to do with a 100% success rate on HackerRank. This is how we are sitting today. After that, when we came to Makamai, we felt that what we have learned in 4 years, what took us 4 years to learn, that we are making so many struggles, tears, questions, forgetting, crying, shouting, this thing should not take you 4 years. Because if it took us 4 years, then we want that it should not take any other kid 4 years. So this is the purpose of starting this whole channel."
    },
    {
      "start": 720.26,
      "end": 746.93,
      "text": "So it is not that we are talking against or against all these teachers. Whatever we are today is because of the teachers. We have studied something or the other from every teacher. And we have studied very well and very minutely. I have a quality that even if we don't know anything, I have a consistency. We do that work continuously. We don't leave it in the middle. So because we were able to be consistent by the grace of God, we have passed this."
    },
    {
      "start": 746.93,
      "end": 773.76,
      "text": "If a normal kid asks 400 questions and then he thinks that we have forgotten everything, then he does not have the courage to start again. That is why this process is a brute force process of learning. Do not study with brute force. Study pattern wise. If you are a new kid and are watching till here, then there is a playlist. In the playlist, there is this playlist of DSA pattern. Follow it completely. In a very short time, you will learn to study DSA in a very optimized way. And after this, you will not need to write any question. Because writing a question..."
    },
    {
      "start": 773.76,
      "end": 800.51,
      "text": "Sumit sir used to say that if you want to do it, then do it. If you don't do it, then you will leave Microsoft. But to do 1200 questions, you need a lot of hard work, a lot of time, and a lot of hard work. So, Sumit sir also used to say that if you want, you can do 1200 questions. We also give you a guarantee that if you do 1200-1000 questions, then there will be no question that will not be made by you in the interview. If you do it completely. But,"
    },
    {
      "start": 800.51,
      "end": 828.96,
      "text": "Obviously, you don't want to write. If you don't want to write and you don't have much time, then DSA pattern is the only way to go. Follow the DSA pattern. If you watch one or two lectures of Sliding Window, you will understand what we are talking about and how we are teaching. Thank you and today's video will come soon. So, we thought that we should tell you my journey because you should know what is my brother's background, what is his qualification. We are not a candidate master or anything. You can see on the code shape."
    },
    {
      "start": 828.96,
      "end": 854.38,
      "text": "I might have participated in 17 contests in M.Tech. The highest rating is 1818. 1818 means 4 stars. So, it's okay. I did the course at that time. I didn't understand it. I left it. But it's okay. So, this was my background. This is my educational qualification, DSA qualification. Whatever you want to call it. So, let's meet in the next video. Thank you very much."
    }
  ]
}
```


9. transactions/t_18.json


```
{
  "video_id": "91qkzhmEO48",
  "title": "I Will make Binary Search Easy For You | DSA Patterns Course | C++ Java Python",
  "language": "en",
  "model": "large-v3",
  "duration": 1605,
  "segments": [
```

```
    {
      "start": 0.0,
      "end": 25.98,
      "text": "Before studying, read this comment, which we have been shouting for so many
days. At least one person has understood it. The rest must have also come. We have seen its
comment, so we will tell you. We have been trying to explain this thing to you for many days.
That you have to learn to think from me. Don't stay behind code and notes. We have seen many
children that till now while studying DSA, he does not have a copy. He neither has a copy nor
a pen. We had seen our friends in college that"
    },
    {
      "start": 25.98,
      "end": 40.3,
      "text": "I used to open the laptop and look at the screen for a long time. I used to ask
him, what are you doing? He used to say, I am reading DSA. I said, are you reading DSA? Where
is your copy? Where is the pen? He said, no, no, we will look at the question and if we
understand it, we will code it. Otherwise, we don't do it."
    },
    {
      "start": 40.3,
      "end": 58.35,
      "text": "this kid was studying DSA even after 5 years, he is still studying DSA, he
didn't get it nor will he ever get it so don't look at DSA in such a way, nothing is coming,
just take a copy pen, you watch all these videos, right? just take a copy, take a pen, a
pencil, whatever you have, iPad, notes, whatever you have, just take it"
    },
    {
      "start": 58.35,
      "end": 87.18,
      "text": "When we used to study, we used to watch someone's video or a video of a striver.
 We used to watch someone's video. We used to copy that this is a class, this is not a video.
Look, we are not that beautiful to look at. We are not heroes that you come to see my face.
This is a type of study, right? So like you study in school, you study in college, this is
also a type of study. So take it with you so that you can write something in it. First of all,
the child comes and starts searching for where to get the code. There is a very good website
to find the code. Its name is chatgpt.com. There you will get the code of every type."
    },
    {
      "start": 87.18,
      "end": 110.82,
      "text": "Where are the notes? You make the notes. My notes are cheating. My code is
cheating. What will happen? Learn from us how to think. We will tell you something. Think by
yourself. Attach it to your mind. Once you understand, try to code it yourself. You will learn
by doing this. My cheating and stealing will not be of any use. Let's start. Today's question
is minimum in sorted array. Rotated sorted array."
    },
    {
      "start": 110.82,
      "end": 137.68,
      "text": "Leetcode 153. So we will encourage that if you are watching the previous 2-3
videos, then you should definitely try it once. Pause the video before moving ahead. Go and
read the questions and try it. And the new students, check the description. And the students
who are still here, either you have played it or you will not think about it. Come, let's
start. We will explain the questions to you. First of all, we will understand the questions.
After that, we will see why it is a question of binary search."
    },
    {
      "start": 137.68,
      "end": 164.85,
      "text": "After that, we will see how it is related to what we were studying till now.
And then when these three things will be done, then you will know that the question is of one
minute. The question is not more than one minute. If you understand everything till now, then
the question will come to you in one or two minutes. You will come to do the code, you will
come to write, you will come to submit, everything will be done. Let's go. You have been told
that you have been given a sorted array. You have been given a sorted array. Okay. You have to
find its minimum."
    },
    {
      "start": 164.85,
      "end": 192.26,
      "text": "Find the minimum number, not the index. If we find the index, we will get the
number. We have to find the minimum number. We have to return or find. Some people will say
that this is stupid. The minimum number of the sorted array is in the beginning. The minimum
number of the sorted array is the array of 0. So we will return it. Very good, return it. But
what is the twist in this? This array is not only sorted, it is also rotated. What does this
mean? Now,"
    },
```

```
    {
      "start": 192.26,
      "end": 218.11,
      "text": "Rotation, what does rotation mean? Remember when we did the question of link
list rotation, what used to happen? If you used to say that we are rotating something k times,
then it used to happen that suppose 5 people used to stand in the link list, 5 people used to
stand and we used to say that k is equal to 2 times rotated k. So what used to happen by
rotating once? That this person, the first person used to come to the second position. The
second person used to come to the third. The third person used to come to the fourth."
    },
    {
      "start": 218.11,
      "end": 242.7,
      "text": "4th person used to come on 5th position and 5th person used to come on 1st
position because 6th position was not left so this used to happen this used to happen this
used to happen 1,2,3,4,5 after rotation this used to happen 5,1,2,3,4 one more time because
this was one rotation this k is equal to 1 times of rotate if we had done one more rotation
then we had to repeat the same thing in which this 1st person used to come here"
    },
    {
      "start": 242.7,
      "end": 272.88,
      "text": "This one used to come here, this one used to come here, this one used to come
here, and the last one didn't have any space left, so it used to run in the beginning, so it
used to come here. Come on. It used to happen that you can say that this is two times rotated.
Same thing is done in this array, same thing is done in this array, but, but, you are not told
the value of k. You don't know the value of k, because if you know the value of k, then it
will be very easy. You don't know the value of k. And one more thing, you don't know if the
array is rotated or not."
    },
    {
      "start": 272.88,
      "end": 299.04,
      "text": "Is the array rotated? It is not even certain. It means that it is not even
certain. It can be rotated. It can not be rotated. It can be rotated. It can not be rotated.
It can be anything. It means anything in the world. For example. See. Now slowly. Now come to
this question. Let's think. Now till here you understood the question. That you have been
given a rotated sorted array. You have not been told rotation. It is said possible. It can be
possible that it is rotated. You have to find its minimum element."
    },
    {
      "start": 299.04,
      "end": 324.46,
      "text": "So see, how to think of any question, how to think of any thing, that's why we
say, here comes the pen copy that we were talking about, right? That's why we are not saying
that we are selling pen copy, no. What happens with pen copy is that at least, suppose you
read the question, you read the question like this, read it 3-4 times in English, you don't
understand anything, you are looking at it, looking at it, after 15 minutes, you message on
Instagram that your course is useless, we see the 15 minute question, we don't understand
anything, no."
    },
    {
      "start": 324.78,
      "end": 354.29,
      "text": "If you don't understand anything, then you must know how to make a drawing.
Everyone knows how to make a drawing. Everyone knows how to make an array drawing. Make an
array drawing. Write down the input. Do indexing. See the output. If you do something, then
you will be stunned that after 1-2 months, how much this thing is going to benefit you. Come.
Assume that you have an array like this. 1, 2, 3, 4, 5, 6, and 7. So this is a sorted array.
This is a sorted array. Okay. And this is 6."
    },
    {
      "start": 354.29,
      "end": 383.26,
      "text": "Now, if it is not rotated, then what is its minimum? Array of 0, which you all
know very well. Where will be the minimum of sorted? In the beginning. Okay. But he is saying
that it is possible that it is rotated. It is possible, isn't it? It is possible. Assume that
we rotate it 3 times. It will give you after rotating. The question will give you after
rotating. Okay. Assume that if you rotate it 3 times, then what will happen? See. We will do
it 3 times, right? So, first of all, this will happen in rotation. 7, after that 1, 2, 3, 4, 5,
 6."
    },
    {
      "start": 383.26,
      "end": 411.06,
      "text": "what happens in every rotation, one last element comes first in every rotation,
one last element comes first and the rest of the things shift one by one, this happens in
```

```
every rotation, you can do it and see this happened once, if we do it twice, then 6, 7, 1, 2, 3, 4, 5 this happened twice, do it one more time take 5 here, 5, 6, 7, 1, 2, 3, 4 this area will be given to you, this area will be given to you as input now I have not told you from before that it is rotated or not, this is the input"
    },
    {
      "start": 411.06,
      "end": 437.65,
      "text": "You have been told that this is a sorted error, which may be rotated. You have to find its minimum, which is this. So what happened? We have made a drawing of the question, we have seen the question, we have understood, everyone knows this much, it is easy to do this much. Now think about it, what to do now? See, now those who are thinking, pause, as soon as it clicks on you, that yes, it has come to the mind, pause the video as it is, go and see it by coding it."
    },
    {
      "start": 437.65,
      "end": 464.29,
      "text": "now suppose you will be given an array like this so see carefully how this array will look like this array will be given like this that 5, 6, 7, 1, 2, 3 and 4 1, 2, 3, 4, 5, 6 this is your array now see carefully what you have to find you have to find the minimum you have to find the minimum ok now this array"
    },
    {
      "start": 464.29,
      "end": 492.82,
      "text": "How was this array before? We are all assuming that you don't know anything about k. How was this array before? This array used to look like this. Let's not make it an array. Let's just write the number. How did this array look originally? This array looked like this. 1, 2, 3, 4, 5, 6, 7. Now if someone rotates it by k times, what happens to it? Rotating it by k times means that the last k elements will go to the beginning."
    },
    {
      "start": 492.82,
      "end": 521.94,
      "text": "And in the beginning, n minus k elements, n is equal to 7, k is equal to 3, means the last 3 elements will go to the beginning. And the beginning n minus k elements, means the beginning 4 elements will go to the end. So, both of them reverse their position in a way. So, they become like this. 5, 6, 7, it comes here."
    },
    {
      "start": 521.94,
      "end": 545.57,
      "text": "And 1, 2, 3, 4. This comes here. This is what happens. This goes here. And this goes here. This is what rotation means. This is what rotation means. Now, now, now, look very carefully. Now, what is this? These are two parts. What is this? These are two parts. This is your part 1. This is your part 2. If we call an array,"
    },
    {
      "start": 545.57,
      "end": 573.9,
      "text": "that according to k, divide it into two parts. Keep the n minus k of the beginning in one part. Keep the last k in one part. So, what is happening is that part 1 is coming here instead of part 2. And part 2 is coming in the beginning. So, this is happening. There are two parts in the array. This is it. Okay. If, if, if someone tells you, now see, what is the whole game? The whole game is of guessing. Guessing. What will we guess? Index. Now, people will say that we have to return the number in this, so why are you guessing the index?"
    },
    {
      "start": 573.9,
      "end": 602.58,
      "text": "We are guessing the index because if you guessed the index, then you have to do the return array of index. That's the only difference. Okay. If someone tells you, let's say you guessed an index, let's say you guessed an index, let's say you guessed an x index. If someone tells you, if someone tells you that that x belongs to part 1 or part 2, then what will happen? Let's think. Let's think. Look."
    },
    {
      "start": 602.58,
      "end": 618.98,
      "text": "Now open your eyes, slowly this question will open, slowly you will understand that whatever you were studying for the last 2 days is interrelated, everything is related. See, how was Eric in the beginning, without rotation, how was he? 1, 2, 3, 4, 5, 6, 7."
    },
    {
      "start": 618.98,
```

```
      "end": 645.14,
      "text": "k can be anything. Let's assume that k is 3. Let's assume that k is 3 just to
explain to you. So what do we say? What does k 3 mean? The last 3 elements make a separate
part. The beginning n minus k makes a separate part. And what does rotation mean? The elements
of part 2, part 2 comes first and part 1 comes later. So look, if we look at every element,
every element of array either belongs to part 1 or part 2."
    },
    {
      "start": 645.14,
      "end": 674.51,
      "text": "In part 1, there is n-k. In part 2, there is k. So, every element will be
either part 1 or part 2. So, in the beginning, see. In the beginning, see. Which part is 1? 1
is part 1. Yes. 2 is part 1. 2 is part 1. 3 is also part 1. And 4 is also part 1. Because in
the beginning, n-k elements, we say that this is the first part, which we will take in the
last. Okay. But, whose is 5? 5 is part 2. Whose is 6? 6 is part 2."
    },
    {
      "start": 674.51,
      "end": 704.53,
      "text": "Yes. Whose is 7? 7 is of part 2. Every element is a part. Every element is in
the same part. Now you must be thinking that we used to say monotonicity. That the answer of
your guess will be the same. Now see. Now when you rotate. This was the original area. This
was the original area. When you rotate, how will the area look? The area will look like this.
5, 6, 7. After that, 1, 2, 3, 4. Let's write the part again. Let's write the part again. Which
was 5?"
    },
    {
      "start": 704.53,
      "end": 731.49,
      "text": "5 was of part 2. 6 was of part 2. 7 was of part 2. And 1 was of part 1. 2 was
of part 1. 3 was of part 1. 4 was of part 1. Now, what is your answer? Your answer is this.
What is this? Part 1's first occurrence."
    },
    {
      "start": 731.49,
      "end": 747.58,
      "text": "Look carefully. This is the first occurrence of part 1. Yes. This is the first
occurrence of part 1. Because your array looks like this. When your array was sorted, you said
that in the beginning, n-k was part 1. So, you got to know that the first 4 things are part 1."
    },
    {
      "start": 747.58,
      "end": 773.47,
      "text": "And the last three things are part 2. When you rotate the pair, all the
elements of part 2 come here. So now the rest of the beginning is part 2. And the last four
are part 1. What do you want in the answer? What do you want in the answer? You want 1 in the
answer. Means you want the first occurrence of part 1. So now if you guess something."
    },
    {
      "start": 773.47,
      "end": 791.89,
      "text": "Now, if you magically get to know whether this is part 1 or part 2, let's say
you come here, you get to know that this is part 2. If this is part 2, then where should you
go? Where will you get part 1? If you get part 1 on the right, then you should go on the right.
 If you guessed something and got part 1, then what would you have done in the first
occurrence?"
    },
    {
      "start": 791.89,
      "end": 820.43,
      "text": "we used to keep this, we used to keep this and we used to tell the shopkeeper
that now we will go to the left, we will go to the left and see if we find anyone else because
it is possible that the answer is there, it is possible that someone else is present on the
left, then we will take him, if we do not find anyone on the left in part 1, then we will take
you, then where did you have to go here, left and from here right, the whole question is
complete, there is a small problem, there is a small problem, this is part 1 or"
    },
    {
      "start": 820.43,
      "end": 843.3,
      "text": "Part 2, how will you know? How will you know? Because this is the main point of
binary search, that when you guess something, the answer of that guess, you should either know
in order of 1, or you should know in order of n. This used to happen, right? Okay. So,
everything is sorted. If your array is seen in one way, then you will get it in this way, that
part 2, part 2, part 2, part 2."
    },
    {
```

```
      "start": 843.3,
      "end": 852.78,
      "text": "Part 1 will start all at once and then all will be of part 1. So, how is the
answer of the guest? Monotonic."
    },
    {
      "start": 852.78,
      "end": 880.67,
      "text": "The answer is monotonic. It means that in the beginning of many parts 1,
suddenly part 2 comes and then part 1 comes continuously. There is no mix and match. It
doesn't happen that part 2 is there, part 1 is there and then part 2 comes. No, no, no.
Continuous part 2, continuous part 1 means monotonic. But there is a small problem that part 1
and part 2 are ours, right? Part 1 and part 2 are ours. How will we know?"
    },
    {
      "start": 880.67,
      "end": 911.34,
      "text": "Let's see how it will be known. One more thing, you have been told that array
is unique. Array is unique. What is array? It is unique. Okay. Now let's see what is part 1
and part 2? How is part 1 and part 2 known? See. This is your array. Suppose it is rotated
sorted array. So this is your 5, 6, 7, 1, 2, 3 and 4. 0, 1, 2, 3, 4, 5 and 6."
    },
    {
      "start": 911.34,
      "end": 943.28,
      "text": "This is your array, okay. How will we know? How will we know if this is part 2
or part 1? Now, what is part 2 in this? See, these three are part 2. This is P2, this is P2,
this is P2. And the rest are part 1. And the rest are part 1. Now, let's make a drawing of
this array. Make a drawing of this array. See how the drawing looks. This drawing looks like
this. First, 5 comes here. Then, sorry. Here, 5 comes somewhere. Then, 6 comes."
    },
    {
      "start": 943.28,
      "end": 972.21,
      "text": "Then 7 comes. By doing this, it climbs up. It suddenly falls and falls very low.
 Where? Straight on 1. It falls straight on 1. It falls very low. It falls very low on 1.
Let's make it lower. It falls very low on 1. Okay. After this, it climbs a little. After this,
it starts climbing. From 1, it goes to 2. From 2, it goes to 3. From 2, it goes to 4. Now see."
    },
    {
      "start": 976.3,
      "end": 1006.58,
      "text": "Pay attention to one thing. All the elements of part 1... Sorry, this is part 2.
 This is part 2. And this is part 1. This is part 1. This is part 1. All the elements of part
2 are more than all the elements of part 1. All the elements of part 2 are more than all the
elements of part 1. See. 5 is more than all the elements of part 1. 1, 2, 3, 4 are the most."
    },
    {
      "start": 1006.58,
      "end": 1035.92,
      "text": "6 is more than every element of part 1. 7 is more than every element of part 1.
And this had to happen. This had to happen. Because how did part 2 and part 1 become? Some
array was sorted. Some array was sorted till here. What did you do? You broke it from here.
You broke it from the middle. And you took this part and put it in the beginning. So now this
part obviously will be bigger than all the parts at the back. Sorted array means the same
thing. That your array was like this. 1, 2, 3, 4."
    },
    {
      "start": 1035.92,
      "end": 1065.23,
      "text": "5, 6, 7. You say that we will make a separate part of this and we will add it
in the beginning. The obvious thing is that if you make a part of the last things, then if
this is your part 2 and this is your part 1, then the obvious thing is that every element of
part 2 will be bigger than your part 1 element. Because it is a sorted array and you are
taking it there. So see. Now see. Slowly open your mind and think with me. This is your array.
5, 6, 7. 1, 2."
    },
    {
      "start": 1065.23,
      "end": 1092.85,
      "text": "3, 4. What is your problem? If you will guess an index, then someone will tell
you that is it part 2 or part 1. So we are trying to find something which is common in part 2.
Which is common in part 2. Let's see what is common in part 2. 5. It is bigger than every
element of part 1. So if it is bigger than every element, then this index is 0, 1, 2, 3, 4, 5,
6. Means,"
    },
    {
```

```
      "start": 1092.85,
      "end": 1118.9,
      "text": "5 is bigger than every element of part 1. Who is the biggest element of part 1? Part 1's biggest in part 1. Who is it? 4. Isn't it 4? The last one. The last one. The last index one. Array of 6. Array of 6. It's a natural thing that part 1's biggest is 4. Look. 5 is bigger than 4. Yes. 6 is bigger than 4. Yes."
    },
    {
      "start": 1118.9,
      "end": 1148.59,
      "text": "7 is bigger than 4. But... But... Now come to part 1. 1 is not bigger than 4. 2 is not bigger than 4. 3 is not bigger than 4. And... 4 is also not bigger than 4. So we got it. We got it. Look, look, look. Yes, yes. We got a trick. 5, 6, 7. 1, 2, 3, 4. 0, 1, 2, 3, 4, 5, 6. If... If..."
    },
    {
      "start": 1148.59,
      "end": 1169.36,
      "text": "Array of guess, if array of guess is larger than array of n-1, then this is part 2. Else, this is part 1. See, it will be in all. Array of guess, if we see 5, 5 is greater than 1, part 2. 6 is greater than 4, part 2. 7 is greater than 4, part 2. But,"
    },
    {
      "start": 1169.36,
      "end": 1198.67,
      "text": "1 is not greater than 4, part 1. 2 is not greater than 4, part 1. 3 is not greater than 4, part 1. And 4 is not greater than 4, part 1. So, this is also monotonicity. This is a strange monotonicity. Part 2, part 1, we had seen. This is also monotonic. What is monotonic? If you ask a question, if you ask a question, that what? Array of guess is greater than array of n minus 1. So, you got an answer like this. Yes. Yes. Yes. No. No."
    },
    {
      "start": 1198.67,
      "end": 1214.96,
      "text": "no, no, see it's monotonic, yes, yes, yes, turning point is suddenly no, no, no, no, no, and in a way you can say that where there will be no for the first time, where there will be no for the first time, this will be your minimum element, this same thing we understood from part 1 and part 2, this is P2,"
    },
    {
      "start": 1214.96,
      "end": 1243.12,
      "text": "This is P2, this is P2, this is P1, this is P1, this is P1, this is P1. Because how do we know P2? If the area of gas is greater than the area of n-1, then it will be P2. If the area of gas is not greater than n-1, then it will be P1. So, by doing this, we got to know. Let's see a small dry run of this. We are sure that this will be clear. If it is not clear, then tell us and we will explain again. That is why we are walking slowly because now we don't want to do what we used to do earlier."
    },
    {
      "start": 1243.12,
      "end": 1272.13,
      "text": "So earlier you used to say that explain everything in one question, make 5-6 questions in one question. What happens is that the 5th question gets spoiled. This is your array. You don't know what it is. No problem. Let's guess. Low is equal to 0. High is equal to 4. What is the initial guess of both? The mid value of 0 plus 4 by 2 is 2. So you came here. Okay. Now, is this part 1 or part 2? How will you know? Array of guess."
    },
    {
      "start": 1272.13,
      "end": 1299.63,
      "text": "Array of guess greater than array of n-1 means, what array of 2 is greater than array of 4? What 5 is greater than 2? Yes, yes, means this is part 2, means this is part 2. Remember carefully, in which format was your array? Your array was like this, p2, p2, p2, p2, p2, then p1, p1, p1, p1, means continuously p2, then continuously p1. So, if you are in part 2, if you are in part 2,"
    },
    {
      "start": 1299.63,
      "end": 1316.38,
      "text": "So where should you look? You should look at the right. How do we look at the right? By doing low is equal to guess plus 1. So what will be the next low? Is equal to 3. High is equal to 4. What will be its guess? 3 plus 4 by 2. 3. 3. Come. Coming."
    },
    {
      "start": 1316.38,
      "end": 1344.75,
```

```
      "text": "What? What? What? Array of gas is greater than array of n minus 1. What? 1 is greater than 2. No. No. No means this is part 1. If this is part 1, then what did you do? If this is part 1, then what did you do? You would have written it in the answer. Write the guess. And look to the left to see if there is anything else. First occurrence. Remember what happened in the first occurrence. Explain it very nicely. First occurrence. Please look at it. What was there in the first occurrence? We will look to the left. How do we look to the left? High is equal to gas minus 1."
    },
    {
      "start": 1344.75,
      "end": 1374.75,
      "text": "Low is equal to 3. High is equal to 2. Your loop is over because low is greater than high. What did you get in the answer? Rest is equal to 3. Means this index. If you return the array of 3, will you be able to return 1? Yes, you will be able to return 1. Yes, you will be able to return 1. One more thing. If the array is not rotated. If the array is rotated. It is possible that it is not rotated. In the question, it is possible that the array is like this."
    },
    {
      "start": 1374.75,
      "end": 1400.77,
      "text": "2, 3, 4, 5. 0, 1, 2, 3, 4. So, let's see if it works or not. So, what is the error? See, there is no error rotated in this. So, all the things in this are the same. P1, P1, P1, P1, P1, P1. There is no P2 in this. There is no P2 in this. So, let's see what is the error in that. Low is equal to 0. High is equal to 4. Gas is equal to 2. Come."
    },
    {
      "start": 1400.77,
      "end": 1429.5,
      "text": "Now see, what do we have to check? Is it P1 or P2? It may or may not be P2. What do we know? We have to check. Is the area of gas greater than the area of n-1? No. No means this is P1. But now what will we do? We will write this in the result and we will go to the left and see. So go to the left and see. Low is equal to 0. High is equal to gas minus 1. Now we have come here. Now what will be the gas? What will be the gas? Zero. Okay. We have come to zero."
    },
    {
      "start": 1429.5,
      "end": 1458.61,
      "text": "Now, what will we do? Now, what will we do? Now, what will we do? What? Array of guess is greater than array of n minus 1. No, 1 is not greater than 5. Means, this is also t1. What will we do? We will write this in the answer. And, we will go to the left. We will go to the left. Come, let's go to the left and see. Low is equal to 0. High is equal to guess minus 1. Minus 1. Your program is over. What is the result value? 0. What is array of 0? 1. Is this minimum? Yes, this is minimum. Yes, this is minimum. Come, let's code and show you."
    },
    {
      "start": 1459.15,
      "end": 1491.52,
      "text": "int at this point of time, you should really pause this video and do it yourself. After that, do it and then come and check what you did wrong. What do you have? Vector int this. Int n is equal to a dot size. Int low is equal to 0. High is equal to n minus 1. We will keep a variable for result which will be minus 1. Come, let's start."
    },
    {
      "start": 1491.52,
      "end": 1520.88,
      "text": "While low is not greater than high. Okay. I and T. Guess is equal to what? Low plus high by 2. Low plus high by 2. What do we have to check? Is it part 1 or part 2? If array of guess is greater than array of n minus 1, then what is this? This is part 2. Part 2 means, what was the array? Part 2, part 2, part 2, part 2."
    },
    {
      "start": 1520.88,
      "end": 1549.71,
      "text": "Part 1, part 1, part 1, part 1. So you need to go right. Low is equal to. Guess. Plus 1. Else. Else. Means this is part 1. Very good. Write this in answer. Store this in answer. But. Tell that shopkeeper that we want to see more. Because what occurrence are you looking for? First occurrence test. You want to find first occurrence. So. High is equal to. Guess. Minus 1. Close this. Close this. And."
    },
    {
      "start": 1549.71,
      "end": 1577.81,
      "text": "In the end, return array of rest. Don't do rest. It doesn't ask you for the minimum index. It only asks you for the number. In this case, return array of rest. So, this is your small code. Time complexity, login. Space complexity, order of 1. This is done. So,
```

```
this is your binary search code. Now, some people will say that we will do it with linear
search. It will be done with linear search too. You search minimum. So, that doesn't make
sense. Why do we have to do linear search? So, there is no point in reading this."
    },
    {
      "start": 1577.81,
      "end": 1604.08,
      "text": "So what was this question? This was the question of first occurrence. There was
a little trick that you have to think about part 2 and part 1. You can think about it in
various ways. Part 2 and part 1, we are telling you to understand that you have to understand
what monotonicity is. The day you start seeing this monotonicity, that day you will become a
binary search. So let's code this and see you in the next one."
    }
  ]
}
```


10. transactions/t_19.json


```
{
  "video_id": "93copdtW4ZQ",
  "title": "No one Has Explained Dijkstra's Algorithm Like This On The Internet!",
  "language": "en",
  "model": "large-v3",
  "duration": 3117,
  "segments": [
    {
      "start": 0.0,
      "end": 24.4,
      "text": "Welcome to Pratyush with Pratyush. My name is Pratyush Narayan. Today's topic
is very controversial. We will learn the Dijkstra algorithm from the beginning to the end. But
before teaching, let me tell you a few things very quickly. First, this is a little hard topic
in the graph. It is not hard. It is hard because it is a different algorithm. So if you do not
know the algorithm, you will not be able to make a question of it. That is true."
    },
    {
      "start": 24.4,
      "end": 44.29,
      "text": "It is not that you will think. No, Dixitra was a person, he thought. You will
have to learn that. The first thing is that it is a little hard topic. Sorry. The chance of
him coming in the interview is 2 by 5. That means, how much? 40% chance. I mean, all the
interviews we have given, Dixitra's question has not been asked to us till date. So see, you...
"
    },
    {
      "start": 44.29,
      "end": 73.84,
      "text": "If you want, you can skip the whole Dixitra, but the problem with this is that
there is not only one question in Dixitra or one lecture that you will skip. In the future, we
will see three to four graph questions like GPS, route of flight and many other questions. It
is a standard question. It uses Dixitra directly. So you should not leave it, but the chance
of coming to the interview is 40%. But now you are taking a lot of risk."
    },
    {
      "start": 73.84,
      "end": 101.92,
      "text": "If you got it in the interview and you were reading it, then you will make it
immediately. There will be no need to think about it. If you got it and you didn't read it by
chance, skip it. It will never be made by you in any birth. In any birth, you can't think to
yourself that this is Disclosure Algorithm. No, it can't be thought. So you take the risk
according to yourself. And before moving forward in this lecture, you should know 2-3 things.
Before today, we never said that you should know this. But before reading Dixitra, you should
know 2-3 very particular things."
    },
    {
      "start": 101.92,
      "end": 130.19,
      "text": "First of all, if you are watching this graph series, then in the first or
second lecture, in the second lecture, we talked about how adjacency is made into a list. How
adjacency is made into a list. From whom? From some edges. If edges were given an array, then
how adjacency is made into a list. But till now, we were making a list on an unweighted graph
or on a non-weighted graph. Pay attention, till now, whatever questions we had done, in 9-10
episodes, there were about 12-15 questions, right? In all of them, non-weighted things were
going on. In all of them, a non-weighted graph was being made."
    },
    {
      "start": 130.19,
```

```
      "end": 159.34,
      "text": "Dijkstra algorithm is not for non-weighted. Dijkstra is for weighted graph. Dijkstra's whole application is on weighted graph. So this is on weighted graph. So first of all, you should have seen the episode 2 of graph series. And how does it make you adjacency list? It should be almost almost written in your mind. First thing is this. Second thing, you should have seen the previous lecture of graph series, graph playlist. Shortest path. Shortest path in."
    },
    {
      "start": 159.34,
      "end": 186.48,
      "text": "Shortest path in non-weighted graph. Shortest path in non-weighted graph. You will have to watch this lecture. So that you get an intuition. What is shortest distance and what happens. This is the second thing. Third thing. You have priority Q. Priority Q. Or heap. Especially min heap. Min heap. If you are watching this pattern series. You know all this. If you are a new student. You know what is min heap. And what is min heap of pairs."
    },
    {
      "start": 186.48,
      "end": 210.19,
      "text": "This should be understood because these three questions will be used in it, these three concepts will be used in it. That is why we are saying that it is a little hard topic. You will get the link of all these three things in the description. You will get the link of all three points. After that, continue this. If you do not know these three things, then it will not be of much use. You will not understand much. Okay, let's move ahead. If you are still here, very good. Drink water in the future. A little long lecture is going to happen. Let's go."
    },
    {
      "start": 210.51,
      "end": 239.34,
      "text": "Now let's talk about the Dijkstra algorithm. Dijkstra algorithm is one of the most famous algorithms on this planet. If you are a computer science student, then it is not possible that you don't have to listen to Dijkstra in your college. What is the use of Dijkstra? Imagine you are given a graph. You have got a graph. What kind of graph is it? Graph is weighted. Weighted graph. Not in unweighted. There is no need in unweighted. We can do it. Unweighted means that every edge has a weight of 1. So that is it. We can do it. But specially, weighted graph is"
    },
    {
      "start": 239.79,
      "end": 269.68,
      "text": "It can be undirected or it can be directed. There is no problem in that. What does Dijkstra algorithm do? Suppose there are n nodes in the graph. How many nodes are there? N nodes. Means there are n circles. There will be some edge between them. And there will be some weight of every edge. Means your graph will look something like this. 0, 1, 2, 3. Your graph will look something like this. This is your graph. Suppose its weight is 2, 3, 4. This is 3 weight. You will be given one source. Source, node. Source, node. Anything."
    },
    {
      "start": 269.78,
      "end": 298.99,
      "text": "It can be anything. 0, 1, 2, 3, it can be anything. From this one source, from this single source, from this one source, all the rest of the nodes, all the nodes, have to find the shortest distance. All the nodes have to find the shortest distance. Meaning, if the source gives you 0, then from 0 to the rest of the nodes, what is the shortest path or shortest distance in which you have to find the shortest distance? That you have to"
    },
    {
      "start": 298.99,
      "end": 328.96,
      "text": "from the Dijkstra algorithm. If you have watched the last lecture, then what we did was the same thing. Shortest path in this graph. But it was non-weighted. It was non-weighted. It was the same. That you will get a source. From source to other nodes, the shortest distance has to be taken. But it was non-weighted. It means that the weight of every edge is 1. The weight of every edge is 1. You can apply Dijkstra in that too. It's not that you can't apply it in that. Non-weighted means that the weight of every edge is 1. Simple. But when the weighted graph comes here,"
    },
    {
      "start": 328.96,
      "end": 358.03,
      "text": "So whatever we had learnt in the previous lecture about BFS, that if there is zero, then push the neighbor of zero, then do this to the neighbor of that, this BFS will not work in that. This BFS will not work in that. Why will this BFS not work in that? We will tell you later. But Dijkstra mainly does this. If you get a weighted graph, whether it is directed or undirected, it does not matter. And you have to get the shortest distance from a single
```

```
source to all the other nodes, that means you have to make an array. What do you have to make?"
    },
    {
      "start": 358.03,
      "end": 382.64,
      "text": "0, 1, 2, 3 means how much to go from 0 to 0, how much to go from 0 to 1, how
much to go from 0 to 2, how much to go from 0 to 3. So what will you have to use? Dijkstra
algorithm solves this problem. Single source shortest path in weighted undirected or directed
graph. This is the use case of Dijkstra. Let's move ahead and understand it clearly. Let's
move ahead. Come."
    },
    {
      "start": 382.64,
      "end": 403.73,
      "text": "Now we told you that Dijkstra is the shortest path to every node from a single
source. Assume that you are an engineer in Google Maps. You have this map of India. Which five
cities of India have you connected to? Ranchi, Patna, Bangalore, Delhi, Kolkata. Now I have
also told you about the roads. These edges are direct roads."
    },
    {
      "start": 403.73,
      "end": 430.72,
      "text": "From Ranchi to Patna is one road, from Patna to Bangalore is one road, from
Delhi to Bangalore is one road, from Delhi to Kolkata is one road, from Kolkata to Bangalore
is not one road. So, it is fine. The road will be designed as it is. Very good. I have also
told you the distance of these roads. How many kilometers each road is. Now, it is not
unweighted that each road is one kilometer. No. Some roads are two kilometers. Some roads are
ten kilometers. It is possible. Now, what is given to you as a source? Ranchi. What do you
have to do? From Ranchi, from Ranchi,"
    },
    {
      "start": 430.72,
      "end": 460.05,
      "text": "find the shortest distance from Rachi to every city. Listen very carefully.
This Dixra algorithm tells you the route from Rachi to other cities. If someone asks how to go
from Patna to Bangalore, you don't know. How to reach every city from Rachi in the shortest?
This comes. How to reach Bangalore from Patna and Kolkata from Bangalore? This is not the work
of Dixra."
    },
    {
      "start": 460.05,
      "end": 489.07,
      "text": "From the single source to the rest of the notes. Now someone will ask, how do
you go from Delhi to Patna? No, for that you have to apply Dixit from Delhi. If you have
applied it from Ranchi, then only you will get the answer from Ranchi. It can be Patna from
Delhi, you may get the answer or you may not get it. Now see, how will we start this? Now we
will have to think about what Dixit must have thought, how he must have started, what his
thought process must have been. See, first of all, you have got the source, Ranchi. So, in the
beginning, you don't know anything. In the beginning, you are an engineer, you have no idea."
    },
    {
      "start": 489.07,
      "end": 518.74,
      "text": "In the beginning, we assume that the distance of each city is infinity. In the
beginning, you don't know anything, right? You don't know anything when you are starting. So,
how far is each city right now? You don't know. When you don't know and you want to find the
minimum, assume that it is infinity. Now, we don't know what infinity means. So, what do we
do? On Patna, infinity, infinity, infinity and infinity. Meaning, we don't know about these
four. We don't know if we haven't started yet. What is the distance of Ranchi? Is the distance
of Ranchi also infinity? No."
    },
    {
      "start": 518.74,
      "end": 547.98,
      "text": "When the source is Ranchi, then how far is it from Ranchi to reach the source?
Zero. The distance from Ranchi to Ranchi is zero. What will we do? Take this, zero. This is
your initial state. So you have written the distance on each city. You know the distance from
Ranchi to Ranchi is zero. If you don't know the distance from any other city, then the
distance from all other cities is infinity. Very good. Now we will do one thing. We will do
one thing. Look. Now we will do a trick which we call relaxation."
    },
    {
      "start": 548.02,
      "end": 573.95,
      "text": "Relaxation. Relaxation. What is relaxation? You pick up a city whose current
distance is minimum. Whose current distance is minimum. Which city do you pick up? Whose
present distance is minimum. Why is it minimum? You will find out later. Assume for 5 minutes
```

```
that we are"
    },
    {
      "start": 573.95,
      "end": 603.34,
      "text": "Out of all the cities, let's pick a city whose distance is minimum. So which
city's distance is minimum now? In the beginning, which city's distance is minimum? Obviously,
it will be the minimum of Karachi because the rest are all infinity. The rest are all infinity,
 right? So in the beginning, it is obvious that you will only get the source which is minimum.
Look, pick the source. What did you pick? Look, what did you find out in this? What did you
find out by looking at this note? You found out one thing that"
    },
    {
      "start": 603.6,
      "end": 633.84,
      "text": "The distance from Ranchi to Ranchi is 0. If you know that the distance from
Ranchi to Ranchi is 0, then tell me one thing. Patna and Delhi, which are the direct neighbors
of Ranchi, can you calculate their distance as well? Yes, you can."
    },
    {
      "start": 633.84,
      "end": 661.01,
      "text": "Yes, you can. Why? You got to know that your target is to get the distance from
sources to all the cities. Fair enough. You got to know that the distance from Ranchi to
Ranchi is zero. Meaning, you can get from Ranchi to Ranchi in zero kilometers. Very good. You
also know that because Patna and Delhi are direct neighbors of Ranchi. One road goes to Patna
and the other to Delhi. So, can you calculate the distance between Patna and Delhi? Yes, you
can."
    },
    {
      "start": 661.01,
      "end": 691.07,
      "text": "Can you calculate the distance from Ranchi to Bangalore now? No. Because there
is no direct road from Ranchi to Bangalore. There are two roads from Ranchi to Bangalore. One
road goes to Patna and the other one goes to Delhi. So can you calculate the distance between
Patna and Delhi? Absolutely. We call this thing relaxation. Whenever you catch a city, like we
are catching Ranchi right now, we can calculate the distance of its direct neighbors. Because
you get this distance. What is this distance? From source,"
    },
    {
      "start": 691.07,
      "end": 718.78,
      "text": "distance from Ranchi to this node is zero. So can we calculate the distance of
all the immediate neighbors of this node? Yes, we can. So what will we do? The first neighbor
of Ranchi is Patna. If the distance from Ranchi to Ranchi is zero, and the road from Ranchi to
Patna is 2 km long,"
    },
    {
      "start": 718.78,
      "end": 748.35,
      "text": "Patna. So how much time will it take for the source to reach Patna? The
distance of Ranchi is 0 plus the distance of the road from Ranchi to Patna which is 2 is equal
to 2. Calculated? Very good. Let's do the second one. What is the second one? Delhi. Apply the
same technique in Delhi. If it takes 0 to go from Ranchi to Ranchi and the direct road from
Ranchi to Delhi is 10 km long, then how much time will it take to go to Delhi? Whatever is 0
plus 10. How much?"
    },
    {
      "start": 748.35,
      "end": 777.92,
      "text": "0 plus 10 is equal to 10. Okay? Both are done. There is no other direct route
than Ranchi. Very good. I hope you understood this much. It's a very good thing. Let's move on.
 Now, now, now, look. First, first, when you started, how much distance is written in Patna?
Infinity. When you calculated, how much distance did you get? 2 km. How much distance was
written in Delhi? Infinity. When you calculated, how much distance did you get in Delhi? 10 km.
 Now, now,"
    },
    {
      "start": 777.92,
      "end": 807.1,
      "text": "Relax, you will do that note with the current answer. The answer that is
written now, now infinity is written in both. If you get a better answer than that, if you get
a better answer than that, we relax it and update the answer. What does it mean to say? You
used to think that it takes an infinity kilometer to go to Patna. Now you have come to know
after you relaxed, that there is no infinity in Patna. There are two in Patna, which is"
    },
    {
```

```
      "start": 807.1,
      "end": 836.13,
      "text": "Less than infinity. So what will you do? You will update the distance of Patna. You will update the distance of Patna. With whom will you update? With 2. Because you have got a new information. You have updated it. Similarly, you used to think that it takes an infinite kilometer to go to Delhi. Now you know that it is not like that. It takes only 10 kilometers to go to Delhi. Which is less than the current answer. So what should you do? You should update Delhi too."
    },
    {
      "start": 836.13,
      "end": 863.25,
      "text": "What should you write after updating the distance of Delhi? You should write 10. Very good. You must have understood this much. This is very simple. It is going very simple till now. Take out the distance. Calculate it. And if the distance is less than the current distance, then write it in the answer. Very good. You are understanding till now. Now let's move ahead. Now a little complex part is coming. You are understanding till now. Very good. Very good. You are going very well. 60% is done. Okay. Okay. So till now you understood that whom we had processed?"
    },
    {
      "start": 863.25,
      "end": 892.54,
      "text": "Ranshi 0 was processed and updated to Patna and Delhi. Everyone understood till here. Now let's see. This processing should not stop. Processing will continue. The process of picking up each city will continue. So see, how many cities do you have now? Assume that there are only 5 cities. We are not going through any of them. There are 5 cities. Okay. Ranshi has been processed. Now don't do Ranshi again and again. If you pick up Ranshi again and again, then the same thing will happen again. Now you have done Ranshi once. Now don't keep doing Ranshi again and again. It's not stupid."
    },
    {
      "start": 892.54,
      "end": 921.34,
      "text": "Separate Ranchi. How many cities are left? These two are infinity. Separate them too. What does infinity mean? It means that there is no information about it. Separate these two too. How many cities are left? The one we just updated. Patna. We know that the answer to go to Patna is 2 km. Delhi. We know the answer to go to Delhi. We know till now that it is 10 km. Okay. Remember that this is not the final answer. Till now. Till now. We have an idea after seeing Ranchi. Very good app. From these two, who should we process first?"
    },
    {
      "start": 921.34,
      "end": 947.68,
      "text": "You can do both. You can do both. You can do both. You can do both. You can do both. You can do both. You can do both. You can do both."
    },
    {
      "start": 947.68,
      "end": 975.86,
      "text": "If it is less than the current answer, then update the answer. This was the work. This work, same work. You can do it from Patna as well as from Delhi. But why are we saying that the answer of the city is small, the green value written in the city, that is a small value, process it first. Why are we saying this? What is its logic? Look, let's assume that you say that we don't agree with this, we don't agree with this, we will process Delhi first or we will process it in random order."
    },
    {
      "start": 975.86,
      "end": 1005.42,
      "text": "Do it. See what will be the problem. You come to Bangalore for some time and concentrate. If you had processed Patna first, whose current distance is 2 km, then you would have done the calculation of Bangalore because there is a direct road between Patna and Bangalore. So how much distance do you get to go to Bangalore? See, Patna says that there are 2 answers. What does this 2 mean? That the distance to go to Ranchi to Patna is 2. And there is a 4 km road between Patna to Bangalore. So how much distance will it take to go to Ranchi to Bangalore?"
    },
    {
      "start": 1005.42,
      "end": 1032.22,
      "text": "which is the distance to Patna plus this road to Bangalore. So, the distance to Bangalore is 6 km. If you were to process Patna, if you don't agree and start processing Delhi, then do the same thing. There is a direct road from Delhi to Bangalore which is 5 km long. It will take you 10 km to reach Delhi from Ranchi. There is a road from Delhi to Bangalore which is 5 km long. So, this route will take you 15 km to reach Bangalore from Ranchi."
    },
```

```
    {
      "start": 1032.22,
      "end": 1059.17,
      "text": "10 plus 5, 15 km. So you write Bangalore 15 km. Is the answer to Bangalore
really 15 km? No. The smallest route to Bangalore is this. You first go to Patna from Ranchi,
2 km. Go to Bangalore from Patna and reach 6 km. The smallest route to Bangalore is not this
one which you are doing now. That's why the first thing is that you are processing the larger
one. You are doing a useless work. See, you will get the answer from that."
    },
    {
      "start": 1059.17,
      "end": 1088.98,
      "text": "You first do Bangalore 15, then you do more work, then you process Patna and
then see that no, no, 6 is smaller than 15, then again. There is useless processing going on.
Second thing, do you have 10 optimal answers to reach Delhi? No. The answer to Delhi is also
wrong. You just have an idea. Now look at the graph. If we tell you that what is the shortest
route from Ranchi to Delhi? So will you go from this route which takes 10 kilometers? No. What
will you do?"
    },
    {
      "start": 1088.98,
      "end": 1113.86,
      "text": "If you want to go from Ranchi to Patna, 2 km. If you want to go from Patna to
Delhi, 3 km. So, the actual distance of Delhi is not 10 km. The actual distance of Delhi is 5
km. That's why, if you process Delhi now, then first you are processing at the wrong distance.
Which you don't know now. When will you know the distance of Delhi? When you process Patna.
When you process Patna, then what will happen?"
    },
    {
      "start": 1113.86,
      "end": 1133.04,
      "text": "From Patna to Delhi is the road. Patna's distance is 2. So what calculation
will Patna make of Delhi? That from Ranchi to Patna it seems like 2. 2 plus 3 is 5 going to
Delhi. Is 5 less than the current one? Yes, it is less than the current one. So do one thing.
Cut 10 and update 5. So the actual distance of Delhi, if you knew after Patna,"
    },
    {
      "start": 1133.04,
      "end": 1157.42,
      "text": "But you didn't listen. You said that we won't process Patna. We will process
Delhi first. So there are two mistakes. First of all, you took the wrong answer. You are
processing it at 10. And because of this, you took the wrong answer of Bangalore and Kolkata.
Wrong answer will come out, right? What will you get? Write 15 for Bangalore and 11 for this.
This doesn't have 11. This is all dependent on your correct knowledge of Delhi. But you made a
mistake. That's why."
    },
    {
      "start": 1157.46,
      "end": 1184.26,
      "text": "You don't have to do it again and again and again. That's why you process the
smallest one. Think of it in a logical way. If you are given two choices. If you are given two
choices. That look, there are two cities. Patna is 2 km away and Delhi is 10 km away. Now you
see by exploring. So logically think that where will a person go first? You are being told
that brother, it happened on Ranchi. Patna is 2 km away and Delhi is 10 km away."
    },
    {
      "start": 1184.26,
      "end": 1212.67,
      "text": "Do further processing. So no one will say that we will go 10 km away first. No.
People walk slowly. They walk step by step. You will go to Patna first. You will do processing
from Patna. After that, you will go to the distance from Patna. No one will say that we will
do 10 km first. No one does that. That's why it is called greedy algorithm. You are getting
greedy. For whom are you getting greedy? The city's distance will be the lowest."
    },
    {
      "start": 1212.67,
      "end": 1239.42,
      "text": "We will process it. We will process it. So who should you process? And why will
you do it? We have already explained it to you. That two mistakes. It will be asked in the
interview. Why are you doing it? It is possible. It will be asked. It is not so deep to ask.
But okay. We have told you to tell. Because we don't want you to remember it. There is no use
in remembering it. Don't ever remember it. That's why we do it. Because you don't have the
right information about Delhi. You are processing Delhi forcibly. Let Delhi's right
information come. Do Patna."
    },
    {
      "start": 1239.42,
```

```
      "end": 1268.7,
      "text": "It is possible that there is no such information about Patna. There can be
anything. But if Patna is 2 and Delhi is 10, then what is the probability of Patna being more
correct? What will happen if it becomes less than 2? Will it become 1? It has less probability.
 But the probability of it being 10 to 5 is very high. That is why we will process this. Patna
to Delhi. Delhi is in the pipeline. Delhi is sitting in the pipeline. Come. Who did you
process? Patna to. What is written on Patna? 2. What does this mean?"
    },
    {
      "start": 1268.7,
      "end": 1293.76,
      "text": "ki ki iska ek hi matlab hai ki ranchi se patna jaane me 2 km lagta hai same
relaxation kijiye same relaxation kijiye relaxation ka kya matlab hai patna se direct road
kahan kahan jaata hai toh dekhiye patna se pehla direct road toh ranchi jaata hai karke dekh
lijiye ab dekh lijiye patna se ranchi jaake"
    },
    {
      "start": 1293.76,
      "end": 1318.19,
      "text": "If you reach Patna in 2 kms from Patna to Ranchi, then you will have to answer
4 kms from Ranchi to go back to Ranchi. Should you answer from 0 to 4 kms from Ranchi? No. Are
we looking for the maximum part? No. The answer from Ranchi is already minimum 0 kms. So,
there is no benefit. This is nothing. This relaxation has failed. Next. There is a 4 kms road
from Patna to Bangalore."
    },
    {
      "start": 1318.19,
      "end": 1345.58,
      "text": "It takes 2 km from Ranchi to Patna. It takes 4 km from Patna to Bangalore.
Overall, how long will it take to go from Ranchi to Bangalore? 2 plus 4 which is 6. Is this 6
less than Bangalore? Is it less than infinity? Absolutely less. So what happened to the first
relaxation? What happened to the first relaxation? What did you write after updating
Bangalore? 6. Why did you write it? It takes 2 km to go to Patna."
    },
    {
      "start": 1345.58,
      "end": 1375.26,
      "text": "From Patna to Bangalore, it takes 4. From Ranchi to Bangalore, if we go this
route, how many kilometers will it take? 2 plus 4, which is 6 kilometers. Very good. One more
relaxation happened. One more, which we talked about in the previous video, Delhi. Delhi is
also its friend. Do the same thing. From Ranchi to Patna, it takes 2 kilometers, which is
written in its answer, which you are knowing here. From Patna to Delhi, it is a 3-kilometer
road. So, if we go this route from Ranchi to Delhi, how many kilometers will it take? 2 and 3.
2 and 3. 5 kilometers. What?"
    },
    {
      "start": 1375.26,
      "end": 1404.7,
      "text": "This 5 is less than the current answer of Delhi. Yes, it is less than the
current answer. Update Delhi, brother. Update Delhi. Update Delhi. What do you write? 5
kilometers. See, now Delhi is correct. Now Delhi is correct. Earlier Delhi was 10. You are
using it forcibly. Now Delhi is correct. Okay? Has Patna work been done? Yes, Patna work has
been done. Okay? Patna work has been done. We will not do it again. Okay? Now see. Now see.
Come again. Come again. Ranchi. Ranchi does not need to be processed again and again. Ranchi.
Ranchi did not relax, right?"
    },
    {
      "start": 1404.7,
      "end": 1433.78,
      "text": "Means, when will we process it? The original distance of Ranchi was 0. Now, if
Ranchi would have been minus 2, then we would not have done it. Next, which city do you see
that you should process? See, Patna 2. We have done it just now. We will not do Patna again
and again. Delhi 5, Bangalore 6. This is a poor infinity. We will not do it. Who should be
processed? Minimum one. Who should be processed? Minimum one. Why? I have told you. Who is
minimum?"
    },
    {
      "start": 1433.78,
      "end": 1461.95,
      "text": "Delhi 5. What does Delhi 5 tell you? Delhi 5 tells you that it takes 5 km from
Ranchi to Delhi. Now, how does it take 5 km from Ranchi to Delhi? I don't know. It could be
that you have come directly from Ranchi. It could be that you have travelled. I don't know.
But, the shortest path from Ranchi to Delhi is 5 km. Very good. Now, relax. Come. There are 4
roads from Delhi. Come, let's do it one by one. Try to relax Ranchi."
    },
    {
      "start": 1461.95,
```

```
      "end": 1491.02,
      "text": "Delhi is saying that it takes 5 to reach me and the road to Ranchi is 10 km. So
should we make it 15? No, Ranchi is already 0. If 15 were less, we would have updated. No need
to relax. No need to relax. No problem. Don't do it. Delhi to Patna is a road of 3 km. It says
that it takes 5 to reach me and it takes 3 to reach Patna. So should we make Patna 5 plus 3,
8? No, no need. Patna is already sitting on 2. Don't increase it more than 2. If you get less
value than 2, we will increase it. Not more than 2."
    },
    {
      "start": 1491.02,
      "end": 1517.57,
      "text": "This relaxation has also failed. Come to Bangalore. Delhi says that it takes 5
km to reach there. And there is another 5 km road from Delhi to Bangalore. So the total will
be 5 plus 5 is 10. Should we update Bangalore? No. 10 is very good. There is no need to update.
 Relaxation has failed. Don't do it at all. This also failed. Come to Kolkata. Delhi says that
it takes 5 km to reach there. There is a direct road of 1 km between Kolkata and me."
    },
    {
      "start": 1517.57,
      "end": 1546.58,
      "text": "So how much time will it take to go to Kolkata through me? 5 plus 1, 6. What is
6? It is less than this. It is absolutely less. Who should be updated? Kolkata. Who should be
updated? Kolkata. Relax Kolkata. Very good. Is Delhi's work over? Very good. It is over. Then
see. What all changes happened? What all changes happened? See. Now change. This also happened.
 This also happened. This did not happen. This happened last time. Now let's do them one by
one. This is also 6. This is also 6. Who will we do? Your wish. Your wish. Whoever you do."
    },
    {
      "start": 1546.58,
      "end": 1571.44,
      "text": "Try Bangalore. How many people are connected to Bangalore? Two people.
Bangalore says, it takes 6 km to reach me. Patna is a 4 km road between Patna and me. 6 plus 4,
 10. Should we update Patna? No. Patna is already 2. It failed. Same thing. 6 plus 5. He will
say, should we make Delhi 11? Delhi said, we don't need 11. It also failed. Bangalore is over.
Come to Kolkata. Kolkata says, it takes 6 km to reach me."
    },
    {
      "start": 1571.44,
      "end": 1600.96,
      "text": "My only direct neighbor is Delhi. So he will say, should we make Delhi 6 plus 1,
 7? Delhi says, we don't need it. We are already sitting comfortably on the path. This too,
relaxation failed. Now see, everything is over. Now you will repeat it as many times as you
want. Now nothing will change. We will tell you in the code and in the future we will tell you
what is the use of heap etc. But we are telling you very simply that now do anything. Take
someone and relax. Nothing is going to change. See what has come out. How much distance is
there from Ranchi to Ranchi?"
    },
    {
      "start": 1600.96,
      "end": 1630.43,
      "text": "Zero kilometer. Which should be there. How much is the shortest path from
Ranchi to Patna? Two kilometers. Which path is that? This path. Very good. Path from Ranchi to
Delhi? Five kilometers. How? Two, three. Ranchi to Patna, Patna to Delhi. Two plus three, five.
 Looking good. Distance from Ranchi to Bangalore? Six kilometers. Which road to take? Ranchi
to Patna, Patna to Bangalore. Very good. Distance from Ranchi to Kolkata? Six kilometers.
Which road to take? Ranchi to..."
    },
    {
      "start": 1630.43,
      "end": 1658.32,
      "text": "Delhi to Kolkata. The whole distance is set and this is your final answer. This
algorithm is called Dijkstra algorithm in Hindi. What is this algorithm called? Dijkstra
algorithm. Now let's move ahead. Code and heap and fee, whatever it is, now let's move on to
that. If we had told that earlier, then you would have taken a rut, it would not have been
useful. Now you have an understanding of what to do. Let's move ahead."
    },
    {
      "start": 1658.38,
      "end": 1686.88,
      "text": "So, we have reset everything. It is the same graph. Reset means that we have
deleted all the answers. First of all, we have made an array. Now, we are moving towards the
code. Slowly, we are moving towards the dry run because we had made a talk above. An interview
does not come out by making a talk. We will also show the code. Now, look at the dry run. This
is your array. What is the array? Ranchi, Patna, Bangalore, Delhi, Kolkata. Now, we know that
because our source is Ranchi, so we have written the distance of Ranchi to Ranchi as 0. We
have made the rest as infinity like the last time. Very good. Now,"
    },
```

```
    {
      "start": 1686.88,
      "end": 1714.22,
      "text": "What we were saying is, which city are we processing every time? Whose current distance, whose updated distance, whatever you call it, the current answer should be the minimum. Okay? So see, whose is the minimum? Ranchi's answer is zero for now, the rest is infinity. What we will do is, we will make a min-heap. That's why we said it is important to read the heap, because it contains heap. We will make a min-heap. How will we make a min-heap?"
    },
    {
      "start": 1714.48,
      "end": 1743.38,
      "text": "There will be two things in pair type. What will be the two things in it? First of all, there will be this distance in it. Distance means your answer. The answer that is written in the actual result. See this distance. This distance and the name of the city. The name of the city means the name of your integer. We are doing it to explain you the city. We will push this type of entry. Whenever we will update the distance of any city. Means we will relax it. We will get its good distance. We will get less distance."
    },
    {
      "start": 1743.38,
      "end": 1763.09,
      "text": "So we will push it in the heap so that it can be processed at some point. And what does mean heap guarantee? Because you are pushing a pair here. What pair are you pushing? What is the first of it? Distance means what is written now. And city means your node. That node can be anything. Integer, character, array, it doesn't matter."
    },
    {
      "start": 1763.12,
      "end": 1792.42,
      "text": "So, what will the min-heap give you at the top? What will it give you at the top? The one whose distance is the lowest now. So, to start, what will you do by pushing the infinity? There is no point in pushing the infinity. Push the ranchi. So, what will we push in our heap? Suppose this is your heap. Let's make a heap here. This is your heap. Okay? What is this? This is your min-heap. What did you push in this? Zero and ranchi. Zero and ranchi because it is the same in the beginning."
    },
    {
      "start": 1792.42,
      "end": 1819.98,
      "text": "Very good. Start processing. Until this heap is empty, we will process it. And you will see how you get the final result. See. First of all, you process. So what do you get? Zero and Ranchi. Because there is only one element at the top. So this will come out. Pop it with the heap. Because now its work is done. Okay. Relax this man. What do you do to this man? Relax. What does relax mean? Look at all the neighbors of this. Look at all the neighbors of this. Okay."
    },
    {
      "start": 1819.98,
      "end": 1849.3,
      "text": "For every neighbour, what do we have to see? What do we write in this? First of all, we write D is equal to 0. What do we have to see for every neighbour? If D plus weight, weight means the weight of that neighbour, the weight from Ranchi to Patna is 2, right? So if this D, which you have got from the heap, the distance from Ranchi to Ranchi is 0, plus weight, plus this weight is less than whatever is written in your distance array. Distance of neighbour, distance of neighbour means"
    },
    {
      "start": 1849.3,
      "end": 1877.2,
      "text": "If we talk about Patna, suppose your current neighbour is Patna. So, if 0 plus the weight of Ranchi to Patna, which is 2, less than the distance of Patna. What is the distance of Patna written? Infinity. This means that you have got a better distance, a short distance. This is called relaxation. You have to do two things in this. What do you have to do in this?"
    },
    {
      "start": 1877.55,
      "end": 1907.42,
      "text": "Distance of neighbor is equal to d plus weight. What do we do after removing the answer of patna? 0 plus 2. What do we write here? 0 plus 2. Here we write 2. Update is done. And push this in heap. So that next time in processing, we can pick up patna. Where do we pick up processing? From heap. Push this. What will we push in this? What will we push? We updated the distance of patna in this."
    },
    {
      "start": 1907.42,
```

```
      "end": 1934.91,
      "text": "D plus weight, which is 2. You and Patna push it. T and Patna, push it in the
heat. What did you push? You and Patna. Done? Ranchi is not over yet. Ranchi has another
neighbour. Who? Delhi. Same thing will happen in Delhi. First of all, what will happen? If D
plus weight, how much is Ranchi and Delhi's weight? 10. 0 plus 10, which is 10. If it is less
than Delhi's current distance, what is Delhi's current distance? Infinity."
    },
    {
      "start": 1934.91,
      "end": 1962.26,
      "text": "First of all, what should we do? We are doing the same thing. We are doing the
same thing that we explained earlier. There is no difference. 10. Push also so that Delhi
comes in processing next time. If we do it sometime, then it will come. What did you push? 10
and Delhi. Ranchi's work is over. Ranchi had two neighbors. Look, they processed both the
neighbors. If that relaxation is successful, that means if this D plus weight is less than its
current answer, then let's do two things. Let's update its answer too."
    },
    {
      "start": 1962.26,
      "end": 1991.52,
      "text": "And we insert it in the hip so that it can be processed. Now look, Patna was
processed, that's why Patna was processed to other people too. That's why. Ranchi's work is
done. Remove it. I mean, let's move ahead. We have removed it. Let's move ahead. Ranchi's work
is processed. Look, in the hip, earlier it was one. Now two elements have come. Now, who will
we lift? The one with minimum distance. Why? I have explained. Whom will you lift? Lift Patna.
Bring Patna out. Patna."
    },
    {
      "start": 1991.52,
      "end": 2019.86,
      "text": "Bahaar leke aayiye bhaiya. Ye kya batata hai? D is equal to 2. Matlab, Patna
tak paanchne ka distance 2 km hai. Patna ko pop kar dijiye. Patna ko hata dijiye. Ab, jo kaam
Ranchi ke saath hua tha, same kaam Patna ke saath kijiye. Patna ke ketne neighbor hai? Patna
ke ketne direct neighbor hai? Pahela neighbor to Ranchi hai. Ranchi ko kya relax karenge? Nahi
karenge. Kyun? Kyunki relax karne ka logic kya hai? D plus weight should be less than distance
of neighbor. Less than distance of neighbor."
    },
    {
      "start": 2020.02,
      "end": 2049.38,
      "text": "So, if your neighbour is Ranchi, then the current distance of Ranchi is 0. Now
you are saying that D means 2 and plus this weight 2. Is 4 less than 0? No, Ranchi is not
relaxed. If Ranchi is not relaxed, then Ranchi will not even go in the heat. Ranchi's work is
over. Look at Patna's next neighbour. Delhi. Yes, this will be relaxed. What is its? D. What
is the distance to Patna? 2. What is the weight between Patna and Delhi? 3. So what? 2 plus 3."
    },
    {
      "start": 2049.38,
      "end": 2078.03,
      "text": "Is less than. Is less than. Delhi's current answer. Distance of Delhi. How much
is the distance of Delhi? 10. Yes. Yes. Yes. So what will happen? Two things will happen.
Delhi's answer will also be updated. What will happen after Delhi's answer is updated? 5. And.
It will be pushed in the heap. What will be pushed in the heap? 5 and Delhi. 5 and Delhi. 5
and Delhi. See. There were already 10 Delhi in the heap. Which happened last time. Patna
pushed one more. What did it push?"
    },
    {
      "start": 2078.03,
      "end": 2102.42,
      "text": "5 Delhi. 5 Delhi pushed. What will this make sure? When you pick up for
processing next time, then you will not be able to pick up 10 Delhi. You will pick up 5 Delhi
first. You will move ahead of Delhi's shorter correct answer. You will not do 10 Delhi first
process. Because it is a mini heap. That is why we say that we should process the shortest one.
 See the answer of Delhi. It has fixed itself. It has fixed itself. Okay."
    },
    {
      "start": 2102.42,
      "end": 2132.8,
      "text": "Patna's one more neighbour is left. Which neighbour is left? Bangalore is left.
How much weight is there? 4. How much was Patna's D? 2. How much is 2 plus 4? 6. What is 6? Is
it less than Bangalore's current answer? Yes, it is less. What do we do? First of all, update
Bangalore. How many? 6. Push it. In which? In the heap. 6 and Bangalore. Patna's work is done.
Patna's three neighbours were there. Everyone's work is done. Delete Patna. Meaning, we are
moving from here. We will move towards the next, right?"
    },
    {
      "start": 2132.8,
```

```
      "end": 2163.06,
      "text": "Let's go. Take it out again. Now, who will get out? Minhip means the one whose distance is the least. Out of these three, whose distance is the least? 5. 5 and Delhi. Take it out by pulling it. Take it out by pulling it. What will come out? 5 and Delhi means, what is the distance to Delhi? 5. Start relaxing Delhi's neighbours. Delhi's first neighbour is Ranchi. Will it be right to relax Ranchi? No, it won't be right. I have already told you. Patna. Will it be right to relax Patna?"
    },
    {
      "start": 2163.06,
      "end": 2191.6,
      "text": "5. And its weight is 3. 8. What is the answer for Patna? 2. No benefit. Very good. Bangalore. What is its 5? 5. 5 and 5 is 10. Bangalore's answer is 6. No benefit. Kolkata. Yes. Kolkata. Yes. Kolkata has a benefit. Neighbor is equal to Kolkata. Yes. How much will Kolkata be? Its D is 5. It means it is 5 to Delhi. Its weight is 1. So 5 plus 1. 5 plus 1. To go to Kolkata."
    },
    {
      "start": 2191.6,
      "end": 2221.76,
      "text": "Is this less than the current answer of Kolkata, which is infinity? Absolutely less. What two things will we do? We will update the answer of Kolkata and write 6. And we will push this. In which? In heap. 6 and Kolkata. 6 and Kolkata. Very good. The work is over. Delhi's work is over. Nothing else will happen in Delhi. Okay? Remove it. Remove it in Delhi. Let's go. Take out the next. Take out the next. It is not over yet. Then take it out. Whose distance is the least among the three? 6, 6."
    },
    {
      "start": 2221.76,
      "end": 2251.2,
      "text": "Two people are less. It's up to you. It doesn't matter according to the logic of the heap. Take out Kolkata. Take out Kolkata. Take out Kolkata. 6 and Kolkata. Remove it from here. Let's see Kolkata. Let's see if Kolkata can relax anyone. Let's see. It has D6. There is only one direct number of Kolkata, Delhi. 6 and plus 1. 7. What is the current answer of Delhi? 5. There is no benefit. Kolkata is useless. Remove it. There is no benefit."
    },
    {
      "start": 2251.2,
      "end": 2280.91,
      "text": "Take out the next one. What will come out? What will come out? 6 and Bangalore. Take out the next one. 6 and Bangalore. Take out the next one. What is its D? 6. Try to relax. Nothing will happen. Take out the next one. How many neighbours does Bangalore have? 2. Patna and this. Patna will say 6 and 4. It should take 10 to go to Patna. Patna will say it takes 2 km. We don't need it. He will say mine is 6 and this is 5. It should take 11 to go to Delhi. Delhi will say mine is already 5. We don't need it either. Nothing. Useless. Useless. Useless. Yes. Yes. Yes."
    },
    {
      "start": 2281.2,
      "end": 2308.72,
      "text": "Take out the next one. Take out the next one. What will come out? There is only one left, so it will come out. 10 or Delhi. Now look very carefully here. Now look very carefully here. Look very carefully. Now, if you want, you can process it. There will be no benefit. There will be no benefit. Now do it. We are trying to tell you one more important thing here. What is its D? 10. Okay. Is Delhi's current answer 10? No."
    },
    {
      "start": 2308.72,
      "end": 2335.01,
      "text": "When did this 10 Delhi come? This 10 Delhi, no, remember when it came. When you processed Ranchi and wrote Delhi, then this was the answer. Delhi has become correct after that. Delhi is now 5. So, should you process this in any birth? Means, should you try to do something? No, because this is the wrong answer. This was the answer of some earlier. We thought this earlier Delhi was 10 km."
    },
    {
      "start": 2335.01,
      "end": 2363.38,
      "text": "Now, Delhi is not 10 km. Now, Delhi is 5 km. So, add one more small condition. If D greater than distance of this Delhi node. What does this mean? This is an old value. This is an old value of Delhi. Delhi's current value is 5. So, if 5 came from here, from the heap, then we would have tried once. 10 is coming from here."
    },
    {
      "start": 2363.38,
      "end": 2391.09,
      "text": "So it means that there is no logic to process it. Delhi's correct answer is 5,
```

which had changed later. And we would have processed 5 Delhi as well. Why? Because it is a min-heap. 5 Delhi would have left before 10 Delhi left. Now 10 Delhi has come. Do you want to touch 10 Delhi? I mean, do you want to think about it? When you are able to see that this answer is old. How are you knowing that this answer is old? Because distance of Delhi"
    },
    {
      "start": 2391.09,
      "end": 2417.79,
      "text": "It is more than 10. Obviously, it is 10 years old. So, shouldn't you process it? Absolutely not. If it is so, if it is so, then continue. There is no need to look at the neighbor of Delhi. There will be no benefit from it. This is an old stale value. Remove it and move ahead. Finish it. There is no harm in it. I hope you have understood. Why are we saying that Delhi's answer is 10 years old? Because Delhi's current answer is 5."
    },
    {
      "start": 2417.79,
      "end": 2446.8,
      "text": "I mean, someone must have made Delhi 5. Patna had made Delhi 5. This 10 is very old. This is very old, some rotten value. Are you trying to do something with 10? No. 5 and Delhi has already been processed. Whatever he had to do, he has done. He had to do Kolkata, he has done. Now, even if you process it, it won't make any difference. You do it and see. 10 here and there. What will happen with this? This will waste time. What will happen with this? Time will be wasted. We are doing it again and again. Don't do stale value process. Okay? Let's go."
    },
    {
      "start": 2446.8,
      "end": 2475.09,
      "text": "Do this. Your heap is empty. Have you got a correct answer? 0, 2, 6, 5, 6. Yes, sir, you have got a correct answer. This is how Dijkstra works. Now, let's write the code. In the code, we will tell you everything. In the code comment, if possible, we will give it. The first part of the code is how to make an adjacency list for weights. How to make an adjacency list for weights and how to write this Dijkstra. How to write the story that we just told you. Let's print it together."
    },
    {
      "start": 2475.09,
      "end": 2503.42,
      "text": "So the first thing that comes to mind is how to make an adjacency list. Till now we were learning non-weighted. How to make it for weighted? There is a little tweak, not much. See, how did you make an adjacency list? Let's assume there is a graph like this. Let's assume there is a graph like this. This was your graph. Suppose this was your graph. This was a non-weighted graph. So what did you do? You used to make a vector of vector of int. There are three types of it. 0, 1, 2."
    },
    {
      "start": 2503.42,
      "end": 2533.33,
      "text": "There used to be a vector in everything. What did you write in that? What did you write in the vector of 0? Two neighbors of 0, meaning 1 and 2. Did you write this? Okay. What did you write in the neighbor of 1? One neighbor of 1 would be 2. I am telling you about the directed. It will be the same in the undirected. You didn't write anything in the neighbor of 2. You used to make it like this, right? Okay. Now weight is involved. Now you are saying that its weight is 2, 5 and 3. Now weight is involved. Now, when you are writing 1, 2 here,"
    },
    {
      "start": 2533.33,
      "end": 2561.49,
      "text": "Now, writing 1, 2 will not work. Now, there are two neighbors of 0, 1, 2. But, now you have to write pair here. Now, you have to write pair here. Meaning, you have to make an array by doing this. You have to make it like this. It will remain the same. 0, 1, 2. Now, you have to write pair instead of single integer. What is the first pair written? 1. It is going to 1. And what is its weight? 2."
    },
    {
      "start": 2561.58,
      "end": 2590.83,
      "text": "And what is the second pair? 2. How much is the weight of 2? 5. This is done. 1 is replaced by a pair. 2 is replaced by a pair. Where 1 was, it is 1. With that, its weight. 2 has to be replaced by a pair. 2 and its weight. Similar thing. What did you write in 1? 2. Okay. 2 has to be replaced by a pair. 2. And what is the weight of 2 from 1? 3. And this will remain empty as it is. We will do the same thing. So,"
    },
    {
      "start": 2590.86,
      "end": 2618.74,
      "text": "We will tell you. Let's write the code on GFG. You have to return a vector of

```
int of Dijkstra. What is given to you? Int n means this node and you are given a vector of int edges. And you are given a source. This is given to you."
    },
    {
      "start": 2619.06,
      "end": 2647.25,
      "text": "Keep in mind, what edges were given to you till now? 0 to 1 is one edge, 0 to 2 is one edge, 0 to 3 is one edge. Now, because this is a weighted graph, what will be given to you? 0 to 1 is an edge and its weight is 2. 0 to 2 is an edge and its weight is 5. 0 to 3 is an edge and its weight is 6. This will be given. Meaning, every edge, every single edge,"
    },
    {
      "start": 2647.25,
      "end": 2676.37,
      "text": "Now, it is of 3 sizes. Previously, every edge was of 2 sizes. In that, what did you say? Its 0 is the source. Its 1 is the destination. Now, its 2 is its weight. It is the weight of this edge. Practice once or twice. Otherwise, you will understand the graph by making it yourself. The harder it seems to you, the harder it is not. It is an extension. You will understand when you write the code. Look. I have given you this. First of all, make your adjacency list. How? Look. Vector of"
    },
    {
      "start": 2676.53,
      "end": 2706.45,
      "text": "Vector of. Now, what did we write here? Int. What did we say? Replace every int with pair of int, int. What did we write here? Pair of int, int. Pair of int, int. 1, 2, 3. See this. Same. Here. Here, your int used to be. What did we make this int? Pair. Because every number will become a pair. Every number will become a pair. Earlier, every number used to say that this is the destination. Now, this is the destination. And its weight is also there. So, it became a pair."
    },
    {
      "start": 2706.45,
      "end": 2735.57,
      "text": "Now see how the graph will be made. What happened? For int i is equal to 0, i less than edges dot size, edges dot size. They also used to do this. They also used to do this in normal. Now, first of all, find the source. Source is equal to edges i 0. Very good. You know this. Destination is equal to edges i 1. You also know this. Now what did we say?"
    },
    {
      "start": 2735.57,
      "end": 2765.79,
      "text": "Now, you will have three values in one edge. Not two values. Source, destination and weight. What will be the weight? int w is equal to edges i2. This is your weight. Source, destination, weight. Now, because you have been told that this is an undirected graph. What is it doing? Array of s dot pushback. Now, what pushback did we do in non-weighted? Only d. Now, d means destination. Only destination will not work."
    },
    {
      "start": 2765.79,
      "end": 2794.37,
      "text": "Earlier, what did you push? Only 1, which was the destination. Instead of 1, what do you have to push? A pair in which the first is the destination and the second is the weight. So instead of this, push a pair. What do you push? Destination and weight. Look at this. It is undirected. Push it in reverse. AD dot pushback. What? Earlier, only source was pushed. Source and weight together. Instead of integer, we are pushing pair. Your adjacency is made. Look at this."
    },
    {
      "start": 2794.37,
      "end": 2823.7,
      "text": "We will suggest you to go and make it yourself in the copy and see how your graph is looking, then you will have more understanding. Because now we are going to do the majority of the questions on the weighted graph. The era of unweighted is over. The era of unweighted is over. Now we have come to weighted. This is yours. Now what do you have to make? Now we can start like this. So first of all, you make a min heap of pairs. Oh God, the syntax of min heap of pairs is so bad. Come on, make a min heap of pairs."
    },
    {
      "start": 2823.7,
      "end": 2855.06,
      "text": "How is it? A pair of int, int. A pair of int, int. Vector of, I don't know why there is such a syntax, but this is the C++ one. We have already taught the Java Python people how to make it, so make it as it is. A pair of, we can't write the code of all three, obviously. This is done, this is done, and we are also looking at the syntax and writing it, but we don't remember its syntax. A pair, what does it do? One, two, three."
    },
```

```
    {
      "start": 2855.06,
      "end": 2885.98,
      "text": "PQ. What is this? Min heap of pairs. Now we will make a distance array.
Distance array means the array with infinity above it. Vector of int distance. What will
happen? N and infinity. Infinity means int max. Int max. This is done. Now start your game.
What does it mean to start the game? First of all, zero the source distance. How much time
does it take to go from source to source? Zero time."
    },
    {
      "start": 2885.98,
      "end": 2912.24,
      "text": "It takes 0 km. Done. And push this. In which? In heap. What are we pushing in
heap? Distance and node. Means distance and source. Push done. Now start your game. Let's
start our game. Go to the next page and start your game. So while pq.notempty. Until it is
empty, we will play our game. Do this."
    },
    {
      "start": 2912.24,
      "end": 2938.98,
      "text": "Take out a pair. Take out the top one. It will give you the minimum distance.
It has a guarantee. PQ.top and PQ.pop. PQ.pop. Okay? Now, what is the first thing in your P?
Its distance. So, int d is equal to P.first. Very good. And what is the node? Its second one.
Node is equal to P.second. Meaning, what is this?"
    },
    {
      "start": 2938.98,
      "end": 2966.13,
      "text": "D is equal to 2. Node means Patna. Okay. First of all, do what we taught you.
What did we teach you? Where did it go? Did we delete it? Oh, man. We didn't have to delete it.
 Yes. This one. If D is greater than distance of node, which means, which is this? Old. If we
get 10 Delhi, we won't do that process. We'll go ahead. Processing means we won't even look at
its neighbor. We won't do anything. If."
    },
    {
      "start": 2966.32,
      "end": 2995.84,
      "text": "D is greater than distance of node. It means, the current answer of node, if D
is greater than that, it means it is an old value. What is this? It means, it is an old value.
Ignore it. Continue. Ignore means continue. Yes. Now, if it is not like this, then you will
have to process. Now, rotate on all its neighbors. How do we rotate on neighbors? For int j is
equal to 0, j less than array of node dot size."
    },
    {
      "start": 2995.84,
      "end": 3026.37,
      "text": "Yes, and j++, j++. Now, what did we do there? Neighbor, int, neighbor, int,
neighbor is equal to array of node j. What is array of node j? It is a pair, a pair, whose
first person is that neighbor, second person is its weight. So, first, first, first is its
neighbor, and int, weight is equal to array of node j."
    },
    {
      "start": 3026.37,
      "end": 3056.22,
      "text": "Second. Did you understand? Did you understand? Okay. Now, look. If D plus
weight is less than distance of neighbor, which means, the answer of neighbor right now, is it
less than this D plus weight? Meaning, D means it's still on the node, and there's weight to
the neighbor from the node, so it reached there. If it's less, then what will we do?
Relaxation. How many steps are there in relaxation? Two steps. First of all,"
    },
    {
      "start": 3056.22,
      "end": 3080.82,
      "text": "Update the distance of neighbor. Update the distance of neighbor. Is equal to
weight plus D. And push in the heap. What is pushed in the heap? Distance means D plus weight.
And this neighbor. And this neighbor. This is pushed. See this. Close. Close. Close. In the
last, return."
    },
    {
      "start": 3080.85,
      "end": 3109.68,
      "text": "Rested. See this is the code. It is a very big code. We know that it is a very
big code. We know that. We will try to give it in the comments today. We will definitely try
to give it. This is a very big code. So see this. From start to finish code. What does this
code say? What did you make so far? Made an adjacency list. We did some of our heap. What is
this? Main logic. What is this? Relaxation. It works like this. My condition is down today. I
don't know how long it is. But it is fine."
```

```
    },
    {
      "start": 3109.68,
      "end": 3116.82,
      "text": "So it took a lot of hard work. So do it. Do its code. Do its question. Let's
meet in the next video."
    }
  ]
}
```


11. transactions/t_2.json


```
{
  "video_id": "-CvRpL_iUKo",
  "title": "DP Series Episode-2 : When And How To Use DP",
  "language": "en",
  "model": "large-v3",
  "duration": 1867,
  "segments": [
    {
      "start": 0.0,
      "end": 28.06,
      "text": "Before starting the video, look at the code. How many lines are there? I will
count them and tell you. One line, two lines, three lines, four lines. Okay? And five lines.
Let's count the brackets. It is a function of five lines. It is a very small function. If you
run it on your laptop, run it by looking at it now. Int n, means you will run it by looking at
the value of n. If you give the value of n as 5, then it will complete. Means from pressing
the run button to the output. It will take 15 nanoseconds. Nanoseconds means second by"
    },
    {
      "start": 28.06,
      "end": 56.72,
      "text": "Billion divided by such a small amount of nanosecond, even a human can't even
know. If you give n is equal to 20, it will take 22 microseconds. Microsecond means by 10 to
the power 6. Very very small amount of time. If you run it on n is equal to 50, it will take
you 40 seconds. Means the time that has been till now since the beginning of the video, it has
been 40-45 seconds. How much time? 40 seconds. Very less time. If you pass n is equal to 100
in this, now pass it and see. How much time will it take you?"
    },
    {
      "start": 56.72,
      "end": 79.38,
      "text": "36,000 years 36,000 years 36,000 years 36,000 years it will take you to run
this program if you just increase the value of n to 100, not more, 50 to 100 means to give you
an idea, human history means since humans started living on earth, we have been doing this for
about 5000 years"
    },
    {
      "start": 79.38,
      "end": 106.93,
      "text": "If the world's first person had run this program, it would not have been
completed yet. Because it will take 36,000 years to complete. Why will it take? We are telling
you this because it is important for you to understand why DP is asked so much in interviews.
Why is DP so tough? Why is DP so important? This is why it is important. If you write it like
this, it will take you 36,000 years. If you write it with DP, it will take you 5 to 100 years."
    },
    {
      "start": 106.93,
      "end": 135.92,
      "text": "Millisecond. Millisecond means 1000th part of a second. So see, 36,000 years,
from where to where, means it won't take even a second. That's why it's important to study DP.
Now watch the video, you'll get some interest. Let's go. See, before starting DP, you'll have
to understand that DP... We hope that you came here yesterday, so you must have watched my
lecture of DP day 1 and recursion. If you haven't watched recursion, then see, there are 9
lectures, it won't take more than half an hour. Watch that and come here. See. The biggest
thing to learn DP is..."
    },
    {
      "start": 135.92,
      "end": 160.38,
      "text": "The negative side is that once a person reaches DP, that is, once we had
learned DP, when we were in college, we had learned DP from somewhere. After that, you will
feel that if you are getting a hard question of any leetcode, then your mind will go first to
put dynamic programming in it. And you will start putting dynamic programming in it. After
half an hour or an hour, you will understand that it will not take DP. DP is not a question,
it is a question of heap, it is a question of binary search. So the biggest problem is that
```

```
because it is so..."
    },
    {
      "start": 160.38,
      "end": 189.58,
      "text": "is considered a hard topic, then you will think that if we get a DP, then we
will try to put a DP in every question and see if it can be made because there is so much
benefit. I mean, we said above that it has been reduced from 36,000 years, so you will think
that we put it in every question. No. See, to put a DP, first of all, when, means when you
have to think that whether we have entered dynamic programming or not. First of all, that
question will be solved by recursion. That question will be solved by recursion. But there is
a problem in this too."
    },
    {
      "start": 189.58,
      "end": 205.66,
      "text": "In computer science, any DSA problem can be solved either by recursion or
iteratively. There are only two ways in the world. What we mean to say is that to solve any
DSA question, there are two ways to solve it. One way is recursion."
    },
    {
      "start": 205.66,
      "end": 232.93,
      "text": "One way is iterative. Iterative means where you write a loop. For loop, while
loop, call the loop. Recursion means that you call the same function again and again and use
its answer to make your answer. So see, any DSA question can be done in both ways. There is no
such question that can't be done with recursion and can be done with loop. There is no such
question that can't be done with recursion and can't be done with loop. There is nothing like
that. So then you will say that this is very stupid. You are saying that"
    },
    {
      "start": 232.93,
      "end": 261.2,
      "text": "To apply DP, check if recursion will be applied or not. If you say recursion
will be applied in all, then DP will also be applied in all. No. See. First thing, recursion
will be applied in all. It's good that you understand that recursion will be applied in all.
Pick any program. Sum of array, product of array, print multiplication table of two. Pick any
program, we will apply recursion in it and show you. It's good. If, if, plus, plus, plus,
means recursion is applied, okay. Or, if you,"
    },
    {
      "start": 261.2,
      "end": 285.52,
      "text": "If you are solving the same thing over and over again, what does this mean?
What will be applied there? Dp will be applied. If you have written recursion, what do you
pass in recursion? What does recursion mean? It is a function. So, it is a function. And in
this,"
    },
    {
      "start": 285.52,
      "end": 314.7,
      "text": "value can be passed, number can be passed, array can be passed, array index can
be passed, string can be passed, string can be passed, anything can be passed. If your
recursion is doing the same thing again and again, means for example, this n is being passed,
you can see that my recursion, first we run fun for n is equal to 5, it worked. Then we are
seeing that fun n is equal to 5 came ahead. Okay, solve this too. Then we are seeing that fun
n is equal to 5, what is happening?"
    },
    {
      "start": 314.7,
      "end": 343.23,
      "text": "If you are solving the same recursion value again and again, it means that the
question will require dynamic programming. If you are solving new problems every time, i.e. if
your function, fun, we know that recursion occurs many times, but if the value of n is
different every time, i.e. it is never happening that you are solving n in the same thing,
then don't use dynamic programming even if you have forgotten it, it will be of no use."
    },
    {
      "start": 343.23,
      "end": 372.27,
      "text": "The interviewer will immediately reject you because if you don't understand
whether you are solving again and again or not, then you will be rejected. This process of
doing it again and again, some people call it overlapping sub-problems. Why is it called
overlapping? Overlapping means doing the same thing again and again. The example of the tea
vendor yesterday, what was the tea vendor doing? He did the calculation of two teas at once."
    },
    {
```

```
      "start": 372.27,
      "end": 400.29,
      "text": "Then someone asked him about two tea and one biscuit. So, he was again
calculating two tea and one biscuit. What does this mean? The tea vendor is doing the same
thing again and again. That's why the one who was going to write on the board worked there. If
the tea vendor solved different problems every time. For example, if he told once for one tea.
Okay. Second time for one biscuit. Third time for one fan. After that for one pakora. So, this
is a different problem, right? What will you write and do in this?"
    },
    {
      "start": 400.29,
      "end": 424.9,
      "text": "The whole purpose of DP is to write the answer to the first problem that you
have solved in your diary or board. If you are getting new problems again and again, then why
are you writing the previous one? As the previous problem will never be repeated. What is the
need to write the answer to that? If you are in a job, for example, if you are in college,
your teacher gives you different questions every day. Every day he gives you new questions."
    },
    {
      "start": 424.9,
      "end": 454.59,
      "text": "So why would you do the revision? Why would you remember the previous question?
Because you know that if you have asked this question today, you will have to solve a new
question tomorrow. So what is the use of remembering? But when you had a 10th board, what used
to happen in it? Every year, every year, some questions are the same. That means the question
that came 5 years ago is getting repeated. The question that came 3 years ago is also getting
repeated. What do people do in that? They write down the past year paper that if it comes,
they will write the answer immediately."
    },
    {
      "start": 454.59,
      "end": 483.14,
      "text": "This is the difference. In which? Where you will apply DP and where you will
not. Don't worry, we will understand this from the code also. But it is very important for you
to keep in mind that DP should not enter everywhere. As soon as you see a problem of elite
code in which you don't understand, then start applying DP. No. First of all, is your
recursion, is your recursion solving the same problem again and again? Only then you have to
think about DP. And the second question is how? If you think, if you know that dynamic
programming will take place,"
    },
    {
      "start": 483.14,
      "end": 500.34,
      "text": "How to apply it? What to do to apply it? Now, we will explain it to you in code.
 So, we hope that you have understood it. Understand it very carefully. If possible, try to
repeat it again. Keep this in your mind. That you have to check this before any DP problem.
Okay? Let's go."
    },
    {
      "start": 500.66,
      "end": 530.14,
      "text": "Now we will explain this concept in real world that how you will know. See this
is a very simple question. Means you will be given n and you have to print 2xn. So you will do
that print 2xn. But we will solve this with recursion. Why? Because we want to explain you
something. Imagine that this is the question of your leetcode DSA. First of all try to write
its recursion. What is recursion? We will advise you again. First of all there is a base case
in recursion. What does base case mean?"
    },
    {
      "start": 530.14,
      "end": 555.74,
      "text": "What will be the answer of the smallest input? If the value of n, suppose the
equation is that n will always be greater than 1. It means if it is 1, then it is fine. The
smallest can be 1. So, first of all, in the base case, if n is 1, the smallest input can be 1.
What will be the answer of that? Do you know? What will be the answer of that? The answer of 2
into 1 is 2. So, in the base case, if you get n is equal to 1, then you..."
    },
    {
      "start": 555.74,
      "end": 584.16,
      "text": "Be very confident and comfortable and return 2. The base case is done. After
that, what happens? If you get an answer of n minus 1 or any smaller value than n, what does
that mean? Suppose n is equal to 5. So, what should your answer be? 2 into 5 should be 10.
Suppose you don't know how to do multiplication. You don't know how to do 2 into 5. You don't
know how to write 2 into 5. It's okay. If someone tells you the answer of 2 into 4,"
    },
    {
```

```
      "start": 584.16,
      "end": 613.42,
      "text": "If a man from magic tells you the answer of 2x4, which means 8, can you get 2x5
now? Yes, you can. If 2x4 is 8, then what is done to get 2x5? 2 is added to it. So, if you do
8 plus 2, its answer becomes 10. Meaning, if you are asked 2x5, if a man from magic tells you
the answer of 2x4, then what will you do? Whatever answer you have told for n-1, which means
for 4, you will add 2 to it and give it. So, this is what you have to do. You have solved this
as well."
    },
    {
      "start": 613.78,
      "end": 636.51,
      "text": "How to make your own answer from a small answer? How to make your own answer?
So, its recursion will be very simple. What will be its recursion? See, int, this is your
function. You will write, there will be an npass in it. There will be an npass in it. Means,
whatever is your main function or lead code function, suppose it passes you 5 and gives you 5.
Now tell me, what is 2 into 5? So, first of all, you have written your base case."
    },
    {
      "start": 636.51,
      "end": 664.7,
      "text": "n equal equal to 1. If n is 1, then what do we have to return? We have to
return 2. Return 2. Why? Because we have written 2 in the base case. If, if, let's assume,
let's assume, with magic, with magic, someone tells you the answer of n minus 1, which means,
if n is equal to 5 was passed, you passed n minus 1 on the same function, and let's assume
that with magic, this person will tell you 8."
    },
    {
      "start": 664.7,
      "end": 688.99,
      "text": "How will it tell you? I don't know, brother. I don't know. It doesn't matter to
us. It will tell you with magic. If you get to know this answer, that the value of 2 into 4 is
8, then what do you have to do? Return answer plus 2. The game is over. You will get it. Come,
let's go and see it. What is this? This is a recursive code. Why? Let's tell you now. Look.
Assume. Assume your beginner."
    },
    {
      "start": 688.99,
      "end": 717.58,
      "text": "Means, what you have got is 4. Means, you have to tell n is equal to 4. What is
2 into 4? Now, run this. Run this. What is the first line of this function? Is n 1? No, it is
not n 1. Okay. After that, what does it do? Answer is equal to fun of n minus 1. Means, it
calls this function. It calls this function for a small value. For whom? For a small value.
Very good. Now, as soon as it is called, we go to this. Fun n is equal to 3."
    },
    {
      "start": 717.87,
      "end": 746.82,
      "text": "We are going a little fast in this because it is very deep and it has been done
in many ways in recursion. So we don't want to waste your time by telling you the same thing
again and again. n is equal to 3. It will do the same thing. It will do the same thing. What
will it do? Fun. n is equal to 2. It did the same thing. Okay. Run n is equal to 2. Fun. n is
equal to 2. It will do the same thing. Answer is equal to fun of n is equal to 1. Okay. Call
on n is equal to 1. Fun."
    },
    {
      "start": 746.82,
      "end": 772.67,
      "text": "n is equal to 1. Now, 1 has come. What does your function say? If n is 1, then
don't call further. Return 2 immediately. So, what will it return? n is equal to 1. What will
it return? It will return 2. And this function is finished. Where will its value come? Its
value will come in this answer. Meaning, we are in this function. What is the value of your
answer here? 2."
    },
    {
      "start": 772.67,
      "end": 797.46,
      "text": "Now, what is its function? Whatever answer you have got, add 2 to it and return.
 What add? Add 2 and return. So, what will it return? Return. Whatever you have got, add 2 to
it and return. Meaning, 4 is returned and this function is also complete. This 4, where is the
answer? In the answer of n is equal to 3. Meaning, what will be the answer of n is equal to 3?
4. What else is n is equal to 3?"
    },
    {
      "start": 797.46,
      "end": 820.66,
      "text": "Whatever the answer is, add 2 to it. That's it. This is also done. This 6,
```

```
where will it come? n is equal to 4k, which was the main function, which was the first function. Very good. Now, you have got 6. What is the work of n is equal to 4? Brother, whatever the answer is from there,"
    },
    {
      "start": 820.66,
      "end": 847.1,
      "text": "In 6, add 2 and return 8. Where will this 8 go? Your domain function or where you are printing. Look once. Should the answer of m is equal to 4 be 8? Is 2 into 4 8? Yes, sir. It is absolutely 8. So, this recursion is working well. So, you have learned to write recursion. You have written recursion. But, but, is this a question of dynamic programming? Look once carefully. Look once. This fun."
    },
    {
      "start": 847.1,
      "end": 876.67,
      "text": "This fun has been called many times. The first fun was this. After that, we called this. After that, we called this. After that, we called here. Is the N written in the fun, the value of N is changing every time, right? Do you see anything repeated anywhere? Do you ever see that N is equal to 4 is calculated here? N is equal to 4 is the same in the future? Do you see any instance where you have overlapping subproblems? Meaning, once you have called fun N is equal to 4,"
    },
    {
      "start": 876.67,
      "end": 902.13,
      "text": "Then in the future, N is equal to 4 is being called. Can you see anything like this? No, you can't. If we explain to you line by line how the recursion was called, this was the recursion. First of all, N is equal to 4 was called. This person called N is equal to 3. This person called N is equal to 2. This person called N is equal to 1. And N is equal to 1 did not call anyone. He went back."
    },
    {
      "start": 902.13,
      "end": 931.68,
      "text": "Do you see any repeated number in this call stack? No. Why not? Because this recursion is unique. Every number, 4, 3, 2, 1, has been called only once. In your copy, you will not see that the function of n is equal to 2 has been called 3-4 times. No. That means, in this question,"
    },
    {
      "start": 931.68,
      "end": 959.68,
      "text": "These overlapping sub-problems are solving the same input over and over again. This is not the case. If this is not the case, then should you put DP in it? No, you shouldn't.  You shouldn't even forget to put it. You can put it, but it won't be of any use. If you put DP in it, it means rejection. This is what we were explaining to you. Look at it. Write the recursion in the copy. Think about it. You will get to know whether you are solving it over and over again or not. In this question,"
    },
    {
      "start": 959.68,
      "end": 984.86,
      "text": "Even if you forget, you should say that we will put DP in this. You should not say at all. Why? Because this question has a recursion. But there are no overlapping sub-problems. We are not calling the same problem again and again. In the same recursion, that input is not being repeated. That input is unique every time. Every time it is different. That means you are solving different questions every day. If you have to solve different questions in every recursion call every day, then what is the purpose of writing?"
    },
    {
      "start": 984.86,
      "end": 1009.81,
      "text": "This is a perfect example where dp is not applied in recursion. In the next example, we will show you when dp will be applied and you will know that dp will be applied in this and we will also show you how to apply it in one line. Let's move ahead. Now look at this question. You can also get this question anywhere in the lead code. This is also a question of the lead code. How much is this lead code? This is the lead code number."
    },
    {
      "start": 1009.81,
      "end": 1039.79,
      "text": "509. 500. It gets blocked because of the camera. It is 509. Okay. What have I told you in this? I have told you that I will give you an n. You will get an n input. On top also n. Here also n. You have to tell the value of its fibonacci. What is the value of fibonacci? I have also explained that to you. See. If. If I give you n is equal to 0 by mistake. Then return 0 to me. If I give you n is equal to 0. Then return 0 to me. If I give
```

```
you n is equal to 1 by mistake. Then return 1 to me. Very good. What does this mean? This is your base case."
    },
    {
      "start": 1039.79,
      "end": 1061.57,
      "text": "What can be the smallest input? One 0 and one 1. The answer to both of them is straightforward. That brother, the answer to n is equal to 0 is 0. The answer to n is equal to 1 is 1. Very good. If you get n greater than that. Meaning if you get n 2, 3, 4, 5, 10. Then what do you have to do? You have to return Fibonacci of n. How will you find out its value? Whatever."
    },
    {
      "start": 1061.57,
      "end": 1088.34,
      "text": "n-1 of Fibonacci. Meaning, if you are given n is equal to 5, then whatever the answer of 4 is, from anywhere, it doesn't matter, whatever the answer of n-1 is, or plus, plus, whatever the answer of n-2 is, let's add them and return them. Meaning, if let's say, you are given n is equal to 5, then look, you have to tell what is the value of n is equal to 5. You don't know, but you are given a formula, that brother, whatever,"
    },
    {
      "start": 1088.34,
      "end": 1113.97,
      "text": "n is equal to the value of 4. Take that out of the magic. I don't know from where you will take it out. Take it out. Plus. Plus. Whatever n is equal to. Whatever n is equal to. Take out the value of 3 from somewhere. It doesn't matter to us. Do this and return it to us. So this is. Recursion means. Recursion's brand ambassador question. I mean. I have explained both things in the question. What have I explained? This is your base case."
    },
    {
      "start": 1113.97,
      "end": 1142.77,
      "text": "And this is your second point that how to make your answer from a small answer. This means that if the recursion is in the film, then the question is in the film. There is some relation that this question is explained by the recursion and this question is explained by the recursion. So both things have been given to you. So let's quickly write the recursion of this. What will you write in the recursion? See, we will write int fun and you will get an int n obviously. First of all, write your base case that if n is equal to 0 and"
    },
    {
      "start": 1142.77,
      "end": 1169.1,
      "text": "n equal equal to 1 hai. Kya return kar dijiye? n hi ko return kar dijiye. Kyun hai saam likh rahe hain? Kyunki dekhiye, jab n 0 hai to answer 0 hai, jab n 1 hai to answer 1 hai, to hum return n kar diye. Toh ho gaya. Aap isko alag bhi likh sakte hain. If n equal to 0, return 0. If n is equal to 1, return 1. Jaise man kare base case likhiye. Uske baad, ab aapko chote chote answer chahiye. Upar ke multiplication mein aapko bhi chota answer chahiye tha, n minus 1 ka. Isme aapko 2 chota answer chahiye. Khansa?"
    },
    {
      "start": 1169.1,
      "end": 1197.1,
      "text": "One small answer of n-1 and one small answer of n-2. When you get both the answers from somewhere, from the magic, then you will join them and return them. So see, in this, first of all, int a1 is equal to this function called for n-1. And int a2 is equal to this function called for n-2. Suppose, by the grace of God, you get these two by magic. Your work is very easy. You just"
    },
    {
      "start": 1197.1,
      "end": 1219.18,
      "text": "Return these two together. It's over. This is also a code of 5-6 lines. This code was there in the beginning of the video. So, this is the code of Fibonacci. Now, you have written the recursion in this. Now, should we put DP in this or not? What is its litmus test? If you are solving the same problem again and again, you can put DP in it. Let's see. We will call it n is equal to"
    },
    {
      "start": 1219.18,
      "end": 1243.34,
      "text": "For 4, let's run it and see what happens for n is equal to 4 and we will get to know on the way whether it is happening or not. You will be able to see it in the copy. Don't think that we have to run it a lot. Nothing. We will run it 3-4 times and you will be able to see everything in the copy. Let's go. Let's start. Whom will we start with? In fun, n is equal to 4 is passed to you and you are given 4 as the lead. What will happen first? a1 will come out. Fun, for n is equal to 3."
```

```
    },
    {
      "start": 1243.34,
      "end": 1271.1,
      "text": "After that, when this A1 runs completely, when this A1 runs completely, after
that, we will call n is equal to 2. We put tick in it. Tick means it is running. Tick not
means that line is not running. So, the first line will run on n is equal to 3. You will jump.
Come, jump with me. Fun n is equal to 3. Come. Same work will happen. A1 is equal to fun n is
equal to 2. A2 is equal to fun n is equal to 1. This will happen."
    },
    {
      "start": 1271.1,
      "end": 1300.24,
      "text": "In this too, run the first line. By ticking on the first line. What do you want
on that? On n is equal to 2. Fun. Come on n is equal to 2. In this too, a1 is equal to fun of
n is equal to 1. Okay. a2 is equal to fun n is equal to 0. Okay. Okay. On this too, tick mark.
After tick mark, what do you want? On n is equal to 1. On n is equal to 1. Come on, write. Fun.
"
    },
    {
      "start": 1300.24,
      "end": 1328.66,
      "text": "n is equal to 1. Now, n is equal to 1. You have come to the base case. Because
in your function, it is written that n is 0 or 1. So, you have to return n. So, what will it
return? Return 1. And this function will end. This function will end. Very good. Where will
this 1 come? Here. Its answer has come. Now, run its second line. Tick mark on the second line.
 Run. What will happen? Fun n is equal to 0. What will it return? 0. Because this is the base
case. Return 0. And this function?"
    },
    {
      "start": 1328.66,
      "end": 1359.25,
      "text": "It will end. Where will its answer come? It will come to 0. Now, there is
nothing left to do in this function. What does it have to do? It has to join both and return.
1 plus 0 is equal to 1. It returned 1. And this person also ended. This person also ended.
Where will this 1 come? It will come here. This 1 will come here. Now, look at this. Now, an
interesting thing has started. Now, let's save the other n. It has ended. It will work, right?
Look. n is equal to 1. Run it again. Come here again. Fun."
    },
    {
      "start": 1359.25,
      "end": 1385.66,
      "text": "n is equal to 1. What will he do? Return 1. And this person will be finished.
Where will this 1 come? It will come here. What is the work of this function? Return 1 plus 1
is equal to 2. After returning 2, its work will also be finished. Where will this 2 come? It
will come here. Now you have come to your original function, but its A2 line is running. Its
just this line is running. A2 has to run."
    },
    {
      "start": 1385.66,
      "end": 1413.97,
      "text": "Run A2. Run A2. Tick mark it in A2. Run it. Where will it run? Fun. N is equal
to 2. What will it do? A1 is equal to fun. N is equal to 1. A2 is equal to fun. N is equal to
0. Run it one by one. Run it. Fun. N is equal to 1. What will it return? Return 1. And this
function will end. Where is this 1? Here."
    },
    {
      "start": 1413.97,
      "end": 1445.01,
      "text": "n is equal to zero ka chalna baaki hai. To chalao, chalao, jo jo baaki hai sab
chalana padega. Fun, n is equal to zero. Chalao usko. Return, zero. Aur karke ye function
khatam ho jayega. Yaha pe aage zero. Ab isadmi ka kaam kya hai? Return, one plus zero is equal
to one. Ye karke, ye bhi khatam ho jayega. Aur ye kaha pe aajayega? Yaha pe aajayega. Kya
return kiya hai ye? Two. Ab n is equal to four ka kaam kya hai? Return, two plus two is equal
to?"
    },
    {
      "start": 1445.01,
      "end": 1474.93,
      "text": "So, what will be your final answer? 4. Now, repeat the same exercise. Do you
see repeated sub-problems? Do you see repeated functions anywhere? Yes, you can see them. How
many times do you see n is equal to 2? 2 times. When n is equal to 2 was solved here for the
first time, you solved it here again later. Remember. How many times do you see n is equal to
1? 1 time."
    },
    {
      "start": 1474.93,
```

```
      "end": 1497.26,
      "text": "2 times. 2 times. How many times is n equal to 0? 2 times. It means that you
are doing the same thing again and again. You are doing the same thing again and again. What
will have to be applied to it? Dp will have to be applied. It is written on its face that Dp
will be applied. Why? Because,"
    },
    {
      "start": 1497.26,
      "end": 1516.19,
      "text": "n is equal to 2, once it was solved, n is equal to 2's answer was known from
here, whatever its 1, 1, 0, whatever it is. When you solved n is equal to 2, you knew that n
is equal to 2's answer is 1. Why did you run it again? You ran it, then you had to run n is
equal to 1."
    },
    {
      "start": 1516.19,
      "end": 1544.34,
      "text": "Then you have to run n is equal to 0. Why are you doing this again and again?
If you had written n is equal to 2 here in the diary, then when you would have come here, then
you would have just looked at the diary and told it immediately. You would not have to do that
much call over. And now we have taken n is equal to 4, so you must be thinking what is the
problem in this? So it is repeating only once, right? No, it is not happening in this. If you
run it for n is equal to 10, it will be very long. You will see that one thing is repeating 10
times."
    },
    {
      "start": 1544.34,
      "end": 1571.63,
      "text": "You are repeating the same thing 20 times. This is why DP is applied in this
question because you are doing overlapping subproblems. How to apply DP in this? I will tell
you that too. It is a 5 minute work. Now, the main part. How to apply DP? So, DP is not
applied anywhere. There is no problem that it has to be applied. See. First of all, write the
recursion. We have written the recursion of Fibonacci. It is the upper code. We have just
compacted this line to tell you. It doesn't matter. Write the recursion. After that,"
    },
    {
      "start": 1571.79,
      "end": 1576.66,
      "text": "Don't touch the base case. Leave the base case as it is."
    },
    {
      "start": 1576.66,
      "end": 1605.97,
      "text": "Don't touch the lower part too. After the base case, don't do any calculation.
Find a line in the middle, in the base case and in the rest of the part. Whatever is changing
in the function, the input in this function, the input that is changing in this function means,
 what is the input of this function? int n. n was changing again and again. Sometimes it was 4,
 sometimes it was 3, sometimes it was 2. If there was an array question in recursion, what
changes in that? The index changes again and again. Did you read recursion?"
    },
    {
      "start": 1605.97,
      "end": 1632.46,
      "text": "i is equal to 0, i is equal to 1, n is equal to 0 whatever thing is changing
and whatever input you noticed that this input is getting solved again and again what to do
with that? on top of your function on top of your function in global what to do? unordered map
means hash map you can take array also but we take unordered map int"
    },
    {
      "start": 1632.46,
      "end": 1656.16,
      "text": "I N T. Keep it's name as DP. Keep it's name as DP. Is it over? Yes. What is
this? This is your diary. What will we write in this? Whatever input will come, first of all
you will have to check it. First of all you will have to check it. See, if DP dot find and not
equal to DP dot end."
    },
    {
      "start": 1656.16,
      "end": 1685.71,
      "text": "That's why we say watch the HashMap video. There is a video in my playlist. One
video to master HashMap. It's a 30-minute video. It will be clear in HashMap. What does this
line mean? That in your DP, N is written. Something is written with N. It means that you have
already solved this N in your lifetime. After that, don't do anything. Return DP of N. Return
DP of N. You do it with an array. Do it with something. What is its purpose?"
    },
    {
      "start": 1685.71,
```

```
      "end": 1715.31,
      "text": "If you know that this problem is already solved, if it is written in your diary,
 then don't go ahead in recursion. Directly from here, do return dp of m. If not, then you
will have to do it in the last. You will have to do it in the last. But, now you have to write
it. Suppose, for the first time, n is equal to 3 came. It was not written in your diary. No
problem. But when you are solving the answer of n is equal to 3 here, before the answer,"
    },
    {
      "start": 1715.31,
      "end": 1742.61,
      "text": "Before returning the answer, before returning the answer, let's erase this line
a little bit. Before returning the answer, first do your work, do your work, if it is not
written in your diary, do your work, see. Int, suppose we take out a1, fun of n minus 1, okay,
it came out. Int, a2 is equal to fun n minus 2. Make your answer final as well. Int, answer is
equal to"
    },
    {
      "start": 1742.61,
      "end": 1761.92,
      "text": "a1 plus a2. Now, you had only one thing to do, which was to return this answer.
Before returning it, write it in your dp. Here it is. After that, return this answer. Here it
is. This is your dp code. What are we doing in this? Look. First of all, you get n."
    },
    {
      "start": 1761.92,
      "end": 1782.24,
      "text": "Let's run the base case as it is. You have a hash map named dp above. What does
it do? It writes the answer of every n. It remembers the n you solved once. Very good. After
the base case, you check if it is already in your dp. If it is already in your dp,"
    },
    {
      "start": 1782.24,
      "end": 1811.23,
      "text": "You return its value from there. That means, suppose when you solve n is equal
to 2 for the first time, it will not happen. But when you solve n is equal to 2 for the second
time, you will know that I have this. Without all this drama, return it. If you don't get it,
suppose you are solving that n for the first time. This n is equal to 2 is being solved for
the first time. Now the question which is being solved for the first time, where will the
answer be from? No problem. Solve it. Keep the as it is recursion below. Before returning the
answer,"
    },
    {
      "start": 1811.23,
      "end": 1835.74,
      "text": "Write this line. What is this? This tells your DP to remember this. Remember
this. If it comes next time, we will look at it and write. Remember this. This is the DP code.
We will add 2, 3, 4 lines. Any recursion code will become the DP code. What is this thing
called in English? Memoization. Memoization spelling is wrong. Memoization means to remember."
    },
    {
      "start": 1835.74,
      "end": 1851.52,
      "text": "If you already know the answer to any question, then look at it in your diary
and return it. If you don't know, solve it for that time once and then write it in your diary.
It's a simple game. If the tea vendor doesn't know about a tea, it's okay, calculate it."
    },
    {
      "start": 1851.52,
      "end": 1866.06,
      "text": "After that, write it on the board. Later, when someone asks for tea, look at
the board and tell them. This is the concept. There is no such rocket science. There is
nothing like this in DP. This is the concept. So go and submit the Fibonacci number. And run
it and see. And meet us in tomorrow's lecture."
    }
  ]
}
```


12. transactions/t_20.json


```
{
  "video_id": "9a4A6CbrLCo",
  "title": "The Only Video You Need TO Master HASHMAPS | DSA Patterns Course Cpp JAVA Python",
  "language": "en",
  "model": "large-v3",
  "duration": 1878,
  "segments": [
```

```
    {
      "start": 0.0,
      "end": 22.86,
      "text": "Welcome everyone. Before starting, if you have read the HashMap or not, it
doesn't matter. If you haven't read it, then it's a good thing. It's God's grace that you have
come here. If you have read it, then you haven't read it in this way. This is my guarantee.
And let's start. Those who have come to the channel for the first time, you can see it in the
description. Read it from the first video. You can also read it from here. You can watch this
demo video that how a teacher teaches and then you can go ahead. Let's go."
    },
    {
      "start": 22.86,
      "end": 45.15,
      "text": "HashMap is a very small, lovely, and useful topic. You guys have been using
HashMap in many questions from the last 40-41 videos. But we haven't studied HashMap yet, so
we thought of covering it. It's lovely because most of the questions on HashMap are easy to
medium. What does HashMap mean? It means the question that only has HashMap."
    },
    {
      "start": 45.15,
      "end": 72.43,
      "text": "There are some questions in which HashMap is applied. That is fine. That
question can be hard or anything. But such questions in which only HashMap is applied, means
to make that question, you should get only HashMap. The number of such questions is not too
much and if it is, then it is easy to medium. Means we have not seen any hard question of
HashMap till date. Means there is not much to do in it. That's why, let's assume it is mostly
easy. In one to two days, this whole topic will be over. But we understand what HashMap is. So
what is HashMap? See."
    },
    {
      "start": 72.98,
      "end": 97.52,
      "text": "To understand HatchMap, take the example of a diary. Assume that you are a
person and you are living your life. You live your life day-wise. I will tell you what is day-
wise, but you live your life day-wise. And you like to write a diary, as many people write it
now. We used to write it when we were kids and then we got used to it. What you do is, at the
end of every day, suppose on 5th December,"
    },
    {
      "start": 97.52,
      "end": 123.09,
      "text": "That day, what was the most memorable thing that happened to you in the whole
day? Suppose, on 5th December, you wrote that papa bought us a bike and gave it to us.
Something like that. The whole story of buying a bike. On 5th December. Okay. Then on 6th
December, you write that today we fell off the bike and my leg broke. Okay. The leg broke.
Then on 7th December, you write that today my papa sold that bike. Because my leg broke."
    },
    {
      "start": 123.09,
      "end": 151.44,
      "text": "on 8th december you write that today we will eat pizza so this is an example ok
now you are writing diary and filling it daily you are writing diary day wise there was one
page in diary and you write the most memorable event of that day in future if it is 20th
december on 20th december you suddenly think that ok now i have forgotten its been 15 days
tell me one thing"
    },
    {
      "start": 151.44,
      "end": 179.18,
      "text": "What happened on 7th December? What happened in my life on 7th December? What
was the best event in my life on 7th December? So, let's open our diary and see, because if
you don't remember, you have forgotten, it's been a long time, your life is very happening, I
mean, you understand that your life is very interesting, a lot of things happen. So, let's
open 7th December and see. You got to know that my father had sold that bike that day. So, you
got to know immediately that he had sold the bike. Did you open the diary? You saw it, okay.
This thing is called HashMap. This exact thing is called HashMap. In this,"
    },
    {
      "start": 179.18,
      "end": 204.13,
      "text": "Now I will tell you what is next, but all these things, 5, 6, 7, 8, December
are your keys and all your events, whatever you wanted to store your data, these are your
values. Some of you must have heard that things in HashMap are stored with key values. Exactly
key value means what happened with me. Now let's move a little further. If you understand this
example, then you have learned HashMap, but"
    },
    {
```

```
      "start": 204.13,
      "end": 228.98,
      "text": "Okay, if you write a diary, then there is no programming, so it's okay. Let's
move forward a little. We will give an example because then it remains in the mind. Okay, look.
 When is the use of HashMap? HashMap is your diary. HashMap has only one use, it reminds you
of things. It helps you to remember things. It helps you to remember things. Not just one
thing, it can help you to remember a lot of things. Means, you write your diary."
    },
    {
      "start": 228.98,
      "end": 258.64,
      "text": "It is not that you can write only one day's data in your diary. You can make
your diary for years. Means diary of 10 years. What happened 10 years ago? Means you can also
ask that what happened on December 12, 2014? So if you have started writing your diary till
then, then you will get it in your diary. So you can open that page of December 12 and see
what happened in my life 10 years ago. Okay? So this can also happen. So he remembers and he
can remember a lot of things. He can remember a lot of data. Not one or two. Many. Means he
can remember lakhs of things. Okay."
    },
    {
      "start": 258.64,
      "end": 286.38,
      "text": "So, what is the work of this? To remember. Okay. But, but, but, whatever data
you write in it, it is not random data. It doesn't mean that you have entered anything in the
diary. It doesn't mean that you have made a picture. You eat pizza and you can't make a
picture of pizza in the diary. No. Then you won't understand. You have to follow a format.
Call it format, call it rule, call it system, call it anything. You have to follow one thing.
What is that thing? You have to store your data in pair. Pair means pair."
    },
    {
      "start": 286.38,
      "end": 307.71,
      "text": "What is that combination? That combination is key and value. Key and value.
Okay. Now, what is key and value? See. What is value? To understand it more easily, value.
Value means whatever data you want to store, you call it value. Whatever data you want to
store, which is your actual data."
    },
    {
      "start": 307.71,
      "end": 335.28,
      "text": "Your actual data, like your bike, your broken leg, your pizza, your jogging,
etc. But that data is either when it happened or any of its identifiers like when it happened,
where it happened, how it happened, by whom it happened, we call these things as key."
    },
    {
      "start": 335.28,
      "end": 357.9,
      "text": "Key means the identifier of your data and you decide it. It is not decided from
the beginning. You decide the key and you decide the value. Value means your original data
means what you want to store in real. Key means you are writing the diary in what way. For
example, if we are"
    },
    {
      "start": 357.9,
      "end": 387.9,
      "text": "We write diary day wise. Means we write diary once in every day. So my diary
will look like this. My diary will look like this. On 5th December something happened. On 6th
December something else happened. So my diary will look like this. But it is not necessary
that you also write diary like this. It is possible that you write diary according to the week.
 It is not necessary that everyone has so much time that they write daily at night. No. It is
possible that you think that nothing happens with me daily. If something happens in a week
then we will write it for a week. So how will your diary look like? Your diary will look like
this. In week 1."
    },
    {
      "start": 387.9,
      "end": 407.55,
      "text": "something happened to me in week 2 something happened to me in week 2 something
happened to me in week 3 something happened to me in week 3 your diary will not be visible day
wise your diary will be visible week wise it is possible that there is another person who
writes a diary once in a month it is not necessary that he writes once in a week he writes
once in a month his diary will look like this this happened to me in January"
    },
    {
      "start": 407.55,
      "end": 434.29,
      "text": "this happened with me in February, this happened with me in March, then his
```

```
diary will look like this. Now look, the data is the same in all, that is, what is written in the data that what happened with me, that is fine. But what are you choosing as its identifier? We are choosing that we write daily, that is, we will write daily diary, then we choose that there will be one event on 5th December, then there will be one event on 6th. You choose the week, you say this happened in the first week of December, this happened in the second week. Someone else chooses the month, he says this happened in January."
    },
    {
      "start": 434.29,
      "end": 457.94,
      "text": "It doesn't matter which day of January it is. This event must have happened in January. The bike was bought in January and the bike was sold in February. He doesn't know which day it was because he writes this. These are his keys. These are your keys. These are my keys. So, keys means identifier. It is not necessary to write in the diary according to the day or time. It is not necessary. Look."
    },
    {
      "start": 457.94,
      "end": 483.22,
      "text": "Diary means anything. You don't have to write the time. For example, you are a shopkeeper. You can make your diary in such a way that you have to borrow money from someone. You can also do this. You have to take Rs. 10,000 from Ramlal. You have to take Rs. 500 from Shyam. You have to take Rs. 500 from Rahul."
    },
    {
      "start": 483.41,
      "end": 502.19,
      "text": "Now, what is your data at this time? Your data is that how much money you have to take back, 10,000, 500,000, but what are your identifiers? Your identifiers are these people, from whom you have to take it back, from whom you have to take it back, then you have to take 1000 from Ramlal, from Shyam, so whatever the key value is,"
    },
    {
      "start": 502.61,
      "end": 532.38,
      "text": "Key value. Key value depends on you. You can tweak it program by program, use case by use case. But remember one thing, insert will be like this only. Meaning, if you will store anything in HashMap, then it should be in the form of key and value only. What is key, what is value, it depends on you completely. You can do it in any way. Okay. This was about storage. This was about storage. That how you will store or how you will write. Very good. But, "
    },
    {
      "start": 532.38,
      "end": 562.08,
      "text": "What will happen just by writing? I mean you will write but what is its work? Why will we write? We are not interested in writing something useless. Why will we write? We will not write. What is its work? Work, work, work. Okay. We had told you some time ago that you can write your data in the form of key and value. There is no other way to write data. You can write your data in the form of key and value. Okay. What is its work? If you ever have to ask any such question that of any particular key, of any particular key,"
    },
    {
      "start": 562.08,
      "end": 589.3,
      "text": "What is the value? What is the value? The answer to this question can be given to you by HashMap. The answer can be given. And only this question can be answered. Only this exact question can be answered. I mean, there are some other small things. But mainly, what is the value of any particular key? What is the value of any particular key? The answer to this question can be given to you by HashMap."
    },
    {
      "start": 589.3,
      "end": 619.25,
      "text": "You will know how the key will be decided as you entered. For example, in the above diary, you can ask what happened on 6th December? What happened on 6th December? What happened? This answer will be given to you in the hash. But you can't ask what happened on 6th December in this diary. Why? Because you didn't write it according to the day, you wrote it according to the week. In this, you can ask the question, what happened in week 2? It can tell you. In this, you can't ask the week, you can't ask the day, you can ask in any month."
    },
    {
      "start": 619.25,
      "end": 646.75,
      "text": "What happened in June, he can tell you. So, as you write, see, this is a very natural thing, that whatever you write, you will read it. It is not like you write something else, that is, you write a film script and read Mahabharata, it will not be like that.
```

```
Whatever you write, you will read it. So, as you write, you can ask the same question. So, this is a particular question. This is a particular question that tell us the value of a particular key. So, he will tell you immediately. There are 2-3 more things."
    },
    {
      "start": 646.75,
      "end": 667.81,
      "text": "How many entries do you have? What is the meaning of how many entries? What is the number of key value pairs? Pairs are not total. For example, if your hash map is K1, its value is V1. K2, its value is V2. K3, its value is V3."
    },
    {
      "start": 667.81,
      "end": 696.29,
      "text": "This is your diary in which you can see money. If you ask it how many entries are there in it, or if you ask in words, what is your size, it will tell you how many pairs are there in it. How many pairs, not how many values, three pairs. So, what will the size tell you? How many entries are there? How many pairs are there? How many key value pairs are there at this time? So, it can tell you. Okay. You can ask one more question. You can ask one more question that any particular"
    },
    {
      "start": 696.29,
      "end": 722.5,
      "text": "any particular key means any key you take the name of any key whether it is there or not whether it is there or not means if there is a diary then do you have the date of 30th December or not it will immediately tell whether it is there or not yes or no it will tell yes or no it will tell it is called finding it is called find whether the date of 10th December is there or not so it will tell whether the date of 10th December is there or not"
    },
    {
      "start": 722.5,
      "end": 751.87,
      "text": "So this was an overview of what is a hash map, what are the powers of a hash map, what are the major powers of a hash map. What is the value of a key, you will have to tell the name of the key. Tell me the value of December, is it the value of December or not. So he will tell you. How many entries means, how many key value pairs are written, this is a very simple question. And is there any particular data for the key, means did we write anything on 10th December? If we had written, then he will tell you, if not, then he will not tell you. Leaving these three, leaving these three questions,"
    },
    {
      "start": 751.87,
      "end": 771.98,
      "text": "You don't have any other power in the HashMap. You can't take any other work from the HashMap. For example, sometimes you will feel like using the HashMap. No, no, no, don't do it. For example,"
    },
    {
      "start": 771.98,
      "end": 794.98,
      "text": "Assume that you have a hash map. If we ask a random question, especially in the prefix sum, Assume that you write a diary according to the day. For example, you write like this, 5th December, 6th December, 7th December, like this. If we ask a random question that, do one thing, you tell me what happened before 10th December. 10th December."
    },
    {
      "start": 794.98,
      "end": 815.82,
      "text": "What happened before? We don't know what happened. You can ask day by day. What happened on 7th? We will tell you. What happened before 10th December? We don't know. What happened after 10th December? We don't know. You will have to ask the exact question. You will have to ask the exact question. I am not that smart to tell you what happened after 10th December and what happened before and what happened in between."
    },
    {
      "start": 815.82,
      "end": 845.81,
      "text": "What happened between 10th and 15th December? We don't know. We also don't know whether it was on 10th or 15th December. So how will we tell you what happened in between? So these are all stupid questions. HashMap can't help you in this. Okay? There are some more questions that you might think are good but they are also stupid questions from HashMap's point of view. What can we say now? What was the last entry?"
    },
    {
      "start": 845.81,
      "end": 868.82,
```

```
      "text": "you don't know, you don't know the hash map because the hash map that we are reading is not sorted like a diary you must be thinking that if we write a diary then it will open the last page of the diary and see that no no no this is not sorted if you enter an entry once then that entry must have been written on the diary at one place now if you say then it will find it and give it to you immediately but it doesn't know if it is written on the last page or on the first page so you can't ask which entry was last"
    },
    {
      "start": 868.82,
      "end": 896.06,
      "text": "What was the first entry? I don't know what was the first entry. I will tell you entry by entry. I mean, I will tell you day by day. What was the first entry? What was the last entry? What was the most entry? What was the least entry? What happened between these two entries? How many entries are there between these two entries? How many entries are there less than 20th December? I don't know. I won't be able to answer all these complex questions. Yes."
    },
    {
      "start": 896.06,
      "end": 917.04,
      "text": "Ask them what happened on the 15th. Tell them whether you have the 15th or not. They will tell you whether you have the 15th or not. They will also tell you what happened on the 15th. So, for all these stupid questions, we are saying stupid because we don't know the answer to the HashMap points. So, whenever you feel that the question is complex,"
    },
    {
      "start": 917.04,
      "end": 946.1,
      "text": "you are not asking about the particular key, you are asking some weird questions that tell me there are two keys, no, he doesn't know all that, this is just a tool, this is a diary, you can read what is written in it, what is written is not analyzed, means there is no software running in it that will analyze what is written for you, no, you can read what is written, it is a simple thing, you can't do more analysis and more of that, okay, so all these questions don't work in this, let's go, so this is a basic one, now let's come to when to use the hash map, hash map,"
    },
    {
      "start": 946.1,
      "end": 968.35,
      "text": "When to use it? It's a simple thing. Whenever you feel that you have to remember old data in some question, then you have to use HashMap."
    },
    {
      "start": 968.35,
      "end": 996.7,
      "text": "You have to remember the old data, but to remember it, what kind of data do you have to remember? I mean, there can be a lot of things in the old data. You can try to remember the graph data. You will get the intuition that you have to remember the old data and whatever questions you ask, whatever questions you ask, I mean, why are you remembering everywhere? Will you need it? If you need it, then think about it. You always imagine that we will ask a question. We will ask a question from HashMap to tell us its answer. Whatever question you ask,"
    },
    {
      "start": 996.7,
      "end": 1022.18,
      "text": "It should be an exact question that what is the value of a key? What is the value? If that exact question is not that whether a particular key has a value or not or whether there is a particular key or not. So if these two questions are not exact in the same way, if your questions are more complex than this, like we saw in the prefix sum, remember in the prefix sum,"
    },
    {
      "start": 1022.18,
      "end": 1043.92,
      "text": "the sum less than or equal to k. You used to ask how many sums have come before you that are greater than k. You don't have any idea about that. If you ask if there is a particular sum or not, it will tell you. Don't ask how many sums are greater, how many sums are greater and what is the sum's root. Don't ask that. If your questions are not one of these two simple questions, then you don't have to use HashMap. If your questions are more complex than this,"
    },
    {
      "start": 1043.92,
      "end": 1070.93,
      "text": "then you will have heap or stack, we will see that later but if your question is so simple and you think that you have to remember some things then you will understand how to remember by inserting the value of a particular key and your question is simple enough that
```

```
yes, tell me the value of a particular key then you can use HashMap with an open mind if the question is complex then don't use HashMap at all so this was a small"
    },
    {
      "start": 1070.93,
      "end": 1101.6,
      "text": "Intro. Let's quickly see how to write this for those who don't know. First of all, C++. In C++, the hash map is called unordered map. There is a map too but it is ordered. It is of a different type. It is not that useful. You can use a lot of things better than sorted maps. But what we are interested in is that the real hash map is unordered. That's why we said it is unordered. We will tell you the meaning of unordered. So, let's write it like this. Unordered map."
    },
    {
      "start": 1101.6,
      "end": 1127.39,
      "text": "Now we will tell you that you insert things on key and value. So there will be a data type of key, that means what you want to store in key and there will be a data type of value. One more thing, you can't do this that once you decide what the format of my key is and what the format of my value is, then don't go back and forth from it. You can't write one page in the diary that an incident happened with me on 6th December."
    },
    {
      "start": 1127.39,
      "end": 1155.07,
      "text": "The next event you write is that this happened to me in January. No, no, no. Either you write day-wise or month-wise. Think about it first. If you have written 6th December, then don't write January-February. Then write that this happened to me in 2014. What does this mean? Then how will you find out? Then how will you find out? What are you trying to do? First you write that this happened on 6th December. Then you write that this happened in January. Then you write that this happened in 2014. Tomorrow you will write that this happened to me at 5 o'clock. It's not like that. There is no such thing."
    },
    {
      "start": 1155.07,
      "end": 1181.12,
      "text": "It is not a waste to write anything in it. Then you will make a drawing. So you don't have to do all this. Think about it with an open mind once. What do you want in the key? Do you want a day in the key? Do you want hours? Do you want a year? Do you want a person? Yes.  You write one entry that this happened to me on December 6th. Write the next entry that Ramlal took 5000 rupees from us. What is this? So make a different diary. If you write everything in the same diary, then how will you find him?"
    },
    {
      "start": 1181.12,
      "end": 1207.46,
      "text": "It's a natural thing to write all the subjects in the same diary and then find out how to do it. So, think from your mind what to write in the diary and what should be in the key and what should be in the value. That's why it is said that the key is our data type. Now, it is not necessary that if you are writing an integer, then there should be an integer in the value as well. No, no, no. Write an integer in the key. Write a string. What does integer mean?"
    },
    {
      "start": 1207.46,
      "end": 1234.82,
      "text": "write any number that this happened to me on 10th, write string, I got 5000 from Ramlal, so Ramlal became a string, you put a whole vector in it, a whole vector, a whole array, any data structure, whatever you want, put that data structure in it, you go ahead and put complex complex data structure in it, you can put another hash map in the key in the hash map, that is also allowed, means anything, anything, but it should be uniform, it should be one, it should not be that you"
    },
    {
      "start": 1234.82,
      "end": 1258.74,
      "text": "make an integer key and then another string. No, don't mix it. If you keep integer, then keep integer. And it is not necessary that if key is integer, then value is integer. In value also, write anything. In value also, write anything. In value also, write date, integer, string, character, float, complete vector, complete set, and write a new hash map. Anything. Whatever you want. But be consistent."
    },
    {
      "start": 1258.74,
      "end": 1289.06,
      "text": "you don't have to do this that on 6th December there was pizza in my house that means write a string like this and then on 7th December make a drawing here so this is a bad
```

```
thing if you want to write a string then write a string if you want to make a drawing then
make a drawing in all of them if you can do this then make a drawing in all of them every
December means drawing on 6th December drawing on 7th December drawing on 8th December drawing
on 8th December drawing on 8th December drawing on 8th December drawing on 8th December
drawing on 8th December drawing on 8th December drawing on 8th December drawing on 8th
December drawing on 8th December drawing on 8th December drawing on 8th December drawing on
8th December drawing on 8th December drawing on 8th December drawing on 8th December drawing
on 8th December drawing on 8th December drawing on 8th December drawing on 8th December
drawing on 8th December drawing on 8th December drawing on 8th December drawing on 8th
December drawing on 8th December drawing on 8th December drawing"
    },
    {
      "start": 1289.06,
      "end": 1317.73,
      "text": "That's why whenever we make a hash map like this, we have to give two data
types. The first data type is of key and the second is of value. For example, you want to make
it so that both are integers. So you do this. After that, you have to give the name of the
hash map. After that, everything becomes simple. If you want to write an entry, then don't do
anything. You have to write the value of the key. You will always write the entry according to
the key."
    },
    {
      "start": 1317.73,
      "end": 1341.17,
      "text": "Suppose you write the value of f1 is going to be 10. You can write your entry
like this. If you want to increase the entry of any key, then you simply write f1++. What will
happen with this? If the entry of f1 is present, suppose if 10 is already present in 1, then
this 10 will increase to 11. If 1 is not present, then 1 will be inserted with 1 value. So by
default,"
    },
    {
      "start": 1341.17,
      "end": 1365.12,
      "text": "The hash map assumes that the default value of the key is 0. It assumes the
default value. If you have done F1++, then suppose 1 becomes 11. In the next line, if you do
F2++, then 2 is not written in your diary. No problem, there will be no error in this. It will
write 2 and put the value 1. Why? Because it will assume that the previous value was 0. Now if
you have done it, then your value will be 1. So this will do your work. So this is what
happens."
    },
    {
      "start": 1365.12,
      "end": 1392.85,
      "text": "If you want to erase any value, again, you have to tell the key to erase, you
have to tell the particular key to erase, that erase this key entry. So, you can say that f.
erase . What will happen with this? It will lock the entire entry. It will end. Now, if you do
f.size, f.size, then it will say 1. Why? Because one pair is present. One pair is present.
This is the hash map. We said that there are very simple functions in this."
    },
    {
      "start": 1392.85,
      "end": 1419.71,
      "text": "you don't have to remember much, in C++ you don't have to remember anything at
all, in C++ whenever you have to insert a key, then there is no need for any trick, f of, f is
your name, whatever your name is, f of any particular key, whatever value you want to give,
give it here, if you want to erase someone, then f.erase for key, if you do this, it will be
erased, yes, if you want to see if there is any key or not, its syntax is a bit difficult,
look at its syntax carefully,"
    },
    {
      "start": 1419.71,
      "end": 1451.5,
      "text": "If you want to check if there is any particular key, i.e. if there is any entry
of that key, then you have to write this. If f.find is equal to f.end, this means that it is
not there. This means that it is not there. Equal to f.end means that it is not there. And if
f.find is not equal to f.end, this means that it is there."
    },
    {
      "start": 1451.5,
      "end": 1480.45,
      "text": "we know this syntax is a bit difficult but what we can do now this is it, you
will remember it while writing while writing everything happens so you can do this so we think
this is enough and we have told you that you can do any operation on f key you can do minus
minus on f key you can do plus plus on f key if you ever need its value to print then you can
do int val is equal to f of any of your key"
    },
    {
```

```
      "start": 1480.45,
      "end": 1510.86,
      "text": "Yes, you have to pass that key, that which key's value, means there is no random value like this. You have to tell the key that which particular key's value you want, then you will get it. By doing this, you can give this, okay? It's a good thing. So, this is all there is in C++, right? So, we think it's done. Yes, one more thing. It is not required much, but sometimes it is required. So, if you want to loop on the hash map, for example, your hash map sometimes looks like this, that 1 is 10, 3 is 5, 2 is 4."
    },
    {
      "start": 1510.86,
      "end": 1541.41,
      "text": "and 4 is 20 again again again this is your hash map but it is not necessary that it is sorted it is not necessary that entry of 1 will come first then of 2 then of 3 then of 4 this can be done in any order you don't know so when you will do loop then you have to write the loop like this for auto i equal to f this is your hash map you can remove key like this int key is equal to i.first i.first int val is equal to"
    },
    {
      "start": 1541.41,
      "end": 1560.26,
      "text": "i.second and then you can do anything with the key value so what will happen with this? first of all let me tell you according to this hash map first of all you can get any key value pair but all of them will be available all of them will be available in which order? this is not certain so you will get 1 such 10 which is the key 3 and 5 you will get this"
    },
    {
      "start": 1560.26,
      "end": 1588.62,
      "text": "Then you will get 2 and 4, then 4 and 20, you will get this. We have written this in this order because we are assuming that my hash map will be like this. These four can be any order. It is not necessary that whatever you have inserted, it will be the same or whatever you have inserted, it will be the opposite. Anything can happen because what is this? We are saying it again and again. Unordered map, its name is unordered, see. Its name is unordered map. Unordered means it has no order, so its name is unordered map. Let's go. See in java, in java also it is almost almost same, nothing much happens. In java, it is almost almost same."
    },
    {
      "start": 1588.62,
      "end": 1621.1,
      "text": "in that you write hashMap hashMap sorry, M is capital so in that you write hashMap same, what do you want? if you want integer integer and integer give its name f equal to new hashMap like this this is syntax, no one can explain you anything in syntax you have to remember"
    },
    {
      "start": 1621.1,
      "end": 1649.94,
      "text": "Now if you want to insert a key or update the value of that key then you can do this f.put So what will happen with this? One key will be inserted whose value will be 10 If you want to change it again, then write again 1 and 20, what will happen with this? New thing will not be inserted, this value will be changed to 20 The method of finding is easy with C++, so in this you write if"
    },
    {
      "start": 1650.16,
      "end": 1678.06,
      "text": "f.contains key 1 so in this you get true or false if you want to remove something then again you have to tell the key which key you have to remove f.remove there is erase and here is remove so suppose 1 what will happen with this? this entry will be removed this entire entry will be removed"
    },
    {
      "start": 1678.06,
      "end": 1707.47,
      "text": "And there are small things like f.size. This is also in C++ and Jaon. What does it give you? Number of pairs. Number of pairs. Remember. Number of pairs means, in one way, total number of unique keys. Yes, one more thing. When you make a hash, then there are keys here. 1, 2, 3, 4. Suppose this is 10 times, this is 20 times, this is 30 times, and this is 40 times. This is how something is made. But, all these keys are unique in this. Keys are unique. Keys are..."
    },
    {
      "start": 1707.47,
      "end": 1736.18,
```

```
      "text": "Unique. Duplicate entry is not possible. It means that two values of one key
are not possible. It is not possible that one is 1 and 10 and another entry is 1 and 20. No,
no, no. This is not possible. Why is it not possible? Because when you ask for the value of f ,
 what will you get? Will you get 10 or 20? You don't know because there are two. How will he
know what to give? That's why keys are unique. Values can be duplicate. It means that the same
value of two keys can be there. The value of 1 can also be 10 and the value of 2 can also be
10. It is absolutely correct. But"
    },
    {
      "start": 1736.18,
      "end": 1765.33,
      "text": "Two keys can't be the same. This is not allowed. This is absolutely allowed.
That's why keys have to be unique. That's why when you write F1 is equal to 10, then it makes
it 1 or 10 if it doesn't happen before. After that, when you write F1 is equal to 20, then 1
doesn't become 20. It doesn't make a new entry. This 10 becomes 20 mid. So now this one has
disappeared. You have updated it. So remember that keys will never be duplicate. Keys will
always be unique. Keys will always be unique. Unique. Unique. Yes."
    },
    {
      "start": 1765.33,
      "end": 1794.29,
      "text": "that's why you can understand f.size in that that what is the number of keys in
that number of keys because it tells the number of pairs there is a key in every pair which is
unique so it will tell you that what is the number of keys so we think that you must have
understood let's see python, python is more simple there is no data structure in python as
such so in python you just do f is equal to like this after that you have to put so just f1 is
equal to 10 then f2"
    },
    {
      "start": 1794.29,
      "end": 1822.88,
      "text": "is equal to 20 then what you have to do is you have to find if 1 in f what does
this say? this key, is it in this map or not? if you want to delete then delete f of 1 what
will happen with this? it will be removed or erased which we read while going up if you want
to tell size then len of f this tells you the size what does this tell you? size so this is
python, python is very easy because"
    },
    {
      "start": 1822.88,
      "end": 1851.5,
      "text": "Key value data structure is not a concept. In Java and C++, it is important to
define the key value data structure. And then work according to that. It is not that it is
different. If you define integer integer. If you do f of 1 is equal to 10, then it works. But
if you go ahead and do f of a is equal to 20. Then this will be wrong. Because you said that
the key will be integer and the key will be character. So this will be wrong. Pay attention to
this. You will understand it as it goes on. So this is a complete."
    },
    {
      "start": 1851.5,
      "end": 1874.99,
      "text": "The intro of HashMap is over, only the questions are left. From tomorrow, there
will be 3-5 questions. By tomorrow or day after tomorrow, it will be over. Those who are still
here, tomorrow an important video is going to come. Watch tomorrow's video for sure because we
are doing some changes in this whole sheet. We are removing some questions and adding some
questions. So watch tomorrow's video for sure. Don't miss it. And let's meet tomorrow, right?"
    }
  ]
}
```


13. transactions/t_21.json


```
{
  "video_id": "9wc8HZH_sh4",
  "title": "DSA PATTERNS Episode 16 | Sliding Window Pattern: Amazon Interview Question",
  "language": "en",
  "model": "large-v3",
  "duration": 1193,
  "segments": [
    {
      "start": 0.0,
      "end": 19.01,
      "text": "Before my kids ask me to make a video about studying, they won't watch this.
This is a video about studying. Listen carefully. The new students who are coming, you will
know in this video that we have been shouting and teaching for so many days, why your Stryver
brother, your Lovebubber brother, your college, your college, any college, this man is better
than that."
```

```
    },
    {
      "start": 19.01,
      "end": 46.85,
      "text": "Look at the question on your screen. What is this question? It is a hard
question. The lead code is a hard question. Facebook, Amazon, LinkedIn, Microsoft, whatever
company name you can think of, it has been asked in all of them. This question has given a
hint of four ways. It means that it is very hard. Now look, the problem is that if you have
studied from these three brothers or any of your brothers, if this question is in their course,
 then you can make this question. But if this question is not in their course, then you will
not be able to make this question."
    },
    {
      "start": 46.85,
      "end": 72.66,
      "text": "and always remember one thing that in interview questions don't come from
anyone's course or sheet because all interviewers are not taught by us so all are not taught
by them that's why we teach you patterns or templates what do we teach? template or rules once
you learn these rules then any question will be made you don't need to write question by
question because how many questions will you write? 500, 1000, 10000, 20000 there is no limit
to it come"
    },
    {
      "start": 72.66,
      "end": 101.97,
      "text": "Let's open this question and see how many fake and stupid questions are asked
in any company like Amazon. What is the question saying to you? You are given two strings, S
and T. Its length is N and its length is M. You have to search in S. What do you have to
search? You have to search minimum window. What do you have to search? You have to search
minimum window substring. You have to search this. But what is its condition? There must be
some condition."
    },
    {
      "start": 101.97,
      "end": 129.38,
      "text": "Its condition is that the string of T is present in this window including
duplicates. Its character is present in this window. It doesn't mean by order. For example, if
T is A, B, C, then you make any window in S. For example, you want to make this window. So,
somewhere A, B, and C should be present in this. May be B is present here."
    },
    {
      "start": 129.38,
      "end": 152.35,
      "text": "It can be A or C. So, B, A, C should be present once. If your string is A, B, C,
 C, then whatever window you choose, C should be there twice. C should be there twice,
wherever it is in the window, in any order. But C should be there twice, A should be there
once, B should be there once. Apart from this, anything else can happen."
    },
    {
      "start": 152.35,
      "end": 181.92,
      "text": "It should happen. And you have to find minimum sub-string of such window. What
to find? Sub-string. Not length. Not length. Not length. Sub-string. Means you have to return
any sub-string of such window. This is your question. Now, my kids, who have read, who have
revised sliding window, Pause it. Think for yourself. The new kids who are watching, come. You
won't know the template or pattern. We will tell you. Any sliding window question, any sliding
window question in the world, is of two types."
    },
    {
      "start": 181.92,
      "end": 211.82,
      "text": "One is max window and the other is min window. This question is of min window.
In any question of min window, the condition in which you have to return the window, if there
is some sum, then it should be greater than or equal to. Greater than or equal to something.
The sum is not necessary. Whatever you ask, it will be greater than or equal to something in
the condition. What is the condition in this question? In this question, in S,"
    },
    {
      "start": 211.82,
      "end": 239.98,
      "text": "A sub-string or a window in which the entire character of T is present. And it
should be present in the same frequency. The entire character should be present. It should be
present. It should be present. It means, the entire character of T should be present. After
that, you can do anything. So, it means that whatever frequency or sub-string will be there,
it should be at least greater than or equal to T."
    },
    {
```

```
      "start": 239.98,
      "end": 267.94,
      "text": "At least it should be bigger than that. It should be full of T. Apart from that,
 anything should be there. Did you understand? So, you are getting the minimum pattern. You
got the condition too. Come. Come, let's see. How many steps were there? What was step 1? What
was step 1? See, we are not telling you all this new. We are telling you all this. You will
see. You will see the last 2 lectures. You will know this flow chart. Okay? You don't know? No
problem. Come, let's tell you in 2 minutes. Step 1 is."
    },
    {
      "start": 267.94,
      "end": 297.12,
      "text": "we have to identify what kind of question it is. So, we are getting to know
that it is a sliding window question. So, if someone has two opinions, if someone has a doubt,
it is not according to me because the substring is asking. It is a string question. A
condition is given. Perfect example of sliding window question. Very good. Let's come to step
2. Is it a fixed or a variable question? Is it a fixed or a variable window question?
Somewhere you are being told to give a substring of length. Either give a substring of length
this much or give a minimum substring."
    },
    {
      "start": 297.12,
      "end": 326.82,
      "text": "As soon as I asked you to give minimum or maximum, it became a question of this
variable. Very good. Step 3. We have to play here. What is the information of your window? And
how will you store it? Information means, whatever window you are holding, whatever window you
are holding, what is this window? What will this window tell you? What do you want to know
from this window? So, what do we want to know from this window? Is the entire character of T a
frequency or not?"
    },
    {
      "start": 326.82,
      "end": 353.68,
      "text": "For this, what will we store the information of this window? Frequency.
Frequency means HashMap, HashMap or Array, anything. Means, all the characters in this window
have a frequency. Only then you will know, right? For example, if a window of S is made in
which A is twice, B is once and C is twice. Your T was of this type in which A was once, B was
once and C was once."
    },
    {
      "start": 353.68,
      "end": 379.31,
      "text": "So, will these two match or not? So, how will we know if it will match or not?
See, what was your T? Your T is this. Means, A was in T once. So, A should be in this once. Is
A more than 1? Yes, it is more. So, very good. This matched. B was in this once. So, B should
be in this window once or more than once? Is it more than or more than 1? Yes, it is. Very
good. C?"
    },
    {
      "start": 379.31,
      "end": 407.82,
      "text": "T has 1 bar which means in S window also, C should be 1 or more than 1? Is it?
Yes it is. So what is your information? It is correct. When is information wrong? For example,
let's take a window where A is 1 bar and B is 1 bar and C is not. And your T is given that A
should be 1 bar, B should be 1 bar and C should be 1 bar. So now see, your T says that A"
    },
    {
      "start": 407.82,
      "end": 433.74,
      "text": "Once or more than once, that is very good. Once or more than once, that is also
very good. But your T says that C should also be more than one or more than one. But what is
your C? C is zero. If C is not there, then C is zero. If C is not there, then what does it
mean? C is zero. C has not come even once. So this information is that what happened to you?
It is wrong. It is wrong. Now see. Now see. How will you know if it is wrong or right?"
    },
    {
      "start": 433.74,
      "end": 460.26,
      "text": "It is very simple. What are they doing in every question of the string? If you
want to find any frequency and you don't even know the hash map. What you don't know is that
you don't even know the hash map. What is simple is that take an array of 256. How much size?
256. Why 256? Because my birthday is on 256. No, no, no. In C++, the total character is 256.
In any programming language, the total character is 256. You take an array of 256 size. Make
all of them 0."
    },
    {
      "start": 460.26,
```

```
      "end": 483.07,
      "text": "When you want to include a character's frequency, increase it. When you want to remove a character's frequency, decrease it. By doing this, your frequency will be created. If it is an array, then it is a simple array. No hash map is being used in it. Now see. Now see. Now see. What was the next? Information. Is it right or wrong? Information. Is it right or wrong? Is it right or..."
    },
    {
      "start": 483.07,
      "end": 507.66,
      "text": "This is wrong. How will we know this? Because this is a major part of our flowchart. How will we know this? What did we do last time? For example, there will be a window of S and T is ABC. T is ABC. T will be given to you. So first make this thing of T. What should you make? Make an array of 256. Make an array of T. So what will happen? In this, A, B, C, on any index, there will be 1."
    },
    {
      "start": 507.76,
      "end": 534.7,
      "text": "Okay. Now suppose your S has a window in which A is there, B is also there, but C is not there. Now this information, now this window, now this window, and anything else can happen. Anything else can happen. There is no difference. D can be 10 times, E can be 15 times. There is no difference. Where is the difference between us? This. It should be greater than this. We will check from the character in T. It should be greater than that. You can keep anything else. Now see."
    },
    {
      "start": 534.7,
      "end": 558.35,
      "text": "How will we know if it is the right window or the wrong window? Check it out. What is the question telling you? A, B, and C, the characters in T, their frequency, in which? In the window area. In the window area, should it be equal or greater than that? So check it out. If A's frequency is 1, then is it equal or greater than 1? Yes, it is."
    },
    {
      "start": 558.35,
      "end": 585.55,
      "text": "If B has 1 here, then is it greater than or equal to B? Yes, it is. But if C has 1, then is C greater than or equal to 1? No, it is 0. So what happened to this window? This window is wrong. It doesn't matter whether D has 10 or E has 15. We don't need that. We just have to match the frequency of the characters in T. Okay? Now you know how to decide whether the information is right or wrong. If you know this much, then come now. We will solve the question in 2 minutes."
    },
    {
      "start": 585.81,
      "end": 615.66,
      "text": "Minimum window. What was the flowchart of minimum window? What was the flowchart of minimum window? From where did the high go? Those who are my students, pause it and make it yourself. We have cleared it for you. From where did the high go? From 0 to N. After that, what did we do? Remember yesterday's lecture. Those who are coming directly after seeing the day after tomorrow, don't watch it. Watch yesterday's lecture. Yesterday's lecture. In yesterday's lecture, the minimum flowchart was told. In the day after tomorrow's lecture, the maximum flowchart was told. The same flowchart will be used in every thing in the world. Come on. What did we do?"
    },
    {
      "start": 615.66,
      "end": 643.89,
      "text": "What to do after this? What to do before this? You have to include high in your information. Where to include high? How will you include it in your information? You will increase the character of high in the array. It's simple. Now, as long as the information is correct, correct, correct, not wrong. Those who are watching the day after tomorrow, they must be thinking that they are saying the opposite. Watch tomorrow's please."
    },
    {
      "start": 643.89,
      "end": 673.86,
      "text": "till the time the information is correct, we have to do two things. First of all, we have to write this in our result. Why? Because we want this information to be correct. We have to write this in our result. And the next thing we have to do is, we have to remove the low from our information. We have to remove it from our information and increase the low. This is all we have to do. We have to close the high and return our result. This is the pattern of any minimum window substring."
    },
    {
      "start": 673.86,
```

```
      "end": 697.46,
      "text": "Now, let's see how we will do this. First of all, we will make two arrays. We
will make two arrays. We will make two arrays of 256. We will make two arrays of 256. Why? One
array is for your S window and one array is for your T. It is for T. Now, suppose this is your
T array. So, give it a name. Need. Needed. Needed. This much is required. This much is
required. What we are saying is condition."
    },
    {
      "start": 697.46,
      "end": 727.2,
      "text": "Sum greater than equal to K. So what is K? K is needed. Needed means K. It
should be bigger than this. And what name will you give to this? You give it have name. How
much is it now? I have how much now? Now see. Needed, how will you know how much is needed?
Traverse T. Traverse T. Traverse T. And whatever character is in T, do it in needed. Means, if
T is A, B, C, then first A, do A from 0 to 1. If B is there, then do B from 0 to 1."
    },
    {
      "start": 727.2,
      "end": 755.47,
      "text": "If there is C, then make it 1 from 0. If there is another C, then make it 2
from 1. First of all, make your needed. Your needed is made. Now come, we will start our
sliding window. Write, for high is equal to 0. High less than n. This is nothing new. We are
doing the same thing again and again. First of all, what we have to do? We have to take high
inside. Take high inside. What is the name of your window? It is have."
    },
    {
      "start": 755.47,
      "end": 784.38,
      "text": "In half, who has to be taken in? High has to be taken in. The character of high.
 So your S, high. S, high. And plus plus. High has come in. Now what to do? Till the time the
information is correct, till then you have to do something. Come, write low. While. Now see.
Information is correct or wrong, we have told you how the calculation will be done. We will
write a function of that. Now we will write a function. We will write it now and show it. For
now, assume the name of that function is M. Fun. What is the name of that function? Okay, it
is Saheed."
    },
    {
      "start": 784.38,
      "end": 813.44,
      "text": "The name of the function is sahi. And what were you doing in sahi? You were
comparing have with needed. Needed should be in have. So what will we pass in this? Have and
needed. We will show you now. And close this. Meaning? Till it is correct. Till it is correct.
Till it is correct, what do you have to do? You have to store the result. One more thing. Here
you don't have to give length to the result. Here you don't have to give length to the result.
You have to return the whole substring."
    },
    {
      "start": 813.44,
      "end": 842.35,
      "text": "we have to return the substring to do this, if we had done length then what we
would have done is we would have written the result as result is equal to minimum of result
and length ok so first of all you have to remove the length first of all you have to remove
the length so remove the length first of all we used to remove this every time you remove the
length means of this window which is right now its length is so and so you can do one thing
which is not known in substring we will tell you if if rest greater than l length"
    },
    {
      "start": 842.35,
      "end": 867.28,
      "text": "We used to say that if res is greater than length, then there is a benefit of
updating. Otherwise, there is no benefit. We need minimums. So, first of all, res is equal to
length. Do one more thing. How do you get substring in C++? To get substring, you have to pass
two things. You can get substring of any string. How? You have to pass its starting point and
you have to pass its length of substring. For example, if you have A, B, C."
    },
    {
      "start": 867.28,
      "end": 885.06,
      "text": "0, 1, 2. So, if you write 0 in starting and write 2 in length, then it will
take A and B. So, you will get its answer AB. If you write 3 in length, then it will take A, B
and C. So, its answer will be ABC. Meaning, do starting point and start taking the length's
substring from there. Now, see."
    },
    {
      "start": 885.06,
      "end": 912.46,
      "text": "You got to know the length from here. The length is going to be the result of
```

```
this. The length is going to be the result of the final. And how will the starting point be changed? Do you need the final every time or do you need the minimum substring? For that, you need two things. One is the length of that substring which we will store in the rest again and again. It will come in the last. But, starting point. What will be the starting point? The starting point of any window is always low. The meaning of window is low to high. So, you take one more variable and in start,"
    },
    {
      "start": 912.46,
      "end": 941.34,
      "text": "At the start, store the low. When are we doing this work? When your rest is bigger than the length. That's when we are doing this work because we want the minimum. We used to do the same work in one line. Rest is equal to minimum of length and rest. Just because we have to give a substring in it, then you will have to do it like this. You can do it in any other way according to your thinking. What else do we have to do? Close it. The first work is done. What was the first work? As a result, we took the low one. Now that window. Now what do we have to do? Let's remove the low. Let's remove the low. How will we remove the low? Let's remove the low quickly."
    },
    {
      "start": 941.34,
      "end": 971.18,
      "text": "s low, minus minus, the game is over. Make the low plus plus. That's it. This while loop is over. After that, do we have to do anything? No, we don't have to do anything. This high loop is over. High's fall loop is over. Now, what we used to do? We used to do return result. But, what do you have in result now? Now, you have length in result. There is length in result. And what is there in start? There is starting point in start. So, how do we get substring? We have to do substring return, right? Do it, man. Do it. Return."
    },
    {
      "start": 971.18,
      "end": 1000.14,
      "text": "S. We have to return the window of S, right? Substr. We write Substr like this. First of all, what do we give here? Starting point. And length. Where will the length be stored? In the result, the minimum one. That's it. Hard question is solved now. It took 16 minutes even after giving so much lecture. This is the hard question of the lead code. Hard question with 4 types of hints. Means, it's a good hard question. It's solved. What did we use in this? We used a simple template. Did anything change in this rule?"
    },
    {
      "start": 1000.14,
      "end": 1028.56,
      "text": "Did anything change in this rule till now? What did you do in minimum? Tell us one more step which is extra for you. Oh, what did you do? This is not right. Yesterday you said this, today you did this. No. We are doing the same step. Run high. Take high in result. Till the time information is correct, take it in result and keep increasing low. In the end, return the result to it. Now come, what we told you. Information is correct, how will we find out wrong? Come, now many people have problem in writing code. We will write it. How did we find out right or wrong?"
    },
    {
      "start": 1028.56,
      "end": 1058.53,
      "text": "First of all, what were we doing in right and wrong? What were we passing? We were passing two things. Vector, int, have. Have means the one which is in the window of S. Very good. And what was the next vector we were passing? The one which is needed. What is it? Needed. What was the name of needed? Needed. Okay. What was its logic? That the value in need, there will be any value in need. In need, the value of A will be 1, B will be 2, C will be 3, anything will happen. Rest will be 0. So, the value of the character in need,"
    },
    {
      "start": 1058.53,
      "end": 1083.9,
      "text": "The value of that character in have should be equal to or greater than that. Come, you will get to know by simple loop. What is the size of both your arrays? 256. That is why 256 is known, it is very easy. Come, run it, take 256 and do i++. Now, when will this information be wrong? When will this information be wrong? When the value of have will be less than the value of need."
    },
    {
      "start": 1083.9,
      "end": 1107.46,
      "text": "this is the same, if have a is 0 and need a is 1 then this will be wrong so run an if that if have i less than need i at any time at any time in 256 then return false means it is not correct function name is correct now if this loop is complete if this loop is complete 256 times means that you never caught this condition"
    },
```

```
    {
      "start": 1107.46,
      "end": 1137.34,
      "text": "Your this condition has never been executed. What does it mean that this
condition has never been executed? That the i of have is always equal to the i of need, so it
is greater. This is our condition to be correct. Here, quickly, quickly, return true. Take
this. We have written the right function. What is this? We have written the right function.
Take this. Say right. Say right. What is the name of this function? Say right. Right. This was
being used right here. Where? Here. This was being used right here. See. Same happened. The
question is over."
    },
    {
      "start": 1137.34,
      "end": 1166.8,
      "text": "this question and if you explain this way to an interviewer, then he will be
shocked that what have you told, he is telling it in a systematic way what will be the
opposite of this, that you will be hanged, the striver brother was hanged, there he will ask
that why are you doing this, there he will ask you that why are you doing this, why are you
doing this, from where have you hunged, you will come to know that he does not know and those
who are new kids, they must be thinking that you also hanged, you also said that till the time
the information is correct, keep doing this, see tomorrow's lecture, nothing is hanged"
    },
    {
      "start": 1166.8,
      "end": 1192.74,
      "text": "I have to tell you this quickly, so I have told you. Nothing has been fixed. It
has been told logically that why it shrinks. The logic of both shrinking, in the maximum
window and in the minimum window, why it is different, why it is upside down, everything has
been told. Try it carefully. Enjoy it. And go and submit this question now. It is a very
important question from the perspective of the interview. So, you can joke on your side, but
it is a very important question. Isn't it? Let's meet tomorrow. Tomorrow, we will start the
link. Isn't it? Let's go."
    }
  ]
}
```


14. transactions/t_22.json


```
{
  "video_id": "A3sZWwalzz4",
  "title": "We Crossed 50K! Let's Study Bellman-Ford NOW",
  "language": "en",
  "model": "large-v3",
  "duration": 2093,
  "segments": [
    {
      "start": 0.08,
      "end": 19.44,
      "text": "Before we start the video, we would like to share something with you. You must
be knowing that our 50,000 or 51,000 subscribers has been completed. We have seen a lot of
congratulations and comments. Thank you very much. We haven't celebrated it much because I
don't have a major role in it."
    },
    {
      "start": 19.44,
      "end": 46.06,
      "text": "Whatever happened, we didn't even know when we started that so much will happen,
 it will happen so soon. So whatever happened, it happened because of you, because of God or
because of anyone. So it's okay. But we liked it. Knowing that we are able to help you in some
way means the hardships or the difficulties that we faced while studying, the difficulties
that we faced while learning DSA, you should be a little less than us. This is always my
effort."
    },
    {
      "start": 46.06,
      "end": 73.86,
      "text": "We really like it when people read this kind of pattern and we know some kids
by name who were asking in the comments 2-3 months back what is index, what is time complexity
and what is array. Those kids have read tree, recursion, graph and in the future they will
read DP and in such a small amount of time. If traditionally we would have taught that first
the whole array should be taught by doing 500 questions then we don't think it would have been
possible."
    },
    {
      "start": 73.86,
      "end": 88.43,
```

```
      "text": "So if any one of you can read DSA and finish DSA because of me or because of the channel, then we will be very grateful for that. And whatever has happened, I am very grateful. We don't talk about all this emotionally."
    },
    {
      "start": 88.43,
      "end": 117.71,
      "text": "And if you are on this channel, if you are watching this video and you are not studying the DSA pattern, then you are missing a lot. I will tell you. Because the most efficient and the smallest way and the best way to study DSA is this pattern wise. In a very short amount of time, go ahead a lot. If you want, you can do a course of 6 months, 8 months, 10 months. Keep studying, keep studying, keep studying. It is a brute force technique. But in a very short amount of time, study DSA in a very good and efficient way. For that, you should definitely study the pattern."
    },
    {
      "start": 117.71,
      "end": 147.23,
      "text": "And that's it. Let's move ahead. What is today's topic? Bellman-Ford. So what were we doing in the graph series till now? We have done a lot of things till now. The most recent thing that happened was that we saw the Dijkstra algorithm. What did we see in the Dijkstra algorithm? We understood it and made its question. So what was the main motive of Dijkstra? That you were given a single source, right? And you used to get the shortest path to all other vertex or all other nodes."
    },
    {
      "start": 147.23,
      "end": 170.13,
      "text": "This is what used to happen in Dijkstra. That there is a single source and you get the shortest path till the other vertex. But there was a big problem in Dijkstra which we had discussed. If there were negative weights. If there were negative weights. Either there was a negative weight cycle. Or there was a negative weight cycle."
    },
    {
      "start": 170.13,
      "end": 194.8,
      "text": "So Dixitra didn't work. What does it mean to not work? Dixitra never stopped. He didn't understand that now he has to stop. For example, if someone gave you a Google map that this is Ranchi and this is Patna. Now the road from Ranchi to Patna is of 2 km. How many km is that? 2 km. The road from Ranchi to Patna is of 2 km. And the road from Patna to Ranchi"
    },
    {
      "start": 194.8,
      "end": 224.64,
      "text": "It is of minus 10 km. It doesn't happen, but it can be given in the question. Now in this case, Dixitra never stops. Why doesn't it stop? We have explained this in the previous video very calmly. Look, Dixitra will start by giving 0 to Rachi and giving infinity to it. Now if you relax from Rachi, then your Patna will become 2. Now when you relax from Patna, then Rachi will become 2 minus 10, minus 8. Now Rachi will become less. If Rachi becomes less, then Patna will become less. Patna will become minus 8 plus 2, minus 6."
    },
    {
      "start": 224.64,
      "end": 250.88,
      "text": "If Patna is reduced, then RANCHI will be reduced again. Now RANCHI is reduced to minus 16. In this way, it will keep on moving and it will crash to infinity. Because it will not understand when to stop. It kept on moving and it kept on knowing that RANCHI is also decreasing and Patna is also decreasing. So it kept on moving. Why did this happen? Because there is a negative cycle here. This cycle RANCHI to PATNA to RANCHI is a cycle. What is the total value of this cycle?"
    },
    {
      "start": 250.88,
      "end": 273.31,
      "text": "2 and minus 10, minus 8 km. So, this is a negative cycle. So, in such a case, Dixitra gets stuck or it doesn't work. To solve this disease, to solve this limitation, there is a new algorithm and it is a very easy algorithm. It is a very easy algorithm from Dixitra. What is its name? The name of that man or that algorithm is Bellman-Ford. Bellman-Ford. Bellman and Ford will be two men."
    },
    {
      "start": 273.31,
      "end": 302.4,
      "text": "These are the two algorithms. And don't think that there is a lot of rocket science by listening to the algorithms. It is a very simple and very small thing. Bellman-Ford also does the same thing. What is the same thing? It gives you from single source to all other nodes. What does it give you? It gives you the shortest path. It is a good thing. And the
```

```
limitation of Dijkstra is that Dijkstra sometimes does not work in negative weight and in
negative weight cycle it does not work at all. Bellman-Ford solves that. How?"
    },
    {
      "start": 302.4,
      "end": 316.82,
      "text": "Bellman-Ford also works in negative weights. And if there is a negative cycle
in the graph, Bellman-Ford can also detect it."
    },
    {
      "start": 316.82,
      "end": 345.28,
      "text": "If you are given a map where there is a negative cycle, where you can't find
the shortest path, because it is a cycle, meaning the more you walk, the less it will be. So
where Dijkstra used to get stuck in this, Dijkstra used to walk infinity, Bellman Ford will
detect this and tell you that there is a negative cycle, this can't be done, this can't be
done. Why? Because if there is a negative cycle, then you can't do this shortest path, because
what will happen in that, the more you walk, the less it will be. This is the same thing that,"
    },
    {
      "start": 345.28,
      "end": 369.78,
      "text": "The more you work, the more you like it. So you will keep working and working
and working. So when will you stop? Bellman-Ford detects this thing that stop, stop, stop.
There is a negative cycle in the graph. Its solution is not possible. You stop. So it will not
crash your program by going to infinity like Dijkstra. And this is very, very easy from
Dijkstra. It is very, very easy to understand and to write. You will see that you will know in
the code that where in Dijkstra you..."
    },
    {
      "start": 369.78,
      "end": 383.79,
      "text": "15 types of things had to be done. Make a heap, do this, do that, do that, do
all this. Bellman-Ford is a very beautiful, very cute algorithm. It is a very intuitive
algorithm. So today we will learn Bellman-Ford. After learning Bellman-Ford, we will do a few
questions of it and you will understand Bellman-Ford completely."
    },
    {
      "start": 383.79,
      "end": 413.49,
      "text": "In any interview, if you are given a choice that do this, do that, then you
don't do Dijkstra, you do Bellman-Ford. Bellman-Ford is easy, efficient and intuitive
algorithm. Writing is small, understanding is small, it is a very cute algorithm. So let's go
ahead and start with the example of understanding Bellman-Ford. So look, again we made a
Google map, okay? And you must be thinking that it is very complex. We have made it complex on
purpose to explain our point. Okay? You too have been given a source like Rachi and on other
cities,"
    },
    {
      "start": 413.49,
      "end": 435.7,
      "text": "What Dixitra used to do? He used to do a lot of things. He used to do this and
that. Bellman and Ford came and said, don't do so much work. There is no need to be so complex.
 He said, okay. Now he asked, do one thing. In this map, Rachi is your source, understood.
Which city is the most complex? Complex means, where there are multiple paths to go. So he
said, Delhi."
    },
    {
      "start": 435.7,
      "end": 464.61,
      "text": "Delhi is the most complex. Why is Delhi the most complex? See, if you want to
go from Ranchi to Bangalore, then there is only one path in the whole map. You will have to
take this road from Ranchi to Bangalore because there is no other road in Bangalore. Now see,
there is no other road in Bangalore. There is only one road. Similarly, there is only one path
to go to Patna. Which path? Ranchi to Bangalore, Bangalore to Patna. There is no other path.
There is one path to go to Kolkata. Ranchi to Bangalore, Bangalore to Patna, Patna to Kolkata.
To go to Delhi, if you want to travel from Ranchi to Delhi, there are four paths. Which?"
    },
    {
      "start": 464.61,
      "end": 493.84,
      "text": "First, you can go directly from Ranchi to Delhi. Second, you can go from Ranchi
to Bangalore. Third, you can go from Ranchi to Bangalore, from Bangalore to Patna, and from
Patna to Delhi. Fourth, you can go from Ranchi to Bangalore, from Bangalore to Patna, from
Patna to Kolkata, and from Kolkata to Delhi. So, Delhi is the most complex city. Now look,
there is only one path in the other three. Decision making becomes hard when you have to see
multiple choices. So, in this map, Belmond Wood asked to tell the most complex city."
```

```
    },
    {
      "start": 493.84,
      "end": 522.78,
      "text": "He said Delhi. He said do one thing. Write down the entire route to go to Delhi.
 Now you have told that we will go from here to there. Write down the entire route. Write down
one place and see. So people said okay, let's write. One man said, you can directly go to
Delhi from Ranchi. He said okay. Second, you can go to Bangalore from Ranchi. You can come to
Delhi from Bangalore. He said great, write more. You can go to Bangalore from Ranchi. You can
come to Patna from Bangalore. And you can go to Delhi from Patna. He said very good, write
more."
    },
    {
      "start": 522.78,
      "end": 552.35,
      "text": "He said, we can come to Bangalore from Ranchi. That's a very good thing. Then
we will go to Patna. Then we will go to Kolkata. And we will come back to Delhi from Kolkata.
He said, very good. Is there anything else to write? He said, no, there are four to write. Now
see, the final answer of Delhi can be any of these four paths. The shortest distance of the
path will be the answer of Delhi. So Bellman-Ford said, okay. Now do one thing. Write edges.
Write whatever edge is given in this graph at one place. Like you are given the vector of
edges. So Bellman-Ford said, write."
    },
    {
      "start": 552.35,
      "end": 577.49,
      "text": "We have written all these edges in total. Look, there are 7 edges. From Kolkata
to Delhi, from Patna, whatever is there and its weight is written here. Okay. Now Bellman-Ford
said, do one thing. Mark Ranchi with 0. Mark the answer of Ranchi with 0. Because Ranchi is
the source. You don't know the rest of the 4. What do you mark the rest of the 4 with? Mark it
with infinity. And make an array like this. Which was also made in Dijkstra. Okay. Now,
Bellman-Ford said,"
    },
    {
      "start": 577.49,
      "end": 604.56,
      "text": "Do one thing. Look at all the edges. Look at all the edges and do their
relaxation. What does relaxation mean? If an edge tells you that source is destination edge
and its weight is WT weight. What does relaxation mean? What did we read in Dixra? If the
answer of destination which is written in this array that is greater than"
    },
    {
      "start": 604.56,
      "end": 630.24,
      "text": "Result of source. Means the answer of source. What happened? Which is the
answer of source. Plus weight. If this is the case. Then what do you do? Update it. Update it.
With whom? Result of source plus weight. This is relaxation. If the value of Ranchi is 0.
Which is the source. And there is a road of 2 km from Ranchi to Bangalore. So 0 plus 2. 2."
    },
    {
      "start": 630.24,
      "end": 656.54,
      "text": "If the current value of Bangalore is more than 2, then update Bangalore. This
is called relaxation. So, what did Bellman Ford say? Do one thing, relax all the edges. Hold
all the edges and relax. So, see, if you hold and relax the edges, then what will happen?
Assume that you are given the edge in this order. So, now this is your situation. So, what did
Bellman Ford say? Come on, relax. Let's see what happens one by one. Hold the starting edge."
    },
    {
      "start": 656.54,
      "end": 684.66,
      "text": "He is speaking from Kolkata to Delhi and what is he saying? It is one-way. But
you see that the source from Kolkata to Delhi, Kolkata's own value is infinity. What does
infinity mean? That we don't know right now. Infinity means that we don't know anything. We
don't know the answer at all. Don't relax this. If the answer from source is infinity, then
what are you doing? If you don't know how to reach Kolkata, then what are you relaxing from
Kolkata? Don't do anything. Leave it for now."
    },
    {
      "start": 684.94,
      "end": 712.29,
      "text": "Same thing. Patna to Kolkata. Patna itself has infinity. So what will you do by
relaxing from Patna to Kolkata? Now someone is saying that relax from Patna to Kolkata. Now
there is an infinity distance to reach Patna. So what difference will it make to Kolkata? It
will also become infinity. Leave it. Leave it for now. Okay. Patna to Delhi. Same story. Patna
itself has infinity. So what will it tell others? Leave it. Patna to Bangalore. Same.
Bangalore itself has infinity. What will it tell others? Leave it."
```

```
    },
    {
      "start": 712.29,
      "end": 733.92,
      "text": "Bangalore to Delhi, same. Bangalore has its own infinity. What will it look
like in Delhi? Leave it. But, but, look. Ranchi to Bangalore, Ranchi to Bangalore. Is the
distance of Ranchi infinity? No. Do you know the distance of Ranchi? Zero. It's a good value.
Very good. Relax. Ranchi to Bangalore is an edge whose weight is 2. Meaning, Ranchi to Ranchi,
it looked like zero."
    },
    {
      "start": 733.92,
      "end": 760.21,
      "text": "If the road to Bangalore is 2, then what can be the value of Bangalore? 0 plus
2 which is 2. Is this 2 smaller than this infinity? It is absolutely smaller. What should we
do? Relax Bangalore. What does it mean to relax? What does it mean to relax? Make this 2. Make
this 2. Make this Bangalore 2. Take this. What did we do to Bangalore? We made it 2. We made
Bangalore 2. Very good."
    },
    {
      "start": 760.21,
      "end": 787.01,
      "text": "There is one more day left to relax. From Ranchi to Delhi, 20. What is 20 from
Ranchi to Delhi? 20. Ranchi itself is 0. Ranchi to Delhi is a 20-kilometer road. So 0 plus 20,
20. Is this 20 smaller than the infinity of Delhi? It is absolutely smaller. What should we
do? Relax Delhi, brother. Relax Delhi. Relax Delhi immediately. What should we write after
relaxing Delhi? 20. Relax Delhi and write 20. Is it over?"
    },
    {
      "start": 787.01,
      "end": 816.85,
      "text": "In this iteration, you have seen all the edges, right? You have seen all the
edges one by one. Now, if the upper edge is not relaxed, then what can we do? We did what
happened. Do you see an interesting thing? A very interesting thing must have been found.
There are two things in this. Rachi to Bangalore and Delhi. Bangalore and Delhi have been
updated. Okay? This first path, the smallest path, how long will it take to go to Rachi to
Delhi? Have you got its answer in this? Will it take 20 kilometers? Yes, you have got it."
    },
    {
      "start": 816.85,
      "end": 843.01,
      "text": "The direct road of Ranchi to Delhi, which you had written that this can also be
a path, the smallest path. Have you got its value that its value is going to be 20? Yes, you
have got its value. You have got its value in one way. You don't have to write it. You have to
think that its value is 20. Remember that you have got it. How many times have we relaxed? We
have relaxed even once. We have got its value. What did Bellman Ford say? Okay."
    },
    {
      "start": 843.01,
      "end": 867.63,
      "text": "You have three more left. It's not like your program is over. If someone asks
you, what is the path from Ranchi to Bangalore to Patna to Kolkata to Delhi? You don't know.
You don't know. Okay? And you haven't got the answer yet. Bellmanford said, do one thing. What
you just did, lifting all the edges and relaxing, do it one more time. Do it one more time.
Someone asked, same thing? Yes, same thing. Do it again. Come, let's see what happens. Repeat
the same thing."
    },
    {
      "start": 867.79,
      "end": 896.54,
      "text": "Kolkata to Delhi. Will we relax? Kolkata itself is infinity. No, we won't.
Patna to Kolkata. Will we do it? Patna itself is infinity. No, we won't. Patna to Delhi. Will
we do it? No, Patna itself is infinity. No, we won't. Bangalore to Patna. What can we do? Yes,
Bangalore was infinity last time. Now, there is a value in Bangalore. Do it. Bangalore to
Patna whose weight is 2. So, Bangalore's value is 2. There is an edge whose weight is 2. 2
plus 2 is 4. Is this 4 less than Patna's infinity? Absolutely less."
    },
    {
      "start": 896.54,
      "end": 925.76,
      "text": "Relax Patna. Relax Patna and write 4 in Patna. Write 4 in Patna. Relax. Go
ahead. Go ahead. There's more to relax now. Bangalore to Delhi, whose weight is 20. Let's try.
Let's try. Bangalore's own answer is 2. This is 20. What will be 2 plus 20? 22. Is this 22
smaller than this? No, it's not smaller than this. So, let's not relax this. Let's not relax
this. Or do one thing, to make you understand,"
    },
    {
```

```
      "start": 925.76,
      "end": 955.46,
      "text": "We will try to explain it to you. We will try to explain it to you. We will try
to explain it to you. We will try to explain it to you. We will try to explain it to you. We
will try to explain it to you. We will try to explain it to you. We will try to explain it to
you."
    },
    {
      "start": 955.46,
      "end": 984.88,
      "text": "2 plus 17, 19. Is this 19 smaller than the one in Delhi? 20? It's absolutely
smaller. Relax Delhi again. Relax Delhi again. And what do you make? Make Delhi 19. Make Delhi
19. Make Delhi 19. Yes. Yes. It's done. Very good. Come. The edges are still left. Ranchi to
Bangalore, 2. So if we do Ranchi to Bangalore, the same answer will come. There will be no
benefit of doing it. No problem. Ranchi to Delhi, 20. Ranchi to Delhi, 0 plus 20. It's not
smaller than 19, so it's fine."
    },
    {
      "start": 984.88,
      "end": 1014.16,
      "text": "One more time, the whole edge got relaxed. One more time, it got relaxed.
Remember, how many times have we done this? Till now, we have done it two times. Here, we
write it somewhere. We have done it two times. Once before, once now. Now, what did you get to
know? Now, see what you got to know. Delhi got updated again and became 19. Which path has it
become 19? Ranchi to Bangalore and Bangalore to Delhi. So, did you get the answer for this
path too? Yes, you got the answer for this path too. What answer did you get? You got 19."
    },
    {
      "start": 1014.16,
      "end": 1038.62,
      "text": "Did you get the answer for the other two? No, you didn't get the answer for the
other two. But by doing the same thing one more time, by doing the same relaxation, you got
the answer for the other path as well, 19. That means you are doing something right. But did
you get the old answer now? No, you didn't get the old answer. Until we don't cover this
complex path, how will we get the old answer? What will Bellman Ford say? You guessed it right.
 Do it one more time."
    },
    {
      "start": 1038.62,
      "end": 1068.43,
      "text": "Again. What do you do once again? Hold all the edges and relax. Repeat the same
thing. Let's repeat. Kolkata to Delhi. Kolkata itself is infinity. We won't do it. Patna to
Kolkata. We can do it. Do it and see. Patna itself is 4. This is weight 2. 4 plus 2 is 6. Is
this 6 less than infinity? Absolutely less than infinity. What do you do to Kolkata? Relax and
make it 6. Relax Kolkata and make it 6. Yes. Yes. Relax Kolkata and make it 6. Let's go."
    },
    {
      "start": 1068.75,
      "end": 1097.89,
      "text": "Patna to Delhi, 13. Let's see. Patna itself is 4. This weight is 13. 4 plus 13,
17. Is this 17 less than the current answer of Delhi? Yes, it is absolutely less. Update Delhi
and make it 17. Breaking news and your Delhi has been updated. What has become of your Delhi?
17 has come to your Delhi. 17 has come to your Delhi. Very good. Keep moving forward. Patna to
Bangalore, same answer will come. Let's see. Delhi to Bangalore, no benefit."
    },
    {
      "start": 1097.89,
      "end": 1119.62,
      "text": "Ranchi to Bangalore is of no use. Ranchi to Delhi is of no use. This iteration
is also over. How many times have we done this iteration? 3 times. 3 times. Yes. 3 times. 3
times. Now look. Delhi has been updated again and has become 17 from 19. What kind of 17 is
this? Ranchi to Bangalore. Bangalore to Patna. Patna to Delhi. Did you get the answer for the
third one?"
    },
    {
      "start": 1119.62,
      "end": 1148.45,
      "text": "Yes, you also got the answer of the third one. But, do you think your work is
over? No, your work is not over completely. What will Bellman Ford say? One more time, one
more time, one more time. Let's do it one more time. Kolkata to Delhi, there is one weight. Do
it and see. Kolkata itself is 6. This weight is 1. 6 plus 1, 7. Is this 7 less than Delhi's
17? It is absolutely less. What will you do by updating Delhi? 7, by updating Delhi,"
    },
    {
      "start": 1148.45,
      "end": 1172.21,
      "text": "Do 7. What do you write after updating Delhi? 7. Come on, write 7. To relax, do
```

more. Patna to Kolkata, there will be no benefit. Do it. There will be no benefit. Same will remain. Patna to Delhi, there will be no benefit. Bangalore to Patna, there will be no benefit. Bangalore to Delhi, there will be no benefit. Ranchi to Bangalore, Ranchi to Delhi, there will be no benefit. One more is over. How many times have you repeated the same thing? Four times."
    },
    {
      "start": 1172.21,
      "end": 1200.21,
      "text": "Four times. Which one? The process of relaxing the whole age. We have repeated it four times. Your Delhi has changed again. Your Delhi is now seven. What path did you get the answer for? Ranchi to Bangalore, Bangalore to Patna, Patna to Kolkata, Kolkata to Delhi. Did you get the answer of the fourth one, the most complex one? Yes, you got it. Now, when you have done it four times, Bellmanford says,"
    },
    {
      "start": 1200.21,
      "end": 1229.14,
      "text": "Stop it. Stop it. Now your answer is complete. The array that you have made, this is your answer. Bellman-Ford tells you, four times, stop it. This is your answer. But why? Why isn't Bellman-Ford saying, do it one more time? Bellman-Ford, why isn't he saying, do it one more time. Do it two more times. How did Bellman-Ford know that it stopped? He found out because in this program, number of cities, number of nodes, how many are there?"
    },
    {
      "start": 1229.14,
      "end": 1259.14,
      "text": "5. How many are the number of nodes? 5. How many times does Bellman-Ford tell you to do this relaxation? 4 times. How many? 4 times. That means, n-1 times, n-1 times. But why did n-1 times come? Because, in the worst case, in the worst case, this path, means, if you want to reach a destination from source, and you have multiple paths,"
    },
    {
      "start": 1259.14,
      "end": 1288.08,
      "text": "In the worst case, your number of edges will be 4. Your number of edges will be 4. Why am I saying this? Because, let's say you want to travel from Kashmir to Kanyakumari. You want to travel from Kashmir to Kanyakumari in India. If someone tells you to do whatever route you want to take, how many states or edges will you travel in? In the worst case,"
    },
    {
      "start": 1288.08,
      "end": 1313.7,
      "text": "How many edges will you travel? If there are 29 states in total in India, then in the worst case scenario, you will travel 28 edges. Why? Because you will not repeat any city. If someone asks you to find the part of Kanyakumari from Kashmir, then you will not start from Kashmir and go to Uttarakhand."
    },
    {
      "start": 1313.7,
      "end": 1342.22,
      "text": "You went to Himachal Pradesh from Uttarakhand and came back to Uttarakhand from Himachal. If you want to go from Kashmir to Kanyakumari, if someone asks you to choose the longest route, whatever you want, then too, every state will visit only once, right? Why are you visiting a state again and again if you want a shortest distance? If you have come to Uttarakhand, went to Himachal from Uttarakhand and came back to Uttarakhand, what is the logic? You don't go to Uttarakhand better than that. This is the logic that if you want to go to Ranchi to Delhi,"
    },
    {
      "start": 1342.22,
      "end": 1367.41,
      "text": "So your biggest path can be of 4 edges or n-1 edges. That can be this path. Apart from this, there can't be a path bigger than this. If you are doing this, that you went from Ranchi to Bangalore, then Bangalore to Patna, then from Patna to Bangalore, then from Bangalore to Kolkata, then from Bangalore to Kolkata, then from Bangalore to Patna, then from Bangalore to Kolkata, then from Bangalore to Patna, then from Bangalore to Patna, then from Bangalore to Kolkata, then from Bangalore to Kolkata, then from Bangalore to Patna, then from Bangalore to Patna, then from Bangalore to Kolkata, then from Bangalore to Kolkata, then from Bangalore to Kolkata, then from Bangalore to Kolkata, then from Bangalore to Kolkata, then from Bangalore to Kolkata,"
    },
    {
      "start": 1367.41,
      "end": 1393.38,
      "text": "That's not the case. That's why Bellman-Ford says, if you have n vertices and you relax your edges by n-1 times, then you will definitely get the answer. Why? Because the

```
most complex part of the scenario we have written was the last one. How many edges are there in the last? Ranchi to Bangalore, 1 edge. Bangalore to Patna, 2 edge. Patna to Kolkata, 3 edge. Kolkata to Delhi, 4 edge."
    },
    {
      "start": 1393.38,
      "end": 1417.92,
      "text": "You can't make a path more than 4h, whereas n is equal to 5. If you are making a path more than 4, then you are doing something wrong. Either you have got a negative cycle. You have got a negative cycle. It means you have got a cycle in which if you rotate it again and again, it starts decreasing. So, what does Bellman Pod say? If your n is equal to 5, then.."
    },
    {
      "start": 1417.92,
      "end": 1445.55,
      "text": "n-1 times, if you relax all the edges, then you will get all the answers. You will get all the answers. There can be no better answer than this. I mean, if you look at Delhi, what path will be better than this? We have covered all the paths. From the simplest path to the most complex path. There is no other complex path in which there are other edges to compare. I mean, when you used to do this, what used to happen? First of all, you only knew the answer of Ranchi."
    },
    {
      "start": 1445.55,
      "end": 1464.29,
      "text": "You relaxed once and got the answer of Bangalore. You relaxed three times and got the answer of Patna. You relaxed again and got the answer of Kolkata. You relaxed again and got the answer of Delhi. This is what happened to you. You knew the answer of Ranchi from the beginning."
    },
    {
      "start": 1464.29,
      "end": 1484.4,
      "text": "In the 1st relaxation, you got to know about Bangalore. In the 2nd relaxation, you got to know about Patna. In the 3rd relaxation, you got to know about Kolkata. In the 4th relaxation, you got to know about Delhi. You will definitely get to know. You don't need to do the 5th relaxation because you will definitely get to know. It's not possible that you won't get to know because you already know about Ranchi. How much is left?"
    },
    {
      "start": 1484.4,
      "end": 1513.63,
      "text": "Rest, n-1 vertex remains. If you relax n-1 times, then it is a sure thing to find out the rest of 4. You can't find a smaller answer than this. You can do anything and see. That's why Bellman-Ford tells you to relax n-1 times. You will get the answer. But there is one more thing. We told you in the beginning that it detects negative cycles. What does it do? It detects negative cycles. For that, how to detect? Till now, nothing has happened like detection."
    },
    {
      "start": 1513.63,
      "end": 1540.91,
      "text": "Let's make a very simple map of two cities. Ranchi and Patna. The map of two cities is very big. Ranchi to Patna will go one edge and Patna to Ranchi will go one edge. Let's write the weight. Let's assume that Ranchi to Patna goes in two and this goes in three. Your source is Ranchi. What is the weight of Ranchi? The answer is zero. What will Bellman Ford say?"
    },
    {
      "start": 1540.91,
      "end": 1568.16,
      "text": "How many vertexes are there? There are two vertexes. You already know the answer of Ranchi, right? Zero. Relax once. How many? N minus 1. Relax once. You will definitely get the answer of Patna. Relax once. Relax Ranchi. Now if we relax Ranchi, then what will happen? Zero plus two, two. What will Patna become? Patna will become two. Finished. Bellman-Ford said, finished. There can't be a better answer than this. But, but, if there is a negative cycle in this graph, then its answer will not come."
    },
    {
      "start": 1568.16,
      "end": 1595.84,
      "text": "Bellman-Ford says, do n-1 times. Get out of the loop. Do it one more time. Do it one more time. What do you do one more time? Relax all the rest of the edges. If you don't get a good answer in that, if in that relaxation, in your nth relaxation, you have to do n-1. Do nth and see one last time. If you don't get a good answer in your nth relaxation, if you don't get a good answer, which you shouldn't get,"
```

```
    },
    {
      "start": 1595.84,
      "end": 1624.96,
      "text": "Everything is fine. You can return it. What will happen in the end relaxation? Relax both of them. Relax Ranchi to Patna. 0 plus 2, 2. Is this 2 less than this? No, 2 is not less. You didn't get a good answer. Very good. Leave it. Relax this edge too. 2 plus 3, 5. Is 5 less than this? No, 5 is not less than this. You didn't get a good answer. Everything is fine. There is no negative cycle. You can return it comfortably. Ranchi's 0 and Patna's 2."
    },
    {
      "start": 1624.96,
      "end": 1652.46,
      "text": "But if there is a negative cycle in it, now see, make it negative. Take this. This cycle is now negative minus 1. This cycle of Ranchi to Patna to Ranchi has become a negative cycle. Now see what will happen. Now when you will relax this nth bar, then see what will happen. Relax this. Ranchi 0, 0 plus 2, 2. Is it smaller than 2, 2? No, it is not small. It is a good thing. But now relax this edge. See what will happen."
    },
    {
      "start": 1652.46,
      "end": 1677.6,
      "text": "Patna's answer is 2. This edge is minus 3. 2 plus minus 3. What happens? Minus 1. Minus 1 is a small answer. It means you got a good answer. When? By going to nth iteration. By getting a good answer in nth iteration, it means negative cycle is present in your graph. Negative cycle is present in your graph. Don't do anything. Return minus 1. This graph can't do anything."
    },
    {
      "start": 1677.6,
      "end": 1701.71,
      "text": "Keep doing it, you won't get any benefit. Why won't you get any benefit? Because the more you roam in this cycle, the smaller the answer you will get. So when will you stop roaming? This means, in this, the shortest distance can't be computed. What is there here? There is a negative cycle here. There is a negative cycle here. Are you understanding or not? So what does Bellman-Ford say? Bellman-Ford says, if n nodes are there,"
    },
    {
      "start": 1701.71,
      "end": 1714.0,
      "text": "Close your eyes and do n-1 relaxation. Why do you do n-1 relaxation? Because you already know Rachi. If you relax 4 times, you will get the correct answer of 4. Very good. After that,"
    },
    {
      "start": 1714.0,
      "end": 1741.74,
      "text": "testing karne ke liye ek aur baar karo, ek aur baar karo. Agar us ek aur baar me koi sahi answer nahi, koi acha answer nahi mila, koi update nahi hua, very good, return kar do. Agar us nth baar me, agar aapko pata chalene lagta hai ki nahi, abhi bhi chota answer mil raha hai kisi ka to, wo graph ekdum gone case hai, bekaar graph, negative case ka graph hai, usme minus one return kar do. Yeh hota hai Bellman-Ford. Thik hai. Toh umeet karte hai ki aapko samajh me aaya hoga, bahut acche se."
    },
    {
      "start": 1741.74,
      "end": 1767.54,
      "text": "We have put it in whatever we knew to teach. We don't think it is better than this that it could have been explained and it will be explained. You will remember this for the rest of your life. Come, let's write its code and show you how it is done. You have already seen dry run. So it is fine. Let's write its code. What is there in the code? You have to return a vector int. You have a graph of edges. What kind of edges are there? You have a source, destination and"
    },
    {
      "start": 1767.54,
      "end": 1798.66,
      "text": "weight. This is the edge of vector of vector. As it was in Dijkstra. So this is your Bellman. First of all, the number of vertex is given. And vector of vector of int of edges. Okay? This is given to you. And a source is given. A source is also given. Okay? Let's go. First of all, what should we do? Make one of those. Distance array. Result array. Result of n size. And what are we calling everyone? 1e."
    },
    {
      "start": 1798.66,
      "end": 1828.27,
      "text": "1E8 means 10 to the power 8. Do int max of this. We don't want to do int max.
```

What does 1E8 mean? 10 to the power 8. If you write 1E9, then it will be 10 to the power 9. After that, it is very good that you don't have to make any adjacency list in this. Why don't you have to make it? Because what is your whole work? To relax edges. So, edges are already given in edges. So, what is the use of making adjacency? It is very good. We have already said that it is going to be a very small code. You don't have to do anything in this. After that,"
    },
    {
      "start": 1828.27,
      "end": 1857.38,
      "text": "The answer of the result of source is 0. So, make it 0. Is it okay? Start the relaxation. How many times? n minus 1 times. For int i is equal to 0. i less than n minus 1. i plus plus. Do i less than n minus 1. Not n. Because n minus 1 times to relax. Not n times to relax. What happens? Relaxation means loop on all edges. Loop on all edges. int j is equal to 0."
    },
    {
      "start": 1857.38,
      "end": 1887.76,
      "text": "j less than edges dot size and j plus plus. Okay. Take out all three things. Int source is equal to edges of j's 0. Its first one. Int d is equal to edges of j of 1. And int weight is equal to edges of j of 2. Okay. Now, who to relax?"
    },
    {
      "start": 1887.76,
      "end": 1918.85,
      "text": "If result of S is not equal to 1E8. Means if suppose source itself is infinity. Which you want to relax. From this source to this destination. Where we will relax it. Source itself is infinity. So what will relax anyone? So if it is not 1E8. 1E8 means in our case also infinity. Or result of D is greater than result of S plus weight. This is what relaxation means. So what will we do?"
    },
    {
      "start": 1918.85,
      "end": 1949.25,
      "text": "Result of D is equal to result of S plus. Wait. It's done. Relax. It's done. Relax. Close this. Close this. What happened? What did you do? N minus 1 times up. Relax. You should get the right answer by now. You should get the right answer by now. But for testing, relax one more time. Relax one more time. Repeat this J loop again. Repeat this J loop again. For INT, J is equal to 0."
    },
    {
      "start": 1949.25,
      "end": 1976.8,
      "text": "J less than edges dot size or J plus plus. Again write everything. I N T S is equal to edges of J 0. I N T D is equal to edges of J 1. I N T W T is equal to edges of J 2. Is it correct?"
    },
    {
      "start": 1976.8,
      "end": 2004.9,
      "text": "Try to relax. Check if it can be relaxed or not. If result of S not equal to infinity, which means 1E8, and result of D is greater than result of S plus weight, which means we are still getting the right answer. How can we get the right answer in nth iteration? If so, then you have been asked to return a vector of minus 1. Return. Return."
    },
    {
      "start": 2004.9,
      "end": 2032.27,
      "text": "Minus 1. I have asked you a question. GFG has asked that if you get this, what does it mean? What is this? This is a gone case. This is a gone case with a negative cycle. This is a graph of a negative cycle. No one can do anything in this. Return one vector which has minus 1. Which has minus 1 filled in it. So return it like this. And close this loop. If this doesn't happen, then what will you return after being comfortable? Your result. That's it. "
    },
    {
      "start": 2032.27,
      "end": 2062.03,
      "text": "See how cute and cute this code is. And its intuition is not like this that you have to remember a lot. You don't have to remember anything. You have to relax n-1 times. Why? Because you already know the source. The rest of the n-1 vertex will be found in n-1 time. Because there can't be any path in which there is more edge or more complex path. And for testing, relax one more time. For what? For testing. If testing fails, in nth relaxation, if you are getting the right answer, if you are getting a good answer,"
    },
    {
      "start": 2062.03,

```
      "end": 2091.9,
      "text": "It's a negative cycle. Tell the graph to remove this graph. Return minus 1 to
all. If you don't get it, then very good. Your Bellman-Ford is complete. So, you have learned
Bellman-Ford. When to use Bellman-Ford? Whenever single source, shortest path, and negative
weights are given to you. Negative weights. Whenever negative weights are given, don't use
Dijkstra. We say, don't use Dijkstra even usually. Make Bellman-Ford your companion. Make it
your companion. And let's meet in some next video."
    },
    {
      "start": 2091.9,
      "end": 2093.0,
      "text": "Let's code it."
    }
  ]
}
```


15. transactions/t_23.json


```
{
  "video_id": "AflRvP5LFFc",
  "title": "Stop Struggling with Recursion: Combination Sum Explained Simply",
  "language": "en",
  "model": "large-v3",
  "duration": 1565,
  "segments": [
    {
      "start": 0.05,
      "end": 16.98,
      "text": "Today's recursion question is hard. It is asked in Amazon, Meta and many other
companies. But there is a little twist in it. For this twist, we want to give you an example
of chocolate as an intro. So after listening to this example, the next explanation will sit in
your mind. So we are making this later."
    },
    {
      "start": 16.98,
      "end": 42.11,
      "text": "explain pehle ki hai humko karte karte strike ki yeh bata dena chahiye tha to
yeh bahut dhyan se sun lijiye ga hum yeh bhi asha karte hai ki aap pehle ki recursion videos
dekhte hu aara hai aur aap bilkul nahi yaha pe nahi hai kyunki aaj jo 5 step flowchart tha
uski baat hone wali hai thik hai to vo dekh lijiye nahi bache vo pehle dekh lijiye shuru se
yeh video padhi rahi gaya isa nahi ki hum isko delete kar denge maal lijiye aap kisi chocolate
factory me hai thik hai yaha pe bahut saare chocolate dekh sakte hai Cadbury, KitKat, Munch
perk sab rakha hua hai"
    },
    {
      "start": 42.11,
      "end": 71.6,
      "text": "You have choices. The manager has told you to go from here to here. Go from
here to here. Okay? Go in a line. Okay. If you go to every chocolate, you have choices. If you
don't want to eat chocolate, then you can go ahead. No problem. No issue. You can go ahead. If
you want to eat this Cadbury, then you can't eat it once. You can eat as much Cadbury as you
want. I mean, there are a lot of chocolates in Cadbury. You can eat as much as you want. Okay?
But, if you do this, that you eat two Cadbury and go ahead,"
    },
    {
      "start": 71.6,
      "end": 101.15,
      "text": "After that, you can't come back to Cadbury. After that, you can't say that you
have eaten KitKat, you will come back. No, no, no, no, you can't do that. As much as KitKat,
if you don't want to eat Cadbury, go ahead. If you don't want to eat KitKat, go ahead. There
is no issue. But, if you want to eat, then you can eat as much Cadbury as you want. But, if
you go ahead, then you can't eat Cadbury. So, what do you do in this? The next question we are
going to explain is its heart, its base."
    },
    {
      "start": 101.15,
      "end": 130.43,
      "text": "What will you do? If you don't want to eat Cadbury, you will move forward. If
you don't want to eat, then what will you do? You will move forward towards KitKat. But
suppose you don't know how much Cadbury you want to eat, then what will you do? What will you
do? You will take out one Cadbury and eat it. You will take out one Cadbury and eat it. But
you will not move forward by taking out that one Cadbury. Because it is possible that you will
want to eat one more. The manager has clearly said that if you move towards KitKat, then you
cannot come back to Cadbury. You cannot come back."
    },
    {
```

```
      "start": 130.43,
      "end": 143.82,
      "text": "You have to move forward. What will you do? You will take out one Cadbury. You
will eat it and stay there. You will stay there. You haven't moved forward on Cadbury yet.
After one Cadbury, if you feel that it's done, you will move forward."
    },
    {
      "start": 143.82,
      "end": 170.27,
      "text": "But if you think that you want more, you will eat one more Cadbury and will
stay there and will not go ahead. Because you will go ahead when your heart will be full. You
don't know when your heart will be full. You took out one more Cadbury and you are still on
this. You are still on Cadbury. You ate one more Cadbury and you are still here. When you will
feel that you don't want to eat Cadbury. When you don't want to eat Cadbury."
    },
    {
      "start": 170.27,
      "end": 197.47,
      "text": "Then you will go to KitKat. Then you will go to KitKat. You will do the same
thing on KitKat. You stand here. You know that you like KitKat. So you eat one KitKat. But if
you eat that KitKat, you are not moving forward. You will stop there. You will stop there. If
you feel like eating one more KitKat, then eat one more KitKat. But still you don't move
forward. You have to stop there. When you feel that you don't want to eat KitKat, then you
move forward. Then you move forward. So what are you doing?"
    },
    {
      "start": 197.47,
      "end": 217.12,
      "text": "If you are standing on a chocolate, if you think that you don't want to eat it,
maybe you are full or you don't like it, you don't like it. If you don't want to eat, if you
don't want to eat, you go to the next chocolate very easily. But, but, but, if you want to eat,
 if you want to eat, then you..."
    },
    {
      "start": 217.12,
      "end": 240.8,
      "text": "Eat and don't go to the next one. Eat standing there. Because you don't know
how much to eat. You think you have to eat 100. Stand here because the manager will not let
you come back. If you want to eat, then you eat standing here. You must be thinking this is
very stupid. What is he telling? But we are telling this because it is very relatable. You
will do this only. You will not do anything else. You don't know how much you can eat."
    },
    {
      "start": 240.8,
      "end": 263.9,
      "text": "You will feel like eating a lot of food. So you will do the same thing. Until
you don't feel like it, you will keep eating it and stay on the category. When you feel that
you don't want to eat any chocolate, then you move on to the next chocolate. So this is a very
relatable and very human example. In the next video, you will understand why we talked about
it. It was important to talk about it. You didn't understand it. We felt that we won't
understand it. Now move on. Now you should understand. And let's start the video."
    },
    {
      "start": 263.9,
      "end": 289.36,
      "text": "So, first of all, let's explain the question. You may think that the question
is big, but those who are watching the recursion series will think that yes, it is a very
basic question, that we do the same thing again and again. So, you will feel that if you are
watching all of them, then if you stumble on this channel for the first time, then you will
not feel that. Let's go. You have been told that you have been given an array, which has
distinct integers, right? Distinct integers and positive integers are also given."
    },
    {
      "start": 289.36,
      "end": 316.59,
      "text": "Distinct positive integers. Means all are different from positive numbers. Okay.
 You have been given a target. You have been given a target also. Okay. What is your work? You
have to find all the unique combinations of array. All the unique combinations you have to
find. But which unique combination? Whichever sum. Whichever sum. Equal equal to target. So we
have seen such a question earlier. Where you have to find."
    },
    {
      "start": 316.59,
      "end": 342.58,
      "text": "You had to find all the permutations. For example, if the array is 1, 2, 3,
then you had to do everything. Do an empty array, do a 1, 2, do a 1, 3, do a 1, 2, 3. It means
all the subsets. Remember the subset printing. It is the same here, but there is a small
```

```
problem in it. That you don't know the same subset. What do you understand the combination? You understand the combination of subsets. What does the subset mean? There will be some elements of the array, there will be nothing. It is the same. But whatever subset you are,"
    },
    {
      "start": 342.58,
      "end": 371.95,
      "text": "Subset whatever you do, those whose sum is equal to target, you have to include that subset in your answer. So what is this problem? Same problem, generate subset, but the subset's sum is equal to this target, which is given to you in the question, only that subset has to be included in your answer. And one more small thing, one more small thing is different,  that what you used to do in subset, for each element, how much choice did you have? Either you used to take it, or you didn't take it. If suppose this is your array,"
    },
    {
      "start": 371.95,
      "end": 379.04,
      "text": "We want to say 1, 2, 3. And you make its subset. So suppose you make 1 and 2. So see 1."
    },
    {
      "start": 379.04,
      "end": 408.34,
      "text": "It will come only once in this subset. But there is one more unique thing in this. In this particular question, in this combination sum, you can take any element as many times as you want. You can take it as many times as you want. You can take the same element again and again. It doesn't mean that you don't want to take the element. But if you have taken that element, you want to take one more one, you can take it. You want to take one more one, you can take it."
    },
    {
      "start": 408.34,
      "end": 437.78,
      "text": "For example, if we take an example of an array. An array is given to you like this."
    },
    {
      "start": 437.78,
      "end": 457.17,
      "text": "3, 6, and 7. 0, 1, 2, 3. Okay? Your target is 7. So look, we can do one thing in this. Take 2 in one subset. Take this 2. Okay. Take one more time. One more time. I don't feel like it. Take one more time. Okay? Take it. Take one more time. Now leave 2. Now take 3. Now take 3."
    },
    {
      "start": 457.42,
      "end": 486.11,
      "text": "Look at this. What will be its sum? 7. So this is your answer. What else can you do? Only 7. This is also your answer. Is there any other answer possible besides this? No, there is no other answer possible. Now see, you can take all the permutations and combinations you want to do. There is no other answer possible whose sum is 7. Only these two answers are possible. So these two subsets should be in your answer. These two should be in your vector of,  vector of integer. Come, let's solve this with our flowchart."
    },
    {
      "start": 486.11,
      "end": 513.23,
      "text": "In the previous recursion video, we made a stepwise plan. It was a five-step plan. The first step was to identify the stages. We are doing this revision. After that, you can see how to go one by one stages. What are the choices? What are the choices in every stage? What can you do? Either do whatever you want or if there is a condition. After that, that choice. Make one choice. During that make one choice, you had to write that choice in your diary."
    },
    {
      "start": 513.23,
      "end": 542.56,
      "text": "You had to go to the next stage and then come back and reverse it from the diary. And when you finish all the choices, then look at your diary. Whatever will be in your diary, that will be your answer. This was a five-step plan. Let's see the five-step plan. Come on, let's see what we do. See, with this five-step plan, we said that any question can be made.  Till now, any question can be made. Come on. First of all, what are the stages? Meaning, how many chances do you have to make a decision? So see, stages."
    },
    {
      "start": 542.56,
      "end": 569.28,
```

```
      "text": "How many chances do you have? You have an array. Every element of the array is
a chance for you. You have to make a decision whether to take it or not. Every element is a
stage for you. Every element means every index is a stage for you. Every index is a stage for
you. It's simple. In any array or string question, what will be the stages? The index or the
element of that will be the stages. How to go to different stages? With the help of index."
    },
    {
      "start": 569.28,
      "end": 599.18,
      "text": "With the help of index, we will start from index 0. And when my IDX will be
equal to n, it means that all the stages are over, then we will stop. What is choice? The most
important question. What is choice? What is choice at every stage? What is choice at every
stage? See. Every stage means that when you are standing on any one index, suppose you are
coming here, 2, 2, 3, 0, 1, 2. You are standing here. What choice do you have at this stage?
What choice do you have at this stage? See."
    },
    {
      "start": 599.41,
      "end": 627.95,
      "text": "The first choice is a very simple choice that you don't have to take. You don't
have to take. See, in this question, you feel the same complex, right? That you can take one
element again and again. But taking it again and again doesn't mean that you have to take it.
The best choice is that you don't have to take it. Absolutely ignore it and don't take it.
This is the easiest choice. You don't have to do anything in this. You have to go straight to
the next index. In this, you have to go straight to the next index. Okay. You have one more
choice. You can take it. You can take it."
    },
    {
      "start": 627.95,
      "end": 652.62,
      "text": "You can include this index in your answer. But what is the catch here? When we
used to do generate subset or any other problem where you could take it or not, you used to go
to the next stage. What do you mean by this? Suppose you were standing at door 1. You had two
choices. One, you could go to the left."
    },
    {
      "start": 652.62,
      "end": 678.21,
      "text": "And you could go to the right. So no matter whether you choose left or right,
in both cases, you would reach door 2. In both cases, you would reach door 2. If you open left
on door 1, then also you would reach door 2. If you open light on door 1, then also you would
reach door 2. But it is not like that here. Here, let's assume that you can open door 1
repeatedly. You can open door 1 repeatedly. What?"
    },
    {
      "start": 678.21,
      "end": 705.71,
      "text": "we can open it again and again. Meaning, it is also possible that you come to
door 1, you have decided that we have to take left, but after taking left, you come back to
door 1. You come back to door 1. Sorry, after taking left, you come back to door 1. Where did
you come? You came back to door 1. You can also do this. After taking right, you come back to
door 1. You come back to door 1. Okay. So, meaning, we came to know that"
    },
    {
      "start": 705.78,
      "end": 728.22,
      "text": "On any one index, suppose you are standing on idx is equal to 0. The best
choice can be that you don't have to take it. This is useless, you don't have to take it. This
is absolutely right, you don't have to take it, so you go to the next stage. What will you do
on idx is equal to 0? You come directly to idx is equal to 1, you will take a decision on it.
But if you have to take it,"
    },
    {
      "start": 728.22,
      "end": 747.6,
      "text": "So, you have to take it no matter how many times you take it. Now, take it once.
 Now, once IDX is equal to 0. Assume that it was 2. So, write it in your diary once. But you
can take it again and again. You can take it twice. You can take it three times. You can take
it four times. If you take it, then don't go to IDX is equal to 1 now. Because if you go to
IDX is equal to 1,"
    },
    {
      "start": 747.6,
      "end": 772.78,
      "text": "So how will we get a choice to take it again? We need a choice to take it again.
 We need something on which we keep coming again and again. When you take it, don't increase
the index. Come back to IDX is equal to 0. Come back to IDX is equal to 0. But, but, but, now
```

someone will say, sir, how will this end? This will never end. Why won't it end? Because suppose IDX is equal to"
    },
    {
      "start": 772.78,
      "end": 801.89,
      "text": "You are standing at zero. Your element is 2. Your diary is empty in the beginning. Your diary is empty in the beginning. So what did we say? If you don't want to take it, then go to idx is equal to 1. If you want to take it, then you keep coming back to idx is equal to 0. So you will say, okay. So let's assume that we have taken it. And what have we written in our diary? That we have taken 2. Okay. And you are saying that take it and come back to idx is equal to 0. Again idx is equal to 0 will work. You will say, take it. Write it in your diary. And come back to it. Then that will work. Come back to it."
    },
    {
      "start": 801.89,
      "end": 831.14,
      "text": "Then the same thing will happen. When will this end? Will it go to the infinite loop? No, it won't go. Why won't it go? Look, what does the question tell you to make it easy for you? The whole number is positive. The whole number is positive. The whole number is positive. Whose sum should be equal to? Equal to the target. Your sum should be equal to the target. Okay. Suppose at some stage, at some stage, you have come to some idx is equal to 2. Okay, suppose you have reached somewhere. Your current sum,"
    },
    {
      "start": 831.14,
      "end": 860.22,
      "text": "Assume the current sum is 10. Your current sum is 10. Your target is 13. Your target is 12. Your target is 12. Okay. Now what will we do? Assume the array of IDX, the element you are trying to take or not to take, that is 5. That is 5. Okay. Should you think of taking IDX is equal to 2 even once? No. Why not? Because if you take it, if you take it,"
    },
    {
      "start": 860.22,
      "end": 881.01,
      "text": "If you include this 5, then you will have to include it in your sum. Your sum will become 15. Now whatever happens in the future, all the positive integers are in the future."
    },
    {
      "start": 881.01,
      "end": 904.37,
      "text": "Can the target be equal at any time? No, it can't be. You can do anything in this. Don't take anything in the next one. Take the next one 100 times. Do anything. All are positive numbers. 15, 12 cannot be made without negative numbers. So, we got a funda here, right? If you come to any IDX and if you come to know that the sum here, what is the sum? Sum is your"
    },
    {
      "start": 904.37,
      "end": 933.65,
      "text": "diary. Sum is like your diary. Whenever you take an element, when you decide to take it, then what will you do? Sum plus is equal to array of idx. And when that work is done, then how will you reverse? Just the opposite. Sum minus is equal to array of idx. But if you get to know on any index, if you get to know that sum plus array of idx greater than target will be done. It will be greater than target."
    },
    {
      "start": 933.65,
      "end": 962.29,
      "text": "In this case, how many choices do you have left? Only one choice is left, that you don't want to take. What? You don't want to take, brother. You don't want to take. Absolutely don't want to take. Absolutely don't want to take. There is no benefit of taking. And if this doesn't happen, and if this doesn't happen, then you have two choices. So you have two choices. You are standing at IDX is equal to zero. The first choice you have, the choice of not taking is always there. The choice of not taking is always there. Go to IDX is equal to one. And if you want to take, and if you want to take, then take it."
    },
    {
      "start": 962.29,
      "end": 993.36,
      "text": "But because you can take it again and again and again in this. Revisit IDS is equal to 0. Revisit IDS is equal to 0. Give yourself one more chance. Brother, take as much as you want to take. After that, stop. After that, stop a little. Okay. After that, what to do? Stop a little. Let's go. So, we hope that you have understood this thing. That what we have to do in choices. Look, in choices, you have understood the choice on the stage. That you have to take or not to take. Okay. But what stage do you have to go without taking? Do you have to go

```
to the next door without taking?"
    },
    {
      "start": 993.36,
      "end": 1018.42,
      "text": "But if you have taken it, then because you have the opportunity to take it
again and again, then you revisit it again and again at the same door. When does this thing
end? When you come to know that if we take this index, then my sum will be greater than the
target and then it will never be equal to the target, which is not my aim. So you will stop
there, you will say that there is no question of taking it, we will not take it. Come on, now
write it in code. First of all, what will you need for the stage?"
    },
    {
      "start": 1018.42,
      "end": 1045.25,
      "text": "You will need an index. So we will write what all we will pass in the function.
First of all we will pass the array. We will pass the array's n. We will also pass the index.
And what else will we pass? Your diary. The most important thing. When you take a choice, you
write in your diary. So here you are getting a subset return. So one obvious thing is that one
will be your diary. One will be your diary in which you will write choices."
    },
    {
      "start": 1045.25,
      "end": 1074.58,
      "text": "The obvious thing is that you need all the diaries. You need all the good
diaries. What will be written in the diaries? Whatever choice you have made, whatever subset
you have made. You will need a vector of, vector of int. The collection of all diaries. This
will be your result. And let's do one thing. Let's take an int sum variable. What will this
sum tell us? Sum is also a type of diary. Whenever we take an element, we will do two things.
Whenever we take an element, we will do two things. First of all, we will push it back in the
diary."
    },
    {
      "start": 1074.58,
      "end": 1101.63,
      "text": "First of all, we will push it back in the diary. If we take it, and what we
will do is, sum plus is equal to array of i. This is done. When its work is done, then we will
reverse both. First of all, reverse this. Sum minus is equal to array of i. And then reverse
this. What is the reverse of push back? Pop back. So instead of one diary, we will take two
diaries. Because we are working on the sum again and again, so we want to keep the sum easy.
That's why you can do it at once if you want. You can take out the sum of the diary again and
again."
    },
    {
      "start": 1101.63,
      "end": 1129.04,
      "text": "There is no problem. So what does the diary contain? The diary contains what
you have chosen till now. And it contains the sum. What is the sum of whatever is in your
diary? 2 to 7. So these two things are your diary. Okay? So we will do these things. Come,
let's see its algorithm or code. So what will we do? First of all, we will void. Come. Void
fun. In this, first of all, my array will pass. Vector int a."
    },
    {
      "start": 1129.17,
      "end": 1157.5,
      "text": "int n, an int idx, which is my current one. You have to take a decision on this
current index. After that, we will pass a vector int diary. Vector int diary. And then we will
also take an int sum. And where is the answer of all the diaries? What do you need in the
final answer? All the subsets. So vector of vector of int. Whatever your diary is, its vector.
Vector of vector of int. This will be your final answer. Okay, come."
    },
    {
      "start": 1157.5,
      "end": 1186.51,
      "text": "First of all, when did you stop? When did you stop the recursion? When did you
have no choice left? When did all the stages end? When did all the stages end? If idx is equal
to n. Everything is over. Now what did you have to do? Generally, you used to write what is in
the diary. Generally, you used to write what is in the diary. But, which subset do you want
here? Whose sum is exactly equal to the target. Yes, see, you have to take a target too. You
have to take a target too. Int."
    },
    {
      "start": 1186.93,
      "end": 1215.66,
      "text": "target. If idx is equal to 1, it means the game is over, you don't have to do
anything. But, if your sum is equal to target, if your sum is equal to target, then what do
you do? rest dot push back diary. Push back diary. So, it means that diary is correct.
```

```
Whatever is in that diary is exactly. And what do you do? Return it. Now, your whole is over. So, what happened to you? Over. All stages are over."
    },
    {
      "start": 1215.66,
      "end": 1236.59,
      "text": "Now let's decide on choices. So what we thought was that if the sum of the index is smaller than the target, then you can take it. If it is bigger than the target, then you can't take it. But there is a choice of not taking it. In both cases, you always have a choice of not taking it. So make the choice of not taking it earlier."
    },
    {
      "start": 1236.59,
      "end": 1266.02,
      "text": "We will not take this stage, we will ignore it. If you are not taking anything, then there is no point in writing in the diary. What do you write in the diary that you take? If you have done 10, then you have written 10 in the diary. If you have done 12, then you have not done beta, then why are you writing it? There is no point in that. Go straight ahead. A, N, IDX plus 1. If you don't want to take, go straight ahead. If you don't want to take, then why will you come again and again? You don't want to take, right? After that, your diary, nothing changed in it."
    },
    {
      "start": 1266.02,
      "end": 1289.41,
      "text": "Your sum, your result, and your target. Nothing has changed. What is this? Don't take. Don't take. Choice 1. Now, choice 2 is that you have to take. Choice 2 is that you have to take. But there is a check for that. If taking this will increase the sum from the target, then why will we take it? So, check this. If array of index plus sum."
    },
    {
      "start": 1289.41,
      "end": 1319.02,
      "text": "If it is less than or equal to the target, that means if the sum is less than or equal to the target, then there is some hope. You can take some hope. But if it is not like that, then you cannot take it. So if it is like that, then you can take it. You can take it. You have a choice. You can take it. Try it and see. There is nothing in it. What did I say? Before making any choice, if you are including someone, first of all, diary.pushback. Write in the diary that we are taking it. We are taking it to remember. Array of IDX."
    },
    {
      "start": 1319.09,
      "end": 1349.23,
      "text": "Along with diary, update sum too. Sum is equal to sum plus array of idx. Very good. Now take this. Now take this and move forward. Where do you move forward? Not on the next index. Come on the same. Because it may be that you want to take it again next time. If you want to take 2 twice, then we will take it next time. We will take it next time. Let's take it once now. We will take it once later. So what to do? a, n, idx. Not idx plus 1, idx. Diary, sum, result, target."
    },
    {
      "start": 1349.23,
      "end": 1377.22,
      "text": "Done? Now, we have come back. Reverse these two things. Whatever you did earlier, reverse it. Diary, dot, popback, reverse of pushback, popback. Sum is equal to sum plus array of index. Means, if you join sum, what will be its reverse? Reduce array of index, it will be reversed. Sum is equal to sum minus array of index. Close this. Now, have you done both choices? Means, have you done choice 1?"
    },
    {
      "start": 1377.22,
      "end": 1407.52,
      "text": "Choice 2. Is there any other choice other than this? We can take it. Or we can not take it and we can take it many times. There is no other choice other than this. So it's over. There is no other choice other than this. Return. This is your function. See what step it follows. Identify the stages. Identify the choice. Identify in which condition you can take the choice. This was its complex part that we are going the same way. Because you can take the same element again and again. Again and again."
    },
    {
      "start": 1407.52,
      "end": 1439.15,
      "text": "We can open the same element. How will we run this? We don't have to do anything. First of all, what should we make from the main? Int n is equal to a dot size. Is it done? Int sum is equal to 0. In the beginning, the sum will remain 0. Make an empty diary. Vector int. Make an empty diary. Make an empty answer. Vector of. Vector of int. Make an empty answer. And call this. From where? From the beginning."
```

```
    },
    {
      "start": 1439.15,
      "end": 1469.95,
      "text": "n, call from 0, take this from 0, int idx is equal to 0, take it, no problem,
come, so take it, and array of n, idx, diary, sum, result and target, when you will complete
this, when a person will complete this work, then whatever you have to fill in the result,
will be filled, return, result, this will happen in your main function, this is your recursion,
"
    },
    {
      "start": 1469.95,
      "end": 1498.34,
      "text": "I hope you are able to understand, especially this part. If you are not able to
understand this part, then please comment. I am ready to explain it again. You should be able
to understand this part of what you are doing repeatedly. That is why I am asking this
question because the question is a little different. So this part is important. The rest is
the same. The rest is a generate subset. There is nothing in it. Everything is the same. Why
are you coming to the same element repeatedly? And how will it stop? If you don't put this,
then it never stops."
    },
    {
      "start": 1498.34,
      "end": 1528.18,
      "text": "Now you are stuck like this. You have come to door 1. And you have written like
this. To take right, come back to door 1. So he will come back to door 1 with right. Then come
back. Take right and come back here. Okay, then come back. Take right. He will get stuck. This
is an infinite loop. It is very important to write this to finish it. When can you take this
choice? That's why think very consciously. Which choice can you take when? It is very
important. There is a choice. But every time that choice is not there. Every time. It is not
that you can open left and right. No. Open with your mind."
    },
    {
      "start": 1528.18,
      "end": 1553.78,
      "text": "Can we take this choice or not? Are we taking it wrong? There is a choice of
not taking it. There is always a choice of not taking it. Suppose the question tells you that
you have to take all the elements at least once. You can't say that you don't have to take it.
Then you have to change it. So you don't have this choice. There is no choice of not taking it.
 So you have to write some other code. It depends on the question. But at every stage, what
are the choices?"
    },
    {
      "start": 1553.78,
      "end": 1565.01,
      "text": "What are your choices? This is the main game of recursion. You have to
understand this. So do ask this question. It's a very good question. It's a fan-favorite,
company-favorite question. We'll meet later."
    }
  ]
}
```


16. transactions/t_24.json


```
{
  "video_id": "BdpT6_jotcM",
  "title": "Best Explanation on Reverse LinkList Nodes in K Group | DSA Patterns CPP JAVA
Python",
  "language": "en",
  "model": "large-v3",
  "duration": 3144,
  "segments": [
    {
      "start": 0.0,
      "end": 29.17,
      "text": "Today's question is a highly requested question. Reverse nodes in K-group in a
linked list. This is a hard question on Leetcode. Personally, this is a very close question to
my heart because I had an interview in Arista a year ago. In 35 plus LPA, it was asked the
same question that make reverse nodes in K-group. We could not make it and after that we were
very sad. And before teaching this, we have not only read it ourselves, we have seen the
entire video. We saw this question of Stryverji, we saw this question of pepcoding, we saw
this question of lovebubber. We felt that everywhere it was told that how to solve this
question."
    },
    {
      "start": 29.17,
```

```
      "end": 57.17,
      "text": "that when this question is asked that what group to solve, then solve it like
this. That is not the right way because if you solve this, I have two more questions that you
will not be able to solve. That is why today we will see how to solve any question of this
type. Because it is possible that you may not be asked what group to solve, you may be asked
something else. If you look at the sheet, then above it and below it, there are one question
on both of them that will not be made by you. If you will learn only this that remember this,
how to make this and after that we will give the rest. Come and in this video, especially we"
    },
    {
      "start": 57.17,
      "end": 83.82,
      "text": "We are going to show you by coding because we think that in this video, in this
question, or in this way, it will be beneficial for you to see the code. So stay connected,
come with us. Now, before doing the group of reverse nodes in K, we want to ask you one more
question. The name of that question is swap nodes in pairs. What is its name? Swap nodes in
pairs. So we want to ask you this question before that. Why do we want to ask? We will tell
you, let's see what it means."
    },
    {
      "start": 83.82,
      "end": 111.82,
      "text": "If you are new here, then we highly recommend that you watch episode 44. At
least watch episode 44 so that you can reverse it. Yes, watch episode 43 also. So that you can
reverse the linked list and you get to know that when we were given left and right, which was
discussed in episode 44, what we used to do after that. So if you get to know these two things,
 then it will be fun. If you don't get to know, then also it will be fun. What is swap loading
pair? It is a very simple question that you will be given a linked list."
    },
    {
      "start": 111.82,
      "end": 140.02,
      "text": "You can give a link list like this. 1, 2, 3, and 4. You will be given a list
like this. Its address is 100, 200, 300, and 400. You will be given a list like this. Now,
what do you have to do? What do you have to do? You have to pick a pair from this. What do you
have to do? Pair. So, how many pairs are there? The first pair is this. The second pair is
this. Now, reverse every pair on your own. Reverse every pair. Means, write it like this. 2, 1.
"
    },
    {
      "start": 140.24,
      "end": 169.63,
      "text": "Reverse 4 and 3 in the same way. As soon as the arrow is placed, place it in
the same way. This will look like this and this will look like this. But you can see that your
link list is broken. So don't break it. The arrow that is here should come here. And your new
head will be like this. New head will be like this. Do you understand what the question says?
There is a link list. Break it in pairs. Break it in pairs. Reverse each pair."
    },
    {
      "start": 169.63,
      "end": 199.62,
      "text": "You will find that your link list will be broken, then add the link list again.
And in a bigger example, if we see, 1, 2, 3, 4, 5, 6, 7. There are odd number of notes. Do the
same thing in this too. Break it in pairs. The first pair is this, the second pair is this,
the third pair is this, and the fourth pair is 7. There is no one with 7, so 7 will be left
alone. 7 will be left alone. Reverse it."
    },
    {
      "start": 199.62,
      "end": 229.84,
      "text": "1, 4, 3, 6, 5, and 7 to 7, because 7 is alone. How can we reverse a person
alone? Now, 2, 1, 4, 3, 6, 5, and 7. But now your link is broken, you have to join it again.
You have to join it again. And, this is your new head. You have to return this new head. You
have to return this new head. So, this is your question. This is your question. Okay. Look.
Whenever I give you a question, what do you have to do in this? Look."
    },
    {
      "start": 229.84,
      "end": 247.04,
      "text": "First of all, you should know how to reverse. How to reverse any one linked
list, which we have read in Curve and Prev, you should know that. You should also know that,
if someone asks you to reverse how many times, if someone asks you to reverse 2 times, from
any head node,"
    },
    {
      "start": 247.04,
      "end": 276.58,
```

```
      "text": "Reverse always starts from where? From where the head passes, from where it
starts to reverse. Normally, we write that until the null comes, reverse it. It means that
until the linked list is not finished, reverse the entire linked list. But this is not always
necessary. If someone tells you how many times you have to reverse, how many times you have to
reverse. Suppose someone tells you that you have to reverse 2 times. How will you do that? You
should know that. How will you do that? See. For example, you have a list. What do you have? A
list. You have a small list of 4 nodes."
    },
    {
      "start": 276.58,
      "end": 306.22,
      "text": "This is 100, this is 200, this is 300, and this is 400. Here 1 will be there, 2
will be there, 3 will be there, and 4 will be there. This is 200, this is 300, this is 400,
and this is null. Now, it tells you that do one thing, in the head you have 100, reverse it
from 100. But, don't reverse the whole link list. Do it 2 times, reverse it 2 times and show
us. Let's see how we will do it. What you do is,"
    },
    {
      "start": 306.77,
      "end": 336.54,
      "text": "times is equal to 2 times is equal to 2 Now, what did you do? Curr is equal to,
where did you put the curr? On the head, in reverse, we are talking about reverse Where did
you put the curr? On the head From where did you start the prep? From null, it will be the
same, it will be the same but the while condition in which you used to write curr not equal to
null, it means that till the null comes, till then reverse, reverse the entire length list,
you don't have to write this while times minus minus while times minus minus and"
    },
    {
      "start": 336.54,
      "end": 362.3,
      "text": "The same code you used to write, the same reverse code you used to write, write
that only. What did you write? Note star next is equal to curr's next. We used to store curr's
next. We used to rotate curr's next. Where? On previous. Previous is equal to whom did we
rotate? Curr and we used to take curr forward. We used to take curr forward. What will happen
with this? With this, this work"
    },
    {
      "start": 362.3,
      "end": 391.41,
      "text": "it will be 2 times no matter where the null is coming and where it is not it
doesn't matter it will work 2 times so let's see what will happen with this first of all your
curve will be on 100 your prev will be on null let's reverse it once first of all this person
will rotate this arrow will be cut and this will be null after that once it will be reversed
next time your curve will be equal to 200 and"
    },
    {
      "start": 391.41,
      "end": 416.94,
      "text": "Prev is equal to 100. Do one more reverse. What will happen? This arrow will
also rotate. What will happen to this arrow? It will rotate. And in the end, your curve will
be equal to 300. And prev is equal to 200. But now you will stop. Why? Because you had to
reverse twice. To reverse twice means to pull the arrow back twice. So you did it. Where is
the first arrow? The first arrow is this one. The second arrow is this one."
    },
    {
      "start": 416.94,
      "end": 446.38,
      "text": "you can also do this, that you don't have to reverse it all the time if someone
gives you a note that reverse it from here and reverse it so many times then you can do it
very easily with the help of times so this is also easy now why should we tell you this?
because we are going to need it now what does your question tell you? first of all we will see
for even number of notes we will see for odd number later assume you have a question like this"
    },
    {
      "start": 446.8,
      "end": 476.14,
      "text": "3, 4, 5, 6, 100, 200, 300, 400, 500, 600. Any question like this, do something
like this, do something like this, in that you will be told that you have to keep reversing in
a particular group. What do you have to do in a particular group? You have to keep reversing.
For that, see what you have to do. What do you know first? You know first that if someone
gives you left,"
    },
    {
      "start": 476.14,
      "end": 501.3,
      "text": "If someone gives us a left note and tells us to reverse it so many times, then
we are in a state of reversing it easily. We are in a state of reversing it easily. Okay, you
```

```
can do it. See, what have you done? Do what you know. Your linked list is like this. What have you been told? See, what have you been told? That you have to reverse every pair. Do what you know."
    },
    {
      "start": 501.3,
      "end": 529.36,
      "text": "First of all, which pair will we reverse? First of all, the pair 1, which is 1, 2. First of all, you keep left is equal to 100. Okay? Keep left is equal to 100. Now, you have to reverse it 2 times. Assume that here, before reversing 2 times, remove its right. Remove left and right. How will you remove right? How will you remove its right? What will be the right of this pair? Leave it now. How will you remove it? Assume that someone tells you that right is 200."
    },
    {
      "start": 529.36,
      "end": 557.74,
      "text": "You have to reverse from 100 to 200. Okay. Okay. First of all, suppose this is left 1 and right 1. First of all, you have to reverse. So, how will you reverse? By sending left in the reverse function, you will ask it to reverse 2 times. So, when you do this, you will find that your linked list will look like this. Both these arrows will break. What will happen to both these arrows? They will break and this 2 will point at this. This 2 will point at this. Okay. Is the reverse done? Very good. Okay."
    },
    {
      "start": 557.74,
      "end": 585.66,
      "text": "Now, now, now, what will we do? Reverse the next pair. You don't know anything else. If you want to reverse pair by pair, then reverse pair by pair. Let's see what happens. Next, let's assume your left 2 is equal to 300. Let's assume that someone told you this. Let's assume someone told you that this happens. Okay, no problem. How will the right 2 come out? 400. We will tell you how it will come out. Let's assume it comes out. I mean, it's not that hard. From left, you will get the next node. The next node of left will be your right. So, we will tell you. There is no such rocket science in it."
    },
    {
      "start": 585.66,
      "end": 615.44,
      "text": "Reverse this group too. If you reverse this group too by sending the left, then you will find that your arrow has rotated. It rotated. Very good. Let's move ahead. Left 3 is equal to 500. Right 3 is equal to 600. Why am I telling you this much? Because you will have to do this concept again and again. So, it is not good to remember that you should do this. That is fine. But think of it like this. Reverse this too. When you reverse this, then you will find that"
    },
    {
      "start": 615.44,
      "end": 646.06,
      "text": "Your arrow has been turned like this. Okay? This is yours. Very good. But is this the right thing? Have you reached the right answer? No, you haven't reached the right answer because your linked list is broken. Let's see what answer you wanted. Ideally, what answer did you want? Ideally, you wanted this answer. 2, 1, 4, 3, 6, 5. Here, here,"
    },
    {
      "start": 646.06,
      "end": 673.1,
      "text": "here and here and here now we will see how much work we have done this is your answer by the way this is your answer what you wanted this is your answer ok how much work you have done how much work you have done and how much work is left see we will see is 2 to 1 arrow in your answer yes yes it is in my answer you have made 2 to 1 very good very good"
    },
    {
      "start": 673.1,
      "end": 701.97,
      "text": "Is the arrow from 1 to 4 in your answer? No, there is no arrow from 1 to 4. See, there is no arrow from 1 to 4. No, there is no arrow from 1 to 4. No problem. So, mark this. No problem. Is the arrow from 4 to 3 in your answer? Yes, the arrow from 4 to 3 is in my answer. Is the arrow from 3 to 6 in your answer? No, there is no arrow from 3 to 6. Is the arrow from 6 to 5 in your answer? Yes. So, you have made half the answer. Half the answer is left. Half the answer means you have made a small part of it. A small part of it is left. Let's see which parts are left."
    },
    {
      "start": 702.26,
      "end": 731.6,
      "text": "First of all, this is 100 and this is 400. One is left, 100.next is equal to 400. This step was left. Okay. And what is left? 300, 600. 300.next is equal to 600. This step
```

```
is also left. Okay. Now, we can't write it like this by hard coding. Okay. Now, 100.next. 100.
next is equal to what? 400."
    },
    {
      "start": 731.66,
      "end": 759.95,
      "text": "The variable you had, what was it? Left 1, right 1, left 2, right 2, left 3,
right 3. Look at it according to that. Isn't this yours? Left 1 dot next is equal to right 2.
Left 1 dot next is equal to right 2. Yes, 100 dot next is equal to 400. If you had done it, if
you had come here and done it, that left 1 dot next is equal to right 2. If you had done this,
then it would have been done. You would have done this step. And,"
    },
    {
      "start": 760.08,
      "end": 790.3,
      "text": "Left 2 dot next is equal to right 3. Left 2 dot next is equal to right 3. If
you had done this step, then 300 next would have become 600. Your entire answer would have
been ready. And what is your head? What is your new head? 200. What is your right 1? Right 1
is your head. If you had done this step along with reverse, if you had done this step along
with reverse,"
    },
    {
      "start": 790.3,
      "end": 816.43,
      "text": "When you reverse left 2 and right 2, then left 1.next is equal to right 2. Left
1 is the previous left. When you reverse left 3 and right 3, then left 2's next is equal to
right 3. If you want to make a head and store the value of right 1 and return it, then you can
make this question."
    },
    {
      "start": 816.43,
      "end": 845.22,
      "text": "Okay? So, after reversing, you have to make half an answer. After that, you
have to see what other strings we have to connect, what other connections we have to make so
that we can make this question. So, you got to know that we have to do these things. Come,
let's see. Let's see more. Don't worry. If you don't understand, you will understand. I
promise you that by the end of this, you will be able to write this program even in your sleep.
 Let's go. Let's take one more example. Let's take one more example. Odd number of nodes. Odd
number of nodes."
    },
    {
      "start": 845.22,
      "end": 875.01,
      "text": "This is also possible. It will not be always even. Okay. 100, 200, 300, 400,
500, 600, 700. And it can be done on the number of nodes, right? Yes, it can be done. How many
pairs will be made in this? According to me, there will be 4 pairs in this. 1, 2, 3, and 4.
There will be a single node in the 4th pair. So, let's see. Let's start with left 1, right 1.
And we will see what happens in left 1, right 1."
    },
    {
      "start": 875.01,
      "end": 904.75,
      "text": "Let's say, left 1 is equal to 100. How will the left 1 and right 1 come out and
how will the left 2 come out? We will see all that. It is a normal calculation. We are trying
to make you understand that what you have to do in this is that you have to reverse every pair.
 We are doing the same thing that we are putting our focus on every pair and reversing it. And
after doing everything, we see which wires are left and which connections are left and we have
to connect them. Left 1 and right 1 is equal to 200. What does left and right denote?"
    },
    {
      "start": 904.75,
      "end": 931.74,
      "text": "which note to start and which note to reverse, left and right denotes that you
don't need right in reverse function, but you need right somewhere else, that's why we are
writing, just for simplicity we are writing for simplicity, ok when you will reverse this,
then you will get to know that these two arrows are broken these two arrows are broken, ok, no
problem come forward, left 2 is equal to 300 right 2 is equal to 400"
    },
    {
      "start": 931.74,
      "end": 960.45,
      "text": "Reverse this too. How? By sending it in the reverse function or by writing it
there. You have to reverse 2 times. From where? Start from left 2 and reverse 2 times. Do it.
This arrow also broke. This arrow also broke. Sorry. So this arrow became here. Very good.
This is very easy. If someone tells you to keep doing this, then you can keep doing it. It is
very easy for you. Left 3. Yes, it is easy when you have seen the reverse. Now someone will
say that we have never seen the reverse. So why are you seeing this for the first time? 500,
```

```
600."
    },
    {
      "start": 960.45,
      "end": 979.07,
      "text": "If you reverse this, then you will find that both these arrows will also break.
What will happen to both these arrows? They will break. So, this arrow will be made here.
Okay? Even now, one pair is remaining. Now, left 4 is equal to 700. But what will happen to
right 4? Right 4 is equal to null. Because nothing else is left."
    },
    {
      "start": 979.07,
      "end": 1004.27,
      "text": "what was right? starting from which node to which node? so you are always
seeing that if you want to reverse pairwise, how many nodes are there in pair? two nodes, so
there are two nodes in one group, what is there in one group? there are two nodes, so you used
to see that left was 100, right was next to that, so here left is 400 is equal to 700, next to
that is null, so it is fine, you will reverse this also, if you reverse this, nothing will
happen, if you reverse this, nothing will happen, there is nothing behind it, so it will
remain the same,"
    },
    {
      "start": 1004.37,
      "end": 1032.35,
      "text": "Your linkage looks like this. But how should your answer look? Your answer
should look like this. 2, 1, 4, 3, 6, 5, and 7. Let's see. 2, 1, like this, like this, like
this, like this. Which strings are there and which are not? Is there a 2, 1 string? Yes, there
is a 2, 1 string. But is there a 1 and 4 string? No. There is no 1 and 4 string. Okay."
    },
    {
      "start": 1032.35,
      "end": 1062.1,
      "text": "Is the wire of 4 and 3? Yes, you have connected the wire of 4 and 3 in reverse.
But the wire of 3 and 6 is not there. Is the wire of 6 and 5? Yes, the wire of 6 and 5 is
there. But is the wire of 5 and 7? No, the wire of 5 and 7 is not there. So there are no 3
wires. And coincidentally, which 3 wires are there? 100, 400. 300, 600. 500, 700. See."
    },
    {
      "start": 1062.1,
      "end": 1088.29,
      "text": "100.next is equal to 400. What happened here? Left 1.next is equal to right 2.
Left 1.next is equal to right 2. That's why we took right. Previous left's next is equal to
now's right. Previous left's next is equal to now's right. This line is connected. Left 2.next
is equal to right 3. Yes. Left 2.next is equal to"
    },
    {
      "start": 1088.29,
      "end": 1111.01,
      "text": "Right 3. Previous one left next is equal to current one right. Yes. This wire
is connected. This wire is connected. 500.next is equal to right 4. 500.next. Sorry. Now see.
Now it is wrong. Now if you write this. Left 3. See. Left 3.next is equal to right."
    },
    {
      "start": 1111.01,
      "end": 1131.42,
      "text": "4. So what will happen is that 500.next is equal to null. So 500.next will
remain null. But you don't want null. You don't want null. You want 500.next is equal to 700.
Where is this 700 stored? It is stored in left 4. That's why if any part is saved, if any last
part is saved,"
    },
    {
      "start": 1131.42,
      "end": 1151.63,
      "text": "7 is saved alone, 7 is not reversed. Now see, why 7 is saved? Because 7 has
only 7 nodes. What do you need by law? Next to 500 should be 700. Next to 500 should not be
null. So, if any part is saved, if any part is saved in reversing, how will you know that it
is saved? Its right will be null."
    },
    {
      "start": 1151.63,
      "end": 1175.52,
      "text": "its right will be null, which means this whole part can't be reversed, so it's
a remainder part. What is 7? 7 is a remainder part. 7 can't be a pair, so 7 is left. So how
can we reverse in 7? We can't. That's why, left 3.next is equal to, not right 4, left 4.
Because right is null."
    },
    {
```

```
      "start": 1175.52,
      "end": 1205.57,
      "text": "This part is saved. This part is saved. That's why left 3 dot next is equal to
left 4. Left 4. In which? The part that will be saved in the last part. In the last part. How
will we find out? We will tell you. You have added this part too. You have added this part too.
 So, what did we find out? What did we do in total? See. First of all, how many variables do
we need? One you need is left. One you need is left. Means, to track current left. From every
left, from every left,"
    },
    {
      "start": 1205.57,
      "end": 1226.5,
      "text": "We don't do arrow like this. From every left, you have to take out the right.
What do you have to take out? You have to take out the right. Yes. You have to take out the
right. You have to take out the right. Okay? So, left and right are done. What else do you
need? Do you need anything to reverse? No, you don't need anything to reverse. Why? You know
the times, right? How many times do you have to reverse?"
    },
    {
      "start": 1226.5,
      "end": 1254.19,
      "text": "We have to reverse every pair. How many nodes are there in the pair? 2 nodes.
We have to reverse 2 times. Very good. So, it will be reversed. After that, the last
connection that we used to connect. What did this connection say? Previous left. Previous left.
 Dot next. Is equal to current right. Yes. Current right. Current right. So, what do you want?
You will always want previous left. Previous left."
    },
    {
      "start": 1254.19,
      "end": 1284.18,
      "text": "notice, when we reversed the first pair, left 1 and right 1, at this time we
didn't connect any wire, because there was no previous left when you came to the first left,
then previous left was null, so here according to now, if we talk about previous left, then it
was null, so how will we connect the wire of null? yes, after this, when you came to left 2,
then your previous left was left 1, so after that you went connecting the wire, so do you need
one more variable? previous left"
    },
    {
      "start": 1284.18,
      "end": 1313.74,
      "text": "Previous left. Okay. One more thing. Now, this left 1 and left 2, left 3, left
4. Now, it looks easy to see that you have written it like this. But, how is it written in
real? See. For the first time, you know. Now, we see. Slowly, the concept is done. So, we are
slowly entering the implementation. Slowly, they are clearing all the things. See. 1, 2, 3, 4,
5, 6, 7. Okay. 100, 200, 300."
    },
    {
      "start": 1313.74,
      "end": 1341.41,
      "text": "400, 500, 600, 700. Okay? This, this, this. Don't worry. We will write the code
from start to end. Okay. What will be the first left? Obviously, it will be 100. What will be
the first left? It will be 100. It will be the first left, right? Okay. How will we subtract
right from left? How will we subtract right from left? Look. You..."
    },
    {
      "start": 1341.41,
      "end": 1369.14,
      "text": "what does right mean? right means that in that group, here also it is a matter
of group, if you think carefully then it is also a group, it is a group of two nodes, you have
been told that swap nodes in pairs means reverse every pair, reverse every pair, so this is
only a group, how much is the size of the group? 2, okay, so you have left is equal to 100, so
this is one node, how many more nodes will you have to go ahead so that you can keep the right
on the right node, so you can do this for"
    },
    {
      "start": 1370.0,
      "end": 1399.7,
      "text": "Int i is equal to 0. Some people will say, why are you doing this? Let's do
right is equal to left dot next. Let's do right is equal to left dot next. Why are you not
doing this? You will understand in some time. Have some patience. You must be thinking, what
is all this nonsense? Right is equal to left dot next. You must be thinking. Left is 100. You
have to keep right until that group ends. So, there are two groups. So, why are you not doing
right is equal to left dot next? Why are you not doing it? You will know in some time. And you
will be grateful that why are you not doing it. i is equal to 0. i less than."
    },
    {
      "start": 1399.7,
```

```
      "end": 1430.96,
      "text": "What is the size of the group? 2. If you do i less than 2 minus 1. If you do
this and do i plus plus. And before starting, write is equal to 100 initialize. In this loop,
if you do write is equal to write dot next. What will happen? Let's see. First of all, write
is equal to 100. Okay. Now what is written? i is equal to 0. i less than 2 minus 1. Means i
less than 1. So assume when i is equal to 0."
    },
    {
      "start": 1430.96,
      "end": 1455.28,
      "text": "So what will be your right? Condition is satisfied. What will be your right?
Next to right, 200. Okay? I++. I is equal to 1. Now this condition will fail because what have
you written in the condition? You haven't written I less than 2. You have written I less than
1. So now this condition will fail. See, your right has reached 200. Right has reached 200,
which is a trick. You can remove it anyhow. See, from left,"
    },
    {
      "start": 1455.28,
      "end": 1485.25,
      "text": "You have to keep the right on the node where the group ends. The size of the
group is 2. Now, no matter how you think about it, from where we will take out the right, how
we will take it out, you have to keep the right in the right place. It also shows in a way, it
also tells in a way that see, the size of your group is 2. Your left, your left has already
come here on a node. You have also kept the right on this node. So how many elements are left
in the group? How many elements are left in this group except this node? Only one element is
left. In the group of two, you have kept it on one element."
    },
    {
      "start": 1485.25,
      "end": 1501.87,
      "text": "How much is left? 1. So you have to increase it one more time, then right will
increase. If the size of the group was 3, then you would have kept both left and right here.
And leaving this, how many children are there in the group? 2 children. So you would have to
increase it 2 times. That's why we are writing that see, now the size of the group is 2, so
how many times did you have to increase? 1 time."
    },
    {
      "start": 1501.87,
      "end": 1525.1,
      "text": "If the size of the group was 3, then how many times did you have to increase? 2
times. That's why you have to increase 1 less than the size of the group. That's why we have
written i less than 2 minus 1. Not i less than 2. i is equal to 0. i less than 1. So this will
work once. Because the size of the group is 1, so it will work once. By doing this, you can
take out the right. By doing this, you can take out the right. By doing this, you can take out
the right. Okay? Now it is very easy. Now it is very easy."
    },
    {
      "start": 1525.1,
      "end": 1551.74,
      "text": "Now you will reverse it and do it. But, but, but, this is your left 1. This is
your left 1, right 1. You have taken it out by showing it from the head. How will the left 2
come out? How will the left 2 come out? What was left 2? What was above? Left 2 was 300. What
was left 2? 300, 300. Okay. Why did this 300 come? When you reverse one pair, reversing one
pair means that your left was here."
    },
    {
      "start": 1551.74,
      "end": 1578.82,
      "text": "And the right was here. This means that 100 to 200 is reversed. From the next
node, you start the next group. From the next node, you start left 2. Then here goes right 2.
Then from the next node of right 2, you start left 2, 3. Then here goes right 3. After that,
from the next node of right 3, you start left 4. So, you can do one thing. You can take a
variable. What?"
    },
    {
      "start": 1578.82,
      "end": 1608.02,
      "text": "Next left. Yes. Next left is equal to what? Right's next. Right's next. Means.
That current right. Suppose right is at 200. Next to 200. Store 300. When all the work is done.
 When all the work is done. Then what do you have to do at last? Left is equal to next left.
And this process has to be repeated. What do you have to do with this process? It has to be
repeated. It has to be repeated. Let's see this. Let's see what it means. What does this
process mean? Let's see."
    },
    {
      "start": 1608.34,
      "end": 1637.09,
```

```
      "text": "Let's hope that it will be clear. According to me, we are teaching very well. We don't know how well it is going. Okay. Let's go. One. This is a little difficult. When you make a recorded video, you don't get feedback. Like when we teach live, we keep finding out whether you understand or not. The kids stop us saying that we didn't understand this thing. Here we have to guess whether we understood or not. That's why sometimes you think that we are repeating the same thing again and again. So we don't want to take a chance."
    },
    {
      "start": 1637.09,
      "end": 1666.02,
      "text": "because it is possible that you have understood by 90% chance and you have not understood by 10% chance so we don't want to take that chance that it is okay, it must have come because then it doesn't look good, right? let's go what did we say? in the beginning, left is 1 and left is 2 in the beginning, left is equal to 100 how will the right come out? how will the right come out? by running the loop we will increase it once more first of all, we will make right equal to left so right is equal to 100 now if we increase right once more then right will become 200"
    },
    {
      "start": 1666.02,
      "end": 1692.48,
      "text": "What will be right? 200. Now, we will store the next of right for next left because next time it is necessary. What happened? 300. Very good. Now, what to do? Nothing to do. We have to reverse. Where? We have to send left. We have to send left and tell it to reverse 2 times. What to do? Reverse 2 times. So, we are hoping that you will get a reverse program. We will write a function in this. Reverse. What?"
    },
    {
      "start": 1692.48,
      "end": 1720.38,
      "text": "We will send left in this and 2 times, reverse it 2 times. So, as soon as you will reverse, then these two arrows will disappear. Both the arrows will disappear. That's it. Now, now, now. Now, we don't have to do anything. We have to move ahead. But, but remember one thing. We had done one thing after every reverse. Previous left, previous left dot next is equal to current right. So, see. Now, if you are first time, then now your previous left is equal to null. Now, your previous left is equal to"
    },
    {
      "start": 1720.38,
      "end": 1751.68,
      "text": "It will be null. Now this is fine. Now this is fine. But, when you will go to left 2, when you will go to left 2 next time, then you will get the work of previous left. Then you will get the work of previous left. That's why, that's why, we will do two things. First thing, what will we do? If previous left. Means, if any left had come earlier, now it has not come, now it is the first time, then it is fine. But if it had come, then what would we have done? Previous left dot next is equal to what? Previous left dot next is equal to what happened?"
    },
    {
      "start": 1751.68,
      "end": 1781.87,
      "text": "Current wala right, current wala right, abhi ka right, abhi ka right, yes, abhi ka right is equal to right. Okay, ye toh ho gaya. Lekin, ab aage ke liye, aage ke liye, previous left ko yaad rakhna padega, kyunki aapka left badalne jaa raha hai, aapka left hamesha badalta raha hai. Toh, previous left is equal to left. Yes, yes, ho gaya, bas. Ab aap apne left ko badha dijiye. Ab aapne left ko kaise badha dijiye? Left is equal to, isi liye ke next left ko rakhe hue the. Next, left."
    },
    {
      "start": 1783.02,
      "end": 1811.07,
      "text": "Do like this. Remember that now we are dealing for even nodes. Okay, we will deal for that too. So this is your small thing. Okay, let's see what happens. Next, what will happen? Left is equal to next left. So left is equal to what? 300. Where will right start from? From left. How many times will right have to be increased? Once. If we increase it once, what will right become? 400. First of all, what do we have to do?"
    },
    {
      "start": 1811.07,
      "end": 1837.97,
      "text": "store the next left what is the next left? the next of 400 is 500 after that what you have to do? reverse left and 2 reverse left and 2 means start from 400 and reverse 2 nodes start from 300 and reverse 2 nodes and reverse 2 nodes is it reversed? ok now if prev left if prev left what is prev left? yes yes yes"
    },
    {
      "start": 1837.97,
```

```
      "end": 1865.26,
      "text": "Now, the value of your prev left is not null. The value of your prev left is
the previous left. What was the previous left? 100. So, if it is there, then what had to be
done? prev left.next is equal to right. prev left.next is equal to right. Means, 100.next is
equal to 400. 100.next is equal to 400. 100.next is equal to 400. Yes. And next time, before
going ahead, prev left is equal to left."
    },
    {
      "start": 1865.26,
      "end": 1895.54,
      "text": "Prev left is equal to left. So, what happened after changing prev left? 300.
300. Yes. Yes. After that, what do we have to do? Left is equal to next left. So, left is
equal to 500. Come forward. Left is equal to 500. Now, I hope you have understood Dryden. I
mean, there is no complete code. Now, we will write it. We will write it easily. Is equal to,
what will be right? It will start from 500. It will increase once. It will increase once. So,
what will happen?"
    },
    {
      "start": 1895.54,
      "end": 1925.39,
      "text": "600. Yes. First of all, we will store next left. What will be the next left
stored? 700. Okay. After that, we will reverse here. By sending left, and we will say, reverse
2 times. So, it will reverse 2 times. So, what will happen? These two arrows will disappear.
These two arrows will disappear. And this arrow will come here. What to do? If previous left
is there, if previous left is there, what is previous left? Yes, that is 300. So, previous
left dot next is equal to current right. Current right means,"
    },
    {
      "start": 1925.39,
      "end": 1952.56,
      "text": "300.next is equal to 600. 300.next is equal to 600. Yes, yes, yes. And before
going, Prev left is equal to Prev left is equal to left. Prev left is equal to left. So 500.
So what is Prev left is equal to? 500. And left is equal to next left. Left is equal to next
left. Is equal to 700. Okay."
    },
    {
      "start": 1952.56,
      "end": 1980.69,
      "text": "And what are these things happening? These things are happening in while loop,
by the way. We are imagining that we are doing this thing in while loop, in infinite times.
Okay? Now let's see. Now look very carefully. Left is equal to 700. Left is equal to 700. What
will be right? Right will start from 700. But when you will increase right once, then what
will be right? Null. Null. Right will be null. So,"
    },
    {
      "start": 1980.82,
      "end": 1998.66,
      "text": "We have to pay attention in our for loop that you keep increasing right. In for
loop you keep increasing right but you have to pay attention that right is null. Now see right
is null. Now right is null. Why right is null? Right is null because see your entire linked
list is finished."
    },
    {
      "start": 1998.66,
      "end": 2024.7,
      "text": "your complete link list is over and only one part is left which is not complete
now what you had to do remember what you had to do you had come here in this condition you had
one part left which is not complete now your program has come to an end there is no need to
reverse it why?"
    },
    {
      "start": 2024.7,
      "end": 2054.74,
      "text": "Because you had to reverse the pair. You had to reverse by doing 2 2. Now there
is only 1 in this. So how will we reverse 1? It is not even in the question that we will not
do this. But. But. Now you can finish the program. You can finish the program. But. Before
going. Left 3 dot next is equal to left 4 means. The previous left. Every time you used to do
that previous left dot next is equal to right. No. What to do in this? What to do in this?
Previous left. Previous left."
    },
    {
      "start": 2054.74,
      "end": 2084.42,
      "text": "Dot next. Is equal to left. Yes. Left. Means. What was previous left? 500 dot
next. 500 dot next is equal to 7. That's it. Now. Can you finish your program? Can you finish
your loop? Whatever you were running while loop. Is there any need to do anything else? No. We
reached last. There was a part of last. We don't have to reverse it. Previous left has
```

```
connected it. Now you don't have to do anything. Now what you have to do? You have to break your while loop. Break your while loop."
    },
    {
      "start": 2084.42,
      "end": 2109.12,
      "text": "You have to break your while loop. What do you have to do? Your program has been broken. And what do you have to return in the end? Rest. Yes, see one thing. Rest was not made anywhere. What was rest? Rest was the head of your new note. What will always be the head of the new note? The first, first, first right, first right, first right 1, right 1, right 1. The first right will be reversed. See 2, 2, 2. So?"
    },
    {
      "start": 2109.12,
      "end": 2131.73,
      "text": "every time when you will reverse then what you have to pay attention here before doing this if my rest is equal to null rest is equal to right means in the beginning rest will be null as soon as you come to know that one left right is reversed then what you have to do is"
    },
    {
      "start": 2131.73,
      "end": 2164.27,
      "text": "it has to do res equal to write which means it has to do res on the first right so this was its small dry run by doing this you can do the whole code now let's write the code let's write the code and there are some edge cases here and there you will understand how this works what will be given to you first we will write here then we will go to the computer and then we will see node start you will be given any function in which you will be given a node start head"
    },
    {
      "start": 2164.27,
      "end": 2194.51,
      "text": "First of all, if head is null, then what you have to do is return head. After that, start the program. First of all, what you need is a node star left is equal to head. Okay. You need one node star for result which should be null. Result means head, new head. Remember one, what you needed was previous left. Previous left. Okay."
    },
    {
      "start": 2194.58,
      "end": 2224.94,
      "text": "In the beginning, this is also null. We also need right. So, take right. But don't assign right now. Or you can do it as you wish. You can do it or not. This much is done. This much work is done. Let's start while loop. While loop. Till when? Do you know how many pairs are there in it? Do you know how many pairs are there in linked list? No, you don't know.  Because you don't know the length of linked list from before. Use infinite loop for this. Use infinite loop. Now, now, first of all, what did we do? Right is equal to left."
    },
    {
      "start": 2225.07,
      "end": 2255.41,
      "text": "Right to left. And now let's run our loop. How? Now you have to find the right. How? If the right has come to the left, what is the size of the group? What is the size of the group? It is 2 for now. Okay. So run the for loop. Now, now, i is equal to 0. i less than. Imagine here we assume that int size is equal to 2. What is this size? Which size do you want to reverse? I mean, we want to reverse every 2, 2, 2, 2. So size is 2. So what did we see? i less than."
    },
    {
      "start": 2255.54,
      "end": 2285.26,
      "text": "Size minus 1. I plus plus. Now, in this, we used to write right is equal to right dot next. But, there can be a condition that right can be null. There can be a condition that right can be null. So, you do this. If right equal equal to null. Stop it. Break. Don't do it after that. If right is null, then don't do null dot next. Don't do it. If not, then fine. Right is equal to right dot."
    },
    {
      "start": 2287.15,
      "end": 2318.0,
      "text": "Now, your left and right are set. But, but, now you have to check. Now, what you have to do? You have to check. The code may be small, but okay. What you have to do? You have to check. See, if right is not null, then it is normal. Then it is normal. Then the linkage is left. So, let's do that first. If right. Come on. Right is null. No, it is not. Very good. Yes. Now, we can do our own. What can we do? First of all, reverse. First of all, reverse. So, one reverse function. We will write reverse function. No, sorry."
    },
```

```
    {
      "start": 2318.0,
      "end": 2347.76,
      "text": "First of all, keep the one next to right. What will happen to it? It is your
next left. It is your next left. Keep it. Don't leave it. Right dot. Next. Keep it. Very good.
Now reverse it. What will you send in reverse? Left. And how many times do you have to
reverse? Two times. Where was two? Size. Send size. Send size. Reverse it. Reverse it. That's
it. What was the next step? Previous left. If it is"
    },
    {
      "start": 2347.76,
      "end": 2376.29,
      "text": "If previous left. Then what we had to do? Previous left dot next is equal to.
Right now. Right now. Right now. And now update previous left. Previous left is equal to left.
Previous left's work is done. You have to update the result. If result equal equal to null.
Result is equal to right. Result is updated."
    },
    {
      "start": 2376.29,
      "end": 2405.97,
      "text": "Previous left's work is done. Is there any work left? No, there is no work left.
 Left is equal to next left. You have moved on to the next group. It is not over yet. It is
not over yet. This was that if right is correct, then left right. But what did we do in the
last 7? Else. Now right is null. When 7 was null, what did we do? When 7 was null, what did we
do? Did we do something reverse? No, no, no. We just did this. Previous left dot next is equal
to left. So do that here too."
    },
    {
      "start": 2406.38,
      "end": 2437.94,
      "text": "If previous left, then previous left dot next is equal to left. Is it done?
Very good. Now, should we break the loop? No, no. Assume that there is only one list. Assume
that there is only one node. You have only one node. You have only one node in the whole
question. See what will happen in this. Left is equal to 100. You will start while loop. Right
is equal to 100."
    },
    {
      "start": 2437.97,
      "end": 2459.55,
      "text": "If you increase it once, then your right will be null. In that case, you will
feel that the linked list is over and it is done. You have only one node, not even one pair.
You will see that if I have previous left, then previous left.next is equal to left. But in
the beginning, there is no previous left because this is the first left. This is the first
left and there is nothing in the previous left. So you will break it. But pay attention."
    },
    {
      "start": 2459.55,
      "end": 2488.34,
      "text": "your result is still null your result is still null still null still null so
you will return null you didn't have to return null you had to return 100 so for this suppose
if there is only one node you can check here if my result is equal to null see if before this
you get any left or right left or right is fine your result is not null your result is right
but if result is null means you didn't get anything then what should you return"
    },
    {
      "start": 2488.34,
      "end": 2509.01,
      "text": "Result is equal to left. Result is equal to left. Why? 100. It's the same. Left.
 Right is not there. Right is finished. If you are getting result null, that means, you didn't
get anything before this. You didn't get anything. You didn't do anything. So, result is equal
to left. Result is equal to left. So, if result equal equal to null."
    },
    {
      "start": 2509.71,
      "end": 2538.96,
      "text": "Result is equal to left. Yes, now you break it. Your program is finished. What
is happening? This infinite while loop, this while loop, it is breaking the while loop. It is
breaking the while loop. That's it. By closing the bracket of while loop, what should we do
here? Return, rest. Return, rest. So, this is done. We will write the reverse function. We
will write the reverse function. We will write as we write. Come, we will show you its code.
We will show its code on the screen. We will not show it here. So, here is your question."
    },
    {
      "start": 2538.96,
      "end": 2565.33,
      "text": "Come, let's start. So, these are some macros that we keep for ourselves. You
can make it or you can not make it. So, we will not write any new code. Whatever code is
```

```
written, it will be written. Whatever code was written, it will be written. These are some formatting things. You don't need to see this. Let's go. What was written first? If head equal equal to null. Return."
    },
    {
      "start": 2565.81,
      "end": 2596.77,
      "text": "Yes, someone will say that give this code, copy paste this code and give it. No, we will not give it. Why will we give it after copy pasting? We don't understand its significance. Why do you need it? Why do you need a copy pasted code? I mean, we are not able to understand its logic at all. Done. What else did we need? Res is equal to null. Previous left is equal to null. One size, because you have to reverse in pair, so take size equal to 2. Is there anything left to take? No. We will see if there is anything left. Why?"
    },
    {
      "start": 2596.77,
      "end": 2625.74,
      "text": "true, infinite loop, yes, infinite loop, yes, first of all, right is equal to left, after that for int i is equal to 0, i less than size minus 1, i plus plus, see inside that if your right is equal to null, go out, go out, break, break, i loop is happening, i loop is not happening, otherwise right is equal to right dot next, ok, now,"
    },
    {
      "start": 2626.13,
      "end": 2654.98,
      "text": "If you have a right, it means you have got a left right. Very good. Let's do our thing. First of all, what did you do? Node left. First of all, what did you do? Node next. Next. Left is equal to right dot. Next. Very good. Sorry. Right dot. Next. Okay. After that, what did you do? After that, you did nothing. After that, you did reverse. You did left."
    },
    {
      "start": 2654.98,
      "end": 2686.22,
      "text": "We used to send size. Let's see the function of reverse. Pay attention that reverse is not returning anything. It is not returning anything because it is not needed. Void reverse. Node star head. Int. Times. Means how many times do we have to do? How many times do we have to reverse? So, node star. Curve is equal to head. Node star. Prev is equal to null. Bus. While. Times. Minus. Minus. And write the same reverse code. Next is equal to curve's next. Curve's next is equal to."
    },
    {
      "start": 2687.12,
      "end": 2714.67,
      "text": "prev prev is equal to curr and curr is equal to next and we don't have to return anything pay attention we don't have to return previous it's a void function we don't need it because see why did you return previous because the one you reverse we need its new head we know that when you reverse left to right then right will be its new head so we know we don't need it reverse is done now if my previous left then"
    },
    {
      "start": 2714.67,
      "end": 2743.86,
      "text": "Previous left's next is equal to current right. Yes. Current right. Current right. Okay. And previous left is equal to left. Aage kaam aega. Aage kaam aega. Or, result. If result equal equal to null, toh result is equal to right. First right aapka answer hoga. Answer. Matlab head. Aapka head kya hoga? Pehla head. Pehla right. Sab kuch hone ke baad, left is equal to next left. Ye toh hua ki if right hai. Else,"
    },
    {
      "start": 2743.86,
      "end": 2763.31,
      "text": "Else, what does this mean? Everything is over. But you have to do one thing. Previous left dot next is equal to, if, if previous left is there, if previous left is there, if previous left is there, then previous left's next is equal to left, not right. Right is not there, right is not there, that's it."
    },
    {
      "start": 2763.31,
      "end": 2801.95,
      "text": "And if res is equal to null, which means if there is only one node in the linked list, then res is equal to left. Not right, but left. Because the one that is left is left only. The right has nothing. And after else, your break is over. After the while loop, what will we do? Return res. Let's see if something goes wrong. We will debug. It's taking a long time."
    },
    {
      "start": 2801.95,
```

```
      "end": 2832.1,
      "text": "Okay? Yes, 100% efficiency means very nice code, very nice code. Okay. Now, now,
 now, to do this, we came to teach you this question, we came to teach you reverse notes in K
group, but we have wasted 40-45 minutes on this question. Why have you done that? Why have you
done that? Come, let's tell why have you done that? Come, let's tell why have you done that?
Why have you wasted so much?"
    },
    {
      "start": 2832.1,
      "end": 2861.5,
      "text": "So why was it so easy? Because it was so easy. Now come. Now you are ready to
win this question. Reverse in K group. What does the question say? Suppose K has 3 values. So
make a group of 3. Make a group of 3. Leave the last one. Reverse every group. Reverse every
group. 3, 2, 1, 6, 5."
    },
    {
      "start": 2861.5,
      "end": 2891.25,
      "text": "4, 8, 7. When you reverse, this will be done. But, this arrow, this arrow, join
and return. Return this. Isn't this question the same as the previous one? The previous
question was the same. The previous question was the same. Reverse in K. K's value was 2 last
time. This time, K is given. This time, 2 is given. See. Open your mind. Open your mind."
    },
    {
      "start": 2891.25,
      "end": 2922.46,
      "text": "Open your mind. If you don't open your mind, you won't be able to do anything.
6, 7, 8. Come. Make a group of 3. Come and see what is left 1 and right 1. Left 1. Right 1.
What is it? 100. And what is right 1? 300. It's a group of 3. Left 2. Right 2. What is it? 400.
 600. Left 3. What is it? 700."
    },
    {
      "start": 2922.46,
      "end": 2949.58,
      "text": "And what is right 3? Null. Right 3 is nothing. Where is this group complete?
The group is not complete. And in the question, it is said exactly that if a group is not
complete, in the last, if a group is saved, it has to be left as it is. It doesn't have to be
reversed. It doesn't have to do anything. It doesn't have to do anything. Look. First of all,
here, reverse it. Left 1, right 1. It will become such an arrow. It will become such an arrow.
Reverse left 2, right 2. It will become such an arrow."
    },
    {
      "start": 2949.58,
      "end": 2978.78,
      "text": "Left 3, right 3, don't do anything. More, more, more. After reversing this,
after reversing this, what is previous left? 100. What is right now? 600. 100.next is equal to
600. Look. After doing this, after doing this, because right is null here, means your program
is over. What did you do last time? Previous left.next is equal to left, not right, left. 400.
next is equal to"
    },
    {
      "start": 2978.78,
      "end": 3007.58,
      "text": "700, 700, 700. And this will remain the same. And what do we have to return?
The first right, the first right, the first right. This is not exactly the same. Look. 3, 3. 2,
 2. 1, 1. 6, 6. 5, 5. 4, 4. After 4, after 4. What is after 4? What is after 4? Sorry, we also
reversed this by mistake. This will not be reversed. The last group is not reversed."
    },
    {
      "start": 3007.58,
      "end": 3037.79,
      "text": "7, 8, what is after 4? After 4, 7, 8. This is exactly the last question. What
will change? Nothing. The value of size will change. Let's see how easy it will be. We will
not change any code. We will copy and paste the whole code. We will copy and paste one thing.
Let's see. We will copy and paste the whole code. We will copy and paste each line. First of
all, copy and paste the reverse function."
    },
    {
      "start": 3037.79,
      "end": 3067.86,
      "text": "Copy and paste from here to here only. This is... This is annoying. Copy. See
this. Reverse notes in K group. Reverse notes in K group. Lead code hard. Lead code hard.
Nothing is hard. Copy and paste. Copy and paste. K is given here. K is given here. What is K?
This size. This size. This size is equal to 2. This is K. This is K. This is K. Copy and paste
it completely. Copy and paste it completely."
    },
    {
```

```
      "start": 3067.86,
      "end": 3104.53,
      "text": "Now let's see. Pick up the whole thing and copy paste it. Pick up the whole
thing and copy. Paste. On the top, instead of size is equal to 2, size is equal to k. Size is
equal to k. See. See. What did we say? It's exactly the same question. One line, one line has
to be changed. Size is equal to k. That's why, see, we teach you this, right? If we teach you
this first, then you think that, okay, this will be made like this. But you have to think like
this for every question."
    },
    {
      "start": 3104.53,
      "end": 3132.88,
      "text": "think of all the reverse questions like this, first reverse whatever you have
said, then connect it with your mind, so this is the code, now we will not give this code, do
this code yourself, if you will take this code and do it, then we will not understand, if we
will copy paste this code and give it to you, then what will you do, will you paste it too, no,
 don't do it, write it yourself, we have written it ourselves, we have written it in front of
you, it was not that we have printed it from somewhere, no, we have written it ourselves, so
put the habit of writing it yourself,"
    },
    {
      "start": 3132.88,
      "end": 3143.63,
      "text": "and do make this question, both the questions tomorrow we will make a harder
version of this question till then thank you very much for coming"
    }
  ]
}
```


17. transactions/t_25.json


```
{
  "video_id": "Bt9rgQgGf64",
  "title": "DSA PATTERNS Episode 24 | Kadane’s Algorithm Pattern: Quick Revision",
  "language": "en",
  "model": "large-v3",
  "duration": 2294,
  "segments": [
    {
      "start": 0.0,
      "end": 29.28,
      "text": "Welcome to the channel. After completing the complete revision of the cadence,
what will we do with the cadence pattern today? It's over. Let's go. The sheet beats are in
the description. They are all good things. First of all, how to identify? How to identify that
whose question is it? It's the question of the cadence. First of all, there will be an array
question. It will be of the array, not even of the string because the cadence is basically in
the string. Cadence is the sum of all arrays. Sum of all arrays."
    },
    {
      "start": 29.28,
      "end": 59.89,
      "text": "or product of sub-array, something like that. In string, it is not possible to
find the sum, right? The first thing is that the question will be of array only. It will not
be of string or linked list, okay? After that, you have to find some maximum or minimum, okay?
What is plus? You may have to find the sum, you may have to find the product. With sum product,
 yes, okay. You may have to find the sum and product frequency sometimes."
    },
    {
      "start": 59.89,
      "end": 90.03,
      "text": "But basically these will be the two. What will be? Sum of product. More, more,
more. Plus, some condition will be given to you. What will be given to you? Condition can be
given. This is optional. This is optional. Means like, either you have to get maximum sum. And
in that condition is added that you can delete one or you can delete two. Either a condition
is added that your array is of some circular type. Either a condition is added that you have
to get this but you have to get absolute. So any condition like absolute."
    },
    {
      "start": 90.03,
      "end": 120.24,
      "text": "delete it, i.e. give you an option, give you a power in your hand, or limit you,
 give you an option and limit you, that the array is like this, or you have to work like this,
or you have to work like this without doing this, a small condition, right? And where are all
these things? In subarray, in which? In subarray, not in subsequence, not in subsequence, in
subarray, in subarray, okay? Let's go. Whose flowchart is this? Cadence's flowchart, this. In
this, cadence will definitely apply, okay?"
```

```
    },
    {
      "start": 120.24,
      "end": 148.18,
      "text": "Now, let's talk about the steps. You don't have to collect the cadents. How do
the cadents work? Whenever you reach an array of an index i, you should have the best result.
What should be the best result?"
    },
    {
      "start": 148.34,
      "end": 178.22,
      "text": "What does best result mean? It depends on the question. It can be minimum or
maximum. It can be both minimum and maximum. If there is a condition, it can be applied. And
what else? Sum or product. Whatever your question is asking you. If your question is asking
sum, then obviously you will have sum. If your question is asking product, then obviously you
will have product. Okay? You should have this. Now, on a big scale, you have two options. On
which? On i-index. Either, either,"
    },
    {
      "start": 178.22,
      "end": 203.68,
      "text": "Now the old best, i.e. the best up to i-1, connect with it. So this is also
going to end at i. Remember that you always have to make the best result that ends at i.
That's why the best result of i-1, which we call the old best result, was also going to end at
i-1. That means at the end of it, i-1 will come. No matter what comes ahead."
    },
    {
      "start": 203.68,
      "end": 230.64,
      "text": "and you have one option that you make only array of i means make only one array
in which array of i is there so technically this is also ending on array of i that was also
happening so you have these two options you update means again make the one with i and update
your result update your result with whatever is your best ending depending on question this is
the thick one in this we have seen so many questions"
    },
    {
      "start": 230.64,
      "end": 261.01,
      "text": "The first question is standard. What did you see? Max, sum, sub-array. Okay?
And, the first thing we do in this is to define our best ending for i is equal to 0. Define
your best ending for i is equal to 0. What do we call best ending in Hindi? If we talk about i
is equal to 0, then on i is equal to 0, the ending, this is a very important term, the ending,
the ending, all sub-arrays, all sub-arrays,"
    },
    {
      "start": 261.01,
      "end": 291.47,
      "text": "the best answer of all sub-arrays. Best answer. This is how you have to do it.
You should always remember the meaning of best result, that on i, or on i is equal to 0, or
whichever i you are going, initially i is equal to 0, that's why we have written, whichever
you are on the index, write all the sub-arrays ending on that index, their best result should
be stored in your best ending. This is what happens. Now see, if you had such an array, that 1,
 minus 2, 0 and 3, 0, 1, 2, 3,"
    },
    {
      "start": 291.66,
      "end": 316.54,
      "text": "Obviously, how many sub-arrays are possible on i is equal to 0? Only one sub-
array is possible, which is an array of 0 itself. Right? Empty sub-array is not allowed. Only
one is possible. That's why, you will see that in majority of cases, we do our best ending.
And we don't remember. Because only one sub-array is possible, then what will be its sum? It
will be an array of 0. Let's write array of 0. Why? Because,"
    },
    {
      "start": 316.54,
      "end": 344.98,
      "text": "How many sub-arrays are possible to end at 0? Only one sub-array is possible.
What is the sum of this sub-array? Array of 0 only. Because it is only one. We used to write
that in best ending. And for now, if your array had only one element, then your result would
be the same. Because empty is not allowed. Empty is not allowed. Empty is not allowed. Your
array has only one element. So, what will be the maximum sum of sub-arrays? Only one sub-array
is possible. Its sum will be maximum. So, we used to write the result like this. Okay? Let's
go."
    },
    {
      "start": 345.17,
      "end": 375.82,
```

```
      "text": "First of all, the best ending is defined. For i is equal to 0, we have written
the result accordingly. Now what we used to do is, from i is equal to 1, how far did we run
the loop? From i is equal to n. Okay? First of all, what we used to do is, the old best, the
old best, the old best, store it somewhere. To understand, even if you don't do it, it's okay,
but you should do it. Store it as best ending. So what does this old say to you? What is this
old? It was the best ending at i minus 1."
    },
    {
      "start": 375.86,
      "end": 405.31,
      "text": "It was the best ending on i-1. Now, let's find the best ending on i. So, best
ending is equal to... Yes, one more thing. Now, when the question is of maximum sum, then what
should be the meaning of best? Your best sum should contain the maximum. So, what will we do?
Maximum. What two choices were told? Whatever is your old best, whatever is your old best, i.e.
 the best of i-1, either join it and make another subarray. Or it can be this. Because,"
    },
    {
      "start": 405.31,
      "end": 433.12,
      "text": "You have to become the best. So you know who is the best in the old age, you
will join them. For example, you are a politician who fights indiscriminately. You have a
choice of many parties. There is BJP, Congress, Aam Aadmi Party etc. There are many parties.
If you want to join any party, then which party will you join? The party which is the
strongest till now. Suppose you come to know that BJP is the strongest till now. So you will
join BJP. What will you do? BJP and plus us. You will do this."
    },
    {
      "start": 433.12,
      "end": 459.3,
      "text": "If you think that no party is good, no party is looking strong, we are the
strongest, you may know that BJP gets 50000 votes, it gets 2 votes, it gets 10 votes, and we
get 1 lakh votes alone, then why will we meet BJP? So you become yourself. These are the two
options. This is the whole logic. If you want to meet in any party, then meet with the
strongest party, the old ones. Either you want to fight separately, then fight separately."
    },
    {
      "start": 459.3,
      "end": 488.14,
      "text": "You got this from the party. What did you do? What did you do in the old best?
You got this from the party. Very good. Either you fight alone. And its max will be your best
ending. What does this mean? At i. Ending at i. What was the old best? i-1. The new one is at
i. Very good. Now, result is equal to, because we are doing max, so max, whatever is the
result till now, and best ending. Best ending. And we closed the loop. When you will do this
for every i,"
    },
    {
      "start": 488.14,
      "end": 518.72,
      "text": "When you do this for every i, you will get the result. Return. Result. That's
it. This is its template. What do you have to change in this? You have to change that how do
you define the best ending? How do you think about the best ending? And how do you update
here? How do you update this? Rest, it will be the same. Same template, same blueprint. Let's
go. What was the minimum sum error? We just reversed it. What did we do? We just reversed it.
We just reversed it. Everything was the same. The best ending here was max."
    },
    {
      "start": 518.72,
      "end": 547.79,
      "text": "We have cut this and mined it. We have cut the result and mined it. It was a
question of minimum sum. Right? Come. What did we do in maximum product? Here you will get to
know what is the real meaning of reading this pattern. Because if you will read it from the
cadence, then someone will tell you to do one thing, split it from zero and handle the
negative separately. There is no need to do anything like this. It's all error. What did we
tell you? Whenever someone stays on i, then the one at the back,"
    },
    {
      "start": 547.79,
      "end": 575.79,
      "text": "looks for the best for himself like you were in old parties like BJP, Congress
and you you were looking for the best for yourself it is possible that for you the best is
something else and for me the best is something else what does this mean? it is possible that
you think that BJP is good but we think that Aam Aadmi Party is good it can be different for
different people it can be different for different people this was the concept of this
question why? because if array of i"
    },
    {
      "start": 575.79,
```

```
      "end": 602.69,
      "text": "If the area of i is positive, for example, this is 4. What number does it need here? It will need a bigger number here. Bigger means max. It will need a bigger number. Only then it will be able to make something bigger in its life. But if the area of i was negative, then what did we see? That minus 4 needs a smaller number. It is best to get negative."
    },
    {
      "start": 602.69,
      "end": 632.69,
      "text": "We need the smallest number only then we will be able to do something best in our life. For example, if 4 needs 10, then it will become 40. But if minus 4 gets minus 10, then it will be able to become 40. If you give minus 4 10, then it will become minus 40. It will become even worse. Right? This is how we came to know that the array of i needs the best. It needs the best at the back. But how does it need the best at the back? It depends. It depends on whether it is positive or negative. What did we do in this? We defined our best."
    },
    {
      "start": 632.69,
      "end": 663.54,
      "text": "So, we had defined two bests. Best min and best max. And both of them were written as array of 0. What did they mean in Hindi? They were done at i is equal to 0. They were done in the beginning. What does this tell you? That at i is equal to 0, the ending, we are writing this in Hindi again and again because it is important to understand. In the ending sub-array, in sub-array, from sub,"
    },
    {
      "start": 663.7,
      "end": 693.95,
      "text": "What is the max product? This tells you. This is the best max and this is the best min. So max tells you this. What does min tell you? Min tells you the minimum product in the morning that ends at i is equal to 0."
    },
    {
      "start": 693.95,
      "end": 724.11,
      "text": "In the beginning both, because i is equal to 0, there is only one square minimum and maximum, whatever you do, it will come. Good. After this, what did we do? Simple, we ran the loop. i is equal to 1, i less than n, we did i plus plus, very good. Now, now, first of all, do two things, old best max and one old best min. Take it in this, first of all, our best max,"
    },
    {
      "start": 724.21,
      "end": 751.79,
      "text": "and in this we have taken our best mint. Very good, we have taken it. Now what we have to do? Now see, who used to connect Array of I first? Array of I had two options. Either it used to connect with one of the BJP Congress Aam Aadmi Party, which he liked best, or he used to make it himself. Now here there are two bests. One, for you it may be BJP best, for me it may be Congress best. So there are two bests. Now you have to make three values. You have to make int v1 is equal to"
    },
    {
      "start": 752.05,
      "end": 776.42,
      "text": "First of all, your old best max. What is it? Old best max. Connect with this. Connect with this. Okay. What is the second option? The old best min. The old best min. Connect with this. It is possible that your best is different. You think that we will connect with the weakest party instead of the strongest party. Because we will become a star in it."
    },
    {
      "start": 776.42,
      "end": 805.78,
      "text": "This can also happen, right? Some may think that why should we join BJP? BJP is already such a big party. No one will give us any importance. We will join the parties with them. If the parties are the weakest, then they will join Congress. Join them. And the third option is that you fight alone. Don't join anyone. Now see. Now this is done. V1 is done. V2 is done. V3 is done. Right? Now what do you have to do? Now what do you have to do? Now you have to update both. Right? Both. This too. This too. Do it."
    },
    {
      "start": 806.83,
      "end": 850.88,
      "text": "What was the best max? What will be the best max? Maximum of all the choices. So maximum of V1, V2, V3. What will be the best min? Min of all the choices. Min, V1, V2, V3. One minute, one minute. Sorry, sorry, sorry. Forgive me, forgive me, forgive me. We will ask for a lot of forgiveness from all of you. Yes. What was the best min? It was minimum of three. So do minimum. Now, now."
```

```
    },
    {
      "start": 850.88,
      "end": 881.2,
      "text": "When we are updating the result, what do we need in the result? What did you
ask for the answer? Maximum product. So, what was there in the result? In the result and best
maximum, here there were two bests. So, maximum of result, whatever your best min is, it can
be done, and best max. Anything can be done, I mean anything can be done. By law, the min
should not be written, but write it, it doesn't matter. There is no more issue than that. And
by closing the loop, return result."
    },
    {
      "start": 881.2,
      "end": 909.63,
      "text": "See, by reading the same pattern of the cadence, we understand the product,
otherwise we don't understand the product. In the product, because it is negative, it is
positive, it is zero, so you have to split it and run different cadences, we don't like that.
Okay? Come on. After that, what did we do? After that, we did two, after that, we did a very
hard question, right? That, maximum sum, subarray, with, with one deletion, yes, one deletion."
    },
    {
      "start": 909.63,
      "end": 938.43,
      "text": "Here it is said that maximum sum will be given to you and some other condition
will be given that you have to do 1 deletion or 2 deletion or 5 deletion, do anything. Now see,
 it is said with 1 deletion, it is a good thing. Now how will we do best here? Now see,
because you have power, one thing is decided that there is maximum sum, there is maximum sum,
then there is no need of min, there is no need of minimum, because see the sum goes straight,
there is another in the product. Assume that there is a positive number or a negative number."
    },
    {
      "start": 938.43,
      "end": 967.57,
      "text": "Both of them would need the biggest number to do well in life. Both of them
would need the biggest number. The product was upside down so that the negative would need a
small number to do well. No. Here it is straight. One thing we are getting to know is that we
have to take max only. Means best max is a good thing. But because you have a condition. What
condition do you have? That if you have any array. If you have any array 1, minus 2, 0, 3, 0
like this. So first you have to take sub-array. So you have to take all the elements in sub-
array. But now you have been told. Now you can make any sub-array from your mind."
    },
    {
      "start": 967.57,
      "end": 994.51,
      "text": "And if you feel like doing it, then ignore one element. Ignore the sum. Now its
sum will be 1 plus 0 plus 3 is equal to 4. So this can also be the answer. If you don't feel
like doing it, then don't ignore it. I mean, don't delete it. If you feel like doing it, then
you can delete one in any sub-array. Okay. What did we read in this? For this, if you watch
the video, it will be better because it is very deep. What did we read for this? For this, we
read that because you have two options,"
    },
    {
      "start": 994.51,
      "end": 1023.6,
      "text": "See, best means maximum. But maximum can come in two ways. Either you delete or
you don't delete. What did we do? Best, one is no delete. Means, the best result up to here
with no deletions. We had told this to become an array of 0. Because, if there is only one
element array in the beginning, then only one array is possible in which 1 will be there. Now,
by doing no delete in this, you will get 1 sum. Very good. But, we had also said that best 1
delete means,"
    },
    {
      "start": 1024.78,
      "end": 1052.7,
      "text": "Good answer, the best answer, but with one deletion. Now, we didn't give any
value in this. Why couldn't we give? Because, see, 0, i is equal to 0 and this element is 1.
In the beginning, how many sub-arrays are possible which are ending here? Only one sub-array
is possible in which this element is present. Now, there is only one element in this, 1. Now,
you are saying that delete one from this also. So, what will be left in the sub-array? It will
become empty. That's why make it 0. Make it int min, make it anything, make it 0. Let's go. In
the result,"
    },
    {
      "start": 1052.7,
      "end": 1084.8,
      "text": "Now write, write array of 0 only. What can we do? See, this means not 0, this
means nonsensical value. It is not fun to give any value here. We have explained this in
```

```
detail yesterday. It is nonsensical value. Okay. After this, what did we do? Same. For i is equal to 1, i less than n. Very good. First of all, this old one, old no delete and old one delete. Take the value of these two."
    },
    {
      "start": 1084.8,
      "end": 1111.38,
      "text": "is equal to no delete is equal to one delete. First of all, see what is no delete? No delete means the best answer till now without any deletion. This is a normal caden, this is a normal caden. First of all, update this that no delete is equal to max. Either you connect with the old no delete, connect with it, or you make it your own."
    },
    {
      "start": 1111.73,
      "end": 1140.5,
      "text": "That's it. Why won't it connect with the old one delete? Because no delete means the answer till now which didn't have any deletion. But the old one delete gives you the answer till the end but there is one delete in it. So we don't have to connect it to the answer. In which? In no delete. Because no delete means the best answer ending on i which didn't have any deletion. Now where is the deletion? So either you take the no delete at the end which didn't have any deletion and connect it to the array of i. Or you take only the array of i which didn't have any deletion. Okay."
    },
    {
      "start": 1140.66,
      "end": 1164.43,
      "text": "What did we do in one delete? Now see, one more thing. In one delete, this array of i cannot come. What does array of i mean? What does one delete tell you? That in all the subarrays ending on i, the best answer in which one has been deleted. If you take only array of i, then it means that you have only one subarray in which there is only one element. Now see, nothing has been deleted in this."
    },
    {
      "start": 1164.43,
      "end": 1191.12,
      "text": "Where is the delete in this? This is one sub-array, which starts from array of i and ends at array of i. Where is the delete in this sub-array? To delete, you have to delete it. So, it won't come in one delete. That's why it came in no delete and not in one delete. We had explained this yesterday, let's tell it anyhow. Now, where do you have to search for one delete? You have to search for the one delete of i, that whatever happens in i. Okay, that's good. The first thing is that the last one delete that is coming,"
    },
    {
      "start": 1191.12,
      "end": 1220.42,
      "text": "you can take that because it has already been deleted and if you add array of i in that then also it will be one delete in the previous one delete some element has been deleted if you add array of i in that then also it will be one delete so first of all the old one delete add array of i in that but you may also feel that the one delete i that we are taking out in that if we delete array of i only this also I may feel that I want to delete array of i for this what you have to do"
    },
    {
      "start": 1220.42,
      "end": 1249.18,
      "text": "Your i is here. Here is your array of i. You have two types of previous answers. One is no delete. And one is one delete. This is the best one till the end. If you want to delete array of i. Then can you use this one? You can't use this one. Because here already one number is deleted. Now you can't use this to delete array of i. You can use this."
    },
    {
      "start": 1249.18,
      "end": 1272.86,
      "text": "We can use this person. What was the meaning of the old no delete? That the answer till the last has not been deleted. Now, you want one delete on i, where there is only one delete, which is an array of i. So, you have to take this value and write it here as one delete. I am telling you again. The last one, no delete, means that the answer till the last has no deletion. Very good."
    },
    {
      "start": 1272.86,
      "end": 1305.22,
      "text": "If you want to make one delete by deleting array of i, then write no delete in one delete i. What will happen? There is no delete till the end. If you don't count array of i, then your one delete will be made. So write it. Old no delete. That's it. And this is how it happened. After that, what did we do? For both max, this became res, this became one delete, this became no delete. And we used to return it."
```

```
    },
    {
      "start": 1305.22,
      "end": 1336.19,
      "text": "This question is a very hard question. It is a very intuitive question. Watch
yesterday's video. Please join my hands and pray. If you are watching till here and don't
understand, watch yesterday's video. Yesterday, we explained it in great detail. Today, we
explained 5 questions in 30 minutes. Yesterday, we explained only one question in 30 minutes.
Watch that video carefully. Let's go. Now, what have you been told? What was the next
question? Maximum absolute sum of any sub-array. Any sub-array."
    },
    {
      "start": 1336.19,
      "end": 1365.68,
      "text": "Okay. Maximum absolute sum in any sub-array. What did I tell you about this?
Look at the cadence. First of all, you will have to understand what I said in the question.
And the actual question of cadence, which was the maximum sum sub-array without absolute, what
did it say? It used to tell you to make the whole sub-array first. Cadence works the same way.
Make the whole sub-array. Sub-array 1 is done. Sub-array 2 is done. Sub-array 3 is done. And
sub-array 4 is done. I mean, make a lot like this. Okay. Take out the sum of all. Its S1 is
done."
    },
    {
      "start": 1365.68,
      "end": 1395.62,
      "text": "its S2 is done, its S3 is done, its S4 is done, it will continue like this.
Okay. And in all, whatever is the maximum sum, maximum of S1, S2, S3, S4, return it to us.
This was the answer, cadence, normal, which was the maximum sum in the morning. In this
question, absolute has come. What does it say? What does it say to you? It is asking you to
apply an extra layer. What layer is it asking you to apply? It is asking you to make it the
whole morning. Do the same work. Do the same work. Make it."
    },
    {
      "start": 1395.62,
      "end": 1425.36,
      "text": "Okay, I have made it. Then? Then it says, you do the same thing. Take out all
the sums. Take out S1. Take out S2. Take out S3. Take out S4. Now, obviously, when you take
out the sum, some sums will be positive and some will be negative. Obviously, right? What does
it say? Absolute all the sums. Absolute means, if it is positive, then very good. Let it be
positive. If it is negative, then make it positive. Absolute means this. So, what does it say
to you?"
    },
    {
      "start": 1425.36,
      "end": 1454.72,
      "text": "For example, make this as S1, make this as S2, make this as S3, make this as S4.
 Now, what is the sum that you have got? It is positive. This is an extra layer. Now, do the
same thing. Now, do the same thing. Give maximum. Maximum of S2, S3, S4. Return it. Now, you
have to do the same thing. So, this is an extra layer. Now, see, if there is an extra layer,
then you don't have to worry. You just have to see what is happening. What does absolute mean?"
    },
    {
      "start": 1454.72,
      "end": 1482.99,
      "text": "First of all, consider that you can divide the sum into two groups because it
is a question of positive and negative. Positive and negative remains positive and negative
also becomes positive. Now you pay attention. Let's take some negative numbers. Minus 4, minus
10 and minus 14. Make all of them positive. 4, 10 and 14. See, when it was negative,"
    },
    {
      "start": 1482.99,
      "end": 1512.75,
      "text": "What was the biggest number in this? Minus 4. And what was the smallest number?
Minus 14. But when you made it absolute, then who became the smallest number? 4. And who
became the biggest number? 14. So see, the smallest negative number became the biggest
positive number. This is not the case. When you do minus 10, it becomes 10. When you do minus
4, it becomes 4. So minus 10 is smaller than minus 4, but when it becomes absolute, it becomes
bigger. It is behaving the opposite."
    },
    {
      "start": 1513.04,
      "end": 1539.31,
      "text": "Now, do one thing to think about it. You will get the sum of all sub-arrays.
There will be a lot of sum. Divide it into two groups. In which group? One positive and one
negative. Assume S1, S2, S3, S4. These are your positive sub-array sums. If we take them out,
then we will get this only. We will take it out from brute force and divide it. And S5, S6, S7,
 S8. This is my negative one. Now, see."
```

```
    },
    {
      "start": 1539.7,
      "end": 1567.36,
      "text": "When you will absolute all of them, then all of them will remain same because positive remains same. Suppose you have only this group, this is your positive group and this is your negative group. Suppose you have only this group, then do you know the answer before you do absolute or maximum? Yes, you know. There are many positive numbers. What will be the maximum? The maximum number is known that maximum sum will be its maximum. Suppose here,"
    },
    {
      "start": 1567.36,
      "end": 1596.16,
      "text": "This S4 is the maximum. Then you would have known that S4 is my answer. Good thing. Here also, what will you know? Negative. Yes, I just told you. When the smallest negative number is absolute, then it will become the biggest number. Here, suppose the smallest number was S7. You will say that S7 is my answer. Both groups are looking different. Now, combine both groups. What have you done? Your positive group has told you that S4 is your answer. Your negative group has told you that S4 is... What is it?"
    },
    {
      "start": 1596.16,
      "end": 1625.86,
      "text": "S7, my answer is S7. Now you don't know who is the big one and who is the small one. Now do the max of both. What do you do? Do the max and give the answer. Let me tell you again. There are two groups. One is of positive sub-array sum and the other is of negative sub-array sum. What will be the answer of the positive one? Which will be its max. What will be the answer of the negative one when we are doing it with absolute? The smallest element of it will be the answer. Now, even though it has become absolute and it was already positive, so it is absolute. Now we will take out the max of both."
    },
    {
      "start": 1625.86,
      "end": 1656.61,
      "text": "So what is happening in the question? What group is this? What are we doing? First of all, we are finding a normal sum without absolute. We are looking for the maximum sum.  We are calling the maximum sum S1 in that group without absolute. Then we are looking for a minimum sum. We are looking for a minimum sum. We are calling this S2. After that, we are making this S1 and this S2. How? By doing absolute."
    },
    {
      "start": 1656.61,
      "end": 1686.91,
      "text": "by absolute. Now, whatever is the max in these two, return that. This is what we are doing. This is what we are doing. I mean, the group of positive and negative was formed because of this. One is the maximum sum. The maximum sum can also be negative. There is no problem. It can be in the maximum sum. If there is any positive, if we take out the minimum sum, then it is possible that some negative will also come. Because there are negative numbers in the array, so it will come. We are absoluting this and taking it out. So, what does the question say? The question says nothing. Take out an a1."
    },
    {
      "start": 1686.91,
      "end": 1715.57,
      "text": "What will it do? Cadence max. What will it give you? It will give you maximum sum. Take out A2. Cadence min. What is cadence? Min. It will give you negative sum. Absolute this. Absolute this too. Suppose something will come. You got two answers. Now, the answer will be from these two. Max this. Simple logic."
    },
    {
      "start": 1715.57,
      "end": 1745.22,
      "text": "What is an absolute sum? When you look at the array, either a very good positive sum will come and that will be your answer or a very negative sum will come and when we will absolute it, it will be very big. It is possible that somewhere in the array, the sum will be minus 100. If you do minus 100, it will be 100 and it will be big. So either we have to find the best sum or we have to find the worst sum, we have to absolute it and check it. Either we get a very good sum, very good. Even if we get a very bad sum, we still have an option. Why do we have an option?"
    },
    {
      "start": 1745.22,
      "end": 1779.68,
      "text": "because it is absolute. You have to do the same. You know both the programs. You know the program of maxsum and the program of minsum. Run both of them and see which is the maximum. Very good. Let's go. One more question is left. What is left? Circular array. What? Circular array. Circular array. In circular array, it is maximum. Maximum. What does
```

```
this tell you? It is an array. You have to get the maximum. You have to get it as you were getting it. But there is a quality in this array. We said that it will give small conditions in the data. This is 5."
    },
    {
      "start": 1779.68,
      "end": 1805.26,
      "text": "minus 1, minus 2, minus 6, and here is 3. 1, 2, 3, 4. You have to find the maximum sum of this. If you will use normal ketone, if you will find the normal maximum sum, then you will get to know that this 5, this is its maximum sum, then you will give 5. Because after 5, to include this 3, you will have to do this negative part, which will reduce your sum. It will not look good. You will see that 5 will come. But I have told you that this array is circular, means,"
    },
    {
      "start": 1805.78,
      "end": 1835.7,
      "text": "After this 4, the array does not end. After this 4, the array gets back to 0. This is the array. Now you tell me what will happen. Now you see. Now you can do this. Start from here. And take this too. Take this in your hand. So now your sum of 3 and 5 will be 8. Understand it like this. A circular array. This is the array. It won't be like this, but it will look like this. First of all, 5."
    },
    {
      "start": 1835.73,
      "end": 1864.94,
      "text": "then there is minus 2, then there is minus 3, then there is minus 6, and there is one more 3. Now see, I have told you that this is a doughnut. What is it? It is a doughnut. Break a piece from this which is the most sweet, means which is the maximum. The obvious thing is that you will break this piece. You will break this piece. See, this is its maximum. This is what we did here, 5 over 3. If you understand circular, it will look better. Now see, till now we have read caden, we know caden, we also know maximum sum. We don't know circular."
    },
    {
      "start": 1865.49,
      "end": 1892.24,
      "text": "It doesn't come circular. Okay, if it doesn't come, then it's okay. Now tell me one thing. Now you understand how this donut looks, right? Okay. Now see, there can be two scenarios in this. Where is the whole problem coming? It is telling you that it is circular, which means your answer turns. What does it mean to turn? That your answer, see, half of your answer came here and half of your answer came here. It means both turned back and forth. Now tell me one thing."
    },
    {
      "start": 1893.65,
      "end": 1921.39,
      "text": "One thing that you can see in the doughnut is that the red part is the maximum sum. What is this green part? Why are we not including it in the answer? We want the maximum sum. Then why is your maximum sum only red? What is there in the rest of the doughnut? The rest of the doughnut has your minimum sum."
    },
    {
      "start": 1921.39,
      "end": 1951.3,
      "text": "It's a cake and you are told to take the sweetest piece. So you break one piece. You break this piece. Why did you leave the rest? You could have broken that too. If everything was good, you could have broken everything. Why did you leave it? Because it's a rotten piece. This is the minimum. This piece of doughnut is absolutely useless. That's why we are not able to include it in the maximum sum. If none of this is useless, what does it mean? That both the pieces, this and this,"
    },
    {
      "start": 1951.3,
      "end": 1973.09,
      "text": "Both are useless. That's why we don't include it. Because if you include it, your result will be wrong. See, if you include minus 2 in your result, your answer will be 6. We don't want it. We want a good one. So why will we include it? Because it is useless. From here, uselessness starts. It is possible that here positive 3 will be there. Here negative 3 will not be there. See, it is still useless. To include this 3, either you... Sorry."
    },
    {
      "start": 1973.09,
      "end": 2002.34,
      "text": "Suppose here is 1. To include this 1, either you have to include this, then it will become minus 2. And then if you include this, then minus 1 will remain. Either you have to include minus 6, then you have to take 1, then you will remain in minus 5. That's why this part which is left, what is it? It is useless. Very useless. It is a part of min-sum. It is a
```

```
part of min-sum. See. Now, there can be two scenarios. If you understand this donut thing,
then it will be good."
    },
    {
      "start": 2002.34,
      "end": 2029.68,
      "text": "There can be two scenarios. You have to find it in this array. Consider this
array. It won't give you an array by making a doughnut. It will give you an array. Now see.
There can be two scenarios. Either your maximum sum comes somewhere in the middle. What we
have studied. What we have studied is that it comes in the middle. It doesn't rotate. It
doesn't rotate round. Okay. So either it comes in the middle. Which is a good thing. We know
how to get this. This is the question of maximum. Or you get one like this."
    },
    {
      "start": 2029.68,
      "end": 2058.05,
      "text": "or you may have one like this in which your result is round and round means
half of the result is this and half of the result is this in this case, this is case 1, this
can happen this is case 2, this can also happen in this case, you should understand that the
middle part is stuck, this one this is nothing else, this is my minimum sum my minimum sum of
the morning see, if your red part is not round"
    },
    {
      "start": 2058.05,
      "end": 2087.9,
      "text": "then it is a good thing, you can remove it from the caden. If your red part
will rotate, it means your green part will never rotate. It will not happen that both will
rotate, then your green part is straight. Why are we telling this? Because the caden that we
have studied till now, we know caden in straight array only. We know caden in straight array.
So we will have to use our caden in straight array only. But we don't know circular array, we
know straight, so use this only. Understand it like this, suppose this is India, suppose this
is India, BJP"
    },
    {
      "start": 2087.9,
      "end": 2117.36,
      "text": "The entire north, Kashmir, and the entire south of BJP won. What will Congress
win? Congress will win in between. Congress will win in between. It is not possible that
Congress will turn north and south. No, no, no. If BJP is on both sides of the country, if BJP
is here, if BJP is in the north and south, Congress will definitely be in the middle. Congress
will definitely be in the middle. And if BJP is in the middle of the country, then it is
possible that Congress is there. We don't care. We care about BJP."
    },
    {
      "start": 2117.36,
      "end": 2147.25,
      "text": "Did you understand this? If the red part is in the middle, then it is a very
good thing. If the red part is in the middle, then the green part will definitely be in the
middle. Okay. You have two cases. What was the first case? That your maximum was straight. So,
you remove this. A1 is equal to, this is a caden. This is known to everyone. Maximum, sum, sub,
 array. Straight array. Now, don't even think that it is array circular. Because you are
assuming that the answer is correct. The answer is not moving. So, what does array circular
mean to us? You removed caden. Okay."
    },
    {
      "start": 2147.44,
      "end": 2177.95,
      "text": "The next answer, in this you think that mine is wrong. No problem, you have the
green part in the middle. We have proved that it is in the middle. You take out the green part.
 Now what do you do? Take out the min answer. How? By using caden. Use simple caden. Because
the green part is in the middle. Whether it is circular or rectangle, it doesn't matter. Now,
you got the green part. But what do you want? You want the red part. You don't want green. So,
the whole sum,"
    },
    {
      "start": 2177.95,
      "end": 2208.14,
      "text": "Sum of array. Say if you minus this mean answer, which is the green part, then
you will get this red part. You will get the red part. You will get it, right? BJP got 10
votes here. Here 20 votes came. There are 50 votes in the whole of India. So how much will
Congress get? 50 minus 10 minus 20. So 30. It must have come this much. Or reverse it. There
are 50 votes in the whole of India. When will you know that Congress got 30? How much will BJP
get? 50 minus 30."
    },
    {
      "start": 2208.14,
      "end": 2236.37,
```

```
      "text": "20. So, it's a simple math. You have taken out two parts. You have taken out
both the red parts. This is also your red part. Which you have taken out from the cadence. A1.
And this is also your red part. Which you have taken out from sum minus minimum. Take the
maximum of both. Because there are two options. It can come up or down. Take the answer of
both. Take the max. Return it. It's simple. Both the cadence have been explained to you.
Right? It's simple. So, whenever you see."
    },
    {
      "start": 2236.37,
      "end": 2266.26,
      "text": "Don't get scared by looking at the question. Don't get scared by looking at the
question of caden. What to do? Don't get scared. Because the question of caden is very easy.
It means that the whole question contains caden with a twist. With a twist. We have seen a lot
of twist questions. We have seen a product twist in which you had to take two max. You had to
take max and you had to take min in the best. We have seen a one delete twist in which you had
to take no delete and one delete. We have seen an absolute twist."
    },
    {
      "start": 2266.26,
      "end": 2291.18,
      "text": "We have seen the twist of its circular. This is the only twist, not more than
this. So, this is the total of six questions. You have solved it on Kedain. According to my
understanding, this question is more than enough for you to understand Kedain and to make a
new question from you in the interview. So, let's go. Today's review, sorry, revision video is
over. See you from tomorrow and keep reading and keep learning."
    }
  ]
}
```


18. transactions/t_26.json


```
{
  "video_id": "CDo0V5Jbb2o",
  "title": "Introduction to Binary Search Tree! Don't Miss",
  "language": "en",
  "model": "large-v3",
  "duration": 1412,
  "segments": [
    {
      "start": 0.0,
      "end": 27.76,
      "text": "Today's lecture is very important. We are sweating a lot. We are sitting at
home, so the background is a bit difficult. We will start studying BST, which is Binary Search
Tree. Binary Search Tree, you will think that you have studied Binary Tree, so it is special.
It is very important to study it because there will be a lot of questions from it in the
future. This does not mean that Binary Tree is over. Don't think that. We know that there are
a lot of questions. Let's study Binary Search Tree and then we will tackle it. Because it is
necessary to study it. At least it is necessary to come."
    },
    {
      "start": 27.86,
      "end": 51.01,
      "text": "Secondly, the LCA video that we were uploading yesterday, we hope you will like
it. We made a reel on Instagram, it got 1 lakh views. Some people were saying that it uses
controversy. There was no controversy in it. The small thing was that we did not find the
explanation of their video intuitive. Now there is no controversy in it. We will say that it
was not good, then what will we say that it was very good."
    },
    {
      "start": 51.01,
      "end": 79.63,
      "text": "they are good and mine is bad, then we will close the channel. That was not a
controversy, it was a review. I didn't like it. If you want to go and see, then go and see.
There is a solution that you don't understand. Now some people were giving us knowledge that
you don't know. Okay, we don't know. What we studied is simpler. What is that? It is simpler.
Okay, we don't have that much brain. We don't even know that much. What we teach, what we
understand and what we teach, those are simpler approaches. We don't know so much hi-fi.
Neither we will be able to know. Now I am old too."
    },
    {
      "start": 79.63,
      "end": 100.22,
      "text": "So watch this video till the end because there is BST in the video. Why should
you read BST? What is the relation between BST and sorted array? Why is it used to search? And
what is BST? What is the property of BST in order? These 4-5 things will be covered. So watch
the video till the end. This is in the last, so don't leave it. If you don't want to see the
```

```
whole thing, then go and watch it."
    },
    {
      "start": 100.22,
      "end": 130.54,
      "text": "If you don't see this, then you will have a problem in the future. If you go
ahead and ask a doubt that you didn't tell this, you told it, you didn't see it, you saw it
for 5 minutes, you told it for 6 minutes, right? So let's start the video quickly. So let's
start. First of all, let's see what is BST? BST means Binary Search Tree. What were we seeing
till now? Binary Tree. Now what is attached to it? Binary Search Tree. So see, what was a
simple binary tree? One was its node and binary means that it had two beta."
    },
    {
      "start": 130.54,
      "end": 160.1,
      "text": "This was the binary tree. Now binary is also here. So this will also be the
same. This will also be the same. That there will be a node. And it will have two children. At
max there will be two children. Okay. But what is written here? Search. What does this mean?
In any binary search tree. The value of any parent. Let's assume that the value of this parent
is 10. Let's assume that this parent node. Any parent node can be. It is not necessary that it
is root. Any node whose children are called parent node. So if the value of any parent node is
10."
    },
    {
      "start": 160.1,
      "end": 188.54,
      "text": "In binary search tree, the value of its left child will be less than 10. Less
than 10 means it can be 8, it can be 2, it can be 5. Whatever it is, less than or equal to.
Some people write less than or equal to in the question. But generally, it should be less than
10. And the value of the right child will always be greater than 10. It can be 12, it can be
15. So, every binary search tree follows this simple rule."
    },
    {
      "start": 188.54,
      "end": 217.26,
      "text": "The value of any parent node can be anything, be it 10 or anything. The value
of the left child of that parent will be less than 10. The value of the right child of that
parent will be more than 10. It is not necessary that it is of only 3 nodes. It can be a very
big tree. Look, it can be a very big tree. We make a tree like this. This is a binary search
tree. What is a binary search tree? So we are just explaining the binary search tree to you.
It is not very complex. We are explaining. Look, it is like this. Let's assume we write 10
here."
    },
    {
      "start": 217.42,
      "end": 246.7,
      "text": "Now, we have to fill this. Now, here, can we fill 12 here? No, we can't.
Because binary search tree says that any left child's value is less than its parent. So, let's
assume that we fill 5 here. Can we fill 5? Yes, we can. Now, can we fill 7 here? No. Because
this is also a small tree. This is also a parent. This is also a left beta. Whose value of
this left beta will be less than this? It won't be less than 10."
    },
    {
      "start": 246.7,
      "end": 276.22,
      "text": "will be less than their parent. Any left child's value will be less than their
parent. Means, it should be less than 5. So, let's write 2 here. Very good. Now, now, now. Now,
 one more important thing. Here, what should we fill here? Now, you will say, yes, can we fill
anything more than 5 here? Can we fill anything more than 5? No. If, let's say, we fill 15
here. If we fill 15 here, then will it be correct or not? What are we saying?"
    },
    {
      "start": 276.22,
      "end": 298.22,
      "text": "What did we just tell you? That any parent's right child's value is more than
the parent's. Some people will say, yes, okay, then it is more, then fill 15. No, 15 cannot be
filled. Why can't it be filled? Because this 10, this 10, see this. This 10, this 10, what is
this? This is a kind of super parent. What is this? This is a kind of"
    },
    {
      "start": 298.22,
      "end": 322.21,
      "text": "super parent. What is a super parent? You can't fill 15 here. What do you have
to fill in this node? You have to fill greater than 5 in this node. Sorry, what is the symbol
of greater than? Greater than 5 or less than 10. Less than 10. Because this node says that
whatever is on my left side will be smaller than me. On my left side,"
    },
```

```
    {
      "start": 322.21,
      "end": 351.41,
      "text": "Whatever is there, it will be smaller than me. And whatever is there on my
right side, it will be bigger than me. So here you cannot fill 15. Why? Because its parent is
5. So it has to be more than 5. But it has to be less than 10. It has to be less than 10. It
has to be less than 10. So here you can fill 7. For example, you can fill 7. Similarly, now
what to fill in this? More than 10. Suppose you have filled more than 10. Here 15. You can
fill it. You can fill it. Now,"
    },
    {
      "start": 351.41,
      "end": 376.5,
      "text": "What do you have to fill here? Some less than 15, some less than 15. But can
you fill 2 here? No, you can't. Why can't you fill it? This is a parent, a super parent. Any
value on its right should be greater than 10. So on this node, what can you fill? It should be
smaller than 15. Smaller than 15 and greater than 10. Because 10 is here. So smaller than 15
and greater than 10. Let's assume that we have filled 12."
    },
    {
      "start": 376.5,
      "end": 405.57,
      "text": "Now here you can fill anything. What does it say? That we have to be bigger
than 15. And we have to be bigger than 10. So if it is bigger than 15, then it will be bigger
than 10. So here you can fill 20, 25, 100, anything. So here we fill 20. So see, what has
become of this whole thing? This has become our binary search tree. This has become our binary
search tree. Remember this thing that only"
    },
    {
      "start": 405.57,
      "end": 434.51,
      "text": "We don't have to do left and right. It doesn't mean that we don't have to fill
17 here. We have to look at the upper one as well. What does binary search tree say? Any root
says this. Whatever is on my left side is less than me. Whatever is on my right side is more
than me. This is the rule of binary search tree. Now see, is this binary search tree? Yes, it
is. Because it is 10. Smaller than 10 is here. Bigger than 10 is here. Now 5 is here. Smaller
than 5 is here. Bigger than 5 is here. 15 is here. Smaller than 15 is here. And this is bigger
than 10. Bigger than 15 is here. So this is binary search tree."
    },
    {
      "start": 434.58,
      "end": 463.6,
      "text": "So now you have understood the difference between binary search tree and binary
tree. There was no such rule in binary tree. You can put anything anywhere. Whenever binary
search tree will be given to you, it will follow the rule like this. Now the question arises,
why do we have to read this? Why do we have to read binary search tree? We were reading binary
tree. Why do we have to read binary search tree? So binary search tree is to be read like you
used to read array before. Everyone has read array. We have read array vector. And after that
we had read sorted array."
    },
    {
      "start": 464.02,
      "end": 493.42,
      "text": "Why did we read Sorted Array? In Array, it was difficult to search. Remember,
in Array, you have to search linearly one by one. What happens in Sorted Array? You used to
search binary. Remember, you used to search binary. In Sorted Array, we used to search binary.
Its time complexity was order of n. Its time complexity was order of log n. For the exact same
reason, what do you have to read? You have read Binary Tree. Why do you have to read BST?
Exactly, BST is a sort of Sorted Tree."
    },
    {
      "start": 493.42,
      "end": 518.1,
      "text": "You might not find this sorted by looking at it. You might think that it is
here and there. But this is a sorted tree. We will tell you how it is sorted. For this reason,
like you read sorted array in array because you had to save some time in search. In binary
tree, you read BST because you have to save some time in search. Okay. I will tell you how to
save time. Let's take the example of search. Let's take the example of search. Suppose you
have given a normal tree. What have you given? Normal tree. So, we will make a normal tree
here."
    },
    {
      "start": 518.1,
      "end": 540.53,
      "text": "In a normal tree, you can do anything. You can give the value whatever you want.
 We will make a normal tree. So, let's give the value 10, 12, 2, 5, 7, 9 and 6. It is a normal
tree. There is no rule in it. Okay. Suppose someone tells you that in this search, is there 9
```

```
here or not? In this tree, is there 9 or not? Search it. How will we search?"
    },
    {
      "start": 540.53,
      "end": 566.8,
      "text": "Now to search, you have to see every node once. So you can do any kind of
traversal. What is traversal? People get scared of hearing traversal. Traversal means to see
every node one by one. Nothing else. You can either pre-order, in-order or post-order. How can
we do it? We can write a function. We can do a root pass in it. And what can we write in it?
If root is equal to null."
    },
    {
      "start": 566.8,
      "end": 591.25,
      "text": "So we don't have to do anything in null, so return it. If we get it, what does
it mean to get it? If the data of the root is equal to 9, then we will take an answer above,
like we took an answer in LCA yesterday. So what will we do? We will do answer is equal to
root. And return it because we have got it now. If this doesn't happen, then you will have to
search in both. Suppose you come here and search. This is the root, so you come here and
search."
    },
    {
      "start": 591.25,
      "end": 616.53,
      "text": "that what is this 9? This is not 9. Now what will we do? We will have to find
the rest as well. So do one thing, find its left child as well as its right child. So what
will we do? We will call the same function for both. We will call for the left of the root as
well. So it will go there. And we will call the same function for the right of the root as
well. Is it a good thing? We can do it. We will tell you how this normal function will work.
First of all, you will come to f10."
    },
    {
      "start": 616.53,
      "end": 644.22,
      "text": "If you don't get 9 on F10, then what do you do? You call both for left and
right child to search down there. So what will it call? F12 and F7. Run F12. You won't get it
in F12 either. What will it do? It will call both for left and right. So what will it do? For
F2 and F5, you won't get it here either. You won't get it here either. Both will run and get
over. So this is over. Now come to F7. If F7 calls, it will say that this is 9 and not 9. So
what will it do? It will call both of its children."
    },
    {
      "start": 644.22,
      "end": 673.73,
      "text": "It will call both of its children. Now when you will come to F9, you will get
the answer. Very good, good, yes. So what will be stored in the answer? This will be your
answer. You can return it like this. Very good. So this is a normal program. We think that
everyone knows how to write. Search in a binary tree. Now, now, now. Let's do the same thing.
Search in a binary search tree. Search in a binary search tree. Which is our question too.
Which is our question too. Lead code 700."
    },
    {
      "start": 673.73,
      "end": 698.19,
      "text": "Let's make a binary search tree. Now see how we will make a trick. Just keep
watching. Open your eyes and keep watching. Let's make a binary search tree. 10, 5, 15, and 2,
7, 12, and 20. Is this a binary search tree? Absolutely yes. Binary search tree. The whole
rule is applicable. What do we tell you? Search 12. See that search..."
    },
    {
      "start": 698.19,
      "end": 721.9,
      "text": "12. Is it 12 or not? If you apply the above approach, then you can do the same
thing. Search on the root and check left and right. Everyone knows that. But let's do a small
trick. First of all, where do we start searching from? From the root. What do you have to
search? Search 12. Okay. So the first step is that the root is null. The root is not null. The
root is not null. Go ahead. So is its value 12? Is its value 12? Its value is not 12. Okay."
    },
    {
      "start": 721.9,
      "end": 751.66,
      "text": "now its value is not 12 so what we used to do in the upper one we used to look
at left and right but but but remember what was binary search its value is 10 is it smaller
than 10-12 or bigger than 10-12 it is smaller than 10-12 it is smaller than 10-12 ok now tell
me one thing when it is smaller than 10-12 the one you are trying to find is it smaller than
10 yes it is smaller than 10 so is there any benefit in looking at this whole left subtree no
because what does binary search tree say"
```

```
    },
    {
      "start": 751.66,
      "end": 781.55,
      "text": "The whole left subtree of this root will be smaller than this. It will
definitely be smaller. You will not get a bigger value than 10 on the left side. You will not
get it. You will not get it at all. So do you need to see any left subtree? No. There is no
need to see any left subtree. It means that this call of yours, there is no need to call it.
What do you need to call? 10. 10 is smaller than 12. It means you need a bigger value than 10.
12 is a bigger value."
    },
    {
      "start": 781.55,
      "end": 805.2,
      "text": "Where will you get the big value? You will get it on the right side. Come to
the right child. Repeat the same thing. Is it equal to 1512? No, it is not equal to 1512. If
it is not equal to 1512, then is it smaller than 1512? No, it is bigger than 1512."
    },
    {
      "start": 805.2,
      "end": 834.46,
      "text": "If 15 is greater than 12, then will you ever get 12 in the right subtree? You
won't get it because the entire right subtree is greater than 15. 15 is greater than 12. Why
do you have to find it in the entire right subtree? Where do you have to find it? 15 is
greater than 12. You need a smaller value than 15. Where will you get it? In the left subtree.
Come to the left. Come to the left. Exactly. Is 12 equal to 12 here? Yes. This is 12 equal to
12. So what will be your answer?"
    },
    {
      "start": 834.46,
      "end": 864.11,
      "text": "So, see what they are doing here. Every time, every time, in the above example,
if it was not found on the root, then left would also go, right would also go. Good thing.
Here, every time, either left is going or right is going. Both are not going. Both are not
going. Remember. Both are not going. Every time, either left is getting eliminated or right is
getting eliminated. Who does it match with? What do you remember? This is what binary search
is. This is what binary search was. What did they check there?"
    },
    {
      "start": 864.11,
      "end": 887.98,
      "text": "If you could find mid, then what would you do? You would either eliminate left
or right. Here exactly the same thing is happening. This root is acting like your mid. Every
time either left is being eliminated or right is being eliminated. Either left is being
eliminated or right is being eliminated. That is why binary search tree is so powerful."
    },
    {
      "start": 887.98,
      "end": 915.62,
      "text": "What was the time complexity of the above search? In the worst case, you would
have to search for every node. You would have to search for every node. Because, let's say it
doesn't exist. Let's say you want to search for 20. 20 is not there. So, until you see every
node, how will you know whether it is 20 or not? That's why, what is its time complexity?
Order of n time complexity. But, here, what are you doing? Look, at every level, at every
level,"
    },
    {
      "start": 915.62,
      "end": 938.08,
      "text": "At every level, you are seeing one note. At every level. First of all, you see
10 at level 0. After that, you see 15 at level 1. Then you see 12 at level 2. So you are not
seeing every note in this. You are seeing number of levels. Number of levels. Number of levels.
 And in any binary tree, what is number of levels?"
    },
    {
      "start": 938.08,
      "end": 967.57,
      "text": "log n log n log n log n log n log n"
    },
    {
      "start": 967.57,
      "end": 994.83,
      "text": "Nodes. That's why it is important to read BST. This is the power of BST. This
is the power of BST. If we have to write a code for this, it will be a very simple code. First
of all, we want to take a global variable above. First of all, we want to take a global
variable. And what will we write? The same function that we wrote above. In the same one, we
will take a node in it. And whatever you want to find, what do you want to find?"
```

```
    },
    {
      "start": 994.83,
      "end": 1017.66,
      "text": "So, first of all, check that if node is equal to null. Very good. So, if there is no work, then you won't get it here. Okay. After that, check that node's data is equal to k. If it is, then what will we do? Answer is equal to node. And there is no need to do anything further. So, return. There is no need to do anything further. Return. Now, the main work comes. "
    },
    {
      "start": 1017.66,
      "end": 1046.48,
      "text": "Now, if the data of node is not k, then only two things can happen, either it will be bigger than k or it will be smaller than k. Let's see, if the data of node is greater than k, that means you are standing on 10. You wanted to find out what was your target? Your target was 5. Now, see 5, you will get 5 on the left side or on the right side? You will get it on the left side or on the right side? What is 5? It is smaller than 10. Where will you get it? On the left side. Where will you get it? On the left side. Same as binary search."
    },
    {
      "start": 1046.48,
      "end": 1076.43,
      "text": "Is there a probability of finding it in the left or in the right? Or in other words, where will we not find it? We will not find it in the right at all. So where do we have to go? Fun of, go to the left side of the node. What does this mean? 10 is your node and your target is 5. See where you will find 5. 5 is smaller than 10, where will you find it? You will find it in the left side. Go to the left of the node and become K. No, no, if it is not like that, where will you find it? In the right, in the right, in the right. In the right, in the right, yes, in the right."
    },
    {
      "start": 1076.85,
      "end": 1106.0,
      "text": "Node right and K. Yes. And return. Khatam. Khatam. Khatam. Yeh program. Dekhiye. Isme kabhi bhi ham log dono me nahi ja rahe hain. Upar wale code me kaha jaate the. Dono me. Left right dono me. Yeh linear search tha. Yeh to aisa tha ki unsorted array me search kar rahe hain. To kya karoge. Pura array search karo bhai. Aur kya kar sakte. Lekin. Yeh ek sorted array ki tarah behave karta hai. Ek iske tarah behave karta hai. Ek sorted array ki tarah. Batahin ki kaise behave karta hai. To isme ham log kya karene. Har baar. Ya to left ya to right. Ya to left ya to right. Order of."
    },
    {
      "start": 1106.0,
      "end": 1135.84,
      "text": "Log n, time complexity. What is this? Search in BST. Search in BST. What is this? Search in BST. Yes. Now, after this, those who are watching till here, it is a very good thing. Now, you should watch the whole video because we don't dance so much. You will miss a very important thing if you are not watching till here. Now, we said that, we were saying one thing again and again that this BST behaves like a sorted array. But you will say, where is sorted? I mean, it is not visible. It will be visible. Come, let's show. We will show it now."
    },
    {
      "start": 1135.84,
      "end": 1160.91,
      "text": "Let's make a small BST. It was 5. Here it was 15. Here it was 12. Here it was 20. Now we will show you the sorted array. We want to show you. Okay, this BST is made. How many types of things do you know to write? Traversal. Did you teach three types? On the first day. Did you teach pre-order? Did you teach in-order? Did you teach post-order? Did you teach? Okay. What used to happen to all three?"
    },
    {
      "start": 1160.91,
      "end": 1191.89,
      "text": "First of all, when root comes, it becomes pre-order and always left comes before right. After that, when left comes, root comes in the middle and then right comes. Next, when root comes in the last, left and right comes first. So, what was this? This was pre-order, this was in-order, this was post-order. How did we know? If root is in the middle, then pre, if root is in the middle, then in, if root is in the last, then post. Very good. Let's write all three. So, what does pre mean?"
    },
    {
      "start": 1191.89,
      "end": 1222.08,
      "text": "First root, then left child, then right child. So we are not writing the code. We have read the pre-order once. Go and see. We have read it very well. We just want to write its pre-order. If we write the pre-order, then first we will write 10. After that we will
```

```
write 5. Then if it comes here, then we will write 2. After that we will write 7 in the right child of 5. Done. Done. This is also done. After that we will write 15. After 15, we will write 12. Then we will write 20. Okay. We have written the pre-order. Let's write the in-order. What does in-order say? First left."
    },
    {
      "start": 1222.08,
      "end": 1251.18,
      "text": "After that root, then right. So when it comes to 10, then 10 will not print itself. 10 will go to 5. Okay. 5 will not print itself. 5 will first give a chance to its left child. Now 2 doesn't have a left child, so 2 will print itself. 2. After that, when it comes back to 5, then 5 will print itself. 5. After that, it will give a chance to its right child. 7. Okay. Now it went back to 10. 10 will give it to itself. It came to 15. It won't take 15 now. 15 will first give a chance to 12. 12. After that, 15 will take it itself. After that, 20 will take it. See this. This is not rocket science. This is inorderlicantry."
    },
    {
      "start": 1251.57,
      "end": 1275.73,
      "text": "Post-order. Post-order means, first he will give to the left, then to the right, and in the end, he will print himself. So, let's come to 10. He will come to 5. 5 will first give a chance to his left child, 2. After that, he will give a chance to his right child, 7. Then, he will print himself, 5. Now, 10 will not happen. 10 will give a chance to the left child, he will give a chance to the right child, 15. 15 will first give a chance to the left child, then to the right child, then to himself, and in the end, it will be 10."
    },
    {
      "start": 1276.24,
      "end": 1305.55,
      "text": "We have written this traversal. Now see, this is very important. Out of these three, what is this method? We have made an array of the tree. Which array seems to be sorted? See, the array of inorder is sorted. 2, 5, 7, 10, 12, 15, 20. The array of inorder is sorted. The array of inorder is sorted. The other two are not sorted. The other two are not sorted. The array of inorder is sorted. But why did this happen? Did we deliberately take such an example that the array of inorder is sorted? No. What does inorder mean? Left."
    },
    {
      "start": 1305.81,
      "end": 1333.76,
      "text": "Root. Right. In order means. First left child. Then right child. BST says exactly this. Left child will be smaller than root. And root will be smaller than right. This is what BST says. Root. And left child is smaller than root. Right child is bigger. This is what BST says. And in order. Exactly this is what BST does. That's why in order. Any BST."
    },
    {
      "start": 1333.76,
      "end": 1361.41,
      "text": "In any binary search tree, if you print or store an inorder, how will it be? It will become a sorted array. Because this is the work of an inorder. Left first, root later, right last. So, left will be smaller than root, so it will come first. Root will come in the middle, right will come later. Any BST's inorder gives a sorted array and this is going to be very useful in the next question. This is a very important concept. So, this is a small..."
    },
    {
      "start": 1361.41,
      "end": 1390.9,
      "text": "Pre-order and post-order are not sorted. But in-order is sorted and it will be very useful in the future. When you will construct a tree, you will get two things from these three. It will tell you to make a whole tree out of it. It will also tell you, but it was necessary to know in the future. If you write in-order of any BST, then how does it happen? It is a sorted array. It is a sorted array. So see, this is also a small question. And you have learned BST. Like you have learned time complexity of BST."
    },
    {
      "start": 1390.9,
      "end": 1411.12,
      "text": "That's it. BST is enough for the introduction. So make a question about BST. And BST itself... Yeah, that's it. We will teach as much as we can. And now we have come home, so we are teaching slowly. I don't have that much internet to upload. I mean, I can't upload very big videos. I don't have that much internet. That's why it's happening. So let's meet tomorrow."
    }
  ]
}
```


19. transactions/t_27.json

```
{
  "video_id": "CdGBURVBMn4",
  "title": "Episode 10 of DP in 15 Days | Longest Increasing subsequence part 1",
  "language": "en",
  "model": "large-v3",
  "duration": 1550,
  "segments": [
    {
      "start": 0.46,
      "end": 21.92,
      "text": "Hello students, how was the weekend? Today's lecture will be watched very carefully, very calmly. Again, the students who have come in the middle of the DP series, watch the DP series from the beginning. Now, today we are going to do LIS. LIS is very important because we have done so much DP till now. Who was it on? It was on arrays, it was on integer arrays. This is the first question that is going to happen."
    },
    {
      "start": 21.92,
      "end": 45.39,
      "text": "This is also going to be an integer, but this is going to be a different type of question. Till now, we have done Napsack and etc. It is of a different category. This is a different pattern. Like the subset sum was a different pattern. There used to be 3-4 questions from 1 question. This is also a different pattern. Even after doing 1 question, you can do 3-4 different questions. That's why it is of a different category, pattern and importance."
    },
    {
      "start": 45.58,
      "end": 69.01,
      "text": "This is why LIS is more important. LIS means Longest Increasing Subsequence. We will talk about this later. But look at this, it is such a big thing. First of all, today we will explain the recursion of LIS and its memoization. Today's lecture is for that because we don't want to rush it. We want to learn it slowly by doing it in two parts. But LIS's recursion and its memoization. Memoization means writing DP."
    },
    {
      "start": 69.01,
      "end": 96.94,
      "text": "This is not enough, this is not enough. You should get the tabulation of memoization in any interview, which we have already told you that tabulation is your end goal. But you should get the optimized tabulation of tabulation in this. I mean, save some space in the tabulation too. Optimize the space. Okay. Even after this, it should not stop. The interviewer will not even agree with you. The interviewer needs the most optimal binary search approach. Absolutely binary search. Binary search in DP."
    },
    {
      "start": 96.94,
      "end": 111.47,
      "text": "In LIS, there are 1, 2, 3, 4, 5 stages of improvement. We will start by writing recursion as usual, memorization in recursion, but we will not stop there. Today's lecture will cover these two."
    },
    {
      "start": 111.47,
      "end": 141.49,
      "text": "After understanding these two, you will have to see part 2 on which we will do the recursion in tabulation, the tabulation is optimized and lastly, we will solve this question with the approach of binary search. So you should know all this approach. In any interview, if you are asked for a different variant of LIS or LIS, then you will have to do this in most of the cases. Meaning, if we take your interview, then we will make you do this, we will not agree with your recursion. So you open your ears and see, remove the copy and let's go. So see,"
    },
    {
      "start": 141.49,
      "end": 171.39,
      "text": "LIS means L-I-S L-I-S means L-I-S L-I-S L-I-S"
    },
    {
      "start": 171.39,
      "end": 196.53,
      "text": "Subsequence means that you can take any number in an array. You can take any sequence of numbers in an array. For example, if you want, you can take 9 from an array, then you can take 3, then you can take 101. So what does this mean? This is called a subsequence. Subsequence. What does it mean? Subsequence. What is the difference between subsequence and substring?"
    },
    {
```

```
      "start": 196.53,
      "end": 214.21,
      "text": "What does sub-string mean? When we used to do sliding window, we used to read
sub-string. Sub-string means you have to pick up a continuous part. You have to pick up a
sliding window, a window, a chunk. But what does sub-sequence mean? You can pick up a number
by leaving it. Means you pick up 9, then you leave the previous number, then you pick it up."
    },
    {
      "start": 214.21,
      "end": 240.51,
      "text": "Then you can leave the next two numbers, you can do anything. But in this order,
 the order should be maintained. Meaning to maintain the order means 10 and 9. This is a sub-
sequence. But you can't do this that you pick up 9 first. First pick up this 9 and then pick
up this 10 and write this. No, this is not a sub-sequence. The order is spoiled. 9 comes after
10. You can pick up whatever you want. Keep maintaining the order. But there is no need to
pick it up in one window. What is it called? Sub-sequence."
    },
    {
      "start": 240.51,
      "end": 268.62,
      "text": "You should know the difference between sub-sequence and sub-string. So, the sub-
sequence is that you can pick any number. If you want, you can pick any number. That is also a
sub-sequence. If you want, you can pick any number. That is also a sub-sequence. Very good.
But, what is said in that? Longest increasing sub-sequence. Yes, longest increasing sub-
sequence. What does this mean? If we pick this sub-sequence, suppose 9 and 2 and 5."
    },
    {
      "start": 268.62,
      "end": 294.0,
      "text": "Take this sub-sequence. Okay. Is this an increasing sub-sequence? How will we
test this? What is the first number? 9. This is good. What came after 9? 2 came. Means after 9
you went down. You started from here and you went down. This is not an increasing sub-sequence.
 This is a decreasing sub-sequence. So this is not increasing because"
    },
    {
      "start": 294.0,
      "end": 318.88,
      "text": "Increasing means that it keeps increasing continuously. This is not an
increasing subsequence. Look at another example. 2, 5, 3. Is this an increasing subsequence?
Let's check. 2. Good. The first one is fine. What happened after 2? 5. It means you went up
after 2. Very good. After 5, you came to 3. This is a mistake. After 2."
    },
    {
      "start": 318.88,
      "end": 348.86,
      "text": "went up and then came to 3. This is also not an increasing subsequence.
Increasing means it should always be increasing. It should be increasing as much as it is.
This is also not an increasing subsequence. Look at one more. 2, 5, 7, 1, 0, 1. Is this an
increasing subsequence? Yes, yes, yes. This is increasing. After 2, 5 came. After 5, 7 came.
After 7, 101 came. What is this subsequence called? Increasing subsequence. What?"
    },
    {
      "start": 348.86,
      "end": 374.18,
      "text": "Increasing subsequence. Very good. What have you been asked? Now, it is not
that this is an increasing subsequence. It is not like that. That this is an increasing
subsequence. We can write one more and show you. 7, 1, 0, 1. This is also an increasing
subsequence. 10, 18. This is also an increasing subsequence. 9, 1, 0, 1. This is also an
increasing subsequence. 2."
    },
    {
      "start": 374.18,
      "end": 402.67,
      "text": "3, 7, 18. So, these are all increasing subsequences. But, what do you have to
return? You have to return the longest one. What can you make the longest increasing
subsequence? In this question, what can be the length of the longest increasing subsequence?
Your answer will be, you can make it of 4 lengths. You will not be able to make an increasing
subsequence of 5 lengths in this array. You can try that and see. What do you have to return?"
    },
    {
      "start": 402.67,
      "end": 432.19,
      "text": "From the increasing subsequence, you have to return the longest increasing
subsequence. The longest one. You have to return its length. In this question, you will not be
able to return an increasing subsequence greater than 4. And one more thing. When we are
telling you increasing, then is it strictly increasing? Strictly increasing means, if your
subsequence is like this, 5, 5, 10, then you will feel that this is also increasing. No, this
```

```
is not increasing because,"
    },
    {
      "start": 432.19,
      "end": 460.7,
      "text": "You were at 5. You stayed at 5. And then you went to 10. This is not increasing.
 Why? Because this part, this is not increasing. It was constant. Same numbers cannot be.
Increasing means strictly increasing. If it was 6 after 5, then this increasing would have
been subsequent. The number should not be the same. Increasing should keep on increasing. This
is called increasing. And what is your job?"
    },
    {
      "start": 460.7,
      "end": 486.62,
      "text": "the length of the longest increasing subsequence. So, the question will be
clear. Now, let's understand this question. Now, start understanding this question. Suppose,
this is your array. In this, you have to find the length of LIS. So, by looking at this, you
will know that LIS will be 1, 2, 5. If we make this length, it means 3 lengths. We will not be
able to make more than 3. Because the length of the array is 4. It is not possible to take 4.
Can we make it of 3 lengths? 1, 2, 5. Okay."
    },
    {
      "start": 486.62,
      "end": 510.53,
      "text": "But how to solve it, how to solve it, how to write recursion, we will see. So
this question is also a choice question. You should understand this much till now because you
cannot make any set algorithm in this. What does this mean? What we are shouting from the
beginning, when you come to 4, you will see one by one whether you have to include it in the
subsequence or not. You will see one by one. When you come to 4,"
    },
    {
      "start": 510.53,
      "end": 539.39,
      "text": "You don't have any set algorithm that we will not take 4 or we will take 4. It
can't be like this. No, it can't be like this. Meaning, if you say that 4 will remain in my
answer, we will not leave 4 no matter what happens, then your answer is no. Now, suppose you
have kept 4 by winning your answer. Now, because of keeping 4, neither you will be able to
keep 1 nor you will be able to keep 2. You won't be able to keep it, right? So, after keeping
4, 1 and 2 are not possible. You can keep 5. Your answer will be 2. How many should have come?
3. Similarly,"
    },
    {
      "start": 539.39,
      "end": 557.66,
      "text": "You can't say that we won't keep any number. Suppose you don't keep 4. And you
came to 1. Now you are adamant that we won't keep 1. No matter what happens, we won't keep 1
in the world. So if you don't keep 1, then you have to take 2 and 5. So 2 and 5. How many
answers will come? How many should have come? 3. What does it mean to say?"
    },
    {
      "start": 557.66,
      "end": 582.83,
      "text": "You can't take a certain decision whether you will keep it or not. You have to
do both. And as soon as this line comes that you have to do both, you have to do all the
choice. What should come in your mind? Big big recursion. What should come in your mind big
red? Recursion. Good. It means that you have a choice on every index. What is the choice?"
    },
    {
      "start": 582.83,
      "end": 606.77,
      "text": "We have to include this in our answer, which means we want to keep this in our
sub-sequence. Which means the length of your sub-sequence will increase from 1, or we will
leave this and move forward. There can be only two types of choice. Will we keep half? No, we
can't keep half. Will we keep it or not? Okay. But, but, but, now tell me one thing. Suppose
you are at i is equal to 0."
    },
    {
      "start": 606.77,
      "end": 632.05,
      "text": "On i is equal to zero, the element is 4. Now, on the first element, you will
have a choice. On the first element, it is not like that. Now, the first element is the first
element. There is nothing before that. So, you can keep it or not. Your choice is completely
open. I mean, listen to your heart. So, you say, okay, there are two choices. One is the path
of yes, that we will keep it in our answer. If you keep it in your answer, then the length of
your subsequence will increase by one."
    },
    {
```

```
      "start": 632.05,
      "end": 652.99,
      "text": "Very good. Okay. And you have one more path that we will not keep it. If you
will not keep it in your answer, then your answer length will not increase. Okay. But, when
you will take it and come to i is equal to 1 from the yes path, now see, what is there on i is
equal to 1? 1. What number is there on i is equal to 1? 1. What have you done here?"
    },
    {
      "start": 652.99,
      "end": 682.1,
      "text": "Aap yaha pe iss aadmi ko leke aaye hain. Iss aadmi ko leke aaye hain. Yes wale
path mein aap. Yes wale path mein. Matlab aap 4 ko leke aaye hain. Matlab aapka sub-sequence
kuch kuch aisa dikhega ki 4 aagaya hai. Kuch bacha hua hai ab lena. Ab ek baat bataiye. Agar
aap 4 ko leke a rahe hain. Kisko? 4 ko leke a rahe hain. Kya aap duniya me kuch bhi karke 1 ko
le sakte hain kya? Kya aap duniya me kuch bhi karke 1 ko le sakte hain kya? Nahi le sakte hain.
 Jaise hi aap 1 ko lene ka choice lenge na."
    },
    {
      "start": 682.1,
      "end": 710.74,
      "text": "Your longest increasing subsequence will break. If you have taken 4 and come
here. As soon as you put 1 after 4. This is not an increasing subsequence. So leave the
longest. It is not in increasing subsequence. Meaning. If you have taken 4. And you have come
to 1. You will have only one way left in the world. Which way will be left in the world? You
can't take it. You can't take it at all. You have to go ahead. Similarly."
    },
    {
      "start": 710.93,
      "end": 737.14,
      "text": "i is equal to 2 pe aaye. Abhi bhi last kaunsa element liye the aap? 4. Last
kaunsa element liye the? 4. Kya 4 liye ke baad aapko lagta hai ki aap duniya mein kuch bhi
karke 2 ko le sakte hai apne answer me? Nahi le sakte hai. That will not be an increasing
subsequence. Aapke paas sirf ek rasta bachega. No ka. No karke aap kahan pe aayenge? i is
equal to 3 pe. Ab, ab, ab."
    },
    {
      "start": 737.14,
      "end": 763.79,
      "text": "On i is equal to 3, which element was taken last? 4. Now what number is here? 5.
 Now you have a choice. Now you can take a decision if you want. Which element was taken last?
4. After 4, which number came now? 5. So 5 is bigger than 4. 5 is bigger than 4. Now you have
a decision. If you want, you can take 5 in the answer. If you want, you can not take 5 in the
answer."
    },
    {
      "start": 763.79,
      "end": 790.18,
      "text": "Now, how many options will you have? Two options. Yes and no. But, if your
previous element, means your previous taken element, which one? Previous taken element.
Previous taken element. If this person is greater than or equal to your current element, you
have no choice. You have no choice."
    },
    {
      "start": 790.18,
      "end": 810.78,
      "text": "Aapko chhod ke aage jaana padega. Chhod ke aage jaana padega. Aur duniya mein
koi raasta nahi macha hai aapke pa. Lekin, jo a previous taken element, matlab abhi, issse
pehle jo element aap liye the, wo agar less hai issse, tab aapke paas do tarike ke choices
bachengi. Aur,"
    },
    {
      "start": 810.78,
      "end": 837.15,
      "text": "Why are we saying that you should remember the previous taken element? Suppose
you take the first element, 4. Your previous becomes 4. You got 5. You got a new number, 5. So
you can take it. You have a choice. You take 5. Now your previous will not remain 4. Now your
previous will not be 4. Now your previous will become 5. Why? Because suppose after this 5,
there is another 5 in the array."
    },
    {
      "start": 837.15,
      "end": 865.95,
      "text": "If you don't change your previous, let your previous be 4, then you think we
can take this too. No, we can't. Just now, which is your recent element? 5. On this 5, can you
take this 5 more? No, it won't be increasing. You can't take this 5. Suppose the next element
is 10. Now you can take this 10. So now you can take this 10 if you want. And what will become
of your previous after updating? 10. Whose next decision will be on this?"
```

```
    },
    {
      "start": 865.95,
      "end": 888.74,
      "text": "It has to be on the basis of 10. So, you don't just want the index. What do you want? Which element are you bringing previous? Which element are you bringing previous? Which element are you bringing previous? Means, when you stay at i is equal to 0, i is equal to 0, then which element is previous? No element. You have not taken anything. So, you also take previous."
    },
    {
      "start": 888.74,
      "end": 918.42,
      "text": "We can store it in the index. Which element of the index are we taking? When you stay at i is equal to 0, which element is the previous? What did we take before this? Nothing. We just started. What will we take now? Your previous can be minus 1. What can be your previous? It can be minus 1. Okay. How will you decide whether you have a choice or not? If your previous is minus 1, then you have a choice. Previous minus 1 means that you have not taken anything till now."
    },
    {
      "start": 918.42,
      "end": 947.66,
      "text": "then you have a choice. And, and, and, if previous minus 1 is not there, then if this previous is less than your current element, then also you have two types of choices. If these two things are not there, if these two things are not there, you will have only one choice left. You will have a choice to leave it. What? You will have a choice to leave it. You will have a choice to leave it. If that is not the case. So, we hope that you understood this."
    },
    {
      "start": 947.66,
      "end": 977.63,
      "text": "Now let's move towards our code. So after such a big speech, what does the poet want to say? That you need two things. You need an index on which you want to take a decision and you need a previous index. What does the previous index tell you? Which one have you brought before this? If you have brought it, then you will have to choose according to that. Meaning, if we write such a function that int fun, okay? And vector int a, okay? n, how many elements are there?"
    },
    {
      "start": 977.63,
      "end": 1005.39,
      "text": "After that, two things, i and int, prev means, i means the current index and prev means everything. First, write the base case. If the array is finished, how much more can be made? Nothing can be made, son, nothing is finished. Game over, game over, son. Okay? Okay. Now, you have to check. What will you have to check? If previous equal equal to minus 1. What does this mean? When we start, we sell previous as minus 1."
    },
    {
      "start": 1005.39,
      "end": 1022.45,
      "text": "Array of i is greater than array of previous."
    },
    {
      "start": 1022.45,
      "end": 1051.34,
      "text": "which you have taken in the previous. Suppose you had taken 4. The current element is 10. Now 4, 10 will work. So you have a choice. You can take 10 or not. Now you have two choices. The first choice is that you take this element in your answer. If you take it in your answer, then your answer will increase from 1. If you take it, then 1 will increase, right? It increased. Plus, you go ahead. Array, n, i plus 1. And what did we say? If you are taking someone, if you are taking an element,"
    },
    {
      "start": 1051.34,
      "end": 1079.5,
      "text": "Now, this index will be the previous of the next one. If you take 5, then 5 will be the previous for the next one. So, instead of previous, we will pass i because now this index will work as the previous for the next one. Did you understand? Okay. What is the next choice? You take it. You take it. We will not take it. It is not necessary to take it. We will go ahead. Array n i plus 1. Now, if you don't take it, then what will be the previous for the next one? Which was till now."
    },
    {
      "start": 1079.5,
      "end": 1109.57,
      "text": "Let's assume there were 4 previous. This was the array, right? This was the
```

```
array, right? See. 4, 1, 2. 0, 1, 2. This was the array. Let's assume we took 4. So, my previous became 0. You came to 1. You didn't take 1. You didn't take 1. What didn't you take? You can't take it. If you didn't take it, you moved forward. Now, when you move forward and come to 2, what is previous for 2? For 2 also, 4 is previous. Now, how will 1 become previous for 2? When you didn't take 1, how will it become previous? That's why, as it is, previous is passed."
    },
    {
      "start": 1109.57,
      "end": 1133.65,
      "text": "Take this. Very good. Both the choices are done. Now, which of the two choices is bigger? Whichever you think is a good choice. Max of C1 or C2. Meaning, if we take this, we will get some length. If we take this, we will know that it is of 5 lengths. If we don't take it, it is of 10 lengths. So, we will not take it. It is over. Now, if it is not like this. If it is not like this. Meaning, what is it?"
    },
    {
      "start": 1133.65,
      "end": 1159.94,
      "text": "You don't have any choice left. So what do you have to return? You can't take anything from it. Return fun array and i plus 1 and prev. Why is this written? Assume if will fail when? If will fail when? If previous is minus 1, then if will work. Previous is not minus 1. Okay. And array of i is not even bigger than array of previous. Meaning, somewhere,"
    },
    {
      "start": 1159.94,
      "end": 1187.23,
      "text": "Your array of i is smaller than or equal to your array of previous. When this happens, what do you have to do? Nothing. You have to go ahead. You can't take this element in life. We did this. And here, we closed it. Recursion is over. What do we do to do recursion in DT? What are the things changing? i is changing and prev is changing. How much is i? i maximum.
"
    },
    {
      "start": 1187.23,
      "end": 1216.37,
      "text": "It can go up to n. Previous is also an index. It can also go up to n. So what will we do to memorize? We will make an array. How big is it? n plus 1 and n plus 1 size. After that, what will we do? We will pass that array or that BPK vector here. Here, vector of. We are not telling this now because it is the main function. Because it must have entered your mind again and again. Vector of int."
    },
    {
      "start": 1216.37,
      "end": 1247.73,
      "text": "After that, leave the base case. If dp of i or prev not equal to minus 1, then return dp of i of prev. Very good. If this is not the case, then you will have to do it. But before returning, store it in this."
    },
    {
      "start": 1247.86,
      "end": 1275.58,
      "text": "prev equal to max of c1, c2. One more place is returned. You will have to store it here as well. You will have to store it here as well. dp of i of prev equal to fun array and i plus fun and prev. This is your memo is. But there is a very big, very big mistake in this. Very big mistake."
    },
    {
      "start": 1275.58,
      "end": 1305.66,
      "text": "What is that mistake? Pause the video and think for a minute. Otherwise, we will tell you. Let's tell you. You must have paused and thought. What is the mistake in this? When you start this recursion, when you start this recursion, how will you do it? What did we say? When you call this recursion from the main function, you will call it like this. Return fun array n. Where will i start? From 0. And what did we say? What will be the previous in the beginning? We will start the previous by doing minus 1. There is no previous in the beginning."
    },
    {
      "start": 1305.66,
      "end": 1333.52,
      "text": "Now, this means that your previous, which has been passed, what has been passed? Minus 1. Minus 1 means that there is no index. We are using it. But, here you are doing dpi and previous. Suppose, if the value of previous by chance is minus 1, you will pass minus 1 in this array's index. Segmentation fault will crash. What will happen? Crash will happen or unexpected behavior will come."
    },
    {
```

```
      "start": 1333.52,
      "end": 1363.04,
      "text": "What should we do to solve this? Now, the value of previous, the value of i
cannot go to minus, that is known. The value of previous can be minus 1 because you are
passing minus 1. What should we do for this? For this, we will assume, we will assume, what
will we assume? That i and prev, the i and prev in fun, assume, we will assume that fun i prev.
 Where will we get its answer? Or where will we write its answer?"
    },
    {
      "start": 1363.04,
      "end": 1389.07,
      "text": "We will write its answer in dpi and prev plus 1. Prev plus 1 means, suppose i
is equal to 0 and prev is equal to minus 1. Where will we write its answer? In dp of 0 and 0.
Sometimes i is equal to 4 and prev is equal to 2. Where will you get its answer?"
    },
    {
      "start": 1389.07,
      "end": 1418.32,
      "text": "You will get the answer in DP of 4 and 3. We will write the answer in PREV plus
1. Why? Because PREV can be minus 1. If you write simple simple I PREV to I PREV, it will
crash or it will be wrong. That's why we will assume this. See, what is diary? You have
assumed that you will write diary here and see here. It is your choice. Now assume that you
are making a diary or a calendar. Now normally what people do? Normally what happens on Monday
is"
    },
    {
      "start": 1418.32,
      "end": 1447.98,
      "text": "We write on the Monday page. What happens on Monday, we write on the Monday
page. We do this normally. You assume, your diary is your choice. You assume that whatever
will happen on Monday, we will write on the Tuesday page. It's your choice, it's your diary.
Whatever will happen on Tuesday, we will write on the Wednesday page. And whatever will happen
on Friday, we will write on the Saturday page. It's your choice, you write. Assume someone
asks you, okay tell me, what happened on Tuesday?"
    },
    {
      "start": 1447.98,
      "end": 1476.14,
      "text": "So the normal person will see what happened on Tuesday from the Tuesday page.
But because you were doing the opposite, you were doing something different. What will you do?
You will open the page of Webinar Day and tell him. If someone asks you, tell me one thing,
what happened on Saturday? What will you do? You will open the page of Sunday in your diary
and read it and tell him. It is your wish. Your rule, your life, your rule. That's why we want
to make a change in this. What change do we want to make? Here, if"
    },
    {
      "start": 1476.14,
      "end": 1504.38,
      "text": "Prev plus 1. What will you do? Prev plus 1. It's over. You have to do this
everywhere. Here you are looking for the answer. But where you are writing the answer, you
have to change that too. If you write from your rule, then only you will find it. You write
normally and you are looking for another rule. It will be different. So where there is work to
write, here and here. See this. You have written a memoization code. What have you written?"
    },
    {
      "start": 1504.38,
      "end": 1530.11,
      "text": "Memoization has been done. What has changed? We write according to what? That
is your choice. If you want, you can write the previous answer in the previous plus 10. It is
totally your choice. Because you are writing, you are searching, so you can do whatever you
want. So keep these things in mind. That is why I said that all this is not going to be
written. That always the answer is I. No, it is your choice. You can write as you like. So
this was a very important thing to understand."
    },
    {
      "start": 1530.11,
      "end": 1549.42,
      "text": "So, you have done this memoization which we talked about in the beginning of
stages. Where did we talk about? Where did it go? Yes, now you have done this. Where did it
go? Done? Done. Tomorrow, we will do these three. So, don't miss tomorrow's lecture. And
before watching tomorrow's lecture, understand this completely. Let's meet."
    }
  ]
}
```


20. transactions/t_28.json

```
{
  "video_id": "D5jgZ4PnU5o",
  "title": "DFS Cycle Detection | From Basics to Advanced | DSA Pattern Graph Series",
  "language": "en",
  "model": "large-v3",
  "duration": 2978,
  "segments": [
    {
      "start": 0.0,
      "end": 29.42,
      "text": "Whenever you solve the problem of graph in your life, you think that if we
learn undirected, then we will learn directed as well. Then you will understand that you are
making a big fool of yourself. Nothing like that happens. Graph looks good to see. Directed,
undirected, you think that what is it? There will be a small change. No. Today we will do two
questions. Cycle detection. In undirected and in directed as well. And both are very different
from each other. It is not that if you learn undirected, then you will become a master of
directed. Or if you learn directed, then you will not be undirected. Without seeing both,
there is no benefit."
    },
    {
      "start": 29.42,
      "end": 58.26,
      "text": "If you want to learn something, you can do this in this type of video that
learn the undirected art, learn the directed story or you can do the opposite, first go and
watch the directed, whatever you want to do, but never make the mistake that you watch the
undirected and make a story in your mind that yes, if the directed comes, then we will write
it, we will not be able to write it, it will be completely wrong. So come on, it may be that
the video will be a little long, let's start quickly. So come on, let's start with the
undirected first because it is very easy, undirected will always be easier for you to watch
than directed."
    },
    {
      "start": 58.26,
      "end": 86.53,
      "text": "So in the question, you are given an undirected graph. What does undirected
mean? If there is an edge from A to B, then there is an edge from B to A. Just like Facebook.
What happens in Facebook? If you are someone's friend, then he is also your friend. Now it
must be there. Let's take it normally that there is no one who follows on Facebook. If you are
his friend, then he is also your friend. He is undirected. And when you open the link of the
question in GFG, then you will find a very interesting line. Is this a multiple component
graph?"
    },
    {
      "start": 86.53,
      "end": 113.15,
      "text": "It is a multiple component graph. In all the questions we have done so far,
this line was not there. That it can be a multiple component. What does this mean? See, there
is nothing to be afraid of. It is a very easy thing that your graph is broken in multiple
parts. For example, your graph can look like this. Here, suppose there are three nodes
separately. 0, 1, 2. This is a component. But there are not only three nodes in this graph.
There is one more. 3, 4, 5."
    },
    {
      "start": 113.15,
      "end": 141.97,
      "text": "This graph has a total of 6 nodes. But you can see that you can break this
graph into two parts. These two parts are not connected to each other. Meaning, if you start
from 0, you do anything from 0, BFS or DFS, you will never reach 3. You can start BFS or DFS
from 3, you will never reach 0, 1, 2. If you start from 3, you will end up here. If you start
from 0, you will end up here. So, never."
    },
    {
      "start": 141.97,
      "end": 169.6,
      "text": "In the multiple component graph, you can do anything. In any question, you can
either do BFS or DFS. By running a loop, you have to start DFS from all the unvisited nodes.
For example, if you start DFS from 0, then you will cover 0, 1 and 2. But you have to check if
3 is unvisited. Even after starting from 0, if 3 is unvisited, it means 3 is in a different
component."
    },
    {
      "start": 169.6,
      "end": 192.16,
      "text": "There is no difference between 0 and 3. So, you have to start DFS from 3. After
that, you can cover 3, 4, and 5. So, whenever, in any question, if you are told that there are
multiple components, there is no need to do much in this. From every node, from every
unvisited node, if you want to do DFS, whatever you want to do, then you can start DFS. The
```

```
more nodes you have to visit, the more it will be done."
    },
    {
      "start": 192.16,
      "end": 220.61,
      "text": "If someone is unvisited, then you can start again from there. He will do as
much as he has to cover. There may be more nodes. Here 6 and 7 are also there. So you will see
that 6 is also unvisited. Means this graph is broken into 3 parts. Start from 6 also. 7 also.
So all these are normal things. When you will code, then automatically you will understand. So
never be afraid of multiple components. Nothing much happens in it. You can just treat it that
you have given a lot of small small graphs. You have to do all that. Let's go."
    },
    {
      "start": 220.61,
      "end": 246.26,
      "text": "Now, let's come to the main topic that you have to detect the cycle. What to do
in this? Cycle. What does cycle mean? That whenever, suppose, three circles are like this,
this is closed. So, what is this? It is a cycle. You are closed. You started from one place
and came back to the same place. So, it is called a cycle or a loop. Now, there is an open
thing. This is it. Now, there is no cycle here. See, it is open. This is not a cycle. But, if
it was like this."
    },
    {
      "start": 246.26,
      "end": 275.94,
      "text": "This is the cycle. You understand what cycle means, right? Any closed box where
you will get stuck. Once you go in that box, you will get stuck. Like in the link list. There
was a cycle in the link list. There is a cycle in the graph also. You get stuck there. So you
have to detect it. Okay. So see. Let's understand from the example. Suppose you are running
Facebook. You first come to my profile. You first come to my profile. Facebook is an
undirected graph. If I have a friend, then we are also friends. Okay. From my profile, you see
my friends."
    },
    {
      "start": 275.94,
      "end": 305.68,
      "text": "And where did you go from my friend? Suppose you reached Amit. You came to
Amit's profile. Okay. On Amit's profile. Another girl's name was Seema. So you came to Seema's
profile. Okay. Now you can see that Pratyush is written in Seema's friend. What is written in
Seema's friend? Pratyush means Seema was my friend and we were Seema's friends. Here you will
get the cycle. Why? Because you came to Seema's profile. Where did you start? From Pratyush.
You flew from Pratyush to Amit. You flew from Amit to Seema. Now,"
    },
    {
      "start": 305.68,
      "end": 334.34,
      "text": "You can see that in the profile of Seema, it is written Vapas Pratyush, which
is already visited. You remember, whenever we do DFS or BFS in the graph, it is remembered
that we have seen him. It is remembered from visited that we have seen this person. So, as
soon as you will get such a neighbor on Seema, who is already visited, what will it mean? That
Seema is my friend. Does it seem like a cycle to you? It seems exactly like a cycle."
    },
    {
      "start": 334.34,
      "end": 362.3,
      "text": "So an easy approach should come to your mind that we will start doing DFS,
whatever we get unvisited, we will start doing DFS from the main function. We will keep doing
DFS, but if you have done DFS of any node, suppose you come to any node, now what do we get
out of the node? How do we get the neighbor out? We get it out from the adjacency list that
from whom it is connected. If that neighbor is already visited, that means you have already
seen that node,"
    },
    {
      "start": 362.3,
      "end": 391.71,
      "text": "Now, there is the same node in its neighbor. You are going back there. Meaning,
what are you making by rotating? You are making a cycle. You will never be able to get out of
it. So, you can say that yes, there is a cycle in the graph. But, there is a very important
flaw in this. There is a very important bug. In this, you must have understood that you are
getting a cycle. Why? Pratyush amid Seema. Seema is back to Pratyush. Pratyush is already
visited. So, cycle. Okay. But, suppose there is a graph of two nodes. Sometimes, there is a
graph of two nodes. See this. There are only two nodes."
    },
    {
      "start": 391.71,
      "end": 421.78,
      "text": "0 and 1 and these two are connected to each other. This is the graph of two
```

```
nodes. One edge is 0 to 1. You see what we do. Here we make a visited array. This is your visited. 0, 1. In the beginning, both are false. There is nothing in the beginning. What did you see first? That 0 is not unvisited. So what did you start from 0? DFS. So you reached DFS of 0. Very good. What is the first step in DFS? Mark this node as visited. Mark this node that we have seen it."
    },
    {
      "start": 421.78,
      "end": 450.77,
      "text": "So, what will you do? You will make it a true mark. Okay? It is still correct. Okay. Now, what will you get in the neighbor of 0? What will you get in the neighbor of 0? 1. Why? Because 0 to 1 is the edge. So, 0 is the neighbor of 1. And 1 is also the neighbor of 0. So, what will you get in the neighbor of 0? 1. Okay. You got 1. Okay. Now, what do you see? If this neighbor is visited, it means that there is a cycle. But 1 is not visited yet. So, everything is fine till now. Everything is fine till now. Okay. If it is not visited, then what do we do?"
    },
    {
      "start": 450.77,
      "end": 480.02,
      "text": "DFS is on 1. DFS is on 1. DFS is on 1. Everything is fine till now. But, after coming to 1, what is the first thing to do? What do we write after updating 1's visited? True. Okay. Now it is on 1. Now it is on 1. Now, because it is an undirected graph, if 0's neighbor is 1, 1's neighbor will also be 0. Because it is undirected, if 0 and 1 are given an edge, then 0's neighbor is 1 and 1's neighbor is 0. This is when we"
    },
    {
      "start": 480.02,
      "end": 509.89,
      "text": "What do we do? Adjacency of source, pushback destination. And adjacency of destination, pushback source. 0 is also a neighbor of 1. 1 is also a neighbor of 0. Both are neighbors of each other. Now look here, there is a mistake here. 1's neighbor is 0. So what do you say? Neighbor is equal to 0. Now look here, you are stuck. Here you have got a neighbor who is true. Whose visited is true. You will think, okay, 1's neighbor is 0. This means 0 is visited."
    },
    {
      "start": 509.89,
      "end": 537.68,
      "text": "This means that there will be a cycle in this graph. But look at this graph carefully, this drawing. Do you see a cycle here? There is no cycle here, right? Where will you see a cycle? So it is open, right? To make a cycle, if you take an undirected graph, then at least three people are required to make a cycle, right? How can two people make a cycle? How can two people surround someone? It is not possible, right? Two people together, thinking that we should make a wire and surround someone. We cannot surround. To surround, three people are required to make a wire and surround. So there is no cycle here."
    },
    {
      "start": 537.68,
      "end": 567.52,
      "text": "But with your current logic, you have got a neighbor whose visited is true. So you will think that yes, it is a cycle. You will say that yes, it is true. Very good. You don't have to do this. This is a mistake. What is this? It is a fatal flaw. It is a fatal flaw.  This is a mistake, right? You will say that it is a cycle. It is not a cycle. Why did this happen? Why did this happen? Look, this happened because in undirected, it is obvious that suppose you come to 0. Come to 0 to 1. Now the neighbor of 0 is 1. The neighbor of 0 is 1. Okay."
    },
    {
      "start": 567.52,
      "end": 596.53,
      "text": "1's neighbor is 0. So, you have come to 0. You have made 0 true. And you have reached 1. Now, you have reached 1. Very good. Now, 1's neighbor is 0 again. But 0's visitor is true. In this case, you have to ignore it. Why do you have to ignore it? Because 1's parent is 0. 1's parent is 0. You have to ignore your parent in cycle detection."
    },
    {
      "start": 596.53,
      "end": 625.38,
      "text": "You have to ignore the parent. Why? Because in undirected, you can't make a cycle by combining two nodes. You can never make a cycle by combining two nodes. So, when you check whether you have found a visited neighbor, you have to check one more time that the neighbor we are looking at, is he my parent? If he is your parent, ignore him. Don't consider him. If he is your parent,"
    },
    {
      "start": 625.38,
```

```
      "end": 654.75,
      "text": "Do not consider it at all. Otherwise, you will get stuck in this error. You
will say that yes, there is a cycle in two nodes. There is no cycle in two nodes. This has
happened because there is a parent. You have reached the child from the parent. Now the child
is also coming back to the parent. But there is no cycle. There is only one stick. There is no
cycle in one stick. There is no cycle in one stick. So in this case, this is completely
correct. A neighbor whose visitor is true is very good. But in that, you ignore your parent.
Who? Parent."
    },
    {
      "start": 654.75,
      "end": 683.86,
      "text": "We will show this in a bigger example. We will show this in an example of three.
 Take three. One is Pratyush. The other one is Amit. And the other one is Seema. This is how
the graph is made. This graph has a cycle. You can see a regular cycle in this graph. Now we
will write its neighbor on every node. Who are Pratyush's neighbors? Amit and Seema. Pratyush
has two friends."
    },
    {
      "start": 683.86,
      "end": 711.78,
      "text": "How many friends does Amit have? Two friends. Pratyush and Seema. Pratyush and
Seema. Okay? How many friends does Seema have? Again, two friends. Pratyush and Amit. Now, if
it's undirected, then everyone is a friend of each other. No problem. Okay? Very good. Make a
visited array. Make a visited array. This is Pratyush's visited array. This is Amit's. And
this is Seema's. We are telling by name because it's easier to understand by name. False."
    },
    {
      "start": 711.78,
      "end": 741.44,
      "text": "False? False. Everything is false in the beginning. Okay. You come to the main
function. You see which one is unvisited. Suppose you start with Pratyusha. So what did you
do? DFS of Pratyusha reached. DFS of Pratyusha reached. Very good. The first thing to do is to
make Pratyusha's visited true. Make Pratyusha's visited true. Okay. One more thing. When you
reach Pratyusha in DFS, then track who is Pratyusha's parent. In this case, who is Pratyusha's
parent? Pratyusha has been called from the main function."
    },
    {
      "start": 741.44,
      "end": 769.23,
      "text": "If you open your mobile and reach Pratyush, then assume that Pratyush doesn't
have any parent. So in the beginning, we will make it minus 1 that Pratyush's brother doesn't
have any parent. Now for the first time, DFA is starting with Pratyush. He doesn't have any
parent. Very good. Now come. Pratyush's visited mark is done. Start seeing Pratyush's neighbor
one by one. Who is Pratyush's first neighbor? First neighbor is Amit. Okay. Is Amit visited?
No, Amit is not visited. If he is not visited, no problem. Do DFS."
    },
    {
      "start": 769.23,
      "end": 792.1,
      "text": "It's not DFS. If it's not visited, then do DFS. Do DFS. What did you do? You
did DFS. Come back. DFS of Amit has come. Okay? And who is Amit's parent? Whoever you are
doing DFS with, how did you reach Amit? Through Pratyush. So, in Amit's parent, we will pass
Pratyush so that Amit can ignore his parent with cycle detection. Why?"
    },
    {
      "start": 792.1,
      "end": 821.33,
      "text": "Because if Amit will start considering his parent, Obviously, if you have
reached Amit from Pratyush, then Pratyush is visited. Now Amit will say that my neighbor is
Pratyush, he is visited, he is a cycle. No, you don't have to do this. There are two people,
right? There is no cycle in two people. So who is Amit's parent? Amit's parent is Pratyush.
Amit's parent is Pratyush. When we call DFS, then here we will pass the node instead of a
neighbor and a parent. Okay? We came to Amit. The first thing to do is to true Amit's visited.
Very good."
    },
    {
      "start": 821.33,
      "end": 844.26,
      "text": "Let's look at Amit's neighbours. Who is Amit's first neighbour? Pratyush.
Amit's first neighbour is Pratyush. Obviously, it is Pratyush. Now, Pratyush is visited here.
Pratyush's visitor is true. You should have said that it is a cycle. But, is Pratyush his
parent? Yes, Pratyush is his parent. If a neighbour is your parent, ignore him. He has no
benefit."
    },
    {
      "start": 844.26,
```

```
      "end": 870.66,
      "text": "Ignore. Ignore. We will not do DFS on non-present. Due to non-present, we will not do our cycle detection. We will ignore it. Very good. Very good. Let's see the next number. SEMA. Who is the next number of Amit? SEMA. Is SEMA visited? No. Is SEMA parent? No. If it is not visited, then the game is over. If it is not visited, then what to do? We have to do DFS. We have to do DFS. Come. Do DFS. On SEMA. Now."
    },
    {
      "start": 870.66,
      "end": 897.02,
      "text": "Who is the parent of Seema? How did you reach Seema? Through Amit. So who is the parent of Seema? Amit. Okay? The first thing to do is to prove Seema's visited. Now, let's start looking at Seema's neighbours. Let's assume that Seema has two neighbours. Look at Amit first. Look at Amit first just for understanding. Is this Amit visited? Yes, Amit is visited. But..."
    },
    {
      "start": 897.02,
      "end": 922.42,
      "text": "Amit is not Seema's parent. Yes, Amit is Seema's parent. Ignore it. Because this cycle is not parent-child. Ignore it completely. Amit is Seema's parent. We will not do anything on him. We will not do DFS on him. DFS is not like this because visitor is true. So, we will not use it for cycle detection. Ignore it. Why? Amit is his parent. But who is Seema's next neighbor?"
    },
    {
      "start": 922.42,
      "end": 951.06,
      "text": "Pratyush. See here, the game will change. Who is the next neighbor of Seema? Pratyush. Yes, yes, yes. Now, is Pratyush's visitor true? Yes, Pratyush's visitor is true. You have seen Pratyush. You have seen him at least once. But, is Pratyush Seema's parent? No, Pratyush is not Seema's parent. What does this mean? It can mean only one thing in the world. That you start somewhere with Pratyush."
    },
    {
      "start": 951.06,
      "end": 980.45,
      "text": "Now, Pratyush is not his parent. You have reached the limit. Now, you want to go back to Pratyush. It means that there are at least three nodes. At least there are three nodes in between. And this tells you that yes, cycle detection has been done. Pratyush is not his parent. It cannot happen that you have reached the limit from Pratyush. No. You went somewhere else from Pratyush. After that, you reached the limit. Now, you want to go back to your Pratyush. So, what does it mean?"
    },
    {
      "start": 980.45,
      "end": 1007.6,
      "text": "There is a cycle in the graph. What will we do here? We will do cycle is equal to true. And we will not do DFS on the return. The DFS of the limit is over. Come back to Amit. The last neighbor of Amit is also over. Amit's DFS is over. Now come to the return. The only neighbor of the return was explored. Amit. The cycle detection of the return is already done. Let's do it. Is there anything wrong in it? What is its second neighbor?"
    },
    {
      "start": 1007.82,
      "end": 1035.25,
      "text": "What is its second number? The limit. Now there will be no difference. Let's see if the limit is already visited. Yes, the limit is already visited. Is the limit Pratyush's parent? No, Pratyush's parent is none. Pratyush's parent is minus 1. There is no parent. So what will it do here? It will make the cycle equal to true, which we have already done. So there will be no difference. It will be true. This will also end. Now we will come back. What did you find out? That there is a cycle in the graph. In the graph,"
    },
    {
      "start": 1035.25,
      "end": 1065.23,
      "text": "There is a cycle. Yes, there is a cycle in the graph. So, in this way, you had to take care of one thing. Parent. Don't consider your parent as a visited neighbor. Because obviously, when you are undirected, your parent will be already visited. That's why you are born with it. Right? So, this is called a cycle. What is this called? Cycle. Cycle needs at least three people. The purpose of parent is to remove this system of two people. Remove the two people who are connected to each other. Remove them. Look at this. Remove them."
    },
    {
      "start": 1065.23,
      "end": 1095.71,
      "text": "That's all you have to do. Do you want to change a small thing in DFS? You have
```

to pass parent. Write its code. Write its code from beginning to end. You have given edges. You have to make a list from edges. You have to pass the list in DFS. So, let's write its end-to-end code. Today, if someone says that you have not given the code, then we will beat you a lot. What do we write? Let's write its code. You have given a function. Whose name is isCycle. It is a Boolean type function. IsCycle. What have you given in this? First of all, you have given number of."
    },
    {
      "start": 1095.71,
      "end": 1125.25,
      "text": "Vertex means how many nodes are there in the graph. And you are given a 2D vector named edges. We have told all this when we were reading the syntax of the graph. What will be given in edges? Between which vertex? Source, destination. Because this is an unweighted graph. It will be said that there is an edge between source and destination. There is an edge between 0 to 1. There is an edge between 0 to 2. First of all, let's make our adjacency list. How is it made?"
    },
    {
      "start": 1125.25,
      "end": 1155.54,
      "text": "Make a 2D vector. Make a 2D vector. What size? N size. What will it say? There will be a vector in every vertex and that vector will contain its neighbors. It will say that the neighbor of 0 is 1, 2, 3, etc. The neighbor of 1 is 0, 4, 5, etc. It will say this. Okay. What will we do? We will loop on edges. How will we loop on edges? Through for loop. i is equal to 0. i less than edges dot size. Very good."
    },
    {
      "start": 1155.54,
      "end": 1186.45,
      "text": "and I++. Yes, yes, yes. Come on, what is its source and destination in edges? I and T. Source is equal to edges I. Edges I is a vector of two sizes. It has two things, a source and a destination. If you want to find the source, its zero index. And you need destination. What is the destination of edge? Edge is always between two people. It is between 0 and 1. Source is 0, 1 is destination. Edges I."
    },
    {
      "start": 1186.45,
      "end": 1217.44,
      "text": "1. Now because it is undirected, so array of source may push back destination. Meaning source's neighbor is destination. And destination's neighbor is also source. If it is between 0 and 1, 0's neighbor is 1. Pushed. 1's neighbor is also 0. So you have array of destination.pushback pushback source. Very good. You have made your undirected graph. That's all you have to do."
    },
    {
      "start": 1217.44,
      "end": 1245.92,
      "text": "Now, we have to make a visited array. Vector, bool, viz. What size? N size. In the beginning, everything is false. Now, because these are connected components, you should run a loop. Run a loop from 0 to N. Why is this loop? DFS on whatever is unvisited. Because there are different components. You can also say that we will always do DFS from 0 once. No."
    },
    {
      "start": 1245.92,
      "end": 1276.27,
      "text": "As I told you, we will do DFS from 0. It is possible that you will not be able to see the 3 component. That is why, do DFS from all the unvisited. If you do it from 0, then all the connected components in 0 will be covered. After that, assume that 5 is left. Cut it from 5 and see how much will be covered in 5. If, if, if, array of, sorry, visited of, visited of i, visited of i, equal, equal to 0. What to do? Nothing to do. DFS of,"
    },
    {
      "start": 1276.27,
      "end": 1304.19,
      "text": "What is DFS? A. A is your list. After that, number of vertices. You should do it. You don't need it. Node. What is passed in node? I. What is passed in node? I. On which node? And what else do you have to pass in this? I has parent call. So here, because we are starting DFS with this, there is no parent. And one visited array. That's it. Close this loop. In the last, return. Return."
    },
    {
      "start": 1304.19,
      "end": 1332.86,
      "text": "Cycle. What is Cycle? We have not defined Cycle yet. We will make Cycle a global variable. Cycle will be made a global variable. And DFS will update it directly. Now let's write our DFS. Come. First of all, make a global variable. Bull. Cycle. Keep it false in the beginning. In the beginning, assume that there is no cycle. Whatever you want to detect,

```
give benefit or doubt that there is no cycle. If you make it true in the beginning, then every graph will have a cycle. Now write your DFS. Void."
    },
    {
      "start": 1332.86,
      "end": 1360.4,
      "text": "DFS. What does DFS take? First of all, it takes your 2D vector which is your adjacency list. What is adjacency? List. Okay. INT N. Number of vertices. INT node. On which node have you reached DFS? It will take one more extra thing which is parent. Yes. And your visited array. Bull. Viz. It is over. What is the first step of DFS?"
    },
    {
      "start": 1360.5,
      "end": 1390.75,
      "text": "viz of node is 1. This is a visit. This is a visit. Very good. After that, group all the neighbors. How is it grouped? j is equal to 0. j less than. Array of node is a vector which will contain all the neighbors one by one. And j plus plus. Okay. How will we remove neighbors? Array of node. Array of node is a vector of neighbors. And one by one,"
    },
    {
      "start": 1390.75,
      "end": 1421.62,
      "text": "As soon as the neighbour leaves. Now, what do you have to check? If visited of neighbour equal equal to 1. Okay. And, and, neighbour not equal to parent. Neighbour not equal to parent. Means, if this neighbour is visited. And this neighbour is not even a parent. This means that there is a cycle. Yes, cycle is equal to true. Which cycle? We have made this cycle true. We have made this one true. It is over."
    },
    {
      "start": 1421.62,
      "end": 1452.13,
      "text": "Now, check one more time. If visited of neighbor is equal to zero. If it is zero, then don't do anything. Call DFS. Array of n. We have called DFS on neighbor. And who is the parent of this neighbor? From where are you reaching the neighbor? From node, you are reaching the neighbor. And visited. Close this. And return your simply. What is this? This is the code of DFS. Small change. What small change? Parent."
    },
    {
      "start": 1452.13,
      "end": 1481.33,
      "text": "parent is passing. When we call DFS from the main function, then what do we do? Minus 1. Now, if you are starting DFS from 0, then who is 0? Parent. No one is there. You have called. Minus 1. Brother, there is no parent for this. After that, whenever it happens, then neighbor. Who is the parent of neighbor? From where did neighbor reach? From this node. So, node called. So, this will tell you in which? Cycle in undirected. Cycle in undirected. Code this. Pause and code as you want. And move towards directed. Directed."
    },
    {
      "start": 1481.33,
      "end": 1509.97,
      "text": "It can be a little hard, but what will be harder? Nothing is that hard. Let's move on to directed. Let's close this. Yes, so come to directed. And we have already told you that whenever you are solving a graph question, you have solved it for undirected. It was very fun. If your heart is telling you that you will do the same for directed, then you explain to it that you have become stupid. It will never happen in undirected or directed that you will make it using the same logic. There is a basic difference."
    },
    {
      "start": 1509.97,
      "end": 1539.07,
      "text": "We will come to this graph later. We have made this graph after a lot of thinking to explain another point. First, we will tell you that what we have done now, why won't that happen? Why won't that happen? It is important to tell, isn't it? We will just tell the solution, then you will remember. But why won't what we have done now happen? Why not? It is a graph, isn't it? We will tell you that. Let's go. What have we done now? We will tell you.  We were running DFS. Very good. We were running DFS from a node. On any node. Good."
    },
    {
      "start": 1539.07,
      "end": 1565.82,
      "text": "We used to pass it to the parent. Okay. If we get a neighbor who is already visited. Okay. And parent is not there. Parent is not there. So, what did we say? Cycle. This is what we were doing. This was the whole crux. Now, put it here and see. Come on. We first fail this logic. We first fail this logic. In undirected, we were told that"
    },
    {
      "start": 1565.82,
```

```
      "end": 1595.66,
      "text": "You can't make it by combining two people. You can't surround someone by
combining two people. That's why you have to do the parent one. Obviously, how will you make a
cycle by combining two people? Triangle is the first shape that falls. For that, you need
three edges. Two people make a line together. But when you talk about directed, you can do a
lot by combining two people. There are two nodes. Suppose you have a directed graph. If you
have one edge called 0, 1, then you just have to make an edge from 0 to 1 by giving an arrow.
Not from 1 to 0, you have to assume it."
    },
    {
      "start": 1595.66,
      "end": 1625.47,
      "text": "Suppose you are given an edge from 0 to 1. Very good. Very good. But you are
given another edge from 1 to 0. And the graph, JFG can give anything. There is an edge from 1
to 0. Look at this. There is an edge from 1 to 0. Does this look like a cycle? Yes, it looks
like a cycle. Two people made a cycle together. In directed and in undirected, two people were
not able to make a cycle together. Because two people will pull a line. What will happen in
that? But because there is an arrow in directed,"
    },
    {
      "start": 1625.47,
      "end": 1654.32,
      "text": "This is also a cycle. This is also a cycle. You can't say that there are only
two people in this. No, there is a cycle. How? From zero to one. From one to zero. This is
called a cycle. We got stuck, right? We were not able to get stuck in the upper one. You can
say that we got stuck there too. But we want to show you from a drawing perspective. We could
not make a cycle of two people in the upper one. It did not look like a cycle. It looks like a
cycle. This is also a cycle. This is also a cycle."
    },
    {
      "start": 1654.32,
      "end": 1683.7,
      "text": "Now, if you do that in this. Now see, we write its neighbor. 0's neighbor is 1.
1's neighbor is 0. First of all, what will you do? Many people must have understood what will
happen. DFS 0. Take visited. It is visited. In the beginning, false. DFS 0. First of all, what
will you do? You will make it true. Very good. Take its neighbor. Neighbor 1. Is 1 visited? No,
 it is not yet. Very good. If it is not visited, then you don't have to think anything. You
have to come to DFS. You have come to 1. First of all,"
    },
    {
      "start": 1683.7,
      "end": 1710.48,
      "text": "Mark it as true. Take out the neighbor of 1. What did you get? 0. Is 0 visited?
Yes, 0 is visited. But according to your original logic, 0 is its parent. It's parent is 0,
right? 0 is its parent. So will you consider 0? No, you will ignore 0. Ignore it. And no
neighbor of 0 is left. This is over. No neighbor of 0 is left. This is over. With which
thought did you come forward? There is no cycle."
    },
    {
      "start": 1710.48,
      "end": 1739.54,
      "text": "But is there a cycle here? Yes, there is a cycle here. So here, the parent
thing failed. If your graph is like this, let's say your graph is not like this, your graph is
not like this, it's just this. This is not a cycle, this is open. But if you are given such a
graph, then this is a cycle. So here, at least, at least, don't do the parent thing. Don't do
the parent thing. Now, a lot of kids will be happy that if we don't do the parent thing, then
it will be simpler. We were together as parents. What will we do?"
    },
    {
      "start": 1739.54,
      "end": 1769.09,
      "text": "We will do simple DFS. We will do simple DFS. And we will remove this parent
condition. We will remove it completely. On top of that, in undirected, we had already thought
that we will do DFS. If we get a neighbor who is visited, then we will say that there is a
cycle. Now to solve it in undirected, we have added parent. Here we are getting to know that
parent will not work in this. So it is a good thing that we will remove parent. And we will
write the same code. Come, let's write it. Let's write it and show you if it works or not.
Assume."
    },
    {
      "start": 1769.09,
      "end": 1798.75,
      "text": "This is 0. This is 1. This is 2. And this is 0. This is your graph. Is there a
cycle here? Yes, brother. Look at this. This, this, this. Is there a cycle or not? Make a
drawing and see if there is a cycle or not. Now, many children will say that it will be fun
here. Which is the neighbor of 0? Remember. The neighbor of 0, where does it go from 0? So, it
goes from 0 to 1. Not to 2. Not to 2. It is not undirected. It is going from 2 to 0. Means, it
```

```
will be 0 in 2. It is not 2 in 0. Okay."
    },
    {
      "start": 1798.75,
      "end": 1828.3,
      "text": "The neighbor of 1. Where is the edge going from 1? Only on 2. Only on 2. The
neighbor of 1 will not have 0. It is not directed. It is not undirected. Who is in the
neighbor of 2? 0. In the neighbor of 2, 0. Okay. Now, this will work. 0, 1, 2. This is your
visited. In the beginning, everything is false. So, where did you come? First of all, you
reached DFS of 0. You made 0 true. Very good. The neighbor of 0, 1. Is 1 visited? No. Come to
DFS of 1."
    },
    {
      "start": 1828.3,
      "end": 1855.55,
      "text": "Come to DFS of 1. Make this true. 1's neighbor? 1's neighbor? 2. Okay. Is 2
visited? No. Very good. Come to DFS 2. Make 2 1. 2's neighbor? 0. Is 0 visited? Yes, 0 is
visited. And here we have removed the parent's loop. So what will you say? Yes, 0 is visited.
There is a cycle. And there is a cycle here too. So is this the right solution? Absolutely not
the right solution."
    },
    {
      "start": 1855.55,
      "end": 1880.51,
      "text": "That's why we made the graph above. Now let's tell you that this solution is a
good solution. But you will have to add one more small thing in it. We will explain to you why
you have to add it. Let's go. Come. Now whatever logic you had, we will do DFS, is it never
visited or not, we will run it here and see. First of all, see the drawing. Is there a cycle
here? See, 0 to 1 can go, 1 to 3, 3 to 2, but"
    },
    {
      "start": 1880.51,
      "end": 1909.84,
      "text": "This arrow is on this side, so this is not a cycle. If it would have been from
2 to here, then it would have been a cycle. You are understanding the cycle, right? Can you
come back to that place while rotating? You can see. Start rotating from 0. Have you come back
to 0? No. There is no way to go from 2 to 0. Understand this very carefully and finely. People
don't understand this. People think that yes, there is an edge. There is no edge. It is from 0
to 2. It is not from 2 to 0. See something else. You can go from 1 to 3. You can go from 3 to
2."
    },
    {
      "start": 1909.84,
      "end": 1939.79,
      "text": "You can't go anywhere from 2. What is 2? It is a dead end. There is no cycle
here. Call it no cycle. Look at this drawing. No. Can you start from any node and reach that
node by doing DFS? Look at it. Check it according to yourself. By looking at the entire
permutation combination, you will know that no, there is no cycle in it. But our algorithm
till now. First of all, let's write neighbor. Where is the edge going from 0? 1 and 2. From 1
to 3."
    },
    {
      "start": 1939.79,
      "end": 1969.82,
      "text": "From 3 to 2. There is no neighbour of 2. There is no neighbour of 2. There is
no edge of 2. There is no neighbour of 2. Okay? Make it visited. 1, 2, 3, 4, 0, 1, 2, 3. Okay?
In the beginning, all false, all empty. Start your DFS. Start your DFS from 0. Okay? First of
all, it got visited in 0. Now, just for your understanding, just for your understanding,"
    },
    {
      "start": 1969.82,
      "end": 1999.94,
      "text": "Hold the first neighbor 2. It depends on the graph. 2 comes first, 1 comes
first. What difference does it make? Hold 2. Neighbor 2. Is 2 visited? No. Very good. Okay.
Come to 2. After coming to 2, mark it. Is there a neighbor of 2? No. 2 is dead end. You can't
go anywhere from 2. 2's neighbor is empty. Talk on Instagram. 2 doesn't follow anyone. 2
people follow 2. 2's following list is empty. You can't do anything about that profile. You
have to go back."
    },
    {
      "start": 1999.94,
      "end": 2025.92,
      "text": "You will have to go back to your DFS 0, where 2 is running. There is one more
neighbor of 0, 1. Now run 1. If we had run 1 first, then you wouldn't have understood. Now run
1. Is 1 visited? No. Very good. If it is not visited, then you don't have to think about
anything. Come to 1 in DFS. First of all, mark 1 as visited. What is the neighbor of 1? It is
3."
```

```
    },
    {
      "start": 2025.92,
      "end": 2053.52,
      "text": "Is 3 visited? No. Very good. Very good. It's a good thing if it's not visited.
DFS of 3. Mark 3 as visited. Now open your mind and see. Open your mind and see. Who is the
neighbor of 3? 2. Is 2 visited? Yes. 2 is visited. So what will you say? It's a cycle. Here.
Cycle is equal to 2. According to your logic. According to the logic we thought till now. But
is there a cycle in this graph?"
    },
    {
      "start": 2053.52,
      "end": 2082.56,
      "text": "No, there is no cycle in this graph. There is no cycle in this graph. There is
no cycle in this graph. But when you came to 3, you got 2 in your neighbor. And 2 was already
visited. What does this mean? You are saying that there is a cycle. This means that our
approach is a little wrong. It is definitely wrong. Otherwise, it would have worked. This is a
good graph, right? It should have worked. It is wrong. Now I will tell you why it is wrong.
See. Now see. Come back. Where? On Instagram. Now stay on Facebook. We are on Instagram."
    },
    {
      "start": 2082.56,
      "end": 2112.02,
      "text": "If you follow on Instagram, then there is an edge between you. It depends on
you whether it follows you or not. Now let's see the cycle. It is the same graph above, we
have just changed the name. You start scrolling from Pratyush. We write something here that
what are you doing? Where are you now? What color is this? Where are you now? Start on
Pratyush. We are just seeing your behavior that what are you doing?"
    },
    {
      "start": 2112.02,
      "end": 2141.23,
      "text": "He came to Pratyush. He visited Pratyush. Suppose you went to Sachin from
Pratyush. Where did you go from Pratyush? You went to Sachin. Sachin also visited. Very good.
Till now, this is your path. This is your session. Think of it as a session. What are you
doing? You clicked on Sachin's profile from Pratyush. Imagine you clicked here. There is no
one from Sachin. That means Sachin doesn't follow anyone. You can see there is no one from
Sachin. You can't go anywhere from Sachin. Sachin doesn't follow anyone."
    },
    {
      "start": 2141.23,
      "end": 2170.21,
      "text": "What did you do in this case? Where did you come back from Sachin to Pratyusha
by pressing the back button? You came back from Sachin to Pratyusha by pressing the back
button. Now the session of your mind, which is called session, has become Pratyusha again.
Sachin is not in your mind at all. You have come back from Sachin by pressing the back button
to Pratyusha. Now to Pratyusha. Pratyusha follows other people and follows Amit."
    },
    {
      "start": 2170.21,
      "end": 2197.18,
      "text": "Now you went from Pratyush to Amit. In your session or in your scrolling path,
who all are there now? One is Pratyush and one is Amit. Sachin is not in your path. You went
to Sachin and came back. He is not in your path. Okay? Come. Come to Amit. Amit is also
visited. Where is Amit's neighbour? Amit's neighbour is Seema. Come to Seema. Come to Seema."
    },
    {
      "start": 2197.18,
      "end": 2222.61,
      "text": "When you come to SEMA, what is going on in your mind? Pratyush, Amit, SEMA.
Sachin is not in your path. You went to Sachin and came back. It is over. It is a dead end.
Sachin's account is a dead end. Okay. SEMA is also visited. Who is the neighbor of SEMA?
Sachin. You get to know that Sachin is visited. Normally, what did you say upstairs? Yes,
there is a cycle. But..."
    },
    {
      "start": 2222.61,
      "end": 2249.66,
      "text": "Is Sachin in your path? Do you remember Sachin? No, you don't remember that
Sachin is in your path. Why? Sachin's path is different. That's why we are making it like a
tree. Sachin's path is different. Suppose Seema's friend had Pratyusha. Seema's friend would
have done this. So this is a cycle. From here to here."
    },
    {
      "start": 2249.66,
      "end": 2259.46,
      "text": "From here to here, from here to here. So see how it would have been made. If it
```

```
would have been in the path of the border, see, this cycle would have been made."
    },
    {
      "start": 2259.46,
      "end": 2286.78,
      "text": "If you follow the path to the limit, the current session in your mind, you have
reached the limit. You have reached the limit. But if you are reaching Sachin from the limit,
Sachin is not in your path. Sachin is not in your path. He is not in this session. Sachin was
in another path. Sachin was in a different path. That's why you have to do one more thing. You
have to do one more thing."
    },
    {
      "start": 2286.78,
      "end": 2316.64,
      "text": "You have to track the path's array. What? You have to track the path's array
that if you get a neighbour who is visited, like you got Sachin in SEMA who is visited, but is
Sachin somewhere in that current path? Is Sachin somewhere in that current path? I will tell
you one more example of this. This is why I am telling you this because it is very important
to understand this. Suppose you start from Pratyush. You start from Pratyush. You start from
Pratyush."
    },
    {
      "start": 2316.64,
      "end": 2345.5,
      "text": "You came to Sachin. From Sachin, you came to Amit. From Amit, it ended. Amit
doesn't follow anyone. You came to Sachin. Sachin also ended at Pratyush. Now you came to
Pratyush at Seema. When you came to Seema, in your path, Sachin and Amit are not there. Their
DFS is over. Amit was dead end. Amit's DFS is over. Sachin's DFS is over. Now, if you are
getting Amit in Seema's neighbour, is this a cycle?"
    },
    {
      "start": 2345.5,
      "end": 2374.4,
      "text": "No, this is not a cycle. Let's make a graph and show it to you. Is this graph
cyclic? No. If you start from Pratyush, you can go to Seema from Pratyush. You can go to Amit
from Seema. Is there any way to go from Amit to Pratyush? No, there is not. You will not be
able to reach Pratyush from anywhere. This is the concept of the path. The DFS of these two is
over. Now if this is coming in someone's visited, you have to ignore them because"
    },
    {
      "start": 2374.4,
      "end": 2404.13,
      "text": "If you see in this graph, this is not a cycle. This is not a cycle. On the
contrary, on the contrary, those whose DFS is not over, those who are running in it. Suppose
you came to the border. From the border, you reached Raju. Okay? From Raju, you reached Mohan.
At this time, whose DFS? Their DFS is over. Whose is running? These four. You are going in.
You are going in scrolling. Now if from Mohan,"
    },
    {
      "start": 2404.13,
      "end": 2433.44,
      "text": "There is an edge up to the border. That means there is a border in Mohan's
neighbor. And the border is visited. And the border is in this path. So is this a cycle? Yes,
this is a cycle. See, this becomes a cycle. So here in directed, you don't just have to see
visited. You got a neighbor who is visited. And its path of that neighbor should also be 1.
What will we do with the path? When we will start someone's DFS of node,"
    },
    {
      "start": 2433.44,
      "end": 2462.06,
      "text": "When we start the DFS, we will also make the path of node 1. And we will make
visited 1. We always make visited 1. Path of node means the DFS of this node has started. When
we will end the DFS, before the return, we will return the path of node 0. That its DFS is
over. It has moved away from the path. What will happen with this?"
    },
    {
      "start": 2462.06,
      "end": 2493.44,
      "text": "If you get a graph like this, it's a small example. It's a bit hard to
visualize, but think about it. Pratyush goes to Sachin. There's a limit. The limit goes to
Sachin and Pratyush goes to the limit. Is there a cycle in this graph? No. We'll make two
arrays. We'll make two visited arrays. Whose? Pratyush's, Sachin's and Seema's."
    },
    {
      "start": 2493.44,
      "end": 2522.4,
      "text": "And we will make a path array. In this also, one for Pratyush, one for Sachin,
```

```
and one for Seema. In the beginning, everything will remain zero. Path means, is his DFS
active? That means, have you reached here from that person's account? If you went to that
person's account, and it was over, that means there was nothing else, you backed the account.
Backed means it is gone. Now you don't have to consider that account in the cycle."
    },
    {
      "start": 2522.4,
      "end": 2551.04,
      "text": "He went to Sachin's account. There was no one else in Sachin's account. He had
to come back. That means he had to finish the DFS. He had to finish it. That means he went
from the path. See how he will do it. First of all, you came to Pratyush. What did you do? You
closed Pratyush's visited and closed Pratyush's path. Pratyush has come to the path.
Pratyush's DFS has started. Very good. Who is Pratyush's neighbor? Sachin. Is Sachin visited?
Sachin is not visited. So what did you do? You came to DFS of Sachin."
    },
    {
      "start": 2551.04,
      "end": 2580.5,
      "text": "Pratyush's path is still 1. Sachin's path is also 1. Sachin's visited path is 1
and Sachin's path is also 1. Can you go anywhere else from Sachin? No, Sachin can't go
anywhere else from dead accounts. So what do you have to do? You have to go back. But before
going back, zero Sachin's path again. To tell that Sachin has left the picture. Now if Sachin
is someone's neighbor, whose visited path is 1, that doesn't mean that Sachin's path is 0."
    },
    {
      "start": 2580.5,
      "end": 2609.74,
      "text": "Come back to Pratyush. Pratyush has one number left. Where did he go? To the
border. If he goes to the border, then when you come to the border, Pratyush's path is 1.
Sachin's path is 0. It means, it looks like this. Pratyush's path is 1. It means, you have
come to the border of Pratyush. Sachin's game is over. Come to the border. First of all, make
the visited of the border 1. Just like DFS, visited 1. Don't tease the visited. And make the
path of the border also 1. Make the path of the border also 1."
    },
    {
      "start": 2609.74,
      "end": 2639.01,
      "text": "Seema's one neighbor is Sachin. Is Sachin visited? Yes, Sachin is visited. But
is Sachin in this path? No, Sachin is not in this path. Sachin's path is zero, so what does he
have to do? He has to ignore it. He has to ignore it. Okay. Seema has no other neighbor. Go
back. But before going back, Seema's path is also zero. Seema's BDFS is also over. Come back
to Pratyush. Is there anything left to do at Pratyush? No. Go back."
    },
    {
      "start": 2639.01,
      "end": 2668.34,
      "text": "While going, make the return zero as well. This is the concept of DFS path and
session. We have tried to explain. If you have understood or not, then tell us in the comments.
 But in this way, when Sachin's path is over, then it's okay. Ignore Sachin. Sachin's path is
over. Because if you do this, we have shown you here. If you have made this graph. Where did
you make this graph? This one. If..."
    },
    {
      "start": 2668.34,
      "end": 2697.22,
      "text": "If you make a line, then this is not a cycle. You can't go from Sachin to
Pratyush. Sachin's path is over. But if you make a cycle from Seema to Amit, then this is a
cycle. Why? Amit is in the path. Sachin is not in the path. Sachin is not in the path. So the
whole thing will remain the same. Like undirected. We are not writing the code to make a graph.
 You make the graph yourself. See what will happen in this. First of all, bull cycle is equal
to false. Same, same code."
    },
    {
      "start": 2697.22,
      "end": 2730.62,
      "text": "write dfs, void dfs what will be passed? a will be passed a will be passed,
okay? int, pass the node, pass the n as well as the node now in the upper one, the parent was
passed, we don't need the parent in this, we need two arrays the first array is path bool path
or visited or visited bool visited"
    },
    {
      "start": 2730.62,
      "end": 2760.69,
      "text": "It is important to keep visited because if we keep a path, let's say Pratyush
is in the path. It is important to keep visited because you have to track the component as
well. Some people will have a doubt that when you are keeping a path, why do you have to keep
visited? You have to keep visited because you have to track multiple components as well. Let's
```

```
go. The first thing to do is visited of node is equal to 1. And path of node."
    },
    {
      "start": 2760.91,
      "end": 2794.45,
      "text": "Is equal to 1. This means you came to this Instagram account. Now you are
entering into it. When you press the back button of this account from this Instagram account.
Then zero the path. Do the same thing. For int j is equal to 0. j less than array of node dot
size. And j plus plus. Take out its neighbor. Int neighbor is equal to array of node j. If. If.
"
    },
    {
      "start": 2795.31,
      "end": 2825.97,
      "text": "Visited of neighbor. Equal equal to one. And. And. And visited of path. Sorry.
Path of neighbor. Path of neighbor. Equal equal to one. Means. You got such a neighbor. Which
is already visited. And is in this path. And is in this current path. Means. The limit is
being made at the intersection. Amidst the intersection. Amidst the border. Back to the
intersection. There is a cycle. There is a cycle. Very good. That's it. And. Do one more. If."
    },
    {
      "start": 2826.06,
      "end": 2855.92,
      "text": "its visited is false, its visited is false, then you run DFS on it. a, n, run
DFS on neighbor, path, visited. That's it. Now, you used to return DFS. Now you used to return
DFS, but before returning, path of node is equal to 0. Its DFS is over. You are pressing back
button on this account. There is nothing left to do. That's it."
    },
    {
      "start": 2855.92,
      "end": 2885.84,
      "text": "See, this is your small code. So, I hope you understood the code. And one more
small thing. When we have a path, why do we need visited? We need visited because there is one
more thing. Suppose you come to 0. See this graph. There is no cycle in this graph. You come
to 0. Suppose you are not keeping visited, you are just marking the path. The path of 0 is
green. Okay. You come to 2. Come to 2. The path of 2 is 1. You can't go anywhere from 2."
    },
    {
      "start": 2885.84,
      "end": 2911.39,
      "text": "What did we do before going? We made the path of 2 to 0 and came back to 0.
Okay? Good. Do you want to go to 1? The path of 1 is 0. Go and come back to 1. Okay? This is
one problem. There are many more problems. We came back to 1. Now, who is the neighbor of 1? 2.
 Now, is the visited of 2 not there? We will check the visited. There is a path. The path of 2
is 0. So, what will we do? We will do DFS on 2 again. DFS had already been done on 2 once. We
came from 0 to 2 and did it."
    },
    {
      "start": 2911.39,
      "end": 2940.53,
      "text": "But now 2 is over. Now see from 1 to 2 again. So DFS is called again on 2. By
doing this, you will call more and more DFS. And you will get TLE. According to me, you will
get TLE. There can be many more problems. We will not think so deeply. But this is a small
problem that we can see here. That multiple DFS will happen. It came on 2. Now according to
which DFS? Visited means that we will not do DFS on it if it is 1. If you are saying visited,
we will remove it. So you will have to do it according to path."
    },
    {
      "start": 2940.53,
      "end": 2969.68,
      "text": "See, it was on 2. Now when it came to 1, then the path of 2 again became 0. It
went back to 2. So this will keep happening. This is not a good thing. That's why we have kept
both. So you have also learned the directed cycle. We will suggest you to spend some time on
this explanation. Spend some time on code. Spend some time on dry run yourself. Mark in the
array and see what can happen and what cannot happen. This is a dry run question. The more you
learn from dry run, the more you will not be taught by any master."
    },
    {
      "start": 2969.68,
      "end": 2976.66,
      "text": "So now you have learned the cycle very well and covered all the bases. See you
in the next lecture."
    }
  ]
}
```

21. transactions/t_29.json


```
{
  "video_id": "DL8LSXUsfWE",
  "title": "DSA PATTERNS Episode 11 Sliding Window Pattern: Easy & Medium Questions",
  "language": "en",
  "model": "large-v3",
  "duration": 4182,
  "segments": [
    {
      "start": 0.0,
      "end": 28.98,
      "text": "Today's two questions are only two questions for me to read the sliding window. If we had to teach two questions to someone in life and asked them to explain the sliding window, then we would have told these two questions. And the examples we have told, the examples of hiring and firing, that how the company keeps hiring, the work starts, and keeps firing. The same concept is used in the sliding window. On the internet, you will not get such an example and such an intuition. You will get the code, you will get the code everywhere, you will keep copying it, but..."
    },
    {
      "start": 28.98,
      "end": 58.05,
      "text": "If you want to understand intuition, then watch both the questions. If you are watching the video and there are new students, then watch both the questions. If you are watching, then watch both the questions. If you are not watching, then don't watch both the questions. If you watch one and go for the interview, then you won't be able to watch the other one. This is their link. It was simple. Let's start. It's going to be a very good video. It's going to be a detailed video. Please watch it. It's been a long break. It's going to be proper from today."
    },
    {
      "start": 58.05,
      "end": 88.1,
      "text": "From today, the video will also be on and you won't hear the sound of cutting. Those who are coming to the channel for the first time, today is day 2 of Sliding Window Pattern. You will get the sheet from the description. And the question we are going to do today is two questions, we will tell you in a while. You will get the notes of the iPad in the description. You will also get the link of Insta and LinkedIn in the description. And you will also get the code link in the description in CPP, Java and Python. All these things are done."
    },
    {
      "start": 88.1,
      "end": 118.45,
      "text": "Today we are going to do two questions. First question is, given an array, this is not my word, it is written as it is in the question. First we will see the word of the question and then we will see how the pattern is applied, why it is applied, what to do and how to apply it. What you have to do is, you have given an array, what you have to do is, maximum sum of any subarray, any subarray, not sub sequence, subarray."
    },
    {
      "start": 118.45,
      "end": 148.08,
      "text": "Of size K. This is your question. Let's recap what happened the last day. Because many people forgot. When does the sliding window pattern apply? First of all, there should be an array question. Array or list question. Or link list question. In question, subarray or substring should be written. In array list, string also comes. String is also an array. Subarray or substring question should be there."
    },
    {
      "start": 148.08,
      "end": 178.59,
      "text": "And if both of these are true, after that what we are trying to do is to find this. So what are we trying to find? Either we are trying to find max, min of something, either longest, or shortest, anything like that. Either we are doing sum, count, or average, or at most k."
    },
    {
      "start": 178.59,
      "end": 205.82,
      "text": "at most k or at least k at least k exactly k ok we are trying to find the combination of all these things if these two are not true we don't have to come to this step if these two are true then we have to check in this list that are we trying to find the combination of all these things in this question we will see is it an array question? yes it is an array question so this is true"
    },
    {
```

```
      "start": 205.82,
      "end": 234.7,
      "text": "Do we have to work with sub-arrays or sub-strings? Yes, you have to do
something with any sub-array. This is also true. Now we will look at the second list. What do
we have to do with find? Maximum, sum. Maximum, sum. Means, maximum, sum. So, this combination
is also set. If these three things are set, then it will never happen in the world that this
sliding window will not be there. You can do it in some other way, but the best and optimal
way will be its sliding window."
    },
    {
      "start": 234.7,
      "end": 263.78,
      "text": "So you can see that after reading the question of two lines, you understood
that it is a sliding window. So your 50% work is done. The interviewer is 50% happy with you.
It's been so long that you understand what kind of question it is. Now in this graph, tree, dp,
 some people start applying dp, some people start doing something else. No, we don't do all
that because we read the pattern. So after reading the question, we understood that this is a
sliding window pattern. Okay, that's good. Now, now."
    },
    {
      "start": 263.78,
      "end": 293.36,
      "text": "Sliding window pattern. First of all, what is sliding window? This is in the
introduction video. Go and see in the playlist. What is the addition in that? Any question of
sliding window. Any question of sliding window has three steps. What happens in that? There
are three steps. Okay. The first step is identify pattern. The first step is identify pattern.
Meaning, is this a sliding question or not? So, we have already done this work. Very good."
    },
    {
      "start": 293.36,
      "end": 323.28,
      "text": "The second step is to find out the window type. What does window type mean? Is
it a fixed window or a variable window? Is it a fixed window or a variable window? Do you
remember what we said in the first intro? We said that in any array or any list, what does
window mean? Window means this is a window. Now this window moves, means it slides. You must
have seen the sliding door. It slides with the hand. Similarly, this window stays here."
    },
    {
      "start": 323.28,
      "end": 351.26,
      "text": "then this window comes here, then this window comes here, so it keeps sliding.
Now, this window that slides, does it mean that the same size window is sliding every time? If
this window is of 3 sizes, then does it mean that every time, by lifting this window, it is
sliding one or two steps? Earlier, this window was here, of the same size, of 3 sizes. We have
placed 3 here, then we have lifted these 3 and placed them here, then we have placed them here.
 So, this will be a fixed window. It means that the size of the window is not changing. If the
window"
    },
    {
      "start": 351.26,
      "end": 370.22,
      "text": "It is of 3 sizes, so that window is of 3 sizes. It is sliding up from here to
there. That is called FIX. What is a variable window? It means, if this window is like this,
then variable means that it is changing the size of the window. The position of the window
changes. The window is sliding from here to there. The size of the window also means,"
    },
    {
      "start": 370.22,
      "end": 399.22,
      "text": "It is decreasing from the back and increasing from the front. Sometimes it is
decreasing from the back, so the window is getting smaller. Sometimes it is increasing from
the front, so the window is getting longer. Means the first window was like this. Then we cut
it, so the window started looking like this. Then we increased it, so the window started
looking like this. That is called variable window. The second step is that you find out
whether the work of fixed window or variable window is in this question. How will you do that?
We will also tell you how to do that. What is the third step? The third step is"
    },
    {
      "start": 399.22,
      "end": 427.09,
      "text": "Where did it go? What is the third step? Once you have done the work of fixed
and variable window, once you have understood that it is fixed and variable, whatever window
you are starting from, any window you are starting from, if you start from it, it will have a
size and a position. For example, you say that my window starts from start and it is of 3
sizes. So, this is your starting point of the program. So, what is the next step?"
    },
    {
```

```
      "start": 427.09,
      "end": 456.02,
      "text": "Find out its data or its information. This window represents information. It
may be representing the elements inside me. If this window is of 3 elements, it may be telling
me the sum. It may be telling me who is max and who is min. This is called window information.
What does that window mean?"
    },
    {
      "start": 456.02,
      "end": 483.06,
      "text": "When the computer sees that window, then it understands 0, 1. When you see that
window, then what do you understand? It is called information. Meaningful data is called
information. There are 3 elements in the window. 10, 20, 30. This is data. This is just data.
The window is of 3 sizes. In that, 10, 20, 30. But what information is it getting? It is
possible that you are getting information. This is 10, 20. And the sum of 30 is 60. This is
the information of that window. Okay?"
    },
    {
      "start": 483.12,
      "end": 507.18,
      "text": "First of all, find out the information of that window in any way. Now start in
any way. We will discuss all this further. This is just an approach. There are four steps to
solve any sliding window pattern. Apart from this, no step, no question is asked anywhere. Now
what is the fourth step? You have found out the starting of a window. It must be starting from
somewhere and its size is very good."
    },
    {
      "start": 507.18,
      "end": 529.28,
      "text": "make the window slide or change its size as per your requirement make the
window slide or change its size as per your requirement make the window slide or change its
size as per your requirement make the window slide or change its size as per your requirement
make the window slide or change its size as per your requirement make the window slide or
change its size as per your requirement make the window slide or change its size as per your
requirement"
    },
    {
      "start": 529.28,
      "end": 548.4,
      "text": "new window jo bana usme ek aur element add ho gaya ho to uska sam ya to bad
gaya hoga ya ghat gaya hoga ya to change ho gaya hoga ya to same raha hoga us new window ka
information find out kar lijiye new window ka information find out kar lijiye find out kar
lijiye"
    },
    {
      "start": 548.43,
      "end": 570.34,
      "text": "Keep repeating this until you reach the end of the array, or until you get your
answer, or until your window is finished. Any end condition can happen. It means that there
are 4 steps in the pattern of any sliding window. Let's see it again. First of all, identify
whether there is a sliding window or not. Done."
    },
    {
      "start": 570.34,
      "end": 596.02,
      "text": "Is it a fixed or variable window? Is it a fixed window or a variable window?
Will the size of the window keep changing? Or will the size of the window remain the same?
Find that out. Find the information of the window you are starting from. Find the information
of the window you are starting from. Find the information of the window you are starting from.
Find the information of the window you are starting from. Find the information of the window
you are starting from. Find the information of the window you are starting from. Find the
information of the window you are starting from. Find the information of the window you are
starting from. Find the information of the window you are starting from."
    },
    {
      "start": 596.02,
      "end": 625.86,
      "text": "and we are looking at different parts of the array. This is what it means.
There are 10 houses. You can see 2 houses at a time. So you can see 2 houses at a time and you
will know which one is the most beautiful. Now you are looking at the next 2 houses. You will
have to find out the information of that too. Similarly, you will be able to compare by
looking at 10 houses that which house is the most beautiful in 10 houses. So this happens. So
do this. Your program will be solved in 4 steps. Now we will solve our question with these 4
steps."
    },
    {
      "start": 625.86,
```

```
      "end": 655.14,
      "text": "What is your question? You have given an array. Okay. What is the question in
Hindi? What is the question you are trying to say in Hindi? There is a sub-array in an array.
What is a sub-array? If there is an array, any continuous part of it is called a sub-array.
For example, this is its sub-array. Sub-array. Array is also a sub-array. There was a subset.
There was a set. What was its subset? A small part of it. It is the same. Now, what is the
question you are asking?"
    },
    {
      "start": 655.14,
      "end": 684.75,
      "text": "you will be given an input of k the sum of all the arrays in that array you
have to find the sum of all of them you have to find the sum of all of them from that you have
to return the maximum sum you have to return this is the meaning of the question in Hindi this
question is saying that there is an array we are giving you a number k if k is 2 then keep all
the arrays of 2 sizes with you"
    },
    {
      "start": 684.75,
      "end": 714.51,
      "text": "Find the sum of all. In that sum, return whatever is the maximum sum. For
example, if we try to see its example, then it has 100, 200, 300, and 400. Okay? 0, 1, 2, 3.
What did you say? It has K2. It has K2. Now it tells you to give the answer. So, of two sizes.
The sum of two sizes, one is of two sizes, 100 and 200. The other is of two sizes."
    },
    {
      "start": 714.51,
      "end": 742.86,
      "text": "The second sub-array is 200 and 300. The third one is 300 and 400. You have
three sub-arrays. One here, one here and one here. You have to find the sum of all three sub-
arrays. Find the sum of all the elements in that sub-array. It can be S1, S2 or S3. Return the
maximum of these elements. If we look at it according to this question, then what will be the
sum of the first sub-array? 100 plus 200."
    },
    {
      "start": 742.96,
      "end": 771.01,
      "text": "The second one is 200 plus 300. The third one is 300 plus 400. The sum of this
will be 300. The sum of this will be 500. The sum of this will be 700. Who is the most among
the three? 700. Your answer should be 700. Is it correct? Did you understand? Okay. Now, why
does this have a sliding window? If you look at the pattern, you must have remembered that the
sliding window is written in the question. That is a good thing."
    },
    {
      "start": 771.01,
      "end": 800.05,
      "text": "Even if you don't know the pattern, even if you don't understand the question,
then what are we doing? We are looking at 2 sizes of subarrays. First, we looked at this. Then
we looked at this. What happened in looking at this? We looked at 100 and 200 in the previous
subarray. After that, we subtracted 100, there are 200 too. And we added 300. We were taking 2.
 We did something of 2 and got the answer."
    },
    {
      "start": 800.05,
      "end": 821.04,
      "text": "After that, we left the previous one. We left the previous one, 100 here. And
we took the next 300 inside. We took the next 300 in our hand. We took out its sum. Then the
same thing happened with 100 and now with 200. Earlier, we were seeing this. After that, we
crossed 200 also that you also go. And we took 400 inside. So now we are seeing 300 and 400.
Now,"
    },
    {
      "start": 821.04,
      "end": 850.75,
      "text": "now if you try that whatever happened with 100, 200, even with 300 then remove
300 also now 400, but now there is no one to take it inside the array array is finished that's
why we are able to make only 3 saberes, understood? understood? what are you saying? every
time what we are doing is removing one person from the back and taking one person from the
front so the size of the sabere is maintained maintained right? because what you have been
asked in the question that this size is the sabere you have not been asked to remove sabere of
any size, no"
    },
    {
      "start": 850.75,
      "end": 877.28,
      "text": "k is equal to 2, you have to find a sub-array of 2 sizes. Every time you have
to find a sub-array of 2 sizes and tell the sum from it. So, our step 2 was, is our window
```

```
fixed or is our window variable? Our window is fixed. Why? Because the question is asking you to find a sub-array of k sizes. Don't find it with your own mind. If k is 2, any sub-array should be of 2. To say, there is a sub-array of 400 alone."
    },
    {
      "start": 877.28,
      "end": 888.78,
      "text": "200, 300, 400 is also one bag. But why are we not doing this? Because its size will be different. If we take 3, its size will be 3. If we take 400 alone, its size will remain the same. We need 2."
    },
    {
      "start": 888.78,
      "end": 915.01,
      "text": "That's why we are taking two sizes of barrels every time. We are taking two sizes of windows. Window means sliding. Earlier we had taken this window. Now we have taken this window. Now we have taken this window. So the window is sliding. What does it mean to slide? Leave the back one and take the front one. Then leave the back one and take the front one. The window is sliding equally. But every time its size remains the same. It is not changing the size. We are not doing that at the back."
    },
    {
      "start": 915.01,
      "end": 944.58,
      "text": "If we leave one, then we know that we have taken two, then the size will increase. We are also not doing this that we did not leave the back one, we took the front one, then the window will decrease. Right? So, we are not changing the size again, the size is 2 only. So, with this, we come to know that the window is fixed. The window is fixed. The work of step 2 is done. Step 1 was to identify the pattern, we came to know about sliding. Step 2 was, is it fixed or variable? We came to know. Now, let's come to step 3. Now, let's come to step 3."
    },
    {
      "start": 944.58,
      "end": 970.02,
      "text": "What did step 3 say? Step 3 said that whenever you start windows, take out the information of the starting point. Now what is the starting point? Starting point is not written in the question that from where you have to start. So see, starting point is not shown because starting point is very simple. It is telling you that you have to do something with the sub-array of the array. From where will the sub-array of the array start?"
    },
    {
      "start": 970.02,
      "end": 998.38,
      "text": "the start will be correct, it won't be like if you start from the middle, then where will you go? if you start from the middle, then you will go left or right, if you go anywhere, then the other one will be left out. So, this is the intuition, it is natural that you will start from here only. So, this is your starting point. Now, this is the start point. What will be the size of the window? The size of the window is already known, it will be 2. So, what is your starting window? Your starting window is 100 and 200. So, you got to know your starting point, good. You got to know your starting point, good."
    },
    {
      "start": 998.38,
      "end": 1024.9,
      "text": "What is its information? What are we trying to understand from that window? Or what are we trying to do in the question? We are trying to find the name of any subarray. We are trying to find its sum. That's why it is 700. We are trying to find its sum."
    },
    {
      "start": 1024.9,
      "end": 1053.87,
      "text": "then only we will find the maximum sum so what is the information of this? the sum of that window whatever is there in that window, the information of that window means that we will find the sum of that window this is the information of that window now how will you find the sum? now what is the subarray? subarray is also an array subarray is also a small array in that array how will we find the sum of this? will we find the sum properly? one minute"
    },
    {
      "start": 1055.12,
      "end": 1085.41,
      "text": "Yes. How will we get the sum of this? To get the sum of this, we will iterate. How do you get the sum of array? In array, you go from 0 to n-1. How will you get the sum of all array? You will get the sum of all array from 0 to where it ends. Okay. We have got the sum of that 300. Which is 100 plus 200. Step 3 is completed. We are not thinking about code now. We are thinking about how to do this step. It is very important that whenever you solve
```

```
DSA program, DSA problem, then don't think like a computer."
    },
    {
      "start": 1085.41,
      "end": 1107.89,
      "text": "Think like a human. This makes it easier. Why? Because many people start thinking as soon as they read the question that which loop will work in this, which algorithm will work in this, which graph will work in this. You don't have to think like that. Let the computer think. You are a human. Think like a human. Whatever AI is being made today, what is its goal? Its goal is to make the computer think like a human."
    },
    {
      "start": 1107.89,
      "end": 1134.24,
      "text": "What is image processing? What if the computer looks at it from the eye? The computer is thinking like a human being. And you are trying to think like a human being that we should think of a loop. You don't have to do that. We will write the loop later. Think like a human being. You have to find the sum of these two, right? So how will you find it? You will do 100 plus 200. Just do that. Write 300. Okay. Come to step 4. What was step 4? Whenever, now there is a window on 100 and 200."
    },
    {
      "start": 1134.24,
      "end": 1162.46,
      "text": "the next window is going to be 200 and 300 because the window is sliding. 100 is going to be removed and 300 is going to come. Now you have to get the information of that as well. You have to get the information of that as well. The information of that will also be the sum. We have established that the information of any sub-array is the sum of its elements. Now how will we get the sum of that? There is a very good way to get the sum of 100 and 200."
    },
    {
      "start": 1162.46,
      "end": 1189.84,
      "text": "just like the sum of 100 and 200, we can also get the sum of 200 and 300. That's a good thing. So, you iterate from here, you run i from here, and go till here, and you get 500. Very good. How will the sum of 300 and 400 come out? Just like the sum of 200 and 300 came out, you run it from here, go till here, and the sum comes out. But, this is a useless method. Why is it a useless method? Why is it? I am telling you. What you people are saying is that,"
    },
    {
      "start": 1189.84,
      "end": 1218.27,
      "text": "how will we calculate the sum of new window? like we calculated the sum of old window we calculated the sum of old window by 100 plus 200 is equal to 300 if we calculate the sum of new window then we will add 200 plus 300 so we will get 500 if we calculate the sum of new window then we will add 200 plus 300 so we will get 500 if we calculate the sum of new window then we will add 200 plus 300 so we will get 500 if we calculate the sum of new window then we will add 200 plus 300 so we will get 500 if we calculate the sum of new window then we will add 200 plus 300 so we will get 500 if we calculate the sum of new window then we will add 200 plus 300 so we will get 500 if we calculate the sum of new window then we will add 200 plus 300 so we will get 500 if we calculate the sum of new window then we will"
    },
    {
      "start": 1218.27,
      "end": 1247.66,
      "text": "For example, because there are two windows here, you are not able to understand. For example, you have five people. Okay? You have five people. They have come to your house. Okay? It's a good thing. Every person is giving you money. Every person is giving you some money according to him. He is giving five rupees. He is giving four rupees. He is giving two rupees. He is giving ten rupees. He is also giving five rupees. You have total rupees. Now take five rupees and count them."
    },
    {
      "start": 1247.66,
      "end": 1274.99,
      "text": "You have 5, 4, 9, 11, 21, 26. You have 26 rupees. You counted it and sat comfortably on the seat. Now, now, now. This man, this man, the middle one. He says, sir, we will not be able to give you money. We are going home with our money. Give us our money, 2 rupees. My brother will come instead of us. He will give you 1 rupee instead of 2. What happened? You have 5 people. You gave the money. You counted the money and kept it 26 once. Good."
    },
    {
      "start": 1274.99,
```

```
      "end": 1304.54,
      "text": "What does the middle one say? Sir, we are going home. We will take our money. We will not be able to give the money. We sell it to our brother. He will not give two rupees. He will give only one rupee. Okay. Instead of this, his brother will come. Instead of this, his brother will come. How much will the brother give? Brother will give one rupee. And this man will go home with his two rupees. Now you will return his two rupees. Take one rupee from his brother. Now how much money do you have? Now, first you had twenty-six rupees. But if a man goes, you give him two rupees."
    },
    {
      "start": 1304.54,
      "end": 1332.46,
      "text": "Give him 2 rupees and keep his brother's 1 rupee. Now how much money do you have? Now the only way to do this is that you give back all the money that he had. You get empty. Okay. And then say that then we will give the money and then we will count. Then we will put time in it and then you will know that how much money I have. But you have already counted his money. You have already counted the money of these four. Yes. Are these four going anywhere? No. Are these four asking you for money? No."
    },
    {
      "start": 1332.46,
      "end": 1356.59,
      "text": "What is the change? You have to return this 2 rupees. So you return 2 rupees and take 1 rupee from this man. The man gave 1 rupee. You had 26 rupees earlier. Now you have 26 minus 2, 24 plus 1. 25 rupees will be left. This is what a person does. If you already have it, someone is asking you to return your money, then you return it."
    },
    {
      "start": 1356.59,
      "end": 1386.19,
      "text": "Someone else gave it to you and you took it. And you calculated the change and now you know that you have Rs. 25. Earlier it was Rs. 26, now it is Rs. 25. So, what I mean to say is that the calculation of Rs. 26, it was necessary for you to talk to 5 people about how much you are giving. Because you were starting this, before this you had nothing. There was no reference. But now when you know that you have Rs. 26, then you give the money to the one who wants to give it. Minus it. Take the money of the one who wants to take it. This is also required in a bank account."
    },
    {
      "start": 1386.19,
      "end": 1415.33,
      "text": "If you have 1 lakh rupees, you drank a cold drink of 100 rupees, and someone gave you 50 rupees, then what does the bank say? Empty the entire bank account and we will count you again. No, what does the bank do? It gives you 1 lakh minus 100 plus 50 and updates your bank balance. The bank does the same, the human also does the same. So why are we doing useless work there that we are counting again and again? What will we do? What will we do? Again, look again."
    },
    {
      "start": 1415.33,
      "end": 1445.7,
      "text": "300, 400. This is our error. This is the error. Now we will work like humans. We will not work like monkeys. First window, first window, first window. First window which is this window. You will have to subtract the sum of this. Why? Because you had 0 rupees. You had 5 people. You don't have any reference. You don't know what is happening. You will have to subtract the sum of this. So S1 will be 100 plus 200. Is equal to 300."
    },
    {
      "start": 1445.7,
      "end": 1475.42,
      "text": "Very good, we got its information. Now, what is happening in the next window? In the next window, we are trying to take out 100. We are telling 100 to go. And? What are we doing after that? The window is sliding, right? What is the next window? Your next window is this. 200 and 300. What are we doing in that? We are taking out 100 and taking 300 inside. The same thing is happening which was happening earlier. One man said that we are going home, we are going to the village and my brother is coming. So, 100 is going to his village and 300 is coming back."
    },
    {
      "start": 1475.42,
      "end": 1502.43,
      "text": "Now, the S2 will come. Now, the S2 will come. Why are we calculating that we don't have to add 200 and 300 again? What is happening to you? Your previous sum was 300. The previous sum was 300. What is happening to this? Minus 100. 100 is going out. Who is A? Plus. Plus 300. Plus 300. This is happening, right? Now, what is the answer to this? 200 plus 300 is 500."
    },
```

```
    {
      "start": 1502.43,
      "end": 1531.62,
      "text": "Match it. The sum of 200 and 300 is 500. This is happening. Very good. Your S2
is made. Your S2 is made in such a short time. What did you have to do for S2? First, you had
to see the whole array for S2. That there is 200 and 300. The sum of both was working so much.
Now what are you doing? You are removing 100 and adding 300. Do the same thing. Do the same
thing. Whose was S3? S3 means this window. What is happening in this? 200 is saying that we
will go home. 400 is saying that we will come. So come. Come."
    },
    {
      "start": 1531.62,
      "end": 1555.7,
      "text": "500 is written. After this, 200 is saying that we will go home. Go. And 400 is
saying that we will come. How much was its answer? 700. Match it. The sum of 300 and 400 is
700. Now, you understood how the window is happening. How you are working in less time. Very
good. Now, like this, you got all the information. Let's revise once again what we have done.
Fixed window, variable window."
    },
    {
      "start": 1555.7,
      "end": 1584.18,
      "text": "to get the first information you have to do the work of 100, 200 and 300 after
that to get the information you have to do the work, this is the beauty of sliding window
that's why we are reading the sliding window we have to do it again and again if we think that
we will see everyone again and again then take the whole morning you know how to take out the
whole morning, right? take everyone and keep them in different places and then make everyone
so we could have done it that way but why are we reading so many patterns? we are reading to
do this, to think this"
    },
    {
      "start": 1584.24,
      "end": 1610.72,
      "text": "Now, how to write this? Now, we will come to the computer part. Think like a
human being. Now, how to remove this from the computer language? Now, how to do this? 100, 200,
 300, and 400. 0, 1, 2, 3. In the sliding window, there are two pointers. Window."
    },
    {
      "start": 1610.72,
      "end": 1640.74,
      "text": "What is window? It is a sub-array. For a sub-array, you need at least two
things. Start and end. If there is a line or a window, someone says that it is a piece. If
there is a start and end of a piece, then it will become a piece. Otherwise, it will spread.
So, there are always two pointers. The name of this pointer is given by different teachers in
different formats. But, according to me, low and high. This is the standard name of both. Low
means where it starts and high means where it ends. Very good."
    },
    {
      "start": 1640.74,
      "end": 1664.64,
      "text": "So, when we are starting, where are we starting from? From the beginning of the
array, we have to take an array of two sizes. This is the first window. In the array question,
low is always on the index from where your window is starting. So, low will be on 0 because
the window is starting from the 0th index."
    },
    {
      "start": 1664.64,
      "end": 1693.25,
      "text": "Where is the window ending? The window is ending in the first index because we
have to take two sizes. If we take 0 and 1, it will be of two sizes. If we take 3, then where
does it end? At index number 2. With this, we are able to understand the format that when k is
equal to 2 and your k is the size of the window, then your high will always be at k minus 1. 1.
 This. You didn't understand. k is equal to 2 means you have to take two sizes. You are
starting from 0, so how high will your high go? High will always be at that block."
    },
    {
      "start": 1693.25,
      "end": 1720.43,
      "text": "It stays on the thing where the window is ending. So, high will remain on 1.
High, generally speaking, will remain on k-1. In 90% of the cases. Okay? When your size is
okay. If something else happens, don't do this. Okay? Now, now, now. What did we say? To get
the first sum, you have to do that work that you have to see. How will you see? How will you
see? Simple. For i is equal to low, do it. I, run it till high. Not less than high. Not less
than high."
    },
    {
      "start": 1720.43,
```

```
      "end": 1748.66,
      "text": "The size of the array is less than n. Why? Because the index is n-1. We are
keeping the high on the index where it ends. So less than or equal. And i++. Take a sum
variable. Sum is equal to 0 and sum is equal to sum plus array of i. If you do this, if you do
this once, you will get a sum of 100 and 200. At this time, 300 is registered in your sum.
Okay?"
    },
    {
      "start": 1748.66,
      "end": 1776.8,
      "text": "Your low is equal to 0, high is equal to 1. Okay? In any sliding window, this
is a general or standard process. Okay? Here we are doing sum. If it was an average question,
then we would have done average. If it was a frequency, then we would have done anything. But
what happens is, to get the information of the first window, you have to iterate from low to
high. Very good. Now, now, now. Now we have to shift the window. We have to slide the window."
    },
    {
      "start": 1776.8,
      "end": 1804.91,
      "text": "every time while sliding, we have to take the information inside how to slide
the window? for that, first thing, till when will we slide the window? now your low is here
and high is here what does it mean to slide the window? the window which was here, that window
will come here this is it now your low is here and high is here when this window will slide,
then your low will come here and high will come here"
    },
    {
      "start": 1804.94,
      "end": 1833.92,
      "text": "This is what will happen. Low and high. Low starts here and high ends here. If
we slide, then you will have to increase the low and increase the high. If we slide one more
time, then your low will come here and your high will come here. This is what happens. Now if
we slide one more time, then your low will come here. But your high has come out of the array.
Then you understand that your work is done. What is high? High points to that index. Once your
high comes out of the array, then there is no index outside the array. Last."
    },
    {
      "start": 1833.92,
      "end": 1861.47,
      "text": "high can go till here, if high comes out of the array, your window is over,
your programming is over. So, what will we do to slide this window? To slide, there is a
standard way, we use while loop. While, while, and, where can high go? We always see where the
end is. We will see start. Start, look, start will be anywhere. If start is fixed, then start
will be behind high. What does it mean?"
    },
    {
      "start": 1861.47,
      "end": 1889.94,
      "text": "If this window is fixed, in any case, low is behind high, start and end are
behind. If you take one size of low, it will remain the same. If we take one size of sub-array,
 then low will remain the same and high will remain the same. So, either it will remain less
than low-high or it will remain equal. So, we do it on high. When high will come out of the
array, we will finish it. So, while high is less than n. What is n? The size of your array.
What is n here?"
    },
    {
      "start": 1889.94,
      "end": 1913.18,
      "text": "In this case, n is equal to 4. So, high can go till 3. High last can come till
here. As soon as high goes to 4, the program is over. Very good. Come here. Now, which window
are you on? During this time, when the computer has come here, which window are you on? You
are on the first window. Low, high is equal to 1. You are on the first window. Now."
    },
    {
      "start": 1913.18,
      "end": 1941.44,
      "text": "Do you know the current result? Do you know its information? Yes, we do. Why do
we know? We have already taken it out and are sitting comfortably. That's why we know. Now,
here, whatever you have to do, what you have to do with that result, you have to find the
maximum of this result. You have to find the maximum sum of every array. So, your information,
your information sum, you have to find its maximum. So, how will we find it? We will do it.
Like how do you take out the maximum of an array? We will take it out. So, we will take one
more variable."
    },
    {
      "start": 1941.44,
      "end": 1967.92,
      "text": "Result, which is our final result. And now what does result need? Result needs
```

```
maximum sum. So as we find the maximum of array, we will do the same. So res is equal to maximum of res and sum. What does this mean? If the result is less than sum, what is there in sum? There is sum of this window. So if our result is less than the sum of this window, then result"
    },
    {
      "start": 1967.92,
      "end": 1997.94,
      "text": "Make the result equal to the sum of this window. Very good. So, what has come in your result? 300 has also come in your result. Why? Because you can take the result as 0. Okay. This work is done. Now, we will slide the window. How will we slide the window? If it is a fixed size. Now, it is a fixed size. It is a window of only two sizes. How will we slide it? To slide it, first of all, what is this? Low plus plus."
    },
    {
      "start": 1998.16,
      "end": 2028.42,
      "text": "Okay? Low plus plus. Very good. And high plus plus. As soon as you do this, your window slides. Your window slides, right? See, where was it? As soon as you do this, your low was here. High was here. As soon as you did low plus plus and high plus plus, your low came here. High came here. Automatically, your window slides. Your window slides. Okay? Very good. What was the next step?"
    },
    {
      "start": 2028.42,
      "end": 2057.1,
      "text": "we need the sum of this window why? because when we are coming here when we are coming here when we are coming at the beginning of the while loop then we already have to use the sum we want that result to be there why? because the sum of every window is being used immediately so you have to find the result of this window means the sum of this window you have to find out the sum of this window how will we do it? like we did last time what happened here? here"
    },
    {
      "start": 2057.14,
      "end": 2086.5,
      "text": "Low has become plus plus. Means, where was the low before? Where was the low before? The low must be somewhere behind. It has come here. So, the low minus 1 has left its home. It has gone to the village. And, where the high is now, where the high is now. The high was on 2 earlier. The high has now come on 3. So, this man has been added. See, these are very subtle things. These are very subtle things. Understand this carefully. It is difficult in plus plus minus minus. They are saying that low writes minus 1. High does not write minus 1."
    },
    {
      "start": 2086.5,
      "end": 2110.16,
      "text": "Why? Think once and you will understand. Low was here, high was here. Okay? There is no confusion in this. We know the result, so starting. Now, low came here and high came here. What happened? What happened? What happened? What happened in human words? Now, low is here. So, low minus 1. This uncle went home. This happened, right? This happened, right? And..."
    },
    {
      "start": 2110.16,
      "end": 2139.84,
      "text": "Where is the high now? Where is the high now? So, 300 is added to the window. Earlier, it was not 300. Earlier, it was 100 and 200. Now, 100 came out and 300 is added. This makes us understand that low is here. So, we have to remove the low minus 1. And where is the high now? We have to add it here. This is what we understood. We have to remove 100 and add 300. We will write this in the index tab. So, first of all, remove the low minus 1. First of all, sum is equal to"
    },
    {
      "start": 2139.84,
      "end": 2170.03,
      "text": "Sum minus array of low minus 1. 100 has been removed. 100 has been removed from your 300. Very good. Now what is there? We have to add the high one. But, but, but, it is possible that sometime, sometime, sometime, in any case of the world, your high is here. Your high is here. 3 is here. If you did low plus plus, then low also went ahead. There is no problem. Low minus 1 has been removed. If you did high plus plus, then high is equal to 4."
    },
    {
      "start": 2170.86,
      "end": 2200.4,
      "text": "It is possible. What did we check in while? While high less than n. So when high was checked, it was high 3. So we came to while and did its work. But, but, now we did high++, high came here. Now if you try that sum is equal to sum plus a high, high is 4. There
```

```
is no 4 index in array. So your program will get messed up. Segmentation error will come. So, before adding, to check if we are still in array, if,"
    },
    {
      "start": 2200.91,
      "end": 2232.21,
      "text": "high is equal equal to n. Break it. Do you understand why we checked? Because when high was checked, it was 3. It was a very good thing. It did its work. As soon as high is plus plus, high will become 4. So now you don't have any window. Now it is low and it is outside the high area. So check that high is n. If it is plus, break it. Break it by while looping. We will end the program. Okay. If this has not happened, then you have a window. So you do it. So do what you understand. Very good. You do it. A."
    },
    {
      "start": 2232.69,
      "end": 2264.19,
      "text": "High. Now, what happened? It was 300 minus 100 plus 300. Plus A of high, which would have come to 2. Now, what is the condition of your program? Your low is equal to 1. High is equal to 2. And sum is equal to 500. After which? After this statement. This is the condition of your program. Just close the while loop. See it once. What did we do? What did we do? First of all,"
    },
    {
      "start": 2264.19,
      "end": 2292.42,
      "text": "Find the starting sum by iterating. You will have to do it by iterating. There is no other way. After that, what did we do? We ran the loop of high less than n. Very good. As soon as we came to this window, we did the work of this window. There was a sum in this window. Whatever you have to do, you have to store it in the result. Then we shifted the window. This is the part of shift. These two lines are the shift part."
    },
    {
      "start": 2292.42,
      "end": 2319.44,
      "text": "We increased the low and the high. So, the starting increased from 1 and the ending also increased from 1. So, the size remained the same. Now, here, we have found the information of the new window. And it will continue. Now, when will the information of the new window be useful? It will be useful now. Next time, when this will work, then 500 will be updated here. So, your rest will be cut and become 500. Next time, your sum"
    },
    {
      "start": 2319.44,
      "end": 2349.41,
      "text": "If you do something again, then it will become 700. After cutting the rest, it will become 700. After that, your high will be equal to 4. The program will be break. After that, come out. After that, come out. From my understanding, there can be an edge case. If there is K1, from my understanding, this condition, this condition, this condition, you write this before this line. You write this here. Okay? You write this condition here. Okay? This line, this line,"
    },
    {
      "start": 2349.41,
      "end": 2376.69,
      "text": "bring it below this, change it again why? because if k is equal to 1, then it will break now how will this program break? try running it on k is equal to 1, it will fall ok,  that was a mistake so you increase this and decrease the sum so that we stay safe we have to stay safe, we don't have to take any risk this is done, now when you have done all this then your result will be updated automatically by doing something, right? because you saw right? 100, 200, 200, 300, 300, 400"
    },
    {
      "start": 2376.69,
      "end": 2405.57,
      "text": "See the sum of all, its maximum is stored in the result. That's it, now it's over. Now return the result. Return the result. Okay? Return the result. Did everyone understand this? This is for any sliding window. It's very simple now, but the format will remain the same. This is the format. Format, pattern, whatever you want to call it, template. See this. This is its template. Okay? Now return the result."
    },
    {
      "start": 2405.57,
      "end": 2434.24,
      "text": "What is its time complexity? Its time complexity is order of n. Why? Are you doing the array only once? Are you doing the array only once? What is happening every time? Low is equal to 0 and high is equal to 1. Next time you did low is equal to 1 and high is equal to 2. Then you did low is equal to 2 and high is equal to 3. How long will this run? Until high is equal to n. So how many times did this run?"
```

```
    },
    {
      "start": 2434.24,
      "end": 2463.84,
      "text": "From 1 to n, did n times work? If you haven't watched the time complexity video,
 then go and watch it now. It's a very important video, then you won't understand all this.
Then in the comments, you will shout that we didn't understand. Go and watch that. Did n times
work? In every n times, this is about the loop, this loop, while loop, while loop, did n times
work? Every time, what are you doing? You are not doing any work that is dependent on the
input. Every work is independent from the input, means there are 1-1 works. We are doing plus
minus only. So, it can be said that it is constant."
    },
    {
      "start": 2463.84,
      "end": 2494.32,
      "text": "So overall time complexity is order of n. How much is the overall space? Any
space layer? Any space layer dependent on input? No. Order of 1. This is your space. And this
is your time. Is it correct? So you have learned the solution of fixed window problem. Very
good. You have learned this. You will get the code link. Now we will do one more question. The
video is not over yet. You will get the code link. Go and code it. Okay?"
    },
    {
      "start": 2494.32,
      "end": 2524.74,
      "text": "Let's go. Okay. What's the next question? Do the fixed one. Look at both the
fixed variables today. If you understand these two questions, then the 70% sliding window is
over. Okay. What's the next question? Minimum size sub-array sum. Okay. The name of the
question. What's written in the question? What's written in the question? That you have been
given an array which has positive integers. Very good. And you have been given a target. You
have been given a target variable."
    },
    {
      "start": 2524.74,
      "end": 2554.9,
      "text": "Target variable. Okay. What do you have to find? What do you have to return?
Minimal length of sub-array. Minimal length of any sub-array. Sub-array can be of any size. 2,
3, 4, 5. Minimum length sub-array to find. Whose sum is equal to or slightly greater than
target. Okay. Find this. I mean. There can be many sub-arrays in array. Some sub-array can be
of 2. Some sub-array can be of 3."
    },
    {
      "start": 2554.9,
      "end": 2583.71,
      "text": "it can be of 4, of any length, it has 2, 3, 4, it can have many sub-arrays
whose length is 2, okay, of any length of sub-array, if the sum of any sub-array is more than
or equal to the target, you have to return that length, means if any sub-array of 2 sized
window is more than or equal to the target, you have to return 2, this is your question, first
of all, come to the flow chart,"
    },
    {
      "start": 2583.71,
      "end": 2613.17,
      "text": "Is it an array question? Yes. Is it a sub-array or a sub-string question? Yes,
it is a sub-array question. Do you want to find the combination of minimum, maximum and sum or
greater than or equal to something? Yes, you have to find it. After adding all the keywords,
it is saying that there is a 80% chance that it is a sliding window question. It is not a
graph question, it is not a tree question, it is not a DP question, it is not a greedy
question."
    },
    {
      "start": 2613.17,
      "end": 2638.72,
      "text": "till now what we have read yes it is possible that it is a sliding window so
let's assume that it is possible step 1 if you are watching the video from here then don't
watch it at all go and watch it from the beginning and those who have not reached here then it
is a very good thing step 1 what was it? identify pattern identify it has been identified it
has been identified after reading the question step 2 see this a lot"
    },
    {
      "start": 2638.72,
      "end": 2666.86,
      "text": "The two questions in this video, you must be thinking that if there are two
questions, then both are the same. No, this question is very different from each other. One
question is easy and the other is medium. So, the whole level is more. This question is
difficult for, really speaking. If you are a new kid, if you are doing it with a new idea,
then if you will read only that question, then you will be asked this question and no one will
come. Okay? Why? There is only one reason for that. That this is a window of variable size.
```

```
This is a window of variable size. How did we understand this?"
    },
    {
      "start": 2666.86,
      "end": 2695.06,
      "text": "It is not written anywhere that there is a variable size window. It is not
written, right? Yes. It is not even written in the question that what size is the sub-array.
What size is the sub-array? You can look for it. We are not lying. It is not written in the
question that what size is the sub-array. It is not even written in the question that what
size is the sub-array. No, nothing is written. It is just written sub-array. Minimal length
sub-array. So, whenever it is not written in the question that what size is the sub-array."
    },
    {
      "start": 2695.06,
      "end": 2724.72,
      "text": "Either you don't know, you think it can be any length answer. It can be 2
length answer, 3 length answer, 4 length answer. It is always like this. That variable is a
question of size window. It is not fixed, it is a variable. This makes this question difficult.
 And that's why it is important to do this question. That's why it is important to understand
the question in a different way. Okay. Now let's come to the question. What is it saying in
the question? What is it saying in the question? It is saying that you have given the answer.
1."
    },
    {
      "start": 2724.72,
      "end": 2755.15,
      "text": "2, 4 and 4. 0, 1, 2, 3. Good. You are given a target of 4. You are given a
target of 4. You have to see all the sub-arrays. You have to tell the minimum length sub-array
whose sum, whose elements' sum is more than the target. So, one thing is that the information
of sub-array, what was step 3? The information of sub-array, what is that? Sum. The sum of its
elements. Very good. Now, see."
    },
    {
      "start": 2755.76,
      "end": 2779.5,
      "text": "What is the naive solution of this? In the previous question, we had fixed that
what is the length of subarray? We used to start from here and slide it and take it. Every
time we sum it here and there by doing plus minus and taking out the sum. This question is not
like that. What does this question say? You take it, you see the whole subarray with your mind.
 See the whole one length subarray, see the two, see the three, see the four, see any one."
    },
    {
      "start": 2779.5,
      "end": 2807.25,
      "text": "Any sub-array of any category whose sum is more than target 4, return that
number, that length. So this is the variable. Now, sometimes, this has a different format,
sometimes, in the sub-array of variable length, you can't do this that you make low to 0 and
high to something else like 2 or 3 or somewhere else. Why? Last time, where did we keep high?
You might remember. We kept it at k-1."
    },
    {
      "start": 2807.25,
      "end": 2836.14,
      "text": "Because if there is a 2 length subarray, then there is a subarray of 0 and 1, 1
and 2, 2 and 3. That's why. Whenever there is a question of variable size or variable sliding
window, you always have to start with low is equal to 0 and high is equal to 0. Why? Because
no matter how small the subarray is, it will be of 1 length. There is no empty subarray. There
is an empty set, but not here. It doesn't happen that there is no availment in subarray."
    },
    {
      "start": 2836.14,
      "end": 2864.43,
      "text": "There is one element in the sub-array. There should be one element, right?
That's why sub-array is made. That's why we start small and then see what to do next. At some
point, at some point, in the variable, in the variable, we will keep the low at 0 and the high
at 0. With this, our length of current sub-array will be 1. So, this is your starting position
and starting index."
    },
    {
      "start": 2864.43,
      "end": 2894.03,
      "text": "This is your window, it is of same size. This is your low and this is your high.
 This window starts from here and ends from here. Very good. Now what to do? What is there in
the question? Again, think like a human. Don't think like a computer. Think like a human. Look
at the array. What did I tell you in the array? Find and find the minimum length of the sum of
all the arrays. This is the question."
    },
```

```
    {
      "start": 2894.19,
      "end": 2926.13,
      "text": "You have a company. Let me tell you how the logic is built. The company is
Amazon. Now you have to get Amazon to work. What does Amazon do? You must have heard that
Amazon was fired 2-3 days ago. Why was it fired? Because it was fired. You have to get Amazon
to work. Amazon hired people. Amazon doesn't know how many people it can hire. It has no idea.
Amazon hired people. One person hired. It didn't work. One person wasn't able to do it. It's a
very big company. One more person hired."
    },
    {
      "start": 2926.29,
      "end": 2954.78,
      "text": "She kept hiring like this. Her boss didn't pay attention to this. HR kept
hiring her employees. By doing this, 6 people got hired. Now, work started happening. As soon
as she hired 6 people, work started happening. Very good. So, Amazon could have done one thing.
 She would have said, we need 6 people to work. We need 6 people to work. But, now that 6
people got hired, it is possible that one of them"
    },
    {
      "start": 2954.78,
      "end": 2979.17,
      "text": "Some people are very fast and some are very old. It is possible that not all
people have the same job. It is possible that this person does the work of three people alone.
This is old age. This does not work for anyone. This increases the work that another person
has to hire. Now how will Amazon leave? Now it has been hired. Now, for example, there is
another policy that as soon as you have hired, you will have to fire."
    },
    {
      "start": 2979.17,
      "end": 3007.86,
      "text": "This means that first you hired this person, then this person, then this person.
 You have to fire like this. You don't have to fire from the middle. No. You have to fire this
person, then this person, then this person. You have to do it like this. Now Amazon is feeling
that work is happening. But it is possible that the 6 people, it is possible that they don't
need 6, they need 4. It is possible, right? It is possible that the last person who has come,
if he keeps the power of doing work of 6 alone, then how will he know? If he fires, then only
he will know."
    },
    {
      "start": 3008.02,
      "end": 3035.07,
      "text": "For example, 6 people got hired. It is possible that this work is not done by 6
people. It is possible that this work can be done in 4 or 3 people. How will you know? When
you fire a person, only then you will know. What will Amazon do? What does Amazon do? Amazon
will remove this person. It will say that you go and get fired. If you get fired, then there
are only 5 people in the company. Now look at Amazon, it is still working. Earlier it was
working in 6, now it is working in 5."
    },
    {
      "start": 3035.07,
      "end": 3062.66,
      "text": "Amazon will become more greedy. Amazon will think that it is happening in 5
also. It is happening in 4 also. So save one more person's salary. What will it say? You also
leave. You leave. Now work is happening in 4. Now work is happening in 4. Very good. Amazon
will think that do one thing. You also leave. Now work is happening in 3. Now also work is
happening. Now also work is happening. Earlier in 6. It was happening in 3 also. But as soon
as Amazon fires this person. Fires this person. So total is 2."
    },
    {
      "start": 3062.66,
      "end": 3090.9,
      "text": "Now he knows that now the work is not happening, now the work is not happening,
now the work is getting stuck, now the work is not happening. But now you can't hire the one
who has been fired. So now what will Amazon do? Now Amazon will do the same thing. He will go
to college again, he will hire again. One man hired, the work did not happen. Hired one more,
still the work is not happening. Hired one more man, now the work is happening. Now Amazon has
become comfortable and sat down. Now Amazon has become comfortable and sat down. How many
people are working in the company now?"
    },
    {
      "start": 3090.9,
      "end": 3119.6,
      "text": "Right now, only 5 people are working in the company. Do you understand what is
happening? There is a window, right? It doesn't look like a window. 6 people got hired. Then
they fired slowly until the work was done. As soon as the work was done, more people got hired.
 Then the work started. Then what will they do? Now go, you also leave. You also go. Work is
```

still going on. Go, you also leave. As soon as they fired him, the work stopped again. The work stopped again. The work stopped again. What will they do? They will hire again. This is what Amazon does. This is what we have to do."
    },
    {
      "start": 3119.79,
      "end": 3150.05,
      "text": "We have to do the same thing like Amazon. Do you understand or not? If you don't understand, then let me explain. Now, did you understand the logic and the intuition? Now see, what was coming? What was it? 1, 2, 4, 4. 2, 4, 4. 0, 1. Forget everything. Forget for loop, while loop. We will do it by hand and see what happens. Target is 4. Target is 4. Good. First of all, let's go. There is no one in your company right now."
    },
    {
      "start": 3150.05,
      "end": 3179.17,
      "text": "Now, your low is equal to zero. High is equal to zero. Now, no one is working. So, let's hire. Hire this person. What is your sum? Add this. Earlier, it was zero. Earlier, it was a blue and white work. Add this. One is done. Is one greater than your target? Meaning, is it working now? It is not working. We will hire more. Hire more. Hire this person. Sum is equal to one plus two is equal to"
    },
    {
      "start": 3179.17,
      "end": 3207.98,
      "text": "3. Is 3 greater than 4? No. No problem. Hire again. 5. Sum is equal to 3 plus 4. 7. 7 is greater than 4. It started working. When did it work? When you hired 3 people. So, if any earthquake or storm comes in the world, if the world breaks, 3 will be the answer. It is possible that the answer is 2 or 1, but it will be 3. There is no need to go up to 4, 5, 6. We got 3, right? Imagine if there was an answer of 4."
    },
    {
      "start": 3207.98,
      "end": 3235.07,
      "text": "So 3 is smaller than that. You have to give minimal. Remember minimal. 3 is my answer. Write 3 comfortably at one place. Good. What did Amazon do? Remember. As soon as the work started, it started firing. It is possible that one person will do the whole work. We will fire everyone. Good. Fire it. Fire it. Fire this. Fire this. Come. Update your sum."
    },
    {
      "start": 3235.07,
      "end": 3263.7,
      "text": "What happened if you fired? You took your work and computer and left. Six. Six. Is six still more than four? Yes, it is more. It was happening. It was happening even after firing. It was happening even after six. It was happening even after five. It was happening even after four. It was happening even after three. So as the work continues, now see, how much work is happening now? Now, who is in the company? Only this is in the company. What happened? Earlier there were so many people in the company, so work started. Work started, now they started firing. Come."
    },
    {
      "start": 3263.7,
      "end": 3294.45,
      "text": "As soon as it started working, you started firing. Come here. How much length is the window? 2 length. 2 length. 2 is smaller than 3, right? Now, if anything breaks in the world, 2 will be the answer. Earlier, your answer was 3. 2 is smaller than 3. Write 2 here. Write 2 here. Good. It is still working. Fire more. Fire more. Fire this. Remove this. Update the sum. 6 minus 2 is equal to 4."
    },
    {
      "start": 3294.45,
      "end": 3323.63,
      "text": "Is 4 greater than equal to your target is equal to 4? Yes, it is. It is still working. Why? Because this 4 was working alone. It was also happening there that a person was working for 6 people. Very good. Now what is your length? What is your window? Remove this and remove this. This is what you have saved. This is what you have saved. Do this too. Now what is your length? 1. 1. Whatever breaks in the world, I will have one answer. Write 1 here. You will be comfortable. Very good. Now what did you do?"
    },
    {
      "start": 3323.63,
      "end": 3353.63,
      "text": "Now, Amazon is a fool. He doesn't know that there is only one person here. Let's fire this too. Let's fire this too. Let's do it. Let's fire this too. Come. Sum is equal to 4 minus 4 is equal to 0. I fired it. Now, it is 0. Now, it is 0. Is 0 greater than 4? No. Now, the work is stuck. The work is stuck. What will we do? We will hire. The work is stuck. Come. Let's hire this. Let's hire this. Sum is equal to"

```
    },
    {
      "start": 3353.63,
      "end": 3381.22,
      "text": "0 plus 4 is equal to 4. 4 is bigger than 4. Yes, it is. Where is your window? How many people are hiring? Only this one is hiring. Only this one is hiring. 1. 1 is already written. Okay, no problem. 1 was there before. It is still there. Now, work is starting. Fire it. Fire it. Fire it. Remove it. Fire it. Fire it. 4 minus 4 is equal to 0. Work is there. No, work is not happening. You will have to hire. But,"
    },
    {
      "start": 3381.22,
      "end": 3412.02,
      "text": "Who will you hire? Now it's over. All the kids in college are over. RA is over. Your program is over. This was yours. What is the smallest number in this? 1. Return 1. His answer is 1. Why is his answer 1? Because only one person was working. We don't think that you need to understand anything else. In the beginning, you don't have anything. You have to hire and fire. How long do you have to hire? Once you have fired, you are working till you hire. You are not working till you hire."
    },
    {
      "start": 3412.02,
      "end": 3441.57,
      "text": "Write down the work that is going on and start firing from a distance. Okay? Very good. Now come. Now you have thought about it. Now let's start thinking about it in computer language. What will happen? Good. Low is equal to zero. High is equal to. Yes, it is also written that if, assume, there is no sovereign whose sum is greater than or equal to the target. It is also possible that your target is 1 lakh. No one is able to work. You hired the entire college and it didn't work."
    },
    {
      "start": 3441.57,
      "end": 3472.98,
      "text": "then in that case you have to return 0 in that case if you have to return 0 then it will be done your result in result you can take something like int max one more thing whenever you have to return minimum of anything not only in sliding window but in any programming if you have to return minimum then the starting value of your result is int max means infinity if you have to return maximum of anything in life then the value of your result is int min"
    },
    {
      "start": 3473.78,
      "end": 3500.62,
      "text": "If you want to do minimum, start the result from infinity. If you want to do maximum return, start the result from minus infinity. Why? Because if your first value is 10, what will you do? When you want to do minimum return, you will do this, right? Min, the answer you got now is 10 and rest. So what will happen is that when your result is already infinity, if anything comes from infinity, it will be good, then you will take it."
    },
    {
      "start": 3500.62,
      "end": 3530.54,
      "text": "Similar work, when you want to do maximum, you do this. You do this. Answer, res. So, if anything comes, if 10 comes, then 10, whatever it is, if it is better than minus infinity, then 10 will come. For this, let's do this. Okay. Remember. Okay. So, let's come again. Low is equal to 0. High is equal to 0. Res is equal to, because if you want to return minimum, then int max, which you can understand is infinity in computer language. Okay. Int max means integer which can take maximum value. Okay."
    },
    {
      "start": 3530.54,
      "end": 3561.49,
      "text": "Now, what is the information of every window? What is the information of every window? Sum. In the beginning, how much do you have? I mean, how much is the sum? Or how many people are there? Or how many things are there? Is it zero? It is nothing. Neelbeta Sanna. No one has been hired. Okay? Let's go. Now, now, now. Now, what we were talking about hire-fire. Now, how will this hiring and firing happen? In which direction was the hiring? In which direction was the hiring? The hiring was happening in this direction. It was happening, right? One by one, they were hiring till the right side. How was the fire? Fire was also in this direction. That means,"
    },
    {
      "start": 3561.55,
      "end": 3593.31,
      "text": "This is the first window. When this window used to increase, it means higher. The company is increasing. So the window used to increase like this. When the window used to decrease, it used to decrease from here. The one who was hired first, was fired first. It used
```

```
to decrease from here like this. What does it mean? What does it mean to hire? What does it mean to hire an element? To add the value of that element to the sum. What does it mean to fire an element? To add the value of that element to the sum."
    },
    {
      "start": 3593.31,
      "end": 3620.08,
      "text": "reduce it means and who will we add? we will add new element if new element comes in window we will add it we will remove the old element and we will remove the window we did it in the previous question in the previous question we used to remove 100 we used to enter 300 we used to remove 200 we used to enter 400 so whenever the work of entering or the work of hiring or the work of increasing that is done from high"
    },
    {
      "start": 3620.59,
      "end": 3650.61,
      "text": "High will increase then only it will come. And whatever is the work of firing, whatever is the work of firing or whatever is the work of decreasing, the element which has to go to its village, that happens from the side of low. The same thing happened in the previous also. In this, what was the order? In this, high was plus plus and we used to do something with some area of high. We used to sum or something, we used to take it inside in our window. In this, first we removed the area of low and then did low plus plus."
    },
    {
      "start": 3651.18,
      "end": 3680.53,
      "text": "These two are opposite to each other. This is happening. This is hiring process and this is firing process. We will do the same, right? To hire a person, we will increase the high and we will hire him. If we want to fire, we will fire this person first and then we will increase the low that you are not in the company, the company has decreased. Okay? Now what can we do on this? How will we write the code? Writing its code is a bit tricky. Its code is very beautiful. You may not understand its code in the first time. It is possible that you understood the logic. But if we say now write its code."
    },
    {
      "start": 3680.53,
      "end": 3710.96,
      "text": "We will not get the code to write. Okay? Even we didn't know. So, low is 0 and high is 0. Let's make it every time. 1, 2, 4, 4. Same thing, what we did last time. High. We will work on high. As soon as high is out, the program is over. High is then n. Okay? Now, your sum is 0. Now, your sum is 0. You have nothing. And the constraint of your target is 1."
    },
    {
      "start": 3710.96,
      "end": 3739.87,
      "text": "So, in the beginning, there is nothing. So, we can't fire. So, we have to hire. Hire. Sum is equal to plus area of height. Okay. Now, now, now. What is happening? Once you have hired. Now, what was the process? Check that if work is going on. If work is going on. If work is not going on, then there is no problem. If work is not going on, then there is no chance of firing. Now, the company is not stable."
    },
    {
      "start": 3739.87,
      "end": 3769.42,
      "text": "If it is not working, then you have to hire more, then increase the high and do more work. But if the work is going on, then while reducing the company, find the minimum where the work is going on. Work is going on at 6, very good, write 6 in the answer. It is possible that it is in 5, then try in 5. If it is in 5, then write 5. If it is in 4, then write 4. If it is not in 3, then stay there, now we will hire. This is what happens, right? We can write it like this. While, while, one more while, while in while. The sum, what does work mean?"
    },
    {
      "start": 3769.42,
      "end": 3796.75,
      "text": "Sum is this. See this. What does it mean to work? Is your sum equal to the target or greater? If this happens, if you come here, it means work is happening. Somewhere something is happening. No matter how many people, 100 people, work is happening. So, how many people are working together? At this point, how many people are working together? At this point, how many people are working together? Where is your high? Is your high here?"
    },
    {
      "start": 3797.23,
      "end": 3828.4,
      "text": "That's why we are adding it. We are adding high every time. Your low is also here. Low is also here. Low is 0. So, low is equal to 0. High is equal to 0. It means someone is telling you that my start is equal to 0. My end is also at 0. Tell me its length. How will
```

```
its length come out? Its length will come out as end minus start plus 1. I don't understand. There is a sub-array from 0 to 2. Tell me its length. So, which elements are there in this? 0, 1 and 2."
    },
    {
      "start": 3828.4,
      "end": 3858.27,
      "text": "You will think that we will do 2 minus 0. 2 minus 0 is 2. Its length is 3. What will happen to this? 2 minus 0 plus 1 is equal to 3. This is simple math. We have not thought about this yet. We have simple math. So, first of all, take out this. How many people are there in the company? High minus low plus 1. Very good. Whoever is there, he is doing so much work. Res is equal to minimum of res and length. So, res is infinity."
    },
    {
      "start": 3858.27,
      "end": 3887.82,
      "text": "Now he will know that so many people are working, very good. This is done, this is done, you have seen. Amazon has seen that 6 people are working and wrote 6 in its diary. Now fire, how do we fire? How do we fire? How do we fire? How do we fire? We fire by reducing the low. Sum is equal to sum minus array of low. After reducing, what do we do? Increase the low. The one who is fired is no longer a guard."
    },
    {
      "start": 3887.82,
      "end": 3919.02,
      "text": "Okay? Okay? Okay? Is this enough? What did I do? We will explain again. Don't worry, the code is done. Close the while. Close the while. What happened in this while? First of all, we came to high. We will explain from here. We came here. We didn't have anything here. The company is running now. We don't know how it is. We added high. We hired the high one. If work is going on, this section, this means, till"
    },
    {
      "start": 3919.47,
      "end": 3948.21,
      "text": "Jav tak kaam ho raha hai. Jav tak kaam ho raha hai. While loop hai. Ek baar nahi. Loop hai. If nahi hai. Loop hai. Jav tak kaam ho raha hai. While sum is greater than or equal to target. Tab tak. Tab tak kya karenge? Ek ek karke wo length nikaalenge. Answer me likhte jayenge. Answer me likh lo. And fire bhi karte jayenge. Ek ek karke. Ye firing. Ye firing yaha hua. Ye firing kab rukega? Hosekta hai. Ap jab ek admi ko fire kiye."
    },
    {
      "start": 3948.21,
      "end": 3978.3,
      "text": "Even then, some greater than or equal target will be there. This is what was happening. Your window was made till here. Remember the example. The window was made till here. So, you had written 3 in your answer. Very good. You fired 1. You see that 2 and 4 are still working. Okay, then fire more. Why do you have to do it again and again? Do mass firing once. You see that 2 and 4 are also working. Write 2 in your answer. Fire more. Take out 2. Do this too. It is also working. Okay. Do this too. Write 1 in your answer. Fire this too."
    },
    {
      "start": 3978.3,
      "end": 4008.32,
      "text": "then your low will come here. Now you don't have anything. You have fired all the hires. Now you know that you don't have anything. Okay. When you don't have anything, then this vial will break. It will break automatically. As soon as the work stops, you can't do firing. How much power is there to do firing? The work is happening till you fire, till you shrink the window. As soon as the work stops, the firing stops. And we have done all the answers here and there."
    },
    {
      "start": 4008.32,
      "end": 4038.93,
      "text": "Now, you have come here. Now, what happened? You had hired this till one high. You had hired till one high. Now, whom do you have to hire? The next high. Pay attention. High was here. Low was here. Assume that your high has come here. What is the meaning of coming here? That high has been hired. It is possible that it has been fired. We don't know. You had brought the high till here."
    },
    {
      "start": 4038.93,
      "end": 4066.77,
      "text": "we have brought high this high has been hired for sure maybe it has been fired also it doesn't matter he will see this thing he will see firing manager now high plus plus why? next time hire another high you have hired one high you have done your work with it now who has to hire? next college student next college student high plus plus then increase the high see this is a simple program"
```

```
    },
    {
      "start": 4066.77,
      "end": 4098.69,
      "text": "This is simple, very small but very beautiful and tricky Listen carefully,
repeat it carefully If possible, slow down the speed and listen again How is the code written
and what is the meaning of the code We have explained as much as possible, but still there may
be some doubts, ask in the comments Write it in a different way, this is the simple logic Hire,
 see the work, work till it is happening Work, fire till it is happening"
    },
    {
      "start": 4098.69,
      "end": 4128.77,
      "text": "Increase the higher pointer so that we can do hiding next time. Okay? When you
come here, you don't have to do anything. Return. Result. Return result which we were
calculating here. This one. We will return this. When will we return? When this while loop
breaks. When will this while loop break? When high equals to n. High equals to n means no
child is left to do higher. Your high. Your high entered here. As soon as your high"
    },
    {
      "start": 4128.77,
      "end": 4153.62,
      "text": "your window is broken. You need start and end in window. End is broken. So what
window? Nothing is window. Just return it. That was it. Its time complexity is just like
previous question. Order of end time. No space is taken. Order of one space. This is the
beauty of sliding window. Two questions. According to me, very important questions. Very tough
questions."
    },
    {
      "start": 4153.62,
      "end": 4181.7,
      "text": "And both the questions were made with a lot of time. Go and submit your
questions. There is a link to my submission in the description. First, try to read the
pseudocode and try to complete the code. If you don't understand, then look at mine. There is
no use in looking at mine. We will understand it. You won't understand anything. Okay? Two
questions. The sliding window has been cleared. Similarly, we will increase the speed and keep
reading. Keep making consistency every day. And keep reading every day. If you have doubts,
ask. We will try to reply. Okay? See you."
    }
  ]
}
```


22. transactions/t_3.json


```
{
  "video_id": "-g6h0Ok1Buk",
  "title": "EP 03: Level Order Traversal In Binary Tree | Intuition and Template",
  "language": "en",
  "model": "large-v3",
  "duration": 2730,
  "segments": [
    {
      "start": 0.0,
      "end": 13.82,
      "text": "Pura pasina pasina ho gaya ji, matlab baap re, bhoot lamba video ho gaya hoga,
matlab humko lagta hai ki 30-40 minute ka to ho hi jayega. Is video me hum log jo padhne wale
hai na, one of the most important concept hai, bahut saare reasons hai. Sabse pehle to is
concept ka naam hai level order."
    },
    {
      "start": 13.82,
      "end": 38.19,
      "text": "In the last video, we have taught you some traversals like pre-order, post-
order, and in-order. For example, Dadaji first feeds his left child, then he feeds his right
child. Now you will have to traverse in a different way which is called level order. We will
talk about that later. But this is important because till now you were completely unaware of Q.
 Q data structure. We didn't know about it because Q data structure"
    },
    {
      "start": 38.38,
      "end": 61.47,
      "text": "And the main use is used in tree's level order and graph's BFS. And graph's BFS.
 Means the stand alone question of Q data structure is asked very less in interviews. It
happens, not that it doesn't happen, but it is asked very less in interviews. In interviews,
the stand alone question of stack is asked more. The stand alone question of Q is not that
important. That's why I haven't taught it yet. So in this video, you will also learn Q."
```

```
    },
    {
      "start": 61.47,
      "end": 88.19,
      "text": "You will get an introduction of Q. And you will also learn level order. You
will learn a fixed template of level order first. That you should know the algorithm of level
order. Means that is not a thinking algorithm. You should know that fixed template. Based on
the question, you will do small modifications in that template. You will get all three
questions of level order. There are small modifications in all three questions. Means the
first one is pure level order. After that there is a zigzag. After that there is level order 2.
"
    },
    {
      "start": 88.19,
      "end": 116.8,
      "text": "You can make 3 questions in 2-3 lines of change. For this video, I will give
you an advice that keep some patience. Because you are going to learn something new. Why are
you going to learn? And you are going to learn level order. Whose algorithm is fairly complex
according to me. So when there will be a dry run, don't get impatient in it. That I don't
understand, I don't understand. Give it a little time. Give it a little time of 30 seconds, 1
minute, 2 minutes. Because this is such an algorithm. That when we will start in the beginning.
 So first in 1 minute. Suppose we started dry run. So first in 1 minute."
    },
    {
      "start": 116.8,
      "end": 139.73,
      "text": "You will understand 5%. Honestly, I am telling you. I am not saying that I am a
master god teacher. No. You will understand 5% in 1 minute. In the next 2 minutes, you will
start understanding 50%. In the next 3 minutes, you will start understanding 99%. So, as you
will see, there will be a lot of exponential growth. But in the first 1 minute, you will feel
that you are not understanding anything. What is happening? What is going on? What is coming
out? But look with patience."
    },
    {
      "start": 139.73,
      "end": 167.76,
      "text": "I am requesting you to watch it with patience. You will understand it very well.
 This is my promise. For level order, you will get the most detailed video on the internet. We
have seen how much we have studied for the rest of the people. In 6-7 minutes, we have done it
and it is over. This is going to be the most detailed video for level order and queue. Be
patient and watch it. Let's start quickly. So the first question, the base question is level
order traversal. So you will get the leadcode 102. Why is this question a base question?
Because"
    },
    {
      "start": 167.76,
      "end": 195.84,
      "text": "Again, if you understand this question, it will take some time. We will
understand it very slowly. The other two questions are just small variations. It won't take
much time. You have to give as much effort or time as you have to give in this. What does
level order say? You get a tree. For example, your tree looks like this. Its child is 20, its
child is 9, and 20's child is 4, and its child is 5, and 9's child is 10. This is your tree."
    },
    {
      "start": 195.84,
      "end": 224.62,
      "text": "Till now, we have studied three types of things. Pre-order, in-order, post-
order. In the last video, we had discussed what is pre-order and what is in-order. In that,
what used to happen? That root, its left child, right child, or left child, root, and so on,
something used to happen. What does level order say? If you see this tree, if you look at this
tree very carefully, if you make horizontal lines in it, what does horizontal lines mean?
Straight lines. If we make straight lines in it, then see how it will be made. The first level,
 which is the first level? Of root. So, see this line."
    },
    {
      "start": 224.75,
      "end": 252.78,
      "text": "This is one level. If you look at this company, the highest level is the CEO.
The two people under the CEO, CFO and COO, you can make another level for them. For them, you
can make another level. Suppose this is level 0 and this is level 1. So how many people are
there in level 0? One person, 3 people. How many people are there in level 1? Two people. 20
people are also in level 1 and 9 people are also in level 1."
    },
    {
      "start": 252.78,
      "end": 283.06,
      "text": "You can make another line. You can also make another line. Which will cross
```

```
these three. What is this? This is your level 2. How many nodes are there in level 2? There are three nodes. This, this and this. Now this is called level order traversal. When you make horizontal horizontal lines in a tree. Whichever nodes or whichever circle comes in one line. We call that one level man. For example, if we explain this with another example. So what is at the top? Grandfather. Grandfather has two sons. Papa and?"
    },
    {
      "start": 283.06,
      "end": 308.19,
      "text": "Chacha. Okay? Papa has two sons. Let's say Pratyush and Pranjal. Okay? Chacha has one son which is Mohit. Okay? Okay. If we make a level in this, then only Dadaji will come in the top level. Only Dadaji will come, right? What will happen? Let's say Gen Boomer. Gen Boomer or whatever it is. In the next level, Papa and Chacha will come. They will get something else."
    },
    {
      "start": 308.19,
      "end": 338.19,
      "text": "Tell me, okay. In the last level, Pratyush, Pranjal and Mohit will come. What will they be? Gen Z. So if you make a horizontal line, then whoever is on one level, we call them on one level, that they are on one level. What is the level order traversal? That you don't have to do like this, that you have to take root, its left side, you don't have to do like this. You have to go level by level. So if someone tells you to write its level order traversal, then you can write like this, that suppose you will write first, that first there is a man A3. Okay."
    },
    {
      "start": 338.19,
      "end": 367.28,
      "text": "After that, one person is 20 and 9. After that, on the next level, there are 4, 5, and 10. This is its level order traversal. Why is this level order traversal? Because look, first of all, whatever we wrote in level 0, again, in every level, you have to write left to right, or right to left, or question by question, it depends. But, for example, if someone tells you that write this tree in its level order traversal, or level order, in every level, write left to right. What to write in every level? Left to right. So, because there is only one node in level 0, then you will be able to write only one thing, 3."
    },
    {
      "start": 367.41,
      "end": 392.48,
      "text": "There are two people in level 1. Now left to right means first 20 and then 9. So this is level 1. There are three nodes in level 2. They also have to write left to right. Like this. 4, 5, 10. And a big vector will be made of this. So what has happened to you? This thing, what is this? This is level order traversal. What is it? Level order traversal. Level order traversal. Level by level we are going. What will happen here? Grandfather, father, then uncle."
    },
    {
      "start": 392.48,
      "end": 422.48,
      "text": "And in the next one, Pratyush Pranjal and Mohit. So, what is this? Level order traversal. So, see, any question of level order, any complex question of any kind, that in level order, you can say anything, do left to right, right to left, half left, half right, do anything. The thing that is used in this, if you use recursion in this, then it won't be fun and it won't be possible. Recursion, what you did in the previous one, in order, pre-order, that first we take root, give a chance to the left of root, give a chance to the right of root, it won't be possible. It won't be possible. Why it won't be possible, we will tell you."
    },
    {
      "start": 422.74,
      "end": 442.86,
      "text": "If there is any level order question, you have to use Q. Q. We know that we have not studied Q yet. Q because there is nothing special to study in Q. You can study Q in this. In this, you can also tell Q. What is Q? You have studied stack. We told you stack. We told you stack. What is stack? First in, last out. First in."
    },
    {
      "start": 442.86,
      "end": 468.91,
      "text": "Last out means what was the stack? The stack was a structure like this. Suppose you insert 3 things in it. First you insert 1, 2, 3. So when you insert 1, 1 comes here. When you do 2, 2 comes up here. When you do 3, 3 comes up here. When you take it out, how did it come out? First 3 came out, then 2 came out, and at the end 1 came out. So 1 came in first and came out at the end."
    },
    {
      "start": 468.91,
```

```
      "end": 493.01,
      "text": "2 entered second, 2 came out, 3 entered last but came out first. So this was
the stack. Q is just opposite. What does Q mean? First in, first out. First in, first out.
What does Q mean in the cinema hall? Q means line. What happens in the cinema hall? The person
who is first in the queue gets the ticket first."
    },
    {
      "start": 493.01,
      "end": 520.3,
      "text": "The person who stands in the queue first in a ride gets the ticket first. It
doesn't happen that the person who is in the queue first gets the ticket last. No, it doesn't
happen like this. So, Q means first in, first out. What is Q? You understand Q like this. This
is Q. Now, there are two openings or two holes in it. We call this front. And we call this
back. What do we call this? Back. Okay. Same happens. Now, in Q, if you..."
    },
    {
      "start": 520.3,
      "end": 546.59,
      "text": "If you want to push in the queue, similarly you can push like a stack. You can
say, in the queue, push 1. Push one more in the queue. Push one more in the queue, 2. And push
one more thing, 3. Okay? So what will happen in this? The push, suppose you will push from the
back. Meaning, first of all 1 will be pushed here, so 1 will come here. 2 will be pushed, so 2
will come here and 3 will come here."
    },
    {
      "start": 546.59,
      "end": 575.14,
      "text": "This is done after pushing. So this is what Q looks like. Now if you want to
take it out one by one, then from where will you take it out? In the stack, from where did you
take it out? From top. Because only one was open, top. There are two open on this, front and
back. Now we will take it out from front. Now from where will we take it out? From front. What
will Q.front give you? The first element in front, what do you see? One. So what will you get?
One. What will you get? One. After this you will do Q.pop. After this you will do Q.pop. So
from where did it enter?"
    },
    {
      "start": 575.14,
      "end": 605.3,
      "text": "Push. Where did the push come from? From the back. Where will the pop come
from? From the front. From the front. The front one will pop. Do it again. Q dot front. Q dot
front. What will you get? What is there in the front? Two. Two. You got two. Again. If we do Q
dot pop, then Q dot pop. Where will it pop from? From the front. From the front. Whatever is
there in the front, it will pop. Do it again. Do it again. Q dot front. Where will you get it?
Whatever is there in the front. Three. Three. You got three."
    },
    {
      "start": 605.3,
      "end": 633.07,
      "text": "Do it again. Q dot pop. Where will it pop? From the front. So whatever is in
the front will pop. So see, Q is this. Nothing else. If you do Q dot push, then where does
push come from? From the back. Where does push come from? From the back. So when you pushed 1,
2, 3, then 1 came, 2 came, 3 came. When you remove Q dot front, then the person seen from the
front is accessed first. So what happened in this? The order to push was also 1."
    },
    {
      "start": 633.07,
      "end": 661.5,
      "text": "2 and 3. There is an order to exit. Because you are taking out from the front.
The order to exit is also 1, 2, 3. The order to exit is also 1, 2, 3. And when you pop, where
does it pop from? From the front. Whatever element you see in the front, it will blow the
queue. So that's why the order is maintained in the queue. In the stack, the order was
reversed. 1, 2, 3 would enter and 3, 2, 1 would exit. If 1, 2, 3 enters in the queue, then 1,
2, 3 will only exit. 3, 2, 1 will not exit. That's why queue means straight. Queue means
straight."
    },
    {
      "start": 661.5,
      "end": 687.86,
      "text": "The one who enters first will come out first. First in, first out. So, we hope
that you understand this. So, this was a small introduction of Q. There is not much in Q. SPAC
is a very complex topic. Q, according to me, is not a very complex topic. All the complex
questions of Q are in the sub-tree. It is in the sub-tree. There are very few such questions
that Q is very complex. There is not much in Q to make it complex. This is Q. Okay. Now, let's
come to the main topic."
    },
    {
      "start": 688.02,
```

```
      "end": 712.03,
      "text": "Now, when you know the introduction of Q, I am not saying that you have
mastered Q, but you know what is the introduction, where is the push from? From the back. When
you take out the front, the front one comes. When you do pop, where does it come from? From
the front. So, it means first in, first out. Now, look at this tree. Look at the tree of
Dadaji. You have to make a level order traversal of this. Meaning, how many levels do you see
in this? You will see three levels in this. What does three levels mean? This is one."
    },
    {
      "start": 712.03,
      "end": 739.5,
      "text": "Then 2nd and 3rd. So level order traversal means that you have to make
different types of arrays. You have to make one array in which the upper level is there. Upper
level means in which this is there. In which Dadaji is there. This is 1. Level 1 is over. How
many people are there in level 2? 2 people. Papa and Chacha. So you have to make a separate
array in which Papa and Chacha are there. Very good. Level 3. What is level 3? In which there
are 3 people. Pratyush, Mohit and Suman. So you have to return left to right. Left to right.
Remember to return left to right."
    },
    {
      "start": 739.5,
      "end": 756.4,
      "text": "This is a simple level order. Pratyush and Mohit and Suman. Okay? Now what is
this? Three arrays. Three arrays means three levels are represented. Left to right means. This
one. For whom is this one? Say level 0. For whom is this one? For level 1. And for whom is
this one?"
    },
    {
      "start": 756.4,
      "end": 782.27,
      "text": "For level 2. You have to put all of these in a big array and return it. Meaning,
 if you are in C++, then you have to return vector of, vector of int. If you are in Java, then
you have to make a list of list, whatever is there, 2D array, whatever it is. So, you have to
make different arrays. For every level, you have to make an array, combine all of them in the
last and return it. So, this is the question. Now, see how Q will be used in this. And we will
tell you earlier that this is a very non-obvious algorithm. Meaning, this algorithm is not
like,"
    },
    {
      "start": 782.27,
      "end": 809.81,
      "text": "If you don't know anything about level order, then you can think about it. No,
this is a base algorithm. You have to know this. You should know this. According to this, on
the basis of this, you can solve the questions of the other level order. So the basic one,
look very carefully. We will show you the dry run. It is possible that in the beginning, for
30 seconds or 1 minute, you will feel that what is he doing? Look slowly. It will start making
full sense. When we will reach the last level, it will start making sense that okay, how is it
happening? What is happening? Why is it happening? Everything will be understood."
    },
    {
      "start": 810.03,
      "end": 838.26,
      "text": "So first of all, whatever is the traversal of the level order, make a Q. What
will you make? Q. In which one way it will be front and from here it will be back. Okay? Make
a level Q. Okay? This Q will contain people of any level at any time. How will it do? I will
tell you. Okay. So first of all, what will we do? Assume that you will get root. Root means
this. Dadajiwala is called root. This root will remain. I have told you to tell me the level
order. Okay. If the root is null."
    },
    {
      "start": 838.26,
      "end": 855.82,
      "text": "Root null means there is nothing in the tree. So you don't have to do anything.
Return vector is empty. Return it. Root null means there is nothing in the family. The whole
family is empty. If the whole family is empty, then what will happen? You have to return empty.
 Very good. If root null is not there, then you will start. So what will you start? You will
start doing your work. First of all,"
    },
    {
      "start": 855.98,
      "end": 876.78,
      "text": "What kind of queue is this? What does this queue store? There is a stack of
everything, right? What kind of stack is this? There is an integer stack, there is a character
stack, there is a string stack. In the same way, what kind of queue is this? Is it an integer
queue? Is it a character queue? Is it a queue of something else? So this whole circle, what is
called a node?"
    },
```

```
    {
      "start": 876.78,
      "end": 906.38,
      "text": "which is called a node. We will make a queue of a complete circle. Means while
pushing in this queue, we will push the entire circle. We will push only one node. There will
be data in the node. Data means the number. And there will be a left and a right in the node.
So this queue is a node type queue. Whose type is it? The type of node is Q. Node star. If
there are C++, then star. If there are Java, then node. And we will name it Q. We will name it
Q. So this type of node means we will put the entire circle in it. Okay."
    },
    {
      "start": 906.38,
      "end": 930.14,
      "text": "To start, what will we do? To Dada ji, which is your root, enter the root in
this. Enter the root in this because where will the level order start? From the starting level.
 How many people are there in the starting level? There is only one person, the root. So enter
Dada ji in this. Where will Dada ji enter? He will enter from the back. When you write push,
then he pushes in the back. So what will Dada ji do? Here."
    },
    {
      "start": 930.14,
      "end": 960.56,
      "text": "It will push. Now your grandpa has come here. Okay? He has come. Very good. Now
start looking at this Q. Now start your algorithm. So what will we write? That, what will we
write? While, while Q not empty. Means, till this Q is not empty, till then we will do
something. You don't understand, right? Go and see the whole thing. Try and see. You don't
understand, right? There is no harm in seeing. At last, you will understand everything. While
Q dot not empty. Okay. Now we will start looking. Now you see, what is in your Q at this time?"
    },
    {
      "start": 960.56,
      "end": 989.47,
      "text": "Only Dada Ji. Only Dada Ji means only this level. Only this level. Only this
level which is level 0. All the people of level 0 are in this. How many people should be in
level 0? One person. There is one person in this. There is one person in this. So at any time,
at any time, the size of this queue, the size of this queue, it will tell you that how many
people are in that particular level. So first of all, because there should be only one person
in level 0 and the size of the queue is also one. How many people should be in that level?
There should be one person. So what will we do?"
    },
    {
      "start": 989.47,
      "end": 1019.02,
      "text": "We will find the int level size here. That at this particular level, the one we
want to see, how many people are there in it? So you will know what is the size of q now. What
is the size of q now? How much? 1. Okay. If there is one person in this level, then how much
will be the size of its array? The final array that we will make here, how much will be its
size? It will be 1. The number of people in the level, the size of the array will be the same
to store. We will make a small temporary array here."
    },
    {
      "start": 1019.47,
      "end": 1049.07,
      "text": "vector, int, tmp, and what size will it be? Whatever the level size is. So, you
have made an empty vector. Now, let's start the level order traversal. How many people are
there in this level? One person. Whatever the level size is. Now, there are only one person.
Now, the level size is one. What will we do? We will run the loop so many times. We will run
the loop so many times. Means, we will do while. One more while in this."
    },
    {
      "start": 1049.23,
      "end": 1072.21,
      "text": "Level size minus minus means, what does this mean? This loop will run level
size times. Level size is 1 now, so it will run only once. Okay. Now what do you do? Start
popping. So first of all, we will do a pop. Node star t is equal to q dot pop. Sorry, not q
dot pop. First of all, we will see its front. We will see its front. Front."
    },
    {
      "start": 1072.56,
      "end": 1101.89,
      "text": "After looking at the front, we will do q.pop. Now what will happen with this?
When you look at the front, then in front, which is in front? Grandpa. So what came in your t?
Grandpa. What came in your t? Grandpa. q.pop. Where does q.pop come from? Whatever you get in
front, it will pop it. Whatever you get in front, it will pop it. Now what happened? Your
grandpa has left this q. Your grandpa has left this q. He has left for now. And what has come
in your t? The same grandpa has come. Your grandpa has come in your t."
    },
```

```
    {
      "start": 1101.89,
      "end": 1131.38,
      "text": "t is equal to dada ji. Very good. t is equal to dada ji has come. Now, what
will we do? Because you have to insert left to right, so what will you do? The array that you
have made, tmp, so what will we do? tmp dot push back whatever dada ji's data, whatever dada
ji's data, so whatever t's data. Means whatever data will be there, whatever number will be
there, here dada ji is for example, normally number is there, so t dot data. So in your tmp,
in your tmp,"
    },
    {
      "start": 1131.63,
      "end": 1161.17,
      "text": "What has come now? Suppose Dadaji has come. It has been stored in TMP. Very
good. Is it over? No, it is not over. Now Dadaji, you have done the process. But Dadaji's
children will also be there. Dadaji's children will also be there. So now see how the next
level will be made. See how the next level will be made. Now we will play one more game. If t.
left, means if t.left not equal to null, means this t, this node, this node of Dadaji in t, if
its left child is there, if its left child is there,"
    },
    {
      "start": 1161.17,
      "end": 1186.83,
      "text": "then push it in the queue. Similarly, if the right of t is not equal to null,
which means that it has a right child too, your grandfather or whatever node you have, it has
a right child, then push it in the queue. And close this line loop. Now see what happened.
Look carefully, the whole game has changed."
    },
    {
      "start": 1186.9,
      "end": 1215.78,
      "text": "Let's start. When we came first, Dadaji was in our queue. Our level size was 1.
We made a vector of one size in TMP. We have to run this while level size is minus minus only
once. How many people are there in this level? One person. So we have to run this once.
According to the run, what we did is that we took the front of Q in a node T and popped Q. So
Dadaji popped and Dadaji came in T. We pushed Dadaji in the TMP that you made. So your array
is ready."
    },
    {
      "start": 1215.78,
      "end": 1245.54,
      "text": "Now we will see that if grandpa's left child is there, that means if grandpa's
left child is papa, then push him in the queue. Push him in the queue. Where will the push
happen? In the back. So what was pushed in your queue first? Papa was pushed. Now we will see
that if grandpa's right child is there, that means if grandpa's right child is there, then
also push him in the queue. What else was pushed in the queue? Your uncle was pushed. Nothing
else was pushed. It was over. Grandpa's was the only one. It was over. Now you have come out
of this. Now you have come out of this while. From which while? From this while."
    },
    {
      "start": 1245.54,
      "end": 1274.27,
      "text": "came out of this vial. Now, in your tmp, all the people of this level must have
been stored. See, in your tmp, grandpa had been stored. Which level is going on now? Level 0
was going on. Grandpa of level 0 has been stored in tmp. Now what to do? In your big array, in
your vector of vector array, what do you have to do? In that, what do you have to do? res.
pushback tmp. res.pushback tmp. That's it. And this"
    },
    {
      "start": 1274.27,
      "end": 1291.12,
      "text": "while loop, which is this while loop? This one, while loop, while q.not empty,
we have to close this, we have to close this, now what has come in your rest? In your rest,
dadaji has come, so we make a big vector here, your dadaji has come, your dadaji one has come,
now see, again,"
    },
    {
      "start": 1291.12,
      "end": 1314.22,
      "text": "Now we will come to this while loop. Is your queue empty? No, your queue is not
empty. There are two people in your queue. Earlier there was one in your queue. Now there are
two people in your queue. Papa and Chacha. Papa and Chacha. Coincidentally, see what is this
Papa and Chacha? There are people of this level. There are people of this level. So we had
said that when you remove this level size, then all the people should be in that level. This
queue keeps everyone. So see."
    },
    {
```

```
      "start": 1314.22,
      "end": 1342.59,
      "text": "The work of level 0 is done. But how many people should have been in level 1? 2
people, dad and uncle. Is there dad and uncle in your queue? Yes, there is dad and uncle in
your queue. Why does this queue work like this? Look at another cycle, it will be more clear.
Now, when you make level size, then what will come here? 2, not 1. Now there are dad and uncle,
 so what will be the level size? 2. Now, because there should have been 2 people in this level,
 then how many size temporary arrays will you make? 2 size. So what will be your TMP? 2 size
of 1."
    },
    {
      "start": 1342.59,
      "end": 1366.45,
      "text": "Your TMP is an array of two sizes. And your TMP is an array of two sizes. Now,
how many times will we iterate? Level size minus minus. Level size minus minus means 2 minus
minus. Means we will do this twice. How many times will this while loop work? Now it will work
twice. Because there are two people in this level. Papa and Chacha. Now, what will happen?"
    },
    {
      "start": 1366.45,
      "end": 1395.78,
      "text": "Node T is equal to, now run it, let's run it twice, come. First of all, node T
is equal to Q.front. Q.front means, who is sitting in front? Papa. Papa is sitting in front.
So come, by updating T, what should we write here? Hey, by updating T, what should we write?
Papa. By updating T, we wrote papa. Very good. Q.pop. Q.pop means, from where will it pop? It
will pop from front. Who is in front? Yes, this papa. Remove this. Remove this. Okay. Very
good, removed."
    },
    {
      "start": 1395.78,
      "end": 1424.46,
      "text": "Now, now, now. What will we do? tmp.pushback, t's data. Means, whatever data
papa has, he has to push back in tmp. So, just for understanding, you have pushed back papa in
tmp. Very good. Now, same thing. If papa's left child is not equal to null, if papa's left
child is not equal to null, then why should I push him? But, do you see papa's left child? No,
no, no. No, no, no. No problem, no problem. Do you see papa's right child? Yes, papa's right
child is visible. What is visible? Pratyush."
    },
    {
      "start": 1424.46,
      "end": 1453.07,
      "text": "What do we have to do? Q dot push Pratyush means Q dot push. Where will the
push come from? From the back. So Pratyush got pushed. Pratyush got pushed. Look at this. What
happened to Pratyush? It got pushed. Okay? It got pushed. Very good. Now, how many times has
it run? How many times has this while loop run? Once. It ran for papa. It has to run twice.
Because how many times does it run? Level size. It will run for all the people in that level.
Run it one more time. Do the same thing. T is equal to Q dot front. Who is in front?"
    },
    {
      "start": 1453.07,
      "end": 1480.26,
      "text": "Very good. Very good. TMP.pushback, tika data. Very good. TMP.pushback, tika
data."
    },
    {
      "start": 1480.26,
      "end": 1510.16,
      "text": "Is Chacha's left child not equal to Nal? Yes, Chacha's left child is not equal
to Nal. Is Chacha's left child? Is Chacha's left side? Push him in the queue. Push him in the
queue. Who? Where do you push Mohit? In the back. Where do you push Mohit? In the back. In the
back. Yes. Is Chacha's right child? Yes, Chacha's right child is also there. Suman, where do
you push him? Push him in the back. Where do you push Suman? Push him in the back. Now see,
your queue is looking like this."
    },
    {
      "start": 1510.16,
      "end": 1539.54,
      "text": "The work is done. The uncle has also moved. Done. Has this level size loop
worked twice? Yes, it has worked twice. It worked for papa. It worked for uncle. Very good.
What has been stored in your TMP? See. Papa and uncle. On that level. On this level. We were
playing on this level. Papa and uncle. Both have been stored in this. From where? Left to
right. What will we do? Rest.pushback. TMP. So what else has come in rest? Papa and uncle have
also come. Papa and uncle's array has come. Similarly. Finished."
    },
    {
      "start": 1539.54,
      "end": 1567.44,
```

```
      "text": "Now, whose loop is this? Q is empty. Is Q empty? No, Q is not empty. There are three more people in Q. And, coincidentally, who are the three people in this Q? All the people of the next level will be found in this Q. All the people of the next level will be found in this Q. That's why, that's why, this level size, how many people are there in that level to process? How many people are there in that level to process? This tells you. Do the same thing again. Do the same thing again."
    },
    {
      "start": 1567.54,
      "end": 1596.59,
      "text": "Take out the level size. How much is the size of Q? 3. 3. Means in this level, the level at which we are now, how many people are there to process? 3 people. 3 people. Who? Pratyush, Mohit, Suman. All are in Q. Very good. Vector in TMP. If there are 3 people in this level, then how many size arrays will be made? 3 size arrays will be made. So TMP is made. How many size? 3 size. Do the same thing. Level size minus minus means how many? 3 times this loop will run. 3 times this loop will run."
    },
    {
      "start": 1596.59,
      "end": 1618.06,
      "text": "Come on, run it. Run it for the first time. t is equal to q dot front. Who is the man at the front? Pratyush is at the front. Very good. What did t become? Pratyush. What did t become? Pratyush. And q dot pop. Pop means whatever is at the front. Remove it. Whatever is at the front, remove it. Who is at the front? Pratyush."
    },
    {
      "start": 1618.06,
      "end": 1632.72,
      "text": "very good now what we have to do tmp.pushback tika data tmp.pushback tika data"
    },
    {
      "start": 1632.72,
      "end": 1660.27,
      "text": "Now, do you see any left child of Pratyush? No. Do you see any right child of Pratyush? No. So, Pratyush was deleted, but no push was made because there is no child of Pratyush in the next level. So, no push was made. How many times did level 5 run? Once. It has to run 3 times. What will happen to T? Front of Q. Who is in front of Q now?"
    },
    {
      "start": 1660.27,
      "end": 1689.94,
      "text": "mohit mohit mohit mohit mohit mohit mohit mohit mohit mohit"
    },
    {
      "start": 1689.94,
      "end": 1715.79,
      "text": "Don't push it. Can you see Mohit's right child? No, you can't see it. So don't push it. Nothing happened. Mohit got deleted. Nothing happened. No problem. What was the level size? 3. How many times have you run it? 2 times. Run it for the third time. Run it for the third time. The more people are in that level, the more this level size loop runs. Run it for the third time. Do the same thing. t is equal to q.front. Who is in front? Suman. Suman is in front. So what did t become? t became Suman."
    },
    {
      "start": 1715.89,
      "end": 1740.8,
      "text": "Alright. Q.pop. Pop from the front. What do you see in the front? Pop, pop, pop. Your Q is empty. Now, what do we have to do? tmp.pushback. Suman. tmp.pushback. Suman. So, Suman is pushed. Very good. Do you see Suman's left child? No, no. So, we won't push. Do you see Suman's right child? No, no. So, we won't push. Now, Suman is also popped. No push."
    },
    {
      "start": 1740.8,
      "end": 1763.09,
      "text": "Now it is done. Has this been done three times? Yes, it has been done three times. Remember, how many times? It has been done for Pratyush, for Mohit, and for Suman. Your TMP is ready. It is over. Rest.pushback, TMP. What is there in TMP? These three. So in your rest, another array has come in which Pratyush is there, Mohit is there, and Suman is also there. And Suman. Another array has been pushed."
    },
    {
      "start": 1763.09,
      "end": 1792.82,
      "text": "Now, we come back to this. Is your queue empty? Yes, your queue is empty. See, there is nothing in the queue. When there is nothing in the queue, what does it mean? That all the levels of the family are over. How many levels were there in the family? Three. First, grandpa's generation. Then, papa and chacha's generation. Then, Pratyush, Mohit and Suman's
```

```
generation. So, did we cover all three? Yes, we covered all three. Now, there is no generation
left. So, the work is over. What will we do in the last? Here, return. Return."
    },
    {
      "start": 1792.88,
      "end": 1818.32,
      "text": "Let's see what's in res. First of all, Dadaji's generation, L0. Then Papa and
Chacha, L1. At last, Pratyush Mohit Suman's, L2. So you can see what's happening. How did it
happen? It was very important to explain this by dry-running because you won't be able to
think of this algorithm. This algorithm is the base algorithm. Any problem of level order
follows this. So first of all, we will summarize you again. First of all, what did we do? We
made a queue. What did we push in it?"
    },
    {
      "start": 1818.32,
      "end": 1847.36,
      "text": "Dadaji means root. After that, how long did we loop? Until this queue is empty.
After that, how many people are there at that level? That tells us the size of the queue or
the level size. The more people there are, the more times you have to run this while level
size minus minus and make a TMP in which you have to make an array of that level size. Every
time, every time when this level size runs, you will see the front of the queue, you will pop
the queue. After that,"
    },
    {
      "start": 1847.36,
      "end": 1876.86,
      "text": "He will push whatever has come in his array. Now, if Dadaji has a left son,
then push him in the queue. If there is a right son, then push him. Otherwise, let it be. When
this whole thing is done, then you can add that TMP to your rest. You can add that TMP to your
rest. So, what happens with this? First of all, you see Dadaji. Dadaji pushes two people. Who?
Papa and Chacha. Now, you see Papa and Chacha together. You see both of them. So, when you see
Papa, then what does Papa push in the queue?"
    },
    {
      "start": 1876.86,
      "end": 1906.34,
      "text": "Pratyush. And if we look at Chacha, then what does Chacha push? Mohit and Suman.
 Papa and Chacha look at both. Next time, because there are three kids, there are three kids
in generation 3, so let's look at all three of them together. Now Pratyush pushes who? Nobody.
Who does Mohit push? Nobody. Who does Suman push? Nobody. After that, your queue gets empty.
If, let's say, Mohit had a child, if Mohit had a child, then Mohit would also push someone.
Mohit, let's say,"
    },
    {
      "start": 1906.34,
      "end": 1931.98,
      "text": "Arun would have been pushed here. So you would have run it one more time. Now
how many children would have been there in the last level? Only Arun. Only Arun. Because Mohit
had one child. So how many times would this generation have run? Once. So where would Arun
have run? Arun would have come in your area. But who would Arun have pushed? No one. So then
your queue would have been empty. So when your queue gets empty, you understand that the whole
generation is over. The whole family tree is over. So this level order is done like this.
Level order will remain the same."
    },
    {
      "start": 1931.98,
      "end": 1961.44,
      "text": "Where will the question change? The question will change like this. Now here
you are told to do level order traversal. And in every level, in every level, do left to right.
 Do left to right in every level. What does left to right mean? In the first level, if there
is only one, then do it. In the second level, there are two people. Papa and uncle. Write them
left to right. Papa first, uncle later. In the third level, there are three people. So write
them left to right. Pratyush Mohit Suman. So this is the question. What can you say to this?
Write right to left in every level."
    },
    {
      "start": 1961.44,
      "end": 1989.5,
      "text": "Write right to left in every level. Level 1 will be the same, Dadaji. In level
2, write Chacha and Papa. Write right to left. In level 3, write Suman Mohit Pratyush. Write
right to left. This is one way. One more question can be asked. Zigzag. What does zigzag mean?
Zigzag means, in level 0, write normal left to right. In level 0, Dadaji is alone. So, it
doesn't mean left to right. Level 1 will say, write right to left. In level, write Chacha and
Papa. Chacha and..."
    },
    {
      "start": 1989.5,
```

```
      "end": 2018.5,
      "text": "Write Papa. Write this. In level 2, do it again. Come to left to right. Now write Pratyush, Mohit and Suman. In level 0, Dadaji. Left to right. In level 1, do it again. Right to left. Meaning Chacha and Papa. So this is right to left. In level 2, do it again. Come to left to right. Pratyush, Mohit, Suman. So this is a question. I am not making it up. This is a question of zigzag order. So we are giving you the question of zigzag order in the homework. You will have to do very few code changes. Give it in zigzag homework or make it up."
    },
    {
      "start": 2018.5,
      "end": 2054.08,
      "text": "Let's make it. We hope that you have understood the level order. So here is the level order code. And this is how the level order is. Everything remains the same. Now we will tell you about zigzag. What is the meaning of zigzag? What is the meaning of zigzag? We will tell you. The meaning of zigzag is that, now assume that it is like this. Grandfather, father, uncle. One right child of father is Pratyush. One right side of uncle is also Mohit. Okay?"
    },
    {
      "start": 2054.08,
      "end": 2080.83,
      "text": "Ratyush's left is Suman. Means she is Ratyush's daughter. And Mohit's son is Rohan. This is your family tree. What you have been told is to order at zigzag level. Make its array like this. In the first level, it is Dadaji. So there is no debate in that. Dadaji is alone. So Dadaji will remain. In the second level, write it right to left. Means in the second level, right to left. Means uncle and papa should be there. Uncle and papa should be there."
    },
    {
      "start": 2080.83,
      "end": 2110.35,
      "text": "In the third level, again come to the left to right. Alternate again. It means there should be Pratyusha and Mohit. Pratyusha and Mohit. In the next level, come back to the right to left. Alternate again. Left to right, right to left. Left to right, right to left. In the next level, because you have to write right to left, then do Rohan and Suman. Rohan and Suman. Okay, so what happened? Whose is this? L0. Whose is this? L1. But upside down. Right to left. Whose is this? L2. Left to right. Whose is this? L3. Means right to left. Okay, do this."
    },
    {
      "start": 2110.35,
      "end": 2137.22,
      "text": "We have to do the same thing. One thing will change. What will change? We will take a variable. Everything will be the same. The entire code will be the same. Each and every line will be the same. But, we will take one. How do you have to start? What do you have to do in the first level? In the first level, it is said that starting from left to right. Meaning, in the first level, you do left to right. After that, you keep alternating. We will do one thing. That boolean, we will do a left to right."
    },
    {
      "start": 2137.22,
      "end": 2164.72,
      "text": "Left to right is equal to 1. Left to right means we have to do left to right in this level. We have to do that in the first level. Okay. Same. Q dot push, root. Q dot push, root. Okay. Let's make a Q and show. Make a Q. Make a Q and show. This became Q. So what was pushed in the beginning? Grandpa. What was pushed in the beginning? Grandpa. This is your front. And this is your back. Okay."
    },
    {
      "start": 2164.75,
      "end": 2195.04,
      "text": "Let's run it. So, is Q empty now? No, it is not empty. Come. So, what is the first level size? Q's size. So, the level size is 1. Okay. Let's make a TMP too. TMP's array is made, whose size is 1. In this level, only Dadaji is there, right? Okay. Now, what will we do? We will take two indexes. What will we take? We will take index. Int. First index is equal to 0. Now, for whom is this index? Where to start filling TMP? See, you will know. You will know. Int. We will take one last index, which will be level size."
    },
    {
      "start": 2195.04,
      "end": 2222.48,
      "text": "Minus 1. Level size minus 1. Okay. We will do the same thing. But this line where we have tmp.pushback. This green line. Now what did it do? It automatically did left to right. Left to right. Means when you do this, left to right will work. Everything will work left to right. Okay. So what will we see in this? Going inside, in level size. If left to right is there. If left to right is there."
    },
    {
      "start": 2222.48,
      "end": 2248.94,
```

```
      "text": "equal equal to 1 hai, agar left to right hai, tab to simple chalega, tmp dot
pushback, jo bhi, t ka data, matlab ye to normal chalega, ye normal chalega, lekin else, else,
else, agar right to left karna hai, right to left karna hai, to hum log kya karege, tmp last,
iska last index is equal to t dot data, t dot data, and, and, and, last,"
    },
    {
      "start": 2248.94,
      "end": 2277.7,
      "text": "Minus minus and last minus minus. We also take this on first. We don't push
back. If left to right is 1, then what will we do? That tmp first is equal to t.data and first
plus plus. See what happened. See what happened. You will think, oh my god, what have they
written? They haven't written anything. They haven't written anything. Let me tell you. Let's
hope that in code, there are changes in code. Let me tell you how to derive it."
    },
    {
      "start": 2277.7,
      "end": 2307.2,
      "text": "In the very beginning, let's make our Q, okay? Let's make our Q in which Dadaji
will be there. Dadaji will be there, okay? First of all, what will be the level size? 1 will
be there. How many size of array will TMP be made? 1 size of array will be made. First, first
index means, what is the first index of TMP? It is 0. Last index, where will be the last index
of this TMP? Level size minus 1, because no matter how many size of this vector it is, then 1
smaller than that, so its last will also be 0. Okay."
    },
    {
      "start": 2307.2,
      "end": 2335.9,
      "text": "Now, when we fill in this, what will happen? What will be your level size? Once,
 your T will become Dadaji. Your T will become Dadaji and your Dadaji will fly away from here.
Dadaji has flown away from here. Now, what is left to right? Now, in the beginning, in the
first level, left to right is true. Left to right is true or 1? It is 1, right? It is 1. So,
if it is 1, then it is straight. 1 means to fill in normally. So, what will we do? TMP first."
    },
    {
      "start": 2335.9,
      "end": 2361.6,
      "text": "tmp first is equal to dada ji. tmp first is equal to dada ji and first plus
plus. First plus plus. So, what is first? Zero. So, what will be tmp? dada ji. tmp zero is
equal to dada ji and first is done. So, one is done in this. So, in your rest, the first array
is stored, dada ji. It is not so clear in this because it is only one. See in the next. But,
but, in the next level, in the next level, what do you have to do? Alternate. Now, it was left
to right one. Means, left to right had to be done."
    },
    {
      "start": 2361.6,
      "end": 2391.92,
      "text": "In the next level, you have to make left to right equal to 0. So, when this
level size loop ends here, after this, what will we do? Left to right is equal to 1 minus left
to right. 1 minus left to right. This will swap left to right. How does this work? This is a
small trick. If this left to right is 1, which it is, then when we write this, left to right
is equal to 1 minus left to right, then what will be left to right?"
    },
    {
      "start": 2391.92,
      "end": 2420.94,
      "text": "What will be left to right? 1 minus 1, 0. See, it's swapped. If this left to
right was 0 from the beginning, if this left to right was 0 from the beginning, and we say
that left to right is equal to 1 minus left to right, then what would be your left to right
new one? 1. Because what is 1 minus 0? 1. So, this is how it is swapped. You can swap any
binary thing like this. If you write left to right is equal to 1 minus left to right, then if
it was 1 before, now it will be 0. If it was 0 before, now it will be 1. So, this is swapped."
    },
    {
      "start": 2421.01,
      "end": 2450.74,
      "text": "So next time, when we come to the next level of Dada Ji, what will happen in
the queue? What will be stored in the queue first? Papa and Chacha. Because we will do it.
Because we will do it. I am just showing you how it will work. Now come, we have to do the
same thing. What will be your level size? 2. What will be your level size? 2. What will be
your TMP size array? 2 size. Because there are 2 people. 2 size. 0 and 1. Okay. First is 0.
Last is equal to level size minus 1. What is 2 minus 1? 1."
    },
    {
      "start": 2450.74,
      "end": 2477.15,
      "text": "What is 2 minus 1? 1. Okay? Come. First of all, now it has to run the level
size twice. So first of all, what will be stored in t? Papa. What will be stored in t? Papa.
```

```
And pop papa. Now, now we will see that if left to right is 1, if left to right is 1, then now left to right is not 1, now left to right is 0 because we alternate every time. We used to do this on left to right 1. But if left to right is 0, then what will we do? tmp"
    },
    {
      "start": 2477.15,
      "end": 2502.21,
      "text": "We will start filling it with the last index. Last index is equal to t.data. Meaning, what is last? 1. Meaning, we will put papa in tmp's 1. And we will make the last minus minus. We will make the last minus minus. Now, what has become last? Last has become 0. Now, if this level size was 2, then it will work twice. Now, what will become t? Chacha. What will become t? Chacha. What has become t? Chacha. And chacha flew away from here."
    },
    {
      "start": 2502.21,
      "end": 2528.83,
      "text": "Again, we will see if left to right is 1, which is not, now it has become 0. Left to right 0 means you have to fill right to left. We will do the same thing. We will fill t data at the end of tmp. We will fill t data at the end of tmp. What has become last? 0. So we have filled its data at the end of tmp. And last is minus minus, so now last has become minus 1. And your level size was 2, so this is over. See what array has been made here. Uncle and papa. Not papa and uncle, uncle and papa. Because we are filling the array from last."
    },
    {
      "start": 2528.83,
      "end": 2548.13,
      "text": "The only way to fill the array is to decide left to right or right to left. In Q, it was the same. In Q, first there was papa and then there was chacha. Now if you fill TMP from the beginning, then obviously papa and chacha will come. If you start filling TMP from the end, then chacha and papa will come. So this is just a trick. This is just a trick that how did you fill TMP? That is the trick."
    },
    {
      "start": 2548.13,
      "end": 2576.7,
      "text": "If you want, then you can also make a normal TMP. And if that level is not left to right, but right to left, then reverse the TMP. Meaning, you make normal, like you made in the previous one, then you make it of papa and chacha only. And if you get to know that it is right to left, which you will get to know from this boolean, then you reverse it. Then it will become chacha and papa, but it will take a little more time. It is not that good. You can do better than that, that fill the TMP. If left to right is equal to true, then fill it first."
    },
    {
      "start": 2576.7,
      "end": 2592.86,
      "text": "If left to right is equal to false, then fill it with last. There is no rocket science in this. It is a small trick. So this is the problem of level order. Same program will be there, same structure. There are few things. How are you filling it? How are you filling TMP? There will be some changes in that."
    },
    {
      "start": 2592.86,
      "end": 2618.67,
      "text": "So, please go and do these two questions. Level order is the most important concept in trees. And in the future, it also works in graphs and BFS. So, please understand this well. If you don't understand, then see. But we don't think that someone has not understood. According to me, I am sweating a lot. After seeing the dry run, you must be able to understand that yes, this happens at every level. How do you fill TMP? That depends on question to question. Left to right, do it like this."
    },
    {
      "start": 2618.86,
      "end": 2648.21,
      "text": "Suppose the question is that you have to do right to left in every level. So this last one, you do this. You don't need to check anything. You have to fill right to left in every level. So fill it like this every time. You have to fill left to right in every level.  Fill it like this in every level. You have to fill alternate. So see what is the rule in that level. Fill it according to that. So you get to know from this. So the video has become a little long. But it was necessary. It was necessary to explain this much. If we had explained it like this, then it would not have been understood. It would have been understood by dry run.  So please, this is your code."
    },
    {
      "start": 2648.27,
      "end": 2678.75,
      "text": "This is your normal level order code. What will be the change in this? Instead of this line, instead of this line, what will you write? If left to right, if you want to fill
```

```
left to right, then how will you fill it? Put it in the first of TMP. Put the data of T in the first. And make the first plus plus. If this is not the case, if this is not the case, then put it in the last of TMP. If you want to fill right to left, sorry, you put it in the last of it. Put the data of T."
    },
    {
      "start": 2678.75,
      "end": 2708.29,
      "text": "And minus the last one. Meaning? Start from the back. If you fill from the back, then automatically it will be right to left. In Q, the store is left to right only. In Q, how are all the children of every level storing? Left to right. Meaning? In Q, we are storing normally. In Q, we first stored Dada. After that, we stored Papa and Chacha. Meaning, we stored left to right only. After that, we stored Pratyush, Mohit and Suman. So, in Q, it is normally stored. How are you filling in TMP? He will decide whether you will fill left to right or right to left. Did you understand? Get into your mind."
    },
    {
      "start": 2708.29,
      "end": 2728.3,
      "text": "Solve both the questions. We say solve level order 2 as well. There are three questions. One is level order, one is exact level order, and one is level order 2. We say solve all three questions and show us how much material you have. We will meet tomorrow. Let's see. It is the season of Holi. We will meet tomorrow or day after tomorrow. Let's go."
    }
  ]
}
```


23. transactions/t_4.json


```
{
  "video_id": "-gC-QEdpvO4",
  "title": "What If You Could Master Recursion in 3 Easy Questions",
  "language": "en",
  "model": "large-v3",
  "duration": 1818,
  "segments": [
    {
      "start": 0.0,
      "end": 11.44,
      "text": "Today we will do 3 questions in recursion. All 3 questions are very easy. So you will feel that there are very easy questions in recursion. But it is important to do easy questions because you have been doing all these questions in loops till now."
    },
    {
      "start": 11.44,
      "end": 30.69,
      "text": "Today we will do all three questions. Sum of digits, remove all occurrence and check of error is sorted. So all three are different types of problems. One is the problem of number, i.e. how to do number recursion. How to do an array recursion and how to do a string recursion. So all three problems we are sure that you will have done or will do. If you get time, then we will do it from loop."
    },
    {
      "start": 30.69,
      "end": 45.2,
      "text": "but in loop you will not be able to learn recursion so we want to make this easy problem because the first type of recursion in which you used to take the answer of small problems and then return it there should be no problem in that because when we will go to DP"
    },
    {
      "start": 45.2,
      "end": 64.94,
      "text": "or when we go to the tree, then all this will be very useful. If there is confusion in the tree that how to get the small answer and how to return, then nothing will be made in the tree. 50% problems will not be made in the DP also. So listen very carefully. Same with when we will start the problem of backtracking from tomorrow, watch very carefully if you do not understand backtracking."
    },
    {
      "start": 64.98,
      "end": 89.68,
      "text": "it can't be written, then what will happen is that the graph will not be understood in the future. The DFS in the graph is the same. In the future, 40% problems will not be understood in front of DP. Because until the recursion comes, all these problems, all these things do not come. That's why people don't know. Because they don't learn recursion. They see the problem, they think that yes, we will run a loop in this, why do we have to do
```

```
recursion in this? Learn it. The purpose is to teach, the purpose is to teach you."
    },
    {
      "start": 89.68,
      "end": 119.22,
      "text": "I am not an expert in loops, I have done a lot in loops, I have seen a lot of
loops till now. Recursion is a technique that you should know, you should know how to read
recursion, you should know how to dry run. So, let's start quickly. Come, let's start. So, it
is a very easy question, we will solve it very quickly. What is the first question? In the
first question, you have to find the sum of digits. You have given a number n, and your job is
to return the sum of the digits in this n."
    },
    {
      "start": 119.34,
      "end": 138.37,
      "text": "We know that you have done this. How have you done this? You used to do y, n
greater than 0. How is any number's digit taken out? It is taken out by d is equal to n
modulus 10. So when you do n modulus 10, then its last digit is taken out. For example, if n
is equal to 5, 6, 7,"
    },
    {
      "start": 138.37,
      "end": 167.73,
      "text": "and you will take out its modulus, modulus means that when you divide it by 10,
then how much will be left, how much remainder will be left, so when you divide 5, 6, 7 by 10,
then how much remainder will be left, remainder will be 7, so this is the last digit of it,
okay, now you have taken out this last digit, now how do we separate this last digit from the
rest of the numbers, there is a way to separate it, if you take its quotient, quotient means,
remember when you were studying division, then there were two types of things, what happens,
yes,"
    },
    {
      "start": 167.73,
      "end": 196.74,
      "text": "There were two types of things. We used to divide by 10 like this. So first we
will do 10, 5, 6, 7. So we will do this. After that, we used to call this remainder. And what
we used to call this person? Quotient. Do you remember? There must have been division in 3 and
4. So, there is a simple trick. First of all, if you want to extract any number's digit, the
last one, then you do n-modulus 10. You will get the last digit. Now what will we do to
separate the last digit from the other number?"
    },
    {
      "start": 196.74,
      "end": 218.45,
      "text": "n is equal to n by 10. So, when you do 567 by 10, if you take out the quotient,
then it will be 56. So, you see, the last one digit has automatically come out. So, in this
way, we solve it in the loop. But, let's solve it with recursion. Okay. Do the same thing.
Assume that you have an n, which is 567. Is it correct? Okay."
    },
    {
      "start": 219.02,
      "end": 246.4,
      "text": "What we say is that always keep a small part separate. Keep the rest of the
part in your mind that it will be solved. Combine both and get the answer. So what will we do?
Because we can only get the last digit. It is a little difficult to get the first digit. Or we
don't know how to get it. We didn't get it till now. So we can get the last digit of any
number at any time. And we can also separate the rest. Means we can separate these two by
getting it out at any time. So do the same. What does Dikarjan say?"
    },
    {
      "start": 246.4,
      "end": 273.54,
      "text": "Take out a small part of it. Take out 7. Take out 7. The rest of the part's
answer, 56. Whatever number is left, 56. Its sum of digits, assume that someone will tell you
that its sum of digits is 11. Assume. If you add this 11 and this 7, then can you tell your
answer 18? Yes, we can tell 18. So, this condition is fulfilled. You can apply the recursion.
Listen again. Take out a small part for yourself. You will know its answer. The rest of the..."
    },
    {
      "start": 273.54,
      "end": 301.33,
      "text": "Keep the trust that it will be solved. Keep the rest of the trust that it will
be solved. By combining both of them, tell your answer. Write the essay of one page. Think of
the rest of the five pages, someone else will write it. If you combine it in total, then you
will have a six-page essay that you can submit. And write the base case. What can happen on
the smallest input? Meaning, if n is the smallest, then what will happen? Write that base case.
 So, here it is said that n is a positive integer. n is not negative because then it..."
```

```
    },
    {
      "start": 301.33,
      "end": 327.55,
      "text": "If there is a minus, it will be a mess. What is n? It is a positive integer.
Come, what will we do? First of all, int fun. Okay, it takes int. What is its work? To return
the sum of digits of any n. First of all, write the base case. If n is a positive integer,
then n, what can be the smallest? n can be 0. Means, here 1 is given, but assume that n can
also be 0. Let's assume that if,"
    },
    {
      "start": 327.55,
      "end": 352.06,
      "text": "n equal to zero. If you get your number as zero, then what do you return? How
many digits are there in zero? One digit. Which one? Zero. What is its sum? Zero. So, everyone
knows this. Return zero. Now, come to us. First of all, remove your small part. Int d is equal
to n modulus 10. You removed the last small part. Very good. After that, remove the rest. n is
equal to"
    },
    {
      "start": 352.06,
      "end": 382.16,
      "text": "N by 10. The rest of the part is different. The rest of the part is in N. You
have the digit. Take out the answer of the rest of the part using this function. Have faith in
God that he will give you the function. Int. Answer is equal to fun N. On whom is this? On
trust. You will get it. Now, you had 567. You are trusting the answer of 56 that you will get
it. You have already taken out 7. So, what will be the sum of digits? The sum of digits that
you have taken out is D. D plus."
    },
    {
      "start": 382.16,
      "end": 412.08,
      "text": "Answer, return this. Return this. That's it. The program is over. The program
of recursion is very cute. It is a very cute program. So again, what is this? Base case. What
is this? The small part you took out. And what is this? Trust. You took out the answer by
combining both. Come, let's see its dry run. What is happening? Let's see its dry run. Assume
that you are solving at 687. n is equal to 687. So who will be called by the first main
function?"
    },
    {
      "start": 412.08,
      "end": 441.55,
      "text": "Fun, in which n will be 687. Okay, let's run. Is this a base case? Means, has n
become 0? No, n has not become 0. So, okay, we have to move ahead. Remove d. d, what will be
the last digit? 687 modulus 10, means 7. What will be the remainder? 7 will come. What will we
make n? n is equal to n by 10. Means, what will be the quotient of 687 by 10? 68, so that the
last digit comes out of the equation. 68. For whom will we remove the answer? Fun of 68. Let's
go."
    },
    {
      "start": 441.55,
      "end": 470.64,
      "text": "Now, if this is called, then the compiler will go straight here. For whom is
the second fun called? For n is equal to 68. Again, is this base case hitting? Has it become
0? No, it has not become 0. Do the same thing. Now, if we will stamp 68 modules, then what
will be the remainder? It will be 8. What will be n? 68 by 10. So, the last digit will be
removed. What will be n? 6. And, we will leave the answer of this 6, by the grace of God, that
we will get it from here. Again, if we come here, then again, the compiler will come here."
    },
    {
      "start": 470.64,
      "end": 497.01,
      "text": "What will be the fun call for? N is equal to 6. Is it hitting the base case? No,
 it is not hitting the base case at all. So, D will come. What will be 6 modulus 10? If we run
6 from 10, what will be the remainder? 6. And what will happen? When you do 6 by 10, what will
be the quotient? 0. 1 will not come. Think about division in your mind. N is equal to 0. For
this, take out the answer. Fun of 0. Now, what have you done? The compiler will come here."
    },
    {
      "start": 497.01,
      "end": 522.37,
      "text": "Fun of 0. Does this fit the base case? Look carefully at the code. Yes, this
fits the base case. If this n becomes 0, then what do we say? What will be its answer? 0. So
what will it return from here? It returns 0 from here. It is finished. This one is finished.
Where will it go straight? Here. Now what does it do? It has to combine these two to return
the answer. What was the answer returning? D plus whatever answer has come from there. So what
will we do?"
```

```
    },
    {
      "start": 522.37,
      "end": 549.78,
      "text": "What was d? 6. What is the answer? 0. So what will it return? 6 plus 0. Return
6. This is over. Where will this 6 come? It will come here. Do the same thing. 8 plus 6. What
will it return? 8 plus 6. Return 14. This is over. Where will this 14 come? It will come here.
Do the same thing. 7 plus 14. What will it return? 21. Return 21. What will be the answer in
your main? 21."
    },
    {
      "start": 549.78,
      "end": 578.26,
      "text": "Let's cross-check it. 6, 87. If you add it, the answer will be 21. 6 plus 8, 14,
 21. Yes, my answer will be 21. So, this also worked. That's why recursion programs are quite
cute generally. So, this was the first question. Let's move on to the next one. The next
question is quite fake. Check if an array is sorted. Check if an array is sorted. So, you have
given an array. You just have to tell true or false. If the array is sorted, then you have to
say that yes, it is sorted. Yes."
    },
    {
      "start": 578.26,
      "end": 606.51,
      "text": "If the array is unsorted, then you have to say that it is not sorted as false.
It is a very easy question and the difficulty is also easy. But the purpose of doing this is
to learn recursion and see what happens and how to solve all the problems. So let's think a
little about what we will do. If an array like this is given to you, a very big array like
this, 10, 20, 30, and 40, and 50. So looking from above, it seems that the array is sorted. It
seems that it is sorted, okay. But looking from here, it seems that it may be a very big array.
"
    },
    {
      "start": 606.51,
      "end": 627.3,
      "text": "So how does a person think? If someone tells you a very big array and asks you
if it is sorted or not, then what will you do? We will do this, we will start reading it from
here. We will start reading from 10. If we see that 10 is smaller than 20, then we will go
ahead. If we already know, suppose 10 is here and 5 is here. If the array starts like this,
let's assume that 10 and 5 are ahead."
    },
    {
      "start": 627.3,
      "end": 649.94,
      "text": "No matter how big the array is, no matter how beautiful the array is, we know
that 10 and 5 should not be in this order, because 10 is bigger than 5. We will not go ahead,
we will say that it is not sorted. Any person looks like this, that first look at 10, look at
10 from 20, if 10 is smaller than 20, then it is fine, move ahead. Then pick up 20, look at 20
from 30, if 20 is smaller than 30, then it is fine till now."
    },
    {
      "start": 649.94,
      "end": 677.18,
      "text": "pick up 30, look at 40 and say yes 30-40 is a good thing pick up 40, look at 50
and say yes it is a good thing then you come to the end of the array and say yes the whole
array is sorted if you had ever come to know in the journey that suppose here is 30 and here
is 20 like this now no matter what you have done in the back and what is in front if you have
come to know here only that this is 30-20, it should not have been 30-20 30 is bigger than 20
and bigger than the one in front so if any element any element"
    },
    {
      "start": 677.18,
      "end": 706.34,
      "text": "If it is bigger than its next element, then it is not virus-sorted. If all such
rules are followed, then it is virus-sorted. No one looks at it this way. If you are given a
list of 1,000 children, then you will compare it in this way. You are not a scanner that you
will look at it and tell. No, you can't tell. If you imagine that you are given a list of 1,
000 children, 10,000 children, you will look at it this way with your eyes. Let's make the
recursion in the same way. What will we do? Let's take an array."
    },
    {
      "start": 706.34,
      "end": 733.68,
      "text": "Let's take a big array of 10, 20, 30, 40, and 35. So, from the top, we know
that the array is not sorted. Okay, good. But, what did we say? What does the recursion say?
Again, keep a small part. Solve a small part and keep it. Trust the rest. Trust the rest that
you will get the answer. Mix both and give your answer. Okay."
    },
```

```
    {
      "start": 733.68,
      "end": 763.22,
      "text": "So, when you get this array in the beginning, then we say that let's do one
thing. Let's keep this with us. Let's keep this with us. And first of all, check whether the
length of the array, again, base case. What will be the base case? Base case means what can be
the smallest array? What can be the smallest input? So, the smallest input can be that if the
length of the array becomes 0, if there is no array, if there is no array, what will happen to
it? If the length is 0, then is it sorted? Yes, in a way, we can say that it is sorted."
    },
    {
      "start": 763.22,
      "end": 789.94,
      "text": "It can be said in a way that it is sorted. If the length of an array is 1, then
is it sorted? Yes, it is absolutely sorted. If an array is 10, then there is no other element
in this array, then it is absolutely sorted. Yes, it is absolutely sorted. In the base case,
we have written that if the length of an array is 0, it can be said in a way that it is sorted.
 If the length of an array is 1, then also it is sorted. Okay. Now, here we have come to the
conclusion that what to do with the length of an array 5. Okay, the base case is ready. Solve
a small problem."
    },
    {
      "start": 790.26,
      "end": 819.9,
      "text": "Every time, when you were looking from above with your eyes, what were you
doing? Every time, what were you doing with the small things? You were comparing any element
with the one in front of it. You were doing this, right? Every time, remember, what were you
doing? You were not looking at 10,000 children at once. You were doing small, small, small.
This one is smaller than the next one. Yes, we will move on. The next one is smaller than the
next one. Yes, we will move on. So, this is the small, small problem. Solve it here once. This
problem, the small problem, whether one element is bigger than the other or not, this is very
easy. So, assume that you are standing here. Here, on 10."
    },
    {
      "start": 819.9,
      "end": 846.02,
      "text": "Solve a small problem. Is 10 smaller than 20? Ask this question. If the answer
to the question is no, if it is 5, then there is no need to do anything. You can directly say
that it is false. If you get an element which is bigger than the previous one, then whatever
the array is, it cannot be sorted. You can do false from here. But,"
    },
    {
      "start": 846.02,
      "end": 875.44,
      "text": "Assume that here, what did you find out? That 10 is less than 20. So this is
fine. You solved for this. Now, for which array did you solve for? For this whole array. For
whom? For this whole array. Assume that someone tells you that this array is sorted. Which
array is this? This array. From 20 to 35. Assume that someone tells you that it is sorted.
Assume that someone tells you that it is sorted. So you will stand in which position? See. You
are saying,"
    },
    {
      "start": 875.44,
      "end": 905.23,
      "text": "You are saying that 10 is smaller than your previous one. It is correct till
here. Let someone tell you that the whole array is sorted. The whole array is sorted from 20.
That means after 20, 30, 40, 50, 60, 70, 80. It is completely sorted. So won't this whole
array be sorted? If 10 is smaller than 20, and someone told you that it is completely sorted
from here to here, then the whole array is sorted. Are you understanding this now? 10 is
smaller than 20. 10 is smaller than 20. Okay?"
    },
    {
      "start": 905.23,
      "end": 933.71,
      "text": "and if someone tells you that this whole array is sorted, if someone tells you
that 20 to 80 is true, if someone tells you that this whole array is sorted, won't the whole
array be sorted again? Yes, it will be sorted. And in the same case, if it is smaller than 10
to 20, but if someone tells you that the next array is false, the next array is false means
that 40 has also come somewhere in the front, then the next array is not sorted. So should you
return true or false? You should return false because"
    },
    {
      "start": 933.97,
      "end": 963.01,
      "text": "10 is smaller than 20, this is known, but the array from 20 to 80 is not sorted,
 so can the whole array be sorted at any time? It cannot be, why? Because it is not there, if
someone tells you that it is not there, then it is not sorted. We hope that you have
```

understood this. We hope that you have understood this. If we explain this by giving one more example, then you are doing this by saying that you have a lot of parts, you have a lot of parts, okay? Or there is a lot of food, there is a lot of food, so you"
    },
    {
      "start": 963.01,
      "end": 976.13,
      "text": "Let's check one particular dish. Let's check one particular dish. Like we checked whether 10 is our next choice or not. So you get to know that this is good. But you get to know that the other dishes, one of them is bad or some other is bad."
    },
    {
      "start": 976.13,
      "end": 1005.52,
      "text": "So is the whole party good or bad? You will say that it is bad because you have got a bad one in front. And if you check it that it is good and it tells you that the other one is also good, the other 100 dishes are also good. If someone asks you to tell the whole party, then you will say that the whole party is also good because we checked one, the other 99 people are saying that it is correct, then it will be correct. So this is its logic. What will we do? We will see it with a small example. See. 10, 20 and 30. 0, 1, 2. Is it right?"
    },
    {
      "start": 1005.52,
      "end": 1028.72,
      "text": "First of all, we will make a function in which we will pass an index. What index are we looking at now? In the beginning, you have to see from 0. You have to tell the whole array whether it is sorted or not. In the beginning, we will call i is equal to 0. What was the base case? If the array you are looking at, what does it mean? i is equal to 0. You are looking from i to the end."
    },
    {
      "start": 1028.72,
      "end": 1056.1,
      "text": "From i to the end means you are looking from 0 to the end. So what will be the length of the array? There are many ways to find it out. Either you pass n also. Pass array size is equal to n. You can do this. Pass n. We like to pass n. Personally, we like to pass n. n is equal to 3. Now when will the length of the array be 0? The length of the array will be 0 when i is equal to n. When i is equal to 3, then how many arrays will be left from i is equal to 3 to the end?"
    },
    {
      "start": 1056.1,
      "end": 1085.36,
      "text": "Now, how many arrays will be left from here? 0 length array. So, i equal equal to n means 0 length array. And i equal equal to n minus 1 means how many length arrays are you seeing? 1 length means you are just seeing this index till the end. So, in the base case, we can prepare from this only that if i equal equal to n or i equal equal to n minus 1. What did we think in the base case? If array length is 0, then is it sorted or not? Yes, it is sorted. If array length is 1, then is it sorted or not? Yes, it is sorted. Return true."
    },
    {
      "start": 1085.42,
      "end": 1112.85,
      "text": "No need to do anything else. Return true. Okay. Now, how will we solve a small part? Mix the array of i with the next one. So, array of i, if array of i is greater than array of i plus 1, means, this 10 you are seeing, suppose it is bigger than the next one, then do you need to do anything else? No, because it is stuck here. This array cannot be sorted. What will we do immediately? Return false."
    },
    {
      "start": 1113.36,
      "end": 1134.22,
      "text": "Now, if you know that the array of i is less than i plus 1, i.e. less than 10 to 20, it means that it is still correct. Now, what did we say? Ask the answer of the rest of the array from somewhere. How will we ask? What were you doing in the beginning? Solve i is equal to 0. If someone tells you the answer of i is equal to 1, then return it. Now, everything depends on whom?"
    },
    {
      "start": 1134.22,
      "end": 1161.58,
      "text": "Because 10 is less than 20. So this is correct. I is equal to 0 is good. But if the rest of the array is sorted from I is equal to 1, then the whole array is sorted. If the rest of the array is unsorted from 1, then the whole array is unsorted. So now everything depends on the answer of the next one. What are we saying about the answer of the next one? Trust me, this function will give you. What will you return? Return fun of I plus 1 and n. What will this give you?"

```
    },
    {
      "start": 1161.58,
      "end": 1189.15,
      "text": "i plus 1 send the answer. Who did you come to remove? i is equal to 0. Who are
you asking? Tell me the answer of i is equal to 1 send the answer. We will return it. So, this
is your small programming. What kind of function will it be? Boolean type. So, this is its
code. Let's see how it works. See its code. Let's show how it works. Let's assume this is your
array. We are taking an array that is not sorted. So, what will be called first? Fun i is
equal to 0."
    },
    {
      "start": 1189.15,
      "end": 1219.41,
      "text": "Is it fitting the base case? Is it IN or IN-1? No, it is not. So it is not
fitting the base case. First of all, what will we check? Array of 0, is it greater than array
of 1? Is it greater than 10, 20? No, no. Okay, we are safe. If it was, it would have been
false at this time. Okay, we are safe. Now what will we do? Whatever the answer of fun I is
equal to 1, we will return it by closing our eyes. Come, let's come to fun I is equal to 1.
Okay. Let's see what will happen at fun I is equal to 1."
    },
    {
      "start": 1219.41,
      "end": 1245.39,
      "text": "20, is this a base case? No, this is not a base case. Is the array of 1 bigger
than the array of 2? Yes, yes, yes. 20 is greater than 5, 20 is greater than 5. What will
happen? Is there any need to think further? No, false. And this is over. Where will this false
go straight? Here. What will you return? False. So is this array false? Yes, this array is not
sorted."
    },
    {
      "start": 1245.39,
      "end": 1275.97,
      "text": "If it was 50, what would it return? No. Where will we go? We will go ahead. i
is equal to 2. As soon as you reach i is equal to 2, is it hitting the base case? Yes, it is
hitting the base case. What is i equal to? It is n minus 1. So if we just look at this one
element, is it sorted? Yes, it is sorted. Return true. Where does this true go?"
    },
    {
      "start": 1275.97,
      "end": 1302.54,
      "text": "It would come here. Where would this true come? It would come here. Where would
this go? In the main answer. I said, is it sorted? Yes, it is sorted. So, this is how the code
works. Look at this. So, it becomes very easy. Once you start understanding the recursion,
then you will start clicking that yes, it is fine. Means, you have to remember 2-3 things that
in the base case, a small problem should be solved and the result of the rest of the problem
and this small answer should be combined to make the final answer. So, this is the second
question in the clean."
    },
    {
      "start": 1302.61,
      "end": 1331.78,
      "text": "The next question is, remove all occurrences of a character in string. This is
also a very easy question. What does this question say? You will get a string. Suppose you get
an apple word. And you will get a character. Suppose you get p. You have to remove p from this
string and return the string. What is p? One is p and the other is a. So what will we return?
a, l, p. It is a simple question. There is no doubt in this. How do we solve this? Let's take
a string and loop it."
    },
    {
      "start": 1331.78,
      "end": 1361.01,
      "text": "If this character doesn't match, then add it to it. If this character matches,
then don't add it. It's simple with loop, but do it with recursion also. Learn recursion. Try
to make your base in recursion. So simple. You understood the question, right? There is one
character. Remove all of it from that string. If it is the whole character, this word, then
nothing. Return empty string. Remove all of it from that occurrence. Come, let's see again.
First of all, let's think of base case. Let's think of base case. Okay."
    },
    {
      "start": 1361.58,
      "end": 1390.62,
      "text": "What can be the smallest string you can get? The smallest string you can get is
length is equal to 0. Meaning, you can get an empty string. You can get an empty string. Okay,
if it is an empty string, then what should you return? Now, in an empty string, you are told
that suppose this is C. You have to remove this. So, there is nothing in an empty string,
right? So, what will you remove? Nothing. You have to return this. You have to return this.
```

```
You have to return this. Very good? Okay. Your base case is ready. Okay? Now, let's come to
the rest."
    },
    {
      "start": 1390.62,
      "end": 1418.7,
      "text": "Suppose there is a very big string. P and A, T, E, N, B, Z, anything. Very big
string. Okay? This is a very big string. The character you have to remove is P. That is P.
Okay. What did we say? Start looking from the beginning. As soon as you know from the
beginning, start looking. Come. Here they have stood. They have stood on this. Okay? Okay."
    },
    {
      "start": 1419.25,
      "end": 1449.12,
      "text": "First of all, what will we do? When we thought of the loop solution, we told
you in the beginning how to solve it. Let's do loop. If this character is at P, then we will
not add it to the answer, we will move ahead. If this character is not at P, then we will add
it to the answer. Okay. So, let's do one thing. Let's do one thing. Make an answer. Make a
final answer to return. That my final answer is this. My final answer is this. It will remain
empty in the beginning. Okay. Now you concentrate on this. We said separate this. Separate
this. Concentrate on this."
    },
    {
      "start": 1449.12,
      "end": 1479.55,
      "text": "Should you add it in the answer? No, you should not. Why? Because this P
matches your character, so you have to remove it. So you will not add it. Now, the rest of the
string, which is a very big string, we don't know how big it is, what it is, how complex it is.
 If someone tells you the answer of the rest of the string, then its answer is a small answer.
Anything can happen, this can happen to anyone. If there is no P in it, the answer means, what
is the correct answer in the rest of the string? If there is no P in the rest of the string,
then it tells you something."
    },
    {
      "start": 1479.55,
      "end": 1510.02,
      "text": "If you do answer plus small answer, then will you get the result? Yes, you will
get it. Listen again to what we are saying. Your job is to remove p from any string. Suppose
my character is p. If you come to any index, initially you are standing on 0 index. If this
index is not p, you can think like this. This is the rest of yours. Trust that you will get it.
"
    },
    {
      "start": 1510.02,
      "end": 1539.54,
      "text": "Suppose you got this from somewhere. You got this in the small answer. Now, if
this string on which you are standing, this character, if it is not P, then what should be the
answer of the whole string? P plus small answer. This should be it, right? Sorry. Whatever
your string is, whatever your SI is. If it is not this, then it should be this. If this SI is
the character that you have to remove, then what should be your answer? Only small answer.
Only small answer."
    },
    {
      "start": 1539.54,
      "end": 1565.28,
      "text": "This is a logical thing, right? Look, we are standing on this. If this is P, we
have to remove P, then we don't need this. If someone tells us the rest of the answer, then we
will print it and give it. If this is not P, then we need it in our answer. We will add this
to the rest of the answer and return our answer. Meaning, you are writing like this, you have
to write the answer of 6 pages. You are sitting with one page in front. 5 pages have to come
from somewhere else. 5 pages have to come from somewhere else. Now the teacher tells you,"
    },
    {
      "start": 1565.28,
      "end": 1595.1,
      "text": "that you don't have to submit this one page, your handwriting is bad in one
page, assume that you have got good handwriting in 5 pages, teacher says that your handwriting
is bad, your handwriting will not be submitted, then what will you do? you will submit this 5
pages, if teacher says that your handwriting is also good, then what will you do? you will
submit 1 plus 5, 6 pages, this is what you have to do, come let's do this, this is what you
have to do in every recursion, this is the same funda, this is the same formula, print it in
your mind, come let's write a function,"
    },
    {
      "start": 1595.1,
      "end": 1624.69,
      "text": "Which gives us a string. Okay. What will it take? First of all, your input
```

```
string. Okay. The length of input string. An index. Again, it is an array concept. This is the index. We are looking at the last. And your character. Which you have to remove. First of all, write the base case. If i equal to n. Meaning, string is finished. Meaning, there is a string of 0 length left. What do we have to return? Empty string. Empty string. There is no need to use any brain in this. That's it. Now,"
    },
    {
      "start": 1624.69,
      "end": 1652.58,
      "text": "Take out the answer of the remaining part. How to take out the answer of the remaining part? Fun, s, n, i plus 1, c. Assume that it will give you the correct answer. In this answer, this c is already removed. This is a very good answer. Now, you have got 5 pages. If your si is equal to c, means the character you wanted to remove, you are on it now."
    },
    {
      "start": 1652.58,
      "end": 1680.66,
      "text": "So what will you return? The rest of what you have got, the rest of what you have got, what else will you do? Your handwriting is bad, your one page is useless, the rest of the five pages will be returned, that will happen. Otherwise, what will happen? Add them all. Add these two. Return SI plus answer. Close this. Come, let's dry-nut this. See, this is its code. Again, what is this? Base case. What is this? The rest of the trust answer. What is this?"
    },
    {
      "start": 1680.66,
      "end": 1710.82,
      "text": "the small step that we take and mix the answer and return it. This is it. Come, let's do dry learn. Let's assume that what is your string? A, P and E. 0, 1, 2. Whom do you have to remove? P. Come. In the beginning, fun, i is equal to 0. Is this a base case? No, it is not a base case. When will it be a base case? When i is equal to 3, then it will be a base case. Come. First of all, find the answer. Whose? Find the answer from i is equal to 1 till the end."
    },
    {
      "start": 1710.82,
      "end": 1739.09,
      "text": "Call this Fun i is equal to 1 Is this a base case? No First of all, what do you do? Fun i is equal to 2 Take out the answer and sit down Come out This will be called i is equal to 2 Is this a base case? No First of all, what do you do? Fun i is equal to 3 Take out the answer and sit down When you go to i is equal to 3, the base case will be done What is in the base case? Empty string So its answer didn't come It came empty string Now you have to see here"
    },
    {
      "start": 1739.09,
      "end": 1768.45,
      "text": "Is this e, on which you are, 1, i is equal to 2, is this e equal to your p? No, it is not equal to p. So what will be the answer? e plus answer. So what will this return? This string will return in which e will be. Good thing. Where will this go? It will come here. Now, is this i1 equal to your character, which you wanted to remove? Yes, p and p are equal. So we will not add this. What will we return? Just the answer that came earlier. Return e. Where will this e come?"
    },
    {
      "start": 1768.45,
      "end": 1792.4,
      "text": "This will come here. Check i is equal to 0. Did you want to remove a? No, we wanted to remove p. So this is a good string. What will be the return? Return si plus answer means a plus e is equal to ae. This is your final answer. Look carefully. a, p, e. Remove p from this. What will be the answer? ae. So is the right answer coming? Yes, the right answer is coming."
    },
    {
      "start": 1792.4,
      "end": 1817.46,
      "text": "So this is its code. So see these 3 problems. In 3 problems, every problem will take 10 minutes. We have explained a lot, so it is there. In writing, see how many lines are there. 1, 2, 3, 4, 5, 6. Okay? 6, 7, 8 lines will be of this recursion. And the purpose of doing easy-easy problem is to teach you so that in backtracking and dynamic programming, there is no confusion that how to do recursion. So let's meet tomorrow."
    }
  ]
}
```


24. transactions/t_5.json

```
{
  "video_id": "0Kxg0LPGwFo",
  "title": "One Video To Solve All Binary Search Interview Questions | DSA Patterns 2026",
  "language": "en",
  "model": "large-v3",
  "duration": 2260,
  "segments": [
    {
      "start": 0.0,
      "end": 14.32,
      "text": "First of all, it's my happy birthday. We received a lot of wishes. Thank you, thank you everyone. We are 25 years old now. And we had said that the pattern will be finished by the birthday. But due to some delays, it couldn't be done. But I think whatever has happened is very good. And it will be done very soon."
    },
    {
      "start": 14.32,
      "end": 43.86,
      "text": "Today's lecture is the most important of binary search because when people get stuck in binary search, then when they come to the problem of story, you will not be asked in any interview binary search that there is a number, search it and tell me. No, that was the concept. You always have story problems like Coco, aggressive cow. Today in this question, Coco will explain you so well that you will be able to solve all the sheet problems. You will not have to watch that video again for any problem. So listen carefully to Coco's problem because it is a very famous problem and it is a base problem to understand this story problem."
    },
    {
      "start": 43.86,
      "end": 73.34,
      "text": "So in this video, we will not just explain you how to solve this problem, no. We will explain why this is of binary search, why every story problem is of binary search, and how all the things will be solved from the same template. It is exactly one template, every problem has one template. Learn one template, it will happen that you are filling it with value. So listen very carefully, very carefully till the end. If you understand, then don't skip the video. Listen to this video carefully, you don't have to watch the next 5 videos. Do you understand?"
    },
    {
      "start": 73.34,
      "end": 102.06,
      "text": "And again, thank you and let's start the video. Come, first of all we will explain you the question in Hindi. And my target is that you understand this question. You also understand that what is the question of binary search? How to recognize such questions? Because we don't want you to remember that these 5 things are of binary search. No, many questions are possible. We want you to understand that what is this question? How to recognize it? And how to solve it? It will be the same template of everything. So first of all, we will tell you the question in Hindi. So it will be given like a story."
    },
    {
      "start": 102.19,
      "end": 124.3,
      "text": "First of all, I have told you that there is a bear. What is the name of the bear? Coco. It is a female bear. So, its name is Coco. Now, Coco has some bananas. You can see that there are bananas here. And this banana is not kept alone. This banana is kept in a group. Somewhere there are 3, somewhere there are 5, somewhere there are 7. Anything can happen. Now, what can Coco choose? She can choose her K. K means eating speed."
    },
    {
      "start": 124.3,
      "end": 154.02,
      "text": "What? What is K? Eating speed means, how many bananas will she eat in an hour? Coco can decide that herself. We have to eat 3 bananas in an hour or 10 bananas in an hour. Coco will decide that. Okay. What will happen after deciding? Look. For example, Coco decides that my K is 4. It means that we can eat 4 bananas in an hour. So what will happen? In the first hour, let's assume that she came here. She came to this one where 3 bananas are kept. So look, now we can eat 4 in an hour. Here there are 3. So what will she do? She will eat these 3. After eating these 3,"
    },
    {
      "start": 154.02,
      "end": 180.18,
      "text": "It will make it zero. But, its speed of making 1 is still remaining, but it will be wasted. It won't happen that after eating 3, it will move to the 5th one. No. If it is less than the speed, then it will eat the whole banana and stay there for 1 hour. So, its 1 hour will be wasted here. 1 hour means it will occupy here and eat these 3 bananas. When its speed is 4. Okay, it will come here, on 5 bananas. So, see. The banana is 5. How much can it eat in 1 hour? 4. So, what will it do in the first hour?"
```

```
    },
    {
      "start": 180.18,
      "end": 206.38,
      "text": "In the first hour, she will eat 4 out of 5 because she can eat 4. Now 1 banana
will be saved here. Next time, next time, what will she do in the next hour? In the next hour,
she will just make 1 to 0. Means she will eat 1 banana and will stop there. That hour was
wasted. She will not go anywhere to eat 3 more in that hour. So how much did she take here?
Here she took 2 hours to eat. Okay. What will she do if she comes at 7? In the first hour, in
the first hour, she will eat 4 at 7 and make it 3. Okay."
    },
    {
      "start": 206.38,
      "end": 230.53,
      "text": "What will she do in the next hour? What will she do in the next hour? And for
her, if she goes on one pile, then she will eat the whole thing. Because wasting is not like
this, right? That you eat one pile, there are 3 left in it, you have increased somewhere else,
then there is nothing like that. No, no, that is not her target. If she goes on one pile, then
she believes that she will finish it. Now, in the next hour, there are 3 left here. What is
its speed? 4 kilos to eat. So, she will eat all 3 out of 3. And it will finish here too. So,
here also it took 2 hours. So, how much time did it take in total?"
    },
    {
      "start": 230.53,
      "end": 257.71,
      "text": "Here it took 1 hour to eat 3 meals, it took 2 hours to eat 5 meals, it took 2
hours to eat 7 meals, so in total it took 5 hours. So you can see that as you choose K, then
according to K, your hour will increase or decrease. Means the speed you choose, it may take
more hours or less hours, according to that it will increase. Okay, this is the story. What do
you have to do? There are some guards there, not just Coco."
    },
    {
      "start": 257.71,
      "end": 276.93,
      "text": "What is there? Some guards are sitting there, okay? Guards are sitting there to
guard the banana. Now guard, guard has gone out to drink water or tea. Guard will come back in
H hours. Guard will come back in H hours. H hours means something will be given to H. For
example, guard has said that we will come back in 5 hours."
    },
    {
      "start": 276.93,
      "end": 306.86,
      "text": "We are going to have tea or do some work. We will be back in 5 hours. Meaning,
Coco has only 5 hours to eat this banana. Whoever is giving you H, H will be giving you that
the guards have gone out and will be back in this time. So, Coco has only 5 hours to eat the
banana. Okay, this is also done. Okay. Now, what do you have to do? What is the question you
have to do? You have to tell the minimum value of K. Try to remember. Minimum value of K. What
is K? Eating speed. Not its maximum. Its minimum value"
    },
    {
      "start": 306.86,
      "end": 332.0,
      "text": "So that, so that, Coco can finish all bananas. Finish all bananas. Means all
the piles should be finished. All the piles should be finished. All bananas. And its minimum
value of K. Because Coco likes to eat slowly. Coco doesn't like to eat in a hurry. So
obviously, you have to keep some speed. But Coco wants to..."
    },
    {
      "start": 332.0,
      "end": 359.76,
      "text": "It would have been better if my work would have been done at the minimum speed.
So, this is the premise of the question. So, what do you have to return? Minimum value of K.
Minimum value of K. So, see. Now, this is the question. Now, why is this the question of
binary search? Remember, in the first class of binary search, we told you some rules. First of
all, we told you that whatever happens to you, you have to optimize something or the other.
First of all, you have to find a single thing. Single thing or single value. Here, what do you
have to find?"
    },
    {
      "start": 359.76,
      "end": 386.83,
      "text": "Speed. Minimum value of K. What is eating speed? It is a single thing. You have
to optimize it. It means either you have to minimize it. Whatever you are finding. Or you have
to maximize it. Generally. You have to optimize it. You have to minimize it. Okay. After that
what did we say? If you have to find this. You can guess it. If you can guess it. Guess what
you are finding."
    },
```

```
    {
      "start": 386.83,
      "end": 393.9,
      "text": "Do trial and error. Let's see if my work is done or not. And the answer of that
guest means"
    },
    {
      "start": 393.9,
      "end": 423.47,
      "text": "Now whatever you have been told to find, whatever you have been told to return,
you have been told to return the minimum speed of k, what was said in the binary search
question? Return the index where this is present. So we used to guess the index. So whatever
you have been told, you have to guess that. The answer to that guess, the answer to that guess
means that answer is correct, that answer is wrong, that answer is a little wrong, what is
that, yes or no? You should know this in order of 1 or you should know in order of n in some
cases. Order of 1 means in binary search you would know immediately that"
    },
    {
      "start": 423.47,
      "end": 452.45,
      "text": "Is that number on the guess or not? Array of guess equal equal to target. It
went straight ahead. In some cases, you can get order of end time. It is allowed. Okay. And
whatever your answer will be, whatever your answer will be, means, suppose you write all
possible guesses and write all the answers side by side. Write all the answers side by side.
So, what should this be? It should be monotonic. What did monotonic mean? If your answer,
suppose, is coming in yes or no, then it should have been yes, I,"
    },
    {
      "start": 452.45,
      "end": 476.54,
      "text": "yes comes, yes comes, yes comes, suddenly no comes at one place, and then no
comes, and then no comes. It should not be a mixer match that two yes is coming, one no is
coming, two, it should not be like this. And in 99% of cases, you have to find this turning
point only. Remember the example of fan, in the fan regulator, you have to find the turning
point only. Okay. So what is this question? It fits better in all of us, let's see."
    },
    {
      "start": 476.54,
      "end": 500.82,
      "text": "Do you want to find a single thing? Yes, you want to find a minimum eating
speed, which means a single thing. Do you want to optimize the speed? Yes, we want to minimize
the speed. It's not like tell us any speed. No, tell us the minimum speed by which it can work.
 What is the work here? Finish all bananas. Finish all bananas is your condition. Finish all
bananas is your condition."
    },
    {
      "start": 500.82,
      "end": 524.86,
      "text": "Can you guess? Yes, I mean you can guess the speed. You can say anything like
eating feed. Here we guessed. Here we said 4. So what was 4? 4 was my guess. Yes, guess. We
will see what happens. So you can guess. Okay. Do you get the answer of guess in order of 1 or
order of n? Let's see. I don't know. We will have to see. And is your answer monotonic? Let's
see. Let's see. Okay. Come. Now,"
    },
    {
      "start": 524.86,
      "end": 548.62,
      "text": "Those who have understood, we will encourage you to pause the video and make it
yourself, and those who haven't understood, let's see. So remember that in the example of the
fan, there was something like this, that if you remember the example of the fan, then what was
there? That find the minimum speed of the regulator, find the minimum speed, so that it
doesn't get hot, first thing, so that it doesn't get hot, and the bill also comes as low as
possible."
    },
    {
      "start": 548.62,
      "end": 562.22,
      "text": "Bill is also less than less. So why was the bill less than less? Because you
had to define the minimum speed. In minimum speed, the bill will come less than less. But how
should that minimum speed be? In which you don't feel hot. In which you don't feel hot. So
this is also something like this. Tell me the minimum eating speed."
    },
    {
      "start": 562.22,
      "end": 592.02,
      "text": "Minimum eating speed. Now minimum eating speed will say something like I will
keep the eating speed at zero. I will keep it at one. One can be the minimum speed. You can
```

```
eat only one banana. You don't have to eat half a banana. You will eat one banana. If you eat zero, then it will never end. So someone can say that minimum is one. But there is also a condition in it. That you will have to eat all the bananas. You have to eat all the bananas. And in how many hours? In H hours. You have to eat all the bananas in H hours. It is not that you have to delay. No. So that is a condition. That is a condition. That is how you have to minimize. Let's see."
    },
    {
      "start": 592.02,
      "end": 622.29,
      "text": "Assume that you have a pile like this. You have a pile of 3, 6, 7, and 11. 0, 1, 2, 3. Okay? This is your pile. It is a banana pile. And you have been told that in how many hours will the guards come? In 8 hours. Meaning, how many hours do you have to eat? You have 8 hours to eat. Okay. Now what do you have to tell? You have to tell the minimum eating speed. Okay. Okay, so think with your own mind that what can be the minimum eating speed of any person? What can be the minimum speed?"
    },
    {
      "start": 622.29,
      "end": 651.52,
      "text": "Now, any person's minimum speed will be 1. You will eat 1 banana in an hour, right? In an hour, if Coco says that we will eat 0 banana. First of all, there is no need to eat 1 banana in an hour. There is speed, right? Speed is not negative, first thing. Okay. Now, there is no need to eat 0 banana in an hour. Because if you eat 0, then you will not be able to eat, then nothing will happen. So, according to me, your minimum speed, minimum possible speed should be 1. That you will have to eat 1 banana per hour to think anything. Okay. High."
    },
    {
      "start": 651.52,
      "end": 669.23,
      "text": "How much can you eat a maximum banana in an hour? How much can you eat a maximum banana in an hour? Now see, remember what is Coco's behavior? If, if, let's say she goes somewhere, she goes somewhere here, X goes to any one pile, you can go to any one pile in an hour, okay? Let's say there is a banana here, okay."
    },
    {
      "start": 669.33,
      "end": 698.29,
      "text": "If its speed is less than X, then it will do that first it will eat that much, then it will eat the remaining amount and will waste that much. If its speed, suppose X is equal to 5 here, if you also change the speed of cocoa to 10, that we can eat 10 bananas in an hour, then its 1 hour is going to be wasted here because it will eat 5 in an hour and make it zero. But even then it will waste 1 hour here. It will not happen that if you had 5 bananas, then you will get some special benefit from doing 10 speed, you will not get any special benefit. In this pile,"
    },
    {
      "start": 698.48,
      "end": 724.77,
      "text": "In this pile, you can see that 11, where is the most banana kept? It is kept in 11. If you had chosen cocoa speed 11, if you had chosen cocoa speed 11, if you had chosen cocoa's highest speed, highest speed, let's assume that we can eat 11 bananas, then only it will take 4 hours minimum, then only it will take 4 hours minimum. Why? See, it will come here, how much is 3 here? Its speed is 11. All three will be eaten in how many hours? In 1 hour. All three will be eaten in how many hours?"
    },
    {
      "start": 724.77,
      "end": 749.23,
      "text": "Here it will be eaten in 1 hour. It will come on 6. How much is there on 6? All 3 will be eaten in 1 hour. See, you have kept its ring speed at 11. But it is not getting any benefit. It is 6. It will stay there after eating 6. It will come on 7. It will be eaten in 1 hour. It will come on 11. It will be eaten in 1 hour. So, how much is the total? The number of piles are there, it will take that much time. You can see that if there are 4 piles, then it will take you 4 hours. Why?"
    },
    {
      "start": 749.23,
      "end": 765.68,
      "text": "Because when it goes on a pile, it will eat it. If your speed is more than this, then the speed is very high, there is no problem. It will eat the banana and will waste 1 hour there. So it will take you at least 4 hours to know your speed. Now, if you increase your speed more than 11, someone will say that we will keep our speed 1100."
    },
    {
      "start": 765.68,
      "end": 794.45,
```

```
      "text": "you can keep it, it will still take 4 hours, why? 3 is there, your speed is
1100, your speed is 1200, your speed is 2000, it doesn't matter at all, you will have to waste
1 hour there, by this we understand that the maximum element of the array, the maximum of the
piles, in which most of the bananas are kept alone, if you choose that speed, then in 4 hours,
4 means whatever n is there, the size of your array, the piles that are kept, you will
definitely get that, so we understand that, what can we keep highest, we can keep the highest
of the maximum array,"
    },
    {
      "start": 794.45,
      "end": 822.59,
      "text": "That's it. Very good. Okay? Now, what did we have to do in binary search? We
had to guess. What did we have to do? We had to guess. Come on. Let's guess. Guess with me.
Come on, let's see. Let's guess. We have to find two things. Now, what do we have to guess?
What do we have to do? What is this? This is eating speed. Again, we have to say that whatever
we have to find, we have to guess on that. That we will do this, we will do that. This is
guess. What do we have to guess? We will guess whatever we have to find. Till now, we were
finding index. Till now, we were finding index in every question. Index was getting guessed."
    },
    {
      "start": 822.59,
      "end": 848.62,
      "text": "Now what do you have to find? Minimum speed. So guess the speed. Okay. Come,
let's guess. We say that randomly, whatever is in my mind, we will do that. We say 3. Coco
will think that let's try with 3 that we will eat 3 bananas in 1 hour. We will eat 3 bananas
in 1 hour. Come, let's see how much time it will take. So your banana is kept like this. How?
3, 6, 7, 11. Right?"
    },
    {
      "start": 848.62,
      "end": 874.94,
      "text": "Your speed is 3. Let's see how long it will take. And yes, remember that the
guard will come in 8 hours. The guard will come in 8 hours. If it will take more than 8 hours,
then you will not be able to eat. So that's it. You have to eat before the guard comes. Eat
within 8 hours, then it will be done. Otherwise, you will not be able to eat. Okay. Let's see
how long it will take. What is your guess? 3. Means you can eat 3 in 1 hour. First of all, it
will come here. So here it is 3. Very good. So it will eat 3 immediately and make it 0. Here
it will take 1 hour. 6."
    },
    {
      "start": 874.94,
      "end": 895.3,
      "text": "How many hours will it take for 6? See, in the first hour, in the first hour,
it will eat 6 and make it 3. But it is still left. It can't go yet. It is still left to make
it 3. In the next hour, it will eat this 3 and make it 0. So, it took 2 hours here. It took 2
hours here. 7. How long will it take for 7? If your eating speed is 3, you have 7 bananas,
then how long will it take? See, in the first hour,"
    },
    {
      "start": 895.3,
      "end": 921.92,
      "text": "If she can eat 3 out of 7, then she will do 4. If she can eat 3 out of 4 in the
2nd hour, then she will do 1. And in the 3rd hour, she will eat 1 and do 0. But she will not
go further, her hour is wasted. If there is one banana left, then your hour is wasted. 11. Oh
man. In the first hour, she will eat 11 and do 8. After that, she will eat 8 and do 5. After
that, she will take another hour, in which she will eat 5 and do 2. And..."
    },
    {
      "start": 921.92,
      "end": 951.28,
      "text": "It will take one more hour to eat two and get zero. So until it eats the whole
banana, the pile doesn't move forward. Let's count how much time it will take. How much time
did it take here? One hour. Here, two hours. Here, three hours. Here, four hours. Total how
much? One plus two, three, three, six. Ten hours. Ten hours. When will the guard come? In
eight hours. If Coco guesses these three, then how much time does it take? Ten hours. So will
Coco be able to eat three at the speed of three? No. No. No."
    },
    {
      "start": 951.79,
      "end": 974.21,
      "text": "Coco will not be able to eat from the speed of 3. We came to know that there
can't be an answer in 3. There can't be an answer in 3. Okay? It's a good thing. Come, let's
try to do something else. We are telling you this because we want you to solve all the
questions of binary search by looking at this one question. 3, 6, 7, 11. Let's go. 0, 1, 2, 3."
    },
    {
      "start": 974.21,
```

```
      "end": 988.3,
      "text": "We say that the speed that we are guessing right now, whatever we are guessing,
guess 7. 7, it can be guessed because look, what we said that my lowest speed is 1 possible,
highest can be 11, so we said do 7, you can do anything, right? Let's do 7, let's see."
    },
    {
      "start": 988.66,
      "end": 1015.79,
      "text": "What will happen in 7? Now see, we have to make 3 here. How much is its eating
speed? 7. So what will it do? It will eat 3 in 1 hour and stop there. It will eat it and it
will take 1 hour here. 6. What will it do in 6? 6 is less. It will eat 6 in 1 hour and stop
there for 1 hour. Okay. It will take 1 hour. See, it will take 1 hour in every pie. If it has
to make 1 in it, it will take 1 hour. And it will make 1. Okay. 7. 7 too. Okay. Your speed is
7. 7 bananas are there. Eat 7. Eat 7 and stop for 1 hour. 11."
    },
    {
      "start": 1015.86,
      "end": 1044.83,
      "text": "What will happen in the first hour? She will be able to eat 7 out of 11. 4 will
be left. In the second hour, she will eat 4 out of 4 and make it 0. So now 2 is left. Let's
see how much time it took. 1 hour to eat this, 2 to eat this, 1 to eat this, and 2 to eat this.
 1 plus, sorry, 1 here. So 1 plus 1 plus 1, 3 plus 2. 5 hours. Yes, yes, yes. In how many
hours will the guard come? In 8 hours. If Coco eats at the speed of 7, then she will eat in 5
hours."
    },
    {
      "start": 1044.83,
      "end": 1071.81,
      "text": "So, it will eat in 5-6 hours. Okay. So, we came to know that if we guess, then
how do we know the answer? Now, here we did this, but how do we know? Let's see how to know.
Suppose, this is very simple. You just have to think in every problem related to the story,
how to know the answer of the guess. Because, till now, in the binary search, you used to get
the answer of your guess immediately. How? You used to guess on index. So, you used to do like
this, array of guess."
    },
    {
      "start": 1071.81,
      "end": 1094.86,
      "text": "equal to target, so you would have known immediately that my guess is correct,
that my guess is in the less one, that my guess is in the more one, you would have known
immediately, but now you will not know immediately like this, how will you not know, assume
that you have 3, 6, 7 or 11, now k is equal to whatever your k speed is, whatever your
guessing is, you have guessed 3, now how will you know that from these 3, now see,"
    },
    {
      "start": 1094.86,
      "end": 1117.25,
      "text": "After guessing one speed, what can happen? Either you will be able to eat or
you will not be able to eat. This is what will happen. There is nothing in between. Either
Coco will eat all or Coco will not be able to eat. Or, one more thing can be known that either
you will eat in how many hours. See, from every speed, even if your speed is 1, at some point
or the other, you will eat a banana. At some point or the other, you will eat a banana."
    },
    {
      "start": 1117.25,
      "end": 1145.84,
      "text": "It's just a matter of how many hours it takes. If it takes you 100 hours to eat,
 then there is no use. So you will get to know that at what speed we have to know how many
hours. See, how will we know? 0, 1, 2, 3. Now see, your K is 3. Now see, you can think of it
as an example. Suppose your banana is 10. Your speed is 10, what is your speed? K. How many
hours will it take to eat? What does this mean? You can eat 3 in 1 hour. So see,"
    },
    {
      "start": 1145.84,
      "end": 1173.47,
      "text": "In the first hour, if you eat 10 and do 7, then 1. After that, if you eat 7 and
do 4, then 1. After that, if you do 4 and do 1, then 1. After that, if you do 1 and do 0, then
1. Now see, this is not 10 by 3. What is 10 by 3? 10 by 3 is 3. We are talking about integers,
we are talking about computer language. 10 by 3 is 3. But, if 10 modulus 3 is not equal to 0,
which means that a remainder is left,"
    },
    {
      "start": 1173.47,
      "end": 1191.98,
      "text": "Why 10 by 3 is 3? Because out of 10, you can eat 9 first. How long will it take
to eat 9? 3 hours. But even after eating 9, the remainder of 1 is left. It will take 1 more
additional hour to eat this. If this is the case, then 3 plus 1 is equal to 4 answers.
```

```
Similarly,"
    },
    {
      "start": 1191.98,
      "end": 1219.98,
      "text": "Suppose you have to make 7. What is your eating speed? 3. So see, how long will
7 take? If you eat 7 in the first hour and make it 4, then it will take 1 hour. After that, if
you eat 4 and make it 1, then it will take 1 more hour. But the remainder is left. So to eat
this 1, it will take 1 more hour. So 3. Let's see. First of all, 7 by 3. How long will it
take? 7 by 3 is 2. Because we are doing integer division. But 7 modulus 3 is not equal to 0.
So 2 plus 1 is equal to 3."
    },
    {
      "start": 1219.98,
      "end": 1248.0,
      "text": "By doing this, you can find out. Yes, yes, you can find out. See, k is equal to
3. Come, let's see. Remember, we had taken out the answer of k is equal to 3. Where? 10 hours.
Come, let's see if it comes out or not. First of all, 3. First of all, 3. So, what will we do
first? 3 by 3 is equal to 1. And, 3 modulus, 3 is equal to 0. It means, there is no remainder.
It means, in one hour, you will eat and clean it. Nothing will be left. Its answer is 1. Very
good. 6. 6 by 3 is equal to 2."
    },
    {
      "start": 1248.0,
      "end": 1277.04,
      "text": "So, 2. But, 6 modulus 3 is not a remainder. Meaning, if you eat 6, 3 times, 2
times, then 6 will be finished. So, there is no need to stop. Very good. 2. 7 by 3 is equal to
2. But, 7 modulus 3 is not equal to 0. Meaning, if you divide this and eat it, then 6 will be
1. It will take you another hour to eat 1. 2 plus 1. Okay. 11. 11 by 3 is equal to 3. But, but,
 what is 11 modulus 3?"
    },
    {
      "start": 1277.04,
      "end": 1301.22,
      "text": "It is 2. Now see, 2 does not mean that you will take 2 more hours. No. See, 11
by 3 is equal to 9. It means that you will eat 9 out of 11. It means that you will eat 9 in
this and 2 will be left in this. It took you 3 hours to eat 9. That is written here. But, how
many hours will it take you to eat 2? It will take 1 hour. Because what is your speed? Your
speed is 3, right? Your speed is 3. So, 1 hour in eating 2. So, if 11 modulus 3."
    },
    {
      "start": 1301.22,
      "end": 1324.64,
      "text": "is not equal to 0, then whatever r you are counting will be plus plus. So here
also we will do plus 1. How much came? 1 plus 2, 3, plus 2, 5, plus 6, plus 9, plus 10. You
got to know this. How did you get to know this? You choose a speed, traverse on the array, do
this and that for every element. We will write its code too, but I am telling you. How much
time did it take?"
    },
    {
      "start": 1324.64,
      "end": 1335.94,
      "text": "Order of n, order of n, not order 1. You will not know immediately whether it
will happen or not. You will have to look at the whole array. You will have to look at every
pile and calculate whether it will take 10 hours or 11 hours. Okay."
    },
    {
      "start": 1335.94,
      "end": 1365.55,
      "text": "Very good. Order of n. Does it satisfy the condition? How to get the answer of
the guess? If you get it in order of 1, it is very good. It can be in order of n. If it is in
order of n, then you got the answer of the guess. Okay. Now, the next thing is, is your answer
monotonic? Is it monotonic? Okay, tell me one thing. Now, think a little. Think a little. When
we teach, we don't want to teach something magical that you don't understand. We think along
with you. We teach you how to approach. If you make it yourself, then how will you think? Okay.
"
    },
    {
      "start": 1365.55,
      "end": 1395.34,
      "text": "Let's assume that when you guessed your speed as 3, then it can be any pile.
You knew that it took 10 hours. What is this? It is simple speed and time. What is simple? It
is speed and time. It is speed and time. It is a pile. Your speed is okay. You did one more
thing. You chose your speed as 7. How much was it in 7? According to me, it was 7. Where was
it? Did I forget the 7th one?"
    },
    {
```

```
      "start": 1396.14,
      "end": 1420.35,
      "text": "I think it was 7. I forgot the 7th one. It took 4 hours. Let's say it took 4
hours. I don't remember. You can see. If your speed was 7, then it took 4 hours. Okay, tell me
one thing. In this case, what is the duration of your guard's arrival? 8 hours. It means you
have a window of 8 hours. In this case, will you be able to eat cocoa? Will you be able to eat
cocoa when your speed was 3? No. No."
    },
    {
      "start": 1420.35,
      "end": 1435.18,
      "text": "She will not be able to eat at all. Why? Guard will come in 8 hours. It will
take you 10 hours to eat. So if your 10 is greater than 8, you will not be able to eat. But,
speed is equal to 7. Yes. Why? She will finish eating in 4 hours. When will guard come? After
8 hours. Very good. There is enough time. Okay."
    },
    {
      "start": 1435.41,
      "end": 1460.11,
      "text": "Now, realize a very important thing. In speed 3, you said that you won't be
able to eat. Means, when your speed was 3, you couldn't eat. Because your speed will be slow.
If your speed is slow, you won't be able to eat. You took more time. Who takes more time? The
one who is slow. The one who studies slowly, he studies in more time. The one who studies fast,
 he studies fast. This is what happens, right? Okay. Now, if we ask you that will you be able
to eat in speed 2?"
    },
    {
      "start": 1460.11,
      "end": 1487.23,
      "text": "Do you need to do any calculation for this? Isn't it logical that you won't be
able to eat in speed 2? You won't be able to eat in speed 2. If you won't be able to eat in
speed equal to 3, then how will you be able to eat in speed equal to 2? If we are saying that
when your speed was 70 km per hour, then it took you 3 hours to go from Delhi to Jaipur. Now
if someone asks you that if we want to drive at a speed of 40 km per hour,"
    },
    {
      "start": 1487.23,
      "end": 1511.65,
      "text": "Will it take 3 hours or more? It will take more than 3 hours. The speed has
decreased, the time will increase. So, if the speed is equal to 3, it will take 10 hours. If
the speed is equal to 2, it will take more than 10 hours. It may take 11 hours. So, no matter
how many hours it takes, the answer will be 9. Because the minimum time is 8 hours. So, if the
speed is equal to 3, it will take 9 hours. So, what do you know?"
    },
    {
      "start": 1511.65,
      "end": 1533.41,
      "text": "If you try any speed less than this, the answer will be no. It is not possible
that when Koko is eating at the speed of 3, it will take 10 hours and if she starts eating at
the speed of 2, it will take only 2 hours. No, it is not possible. If she eats at the speed of
2, it will take 20 hours and more time. Similarly, if yes is coming in speed 7, if you
increase the speed,"
    },
    {
      "start": 1533.41,
      "end": 1561.68,
      "text": "If you increase the speed, then will it take less than 4 hours? If you were
running at a speed of 100 kmph, then you would have reached in 2 hours. If we were running at
a speed of 200 kmph, then we would have reached in 1 hour. So, let's assume that it will take
only 3 hours. Is 3 hours also a yes? Yes. See, you have to complete within 8 hours. Now, do
you eat in 4 hours? Or do you eat in 2 hours? Or do you eat in 30 minutes? That is your
perspective. So, here also, it is a yes. So, see, this monotonicity has been created. How?"
    },
    {
      "start": 1561.68,
      "end": 1589.15,
      "text": "What did we say? What did we say? From where does your speed vary? From where
does your speed vary? Your speed can be 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11. In this case, up to
1 point, up to 1 speed, all the answers will remain no. Up to 1 speed, all the answers will
remain no. But, 1 speed will come where the answer will be yes for the first time. And beyond
that, the speed you increase,"
    },
    {
      "start": 1589.15,
      "end": 1617.6,
      "text": "Everyone's answer will be yes. Yes. Yes. See if the monotonicity is made or not.
 The monotonicity is made. The monotonicity is made. Who are you interested in? You are
```

```
interested in the yes one. But you have to do it at minimum speed. Now you can say that we will do 11 only. No, we don't have to do 11. The example of the fan. Minimum speed so that you can eat all the bananas. So minimum speed so it is yes. This is your turning point. This is your turning point. What is this? First occurrence. First occurrence."
    },
    {
      "start": 1617.6,
      "end": 1629.78,
      "text": "So, if your guess, again, if your guess comes here, you know that my answer is no, where should you go? You should go right. How do you go right? Low is equal to?"
    },
    {
      "start": 1629.78,
      "end": 1658.96,
      "text": "Guess plus one. If your answer comes on yes, then what did you do in the first occurrence? Remember what you did. First of all, you used to write its answer. First of all, you used to write its answer. And if this is there, then these two are useless. How is the right one first occurrence? You are interested in minimum speed, right? Remember the example of the fan. If you started liking number 8, then why are you trying 9 and 10? Try a little less to save the electricity bill. Try minimum. Go left. But write this. Write this. There is no guarantee. Write this. And?"
    },
    {
      "start": 1658.96,
      "end": 1677.89,
      "text": "Go left, go left, go left. If you do this, what will you get in the end? You will get an answer. Very good. Come, let's write its code and try run it. So, understand this question very well. It is a very famous question. It has been asked almost in every company. Okay? Come, write its code. First of all, you have to write a function."
    },
    {
      "start": 1677.89,
      "end": 1706.32,
      "text": "In this type of question, you will have to write a function, for what? The step to get the answer from the guess, you used to get to know every time. This time, to get the answer from the guess, you will have to do a different process. First of all, let's write int fun. What will you pass in this? Look, first of all, we will pass our piles, which is my piles,  array. We will pass the length of the array. In this, we can also pass what is my speed. Now, what speed have you guessed? Pass that too."
    },
    {
      "start": 1707.15,
      "end": 1728.75,
      "text": "Okay, we need to pass only this much. Now, what is the meaning of this? What is its work? If you pass in this that my K is equal to 3 speeds, then it will tell you the hours. Now see, from every speed, from every speed, we will eat at some point, right? From every speed, we will eat at some point. From one speed also, it will take at some point 2 years, but we will eat. So, if it tells you the hours, suppose it tells you that tell me."
    },
    {
      "start": 1728.75,
      "end": 1753.84,
      "text": "My guess is that if we eat from 3 speeds, how long will it take? Let us return that it will take 10 hours. After that, we will check this in our binary search, that if my 10 is greater than 8, the answer is no. What is 8? The time for the guard to come. But, if my 10 is less than or equal to 8, the answer is yes. According to that, we can take the decision. So,  let us return this as our. Okay. How did we do it? int h is equal to 0. After that, what did we do?"
    },
    {
      "start": 1753.84,
      "end": 1783.22,
      "text": "How long did we run the loop? From the beginning to the end. Okay. Run the loop.  And what did we do? First of all, h is equal to h plus array of i by speed. By speed. Remember what we did. 10 by 3. 10 by 3. That's what we did. That's what we did. If there is 6 bananas, your speed is 3, then 6 by 3. It will take 2 hours. But if any remainder is left, suppose it is 7, then 7 by 3 will take 2. If any remainder is left, not equal to 0, it will take 1 hour to clean that remainder."
    },
    {
      "start": 1783.22,
      "end": 1798.78,
      "text": "So, if array of I modulus speed not equal to 0, then H++. That's it. And return H. What does this mean? It will eat in this much time. It will eat in this much time. It will eat in this much time and finish it."
    },
    {
```

```
      "start": 1798.78,
      "end": 1827.34,
      "text": "Now let's come to the main program. This is your helper function. You will
always have to write helper function in the story problem. This is the reason why people find
this question hard. That they don't understand what to do. You don't have to do anything. You
were getting the answer from the guess earlier. Now you will have to work a little hard. So
the whole hard work is in this. Rest is the guess. What is the hard work in the guess? There
is no hard work in it. Do any guess. But in the process of getting the answer from the guess,
you will have to put a little hard work and a little brain in every question. Now let's come
to our main function."
    },
    {
      "start": 1827.34,
      "end": 1855.6,
      "text": "So, in our coco function, what was given to you? First of all, you were given a
banana, that the banana is kept like this. And you were given a deadline for the guard to come.
 You were told that the guard will come in an hour. It was given like this. Come. First of all,
 int n is equal to a.size. Okay? Now, what did we say? What can be your lowest speed? 1. Okay?"
    },
    {
      "start": 1855.6,
      "end": 1883.62,
      "text": "What can be the highest speed? Max of error. Write max of error by running the
loop. Max of error. We are not writing max of error in this. As we write, write it. Okay? So,
write max of error. Because there is no use of more speed than that. There is no use of more
speed than that. Okay. Come. What will we take? We will take the result. We will take minus 1.
Okay? Yes. And if it happens that it will not be able to eat. It will not be able to eat. It
has not been said in this. But maybe you will get that."
    },
    {
      "start": 1883.62,
      "end": 1912.14,
      "text": "you have to return minus 1 and suppose she won't be able to eat. It's like the
guard came back in an hour and there is no possibility to eat in an hour, so she won't be able
to eat. Suppose in this area, where there were 4 piles, we saw that it will take at least 4
hours. Now the guard will say that I am going for an hour, I will come immediately. So if she
won't be able to eat, then return minus 1, otherwise it won't happen. There is no such test
case in the question. That's it, you are ready. You have done the main work, now what do you
have to do? Low less than equal to high."
    },
    {
      "start": 1912.14,
      "end": 1942.29,
      "text": "We will also do its dry run, don't worry. First of all, guess the middle one.
Why the middle one? We can do it according to ourselves, but it is better to do the middle one.
 Okay. Now, what did we do? int hour is equal to, now by calling this function, what all did
we have to call in this? a, n and your speed, which you have guessed. Okay. Now, you got an
hour, whether it will eat in 10 hours or in 20 hours. Now, how will we know its answer is yes
or no? If hour is greater than God,"
    },
    {
      "start": 1942.29,
      "end": 1964.93,
      "text": "If hour is greater than guard, means it takes 10 hours to eat, guard will come
in 40 hours. What is this? This is no. Yes, how was your answer? See, no, no, no, no, no,
continuous. So if you are on no, then where do you have to go? If you have to go right, then
always look at the diagram and find out. So what will you have to do? Go right. How to go
right? Low is equal to? Guess."
    },
    {
      "start": 1964.93,
      "end": 1990.51,
      "text": "Plus 1. Else. Else means yes. Yes. What did we do in yes? First of all, we used
to note this guess in the result. Because you need first occurrence. If you need first
occurrence, then where will you get it? You can get it in left. Remember the question of first
occurrence. Please see that. It was a nice question. We had explained it very well. Please see
that. Guess minus 1. Close this. Close this. Return. Rest. So see."
    },
    {
      "start": 1990.9,
      "end": 2017.22,
      "text": "It's a perfect binary search question, but what's the difference? What's the
difference? You are doing this work, this one. How much time are you taking in this? Order of
n. Order of n time is taking. And how much time will it take in total binary search?
Approximately log n. So its total time complexity is n log n. Why? Because every time in
binary search, how much time did it take? Log n. Because you used to get the answer of this
guess immediately. You used to get the answer of guess immediately. Now you are not getting it
```

```
immediately. Now you..."
    },
    {
      "start": 2017.22,
      "end": 2045.71,
      "text": "you have to work for it, so n log n. So we said that you have to think about
this work only. In every problem, you have to think about this helper that how will we get the
answer of what we are guessing? Just one thing. If you think, any question will be made. Come,
let's try its dry run. Let's try its beautiful dry run. So, what dry run should we do? Not
that big. Let's do the same example that we were studying. So, 3, 6, 7, 11. 3, 6, 7, 11. Okay?
0, 1, 2, 3."
    },
    {
      "start": 2045.9,
      "end": 2073.62,
      "text": "In the beginning, what did he say? What will be the low? 1, because we will
have to eat 1. What will be the high? The maximum of your array, the maximum pile is 11. Let's
guess. So, what will be the guess in the beginning? 1 plus 11 by 2, means 6. Let's see if it
works in 6 or not. So, let's see what will happen in 6. How much time will it take to eat
this? 3 by 6. Yes, you see. We have 3 eggs, so how much? 3 by 6. 3 by 6 is 0, but..."
    },
    {
      "start": 2073.62,
      "end": 2102.38,
      "text": "3 modulo 6 is not equal to 0, means some remainder, remainder is left, whole 3
is left, so we have to do 1 plus, so 1 hour, so how many hours will it take to eat this? 1
hour, to eat this, 6 by 6, 1 hour, to eat this, 7 by 6, 1 hour, but 7 modulo 6 is not equal to
0, so 2 hours, 11, 11 by 6 is 1, but 11 modulo 6 is 5, means not equal to 0, 1 more hour to
clean it, 1 more hour, so how much happened? 1 plus 1, 2, 4, 5, 6, 6, when will your GATFAP
come?"
    },
    {
      "start": 2102.38,
      "end": 2126.98,
      "text": "Your guard is going to come in 8 hours according to this. So will it be done in
6? Yes, it will be done in 6. Its answer is yes. Yes means write it in the result. Write it in
the result and go left. Let's see if we get anything better or not. So how do we go left? High
is equal to guess minus 1. Now what happened? High is 5. Guess what will come? 3, 3, 3. Let's
see what will happen in 3. Let's see what answer you get in 3."
    },
    {
      "start": 2126.98,
      "end": 2156.83,
      "text": "Now your speed is 3. So let's see in 3. You will get 10. But let's see if it
works with the program or not. 3. What is 3 by 3? 1. Is 3 modulus 3 0? Yes, it is 0. There is
no remainder. There is no need to add anything. 6 by 3 is 2. 6 modulus 3 is 0. There is no
need to add anything. 7 by 3 is 2. But 7 modulus 3 is 1. So one more. 3. 11 by 3 is 3. But 11
modulus 3 is not 0. So there will be remainder. So one more hour. One more hour. How much? 1
plus 2 is 3. 6, 9, 10."
    },
    {
      "start": 2156.83,
      "end": 2182.88,
      "text": "Now it takes 10 hours. When will the guards come? They will come in half an
hour. Will she be able to eat anything? No. No means like this. No, no, no, no, no. Yes, yes,
yes, yes. Yes is in the right of no. Come to the right. How do we go to the right? Low is
equal to guess plus 1. High is equal to 5. What is your guess? 4, 4, 4. Come, let's see 4.
Come, let's see 4. Come, let's see 4. See from 4."
    },
    {
      "start": 2182.88,
      "end": 2203.18,
      "text": "3 by 4 is 0. But 3 modulus 4 is not equal to 0. So it will take 1 hour. 6 by 4
is 1. But 6 modulus 4 is not equal to 0. 1 more. 2. 7 by 4 is 1. But 7 modulus 4 has remainder.
 2. 11 by 4 is 2. But 11 modulus 4 is not equal to 0. 3."
    },
    {
      "start": 2203.18,
      "end": 2232.7,
      "text": "1 plus 2, 3. 5, 8. Exactly 8. So it will be done in 8. Yes, it will be done in
8. Because now let's assume that if you eat in 8 and it is done in 8, then it will be done. So
what is this? Yes. What did we do in yes? We used to update the result. 4. And we used to go
to the left and see once. Once we used to go to the left and see. Low 4 and high is equal to 3.
 Now see. Your low has increased from high. That means this is Betu's thing. That my start
speed is 4 and end speed is 3. So this is Betu's thing. Question is over. What is your answer?"
    },
    {
```

```
      "start": 2232.7,
      "end": 2258.58,
      "text": "4 means 4 is the minimum speed. And there is also, see, if you reduce it by 4,
3, it took 10 hours on 3, it took 8 hours on 4, so 4 is your turning point. So, according to
this, you can do 4. So, that's it for today. And do solve this question. And if you want, you
will get all the questions in the sheet below this. Solve all that. Everything is solved like
this. In the coming days, we will tell you how it happens. Okay. Let's meet."
    }
  ]
}
```


25. transactions/t_6.json


```
{
  "video_id": "14NH9aD7XeY",
  "title": "Tree Series Last Video Construct a Binary Tree from Preorder and Inorder
Traversal",
  "language": "en",
  "model": "large-v3",
  "duration": 1675,
  "segments": [
    {
      "start": 0.0,
      "end": 24.21,
      "text": "Look, today I will give you a disclaimer. The question is not easy. The
question is complex. It is not hard but it is complex. I am sweating a lot while teaching. So,
those who are serious, watch that only. Those who are here and there in the air, don't watch
it. You won't understand it. So, don't waste so much time. I mean, if you watch it for 2-3
minutes, then don't watch it. You won't understand it at all. I will tell you now. It is not
because the teaching is bad. We should teach."
    },
    {
      "start": 24.21,
      "end": 54.03,
      "text": "The question is very difficult. This is the most difficult of all the tree
questions till now. So, you can see that this sheet is over while teaching. So, those who are
serious, see that only. Those who are serious, it's not your business. So, let's go. Let's
start by explaining the question. Let's understand the question. Then we will come to our real-
life example. This time, you have a very strange thing that no tree has been given. No tree
note has been given. You have been given two arrays. Okay. You have been given two arrays. One
array is pre-order."
    },
    {
      "start": 54.03,
      "end": 79.22,
      "text": "In the past, there was a tree. In the past, there was a bridge. What kind of
tree will it be? What will happen to us? It will happen anywhere. Pre-order of that tree has
been given. Another thing of that tree has also been given. In-order has also been given. Yes,
in-order has also been given. Okay. By using these two arrays, you have to generate a real
tree. It means you are a creator or you are a builder. By generating a real tree,"
    },
    {
      "start": 79.22,
      "end": 109.78,
      "text": "you have to return the root. So till now you used to get the root, you used to
do something in it, now you have given an array, that means you have given its pre-order and
in-order traversal. You generate its root and show it. Before moving ahead, a very important
question or concept that you should understand is that why are two things given? If you would
have given a pre-order, what is a pre-order? Root, left, right. Okay? What is in-order? Left,
root,"
    },
    {
      "start": 109.87,
      "end": 139.65,
      "text": "Right. You would have thought that if someone would have given us only one pre-
order, then we would have made it. For example, if someone tells you that there is a tree, its
pre-order is 1, 2, 3. So, in simple, you could have thought that it is 1. So, 1 becomes root.
This becomes left. And this becomes right. So, why don't we make such a tree that it becomes 1,
 2, and 3. Very good. You can definitely make it. But, you have made this tree. You have made
this tree and submitted the paper. We will make another tree. Let's see what will come after
its pre-order."
    },
    {
      "start": 139.65,
      "end": 162.13,
      "text": "Let's make another tree. See. 1, 2, 3. Is it different from your tree?
```

```
Absolutely different. Take out its pre-order. We will come here. 1, 2 on the left child. 3 on the left child. Its pre-order is also 1, 2, 3. See another tree. 1, 2, 3. Its pre-order is also 1, 2, 3. Means."
    },
    {
      "start": 162.13,
      "end": 192.4,
      "text": "If you give us a pre-order of 1, 2, 3, then we can make 6-7 types of tree for you. It means there will be no unique answer. It will not be known whether the tree you made is right or the tree we made is right. The pre-order of both is the same. But, but, but, when you give in-order with pre-order, when you give in-order with pre-order, then things change. See, suppose you have the same pre-order, 1, 2, 3. And your in-order is also 1, 2, 3. Your in-order is also 1, 2, 3. Now see, one thing is certain that you"
    },
    {
      "start": 192.4,
      "end": 220.59,
      "text": "1 will be the root of the tree. If this is a pre-order, then 1 will be the root of the tree. If the root of the tree doesn't come before the pre-order, then it won't be possible. 1. Okay. Okay? Now, if you say, if you say that we will do one thing, we will make this. We will make this. You say that this is the tree, its pre-order is matching. Now, take out its inorder and see. First of all, it will come here. The root itself will not be printed. First, it will give a chance to the left side. So, it will become 2, then 1, then 3."
    },
    {
      "start": 220.59,
      "end": 247.47,
      "text": "Does this match with your inorder? No, this is your inorder. You had to combine both of them and give the answer. But the answer you are giving matches with this. Pre-order is the same, but is inorder the same? No. That's why, with the combination of pre-order and inorder, a unique answer will come. If you do any circle from here to there, it will change. If you have given only pre-order, then a lot of things... I mean, you see, we have made 6-7 types of things in 3 lengths, right?"
    },
    {
      "start": 247.47,
      "end": 276.91,
      "text": "If you give 100, then you will give 1 lakh trees. There are many possibilities. If both are given, then there is only one possibility, a unique possibility. Now we will move ahead and see how to understand it, how to code it. And we will give you an advice that you will understand it stage by stage. Don't expect that the initial explanation will come 100%. It is stage by stage. In the beginning, there is an explanation, i.e. there is an intuition in the beginning. Then there is a little more deep intuition. Then there is an explanation with its number."
    },
    {
      "start": 276.91,
      "end": 306.78,
      "text": "Then there is the code and then there is the dry run. You will understand it stage by stage. You will understand it till the end. And even then, if you don't practice it on the copy pen, then it won't come. We will give this as a disclaimer. Keep patience and let's move ahead. Now see, we have made Dada ji. Dada ji is made, right? Dada ji's work is over. How? Pre-order. Make the first one and make Dada ji. You will know what is left and right. Everything is done. Now let's do one thing. Let's make Dada ji's entire left subtree. Which one? Let's make Dada ji's entire left subtree."
    },
    {
      "start": 306.78,
      "end": 333.31,
      "text": "If we look at the right, then we can make the whole left. So tell me one thing, that you already knew that Dada ji was here till 2. In Dada's left, Papa and Pratyushi can come, which is 0 and 1. So do you have any other array of work? At least when you are looking at the left subtree, then is there any other array of work? There is no other array of work. So we will do one thing, that as long as we are in the left subtree, we will look at the same array. Let's assume that this is the array. How? Let's assume that a low is equal to 0 and"
    },
    {
      "start": 333.31,
      "end": 362.75,
      "text": "High is equal to 1. Rest of the inorder array is not there. Because rest of the inorder array is useless. Dadaji has been created. Dadaji's right is in the right subtree. Left subtree is useless. Now only two things are important for you. Okay. Now come. We have come to the left subtree. Now tell me one thing. We have come to the left subtree. You know that Papa and Pratyush are in the left subtree. Now its next index is 1. Which is Papa. Whose? Pre-order. Remember pre-order. We first looked at index 0 and it became Dada. When we came to index 1, we came to Papa. Now tell me one thing."
    },
```

```
    {
      "start": 362.75,
      "end": 390.11,
      "text": "What should be the immediate child of this? Should it be Papa or Pratyusha? We
are talking about this circle. Should it be Papa or Pratyusha? What should it be? If you say
that we will fill Pratyusha here. Both can be filled, Papa and Pratyusha. Both can be filled.
You say that you will fill Pratyusha in both. If it is so, then tell me one thing."
    },
    {
      "start": 390.11,
      "end": 419.74,
      "text": "That when Dada is here and Pratyusha is saying that we will pay, then how did
Papa come before Pratyusha in the pre-order? How did it happen that when you got the pre-order,
 Dada came, Dada should have come. After that, we got root. After root, left came. Now how did
Papa come in the left? Pratyusha should have come, right? Pre-order should have been Dada and
Pratyusha. How did Papa come? Where did Papa come from? Papa came because Pratyusha cannot be
here. Papa has to be here."
    },
    {
      "start": 419.74,
      "end": 448.3,
      "text": "Papa will have to be here. So if Papa is not here, then your pre-order is wrong.
 You are doing something wrong. Listen again. When we were on Dadaji, we saw that Dadaji has
Papa and Pratyusha on the left. So it can be in the left subtree. Both can come from anywhere.
But when we move forward in the pre-order, we see that Papa is on 1. Can Pratyusha come here
in any birth? No, it can't. Because if Pratyusha comes here,"
    },
    {
      "start": 448.3,
      "end": 467.01,
      "text": "So you first write Dada in the pre-order. Then you write Pratyusha. Then why
did you write Papa? Then you write Papa later. Pre-order tells you in which order the circle
will be formed. As soon as you come to the left circle, the next person who is Papa, you write
that here. Now, now, now, look very carefully."
    },
    {
      "start": 467.01,
      "end": 494.93,
      "text": "The same thing that you repeated in Dadaji. You looked for Dadaji in Inorder.
You told the left of Inorder to come to the left sub-tree. You told the right of Inorder to
come to the right sub-tree. Do the same thing on Papa. Do the same thing on Papa. What should
you do? Look for Papa in Inorder. Look for Papa in Inorder. Where did you find Papa? You found
him on zero. You found him on zero. Now, now, in this, is there anyone on Papa's left? Is
there anyone on Papa's left?"
    },
    {
      "start": 494.93,
      "end": 522.11,
      "text": "No, there is no one on the left of zero. That means, can there be anyone in the
left sub-tree of papa? No, there can't be. No, there can't be. No, there can't be. Who is on
the right of papa? And see, we had told you carefully. When you have come here, you have come
with this array. Low zero, high one. You are considering only this array. On the right of papa.
 What is on the right of papa? There is a reflection. That means,"
    },
    {
      "start": 522.11,
      "end": 550.21,
      "text": "It means that in the right sub-tree of papa, what can happen? It will happen.
It will happen. Either it is one or it will happen. Do you understand or not? Why are you
saying that papa's left sub-tree will not happen? Because in the in-order array in the left of
papa, what do you see? Nothing is seen. So when nothing is seen, the form is papa. There is
nothing in the left of papa. So when there is nothing, then nothing will happen. Nothing will
happen. What happened in the right of papa? It will happen. Now, now."
    },
    {
      "start": 550.21,
      "end": 578.42,
      "text": "Now there is no need to make papa's left because it is null. We are not telling
you the code, we are telling you how to think about it. We will write the code too. Let's make
papa's right. Now when you make papa's right, when you make papa's right, you brought this
array. Now when you make papa's right, then just consider this. Just consider this. Because
dadaji was already there. In this array, how will it be? Low b1 and high b1. Low b1 and high
b1. Again."
    },
    {
      "start": 578.42,
      "end": 607.81,
```

```
      "text": "Which person will be on the right of Papa? The next person in the pre-order
will be Pratyush. So what is this? Pratyush. Repeat the same thing with Pratyush. Where did
you get Pratyush? On 1. Is there anyone on the left of Pratyush? There is someone on the left
of Pratyush, Papa. But which array have you brought? Low 1, high 1. You are considering only
this much. You are seeing only this much array in this order. So is there anyone on the left
of Pratyush? There is no one. Is there anyone on the right of Pratyush? There is no one.
Meaning?"
    },
    {
      "start": 607.81,
      "end": 635.84,
      "text": "Pratyush is a leaf node. Pratyush has left and right. Everything is equal. Is
it done? Papa's work is done. Papa's left is done. Papa's right is done. Come back to Dada.
Dada's left is done. Dada's left is done. See. The original tray was like this. Does the left
look like this? Yes, it looks like this. Now let's make Dada's right. Now let's make Dada's
right. What will happen? What will happen in making Dada's right? First of all, which three
people are there in Dada's right? Rohan, Chacha, Suraj. How did you know? In the inorder array.
"
    },
    {
      "start": 635.84,
      "end": 660.18,
      "text": "Dada's right. So we will just consider so many IRAs in order. From where to
where? Low is equal to 3 and high is equal to 5. Because the rest is useless. Okay. Now. Now
let's talk about right subtree. Come. In the right circle, who should be among the three? The
next pre-order person. The next pre-order person. 3. Why? Why? Why?"
    },
    {
      "start": 660.18,
      "end": 688.18,
      "text": "If you say that we don't want to keep him, we want to keep Rohan here. So when
you want to keep Rohan, then why didn't Rohan come here? How did Chacha come here? See, this
is your route. Left is over. Now starts right subtree. What is Chacha doing here if Chacha is
not here? If Chacha is not here, then what is Chacha doing here? That's why who will stay
here? Chacha. Chacha. Chacha will stay here."
    },
    {
      "start": 688.18,
      "end": 717.84,
      "text": "Same kan chacha pe dhaura haiye. Chacha ko khojye. Kaha mila? Yaha pe. Chacha
ke left me kaun hai? Ek aadmi hai. Rohan. Kaunsa aadmi hai? Low B3 ho gaya. High B3 ho gaya.
Chacha ke left me kya bacha? Sirf Rohan. To chacha ke left me kya hoga? Rohan. Rohan me kuch
hai nahi darudar karne ke liye. Kyunki jab aap Rohan pe aenge. To Rohan ke left me kuch nahi
hai. Right me kuch nahi hai. Jaisa Pratyush ke saath hua tha. Chacha ke right me kaun hai?
Suraj. Chacha ke right me kya hoga? Suraj."
    },
    {
      "start": 717.84,
      "end": 747.17,
      "text": "This is how you are making a tree. This is how you are making a tree. This is
how you are making a tree. This is how you are making a tree. This is how you are making a
tree. This is how you are making a tree. This is how you are making a tree. This is how you
are making a tree. This is how you are making a tree."
    },
    {
      "start": 747.17,
      "end": 774.61,
      "text": "The next step is to find Dada from here. Dada is found. All the items in Dada's
left will go to the left sub-tree. All the items in Dada's right will come to the right sub-
tree. Now make Dada's left sub-tree. What will happen in the left sub-tree? Go one step ahead.
What comes? Papa. This will happen here. Papa is done. Do the same thing. Find Papa here. Go
to Papa's left. If there is nothing in Papa's left, then Papa's left is null. Go to Papa's
right. What is there in Papa's right?"
    },
    {
      "start": 774.61,
      "end": 804.51,
      "text": "Pratyush. Pratyush is the next letter. Very good. Left is complete. Back to
Dadaji. To Dadaji's right. Which three people can be in Dadaji's right? Rohan, Chacha, Suraj.
Come to the right. Who will be the first? The next letter is Chachaji. Find Chachaji. To the
left of Chachaji, here. To the right, here. What does Chacha's left mean? Rohan. To Chacha's
left, see, Rohan is written. To Chacha's right, Suraj. So, to Chacha's right, Suraj is written.
 You will make it like this if you want to make it in copy. If you want to make it in copy."
    },
    {
      "start": 804.51,
      "end": 832.61,
```

```
      "text": "Now look at its code. Now look at its code. It has a lot of things to do. So what will happen to its code? It will write the code and then we will dry run it. Come. So what do you have to return? You have to return a node star. That is the problem. You have to return a node star. So we will make it something. We will make it a make tree. Make tree. You will be given two vectors. You."
    },
    {
      "start": 832.61,
      "end": 862.98,
      "text": "Two vectors will be given. One will be your pre-order and one will be your in-order. What will be your in-order? Okay? Okay. First of all, it is not needed. But what we do is... Is it needed or not? Yes, it is needed. See. What is yours? In pre-order, you will go index by index. So, this is fine. We will do this. Every time. But when you get Dadaji, you will have to find Dadaji here. In-order, Dadaji is on which index?"
    },
    {
      "start": 862.98,
      "end": 892.96,
      "text": "That's how we'll know. Let's do one thing. Let's make an inorder array into a hash map. What will happen is that you won't have to look at the whole thing to find Dadaji. You'll just ask where Dadaji is. Dadaji is here. Then you'll get it. Come. Let's make an unordered map. Unordered. We'll see the code and give it to you. Whose unordered map is it? Int and int. What does this say? Which number is on which index? Because Dadaji is not the real number. Which index is on which index? Okay? In. And one."
    },
    {
      "start": 892.96,
      "end": 922.38,
      "text": "Let's run the loop. int i is equal to 0. i less than in order dot size. And i plus plus. And what will we do? In in order i is equal to i. Means this number is on this index. Very good. After that what will we do? After that we will write a function. After that we will write a function. Let's close this here. And let me tell you how to call. See. After that we will write a function."
    },
    {
      "start": 922.38,
      "end": 951.07,
      "text": "We will write a global variable in this function. index is equal to zero. What does this index say? In which pre-order index are you? You are going to pre-order one by one. You are going to the pre-order index one by one. And from there you make a circle. Once your index starts at zero, then you make a circle of Dadaji. After that, you call the same function for Dadaji's left subtree and right subtree. We make an index and you need a low and high. Why do you need low and high? At this time,"
    },
    {
      "start": 951.07,
      "end": 981.31,
      "text": "Which part of the inorder are you looking at? When you start, you were looking at the entire inorder. That's why you found Dadaji here. Once you found Dadaji, when you went to Dadaji's left, you were just looking at this much. You didn't need this much. You were just looking at this much. When you went to Dadaji's right, you were looking at Dadaji's front to right. So what will we do for this? Low to high. What? Low to high. Okay. So let's write a function. This function will return things. Node."
    },
    {
      "start": 981.31,
      "end": 1012.26,
      "text": "Fun, right? Let's pass a pre-order. That we are doing this pre-order array's index. Pre-order. Done? Very good. Anything else? Low and high. Int. Low. Int. High. Okay? And we will make this hash map global. This hash map. We will make this hash map global. So that it doesn't have to be passed. It can be accessed here. Come. Now come."
    },
    {
      "start": 1012.26,
      "end": 1042.38,
      "text": "First of all, first of all, first of all, tell me one thing. Let's assume, let's assume that the base case will be written later. Let's assume that it is not. First of all, you are on any index. So when will you start? Index is equal to zero. So first of all, make a node of this. Make a node of this. How? Node star. Node is equal to new node. Whose? On whom are we? From whom did we start? From Dadaji. From index. From pre-order index. So."
    },
    {
      "start": 1042.38,
      "end": 1072.05,
      "text": "In the pre-order index, whatever element is there, make a node of it. So what did this work? This means that a node named dada is made. Once the dada node is made, it means that you have seen this in the pre-order. Go ahead, make index plus plus. Index plus plus.
```

```
Done? This was the work of pre-order. Now what is the work? In order, find dada. In order, find dada. Means, find its index."
    },
    {
      "start": 1072.05,
      "end": 1100.74,
      "text": "int id is equal to in our hash map and node's data. Or pre-order, whatever you have made, dadaji. What does this do? It finds dadaji. After that, what did you do? Same function, same function, called for left and called for right. Meaning, when you called for left, then suppose here, where did you get dadaji? On second index. So, what was your id? 2."
    },
    {
      "start": 1100.74,
      "end": 1129.86,
      "text": "When you called for left, what did you do? You had to see 0 to 1 on the left, Papa and Pratyusha. And when you called for right, you had to see 3 to 5. 3 to 5. Meaning, in the beginning, how much was low? 0 and how much was high? 5. Because we were looking at the entire inorder. When you got id is equal to 2, then what happened? You stayed at low is equal to 0, but you came to id minus 1. This became its low and high. What became the low and high of right?"
    },
    {
      "start": 1129.86,
      "end": 1159.7,
      "text": "id plus 1, which is 3, and high. So, this will be your lower high. What does lower high mean? That you are seeing the same in order. When you are on the left of Dada, then just see Papa Pratyush. You don't need to see the entire ending. If you are on the right of Dada, then see Rohan, Chacha, and Suraj. You don't need to see the rest. See. After that, what will we do? Node dot left. Means, on the left of Dada, Dada's left is equal to same function, same function."
    },
    {
      "start": 1159.7,
      "end": 1187.47,
      "text": "Pre-order will pass as it is. From low to id-1 means from the beginning to the end of dada. low id-1 and what is the right of node? 1 pre-order as it is and from dada to the end of dada means from id-1 to high. What will these two do?"
    },
    {
      "start": 1187.47,
      "end": 1217.6,
      "text": "This will make the whole left subtree of Dada. This will make the whole right subtree of Dada. Once this is done, then what have you done? What is happening in this? What is happening in the first time? You made a Dada. After that, you called. So, the whole left subtree has been made. The whole right has been made. Once it is done, then return it. Return node. That's it. This is its code. This is its code. We have not written the base case in it yet. We have not written the base case in it yet."
    },
    {
      "start": 1217.6,
      "end": 1245.47,
      "text": "I haven't written the base case, I'll show you. Remember this. And when we call this from the main, how will we return? Fun. We passed the pre-order in fun. And in the beginning, how much low-high do you want to see? How much do you want to see in the beginning? You want to see the entire in-order. So low is equal to zero. And what will be the high? In-order, the size of the in-order. Whatever the size of the in-order is. And minus one, which means the entire. This is done. This is the code."
    },
    {
      "start": 1245.47,
      "end": 1275.73,
      "text": "If you see its dry run, it will be more clear. Let's make a small tree for reference. Suppose this is your tree. 3, 1, 2 and 4. This is your tree. Is it correct? Let's write its pre-order. If we make an array of its pre-order, then 3, 1, 2, 1. If we make an array of its in-order, then what will it be? 1, 3, 4, 2. Is it correct?"
    },
    {
      "start": 1276.21,
      "end": 1306.51,
      "text": "Come. Is it okay? Okay. First of all, what do we call? First of all, our index is 0. What is the index now? 0. Means, what is the pre-order? We are looking at 0. So, fun. Assume that we are calling on index is equal to 0. We are not calling, we are doing it to explain. And, in the beginning, what is low and high? You want to see the entire in-order. You want to see the entire in-order. Low is equal to 0. High is equal to 3. You want to see the entire in-order. Very good. Now, come."
    },
    {
```

```
      "start": 1306.51,
      "end": 1336.19,
      "text": "First of all, what do we do? We haven't written the base case yet. But first of all, what do we do? We make a circle. Whose circle? We increase the index of the pre-order. It says, you are here now. Make its circle. Increase the index. Next time, its circle will be made. So, first of all, whose circle will be made? 3's. Means, here we have made the circle of 3. And what happened to the index? It became 1. Okay? Very good. After that, left subtree. After that, now this 3,"
    },
    {
      "start": 1336.19,
      "end": 1365.78,
      "text": "Now, find this 3 in order. Where did you find it? In 1. Means, what is your ID? 1. What to do? Call for left and call for right. So, what will we do? 1 will be called with low is equal to 0 and high is equal to ID minus 1. ID minus 1. See, ID minus 1. What will be ID minus 1? 0. And what will be 1 called? Low is equal to ID plus 1. 2. And high will be same. High is equal to 3."
    },
    {
      "start": 1365.78,
      "end": 1395.17,
      "text": "What is this for? For left. What is this for? For right. Okay? Do you understand till now? Come, let's make this. Fun. Remember, our index is 1. Till now, we have only made 3 in circle. Let's go. What is this? Low is equal to 0 and high is equal to 0. Okay. Let's see. First of all, what we have to do? The index. What is the index now? 1. Means you are here. Make a circle of this."
    },
    {
      "start": 1395.17,
      "end": 1423.07,
      "text": "So let's assume that we have made its circle like this. Now what we will do? We will find this 1. Where will we find it? In order. Where did we find this 1? Here it is. Here it is. After that what will we do? Now look carefully. You might be able to see it in the drawing. When we came here, we were looking at this only. We were looking at the left of 3. So there is nothing in the left of 1 and there is nothing in the right of 1. Let's see how our code catches it. What do we do?"
    },
    {
      "start": 1423.07,
      "end": 1450.32,
      "text": "After making idx, increase idx too. Increase idx to 2. Now let's call. What did you get? 0. Where did you get 1? You got it at 0. So what will we do now? To call left subtree, what will happen? Low will remain the same and high id will be minus 1. Okay. For right subtree, low id will be plus 1 and high will remain the same. This will be the call. Okay. Now tell me, is this making sense?"
    },
    {
      "start": 1450.32,
      "end": 1480.0,
      "text": "What does low and high tell you? Look from low to high. It happens everywhere in the sliding window. Look at the area from low to high. It is telling you to look from 0 to minus 1. It is a stupid thing, isn't it? At least look from 0 to 0 and you will understand that look at a block. What happens when you look from 0 to minus 1? It is a useless case. Nothing can happen, isn't it? Look from 1 to 0. How will we look at this? We can look at 1 to 0. We can look at 1 to 2. Low should always be equal to high or small, isn't it? What does it mean to be higher than low and high? There is no special meaning. Which area are you looking at? You are not looking at it."
    },
    {
      "start": 1480.0,
      "end": 1509.46,
      "text": "With this, we have to add another line here. If low is greater than high. If it is greater than low high, what will happen in it? Null. Nothing. Return null. Return null. That's why when you call both on 1, the answer of both is null. So see, there is no beta of 1. This function is over. This function is over. Who did this 1 come from? Who did this 1 come from?"
    },
    {
      "start": 1509.46,
      "end": 1539.74,
      "text": "From the left of 3. From the left of 3. From the left of 3. See. It's done. The left of 3 is done. Let's make the right of 3. Let's make the right of 3. Come. Let's make the right of 3. What's the call? Low is equal to 2. High is equal to 3. What's the index? It's 2. So, let's make a circle named 2. Let's increase the index to 3. That's right. Now, what to do? Search the circle you have made in order. Where did you get 2? You got it here. ID is equal to 3."
    },
```

```
    {
      "start": 1539.74,
      "end": 1570.77,
      "text": "ID is equal to 3. Same thing. Make left and right. What will be its left? Low
will remain low. And high ID will be minus 1. Very good. What will be right? Low will be 3.
Sorry, low ID plus 1 will be 4. And high 3 will remain. Okay. Now this is left. Let's run this.
 Fun. Low is equal to 2. High is equal to 2. Where is your ID? On 3. Meaning?"
    },
    {
      "start": 1570.77,
      "end": 1601.02,
      "text": "Make a circle of 1 somewhere. We have made a circle of 1 here. We have made a
circle of 1 here. Done. And increase the ID. Increase the ID. Keep in mind that your pre-order
array is over. The pre-order array is over. Okay? Now, what to do? The circle you have made,
the circle you have made, look for it. Look for it. Which one? Which one did you make? 3, 1, 2,
 1. How did this become? Oh, sorry. This was 4. This is 4. This is 4. So, you will make 4 here
too. Look for 4."
    },
    {
      "start": 1601.02,
      "end": 1630.85,
      "text": "Where did you get 4? Here. ID is equal to 2. ID is equal to 2. Do the same
thing. Try to call for left and right. So low will remain low. High will become ID minus 1.
Try to call for this. So low is equal to ID plus 1. High will remain the same. Look carefully.
Both are nonsense. You can't see from 2 to 1. You can't see from 3 to 2. It's all over. Where
did it come back? It came back to its left. Meaning? To whose left?"
    },
    {
      "start": 1630.85,
      "end": 1658.18,
      "text": "This one. In the left of 4, 2. We have to do the right of 2 too. But the right
of 2 is a nonsense thing. We can't see from 4 to 3. This is also finished. Where did this come
back? This came back and attached to the right of your main tree. And what will you do in the
last return? This is your root. Is this the same tree that we made? Yes, this is the same tree
that we made. So this is a small dry run. But again, the question is very hard."
    },
    {
      "start": 1658.18,
      "end": 1674.16,
      "text": "So, you would have got help by explaining, but until you do not copy it, it
will not come so easily. It will take time. It is a very good question. And with this, today's
tree series is over. Next time, we will move on to graph or DP. Let's go."
    }
  ]
}
```


26. transactions/t_7.json


```
{
  "video_id": "1yax0YQOb8w",
  "title": "Let's Revise HEAP Data Structure With This Leetcode Hard Question | CPU Task
Scheduler",
  "language": "en",
  "model": "large-v3",
  "duration": 2539,
  "segments": [
    {
      "start": 0.0,
      "end": 24.08,
      "text": "So we were reading a comment actually, so today we thought of doing a revision
of heap, because core scheduler 3, according to me, we think that when we teach you recursion,
then you should come to it, because it is a choice problem, so you should know that the choice
we are doing, then why can't we do recursion and why should we do heap, so we want to teach
you recursion first, then we will come to it. We were seeing another comment, we made a
question, you may have reorganized the ring."
    },
    {
      "start": 24.08,
      "end": 47.79,
      "text": "we thought that this cpu task scheduler is the same type of question and it is
90% correct so we didn't teach it we thought it will be done but today you must be seeing this
code we have written this code by ourselves so this code is a very big code so we shouldn't
have done this the question is actually very complex and we shouldn't have done this so those
who had that comment it was a very helpful comment come we will see this question"
    },
    {
```

```
      "start": 47.79,
      "end": 54.69,
      "text": "It is a very similar question. If you have made a reorganized string, then also
understand this question, think about the approach of this question."
    },
    {
      "start": 54.69,
      "end": 84.16,
      "text": "and writing the code of this question is very implementation heavy you can say
that we will explain it to you in easy language that what to do and what not to do but how to
do that that is not there so that is wrong from us also many times we are doing this submit
from us also a lot of error not syntax error but run time error we also got a lot of errors
here and there so we thought that we should teach this so come and read this after reading
this you can say that 80-85% of your heap is complete the hard questions are left course
scheduler is left two heaps are left"
    },
    {
      "start": 84.16,
      "end": 113.17,
      "text": "we will do recursion dp after that we will make more sense so we will do that
later so see this till the end its code is very hard intuition is not that hard you will feel
it as natural common sense that yes this should be done you will come to the mind that yes
this should be done how to do that is not there in the understanding so let's start quickly so
we did a question about reorganized string if you remember then maybe 1,2,3 lectures before
there was a question in which reorganized string was to be done in that we told you an example
of PT sir"
    },
    {
      "start": 113.17,
      "end": 139.81,
      "text": "We told you an example of PT sir. What did we tell? That he has such a seat in
the class. Exam is going to take place. And he has some students. Like there is a student from
class 8. There is a student from class 9. And there is a student from class 8. By doing this,
there are 3 students. Now, how did he have to make these students sit? Like we sit in the exam.
 How? By doing the alternate. Otherwise, the student starts stealing. So, what did he do?
Because there are 2 students from class 8. 2 students from class 8 and 1 student from class 9.
So, if he..."
    },
    {
      "start": 139.81,
      "end": 153.87,
      "text": "This seat will be filled in such a way that Class 9 is placed here. Suppose
Class 9 is placed here. Now they will have to place both Class 8 here. Because the seat was so
limited. So they would have to place it here and they would have stolen it. They didn't want
this. So what did they do?"
    },
    {
      "start": 153.87,
      "end": 171.17,
      "text": "All of them were seated according to the frequency because their goal was to
finish the child with the highest frequency first so that their tension is reduced. Later, if
one child is left at the end, then they will set him anywhere. But settle the child with the
highest frequency first. So what did they do? Class 8 students were seated here."
    },
    {
      "start": 171.17,
      "end": 193.65,
      "text": "Now they have two children left, one of 9 and one of 8. Again, both the
frequencies are the same. Now one of the children is of 8th class and the other is of 9th
class. So which one will they sit? Try both. Try the 8th class. Can the 8th class sit? It
can't sit. Why? Because it will be stolen. So what did he do? He used to make class 9 sit. Now
class 9 is also round. In the end, which one of the children is left? Class 8. He used to set
it here."
    },
    {
      "start": 194.0,
      "end": 223.95,
      "text": "So, P.T. sir didn't know anything about DSM, but P.T. sir used to make this
question using common sense, according to frequency. Now, in teaching this, we felt that this
question is similar and there are many ways in which CPU scheduler is used. CPU scheduler.
These two questions are exactly the same, except for one very important thing. Listen very
carefully. It's the same problem. Okay? It's the same problem. You have some children. Suppose,
 their name is A, B, C, D, like this. So, you have such children. There is one A, there is one
more A."
    },
    {
      "start": 223.95,
```

```
      "end": 247.79,
      "text": "There is one more A and three kids are of B. You have to arrange them. You have to make them sit like PT sir. But the constraint is not that you have to make the kids sit alternately. In the previous one, you had to make them sit alternately. It will be done. What is the constraint here? There is a gap. Gap is equal to 2. What does this mean? You can make any kid sit. But"
    },
    {
      "start": 247.79,
      "end": 268.46,
      "text": "There should be a gap of at least two seats between two similar children. If you place one child here, then suppose this is seat 1, this is seat 2, this is seat 3, this is seat 4, this is seat 5, this is seat 6."
    },
    {
      "start": 268.46,
      "end": 298.34,
      "text": "This seat will be 7 and so on. Okay. Now you have placed A here. On seat 1. You can place anyone on seat 1. It is your choice. You cannot place A on seat 2. Why? Because it has been said that there should be two gaps between two consecutive A's. There should be a gap of two seats. So here you can place B more. You can place B. So now A's work is done. B's work is done. Next, on this seat, now you have two types of children. Either A or B. Can you place A here? You cannot place A. Why? Because when you place A here,"
    },
    {
      "start": 298.34,
      "end": 324.96,
      "text": "How many seats are there in this A and this A? One seat. It should not be of one seat. There should be a gap of two seats. At least. It is said at least that there should be a gap of two seats. Can you make B sit here? No, you can't make him sit. If you make B sit, then how much gap will be left between these two? Zero seat. It is not allowed. There should be a gap of at least two seats. So this one, seat 3, you will have to keep it idle. No one will be able to sit on this seat. No one will be able to sit on this seat. Go ahead."
    },
    {
      "start": 324.96,
      "end": 347.41,
      "text": "On seat 4, can you make A sit? Yes, you can make A sit. Because if you make A sit here, how much gap is there between these two? Two seats. It works. If you want to keep two gaps, you can keep more than two, but you have to keep two. Here you can make A sit comfortably. This A's work is done. Come to the next seat. On seat 5, can you make A sit? No, you can't make A sit. Because if you make A sit here,"
    },
    {
      "start": 347.41,
      "end": 374.8,
      "text": "How much gap is there between them? 0 seat. We don't want 0, we want 2. We can't make A sit. Can you make B sit here? Yes, you can make him sit. Why? Because how much gap is there between them? 2 seat. 3 seat, 4 seat gap is left. Very good. B's work is also done. Come to the next. In S6, can you make B sit? No, you can't make B sit because B is here. 0 seat gap. You can't make A sit either. Remember, you can't make A sit either. Why? You'll make him sit here, right? How much gap is there?"
    },
    {
      "start": 374.8,
      "end": 399.76,
      "text": "One seat gap. You don't want one seat gap. You want two seat gaps. Again, you can't make anyone sit here. Neither A nor B. This seat is empty. What is this seat? It is empty. Move forward. On seat 7, can you make B sit? No. If you make B sit, then one seat gap will be left. Can you make A sit? Yes, of course. You can make A sit. Why? Between these two, how much gap is there between seat 4 and seat 7? Two seat gap. It works. Two seat gap. It works. Move forward."
    },
    {
      "start": 399.76,
      "end": 428.56,
      "text": "S8, now you will get more seats, because all the kids are not finished yet. This one is finished, but this one kid is remaining. Can you make this kid sit on this seat? Absolutely, you can make him sit comfortably. Why? Because again, there is a gap of 2 seats between them. How many seats did you get? How many seats did you get? 1, 2, 3, 4, 5, 6, 7, 8. You have to return 8. Minimum number of seats to fix all the students, so that there is a gap of at least 2 seats between them. Whatever gap you are given, there should be that much gap."
    },
    {
      "start": 428.75,
      "end": 455.62,
      "text": "So here the question differs. So according to me, the previous question if the
```

```
lead code was medium, this question, although it is medium, this question is quite hard. There are two things here. First, you have been given a gap that there should be at least this much gap between two children. After that, you do not have any constraint of seats. It is not that you have been told that you will get only six seats. Because there are six children, you will get six seats. No, there are six children, you will not get six seats. Use it. You have used eight seats here. You have to tell that you can sit in a minimum number of seats."
    },
    {
      "start": 455.62,
      "end": 479.04,
      "text": "If you wanted, you could have used 100 seats. But what is the minimum number of seats? Can you sit in less than 8 seats? No, you can't. So you have to return 8 seats. So this is the premise of this question. Now let's start thinking how to solve this question. So see, what does common sense say? Common sense says the same thing that was said in the previous question too, that the child who is the most frequent,"
    },
    {
      "start": 479.04,
      "end": 507.01,
      "text": "If A is 3 times, then A is the most frequent in every step. So you should try to make him sit. In the previous video, we had done that the child who was the most frequent, then sir used to try to set him. If this is set, then the rest will also be done in some way. So in every step, one thing is that you should try to make the most frequent one sit. According to common sense, because what will happen is that if you will leave the most frequent one, suppose you had a lot of children and A was the most frequent. Now suppose B, C, D was once."
    },
    {
      "start": 507.01,
      "end": 534.64,
      "text": "B, C, D were one at a time, okay? And A was how many times? It was four times. Now, what will happen with this? Let me tell you. Assume that you are doing it with your own mind that no, what do we mean? We can make anyone sit, so we can make them sit. Now, what will you do? Where will you make B sit? On seat number 1. Is it set? Where will you make C sit? On seat number 2. Because B and C are different, so there is no concept of gap in this. Where will you make D sit? On seat number 3. So, three kids, three seats, work is done. But, now what do you have left? A."
    },
    {
      "start": 534.64,
      "end": 562.53,
      "text": "Okay? Assume that your gap is 2. How much gap should there be between every similar child? There should be a gap of 2 seats. You can make A sit on seat 4. Because A is sitting for the first time, so it's okay. Distance doesn't matter from D. For the next A, you won't be able to make the next A sit on seat 5. Nor will you be able to make him sit on seat 6. Because there should be a gap between every A, of at least 2. You will be able to make the next A sit on seat 7. Similarly,"
    },
    {
      "start": 562.53,
      "end": 591.25,
      "text": "Seat no. 8 and seat no. 9 will have to be left empty. Where will you be able to place the next one? Seat no. 10. Similarly, on the next 11 and 12, you will have to leave a gap again. Where will you be able to place the last one? On S13. You will be able to place it here. So see, this is 1A. 1, 2, 3, 4. How many total seats did you get? 13 seats. You got 13 seats. How many children were there? 4 and 3, 7. Out of 7 students,"
    },
    {
      "start": 591.25,
      "end": 613.49,
      "text": "You are taking 13 seats, which means almost double. Why did this problem happen? This problem happened because you filled BCD in the beginning. You thought BCD is good, lowest frequency, so you filled it in the beginning. You left A, you left the child who was the most, for the last. Now what happened by leaving it in the last, now you have only A left. You are taking approximately 1A for 1A, 3 seats."
    },
    {
      "start": 613.49,
      "end": 643.86,
      "text": "because of placing 1a, 3 seats are spoilt. See, he is spoiling 3 seats. He is also spoiling 3 seats. He is also spoiling 3 seats. He is doing it, isn't he? Yes. If you had already filled A, if you had already tried to fill it, then you would have needed less seats. We will show you. We will also tell you why that happens. See. That is why we are telling you because it is not necessary to remember that the frequency should not be increased. We will use common sense. Come, let's try. Place A. Where? On S1. Okay? The obvious thing is that A cannot sit in S2 and S3."
    },
```

```
    {
      "start": 643.86,
      "end": 669.63,
      "text": "Next, place A in S4. It's obvious that A will not be able to sit in S5 and S6.
Place A on S7. It's obvious that A will not be able to sit on S8 and S9. Place A on S10. There
are four A's. One, two, three, four. We have completed the maximum number of A's. Now, what
all do you have left?"
    },
    {
      "start": 669.63,
      "end": 698.96,
      "text": "B, C, D. All three are one at a time. You can make all three sit anywhere. B, C,
 D. That's it. How many seats were there in total? 10 seats. How many were there above? 13
seats. Why were there 13 seats? Because you had left the most frequent one for last. Now in
order to make him sit, you had to keep more seats idle. Every A had to keep 3-3 seats idle.
But what happened in this? If you look here in a way, every A is making a group like this. He
is making it, right? Every A has to make it. But, but, but, this A 2 seat,"
    },
    {
      "start": 698.96,
      "end": 725.46,
      "text": "You had made it idle, here you have entered B and C. This second one, the one
seat that was made idle, you have entered D there. So you have reduced the idle seat. So the
less you reduce the idle seat, idle seat means empty seat. The less the empty seat will be,
the less the total number of seats will be. That's why common sense tells us that we should
start with the most frequent one. And the most frequent one should be filled at every step.
Now let's see how we will fill and what we will do to fill it. Let's go."
    },
    {
      "start": 725.46,
      "end": 754.1,
      "text": "Now what we will do is, we will think like a master. Suppose you have a class
like this. There are 3 children of A and 3 children of section B. And your gap is 2. That
means, there should be a gap of 2 sets in every similar child. So first of all, as we did in
the reorganized string, let's remove their frequency. First of all, what should we do? In a
question like this, where you need frequency, remove the frequency in the HashMap. So what
should we do? A is 3 times and"
    },
    {
      "start": 754.29,
      "end": 784.1,
      "text": "D is three times. You have taken out a frequency. Now, in common sense, what
did we think? Which child should we try to set first? The most frequent one. The most frequent
in every step. What do you want in every step? The one whose frequency is the highest. What
does this mean by shouting? Heap. Because, which child do you want in every step whose
frequency is the highest for now? So, what does it mean? Heap. Make a heap. What is this? Max
heap. What do you want? You want frequency."
    },
    {
      "start": 784.1,
      "end": 809.84,
      "text": "And what is the frequency of that child? What is the frequency of that child?
What does it say? One is int and one is char. What is its? Pair. What kind of heap? We want
max frequency. Max heap. If the frequency of two children is same, then it doesn't matter.
Take anyone. So let both be max. Let both be max in first and second. So what kind of heap do
you want? Max heap. Why? We want maximum frequency child in every step. That's why we want max
heap."
    },
    {
      "start": 809.84,
      "end": 834.22,
      "text": "What will be stored in maxheat? Frequency and child. So what will we store? 3
and A and 3 and B. What is this? Maxheat is ready. Is it correct? Okay. Now let's try to fill
seat by seat. What do we do after doing one by one? Let's try to fill seat by seat. We were
doing the same last time too. Because what will PT sir do? Try to fill seat 1. Come."
    },
    {
      "start": 834.96,
      "end": 865.68,
      "text": "Try to fill seat 1. We will take a variable seat. We will also see that
implementation. We are just explaining to you. We think like this in our mind. Try to fill
seat 1. Who will we try to fill in every seat? The maximum one whose frequency is maximum. So
in seat 1, you have the first child. Do hip pop. So what is the first child? Its frequency is
3. And your child, what is that child? It is A. Okay. Done. Your first child popped."
    },
    {
      "start": 865.68,
```

```
      "end": 893.01,
      "text": "This person has popped from here. Very good. Now, a decision comes. Can you make this child sit on this seat? Can you make this child sit on this seat? How will we know that? In the previous question, it was very easy. In the previous question, our reorganized string, where did it go? In the previous reorganized string, it was very easy to do this. Why was it easy to do this? Because you used to take an array of a variable, a string. And if you wanted to see, let's say,"
    },
    {
      "start": 893.01,
      "end": 922.27,
      "text": "whether we can place it on index 3 or not. So, you just checked that my res is not equal to this character, so you can place it. But it is not that easy here. Here, suppose you are in seat 1, then you will say that you can place it in seat 1, but suppose you are in seat 10. Suppose, just for example, you are in seat 10. And your gap is 5. Now, here you can place A or not."
    },
    {
      "start": 922.27,
      "end": 950.19,
      "text": "To see this, you have to see everything. You have to see 9, 8, 7, 6. You have to see all of this. Why? Because if A is sitting here, then you can't make it sit on the 10 because there will be a gap of 3. Sorry, you can't make it sit on the 5. Suppose if A is sitting on the 5, then can you make it sit on the 10? You can't make it sit. Why? Because then how many seats will there be here? There should be a gap of 4 seats. How much should there be of 5 seats? If A is sitting anywhere here,"
    },
    {
      "start": 950.77,
      "end": 975.54,
      "text": "from seat no. 5 to seat no. 9, if A is sitting anywhere, then you can't make it sit on the 10. So, it's hard to check this. Now, will you have to run a loop for this? Run the loop of the gap from the present index and then check, then it will be very difficult. We don't want to see so much difficulty in our life. To keep life easy, because you understand this problem, right? What is the problem? Can A sit on this seat or not? How will you know? Now, you can see the gaps in the back."
    },
    {
      "start": 975.54,
      "end": 1004.86,
      "text": "plus 1, whatever is there, the 4-5 seats at the back, whatever you want to see, you will have to see that. If A is sitting there, then it cannot sit here. But see how we will do a trick, we will do a small trick. What we will do is, we will maintain a register. PT sir has a copy, we maintain a register in that. What will we write in that register? What will we write in that register? That A and B, we will make a hash map of both of them. We will say that this is our register. What will we write in this? A can sit in the next seat."
    },
    {
      "start": 1004.86,
      "end": 1030.72,
      "text": "Which seat can A sit on? Which seat can A sit on? Which seat can A sit on? Which seat can A sit on? Which seat can A sit on?"
    },
    {
      "start": 1030.72,
      "end": 1059.76,
      "text": "Seat is at 10, we are trying to make A sit here, but it is written in the free one that A can sit at 12. No one can sit before 12. Now how this will happen, we will see that too. But suppose someone tells you that A can sit at 12, no one can sit at less than 12. So you will immediately know that 10 is less than 12, no one can sit. If it is written here that A can sit at 10, then definitely sit. If it is written here that A can sit at 8, that means A can sit at seat 8 or anyone can sit ahead of 8, then definitely sit. 10 is greater than 8, so this can definitely happen."
    },
    {
      "start": 1059.76,
      "end": 1087.34,
      "text": "Let's assume that in the beginning, every child can sit on seat 1. Anyone can sit on seat 1 in the world. So, on seat 1, your frequency is 3 and child is A. Your child is on seat 1. Can A sit on seat 1? Yes. How did you know? What is free of A? Free of A is 1. So, is free of A greater than or equal to this seat? Yes."
    },
    {
      "start": 1087.34,
      "end": 1116.61,
      "text": "Free of A, what? Greater than equal, greater than or less than? Less than equal to, sorry. Less than equal to seat. Less than equal to seat. Means, A can sit on 1 minimum. Is
```

```
the seat you are trying to make sit bigger or equal to this? Yes, it is equal to this. Very good. Now, what will you do? Now, what will you do? We will make it sit on the seat. We will make it sit on the seat. So, how will we make it sit? First of all, suppose you make A sit on this seat 1. First of all, we will have to reduce one frequency."
    },
    {
      "start": 1116.61,
      "end": 1145.55,
      "text": "We will have to decrease the frequency of A. And suppose after decreasing the frequency, if it would have become zero, then we would not have pushed it in the heat. Because if it becomes zero, then A will be finished. But after decreasing the frequency, it is 2. We will push it back in the heat. What will we do? 2 and A. We will push it back in the heat and move forward. We will move forward. You have placed A on seat 1. Yes, one more thing. Now A has sat on seat 1."
    },
    {
      "start": 1146.13,
      "end": 1174.21,
      "text": "A can sit on which seat? Can it sit on 2? No. How many gaps should there be? 2. No. Can it sit on 3? No. How many gaps should there be? 2. No. Can it sit on 4? Yes, it can sit on 4. Because if you put A here, then there will be a gap of 2. What is the next minimum seat on which it can sit? The seat on which you are now. Seat is equal to 1 plus gap."
    },
    {
      "start": 1174.21,
      "end": 1204.08,
      "text": "plus 1 is equal to 4. So what we will do is, the free register that we have made, this free register that we have made, we will also update this. What will we call this? Free A is equal to whatever was the current value in this, whatever was the current value in this, plus gap plus 1. So A can sit on this. After that, another very important thing that we are saying to do is,"
    },
    {
      "start": 1204.08,
      "end": 1232.93,
      "text": "How did you know whether it will be correct to place A on seat 1 or not? By reading this free diary. You will have to copy this diary too. Where have you placed A? On seat number 1. Now if he has sat on seat number 1, then where will he be able to sit? If A has sat on seat number 1, then you have placed A on seat number 1. Can he sit on seat number 2? No, he cannot. Why? Because there should be a gap between the two seats. He cannot sit on seat number 3."
    },
    {
      "start": 1232.93,
      "end": 1261.71,
      "text": "Seat 4, yes, you can make A sit here. You can make him sit, you don't have to make him sit. You can make him sit on the minimum seat. You can make him sit on 4. Why? Because there will be a gap of 2. So, what will happen? The seat on which you have just made A sit, 1. Plus gap, plus gap, because he has left the chairs, plus 1 on 4. So, while going, what do you have to do? Free A, which is your child, is equal to, wherever you have made him sit, plus gap, plus 1. So, what will happen? 4."
    },
    {
      "start": 1261.74,
      "end": 1293.5,
      "text": "Where can A sit next? 4. This will be useful when we go ahead. When we check on seat 2, then it will be useful. Come, let's move ahead. Seat 1 is done. Let's move to seat 2. Let's see what we will do on seat 2. We will do something. Come, let's see. Seat number 2. Every time, who should try to sit? The one with the most frequency. Whose is it? 3B. It's not 2. It's not A. It's B. Come. Pop this. And let's see. So, what did you get? Your frequency is"
    },
    {
      "start": 1293.5,
      "end": 1318.7,
      "text": "3 and your child or your task is B. Can you make B sit on seat 2? How will you know? B can sit on which seat minimum? On 1. You can make B sit on which seat minimum? On 2. You can make B sit on which seat minimum? You can make B sit on which seat minimum? You can make B sit on which seat minimum?
```

```
      "end": 1343.95,
      "text": "What will we do? We will reduce its frequency. If its frequency had become zero,
 there was no need to push it in heat. But its frequency did not decrease, it did not become
zero, which means B is still left, so you will have to push it again. Come on, push it to B.
And the most important thing, while going, where have you placed B? On seat number 2. Where
have you placed B? On seat number 2. B has sat here. Can it sit on seat 3? No, there should be
a gap."
    },
    {
      "start": 1343.95,
      "end": 1373.02,
      "text": "Seat 4? No, there should be a gap. Seat 5? Yes, it can be. How did you know? 2,
on which you have placed B, plus gap, plus 1. 2 plus 2 plus 1, 5. So what will we do? We will
upgrade free B. How? Free B is equal to whatever seat you have placed on, on the second seat,
plus gap, plus 1. So, where can B sit next? On 5. It was very good till here. You have placed
both the people on both seats, A and B."
    },
    {
      "start": 1373.02,
      "end": 1403.02,
      "text": "Both have worked. From the next seat, seat 3 will become complex. Now open your
eyes and nose and see. Now it will become complex, see. Now see. Now you will not be able to
sit anyone, see. Come, let's try. Seat 3. We have come to seat 3. First A sat, B sat, now we
will sit this. From the hip, who will we remove now? From the hip, who will we remove? If you
see according to the max, then both have the same frequency. Who is bigger between A and B? B
is bigger, so this one will be removed. So remove this."
    },
    {
      "start": 1403.02,
      "end": 1434.0,
      "text": "Take this child out and try to make him sit. What did you find out? There is a
child who has two children left. And what kind of child is this? This is B. Okay. Can you make
him sit on seat number 3? You can't make him sit. How did you find out? Free B. Free of B. It
is 5. You can't make B sit on any seat less than 5. It is greater than your seat. You can't
make him sit. Are you stuck? You are stuck, right? You can't make him sit. What will you do
now? What did you do last time in reorganizing?"
    },
    {
      "start": 1434.0,
      "end": 1463.57,
      "text": "Last time, we used to see that if we can't make him sit, we used to take out
the next max, make him sit, and push him back. This is what we used to do. Last time, if you
haven't watched it, then watch it. In the last reorganized stream, because there was an
alternate, so if A can't sit, then B can definitely sit. That used to happen. Because, what we
used to do, we used to take out another max, whoever it was, and make him sit, and make him go
inside. But, for a little while, for a little while, let's assume that we, this man, this 2B,"
    },
    {
      "start": 1463.57,
      "end": 1492.64,
      "text": "send it somewhere, tell it to stand in the corner, we will see you now, don't
push in the heap, if you push in the heap, then A will come out in the max, then nothing will
happen, okay, tell it to stand in the corner, 2 and B, nothing has been done, no change has
been done, if you know that you can't make it sit, tell it to stand in the corner for a while,
we will see you now, we will handle you, take out the next max from the heap, take out the
next max, what came out, 2A, your heap is empty, your heap is empty,"
    },
    {
      "start": 1492.64,
      "end": 1511.25,
      "text": "What does 2a mean? Its frequency is 2. What child is this? A child. Can you
make A sit on seat no. 3? No, you can't. A will sit on seat no. 4. He can't sit on seat no.
less than 4. He can't even sit on seat no. A. Now you are stuck. What used to happen in the
past? Someone or the other used to sit."
    },
    {
      "start": 1511.25,
      "end": 1540.19,
      "text": "both can't sit now you can't give this seat to anyone neither one nor two this
seat is going to be empty now what will you do now you are stuck now take out both you are
stuck there is nothing then what will you do now let's see ask him to go too stand in the
corner now what can we do ask him to go too stand in the corner there will be a corner
anywhere ask him to stand in the corner with your hands up now we will handle you try to take
out the next but try to take out the next your hip"
    },
    {
      "start": 1540.19,
```

```
      "end": 1567.9,
      "text": "your heap is empty now you can go out now you can go out now see what happened
on seat no. 3, can you make any person sit? no you can't because now take out A because there
were only two people there were only two people in the whole class by counting take out A, A
can't sit sorry take out B, B can't sit take out A, A can't sit now there is no one left in
the heap to do anything there is no one left in the heap to do anything now what will you do?"
    },
    {
      "start": 1567.9,
      "end": 1587.09,
      "text": "This means that your seat is going to be empty. Seat no. 3 is idle. As we saw
earlier, seat no. 3 used to be empty. I have shown it somewhere. See this. Seat no. 3 used to
be empty in the upper seat as well. But, but, but. Okay, if it is empty, then there is no
problem. If it is empty, then okay, it is empty. But the two people who you have taken out and
put in the corner,"
    },
    {
      "start": 1587.09,
      "end": 1611.9,
      "text": "you have to push these two people back into the heap because they are still
there you can't let them go anywhere you have to make them sit so after this suppose this
corner is a vector or an array of pairs this is just a corner where you take out those people
who are not able to sit and store them for some time so that they can be stored for some time"
    },
    {
      "start": 1611.9,
      "end": 1630.66,
      "text": "After doing this, you have to see this whole data structure, this vector. And
whatever is in it, as it is, you have to insert it here. As it is, you have to insert it in
the heap. These two, 2B, 2A, insert both of them here. 2B, 2A, insert both of them here."
    },
    {
      "start": 1630.66,
      "end": 1652.56,
      "text": "as it is, the work of seat number 3 is done, no one could sit on seat number 3,
there is no difference whether they sit or not, as it is, nothing has changed in this round,
in this round neither the free change of A and B, nor the change of A and B frequency, means
this round is completely useless, your seat number 3 is empty, come ahead, come ahead, come to
seat number 4, come to seat number 4,"
    },
    {
      "start": 1652.98,
      "end": 1684.69,
      "text": "Let's hope that you understood this. Because this seat is completely empty.
That's why we have done this corner work. The one who is not able to sit, we will tell him to
go to the corner. Okay, we will insert you later. We did this in the previous also. We did
this in the previous also. We did this in the previous also. We did this in the previous also.
We did this in the previous also. We did this in the previous also. We did this in the
previous also. We did this in the previous also. We did this in the previous also. We did this
in the previous also. We did this in the previous also."
    },
    {
      "start": 1684.75,
      "end": 1714.83,
      "text": "Can you make B sit on seat no. 4? No, you can't because he will sit on seat no.
5 and not on seat no. 4. So, you can't make him sit on seat no. 4. What will you do? You will
tell him to go to the corner. You go to the corner. Whoever you are, you also go to the corner.
 He is stored in the corner. Okay. This man has left the heap. Take out the next child. Try
that because we are trying to take out the one whose frequency is the highest. Try the next
one. Frequency is equal to 2 and child is equal to A."
    },
    {
      "start": 1714.83,
      "end": 1739.92,
      "text": "Child is equal to A. This person has also left from here. Okay. Can we put A
here? Yes, we can put A here. Very nice. Why? A's free, I'm sorry. How much is A's free? 4. 4
can sit on 4. So, put it here. First of all, what should we do? Let's assume that it sits on
seat number 4. It sits on seat number 4. Its frequency is 1. Now, it is not 0. So, push it as
it is. Push 1A. Push 1A here. And,"
    },
    {
      "start": 1739.92,
      "end": 1769.89,
      "text": "Which seat are you sitting on? On 4. Which should be the next free seat? 4 plus
gap. 4 plus 2 plus 1. 7. 7. 7. 7. Okay. Done. Now, it should be done. This is done. The work
of seat 4 is done. But, after this, what should we not forget? Why didn't we tell this in seat
1 and seat 2? Because no one had gone in the corner. The first person had sat. But, after this,
```

```
 you have to see. Is anyone standing in the corner? Yes, 2B is standing. 2B had left earlier. 2B has not gone anywhere yet. 2B, as it is, enter."
    },
    {
      "start": 1769.89,
      "end": 1797.71,
      "text": "As it is, no processing of 2B has been done. Neither the frequency nor the frequency of 2B is over. As it is, put it inside. Come next. Then go. We are taking so much time in this because it is a very important understanding to make. Okay? That's why we are working so hard and doing dry run. Seat number, on which we were sitting? 5. On seat number 5, what person will come out? 2B will come out. Frequency is equal to 2. Child is equal to B."
    },
    {
      "start": 1797.71,
      "end": 1827.62,
      "text": "Can B sit on 5? Yes, it can sit because if you remember, it was 5. Make it sit. Make it sit means reduce its frequency. Make 1B as it is. Make 1B as it is. Till now, remember that no one is standing in any corner. The first person who came out was sitting. Okay. Free B. Free B has to be upgraded. If you have made it sit on 5, then the next one can sit on it. 5 plus 2's gap. 7 plus 1 can sit on 8. Make it 8. Make it 8."
    },
    {
      "start": 1827.62,
      "end": 1859.54,
      "text": "Is there any person standing in the corner in this round? Was there any person standing in the corner in this round? No, there was only one person standing in the corner. Don't do anything, if there is no one in the corner, then there is nothing to do. Go ahead. Go ahead. Go ahead. Seat number 6. What will come out on seat number 6? 1B. Take out 1B again. Frequency is equal to 1. And child is equal to B. Pop this. Now see. Can you make B sit on seat number 6? No, you can't. Why? B can't sit in less than 8."
    },
    {
      "start": 1859.54,
      "end": 1887.28,
      "text": "What will he do? He will say, you go to the corner for a while. For a while. Go to the corner, we will handle you in a while. See the next kid, he has the highest frequency. 1A. Pop it. 1A. Frequency is equal to 1. What is child? A. Can you make A sit on seat number 6? No, you can't. What is free? 7. He can't sit in less than 7. He will say, go to the corner, we will see you in a while. We will see you in a while."
    },
    {
      "start": 1887.28,
      "end": 1917.18,
      "text": "Try to take out the next one. The heap is over and there is no child. What is this seat going to be? This seat is going to be idle. This seat is going to be useless. It doesn't matter. Let it be. What can we do? We don't have it in our hands. The children who were standing in the corner, as it is, put them in the heap. 1B, 1A. As it is, put them in. Which seat went idle? Seat number 6. Look at the example above. What was seat number 6? It was idle, really. Really, it means they are going right. Come, let's go to the next seat. Seat number 7. Until the heap is empty, you have to do this."
    },
    {
      "start": 1917.18,
      "end": 1947.02,
      "text": "till all the kids are finished, till then you have to do this. Seat number 7, remove someone, remove 1B. Remove 1B, remove it, pop it. Can B sit on seat number 7? No. What is the free of B? 8. It can't sit at all. It can't sit at all. So what will we say? Brother, you go to the corner, we will see you in a while. We will see you in a while. Next, come. 1A, remove this, it has a frequency. 1A."
    },
    {
      "start": 1947.38,
      "end": 1970.02,
      "text": "Can A sit on 7? Yes, it can sit. What is A's free? 7. So 7 is greater than or equal to 7. Okay, you can sit. What do we have to do? First of all, reduce the frequency to 0. If the frequency is 0, then the work is over. You don't have to do anything else. You can update it for free or not. There is no point because if A is 0, then we are not inserting it back on the sheet because it is 0. There is no point. If you want to update it for free, then do it. It doesn't matter."
    },
    {
      "start": 1970.02,
      "end": 2000.5,
      "text": "It's finished. Who sat on seat number 7? A came and sat. After doing this, was there anyone in the corner? Yes, this person was in the corner. Put it back. As it is, put it back. Okay. Next M is here. Seat number 7 is filled. Come to seat number 8. Now come to seat number 8. Keep increasing the seats. Until everyone sits, until the heap is finished, you have
```

```
to keep increasing the seats. Seat. 8. What is on top? There is only one person. 1B. 1B. Okay? There is only one person. Remove it."
    },
    {
      "start": 2000.56,
      "end": 2018.8,
      "text": "Can B sit on 8? Yes, B can sit on 8 for sure. What will we do? 0 frequency of B,  B sat on 8 and B's frequency is 0. If the frequency is 0, will we insert it in the heap? No, we will not insert it at all. And it's done. You can update it for free or not, it's up to you. "
    },
    {
      "start": 2018.8,
      "end": 2049.2,
      "text": "Now, is there any person standing in the corner? No, there is no one standing in the corner. So, we will not enter anything. Okay, come to the next. But, you get to know that now your heap is empty. Heap is empty means there is no child. All the children are finished. Now, which seat did you finish at last? Seat number 8. What will you return? 8. You will get 8 seats to make these 6 children sit. See the diagram above. Did it work in 8 seats? Yes, it worked. And it must have worked in this order. You can see from this diagram."
    },
    {
      "start": 2049.2,
      "end": 2080.78,
      "text": "Now let's go to write its code. It is a bit of implementation heavy code. So bear with it. Come let's see its code. So you must have given two things. You must have given vector int a in which you must have given that task. Not vector int sorry vector of char. Vector of char. Vector of char and you have given an int gap. Okay. First of all let's do the work of frequency. So int n is equal to a dot size. First of all do the gap of frequency."
    },
    {
      "start": 2081.26,
      "end": 2111.78,
      "text": "So let's take a hash map for frequency. Unordered map of this type of character and integer. This is the frequency. And let's take a free one too. Because you have to keep it free at the end. When is every person free? Yes, look at this. It doesn't matter how many number of tasks are there. Number of unique tasks can be 26. Because ABCD is up to Z. ABCD is not there. So every task is ABCD."
    },
    {
      "start": 2111.78,
      "end": 2143.15,
      "text": "A, B, C, D can be a maximum of 26. Don't worry that we are taking out the same thing again and again. It is 26, not more. For I is equal to 0, I less than N, I plus plus. We will do two things. First, we will increase the frequency of this task. And in the beginning, which seat can the free of this task sit on? On 1. Everyone can sit on 1 in the beginning. Very good. After that, what will we do? We will take a heap. How?"
    },
    {
      "start": 2143.98,
      "end": 2178.26,
      "text": "Let's write it here. Let's write here. Let's take the heap. We have to take the max heap of pair. Priority. Q. Pair of. What kind of? Integer character. Please note that we made a mistake in coding. So, you won't know why it will be a mistake. Okay. What do we have to put in this? The frequency of all and that task. So, for auto ifreq. Let's do it in the frequency hash map. And pq.push. Let's do it in C++. Let's see in Java."
    },
    {
      "start": 2178.26,
      "end": 2211.41,
      "text": "what we have to do? i.second means first its frequency then that letter hip is ready now what we have to do? seat, from which seat you have to start? int seat is equal to take it from 1 now come, now we have to start filling seat by seat till when we will fill? while pq.not empty till the time this pq is empty we will fill"
    },
    {
      "start": 2212.02,
      "end": 2241.33,
      "text": "What is your goal for every time? Your goal is to find a child who can sit on the seat. And it is not necessary that you get it in the first time. It is not necessary that you get it in the second time. It is not necessary that you get it in the third time. It is possible that you will have to find 3-4 people and 3-4 people will have to stand in the corner.  Only then you will get something. First of all, let's make that corner. We had said that we are sending the child in the corner. Vector of pair of intent care. Whatever will come out of the hip, if it is not of our work, then we will send it in this corner. Call it pulled."
    },
    {
```

```
      "start": 2242.77,
      "end": 2274.54,
      "text": "Whatever you are pulling or popping, we will send it in this corner. This is
the corner. It was here. There was a corner here. This is the corner. No, where was it? This.
This one. Where we were sending it in this corner, this is the corner. Okay. Keep taking out
the children. Keep taking out the children continuously. See. Write one more. while pq. empty
We are writing a while inside this while. Why are we writing? Let me tell you. Take out one
child. Take out one child. How? Pair of"
    },
    {
      "start": 2274.58,
      "end": 2307.47,
      "text": "int char p is equal to pq.top Don't forget to pop pq.top. Now you have to see
if this child can sit here or not. So if, free of, what will be its task? Let's write the task.
 This child has come out. So what is its frequency? p.first. This child is so many times. And
which child is this?"
    },
    {
      "start": 2307.86,
      "end": 2331.09,
      "text": "This child is P dot second. Can this child sit on this seat? Let's see. If free
dot, the child of free, can sit on which seat? If it is less than or equal to seat, then it
can sit here. What did it do in seat? First of all, let's assume it sits here. If P dot first,"
    },
    {
      "start": 2331.6,
      "end": 2361.49,
      "text": "greater than 1, means the frequency you have taken out, suppose if its
frequency is 1, if its frequency is 1 now, then there is no need to push it, there was one
child, one sat, if it is greater than 1, then what will you have to do? pq.push, push it back,
with whom? p.first with minus 1, we have made one child sit, minus 1 and p.second will remain
the same, and change its free also, free of p.second,"
    },
    {
      "start": 2361.78,
      "end": 2391.86,
      "text": "is equal to the seat on which he is sitting plus gap plus 1. Okay? And if he
sits, if he sits, break this loop. The second loop is the checking loop. This is the checking
loop. After one, a kid is picking it up. Until he finds someone, as soon as he finds someone,
break this. Break this. This is the if. If he can sit, break this check-in loop. But if he
can't sit, what will we do? Else,"
    },
    {
      "start": 2391.86,
      "end": 2419.12,
      "text": "If he can't sit, what would he do? What would the kids say? Go play in the
corner. The pull we had, this pull we had, push, back, P. This L has stopped. This check while
has stopped. Checking while. Checking while. Which while? Checking while. The one who was
checking continuously. This while has stopped. Now when this while has stopped, now when this
while has stopped, two things have happened."
    },
    {
      "start": 2419.12,
      "end": 2442.85,
      "text": "Either a child would have sat down due to which we wrote this break and came
out, or the heap was completely empty and no one could sit in it. This whole seat went idle.
Both scenarios are fine. In both scenarios, one of your seats is lost. Either a child would
have sat on one of your seats. If a child has sat, then we put him back inside by doing
frequency and freeing him. If a child is not sitting, then it is fine, then this seat will be
empty. What can we do?"
    },
    {
      "start": 2442.85,
      "end": 2473.15,
      "text": "Come out means try on the next seat but Before trying on the next seat We will
put the kids who were standing in the corner back in the seat without doing anything for i is
equal to zero i less than pull to dot side It is possible that no kid has come No problem, we
will put them back as much as they have come PQ dot push as it is will put Because we haven't
done anything with those kids We didn't make them sit, we didn't do anything for them Now seat
plus plus Seat plus plus Next time"
    },
    {
      "start": 2473.15,
      "end": 2484.98,
      "text": "We'll see in the next one. And the main while loop is closed. When will the
whole game end? When this while loop will come to know that the PQ is empty. What do you have
to return in the end? Seat."
```

```
    },
    {
      "start": 2485.46,
      "end": 2514.34,
      "text": "In last, you don't get seat. Seat minus 1. Why seat minus 1? Because according
to the previous example, suppose in seat number 8, you made the child sit. Now you made the
child sit and come out. But you do seat plus plus one more time in last. So when you will exit
in last, then this seat will be 9. When you will exit, then this seat will be 9. That's why we
have seat minus 1. So seat will be 8. Now you were going to go on 9th seat, but now it is over.
 That's why we have done 8. So see this is a very good code."
    },
    {
      "start": 2514.34,
      "end": 2537.65,
      "text": "We will give you the code today. It is a very good code. We will not say that
it is a simple code. It is a very implementation-heavy program. If you get it in an interview,
it will take 10 minutes to write this code. It will take 10 minutes even in typing. So, this
was the code. We thought that this was a very good question. Rest, we will read the code
schedule when we do the recursion."
    }
  ]
}
```


27. transactions/t_8.json


```
{
  "video_id": "2HZ12B2ZPAQ",
  "title": "DSA PATTERNS Episode 13 Sliding Window Pattern: Important Questions",
  "language": "en",
  "model": "large-v3",
  "duration": 1947,
  "segments": [
    {
      "start": 0.0,
      "end": 26.91,
      "text": "Today is the last day of the sliding window pattern. After this, the sliding
window will be over. But before that, listen to an important thing. It will take 2 minutes. 30
minutes ago, we uploaded a video. It was not a lecture of DSA. It was a different type of
video. It was not that we are dancing in it. But it was a little like how you should study and
why you are wasting time topic wise. Whatever it was, you can go and watch it. It was not a
wrong thing. Now, what happened in that video? Two things happened. One was the views in that
video."
    },
    {
      "start": 26.91,
      "end": 54.88,
      "text": "We will show you a screenshot of it. 1 out of 10 means that is the best video
in 10 videos. There were a lot of comments saying that you are doing this and that by talking
about DSA videos. Don't make videos like this. Which is a good thing because we have said in
the last video that you can comment as you like. But where does the problem happen? Because it
is important to say all this because you may be kids and you don't understand."
    },
    {
      "start": 54.88,
      "end": 81.63,
      "text": "There is a man named Arvind Yadav. You can see his comment, it will still be
there. In his comment, he said, Hey, teach DSA, why do you keep blabbering? If you start
commenting like this, that why do you keep blabbering, teach DSA, then, then see, whatever we
teach you, then you will have to keep some trust, that if we say that we will teach, then we
will teach. Now, we will teach on the channel,"
    },
    {
      "start": 81.63,
      "end": 110.86,
      "text": "Apart from this, if we upload any other video, then it's my channel, right? We
might feel like dancing, we might feel like singing, so we are uploading. The video of
studying is coming. Next time, you will start talking like this, that you should study from
DSA, why do you keep blabbering, why do you keep blabbering? So, nothing will happen with all
these comments. What will happen with all these comments is that we don't teach by writing a
script. I don't have any script in front of me that we say that script here. Whenever we teach,
 we look at the question and teach whatever comes to mind. And you say that you teach well."
    },
    {
      "start": 110.86,
      "end": 140.08,
      "text": "By commenting like this, I will also get some irritation in my mind. Don't be
```

```
thankless. Do you know what thankless means? In Hindi, it is called as Kritagn. Thankless. We don't care about it. We should have what we want. Otherwise, everything is wrong. Don't comment like this. And there are very few students who comment like this. Most of the people comment like this. If you are commenting like, Brother, you have put DSA. What are you doing? It is a very good comment. We have never said anything in it. Nor will we say. But when you will put such a comment, they will say, What are you talking about?"
    },
    {
      "start": 140.08,
      "end": 167.49,
      "text": "There are very few kids like this, so this is a good thing, but we don't like kids like this. This is a public channel, so we can't say that don't watch my video, but if this was my house or if this was my private batch, then we would say that you should go, I mean we don't want you. And this is the entire comment history of Arvind Yadav ji. In the YouTube studio, we get to know what comments he has made till now. Ironically, till now, his 4 comments,"
    },
    {
      "start": 167.49,
      "end": 197.07,
      "text": "they all are commenting on the same type of video on which they are talking about something different from the lecture they like to comment but they don't comment on any lecture that they have read today or there is any doubt in this or they understood this or they didn't understand this they don't have any comment on that means they are just watching this type of video and they are commenting by watching this type of video that don't make this type of video this person is not watching the DSI video so what you are not watching if you are not watching the Gyaan video then Gyaan video will be made so you comment on that but"
    },
    {
      "start": 197.07,
      "end": 226.72,
      "text": "take care of that, nothing goes more than this, nothing goes for me, we don't know you, you don't know us, so nothing goes, mine definitely doesn't go, it's a good thing, so it was a small thing, take care a little bit, that it is online, you think that we will say anything online, we will write anything, what will someone do, no one will do anything, but it doesn't look good, right, so take care a little bit, and what will happen with this, the more you will edit, the lower my teaching quality will go,"
    },
    {
      "start": 226.72,
      "end": 255.28,
      "text": "Isn't it? That's it. That's it with the Gyaan Prachar. Right now, a lot of kids watch only this much. They watch only 3 minutes. Then they will comment that you were teaching Gyaan and you were studying D.S.F. So, keep some patience. Your age is still the same but it's okay. The two questions that we are going to do today are very very important questions. My request to you all is that when we do one question, pause the video at the same time and try to do the second question yourself. Like what happened yesterday? Yesterday we did 3 questions.
"
    },
    {
      "start": 255.28,
      "end": 282.13,
      "text": "In 80% of the comments, it was said that we have done only one and we have studied only one. And the second and third question was done by ourselves. This is what we want, that you should read one question, it is a good thing, it is a new thing to read. But the second, third, fourth, which is the same question, you have to change one line or two lines, you do it yourself. We will do the same today. Today we are doing two questions. We will tell you the first question because it is a little new question. It is not probable that you do it yourself, there is very little probability."
    },
    {
      "start": 282.13,
      "end": 310.03,
      "text": "Second question, if you have done the first one, then the second one is equal to the basic questions. You should do the second question yourself. We will teach you that too, but try to do it yourself. Okay? Let's go. The two questions we are going to do today are longest substring with same letters after replacement, longest subarray with ones after replacement. The pattern sheet is in the description. Go and check it out. Row number 29, 29 and 30 in sliding window pattern. We are going to do this. Let's go."
    },
    {
      "start": 310.38,
      "end": 339.9,
      "text": "Now, let's come to our question. Longest substring with same letters after replacement. Now, after listening to the question, it seems very dangerous but it is not that dangerous. It is a little dangerous. So, listen carefully. Open the copy book, open your ears and ears and listen carefully. What did you say? That you have a string. Okay. And it is of an
```

```
integer. Very good. Now, what does this question say to you in Hindi? Read the question. What does it say in Hindi? That in this string, in this string,"
    },
    {
      "start": 339.9,
      "end": 367.65,
      "text": "By choosing any character, we can make it into any other character. Meaning, we can change it. Meaning, if there is A, we can make it into B, C, D, anything. Any character. Very good. But, but, but, this operation of changing, you can only do it K times. K times. This is a twist. Meaning, if there is K2, if there is K2, in the string,"
    },
    {
      "start": 367.65,
      "end": 396.08,
      "text": "We can change the order of two characters. Here is A and here is B. So, you make it C and make it D. We can do it twice. Not more than 2. We can do it as much as K. So, this is the question. What do we have to do in the question? Sorry. You have to find the length. Whose length do we have to find? Mine? No. You have to find the length of the longest substring. You have to find the longest substring."
    },
    {
      "start": 396.11,
      "end": 426.0,
      "text": "containing same letters same letters means same letters what conditions are you doing this operation? again you understand the question is complex this is not a traditional question means you see the question and you click like this that's why it is a hard question it is written hard on the sheet what is he saying?"
    },
    {
      "start": 426.0,
      "end": 453.63,
      "text": "If you have this string, there will be characters in the string. You have to find the longest substring in the string, which has the same letters. And to make it the same, the change facility that you have given, that you change the character, you can use that too. You have to use that K-bar to make the biggest substring. Okay? Means, assume A, B, C. It is like this, A, B, A."
    },
    {
      "start": 453.63,
      "end": 482.72,
      "text": "and here is C, D, E. You were told that you can change one letter. So, logically speaking, if we make it A, then see, this substring, this one, it has all A. It has a B, but we were told that you can make it. You can make B A. How many times? Once. Can you make this A too? No. Not together. You have to make only one. So, either make B or make this. So, if you make this A, this A,"
    },
    {
      "start": 482.72,
      "end": 512.18,
      "text": "then you have to take this substring whose length will be 2 but this substring's length is 3 you have to return 3 this is the question let's go the first thing is that what we always do pattern, flowchart, keep it ready in your mind whose question is this? what to do in this? I mean I have never heard such a question that you have to flip and you can do it many times, you have to change but is this a string question? array or string, whatever it is"
    },
    {
      "start": 512.18,
      "end": 541.04,
      "text": "String is also a type of array. Whenever we say string, you have to understand array. Yes. Do you have to find a window or a substring in this? Do you have to find a substring or an array in this? Yes, you have to find a substring. Do you have to find a longest substring? Do you have to find a longest substring? Yes. Is there a condition for that? That this should be in the longest substring. Yes, it is given. What is the condition?"
    },
    {
      "start": 541.04,
      "end": 570.67,
      "text": "Whatever letter is in the substring, either it is the same, or it is the same from before, or even if it is different, then you can change it K times. If you are different, then you can change K and make it. So this is our question. Those who don't remember the flowchart, then please see the previous one. Those who don't understand the flowchart, please see the previous one. Those who know the flowchart, they must have understood that this is the question of sliding window. It is the question of sliding window. Okay."
    },
    {
      "start": 570.67,
```

```
      "end": 600.4,
      "text": "This is an easy part. It is easy for you that it is a sliding window question. How many steps were there in the sliding window? There were 4 steps. Step 1. Step 1 was whether there is a question of sliding window or not. Yes, there is. Very good. What was step 2? Is it fixed or variable? Is it fixed or variable? Is it written anywhere in the question that the substring you will find should be of 3 length or 4 length? It is said that it is the longest substring. Spread it as much as you want. Save this condition. Whenever this happens,"
    },
    {
      "start": 600.4,
      "end": 631.41,
      "text": "Variable. Very good. What was step 3? Information. Information. What is information? Whatever window you make, whatever your low to high window will be made, what is its information? Now, what is information? Means, what do you check in that? In that, you have to check whether the whole character is one or not. So, it can be one, like we did in the previous question. In the previous question also, we had to check whether it is unique or not. Is it like this or not?"
    },
    {
      "start": 631.41,
      "end": 660.69,
      "text": "Right? So, what is its information? Its information is also, if we keep the frequency of characters in the hash map, then this information can be taken out. What you have to check is that in this substring, everything is A. Now, how will everything be A? Last time, remember, its length is 4 and its frequency is 4. Okay? So, if the size of this hash map is 1, what is the size of this hash map? It is 1. This means,"
    },
    {
      "start": 660.69,
      "end": 690.66,
      "text": "If the size of this hashmap is 2, then it is 2 elements. If it is 3, then it is 3 elements. It is 3 unique elements. Means, all the elements should be 1. We can't do it in such a way that 1 should be unique. 1 should be unique. This is the thing. A is unique 1. And A is 12. So, this is correct. Step 3. Sorry, step 4. How will the information go ahead? Same. Like, the low will be removed and the high will be taken. When we will remove the low, if it becomes 0,"
    },
    {
      "start": 690.66,
      "end": 719.15,
      "text": "If any frequency is zero, then it will be erased. This is the question of Dictocol. You don't have to do much thumb jam in this. You have understood this. Now, the most important part of the question has come. There is a template of the sliding window variable. If everyone remembers the template or not, then please go and see. What is the template? Low is equal to zero. High is equal to zero. Result is equal to, if it is longest, then int is equal to min."
    },
    {
      "start": 719.25,
      "end": 749.04,
      "text": "If it was shortest, then INP would have been maxed. Very good. First of all, what happens? The loop runs. Okay. From where does the loop run? And where does it run? Where does the high run? We increase the high in the loop. Okay. So write it. See. The template will remain the same. See. 70-80% template will remain the same. I am saying it again and again. The template. Watch the video from the beginning. If one video is missed, then it won't be fun.  Those who have watched the whole thing, they must be enjoying it. We are doing the same template question by question. Same is written here."
    },
    {
      "start": 749.04,
      "end": 777.86,
      "text": "Okay. What happened after that? After that, it was to take the information of high first. You are increasing the high, right? First, take the high inside. So, take it. Take the high inside. How will we take it inside? We will do it like this. What happened? Increase the frequency of the character on high. Very good. Now, what was the part? Till the time this information is wrong, till the time this information is wrong, till the time this information is wrong, till then, we have to increase the low."
    },
    {
      "start": 777.86,
      "end": 805.04,
      "text": "We have to increase the low and decrease the window. This is what we were doing since yesterday. What were we doing in this box? See, the whole game is of this box. It will be the same up and down, but it changes a little in the box. What was it? Till the time, information, till the time, what is it? Information is wrong. It is wrong. Increase the low. What should we do with the low? Increase it. And,"
    },
```

```
    {
      "start": 805.04,
      "end": 833.02,
      "text": "when you increase yellow check here if information is correct if information is
correct correct then remove len how to remove len? high minus low plus one and update len with
your result this is what we have been seeing since yesterday same pattern so the whole game is
of these two boxes"
    },
    {
      "start": 833.02,
      "end": 867.55,
      "text": "The whole game is of this part. The rest is the same from front to back. Come
on. Now come. Now see. Now see what question he wants to tell you. Come on. A, A, B, A, B, B,
A. A. I stopped. Today I don't know how to teach. Okay. A, A, B, A. This is an example. Okay.
0, 1, 2, 3, 4, 5, 6."
    },
    {
      "start": 867.55,
      "end": 896.0,
      "text": "This is given to you. The value of K is 1. Very good. Now we have to do this.
See. Suppose K is not there. Suppose K is not said in the question. It would have been said to
find such a substring. Find such a substring in which the whole letter is the same. If K is
not given, then it is possible that there is such a question too. That if K is not there, then
find such a substring in which the whole is the same. Then it was a very easy question. It was
a very easy question that the size of the hash map should be 1. Is it right? If the size of
the hash map is not 1, then the information is wrong."
    },
    {
      "start": 896.0,
      "end": 925.33,
      "text": "If the size of the hash map is not 1, it is more than 1, then it is wrong. It
will be 1. If it is more than 1, then keep increasing the low. And until the size of the hash
map is not 1, keep increasing it. As soon as it is 1, write it down. This is what would happen.
 But now you have given a K. K means you can change any one element again. You can change any
one element again. Now don't think like a computer. Think like a human. Think like a human."
    },
    {
      "start": 925.68,
      "end": 952.99,
      "text": "Suppose you have this window. This window is running now. Now you are on this
window. It is on this window. How many unique elements are there in this window? There are two
unique elements in this window. A and B. So is your information correct? No, it is not correct.
 So if it was not K, then it would have been wrong. Then it would have increased. Now K is
there. Think like a human. What will you do in this? If you are told that you can change one
thing in this."
    },
    {
      "start": 952.99,
      "end": 979.82,
      "text": "and change it in such a way that all the information is the same. So what do we
do? What do we do? We remove this B and write A here. If we had done this, it would have been
A and the answer would have been 3. We would have done this. We could have done something else.
 There is no point in making B C, there is no point in making B D, there is no point in making
B anything. There is a point in making B A. Right? Okay. Do you try to make A B?"
    },
    {
      "start": 980.14,
      "end": 1008.75,
      "text": "then it would not be possible because only one change is allowed then if this
one would remain then it would become BAB then it would be stuck so think here with your mind
open why not we are making A as B see what we are trying to say let's make it bigger you have
three A A B ok now see A A B this is your substring what we are saying you have to change A it
has been said that I have A ok"
    },
    {
      "start": 1008.75,
      "end": 1038.51,
      "text": "So, either you make B as A or make A as B. This is what happens, right? Look, A
and B are two elements. You have to make them all the same. So, either make B as A or make all
A as B. There are only two options. There is no use in making anyone as C. Now, will you like
to make A as B or A as B? How will you know? Why are we saying that we have to make B as A and
not A as B? So, how do you think? Look,"
    },
    {
      "start": 1038.51,
      "end": 1067.09,
      "text": "The K that I have is equal to A. If it is equal to A, then it is more
```

beneficial to make B as A, then you will get the answer. If you go to make A as B, then you will get stuck. So, how do you see this? See, A is twice here. In the whole substring, A is twice. B is once. So, if you have a limitation, which you have been told that you have K, K power, K times power, that you can change it again, then what will any intelligent person like to do? That what is less"
    },
    {
      "start": 1067.09,
      "end": 1096.74,
      "text": "It is more beneficial to change what is less, right? If it is K1 only, then if you start with more, then why will you start with more? It is not that we have to do everything, we just have to make B. So change what is less, why should we change what is more? Now see, do one more here. Assume that C is also there. C is also there once. Now what will you do? Now what will you do? Now there are three options in this. Now see, A, B, C, three letters have come. So either you make A as C,"
    },
    {
      "start": 1096.74,
      "end": 1127.09,
      "text": "Make both A's C and B's C. And let C be C. Or you make both A's B. Make both B's B. And let C be B. Very good. Or you make both A's A. Make both B's A. And let C be A. What are you doing in this? You will feel that this one is better. Why is this one better? Because A is the maximum. A is the maximum. So let A be A. Change the rest of the rest to A."
    },
    {
      "start": 1127.25,
      "end": 1151.94,
      "text": "What does it mean to say? Suppose there is a very big substance, in that 70% you know that it is already A. 30% is anything, anything, B, C, Z, P, anything. So will you change 70% and make something 30%? Or will it be easy to change 30% and make 70%? 30% because it is less, already it is less. So if in 30% everything is different, make everyone A."
    },
    {
      "start": 1151.94,
      "end": 1173.87,
      "text": "So we will get 30 operations in this. But if you want to change A, then you will get at least 70 operations here. It is like this that there are two parties in the country. There is a BJP and an entire India Alliance. So BJP is in the majority. And India Alliance is a small small small small party. A small small small party is 30%."
    },
    {
      "start": 1173.87,
      "end": 1202.62,
      "text": "If someone wants to change, then change India Alliance to BJP, then there will be one party in the whole country, one party, one rule. But if you try to change BJP and make others, then you will have to do at least 70% of the vote share of BJP. And a lot of parties in India will also have to be changed, then it will take a lot. That's why the thing that is the most in the sub-string, leave it as it is. Change the rest of the thing and make it as maximum as possible."
    },
    {
      "start": 1202.62,
      "end": 1233.23,
      "text": "This should be your algorithm. Now what did you understand from this? Now it will open slowly. We are saying that the question is hard. It took us half an hour yesterday. We didn't make it for many days. So it took time. That's why we are explaining it slowly. If possible, we will not explain the next question today. We will give time to this question. We will do it with time. Assume that this is your low."
    },
    {
      "start": 1233.23,
      "end": 1260.77,
      "text": "This is your high. You had to do this sometimes. Your A is 2 times. B is 1 time. You have 1 power. Now, this can also happen. See. Can you make this substring better than the power you have? You have to find out in this question. If you have K times power, then by finding K times power, can you make the current substring better or not? So, you have to think about this. Can you do it in this?"
    },
    {
      "start": 1260.77,
      "end": 1291.14,
      "text": "Yes, we can do it. Why? Because we can see that K1 is there. A is in majority and A is in maximum, so let A be. And rest B is once, so we can do it once. So it means that K can be either less or equal. But, suppose C is here. Here, here, C is here. Where? Here. Now your high is not here. Your high is here. Means, your hash map looks like this. A2, B1, and C1. Now see, your K1 is here."
    },

```
    {
      "start": 1291.14,
      "end": 1319.7,
      "text": "Can you make this substring good? Let's see. A is in the majority. Leave it. BJP is there. How much is there in the rest of India? 2. So you will have to change 2. Either you make B as A or C as A. Only then it will be complete. A, A, A. 4 times. Otherwise it will get stuck. So do anything. You can't make this substring good. You can't make this substring good. And this is our check. This is our check. That is this information."
    },
    {
      "start": 1319.7,
      "end": 1349.66,
      "text": "Is this information right or wrong? Which substring is right? If K was not given, it would have been very simple that the substring which has all 1 is right. But if there is a substring in which you can use K to change it, then it is right. If there is a substring in which you can't use K to correct it, then it is wrong. This is our whole algorithm. Now see how nicely we will solve it. We will solve it nicely."
    },
    {
      "start": 1349.66,
      "end": 1381.14,
      "text": "Come first. Come first. Write the format again. Write the template again and again. Practice it. I less than N. Okay. High plus plus. Did you understand? Okay. First of all, take the high inside yourself. Take it. Take it. Okay. Now, what do you have to find out? Is everything unique or not? Is it unique or not? You have to find out two things. Unique and maximum. Which character is in maximum?"
    },
    {
      "start": 1381.68,
      "end": 1410.22,
      "text": "What can we do for this? What will be better than HashMap is that you make a big array. Why am I not saying HashMap? Because it is difficult to get the majority from HashMap. How will you know whose frequency is in the majority in HashMap? You will not know that. What will you do in this? Now see, whenever there is a string question, you can use space. It is constant. Why is it constant? I am telling you. You make an array of 26 sizes. What will you make of 26 sizes? Make an array. 0, 1, 2, 3."
    },
    {
      "start": 1410.22,
      "end": 1440.48,
      "text": "here 25, you can make an array of this name this will become your hash map whenever you get capital A keep all of this on 0 initially keep all of this on 0 whenever you get capital A, capital A will start from 1 so change this to 1 whenever you get capital B, change 1 to 1 again if you get B, change this 1 to 2 if you get C, change 2 to 1 this will become your hash map"
    },
    {
      "start": 1440.48,
      "end": 1471.02,
      "text": "Now what will happen is that if you have to find the maximum, then iterate the array. Return the maximum, it will take constant time. Because the alphabet is 26, then the size of the array will also be 26. So we will iterate 26 again and again, there will be no difference. Now see. Come. For. Okay, I had written it, let's do it again. Increase the speed a little, look at 1.2, it is more fun. We also look at our 1.2. First of all, whatever the index of high is."
    },
    {
      "start": 1471.02,
      "end": 1501.39,
      "text": "so F S one more thing one more thing if you make 26 size kare then how will you know that A has to be kept at 0 B has to be kept at 1 and C has to be kept at 2 the value of S the value of S of A is 65 the value of S of A is 65 because we don't want you to fall into that so don't make 26 kare make 256 kare how much? 256 because whatever type of character there is in computer science"
    },
    {
      "start": 1501.58,
      "end": 1530.82,
      "text": "All the types of characters in computer science are 256. Space bar, inverted comma, abcd, small abcd, numbers, all of them are 226. So if you do that, then you don't need to do index here and there. So your a will go to 65 in that array, so you don't need to do anything. Do as you do in the hash map. So make an array, we will put 256 to 0. Come. Do plus plus. Wherever you go, your character will come, very good. Now what is it?"
    },
    {
      "start": 1530.82,
      "end": 1561.14,
```

```
      "text": "this information, this information, is it right or wrong? How will we know? How will we know? That, that, first of all, it is the frequency of that hash map. So, what does this tell you? What does this tell you? What does this tell you? That how many letters are there in it? Not unique letters, total letters. So, you will know this from length. What is length? Length is high minus low plus 1. Length is this. Length means that this is the total character."
    },
    {
      "start": 1561.14,
      "end": 1586.58,
      "text": "Now there will be two things in total character. One will be maximum, whose maximum frequency is, for example, in this case, in this case, in this case
```

```
      "start": 1760.03,
      "end": 1786.56,
      "text": "You will have to remove it again. Remove it. And what was the difference? The
difference was length minus max. Now both length and max have changed. Difference will also
change. Difference will also change. Difference will also change. Change it. Close the while
loop. Now, now we have come here. Now what can happen here? How long are you running the while
loop? Till, till, till difference"
    },
    {
      "start": 1786.56,
      "end": 1816.35,
      "text": "this is greater than k so if this while loop is broken here then it means that
either my difference is less than k or my difference is equal to k both the conditions are
correct you can do both the conditions openly that is correct information that is correct
window you just don't have to do anything length is equal to take it out again what was it? if
you got correct information then don't worry and rest is equal to max of rest and length"
    },
    {
      "start": 1816.35,
      "end": 1845.26,
      "text": "close the for loop, the high one and press return, see same pattern, same
question like yesterday, same pattern, what has changed? something has changed, these three
lines had to be written extra these three lines had to be written extra here we have some
logic, now this logic has to be written extra after thinking the concept is the same, you have
to do the information, when the information is right and when it is wrong, you have to
identify it, the rest is up to you, you have to put it in the same format and till the
information is wrong"
    },
    {
      "start": 1845.26,
      "end": 1875.33,
      "text": "till then you have to keep on teaching till then you have to keep on teaching
till then you have to keep on teaching till then you have to keep on teaching till then you
have to keep on teaching till then you have to keep on teaching till then you have to keep on
teaching till then you have to keep on teaching till then you have to keep on teaching till
then you have to keep on teaching"
    },
    {
      "start": 1875.33,
      "end": 1904.86,
      "text": "Max consecutive 1, look at the question, coded in 2 minutes, submitted and
finished. Both are the same question, almost same question. Not almost, it is the same, it is
easier. If this question was 8 out of 10 difficulty, then it is 6 out of 10 difficulty. So if
you understood this, it will become that. So go, submit it and also submit that and see.
Comment and tell whether it happened or not. If it didn't happen, then we will tell you
tomorrow. If it happened, then it is a good thing, we will let it be, we will start new
patterns from tomorrow. Okay. So this was today's class and see."
    },
    {
      "start": 1904.86,
      "end": 1934.3,
      "text": "You don't have to feel bad about what we said, but you have to understand a
little bit. You also have to be patient. Don't complain that you have not studied, your career
is ruined, you have ruined your whole life. Don't complain, everything will be done. It will
be done as much as you say and it will be done well. It is not that we will come and complete
the food and write something here that yes, let's do it like this. We don't do all that. So
you also cooperate a little bit. If we are increasing by 9 steps, you also increase by 1 step."
    },
    {
      "start": 1934.3,
      "end": 1943.63,
      "text": "We should maintain the decor, right? We will upload this video, it will be a
little late. Those who will not be able to see today, see tomorrow. And let's meet in the next
video."
    }
  ]
}
```


28. transactions/t_9.json


```
{
  "video_id": "4L4MYtxGh9s",
  "title": "Rotten Oranges: Multi-Source BFS | Graph Series",
  "language": "en",
  "model": "large-v3",
  "duration": 2160,
```

```
  "segments": [
    {
      "start": 0.0,
      "end": 27.3,
      "text": "Today we are going to do Rotten Oranges. Initially, I wanted to do two
questions but this question has become very heavy. What does heavy mean? It means that we want
to understand one question very well so that we can do similar questions in the future. Some
people were also writing in the comments that the number of questions is very less. See, never
be dependent on the number of questions. It is not that we are hiding something or we are
thinking that let's not do this question, it will come in the interview. You guys are not my
competition."
    },
    {
      "start": 27.3,
      "end": 55.63,
      "text": "We are not doing this to save some knowledge. Because we are not taking money,
we are giving it for free. So what is the reason to reduce the number of questions? That every
question gets more and more time. 100 questions by doing half incomplete and by making it like
this, it is a very good option to do 20 questions. Do that 20 so solid that you can make 80
without that 20. This should be the goal. Whether the number of questions is less or more,
leave it to us. You have been watching for so many days, so nothing has gone wrong so far."
    },
    {
      "start": 55.63,
      "end": 83.12,
      "text": "So, keep a little bit of trust ahead. Rotten oranges is a very famous question.
I mean, it has been asked in every company according to me. So, look carefully. Those who have
not seen the island one, then you will not understand anything. You go now, run quickly. Those
who have seen it, you will understand why DFS was used in it. And here DFS is not used,
something different is used here. Listen very carefully. Listen with your ears, eyes and nose
open. Let's start. So, let's see first of all."
    },
    {
      "start": 83.12,
      "end": 111.14,
      "text": "What is said in the question? First of all, you have been given a grid of n
into m. See, whenever you get a grid of n into m in your life, and it tells you that you can
move in 4 directions, 8 directions, 6 directions. It is a question of 100% graph. First of all,
 understand and prepare yourself that it is a question of graph. Meaning, either BFS or DFS,
either breadth first search or depth first search. If you haven't seen both, then watch it."
    },
    {
      "start": 111.14,
      "end": 137.82,
      "text": "You have been told that there is no grid in this grid. It means there is a
matrix. Very good. There can be three things in this. Like in the previous question, there can
be two things in the number of islands. 0 and 1. 0 means water. 1 means land. There can be
three things in this. What? 0, 1, 2. There will be only three numbers. There is a matrix of
numbers. Either one number will be 0, 1 or 2. There can't be anything else. Very good. Out of
this, 0 means empty. Okay. 0 means"
    },
    {
      "start": 137.82,
      "end": 164.51,
      "text": "Kali, kuch nahi. Kali ya diwal, kuch bhi hai jaya samajh sakte hai. Acha. One
ka kya matlab hai? Fresh orange. Kya? Fresh orange. Yes. Fresh orange. Taza santra. Okay. Do
ka matlab? Rotten orange. Do ka matlab? Rotten orange. Matlab? Sada hua santra. Very good. Toh,
 aapko ek grid diya hua hai. Isko aap ek boxy ki tarah samajh lijiye. Fal ka tokri diya hua
hai. Fal mein aisa bahut sara khana hai. Kahin pe khali hai. Matlab, koi tokri khali hai."
    },
    {
      "start": 164.51,
      "end": 194.48,
      "text": "Is it a good orange or a rotten one? Every minute is called every minute. What
is every minute? A rotten orange. You must have heard the saying that one spoiled apple spoils
the lot. If you keep one rotten apple in a box, it will spoil the whole box. Similarly, in
this box too. What will happen every minute? Suppose there is a rotten orange somewhere."
    },
    {
      "start": 194.48,
      "end": 219.86,
      "text": "In all four directions, up, right, down and left, if there is fresh orange in
all four directions, then in every minute, this rotten orange has the power to rot all the
fresh oranges. Listen again. In every minute, if there is a rotten orange in any place,"
    },
    {
      "start": 219.86,
```

```
      "end": 247.89,
      "text": "and if there is a fresh orange in any of its four directions, remember fresh,
it should not be in an empty space, fresh orange is kept, then it has the power to rot. The
power to rot means that in a minute all four will rot. If it is kept in four, then all four
will rot. Okay. What will happen after that? This process will keep repeating. Suppose there
was one rotten orange before. Suppose there was one rotten orange before. What did he do? He
rot two oranges around him."
    },
    {
      "start": 247.89,
      "end": 267.18,
      "text": "After that, what will happen in the next minute? It will happen again. He will
start doing it around him. He will start doing it around him. By doing this, the process of
rotting will spread in the basket. What will happen in the basket? It will spread. Consider it
as a coronavirus. You are standing here, you have got a coronavirus. You are your..."
    },
    {
      "start": 267.18,
      "end": 295.04,
      "text": "How many minutes?"
    },
    {
      "start": 295.04,
      "end": 324.85,
      "text": "Pura grid sadh jane me. Pura grid sadh jane me. Matlab pura grid me. Jitne bhi
fresh oranges hai unko sadh jane me. Unko rotten ho jane me. Rotten ho jane me. Yato sadh jane
me. Aur agar koi fresh orange asa hai jo sada nahi hai. Agar koi fresh orange asa hai jo nahi
sadh paya. Bawajood aapke ki aap sab karke dekh liye. Nahi sadh paya. To aapko minus one lota
na hai. Minus one. Agar sadeh sadh gaye. Kitne minute me sadeh sadh gaye. Kitne minute me
sareh santre sadh gaye."
    },
    {
      "start": 324.85,
      "end": 354.75,
      "text": "If an orange is saved till the end, how will it be saved? There was a person in
isolation due to corona virus. There was no neighbour around him. He was left alone in his
surroundings. So, he will be saved. No one will be able to tell him. So, he will be saved like
this. So, an orange can be saved like this. If an orange is saved, then you have to return
minus one. So, this was the question. Now, let's think and understand what can be done in this.
 Now, let's visualize this. Before understanding the question or before thinking about the
question, see how it will work. How will it be in the real world? See."
    },
    {
      "start": 354.75,
      "end": 384.34,
      "text": "This is your grid. You can see 1, 2, 1, and 2 zeros. 2 means that the orange is
rotten. 1 means that the orange is good and 0 means that the orange is empty. You can see that
the orange is rotten. You can see that the orange is rotten. You can see that the orange is
rotten. You can see that the orange is rotten. You can see that the orange is rotten. You can
see that the orange is rotten. You can see that the orange is rotten."
    },
    {
      "start": 384.34,
      "end": 408.3,
      "text": "T is equal to 1 minute. See what happens. This rotten orange has the ability to
rot all around. Very good, right? Now it's the same. See, every minute, every minute, all the
rotten oranges will be working together. It's very important. Since there is only one rotten
orange here, you must be thinking that it's not the same work. Every minute, every rotten
orange will spread all around."
    },
    {
      "start": 408.3,
      "end": 436.11,
      "text": "In coronavirus, if 100 people in the city have corona, then these 100 people
would be spreading it at the same time. It's not like they spread it one by one. No. Every
rotten orange will spread at the same time in every minute. That's why it will expand
exponentially. Look. T is equal to 1 minute. There is only one rotten orange. So what will it
do? It will look around. Can it do something on top? No. On top, the box ends. On the left,
the box ends. On the right,"
    },
    {
      "start": 436.11,
      "end": 463.66,
      "text": "Is there a good orange on its right? Yes, there is a good orange. So what will
it do? What will it do? It will destroy the orange on its right. It will destroy the orange on
its right. See, it destroyed it. Very good. But it is not over yet. Now its bottom. Can it
destroy the orange on its bottom? Yes, it can destroy the orange on its bottom. It destroyed
```

```
the orange on its bottom. It destroyed the orange on its bottom. See. Now its work is over."
    },
    {
      "start": 463.66,
      "end": 493.79,
      "text": "Its work is done. And now it can't do anything else. Now every person can only
spoil his neighbor. Neighbor will do the work ahead. Its work is done. How many minutes? One
minute. Okay. Are all the oranges spoiled? No. They are still left. Okay. Come in the next
minute. Now you will know how this problem will spread. See. T is equal to two minutes. T is
equal to two minutes. Now how many active spoiled oranges are there? See. Its work is done.
Its work is done. How many active spoiled oranges are there? One and two."
    },
    {
      "start": 493.79,
      "end": 522.61,
      "text": "Now, both of them will work together in this minute. First, let's look at this
one. Can the upper one rot? No. The left one has already rotted. The left one's work is over.
The left one has already done what he had to do. Very good. Can he rot the right one? Yes. The
right one is a fresh orange. What will he do? He will rot the right one."
    },
    {
      "start": 522.61,
      "end": 551.09,
      "text": "He ruined it. He ruined it. Okay. Can he ruin the one below? Yes. The one below
is also a fresh orange. He will also ruin the one below. His work is over. His work is over.
But what did we say? In the beginning of this minute, how many oranges were there? Two. This
was also there. This will also work in parallel in that minute. Now let's see what this will
do. Will he ruin his upper one? No. The upper one is already ruined and finished. Will he ruin
his right one? No. Why?"
    },
    {
      "start": 551.09,
      "end": 577.17,
      "text": "Because the right one was already messed up. The right one was already messed
up. So he will not mess it up. He will leave it. Will he be able to mess up the left one? No.
The left one is finished. Will he be able to mess up the lower one? No. Because there is empty
space below. We cannot do anything on empty space. His work is also finished. He could not do
anything to anyone in that minute. Let's go. His work is also finished. How many items are
finished? 1, 2, 3. But."
    },
    {
      "start": 577.17,
      "end": 606.72,
      "text": "Are all the oranges in the basket rotten? No, all the oranges are not rotten.
Let's move on to the next minute. T is equal to 3 minutes. Look, how many active rotten
oranges are there? 1, 2. From whom did these two come out? From this. The work of these three
is over. How many active rotten oranges are there? 1, 2. These two will work in parallel,
together in this minute. Let's first look at this. Will it be able to do anything above us? No,
 the basket is over."
    },
    {
      "start": 606.72,
      "end": 633.65,
      "text": "On the left, the left one is already rotten and finished. We can't do anything.
On the right, the basket is finished. On the bottom, there is empty space. This was absolutely
useless. We couldn't do anything in that minute. Finish this too. Finish this too. It's
finished. Very good. But in that minute, this is also working in parallel. One by one. One by
one is not working. All the active rotten oranges in that minute are working together. Look at
this."
    },
    {
      "start": 633.65,
      "end": 660.91,
      "text": "Will he be able to destroy the upper one? No, the upper one is already
destroyed. On the left? No, the left one is already destroyed. On the right? No, there is
empty space on the right. He won't be able to do anything. But will he be able to destroy the
lower one? Yes, he will destroy the lower one. Why? Because the lower one is fresh. So what
did he do to the lower one? He destroyed it. Now, this man's work is also finished. This man's
work is also finished. In this T is equal to 3 minutes, there were two people."
    },
    {
      "start": 660.91,
      "end": 689.58,
      "text": "Both have finished their work. Are all the oranges rotten? No, all the oranges
are not rotten. You will have to go in the next minute. T is equal to 4 minutes. In 4 minutes,
which are the active rotten oranges? There is only one active rotten orange. There are a lot
of rotten oranges, but all this work is done. How many are active? This one. Let's see what he
```

```
will do in that. Up? No, it was already rotten up. Left? There is empty space in the left."
    },
    {
      "start": 689.58,
      "end": 714.42,
      "text": "Below, the basket is finished. What is fresh on the right? Yes, it is fresh on
the right. So what will it do? It will rot this orange on the right. And now its work is
finished. Now look, four minutes have passed. Have all the oranges of this basket rotted? Yes,
all the oranges of this basket have rotted."
    },
    {
      "start": 714.42,
      "end": 742.02,
      "text": "However, if you have an active rotten orange, you will feel like trying it out.
Nothing will happen with that. Is there any fresh orange left in the jar? No, there is not. So
this is the question I asked. How many minutes will it take for no fresh orange to be left in
the jar? How many minutes will it take? How much will you say? 4 minutes. In 4 minutes, every
fresh orange of this jar will rot. Every fresh orange will rot. Or else,"
    },
    {
      "start": 742.02,
      "end": 772.03,
      "text": "After 4 minutes, there won't be a single fresh orange left in this box. Are you
able to understand or not? Let's look at another small example. Let's take a 2 by 2 matrix.
Here 2 is 0, 0, 1. In the beginning, how many rotten oranges do you have? One. Let's see what
the result will be in 1 minute. There is only one rotten orange."
    },
    {
      "start": 772.03,
      "end": 800.53,
      "text": "Can it do anything on the top? No, the basket is finished. On the left, the
basket is finished. On the right, it is zero, you won't be able to do anything. On the bottom,
it is zero, you won't be able to do anything. So, is this rotten work finished? It has done as
much as it had to do. Cut it. Cut it. Okay. Okay. Now, how many active oranges are left? How
many active rotten oranges are there? Not even one. This was there. Its work is finished. Okay.
 But have all the oranges in the basket rotten? No, all the oranges are not rotten. This is
left."
    },
    {
      "start": 800.53,
      "end": 830.48,
      "text": "Why not? Because it is not allowed to go diagonally. It is allowed to go in all
directions. Now this was rotten. It looked here and there. Now nothing can be done. There is
no other rotten orange that can rot it. What do you have to do in this case? Minus 1. You will
take it till the end of time. Even then this orange will not rot. Why will it not rot? Because
it is placed like this. What do you have to do? Minus 1. It is fresh. It is saved. It is saved.
 If there is no active rotten, then it is saved."
    },
    {
      "start": 830.48,
      "end": 859.65,
      "text": "This is how this question is working. That is why we are telling you very
important thing again and again. In every minute, all the active rotten oranges, active means
those whose work has not been done yet. The orange that has done its work by rotting, the
active rotten oranges will do their work together. Then that minute will be complete. It will
not happen that if t is equal to one minute, there are two rotten oranges, then one minute
will take that the first rotten orange will rot. In the second minute, no. All the rotten
oranges,"
    },
    {
      "start": 859.65,
      "end": 889.09,
      "text": "We will solve them all at once. Suppose in the beginning, there were 2 rotten
oranges. Let's say they solve 6 oranges together. It takes them 1 minute to solve both of them.
 Now these 6 rotten oranges, let's say they solve 18 oranges. It will take them 1 minute too.
It is working in parallel. It is not working alone. Now let's go and explain how to solve it.
Let's go. So in this question, doing DFS will not work. Like you did in the island question."
    },
    {
      "start": 889.09,
      "end": 918.43,
      "text": "You could have thought from the islander that we will go from the top. We will
say, brother, remove one rotten tree. From that rotten tree, we will see how much more we can
stand and we will go and see. No, but that is not happening. In this question, you have ever
been asked in the question of the graph that it is working simultaneously. It means that all
such people, all the defaulted people are working together. Now tell me how much time it will
take when it works together. It means that it is working like this. Look, there was one man in
```

```
the beginning. One man infects three people."
    },
    {
      "start": 918.43,
      "end": 947.01,
      "text": "They infect these three and these three. This is a kind of BFS. Breadth first,
sir. Slowly, slowly. What is BFS? There is one person. He did this, he did this, he did this,
he did this, he did this. After that, what is BFS? One person did three. All three do their
three one by one. Look at this. Look at this. Slowly, slowly, it is spreading. Slowly, slowly,
it is spreading. This tree is spreading slowly. It is happening in the same way. In the
beginning, there was one rotten orange. He will do two."
    },
    {
      "start": 947.01,
      "end": 976.96,
      "text": "These two will do to their two. This will do to their two. It is spreading like
this. It is not happening like this. One rotten orange did to this. Then this one did to this.
Then this one did to this. It is not happening like this. You will get a lot of questions like
this. Where it is working simultaneously. So it will not work by doing DFS. You cannot do this.
 We will catch a rotten one. We will put DFS on that rotten one. No, it is not like this.
Because it is working parallelly. That is why there is a technique here. You can give it a
name in the graph. Give it a name."
    },
    {
      "start": 976.96,
      "end": 1005.82,
      "text": "It is not necessary, but it should be. It is called multi-source BFS. What is
it called? Multi-source BFS. Multi-source BFS is an advanced way to solve such problems. Look,
it is a very intuitive way. What will we do? Look, we will not do much. Whenever the name of
BFS comes to your mind, Q should come in your mind. Whenever the name of BFS comes in front of
you, what should come in your mind? Q. Q data structure. Okay. We had taught you BFS in graph.
How did we teach you?"
    },
    {
      "start": 1005.82,
      "end": 1035.06,
      "text": "Remember the BFS that we did in the graph? What did we do in Q? We put the node,
 then we put all the neighbors of that node in it, then we put all the neighbors of all those
neighbors in Q. We did BFS like this. We will do the same thing here, but what will we do in
the beginning? In the beginning, we will, one minute. So see, we will make a Q and what will
we put in it? At any given minute, what were we doing at the top? All the active rotten
oranges in every minute,"
    },
    {
      "start": 1035.06,
      "end": 1064.46,
      "text": "active rotten oranges that have not been able to do their work yet, they will
be active and they will do their work. We will make a queue in which we will put what? We will
make a queue like this. This is its front and this is its rear. In the queue, at any given
minute, how many active rotten oranges can be there? We will store them there. So see, when
the time starts, when the time starts, so in the beginning, the time starts at zero. So at
zero,"
    },
    {
      "start": 1064.46,
      "end": 1090.59,
      "text": "After running a loop, push all the already rotten oranges in the queue. All the
already rotten oranges in the queue. This is already a rotten orange. Push it. What do you
push? Push their coordinate. Push their i and j. So what do we do here? Push 00. Is there any
other rotten orange? No. And whoever you are pushing,"
    },
    {
      "start": 1090.59,
      "end": 1117.82,
      "text": "change its value to something like minus 1. Why? Because you have to remember
later that this one has been visited. Don't make the visited area separately. What we will do
is that we will make this one minus 2. Push the coordinate and make this one minus 2 later.
Okay, push is done. Now, don't try to make this one more half. Or don't push this one again.
That's why. To remember. Anything. Minus 1, minus 2, minus whatever. Make a value other than 0,
 1, 2."
    },
    {
      "start": 1117.82,
      "end": 1147.81,
      "text": "If possible, make it zero. Empty space is done. There is no work left. So, t is
equal to zero. How many oranges were there already? One orange was there. By running a nested
loop, it was pushed. And by running a nested loop, do one more thing. Take out the count of
```

```
fresh oranges. How many oranges are fresh? So, see. 1, 2, 3, 4, 5, 6. How many are fresh? 6. 6 oranges are fresh. Keep its count. We will tell you later why it is important to take out the count. Done?"
    },
    {
      "start": 1147.81,
      "end": 1170.66,
      "text": "In the beginning, we made a loop. Q is made. We have already put all of them. T is equal to 0. Now, loop on this Q. How? While Q not empty. While Q not empty. And, fresh greater than 0. Means, till one fresh orange is left, you have to process it."
    },
    {
      "start": 1170.66,
      "end": 1200.11,
      "text": "All the fresh should be finished. All the fresh should be finished. As soon as the number of fresh becomes 6 to 0, stop it. After that, you don't need to do it. Okay. Now, see. Here, we will run minute by minute. What? Minute by minute. Like we did in level order traversal. We will run minute by minute. So, see. What is the first thing to do? Do time++. Do time++. What is your time? 1 is done. Now, in this 1 minute, whatever is in the queue,"
    },
    {
      "start": 1200.11,
      "end": 1229.36,
      "text": "they will all work together. After that, the time will increase later. At this time, at this minute, all the oranges in the queue will work together. So what will we do for this? int s is equal to q.size. All the oranges in the queue will work together. You will know in the next diagram. After that, while s minus minus, means whatever it is."
    },
    {
      "start": 1229.36,
      "end": 1258.66,
      "text": "After that, what will we do? Take it out one by one, whatever size it is, take it out, pop it, pop it. So what did we get? We got 0, 0 here. What will happen? Now 0, 0 will rot the oranges on all four sides and if they rot, it will push them in the queue. Let's see. Will it be able to rot on top? No. On the left? No. On the right? Yes. On the right? Yes. But what will we do?"
    },
    {
      "start": 1258.66,
      "end": 1286.75,
      "text": "Right will push this coordinate. 0, 1. And its value will be something like minus 2 or 0. Nothing more. It's done. Let's push it once. Push and reduce. And fresh's value. Fresh's value. Reduce one. Okay. But the work is not done yet. Not here. Not here. Not here. The lower one. Yes. The lower one also has the power to bend. Do the same work in the lower one. What work do you do?"
    },
    {
      "start": 1286.75,
      "end": 1314.85,
      "text": "First of all, push its coordinate. 1, 0. What should we do? 1, 0. The coordinate is pushed. Fresh, because you are sending a fresh orange. The number of fresh is 4. And now change this as well. Do something like minus 1 or minus 2. Do anything other than 1. Do anything other than 1 or 2. Just for marking, it is visited. Now you don't have to work on it. Very good."
    },
    {
      "start": 1314.85,
      "end": 1339.81,
      "text": "Okay. This 00 has done its work. Very good. Now, this loop will be closed. Now, we will come back. Is your queue empty? No. Now, there are two rotten oranges in your queue. So, when time will be plus plus, when time will come on two minutes, these two oranges will work together. What will happen? S is equal to queue's size. How much will the value of S be? Two."
    },
    {
      "start": 1339.81,
      "end": 1369.55,
      "text": "Now, give both of these oranges a chance to rot. Give both of these oranges a chance to rot one by one. First, which one should we pop? 0, 1. Which one? 0, 1. Let's see what 0, 1 has the power to do. 0, 1. You have come to this. Look around this. Can you do something on the top? No. On the left? No. You can't do it on the left. On the left, we will do it minus 1 or something. We will do it only when there is 1 in our direction. Fresh orange means 1. Like, if we get 1, we will do something. Okay?"
    },
    {
      "start": 1369.55,
      "end": 1395.73,
```

```
      "text": "Right? Yes, you can do it in the right. So what to do in the right? There are three steps. First of all, take the right one and push the coordinate. What is the coordinate? 0, 2. What is the fresh one? Reduce one. You have erased one fresh orange. Okay. And the value of this right one, do some minus 1 or minus 2. We will write the code later and show you. Do minus 1 or minus 2. Okay. This work is over. Now,"
    },
    {
      "start": 1395.73,
      "end": 1423.71,
      "text": "Will it spoil the one below? Yes, it will spoil the one below. So what will you push? First of all, 1. First of all, 1. Push 1. The number of fresh is reduced by one more. The number of fresh is reduced by one more. And 1, 1 also. Do something like minus 1 or minus 2. Very good. Its work is done. Its work is done. But, what is the loop going on here? While s minus minus. There were two things. This was also, this was also."
    },
    {
      "start": 1423.71,
      "end": 1448.48,
      "text": "Now 1-0's chance, 1-0 will act in the same minute. 1-0 will also act together. 1-0 will act together. Let's see what 1-0 can do. 1-0 came here. In the upper one, no. In the upper one, the work was already done. In the right, no. In the right, there is 1. But these 1-coms have already pushed this 1. And these 1-coms must have made it minus 1 or minus 2. It won't happen."
    },
    {
      "start": 1448.48,
      "end": 1477.23,
      "text": "Below, below is 0. On the left, on the left the circle is finished, the grid is finished. He will not be able to do anything and will just pop. All three have popped, let's go. In this minute, t is equal to 2 minutes, which two people did their work? Both of them, both of them. Now, come back. Is your queue empty? No, your queue is not empty. And, the queue is not empty. Is your fresh still 0, is it big? Yes, there are still two fresh left, means you have to continue this."
    },
    {
      "start": 1477.23,
      "end": 1506.9,
      "text": "When the phrase will be zero, we will not do it. Now, how many things are left in the queue? These two oranges are left. 0, 2, 1, 1. Both these oranges will work in this. First of all, time++. Means, time increased to 3 minutes. Take out one by one. First of all, take out 0, 2. Let's see what 0, 2 will do. It came here. Can it do anything on the left? No. Up? No. Right? No. Down? No. It was useless."
    },
    {
      "start": 1506.9,
      "end": 1536.88,
      "text": "But there is one more left. He is also acting in this. Take out 1, 1. Who? Take out 1, 1. Take out 1, 1. Let's see what 1, 1 can do. 1, 1 came here. Up? No. It has already happened in the upper part. In the left? It has already happened. In the right? It is 0. In the lower part? Yes. We got the lower part. We got the lower part. What 3 steps do we have to do? First of all, write the coordinate of the lower part here. 2, 1. 2, 1. Write it. What is the number of the fresh?"
    },
    {
      "start": 1536.88,
      "end": 1566.69,
      "text": "One is less. And this 2, 1 is minus 1 or minus 2. Whatever you want to make, make it. It's over. This also popped. This also popped. Was T equal to 3 minutes? Was your size 2? Which two were 2? These two. Is the work of these two over? Yes, it's over. Return it. Is your Q empty? No, your Q is not empty. Is your fresh value more than 0? Yes, the value of fresh is more than 0. It means it is still left. Keep trying. Keep trying. 2, 1 came out."
    },
    {
      "start": 1566.69,
      "end": 1594.46,
      "text": "2, 1. Let's see what 2, 1 will do. 2, 1. Let's see what 2, 1 will do. 2, 1. It popped. Let's see what 2, 1 will do. 2, 1. It came here. Can it do anything to the one above? No, it's already done to the one above. On the left, it's 0. On the bottom, the grid will end. On the right, yes, yes, yes. We got it. We got it. We got 1. What will we do? First of all, push the coordinate. 2, 2. It's pushed. The value of fresh is reduced. Look, the fresh is 0."
    },
    {
      "start": 1594.46,
      "end": 1614.18,
      "text": "You can see that there is no one left. And its value is minus 1 or minus 2. We have done it. Minus 1 or minus 2. We have done it. Okay. It is over. This time is over. This time is over. Okay. We did not increase the time. This time will be 4. We forgot to increase
```

```
the time. Okay. Time is over. Check again."
    },
    {
      "start": 1614.18,
      "end": 1641.9,
      "text": "Is your queue empty? No, your queue is not empty. It is 2-2 in the queue. But
is your fresh more than zero? No, the fresh is zero. So do you need to continue? Do you need
to increase the time? No, you don't need to do anything. Now you will come out of this. Now
you will come out of this. Because what was the whole process? To destroy fresh oranges. Your
fresh is not more than zero. You don't need to do anything. Stop. I have asked you."
    },
    {
      "start": 1641.9,
      "end": 1670.67,
      "text": "Minimum, how many minutes will it take? So, the amount of minutes that had to
take is 4. After that, you come out. A small check. If, even after doing all this, the fresh
is greater than 0, this means that no one can reach such a fresh orange at such a place. Like
in this example. In this example, if you dry run it, then your queue will end. But your fresh
will be greater than 0, i.e. 1 will remain. If you think that the fresh is still greater,"
    },
    {
      "start": 1670.67,
      "end": 1695.23,
      "text": "Return minus 1. Return minus 1. Otherwise, return time. Return our time. Return
time. So, this is how it is working. Again, let's give a small recap. What is the first step?
The first step is to put all the rotten oranges in the queue. Okay? What does it mean to put
them in the queue? They were already rotten. So, don't tease the fresh ones in this."
    },
    {
      "start": 1695.23,
      "end": 1717.65,
      "text": "Put them in Q and change their value to 0 or minus 1. So that they can't go
again and again. Okay. And in this loop, count the number of fresh too. Count the number of
fresh too. After that, have we run a loop? Q dot not empty means Q should not be empty. And
fresh should be greater than 0. First of all, do time++."
    },
    {
      "start": 1717.65,
      "end": 1740.48,
      "text": "Now at this time, whatever is present in that queue, it may be already rotten.
When you have already pushed rotten, then three oranges have come. These three will work at
the same time. What did we do for this? int size is equal to q.size and while size minus minus.
 We will execute these three in this. After that, we will increase the time."
    },
    {
      "start": 1740.48,
      "end": 1767.1,
      "text": "If the queue gets empty, what does it mean? At that moment, how many active
rotten oranges are there? If your queue is empty, it means that no active rotten orange is
left. So if there is no rotten orange left, then there will be no rotting anywhere. Stop it.
Or if your fresh is zero, if the fresh oranges are zero, there is no need to do anything. Come
out. After coming out, if the fresh is still greater than zero,"
    },
    {
      "start": 1767.1,
      "end": 1797.17,
      "text": "Means it is at a place where no one can replace it. Minus 1. Otherwise, return
time. Return time. Come. Look at its code. Look at its code. Like yesterday, we will have to
write two functions. Which people haven't seen yesterday. It is a very good thing. Don't see
it now. First of all, for direction. First of all, for direction. In direction. First of all,
for direction. To do north, east, south, west. Minus 1. 1. 0. 0."
    },
    {
      "start": 1797.65,
      "end": 1826.78,
      "text": "int y4 is equal to 0, 0, minus 1, 1. Very good. One function to get the
validity that the direction you are going in, is it valid or not? int ij, int n, int nm. What
is nm? It is the size of the grid. If i becomes smaller than 0, i becomes equal to n, j
becomes smaller than 0, and j becomes equal to m."
    },
    {
      "start": 1826.78,
      "end": 1856.77,
      "text": "immediately return false that it can't go here, it is coming out of the grid.
Otherwise, return true for fun. It is valid. Now, let's write our main function. Now, let's
write our main function int. This is your lead code function. What is given here? A vector of,
vector of int. Yes, yes, yes. First of all, do the size. If we do a.size, the number of rows
```

```
will come. For the number of columns,"
    },
    {
      "start": 1856.77,
      "end": 1885.62,
      "text": "a.k.a. the size of 0. That means how many columns are there. This row column
will come out like this. After that, what do we have to do? Make a queue. How? Because you
have to insert a coordinate in it, make a pair type queue. Make a pair type queue. int fresh
is equal to 0. To remember the number of fresh oranges. int time is equal to 0. To remember
the time. int ij to run the loop. What do we have to do?"
    },
    {
      "start": 1885.62,
      "end": 1914.3,
      "text": "Do the already rotten oranges. Run a big loop on the grid. For j is equal to 0,
j less than m, j plus plus. Very good. If, if, if, aij, aij equal equal to 2, means already
rotten, then do two things. First of all, push ij in the cube. Because it will be useful. It
is active. Now it is active in the beginning. And,"
    },
    {
      "start": 1914.3,
      "end": 1943.2,
      "text": "Change the value of aij to minus 2. We will do minus 2 every time. Whenever we
will push in q, we will do minus 2. Close it. But the work is not over yet. But the work is
not over yet. If, if the array of ij is equal to 1, if the array of ij is equal to 1, write
else if. So, fresh plus plus. Count fresh. Close i, close j, close i. So now in your q,"
    },
    {
      "start": 1943.2,
      "end": 1974.32,
      "text": "Already rotten oranges have arrived. Now write your loop. While Q dot not empty.
 Q should not be empty. I mean, there should be active oranges in it, which are rotten. And,
and, sorry, a little, this happened. And, fresh should be greater than zero. This should
happen. If fresh is not greater than zero, then what are you doing? First of all, time, sorry,
time plus plus. Time plus plus. Very good. Int."
    },
    {
      "start": 1974.32,
      "end": 2005.94,
      "text": "S is equal to Q dot size. All of them will work now. Give everyone a chance.
How? While doing S minus minus. Okay. First thing, remove that rotten coordinate. Int P is
equal to Q dot front or Q dot pop. Very good. Pop is done. What do we do with this? R is equal
to P dot first. Which means which row is this? Okay. Just for simplicity."
    },
    {
      "start": 2005.94,
      "end": 2032.06,
      "text": "This is not necessary. You like this. c is equal to p dot second. So, which row
and which column has come. Okay. Now, what will we do? It will try to go all four sides. The
way to go all four sides is the island trick. What will we do? For int k is equal to 0. In how
many directions can we go? In four directions. Plus plus. How does it happen? Row is equal to
r plus x of k."
    },
    {
      "start": 2032.06,
      "end": 2061.55,
      "text": "We have explained this very well in the previous video, so please watch it if
you are not able to understand it. If you haven't seen the previous video, then you won't be
able to understand anything. That's a good thing. Now, there are two things that we have to
check. If this row column is valid, if it is valid, if it is invalid, then there is no need to
do anything. Let's move ahead. Valid and if there is a fresh orange here, then we will get rid
of it. We will get rid of the fresh one. We will not get rid of the zero. We will not get rid
of the one that has become minus 2."
    },
    {
      "start": 2061.55,
      "end": 2090.9,
      "text": "We will not do anything. So, it should be fresh orange. It is 1. What will we
do? We will do 3 things. First of all, we will push. First of all, push. Here it is. Push is
done. By changing its value, do something else. By changing its value, do something else. Do
minus 2 only. On the top, minus 2. On the bottom, minus 2. See, what is the purpose of minus
2? Make anything other than 1. Make anything other than 1. And, the number of fresh, because
you are spoiling a fresh orange."
    },
    {
      "start": 2090.9,
      "end": 2119.92,
```

```
      "text": "minus minus kar denge ye aapka if loop bandh hua ye aapka k wala loop bandh hua fir aapka shayad ye size wala loop aur ye aapka main wala loop bandh ho gaya niche aake agar fresh abhi bhi zero se bada hai kuch nahi kar sakte aap return minus 1 nahi to return time ye main loop bandh ho gaya dekh lije code thoda lamba ho gaya shayad dekh lije ye pura code hai"
    },
    {
      "start": 2120.11,
      "end": 2148.54,
      "text": "So this is your code. Time complexity n into m. Space complexity n into m. So see this whole code. This is valid above. And this is your rotten. It is section by section. There is nothing like that. Here we are making. Here we are already doing the rotten one. This is your main loop. Your main loop ends here. You have to return here. Go to this. Code this. Understand this. Dry run it. That is why I wanted to ask two questions today. But it is very heavy."
    },
    {
      "start": 2148.54,
      "end": 2158.83,
      "text": "And it's a very important question, so let's stop at that. So that you guys don't get overloaded. Reduce the number of questions and do it well. So let's go. See you tomorrow."
    }
  ]
}
```


29. transactions/transection.json


```
{
  "video_id": "-AEEZEWOA-E",
  "title": "Master Bellman-Ford: Cheapest Flights Within K Stops",
  "language": "en",
  "model": "large-v3",
  "duration": 1954,
  "segments": [
    {
      "start": 0.08,
      "end": 14.8,
      "text": "Hello kids, welcome. Today my voice is a bit hoarse. I am feeling a bit sick. Look, we have read a lot of things in the graph. A lot of things means we have read a lot of algorithms and various things."
    },
    {
      "start": 14.8,
      "end": 34.8,
      "text": "Today's question is a very famous question. We have been asked this question in many interviews and many of my friends have also been asked this question. One of the most famous question in graph is Cheapest flight within K-Stops. There are many ways to solve this question but today we are going to solve it in the most intuitive way."
    },
    {
      "start": 34.8,
      "end": 62.82,
      "text": "you will understand why we are doing this and why we are doing this and it will be very connected whatever we have studied is the same there is no such out of the world question whatever you have studied is the same here nothing is very different ok see what you have to do cheapest flighting case stops what does this mean let me tell you first of all what will be given to you you have a lot of flight station what does flight station mean suppose 0 is your station or airport 1 is your airport"
    },
    {
      "start": 62.82,
      "end": 89.26,
      "text": "1 airport is 2 and 1 airport is 3. There are 4 types of airports. You will also be given edges. Basically, you will be given edges only. Suppose, one edge goes from 0 to 1. And you will be given a directed weighted graph. Obviously, if you want to find the distance, there will be weight in it. And you will be given a directed graph. Meaning, if there is an edge from 0 to 1, then it is 0 to 1. It is not 1 to 0. Is it good? Okay."
    },
    {
      "start": 89.26,
      "end": 118.02,
      "text": "Let's assume that you have one more edge from 0 to 2 and one edge from 0 to 3. And there is one edge from 3 to 1 and one edge from 2 to 1. This is your airport. Let's assume for a while that the weight of 0 to 1 is 100, which means 0 to 1 is 100 km away. 0 to 2 is 20 km away. 0 to 3 is 20 km away. 3 to 1 is 50 km away."
    },
```

```
    {
      "start": 118.02,
      "end": 148.05,
      "text": "2 to 1, how far is it? Let's assume it is 30 km away. This is your graph. You
will be given two more things. First of all, you will be given a source. What is the source
airport? Let's assume that you have been given source airport 0. Which means, where will you
start from? From the 0 airport. You will also be given a destination airport. Destination
means where you have to reach. Let's assume that you have been given destination 1. That you
have to reach to 1. And, one additional thing which we have not seen till now. These k stops,"
    },
    {
      "start": 148.05,
      "end": 175.5,
      "text": "the value of K will be given. Let's assume that the value of K is given as 1.
What is the value of K? It is 1. Now let's understand the meaning of these things. What is the
meaning of K? The meaning of K stops is the same as when you travel by plane. If you want to
go from Delhi to Ranchi, then many times you go via Jaipur. There is a stop in Jaipur."
    },
    {
      "start": 175.5,
      "end": 202.08,
      "text": "If you go from Jaipur to Ranchi, what do we call it? It's a one-stop flight.
That's what they say. Whenever you travel by plane, let's say you have to go to Ranchi from
Delhi. It will be like this. Let's say you go directly from Delhi to Ranchi. Delhi to Ranchi
is a direct flight. What is it called? It is called non-stop. It is called non-stop flight. It
means you are going directly. There is no stop in between. There is no stop in between. But,
let's say you first go to Bombay from Delhi."
    },
    {
      "start": 202.08,
      "end": 230.34,
      "text": "In Bombay, you halt your plane and then you go back to Ranchi from Bombay. What
do we call this? One stop plane. Why? Because you have taken a stop in Bombay. Suppose you go
to Bangalore before Delhi. You come back to Bombay from Bangalore. You go to Ranchi from
Bombay. What do we call this? We call this a two stop plane. It means you had to go to Ranchi
from Delhi. You stopped at two places in between. You stopped in Bangalore and also in Bombay."
    },
    {
      "start": 230.34,
      "end": 257.01,
      "text": "This is called 2-stop plane. You are given the same stop here. You are given
the value of k here. What does k mean? If your k is given as 1, for example, you can take 1
stop in your journey at max. At max, it is not necessary to take 1 stop. But if you want, you
can take 1 stop. What does 1 stop mean? Suppose you have to go from 0 to 1."
    },
    {
      "start": 257.01,
      "end": 280.11,
      "text": "If you want to go from 0 to 1 directly, you can go directly. No problem. What
is this? This is a non-stop route. k is equal to 0. But if this is 1, you can do this if you
want. You can go from 0 to 2, 2 to 1. Or you can go from 0 to 3, 3 to 1. Both the routes are
one-stop routes. Now using all these things, if you want, you can go directly to 0 stop."
    },
    {
      "start": 280.11,
      "end": 293.6,
      "text": "You can go from 2 to 1 or 3 or whatever you want within K. You have to tell the
shortest distance of 0 to 1. What will be the shortest distance possible? On the condition
that you are at max."
    },
    {
      "start": 293.6,
      "end": 319.34,
      "text": "You can take k stops. At max, how many stops can you take? You can take k stops.
 Meaning, suppose if you were given k is equal to 0. What would you have given? 0. What does 0
mean? Non-stop. Meaning, you have to go directly. You cannot stop anywhere else. So, your
answer would obviously be 100. Why? Because when you cannot take any stop, then you will have
to go directly from 0 to 1. Meaning, it is not good that you cannot explore anywhere else. You
have to go from 0 to 1. Okay?"
    },
    {
      "start": 319.34,
      "end": 347.39,
      "text": "If K is equal to 1, then what would be your answer? 0 to 1 is a possibility. It
is not written that you have to take it. No. At max, you can take it according to your
convenience. If you wanted, you could have taken 0 to 1, which would have taken 100 km. There
is one more route, 0 to 2 and then 2 to 1. In this, 20 plus 30 is 50 km. There is one more
```

```
route, 0 to 3, 3 to 1. In this, 70 km. So, what would be your answer?"
    },
    {
      "start": 347.39,
      "end": 374.5,
      "text": "50 km. Why? Because you can take this route and decide your distance in 50 km.
When? When your k is equal to 1. Now k2, k3, k4 can also be done but nothing will happen. Even
if k2 is done, your answer will remain the same. Even if k3 is done, your answer will remain
the same. So this is your question. You are given source and destination. A graph is present
in front of you. You have to find the shortest path from source to destination. Means you have
to find the shortest distance. But"
    },
    {
      "start": 374.5,
      "end": 402.98,
      "text": "You have one more thing that you have given K means at max you can take K stops.
 How many stops can you take at max? You can take K stops. Notice one more thing here. If we
start understanding it, then we will open it slowly. But from above, a big question tells you
that at max you can take K stops. Now tell me what is the shortest distance? Now let's start
explaining this question to you and tell you how this question is made from what we have read."
    },
    {
      "start": 402.98,
      "end": 422.58,
      "text": "Remember this diagram. You have already seen this diagram in this graph series.
In which lecture? In Bellman-Ford's lecture. In Bellman-Ford's lecture, to explain to you why
Bellman-Ford is used and how Bellman-Ford works, we used this exact root. Assume that you have
this exact root in this question."
    },
    {
      "start": 422.58,
      "end": 442.94,
      "text": "So how did we start explaining Bellman fort to you? We told you that what did
you see in Bellman fort? That in this graph, this whole graph, suppose this is also the route
of your airport. Which is the most complex city? What was the meaning of complex city? Which
city has the most roads to reach? Which city has the most options to reach? We came to know
that Delhi"
    },
    {
      "start": 442.94,
      "end": 472.72,
      "text": "If you want to go from Bangalore to Delhi, then you have a lot of options. What
are the options? First, you can go directly from Bangalore to Delhi. We have written it here.
You can go from Bangalore to Ranchi and from Ranchi to Delhi. We have written it here. You can
go from Bangalore to Ranchi, from Ranchi to Kolkata and from Kolkata to Delhi. We have written
it here. You had one more option. You can go from Bangalore to Ranchi, from Ranchi to Kolkata,
from Kolkata to Bombay and from Bombay to Delhi. This was also an option. What was this
option?"
    },
    {
      "start": 472.72,
      "end": 502.4,
      "text": "This was the option. We also wrote it down. So to come to Delhi, how many
maximum things can you have? There can be 4 types of routes. And because we also said that how
many total number of fairs are there in this? 1, 2, 3, 4, 5. There are 5 cities, right? So the
biggest route that can be possible to come to Delhi, right? Which is the biggest route you can
see? This route. How many edges will there be in it? Maximum 4 edges will be there. How many
edges will there be? 4 edges. Why? Because see, what can be the maximum route? That you can
visit all the cities."
    },
    {
      "start": 502.4,
      "end": 516.24,
      "text": "Go to Bangalore, Ranchi, Kolkata, Bombay, Delhi, all of them. If you go to all
of them, how many edges will you get? 4 edges. Because if you go to 5 cities, you will get 4
edges."
    },
    {
      "start": 516.4,
      "end": 539.42,
      "text": "Can there be a bigger route than this? Absolutely not. Someone said that we
will need 6 edges to go to Delhi. No way. 6 edges means that you are repeating some city. You
might be repeating that you went from Bangalore to Ranchi, from Ranchi to Kolkata, from
Kolkata to back to Ranchi. Then why did you come back from Kolkata to Ranchi? Go properly. The
more you go back and forth, the more..."
    },
    {
```

```
      "start": 539.42,
      "end": 565.07,
      "text": "The distance will increase, the stoppage will increase, so there is no point in
that. When you want to know about the distance, you can't come to Delhi with more than 4 edges.
 When the number of cities is 5, then the maximum route can be 4 edges. There can't be a
bigger route than this. You can't do anything. After this, what did we start doing? There are
4 routes. After that,"
    },
    {
      "start": 565.07,
      "end": 592.86,
      "text": "We started doing relaxation. What did we start doing? We started doing
relaxation. And how many times did we start doing relaxation? 4 times. Why 4 times? Because n
was 5. So we relaxed edges 5 times. And we saw that when you do relaxation 4 times, you will
get the answer for this too. First you will get this, then this, then this, and in the end you
will get this too."
    },
    {
      "start": 592.86,
      "end": 618.21,
      "text": "Now, connect this with this question. If you go directly from Bangalore to
Delhi, then how many stops do you have to take? K is equal to zero. This means that there is
zero stop. This means that when your Delhi is only one edge away from your source, then you
are taking zero stops. You are taking a non-stop flight. Coming directly from Bangalore to
Delhi means that you are taking a non-stop flight. Your K is zero."
    },
    {
      "start": 618.21,
      "end": 645.5,
      "text": "If you are going from Bangalore to Ranchi and from Ranchi to Delhi, then how
many stops are you taking? K is equal to one stop. Two edge means one stop. Bangalore, Ranchi,
Kolkata, Delhi, how many stops will it take? K is equal to two stops. Bangalore, Ranchi,
Kolkata, Bombay, Delhi, how many stops will it take? K is equal to three stops. Now K is equal
to four stops, it will remain the same, there will be no change. What do you understand?"
    },
    {
      "start": 645.5,
      "end": 671.1,
      "text": "In Bellman Ford, when you did one relaxation, who gave you the answer? This man.
 In two relaxation, who gave you the answer? This man. In three relaxation, who gave you the
answer? This man. In four relaxation, who gave you the answer? This man. What does it mean?"
    },
    {
      "start": 671.1,
      "end": 696.56,
      "text": "Suppose in this question, you are given k is equal to zero. So what does that
mean? If you are given k is equal to zero, then you should not be concerned about the weight
of all these roots. k is equal to zero means non-stop flight, means you need maximum one edge.
How will you get the answer to that? As soon as you do relaxation once, you will get the
answer."
    },
    {
      "start": 696.56,
      "end": 724.56,
      "text": "In Bombay to Delhi, in K is equal to zero stop, you will get its answer by
doing relaxation for the first time. So how many times should you do relaxation? Do one
relaxation and your work is done. What is your work? It is done. If you are given K is equal
to one, that means you can take one stop. How many stops can you take? One stop. So you also
need its answer. How many times you will relax, you will get its answer. If you relax twice,
you will get its answer."
    },
    {
      "start": 724.56,
      "end": 750.06,
      "text": "If you have given k is equal to 2, you need this, you also need this answer.
How many times did you get the answer to relax? If you relax 3 times, you will get the answer
to k is equal to 2. If you have given k is equal to 3, how many times did you get the answer
to relaxation? If you relax 4 times, you will get the answer to k is equal to 3. Did you see
any pattern or not? Whatever k is given, whatever k is given,"
    },
    {
      "start": 750.06,
      "end": 776.3,
      "text": "K plus 1 times K plus 1 times Relaxation in Bellman Ford K plus 1 times
Relaxation in Bellman Ford You will get the answer Usually Usually in Bellman Ford Again I am
saying that if you have not watched Bellman Ford's video You will not understand anything
Please go and watch Bellman Ford once If you have forgotten then revise it Usually How many
```

```
times did Bellman Ford relax? N minus"
    },
    {
      "start": 776.3,
      "end": 806.21,
      "text": "One times. And he used to do it once to check. In the end, one more thing about
which? Negative cycle. That's a different thing. But usually, how many times did Bellman-Ford
relax? N minus one times. Because he needed the answer of these four. Because it could be so
complex. How many times do you have to relax? K plus one times. You have to relax K plus one
times. Why? Because if you have K is equal to one, then you just have to use these two. You
don't have to use the lower one. There will be a short distance."
    },
    {
      "start": 806.21,
      "end": 827.76,
      "text": "It is not allowed. If K1 is given, then these two people are not allowed. No
matter how much distance it takes, you can't go. K1 means you have to go in one stop. Either
you go directly or you go in one stop. You don't have to take more than one. If you run
Bellman Ford, how many times? How many times that relaxation? K plus one times. Will you get
the answer? Yes, you will get the answer. You will definitely get the answer."
    },
    {
      "start": 827.76,
      "end": 853.07,
      "text": "Are you able to understand this or not? When you were studying Bellman-Ford,
Bellman-Ford used to do exactly the same thing. In one relaxation, you used to get the direct
answer, which means when Delhi is one edge away from you, when you want to use one edge to go
from source to destination and you want to get its shortest, then you used to get that work in
one relaxation. When you want to go from Bangalore to Delhi in two edges and you want to get
its shortest, you used to get that work in two relaxations."
    },
    {
      "start": 853.07,
      "end": 877.1,
      "text": "We haven't done the same thing. We have connected this one to our k. How much
is k in this one? It has k1. That means to get k1, you have to do 2 relaxations. So whatever k
you are given, if you relax k plus 1 times, you will get the answer of your destination.
Return that answer. It is simple. There is one more small twist. This is very good. I hope you
are understanding this."
    },
    {
      "start": 877.1,
      "end": 898.51,
      "text": "There is a small twist, which I will tell you now. So, the small twist that we
are talking about, look at this example. Suppose you have this airport. Your source is 0,
destination is 1, and your k is 1. Means, you have to reach from 0 to 1, and how many stops
can you use? One stop means, how much edge can be there on the way of 0 and 1? At max, 2 edge,
k plus 1. Remember,"
    },
    {
      "start": 898.54,
      "end": 927.66,
      "text": "k stops means k plus 1 edge stop will be 1 but edge will be 2 so you should
have 2 edge how many times you should relax in that case you should relax 2 times because you
have to relax k plus 1 times ok ok to explain this we make k 0 we make k 0 you have to reach 0
stop now you know that you have to reach 0 stop means you have to go from 0 to 1 directly so
100 will be its answer lets run bellman ford"
    },
    {
      "start": 927.66,
      "end": 957.01,
      "text": "Is the 100 answer really coming or not? K is equal to 0 means how much
relaxation do you have to do? One relaxation. You have to relax 0 plus 1 times. So see, this
is your edges. So what was the meaning of relax? When we were learning Bellman-Ford, what was
the meaning of relax? This was your result. So in the beginning, 0 means 0. So if you want to
reach 0 from 0, then it is 0. And the rest was all infinity. What was the logic of relax? If"
    },
    {
      "start": 957.01,
      "end": 975.86,
      "text": "Result of D is greater than result of S plus weight. So, result of D is equal
to result of S plus weight."
    },
    {
      "start": 975.86,
      "end": 1005.55,
      "text": "weight means if source is an edge and you get to know that the result of the
```

```
destination is greater than the result of the source and its weight then we call this as relax
let's see if we relax just once then do we get the answer or not so first of all relax this
edge we call it as 1 1 2 3 relax this edge so what will it say if"
    },
    {
      "start": 1005.55,
      "end": 1031.49,
      "text": "Result of 1 is greater than result of 0 plus 100. What is result of 1 now? It
is infinity. So, it is big. So, what will we call this? Result of 0 is 0. 0 plus 100. So, we
will change this to 100. We will change this to 100. Relaxed? Look at the next. From 0 to 2,
we will change this to 50. We will change this to 50 from the same logic. Now, now, now."
    },
    {
      "start": 1031.49,
      "end": 1060.61,
      "text": "2 to 1, 2 to 1, see very carefully, this is what we want to tell you, this is
what we want to tell you, see very carefully. Two edges are relaxed, but we have to relax the
entire edge, right? 2 to 1 is an edge whose weight is 20, so what will it say? In this case, S
is equal to 2, D is equal to 1, and weight is equal to 20. What will it see? If result of 1 is
greater than result of 2 plus 20, then what is result of 2 now?"
    },
    {
      "start": 1060.61,
      "end": 1088.91,
      "text": "50 is done. Originally, it was infinity, but you had made it 50 just now. So,
it will say 50 plus 20 and result of 1. What is written in 1? 100. Whatever has happened now.
100 is greater than 50 plus 20. Yes, 100 is greater than 70. So, what will it do? It will
write 70 by removing result 1. It will write 70 by removing result 1. See this. And your
relaxation will end because you have relaxed all three. Now, see."
    },
    {
      "start": 1088.91,
      "end": 1118.26,
      "text": "Now, when you return the answer, what will you return in the answer? Result of
destination, which is 1. In your answer, 70 is being returned. What is being returned in your
answer? 70 is being returned. But, your question was k is equal to 0 stop. What does k is
equal to 0 stop mean? Non-stop. If you see the answer non-stop, you will feel that there
should be 100 in the answer. But, which answer are you returning? You are returning this
answer, 50 plus 20. This is not allowed. Then, how are you returning this? Does this mean that
Bellman-Ford will not work?"
    },
    {
      "start": 1118.26,
      "end": 1148.13,
      "text": "It will work. There is no bug in Bellman Ford. It has a behavior. See, when you
do relaxation once, then what happened? First of all, you relaxed from 0 to 100, which was a
good relaxation. After that, you relaxed from 0 to 2. So, 2 became 50 because of this. After
that, you relaxed from 2 to 1, then your 1 became 70. What did your 1 become? It became 70."
    },
    {
      "start": 1148.13,
      "end": 1175.62,
      "text": "But this is a little wrong in this question. When you do this, you get the
value of 0 to 1 in one relaxation, which was 100. And you get the value of 2 over 1 in one
relaxation, which is wrong. What was your logic about? That when we relax once, we will get
its value. We will relax once again, then we will get its right value. But here,"
    },
    {
      "start": 1175.62,
      "end": 1205.74,
      "text": "In the first relaxation itself, you have taken out the value of both of them
and you are returning the value of both of them. Why did this happen? Because it is a kind of
chain reaction. What is a kind of chain reaction? Why is it a chain reaction? First, you take
out the correct value of 2 from the edge of 0 to 2. In the same time, you use the one from 2
to 1 and take out the correct value of 1. This is a chain reaction. Why? First of all, you use
this to give the correct value of 2. Similarly,"
    },
    {
      "start": 1205.74,
      "end": 1232.4,
      "text": "by using 2 and 1, if you give the real value to 1, then it is wrong, it will
not work like this. You have to make sure in one relaxation that only this value should come
out, its value should not come out in that relaxation. The value of this person should come
out in the next relaxation, then only this thing will work. To solve this, we will use a
temporary array. What will we use? Temporary array. How will we use it? I will explain it to
you now."
    },
```

```
    {
      "start": 1232.4,
      "end": 1260.9,
      "text": "To solve this problem, when we do relaxation once, there shouldn't be a chain
reaction. It shouldn't happen that you get both the answers at the same time. That's wrong for
this question. What you do is, every time before relaxing, suppose your initial configuration
is this. So, we haven't relaxed anything yet. Both are infinity and 0 is 0. What we will do is,
 we will make a copy of this rest. We will make a copy of this rest. What will we make a copy
of? Simple, copy. We will make a copy of this in a temporary array 0, 1, 2."
    },
    {
      "start": 1260.9,
      "end": 1290.98,
      "text": "Copy is done. Before relaxing, now start relaxing. Now start relaxing. While
relaxing, see, first relax this edge, 0 to 1. What will be your S? 0, D, 1. Weight is equal to
100. Now, in the logic of compare, we will write this. If, if, what will we write? If T of D,
T of D, what is this? TMP. This one, TMP is coming. T of D is greater than."
    },
    {
      "start": 1290.98,
      "end": 1316.82,
      "text": "rest of S plus weight then T of D is equal to rest of S plus weight. What is
happening here, understand it very carefully. In this relaxation, the first relaxation that we
are doing, whatever changes you will make in this, whatever changes you will make in this,
when you will change any vertex and relax it, we will change it in temporary array."
    },
    {
      "start": 1316.82,
      "end": 1346.34,
      "text": "But the comparison that will happen with this person, result of source plus
weight, will happen with the old array. What does this mean? You will understand it only when
these three edges are relaxed. First of all, we are relaxing this edge. What does this look
like? T of D. T of D means T of 1. What is T of 1? Infinity. Okay, that's good. It's greater
than result of source plus weight, which means result of 0, which is 0."
    },
    {
      "start": 1346.34,
      "end": 1375.74,
      "text": "Plus weight, plus 100. Yes, if it is big, then it should be relaxed. But the
real relaxation that you are doing, do it in T of D. Don't touch the result. What will happen
with this? T of D. What has become? 100 has become. T of, result of 1 is same. Result of 1 is
not changing. Result of 1 is still infinity. T of 1 has become 100. You will not understand
the benefit of this. You will think, why did I do this? You will understand. You will
understand. See. This edge is done. Let's go to the next edge. Let's go to the next edge."
    },
    {
      "start": 1375.74,
      "end": 1403.5,
      "text": "Next edge is 0 to 2. So, what will be its S? S is equal to 0, D is equal to 2,
weight is equal to 50. Do the same thing. T of D means T of 2, infinity, is greater than
result of source, result of source, which is 0, plus weight, plus weight, which is 50. So, 50,
0, infinity. Yes, it is big, we should relax. But, where to relax?"
    },
    {
      "start": 1403.5,
      "end": 1433.23,
      "text": "T of D, not result of D. So, T of D means, what do you do with T of 2? You do
50. T of 2, 50. T of 2, 50. Very good. Now, come to the third edge. We got a bug in the third
edge. We got a bug in the third edge. We got a bug in the third edge. Now, see, we won't get
that bug. Why won't we get it? Let's tell you. Come to the third edge. 2 to 1. S is equal to 2.
 D is equal to 1. Weight is equal to 20. See. If,"
    },
    {
      "start": 1433.23,
      "end": 1460.74,
      "text": "T of D T of D which is 100 T of D which is 100 T of 1 which is 100 is greater
than Result of source plus weight Result of source plus weight means Result of 2 plus weight
And result of 2 is still infinity Result of 2 is still infinity In this case, no relaxation
will be done Last time when we did not use this temporary So where was the problem? Because"
    },
    {
      "start": 1460.74,
      "end": 1490.7,
      "text": "The result of 2 became 50 because of this edge. When you come to the third edge,
 you think that the result of 2 is 50. 50 plus 20 is 70. It is less than me, so let's relax.
But in this case, nothing has changed in the result of 2. The result array has not changed in
the entire array. Whatever change has happened, it has happened in the temporary array. In
```

```
which? In the temporary array."
    },
    {
      "start": 1490.7,
      "end": 1520.29,
      "text": "1, when you come to relax this, then it sees the previous value of 2. It
doesn't see the new value of 2. The new value is 50, sorry. It sees the previous value of 2,
which was infinity. It doesn't see the current value of 2, which has changed immediately. It
doesn't want to use it. After one relaxation, when this whole relaxation is over, what value
do you have in TMP? Which has come because of this relaxation. This relaxation."
    },
    {
      "start": 1520.29,
      "end": 1549.17,
      "text": "There is no chain reaction that because of 0 to 2, 2 changed. Because of 2 to 1,
 1 changed. They were doing the work of two relaxations in one. Because of this, the work of
one relaxation has been done in one. Now, when this is done, these three things are done, this
is done. What to do in the last? Result is equal to TMP. Write the value of TMP in the result.
Write the value of TMP in the result. What to write in the result? 100 and 50. What to write
in the result? 100 and 50."
    },
    {
      "start": 1549.17,
      "end": 1577.46,
      "text": "Now, when you return, because your case was 0, you did one relaxation. After
one relaxation, your result is this. What will you return? Result of destination, which is 100.
 Is 100 the right answer? It is absolutely the right answer. Because your case is 0. You just
had to return the answer of this root. See, the answer of this root has not been computed yet.
Why? Because we have used a temporary array. This is the whole meaning of using a temporary
array."
    },
    {
      "start": 1577.55,
      "end": 1607.22,
      "text": "that whatever you want to change, do it in a new array whatever you are reading,
 whatever you are reading about the source read that from the results don't remove this and
make it T , then it will be the same problem that you have just edited T then you are reading
that the whole meaning of this is that whatever you are reading about the source, whatever you
are reading about the value of the source plus weight whatever you are doing, read that last
relaxation that last relaxation, i.e. read the result"
    },
    {
      "start": 1607.22,
      "end": 1631.28,
      "text": "Whatever you are writing, whatever you are relaxing and writing new values,
don't write that on the result. Write that in temporary. When the whole relaxation is done,
then the result is equal to. If you make it temporary, then it will be done. This is the
purpose of this. Okay? We hope that this is a very crucial part for you. If you don't
understand this, then your whole question will be wrong. Okay? So, look at it once or twice.
You will understand that it is not that hard. Just that,"
    },
    {
      "start": 1631.28,
      "end": 1658.83,
      "text": "there should be no chain reaction. 0 to 2 is a good thing, but in the same
relaxation if you change 2 to 2, 2 to 1 to 1, 1 to 3 to 3, then the logic of the whole
relaxation will be spoiled. This logic is important in this question. Normally, this wasn't
important in Bellman Ford as well. Because in Bellman Ford, your goal was to get the distance
by hook and crook. But because this is a case-tops, this is important. Now let's show you the
code of this. It's a very simple code."
    },
    {
      "start": 1658.83,
      "end": 1689.39,
      "text": "It is the same code of Bellman Ford. Let's go. If you want to write its code,
then its code will be copied from same to same Bellman Ford. There is not much change in it.
What will be given to you? It will be given to you on int fun or on the lead code. First of
all, you will be given int n which is the number of airports. After that, you will be given
vector of vector of int. Yes, okay. What will be given in this? Edges. After that, you will be
given three more things. Source,"
    },
    {
      "start": 1689.87,
      "end": 1718.13,
      "text": "destination and int k. Alright? Let's go. First of all, what is the work? Make
a result array. Vector, int, result. In this, what will we do? N size and 1E8. 1E8 means, we
have already explained 10 to the power 8. Alright? After that, result of source is equal to 0.
```

```
Because source is 0 in reaching source. Alright? After that, what will we do? Int i. Now,"
    },
    {
      "start": 1718.13,
      "end": 1743.1,
      "text": "Whatever k is given, how many times you have to relax? k plus 1 times. So, for
i is equal to 0, i less than equal to k, i plus plus. Okay? What happened to this? k plus 1
times, relax. Okay? First of all, before starting relax, make a temporary vector. Vector int
tmp is equal to res."
    },
    {
      "start": 1743.1,
      "end": 1773.5,
      "text": "Okay? Make it the same as res because you have to write it. Now you have to
relax all the edges. So how do we relax all the edges? For int j is equal to 0, j less than
edges.size and j++. Okay? Take out the things one by one. s is equal to edges j 0. Where will
your destination be? Edges j 1."
    },
    {
      "start": 1773.5,
      "end": 1803.7,
      "text": "How will the weight come? It will come from 2. Weight is equal to hs j2. Okay?
Relax. Relax. So, first of all, look at this. If result of s not equal to 1e8. Meaning, why
are we saying this? Why are we saying this? Because, if the result is 1e8, it means infinity.
So, if your source is at infinity, then what will relax someone? Meaning, you are saying that
h is going from 1 to here."
    },
    {
      "start": 1803.7,
      "end": 1830.64,
      "text": "Now if 1 itself is at infinity, then what to relax from infinity? Nothing to do,
 leave it. If this is not infinity, then something can happen. And, and, what to do? t of d is
greater than result of s plus weight, plus weight. t of d is greater than result of s plus
weight. Now you will ask, what is the meaning of making t if result of d is greater than
result of s plus weight? You can relax t, destination again and again. Assume, this means,"
    },
    {
      "start": 1830.93,
      "end": 1855.79,
      "text": "Assume that if something goes from 0 to 1 and something goes from 2 to 1.
Assume that the current value of 0 is 50 and its value is 20. The current value of 2 is 10 and
its value is 10. What does this mean? Assume that you got one edge from 0 to 1 and its value
is 50. And you got one edge from 2 to 1 and its value is 10. Why am I saying this? Assume that
when you relax this edge, what will be the value of 1? 70."
    },
    {
      "start": 1855.79,
      "end": 1884.48,
      "text": "When you relax this edge, then the value of 1 will be 10 plus 10, 20. You have
to compare 70 to 20. That's why we have written it here. If this is the case, then T of D is
equal to result of S plus weight. You can understand why T is being used here and result is
being used here. When you are updating, then you can update B in the same relaxation again and
again. Now there is a lot of way to reach 1 where one edge is used."
    },
    {
      "start": 1884.48,
      "end": 1910.82,
      "text": "In the above example, there are two ways to reach 1, where one edge is applied.
So you can update the good answer in 1 by looking at both. But it is important to write the
result of S because the value of the source is taken according to the previous relaxation. Not
with this relaxation. It should not happen that the source has been updated immediately and we
are using it. No, it will be updated in the next relaxation. Is it okay? Is it done? There is
nothing more to do here. Everything is over."
    },
    {
      "start": 1910.82,
      "end": 1940.82,
      "text": "Now, being comfortable here, first of all, see that if the result of
destination is equal to 1E8, which means that it did not get anything, then it is said to
return minus 1. It means that you cannot go to K stops here. You cannot reach up to K stops.
If you have got something, then you must have got it. Return result of destination. See, this
is the code of your entire Bellman-Ford. In this, Bellman-Ford was there. The most important
thing was this part. Why did you have to use temporary array?"
    },
    {
      "start": 1940.82,
      "end": 1952.59,
```

```
      "text": "according to me, we have explained it very well if you don't understand, then
ask in the comment if possible, we will tell you again in the comment is it right? make it and
it is a very important question make it definitely see you in the next lecture bye bye"
    }
  ]
}
```